

# Digital Logic

Generated 9/27/2026, 10:17:10 PM • 64 messages • Source: Shared ChatGPT link  
<https://chatgpt.com/share/6ab94872-f44c-83ee-a18f-ce72d24c56ec>

USER

Digital Logic ....

ASSISTANT

## GATE CS — DIGITAL LOGIC COMPLETE COURSE

Yes. We'll do **Digital Logic** exactly like the other GATE subjects: from fundamentals to GATE-level numericals, with concise notes, shortcuts, traps, and PYQ-style practice.

### Official GATE CS Digital Logic syllabus

#### Digital Logic:

- Boolean algebra
- Combinational circuits
- Sequential circuits
- Minimization
- Number representations
- Computer arithmetic — fixed-point and floating-point

I'll expand this small-looking syllabus into the complete set of concepts actually needed for GATE.

## DIGITAL LOGIC MASTER ROADMAP

### Unit 1 — Number Systems

1. Decimal, Binary, Octal, Hexadecimal
2. Base/radix concepts
3. Base conversions
4. Fractional-number conversions
5. Shortcut conversions:
  - Binary !” Octal
  - Binary !” Hexadecimal
6. Arithmetic in different bases
7. Finding unknown radix
8. Range of n-bit numbers

### Unit 2 — Signed Number Representation

9. Sign-magnitude
10. 1's complement
11. 2's complement
12. Representation ranges

- 13. Negation using 2's complement
- 14. Signed arithmetic
- 15. Overflow
- 16. Sign extension
- 17. Zero extension

### Unit 3 — Codes

- 18. BCD
- 19. Excess-3
- 20. Gray code
- 21. Binary !" Gray conversion
- 22. Hamming distance/basic error concepts

### Unit 4 — Boolean Algebra

- 23. Boolean variables and operations
- 24. AND, OR, NOT
- 25. NAND, NOR, XOR, XNOR
- 26. Boolean laws
- 27. De Morgan's laws
- 28. Principle of duality
- 29. Complement
- 30. Boolean identities

### Unit 5 — Boolean Functions

- 31. Truth tables
- 32. Minterms
- 33. Maxterms
- 34. Canonical SOP
- 35. Canonical POS
- 36. Standard SOP/POS
- 37. Converting between different representations
- 38. Function notation using  $\Sigma$  and  $\Pi$

### Unit 6 — Logic Gate Implementation

- 39. Universal gates
- 40. NAND-only realization
- 41. NOR-only realization
- 42. XOR/XNOR properties
- 43. Gate-count questions
- 44. Multi-level realization

### Unit 7 — Boolean Minimization

- 45. Algebraic simplification
- 46. Karnaugh Maps
- 47. 2-variable K-map
- 48. 3-variable K-map

- 49. 4-variable K-map
- 50. 5-variable idea
- 51. Group formation rules
- 52. Prime implicants
- 53. Essential prime implicants
- 54. Don't-care conditions
- 55. SOP minimization
- 56. POS minimization
- 57. NAND/NOR implementation from K-map
- 58. Quine–McCluskey basic idea

## **Unit 8 — Combinational Circuits**

- 59. Half adder
- 60. Full adder
- 61. Half subtractor
- 62. Full subtractor
- 63. Ripple-carry adder
- 64. Carry propagation
- 65. Carry look-ahead adder
- 66. BCD adder
- 67. Comparator

## **Unit 9 — Multiplexers and Decoders**

- 68. Multiplexer
- 69. MUX expansion
- 70. Boolean function implementation using MUX
- 71. Demultiplexer
- 72. Decoder
- 73. Encoder
- 74. Priority encoder
- 75. Boolean functions using decoder
- 76. Decoder expansion

## **Unit 10 — Sequential Logic Basics**

- 77. Combinational vs sequential circuits
- 78. Clock
- 79. Latch vs flip-flop
- 80. Level triggering
- 81. Edge triggering
- 82. Setup time
- 83. Hold time
- 84. Propagation delay
- 85. Clock skew basics

## **Unit 11 — Flip-Flops**

- 86. SR latch
- 87. SR flip-flop
- 88. JK flip-flop
- 89. D flip-flop
- 90. T flip-flop
- 91. Characteristic tables
- 92. Characteristic equations
- 93. Excitation tables
- 94. Flip-flop conversions
- 95. Race-around condition
- 96. Master-slave JK FF

## Unit 12 — Registers

- 97. Register basics
- 98. SISO
- 99. SIPO
- 100. PISO
- 101. PIPO
- 102. Shift registers
- 103. Universal shift register

## Unit 13 — Counters

- 104. Asynchronous/ripple counter
- 105. Synchronous counter
- 106. Up counter
- 107. Down counter
- 108. Up/down counter
- 109. Mod-N counters
- 110. Counter design
- 111. Ring counter
- 112. Johnson counter
- 113. Counter timing/delay questions

## Unit 14 — Sequential Circuit Analysis & Design

- 114. State
- 115. State table
- 116. State diagram
- 117. State equations
- 118. State assignment
- 119. Sequence detectors
- 120. Moore machines
- 121. Mealy machines
- 122. Moore vs Mealy

## Unit 15 — Computer Arithmetic

- 123 Binary addition
- 124 Binary subtraction
- 125 Signed arithmetic
- 126 Addition/subtraction using complements
- 127 Overflow detection
- 128 Binary multiplication
- 129 Binary division concepts

## Unit 16 — Fixed-Point Representation

- 130 Fixed-point format
- 131 Integer/fraction bits
- 132 Range
- 133 Precision
- 134 Quantization concepts

## Unit 17 — Floating-Point Representation

- 135 Scientific notation
- 136 Normalization
- 137 Sign, exponent, mantissa/significand
- 138 IEEE 754
- 139 Single precision
- 140 Double precision
- 141 Bias
- 142 Normalized numbers
- 143 Denormalized/subnormal numbers
- 144  $\pm 0$
- 145  $\pm$
- 146 NaN
- 147 Floating-point range and precision
- 148 IEEE-754 conversion numericals

---

# UNIT 1 — NUMBER SYSTEMS

---

This is the foundation of Digital Logic.

## 1. What is a number system?

---

A number system is simply a method for representing numbers using a fixed collection of symbols.

Example:

Decimal uses:

$\{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$

So decimal has **10 symbols**.

Therefore its base is:

$\boxed{10}$

## 2. Radix / Base

The number of distinct digits available in a number system is called its **radix** or **base**.

| Number System | Base | Digits  |
|---------------|------|---------|
| Binary        | 2    | 0,1     |
| Octal         | 8    | 0–7     |
| Decimal       | 10   | 0–9     |
| Hexadecimal   | 16   | 0–9,A–F |

Memory trick:

**BODH = 2, 8, 10, 16**

Binary !' 2 Octal !' 8 Decimal !' 10 Hexadecimal !' 16

## 3. Positional value

Consider decimal:

$4725$

Actually:

$4 \times 10^3 + 7 \times 10^2 + 2 \times 10^1 + 5 \times 10^0$

The same principle works for every base.

General representation:

$(a_{n-1} \dots a_1 a_0)_r$

means:

$a_n r^n + a_{n-1} r^{n-1} + \dots + a_1 r + a_0$

where  $(r)$  is the base.

## 4. Binary !' Decimal

Example:

$(101101)_2$

Expand using powers of 2:

$1(2^5) + 0(2^4) + 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0)$

$= 32 + 8 + 4 + 1$

$\boxed{45}$

Therefore:

$$\downarrow (101101)_2 = (45)_{10}$$

### Shortcut

Write binary positional weights:

$$\downarrow 32, 16, 8, 4, 2, 1$$

Take only positions containing 1.

For:

$$\downarrow 101101$$

$$\downarrow 32 + 8 + 4 + 1 = 45$$

---

## 5. Octal !' Decimal

Example:

$$\downarrow (725)_8$$

$$\downarrow 7(8^2) + 2(8^1) + 5(8^0)$$

$$\downarrow = 7(64) + 16 + 5$$

$$\downarrow = 448 + 21$$

$$\downarrow \boxed{469}$$

---

## 6. Hexadecimal !' Decimal

Hexadecimal digits:

$$\downarrow A=10, B=11, C=12, D=13, E=14, F=15$$

Example:

$$\downarrow (2AF)_{16}$$

$$\downarrow 2(16^2) + 10(16) + 15$$

$$\downarrow = 512 + 160 + 15$$

$$\downarrow \boxed{687}$$

---

## 7. Decimal !' Binary

Use repeated division by 2.

Convert:

$$\downarrow 45_{10}$$

| Division    | Quotient | Remainder |
|-------------|----------|-----------|
| $45 \div 2$ | 22       | 1         |

| Division    | Quotient | Remainder |
|-------------|----------|-----------|
| $22 \div 2$ | 11       | 0         |
| $11 \div 2$ | 5        | 1         |
| $5 \div 2$  | 2        | 1         |
| $2 \div 2$  | 1        | 0         |
| $1 \div 2$  | 0        | 1         |

Read remainder:

### Bottom !' Top

$\boxed{101101_2}$

Memory trick:

**Integer conversion: Divide and read UP.**

## 8. Decimal !' Octal

Repeatedly divide by 8.

Example:

$83_{10}$

$83 \div 8 = 10 \text{ quad } R3$

$10 \div 8 = 1 \text{ quad } R2$

$1 \div 8 = 0 \text{ quad } R1$

Read upwards:

$\boxed{123_8}$

Check:

$1(64) + 2(8) + 3 = 83$

## 9. Decimal !' Hexadecimal

Repeated division by 16.

Example:

$254_{10}$

$254 \div 16 = 15 \text{ quad } R14$

14 = E.

15 = F.

Therefore:



$\boxed{FE_{16}}$

---

## 10. Binary !”Octal shortcut

---

This is extremely important.

Because:

$$8=2^3$$

therefore:

$$\boxed{1\text{ octal digit}=3\text{ binary bits}}$$

Example:

$$101111011_2$$

Group into 3:

$$101\text{ quad }111\text{ quad }011$$

Convert:

$$101=5$$

$$111=7$$

$$011=3$$

Therefore:

$$\boxed{573_8}$$

## 11. Binary !”Hexadecimal shortcut

---

Because:

$$16=2^4$$

therefore:

$$\boxed{1\text{ hexadecimal digit}=4\text{ binary bits}}$$

Example:

$$101111101001_2$$

Group:

$$1011\text{ quad }1110\text{ quad }1001$$

$$1011=B$$

$$1110=E$$

$$1001=9$$

Therefore:

$$\boxed{BE9_{16}}$$

**Must-memorize hexadecimal table**

| Binary | Hex |
|--------|-----|
| 0000   | 0   |
| 0001   | 1   |
| 0010   | 2   |
| 0011   | 3   |
| 0100   | 4   |
| 0101   | 5   |
| 0110   | 6   |
| 0111   | 7   |
| 1000   | 8   |
| 1001   | 9   |
| 1010   | A   |
| 1011   | B   |
| 1100   | C   |
| 1101   | D   |
| 1110   | E   |
| 1111   | F   |

You should eventually recognize this instantly.

## 12. Fractional Binary to Decimal

Example:

$$\text{101.101}_2$$

Integer side:

$$1(2^2) + 0(2^1) + 1(2^0)$$

Fraction side:

$$1(2^{-1}) + 0(2^{-2}) + 1(2^{-3})$$

Therefore:

$$4 + 1 + \frac{12}{16} + \frac{1}{8}$$

$$= 5.625$$

So:

$$\boxed{101.101_2 = 5.625_{10}}$$

Remember:

Left of decimal:

$$\{ 2^0, 2^1, 2^2, \dots \}$$

Right of decimal:

$$\{ 2^{-1}, 2^{-2}, 2^{-3}, \dots \}$$

---

## 13. Decimal fraction !' Binary

For integer:

**Divide by 2.**

For fraction:

**Multiply by 2.**

Example:

$$\{ 0.625_{10} \}$$

$$\{ 0.625 \times 2 = 1.25 \}$$

Take integer = **1**

$$\{ 0.25 \times 2 = 0.5 \}$$

Take integer = **0**

$$\{ 0.5 \times 2 = 1.0 \}$$

Take integer = **1**

Therefore:

$$\{ \boxed{0.625_{10} = 0.101_2} \}$$

Memory trick:

### Integer

**Divide !' read bottom-up**

### Fraction

**Multiply !' read top-down**

Very important GATE distinction.

---

## 14. Valid digit rule

For a base  $\{r\}$ :

$$\{ \boxed{0 \leq \text{digit} < r} \}$$

For binary:

Only:

$$\{0, 1\}$$

So:

$$\{(10201)_2\}$$

is **invalid** because digit 2 appears.

For octal:

Digits allowed:

$$\{0-7\}$$

So:

$$\{(786)_8\}$$

is invalid because **8 is not allowed**.

---

## 15. GATE trick — find unknown base

---

Suppose:

$$\{(121)_x = (16)_{10}\}$$

Expand:

$$1x^2 + 2x + 1 = 16$$

$$x^2 + 2x - 15 = 0$$

$$(x+5)(x-3) = 0$$

Valid base:

$$\boxed{x=3}$$

Why not  $(-5)$ ?

Bases are positive, and the base must also be larger than the largest digit.

Largest digit here = 2.

Therefore:

$$x > 2$$

So:

$$\boxed{x=3}$$

---

## 16. Range of an unsigned n-bit number

---

With  $(n)$  binary bits:

Minimum:

$$00\dots 00 = 0$$

Maximum:

$$11\dots 11$$

which equals:

$$\boxed{2^{n-1}}$$

Therefore:

$$\boxed{0 \text{ to } 2^{n-1}}$$

Example:

8 bits:

$$\boxed{0 \text{ to } 2^8-1}$$

$$\boxed{0 \text{ to } 255}$$

Number of different patterns:

$$\boxed{2^n}$$

---

## GATE QUICK NOTES — UNIT 1

Memorize these:

$$\boxed{1 \text{ octal digit} = 3 \text{ bits}}$$

$$\boxed{1 \text{ hex digit} = 4 \text{ bits}}$$

$$\boxed{A=10, B=11, C=12, D=13, E=14, F=15}$$

Unsigned n-bit range:

$$\boxed{0 \leq N \leq 2^n-1}$$

Number of n-bit patterns:

$$\boxed{2^n}$$

Integer decimal !' binary:

$$\boxed{\text{Repeated division}}$$

Fraction decimal !' binary:

$$\boxed{\text{Repeated multiplication}}$$

---

## Common GATE traps

### Trap 1

Reading decimal-to-binary remainders top-down.

Wrong.

For the integer part:

$$\boxed{\text{Bottom !' Top}}$$

### Trap 2

Thinking hexadecimal  $(10)$  means decimal 10.

Actually:

$$\boxed{(10)_{16} = 16_{10}}$$

### Trap 3

Confusing digit F with 16.

$\backslash [ F=15 \backslash ]$

because hexadecimal digits run:

$\backslash [ 0\text{ through }15 \backslash ]$

### Trap 4

Using 4-bit grouping for octal.

Wrong:

$\backslash [ \boxed{\text{Octal}=3\text{ bits}} \backslash ]$

$\backslash [ \boxed{\text{Hex}=4\text{ bits}} \backslash ]$

---

## Mini GATE practice

---

### Q1

Convert:

$\backslash [ 1101011_2 \backslash ]$

to decimal.

Answer:

$\backslash [ 64+32+8+2+1 \backslash ]$

$\backslash [ \boxed{107} \backslash ]$

---

### Q2

Convert:

$\backslash [ 11110110_2 \backslash ]$

to hexadecimal.

Group:

$\backslash [ 1111\text{ }0110 \backslash ]$

$\backslash [ F\text{ }6 \backslash ]$

$\backslash [ \boxed{F6_{16}} \backslash ]$

---

### Q3

What is the maximum unsigned value representable using 12 bits?

$\backslash [ 2^{12}-1 \backslash ]$

$\backslash [ 4096-1 \backslash ]$

$\backslash [ \boxed{4095} \backslash ]$

---

#### Q4

How many distinct numbers can be represented using 10 bits?

$2^{10}$

$\boxed{1024}$

#### Q5

Convert:

$0.375_{10}$

to binary.

$0.375 \times 2 = 0.75 \rightarrow 0$

$0.75 \times 2 = 1.5 \rightarrow 1$

$0.5 \times 2 = 1.0 \rightarrow 1$

Therefore:

$\boxed{0.011_2}$

### Progress

**Digital Logic syllabus:** Started **Completed:** Number-system foundations **Next: Signed number representations — Sign Magnitude, 1's Complement, 2's Complement, ranges, arithmetic and overflow.**

Send **NEXT**.

#### USER

Digital Logic Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).

This is the full syllabus of gate of DIGITAL LOGIC So give me complete each and every thing which is needed data for gate include PYQ's Even everything i 'm not having time to see hours of videos give me a prompt ..

#### ASSISTANT

Absolutely. Since this is the **full Digital Logic syllabus you want to use**, the best approach is to create one master prompt that forces ChatGPT to teach the subject from **zero !' complete GATE level**, without assuming that you will watch lectures elsewhere.

I've also included a strict “**do not miss hidden prerequisite topics**”, **PYQ coverage**, **numerical practice**, **shortcuts**, **revision notes**, and a **NEXT-based chapter system**.

:::writing{variant="document" id="58324" title="GATE Digital Logic — Complete Master Teacher Prompt"}

### GATE DIGITAL LOGIC — COMPLETE MASTER TEACHER PROMPT

I am preparing for **GATE Computer Science** and I have limited time.

I do **NOT** have the flexibility to watch tens or hundreds of hours of YouTubelectures.

I want to learn **Digital Logic completely from this chat** and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE Digital Logic subject expert
  - personal teacher
  - concept explainer
  - notes maker
  - PYQ-oriented mentor
  - numerical problem-solving trainer
  - circuit-design teacher
  - revision coach
  - shortcut and memory-trick creator
  - misconception detector
  - exam strategy mentor
- 

## MY CURRENT SUBJECT

---

Digital Logic

## OFFICIAL GATE SYLLABUS

---

Digital Logic:

**Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).**

Treat the above as the official syllabus.

However, do NOT teach only the literal words written in the syllabus.

Expand every syllabus point into all prerequisite concepts, subtopics, standard circuit types, numerical techniques, representations, formulas and question patterns required to solve GATE questions.

---

## MAIN GOAL

---

Convert the complete Digital Logic syllabus into a **self-contained GATE course inside this chat**.

Assume I may use your explanations as my PRIMARY learning source.

Therefore:

**Do not skip concepts just because they are normally taught in a video lecture or textbook.**

At the end I should be able to:

1. Understand every Digital Logic concept relevant to GATE.
  2. Solve conceptual questions.
  3. Solve numerical questions.
  4. Analyze logic circuits.
  5. Design combinational circuits.
  6. Analyze and design sequential circuits.
-



7. Simplify Boolean functions using every required method.
  8. Solve number-representation problems.
  9. Solve fixed-point arithmetic questions.
  10. Solve IEEE/floating-point questions.
  11. Recognize common GATE traps.
  12. Solve previous-year-question patterns independently.
  13. Revise the whole subject quickly before GATE.
- 

## IMPORTANT TEACHING RULE

---

I am learning this primarily from CHAT.

Never say:

- "You probably already know this."
- "This is basic, so we can skip it."
- "Refer to a textbook for details."
- "Watch a video to understand this."
- "This is outside the syllabus" when it is actually needed as a prerequisite.

Explain every prerequisite necessary to understand the GATE syllabus.

But also avoid wasting time on irrelevant electronics or hardware theory that GATECS does not require.

Teach for:

**Maximum GATE marks per unit of study time.**

---

## COMPLETE DIGITAL LOGIC ROADMAP

---

Before teaching, create a master roadmap and track progress through it.

At minimum, cover all of the following.

---

## UNIT 1 — NUMBER SYSTEM FUNDAMENTALS

---

Teach:

- Number systems
- Radix/base
- Positional representation
- Binary
- Octal
- Decimal
- Hexadecimal
- Conversion between bases
- Decimal !' binary
- Binary !' decimal
- Decimal !' octal

- Decimal to hexadecimal
- Binary to octal
- Binary to hexadecimal
- Conversion involving fractional numbers
- Repeated division method
- Repeated multiplication method
- Unknown-radix problems
- Arithmetic in different bases
- n-bit representations
- Range of unsigned numbers

Include shortcuts for fast base conversion.

---

## UNIT 2 — SIGNED NUMBER REPRESENTATION

---

Teach completely:

- Unsigned representation
- Signed magnitude representation
- 1's complement
- 2's complement
- Representation of positive and negative numbers
- Negation
- Number of representable values
- Range formulas
- Positive zero / negative zero
- Why 2's complement has one extra negative number
- Sign bit
- Sign extension
- Zero extension
- Truncation
- Converting signed values between different bit widths

Create a comparison table for:

- unsigned
- sign magnitude
- 1's complement
- 2's complement

Include all important range formulas.

---

## UNIT 3 — BINARY ARITHMETIC

---

Teach:

- Binary addition
- Binary subtraction

- Carry
- Borrow
- Addition using complements
- Subtraction using complements
- Signed addition
- Signed subtraction
- Carry-out vs overflow
- Overflow detection
- Overflow rules
- 2's-complement arithmetic
- Multiplication basics
- Binary multiplication
- Division basics where required
- Shift operations related to arithmetic
- Arithmetic shift
- Logical shift
- Left/right shifting and multiplication/division by powers of 2

Explain GATE traps involving:

**carry "overflow**

---

## UNIT 4 — BOOLEAN ALGEBRA FUNDAMENTALS

---

Teach from zero:

- Boolean variables
- Binary logic
- Truth tables
- AND
- OR
- NOT
- NAND
- NOR
- XOR
- XNOR
- Buffer if relevant

For every gate give:

- symbol meaning
- Boolean expression
- truth table
- important properties

Then teach Boolean laws:

- Identity law
- Null law

- Idempotent law
- Complement law
- Involution law
- Commutative law
- Associative law
- Distributive law
- Absorption law
- De Morgan's laws
- Consensus theorem
- Principle of duality

Derive important identities instead of merely listing them.

Teach shortcuts for simplifying Boolean expressions quickly.

---

## UNIT 5 — BOOLEAN FUNCTIONS AND REPRESENTATION

---

Teach:

- Boolean functions
- Truth tables
- Literals
- Product terms
- Sum terms
- Minterms
- Maxterms
- SOP
- POS
- Standard SOP
- Standard POS
- Canonical SOP
- Canonical POS
- Sum of minterms
- Product of maxterms
- $\Sigma$  notation
- $\Pi$  notation
- Indexing minterms/maxterms
- Conversion between truth table and Boolean expression
- Conversion between SOP and POS
- Complement relationships
- Finding  $F'$
- Missing minterms/maxterms

Include tricky GATE questions involving minterm/maxterm numbering.

---

## UNIT 6 — LOGIC GATE REALIZATION

---

Teach:

- Universal gates
- Why NAND is universal
- Why NOR is universal
- AND using NAND
- OR using NAND
- NOT using NAND
- AND using NOR
- OR using NOR
- NOT using NOR
- NAND-only realization
- NOR-only realization
- Two-level implementation
- Multi-level implementation
- Bubble pushing if useful
- De Morgan transformation
- XOR/XNOR realization
- Gate-count optimization where relevant

Teach how Boolean simplification affects hardware implementation.

---

## UNIT 7 — BOOLEAN MINIMIZATION: ALGEBRAIC METHOD

---

Teach:

- Algebraic simplification
- Applying Boolean identities efficiently
- Factoring
- Absorption
- Consensus theorem
- Redundant terms
- Finding minimum expressions
- SOP simplification
- POS simplification

For every example show:

Original expression !' identity used !' simplified expression.

Train me to recognize common patterns mentally.

---

## UNIT 8 — KARNAUGH MAP

---

Teach K-map extremely thoroughly.

Cover:

---

- Purpose of K-map
- Gray-code ordering
- Adjacency
- Wrap-around adjacency
- Corners
- Group sizes
- Powers-of-two rule
- 2-variable K-map
- 3-variable K-map
- 4-variable K-map
- 5-variable idea if GATE relevant
- SOP minimization
- POS minimization
- Grouping 1s
- Grouping 0s
- Overlapping groups
- Largest-group rule
- Redundant groups
- Prime implicants
- Essential prime implicants
- Non-essential prime implicants
- Implicant terminology
- Don't-care conditions
- Using X/don't cares optimally
- Minimal SOP
- Minimal POS
- Multiple minimum solutions
- K-map from minterms
- K-map from maxterms
- K-map from truth table
- K-map from Boolean expression

Explain why variables disappear when cells are grouped.

Teach all common K-map traps.

---

## UNIT 9 — TABULAR METHOD / QUINE–McCLUSKEY METHOD

---

Teach from scratch:

- Why tabular minimization exists
- Relation with K-map
- Listing minterms
- Grouping by number of 1s
- Combining adjacent terms

- Dash notation
- Repeated combination
- Prime implicants
- Prime implicant chart
- Essential prime implicants
- Cover selection
- Don't-care handling
- Final minimized expression

Solve complete examples step-by-step.

Do not skip the intermediate tables.

Also explain when GATE may prefer this method over K-map.

---

## UNIT 10 — COMBINATIONAL CIRCUITS: FUNDAMENTALS

---

Teach:

- What is a combinational circuit?
- Input-output relationship
- No memory property
- Propagation delay
- How to design a combinational circuit from a specification

General design procedure:

1. Identify inputs/outputs
2. Assign variables
3. Create truth table
4. Obtain Boolean expressions
5. Minimize
6. Implement circuit

---

## UNIT 11 — ADDERS AND SUBTRACTORS

---

Teach completely:

- Half adder
- Full adder
- Half subtractor
- Full subtractor

For each:

- block idea
- truth table
- equations
- circuit realization
- relationship between circuits

Then cover:

- n-bit parallel adder
- Ripple Carry Adder
- Carry propagation
- Propagation delay
- Worst-case delay
- Carry look-ahead concepts if needed for GATE
- Carry generate
- Carry propagate
- Faster addition concepts
- Adder-subtractor circuit

Include numerical timing questions.

---

## UNIT 12 — MULTIPLEXERS

---

Teach:

- Multiplexer concept
- 2:1 MUX
- 4:1 MUX
- 8:1 MUX
- 16:1 MUX
- Select lines
- Relationship:

For  $(2^n)$  inputs:

number of select lines =  $\lceil \log_2(n) \rceil$

Teach:

- MUX truth tables
- MUX Boolean equation
- Implementing larger MUX using smaller MUXes
- Implementing Boolean functions using MUX
- Choosing variables as select lines
- Using remaining variables at data inputs
- Minimum MUX questions
- MUX tree
- Propagation-delay questions

Give many GATE-style function-realization questions.

---

## UNIT 13 — DEMULTIPLEXERS, DECODERS AND ENCODERS

---

Teach:



## Demultiplexer

---

- Concept
- Select lines
- Implementations

## Decoder

---

- $n$ -to- $(2^n)$  decoder
- Enable input
- Decoder expansion
- Boolean-function realization using decoder
- Decoder as minterm generator

## Encoder

---

- Encoder concept
- Standard encoder
- Priority encoder
- Handling multiple active inputs
- Valid bit

Explain differences clearly:

MUX vs DEMUX vs Decoder vs Encoder.

---

## UNIT 14 — COMPARATOR AND OTHER COMBINATIONAL CIRCUITS

---

Teach:

- Equality comparator
- Magnitude comparator
- 1-bit comparator
- Multi-bit comparison
- Cascading comparators

Also cover any other combinational-circuit concept repeatedly relevant to GATE.

---

## UNIT 15 — SEQUENTIAL CIRCUITS FUNDAMENTALS

---

Teach carefully:

- Combinational vs sequential circuits
- Memory
- State
- Feedback
- Clock
- Clock signal
- Positive edge

- Negative edge
- Level triggering
- Edge triggering
- Synchronous systems
- Asynchronous systems

Timing concepts:

- Propagation delay
- Setup time
- Hold time
- Clock-to-Q delay
- Clock skew if relevant
- Maximum operating frequency concepts

Use timing diagrams wherever needed.

---

## UNIT 16 — LATCHES AND FLIP-FLOPS

---

Teach every important device.

### SR latch

---

- NOR implementation
- NAND implementation
- Truth table
- forbidden/invalid states

### SR flip-flop

---

### D latch / D flip-flop

---

### JK flip-flop

---

### T flip-flop

---

For each teach:

- operation
- truth table
- characteristic table
- characteristic equation
- excitation table
- timing behavior
- applications

Create ONE master comparison table.

---

## UNIT 17 — FLIP-FLOP CONVERSIONS

---

Teach how to convert:

- SR  $\rightarrow$  JK
- SR  $\rightarrow$  D
- SR  $\rightarrow$  T
- JK  $\rightarrow$  SR
- JK  $\rightarrow$  D
- JK  $\rightarrow$  T
- D  $\rightarrow$  SR
- D  $\rightarrow$  JK
- D  $\rightarrow$  T
- T  $\rightarrow$  SR
- T  $\rightarrow$  JK
- T  $\rightarrow$  D

Teach the standard conversion procedure using:

- desired flip-flop behavior
- excitation table
- K-map
- input equations

Focus more on patterns commonly asked in GATE.

---

## UNIT 18 — RACE-AROUND AND MASTER-SLAVE

---

Teach:

- Race-around condition
- Why it occurs
- JK flip-flop behavior
- Clock pulse width
- Methods to eliminate race-around
- Edge-triggered solution
- Master-slave JK flip-flop

Explain intuitively and mathematically where useful.

---

## UNIT 19 — REGISTERS AND SHIFT REGISTERS

---

Teach:

- Register
- Parallel register
- Shift register

Types:

- SISO

- SIPO
- PISO
- PIPO

Then:

- Left shift
- Right shift
- Bidirectional shift register
- Universal shift register
- Serial transfer
- Parallel transfer
- Register timing questions

Explain what happens after each clock pulse.

---

## UNIT 20 — COUNTERS

---

Teach very thoroughly.

Cover:

- Counter concept
- Counter modulus
- MOD-N
- Number of required flip-flops

Formula:

$$\lceil n = \lceil \log_2 N \rceil \rceil$$

Teach:

- Asynchronous counter
- Ripple counter
- Synchronous counter
- Up counter
- Down counter
- Up/down counter
- Binary counter
- Decade counter
- MOD-N counter
- Truncated counter
- Counter state sequence
- Counter design
- Maximum count
- Frequency division
- Propagation-delay analysis
- Maximum clock-frequency questions

Then:

- Ring counter
- Johnson/twisted-ring counter

Include formulas for the number of states.

---

## UNIT 21 — SEQUENTIAL CIRCUIT ANALYSIS

---

Teach the complete method for analyzing a given sequential circuit.

Cover:

- Present state
- Next state
- Inputs
- Outputs
- State variables
- Flip-flop input equations
- Characteristic equations
- Next-state equations
- State table
- State diagram

Given a circuit, teach me how to derive its behavior systematically.

---

## UNIT 22 — SEQUENTIAL CIRCUIT DESIGN

---

Teach:

- Converting specification to state diagram
- State table
- State assignment
- Flip-flop selection
- Excitation requirements
- K-map minimization
- Logic implementation

Do complete examples.

---

## UNIT 23 — FINITE STATEMACHINES

---

Teach:

- FSM concept
- Moore machine
- Mealy machine
- Moore vs Mealy
- Output dependence
- State diagrams

- State tables
- Conversion if relevant
- State reduction/minimization concepts if relevant

Focus especially on GATE circuit questions.

---

## UNIT 24 — SEQUENCE DETECTORS

---

Teach:

- Sequence detector concept
- Overlapping detector
- Non-overlapping detector
- Moore implementation
- Mealy implementation
- State construction
- Handling partial matches
- Designing arbitrary bit-sequence detectors

Solve examples such as detecting:

101 1011 1101

and then harder GATE-style cases.

---

## UNIT 25 — FIXED-POINT REPRESENTATION

---

Teach:

- Fixed-point concept
- Binary point
- Integer bits
- Fraction bits
- Signed fixed-point numbers
- Unsigned fixed-point numbers
- Range
- Resolution
- Precision
- Quantization
- Representable values
- Overflow
- Underflow where applicable

Teach how changing the number of integer/fraction bits affects:

- range
- precision

Provide formulas and numerical questions.

---

## UNIT 26 — FLOATING-POINT FUNDAMENTALS

---

Before IEEE 754, explain:

- Scientific notation
- Binary scientific notation
- Normalization
- Sign
- Significand/mantissa
- Exponent
- Bias
- Precision
- Range

Show examples such as:

`\[ 10110.101_2 \]`

becoming normalized binary scientific notation.

---

## UNIT 27 — IEEE 754 FLOATING POINT

---

Teach completely.

### IEEE 754 Single Precision

---

- 32 bits
- 1 sign bit
- 8 exponent bits
- 23 fraction bits
- bias = 127

### IEEE 754 Double Precision

---

- 64 bits
- 1 sign bit
- 11 exponent bits
- 52 fraction bits
- bias = 1023

Teach:

- biased exponent
- hidden/implicit leading 1
- normalized numbers
- subnormal/denormalized numbers
- +0
- -0
- "+"
- "-"

- NaN
- exponent patterns
- significant calculation
- encoding decimal !' IEEE
- decoding IEEE !' value
- smallest/largest numbers where GATE relevant
- precision
- rounding basics
- floating-point errors
- comparison with fixed point

Solve complete bit-pattern numerals.

---

## PYQ REQUIREMENT

This is extremely important.

For EVERY major chapter:

1. Identify the kinds of questions GATE has historically asked.
2. Mention the relevant GATE year when confidently known.
3. Never invent a PYQ or falsely label a question as an actual GATE PYQ.
4. If exact wording cannot be provided or verified, give a faithful paraphrase and label it clearly.
5. Separate:

### ACTUAL/PARAPHRASED PYQ

from

### PYQ-STYLE PRACTICE QUESTION

6. Explain why each option is correct/incorrect for MCQs.
7. Show full working for numerical-answer questions.
8. Explain the shortest GATE method after showing the proper method.
9. Identify the concept tested.
10. Explain the trap being tested.

Cover old and recent useful GATE CS Digital Logic PYQ patterns.

---

## QUESTION TYPES

Train me for all GATE formats:

### MCQ

One correct option.

### MSQ

One or more correct options.

### NAT

Numerical answer.



For each chapter provide all applicable types.

---

## FOUR LEVELS OF PRACTICE

---

Use four levels.

### L0 — Foundation

---

Very simple questions that verify basic understanding.

### L1 — Standard

---

Direct application of one concept.

### L2 — GATE Level

---

Multi-step or deceptive exam-level questions.

### L3 — Advanced/Trap

---

Questions combining concepts or containing typical GATE traps.

Do not overwhelm me with hundreds of repetitive questions.

Choose enough problems to cover all important patterns.

---

## FORMAT FOR TEACHING EACH CONCEPT

---

Whenever possible use:

### 1. Intuition

---

Explain what the idea actually means.

### 2. Formal Definition

---

Give the precise exam definition.

### 3. Diagram / Truth Table / Equation

---

Whichever applies.

### 4. Step-by-Step Example

---

Start simple.

### 5. GATE-Level Example

---

Increase difficulty.

### 6. Shortcut

---

Show the fastest safe exam technique.

## 7. Common Trap

---

Explain how students lose marks.

## 8. PYQ Connection

---

Explain how GATE asks it.

## 9. Practice

---

Give selected questions.

## 10. Micro Revision

---

Give a 30-second revision summary.

---

# NUMERICAL SOLVING RULE

---

For every important numerical question teach TWO methods whenever useful:

### METHOD A — NORMAL METHOD

Detailed reasoning for learning.

### METHOD B — GATE METHOD

Fastest safe way to solve during the exam.

Clearly identify where shortcuts are safe and where they can cause mistakes.

---

# FORMULA SHEET

---

Maintain a continuously growing Digital Logic formula/shortcut sheet.

It should eventually contain things such as:

- unsigned ranges
- signed ranges
- complement relationships
- minterm/maxterm rules
- MUX relations
- decoder relations
- counter formulas
- propagation delay formulas
- fixed-point formulas
- IEEE 754 formulas
- all other required formulas

At the end provide one consolidated formula sheet.

---

## TRAP BOOK

---

Maintain a separate **GATE Digital Logic Trap Book**.

Examples:

- Carry confused with overflow
- Sign extension performed incorrectly
- Wrong minterm ordering
- Wrong K-map adjacency
- Diagonal K-map cells treated as adjacent
- Creating non-power-of-two K-map groups
- Forgetting wrap-around groups
- Using don't-cares unnecessarily
- Confusing prime implicant and essential prime implicant
- Wrong MUX select-line count
- Decoder input/output reversal
- Latch vs flip-flop confusion
- Characteristic table vs excitation table confusion
- Race-around misunderstanding
- Ripple vs synchronous counter confusion
- Moore vs Mealy output confusion
- IEEE exponent stored value confused with actual exponent
- Hidden bit incorrectly used for subnormal numbers

Add traps as we progress.

---

## MEMORY TRICKS

---

Give short memory tricks wherever helpful.

Examples should be genuinely useful, not childish or confusing.

Create tricks for:

- Boolean identities
  - XOR/XNOR
  - minterms/maxterms
  - K-map grouping
  - flip-flop tables
  - counter rules
  - IEEE 754
  - signed-number ranges
- 

## NOTES-MAKING MODE

---

Write explanations so that I can directly make notes.

Use:

---

- clear headings
- short paragraphs
- equations
- tables
- truth tables
- bullet points
- ASCII diagrams when useful
- compact summaries

Do not produce enormous walls of prose.

However, **concise does NOT mean incomplete.**

---

## IMPORTANT CIRCUIT-DIAGRAM RULE

---

Whenever a circuit is important:

1. Explain its purpose.
2. Show Boolean equations.
3. Show truth table.
4. Show a simple text/ASCII representation if possible.
5. Explain signal flow.
6. Explain how GATE may modify or combine the circuit.

Do not assume I can understand a circuit from its name alone.

---

## TIMING-DIAGRAM RULE

---

For questions involving:

- flip-flops
- latches
- clocks
- counters
- registers
- sequential circuits

teach me how to manually trace the waveform.

Always mark:

- clock edge
- input value
- present Q
- next Q

where relevant.

---

## REVISION SYSTEM

---

At the end of each major unit provide:

1. One-page revision notes
2. Formula/identity list
3. Common traps
4. 5–10 important questions
5. PYQ patterns
6. What I absolutely must remember

At the end of the entire subject provide:

- complete Digital Logic revision sheet
- formula sheet
- Boolean identity sheet
- K-map cheat sheet
- flip-flop master table
- combinational circuit master table
- counter master sheet
- IEEE 754 cheat sheet
- top GATE traps
- high-frequency PYQ concepts
- final mixed mock test

---

## HIGH-PRIORITY LABELS

Label topics:

### ~~0=1~~ MUST MASTER

Very frequently useful/high-value GATE concepts.

### ~~0=2~~ IMPORTANT

Important but relatively smaller.

### ~~0=3~~ SUPPORT

Needed mainly as foundation.

Use this to help me allocate my limited time.

---

## COMPLETION TRACKER

Maintain a Digital Logic syllabus tracker.

After every chapter show only a compact progress line such as:

**Completed:** Boolean Algebra **Current:** K-Maps **Remaining:** Tabular Method !' Combinational !' Sequential !' Number Arithmetic !' Fixed/Floating Point

Never claim the subject is completed until every syllabus item and prerequisite listed in this prompt has actually been

covered.

---

## "NEXT" PROTOCOL

---

Do NOT dump the entire Digital Logic course in one response.

Teach it chapter by chapter in the correct prerequisite order.

At the end of each response tell me exactly what comes next.

When I say:

**NEXT**

immediately continue from the exact point where we stopped.

Do not restart.

Do not ask me what chapter I want next.

Do not unnecessarily repeat previous explanations.

Maintain continuity until the entire Digital Logic course is complete.

---

## BEFORE DECLARING DIGITAL LOGIC COMPLETE

---

Perform a final coverage audit against:

**Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).**

Check every item and prerequisite.

Create a table:

| Topic | Covered? | Revision Done? | PYQs Done? |
|-------|----------|----------------|------------|
|-------|----------|----------------|------------|

If anything is missing, teach it BEFORE declaring the subject finished.

---

## FINAL EXAM PREPARATION

---

After completing the syllabus, provide:

### Phase 1 — Chapter Revision

---

Fast revision of every chapter.

### Phase 2 — PYQ Revision

---

Organize PYQs by concept rather than randomly.

### Phase 3 — Mixed Tests

---

Give mixed Digital Logic tests.

---

## Phase 4 — Error Analysis

---

Classify my mistakes into:

- Concept gap
- Formula gap
- Calculation error
- Misreading
- Trap
- Time-management problem

## Phase 5 — Final Cheat Sheet

---

Give a last-day Digital Logic revision sheet.

---

## TEACHING STYLE

---

Use **simple English**.

Assume I am capable but learning some concepts for the first time.

Prefer:

**intuition !' definition !' equation/table !' example !' shortcut !' GATE question !' trap !' revision**

rather than textbook-style theory.

Whenever mathematics looks complicated, explain what it means before manipulating equations.

For every truth table, explain WHY the output takes that value rather than asking me to memorize blindly.

---

## MOST IMPORTANT INSTRUCTION

---

My problem is lack of time, not lack of willingness.

Therefore give me the **complete information needed for GATE**, while aggressively removing:

- unnecessary historical detail
- irrelevant electronics theory
- non-GATE hardware depth
- repetitive questions
- filler

But never remove a concept merely because it is difficult.

The goal is:

**ZERO IMPORTANT GATE DIGITAL LOGIC GAPS + MAXIMUM TIME EFFICIENCY.**

---

## START NOW

---

First:

1. Show me the final **Digital Logic Master Roadmap** in correct learning order.
-

2. Mark  $\emptyset$ =MUST MASTER /  $\emptyset$ =IMPORTANT /  $\emptyset$ =SUPPORT.
3. Estimate roughly what percentage of the total subject each major unit represents.
4. Then immediately begin **Chapter 1 from the absolute basics**.
5. Do not wait for another confirmation.

...

You can paste this as the **first message in a fresh Digital Logic chat**. It is designed so that after that, you can mostly just keep sending “**NEXT**” and progress through the entire subject without repeatedly explaining what you want.

---

USER

## GATE DIGITAL LOGIC — COMPLETE MASTER TEACHER PROMPT

I am preparing for **GATE Computer Science** and I have limited time.

I do **NOT** have the flexibility to watch tens or hundreds of hours of YouTube lectures.

I want to learn **Digital Logic completely from this chat** and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE Digital Logic subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- numerical problem-solving trainer
- circuit-design teacher
- revision coach
- shortcut and memory-trick creator
- misconception detector
- exam strategy mentor

---

## MY CURRENT SUBJECT

Digital Logic

## OFFICIAL GATE SYLLABUS

Digital Logic:

**Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).**

Treat the above as the official syllabus.

However, do NOT teach only the literal words written in the syllabus.

Expand every syllabus point into all prerequisite concepts, subtopics, standard circuit types, numerical techniques, representations, formulas and question patterns required to solve GATE questions.



---

## MAIN GOAL

Convert the complete Digital Logic syllabus into a **self-contained GATE course inside this chat**.

Assume I may use your explanations as my PRIMARY learning source.

Therefore:

**Do not skip concepts just because they are normally taught in a video lecture or textbook.**

At the end I should be able to:

1. Understand every Digital Logic concept relevant to GATE.
2. Solve conceptual questions.
3. Solve numerical questions.
4. Analyze logic circuits.
5. Design combinational circuits.
6. Analyze and design sequential circuits.
7. Simplify Boolean functions using every required method.
8. Solve number-representation problems.
9. Solve fixed-point arithmetic questions.
10. Solve IEEE/floating-point questions.
11. Recognize common GATE traps.
12. Solve previous-year-question patterns independently.
13. Revise the whole subject quickly before GATE.

---

## IMPORTANT TEACHING RULE

I am learning this primarily from CHAT.

Never say:

- "You probably already know this."
- "This is basic, so we can skip it."
- "Refer to a textbook for details."
- "Watch a video to understand this."
- "This is outside the syllabus" when it is actually needed as a prerequisite.

Explain every prerequisite necessary to understand the GATE syllabus.

But also avoid wasting time on irrelevant electronics or hardware theory that GATE CS does not require.

Teach for:

**Maximum GATE marks per unit of study time.**

---

## COMPLETE DIGITAL LOGIC ROADMAP

Before teaching, create a master roadmap and track progress through it.

At minimum, cover all of the following.

---

## UNIT 1 — NUMBER SYSTEM FUNDAMENTALS

---

Teach:

- Number systems
- Radix/base
- Positional representation
- Binary
- Octal
- Decimal
- Hexadecimal
- Conversion between bases
- Decimal to binary
- Binary to decimal
- Decimal to octal
- Decimal to hexadecimal
- Binary to octal
- Binary to hexadecimal
- Conversion involving fractional numbers
- Repeated division method
- Repeated multiplication method
- Unknown-radix problems
- Arithmetic in different bases
- n-bit representations
- Range of unsigned numbers

Include shortcuts for fast base conversion.

---

## UNIT 2 — SIGNED NUMBER REPRESENTATION

---

Teach completely:

- Unsigned representation
- Signed magnitude representation
- 1's complement
- 2's complement
- Representation of positive and negative numbers
- Negation
- Number of representable values
- Range formulas
- Positive zero / negative zero
- Why 2's complement has one extra negative number
- Sign bit
- Sign extension

- Zero extension
- Truncation
- Converting signed values between different bit widths

Create a comparison table for:

- unsigned
- sign magnitude
- 1's complement
- 2's complement

Include all important range formulas.

---

## UNIT 3 — BINARY ARITHMETIC

---

Teach:

- Binary addition
- Binary subtraction
- Carry
- Borrow
- Addition using complements
- Subtraction using complements
- Signed addition
- Signed subtraction
- Carry-out vs overflow
- Overflow detection
- Overflow rules
- 2's-complement arithmetic
- Multiplication basics
- Binary multiplication
- Division basics where required
- Shift operations related to arithmetic
- Arithmetic shift
- Logical shift
- Left/right shifting and multiplication/division by powers of 2

Explain GATE traps involving:

**carry " overflow**

---

## UNIT 4 — BOOLEAN ALGEBRA FUNDAMENTALS

---

Teach from zero:

- Boolean variables
- Binary logic
- Truth tables
- AND

- OR
- NOT
- NAND
- NOR
- XOR
- XNOR
- Buffer if relevant

For every gate give:

- symbol meaning
- Boolean expression
- truth table
- important properties

Then teach Boolean laws:

- Identity law
- Null law
- Idempotent law
- Complement law
- Involution law
- Commutative law
- Associative law
- Distributive law
- Absorption law
- De Morgan's laws
- Consensus theorem
- Principle of duality

Derive important identities instead of merely listing them.

Teach shortcuts for simplifying Boolean expressions quickly.

---

## UNIT 5 — BOOLEAN FUNCTIONS AND REPRESENTATION

---

Teach:

- Boolean functions
- Truth tables
- Literals
- Product terms
- Sum terms
- Minterms
- Maxterms
- SOP
- POS
- Standard SOP
- Standard POS

- Canonical SOP
- Canonical POS
- Sum of minterms
- Product of maxterms
- $\Sigma$  notation
- $\Pi$  notation
- Indexing minterms/maxterms
- Conversion between truth table and Boolean expression
- Conversion between SOP and POS
- Complement relationships
- Finding  $F'$
- Missing minterms/maxterms

Include tricky GATE questions involving minterm/maxterm numbering.

---

## UNIT 6 — LOGIC GATE REALIZATION

---

Teach:

- Universal gates
- Why NAND is universal
- Why NOR is universal
- AND using NAND
- OR using NAND
- NOT using NAND
- AND using NOR
- OR using NOR
- NOT using NOR
- NAND-only realization
- NOR-only realization
- Two-level implementation
- Multi-level implementation
- Bubble pushing if useful
- De Morgan transformation
- XOR/XNOR realization
- Gate-count optimization where relevant

Teach how Boolean simplification affects hardware implementation.

---

## UNIT 7 — BOOLEAN MINIMIZATION: ALGEBRAIC METHOD

---

Teach:

- Algebraic simplification
- Applying Boolean identities efficiently
- Factoring

- Absorption
- Consensus theorem
- Redundant terms
- Finding minimum expressions
- SOP simplification
- POS simplification

For every example show:

Original expression !' identity used !' simplified expression.

Train me to recognize common patterns mentally.

---

## UNIT 8 — KARNAUGH MAP

---

Teach K-map extremely thoroughly.

Cover:

- Purpose of K-map
- Gray-code ordering
- Adjacency
- Wrap-around adjacency
- Corners
- Group sizes
- Powers-of-two rule
- 2-variable K-map
- 3-variable K-map
- 4-variable K-map
- 5-variable idea if GATE relevant
- SOP minimization
- POS minimization
- Grouping 1s
- Grouping 0s
- Overlapping groups
- Largest-group rule
- Redundant groups
- Prime implicants
- Essential prime implicants
- Non-essential prime implicants
- Implicant terminology
- Don't-care conditions
- Using X/don't cares optimally
- Minimal SOP
- Minimal POS
- Multiple minimum solutions
- K-map from minterms

- K-map from maxterms
- K-map from truth table
- K-map from Boolean expression

Explain why variables disappear when cells are grouped.

Teach all common K-map traps.

---

## UNIT 9 — TABULAR METHOD / QUINE–McCLUSKEY METHOD

---

Teach from scratch:

- Why tabular minimization exists
- Relation with K-map
- Listing minterms
- Grouping by number of 1s
- Combining adjacent terms
- Dash notation
- Repeated combination
- Prime implicants
- Prime implicant chart
- Essential prime implicants
- Cover selection
- Don't-care handling
- Final minimized expression

Solve complete examples step-by-step.

Do not skip the intermediate tables.

Also explain when GATE may prefer this method over K-map.

---

## UNIT 10 — COMBINATIONAL CIRCUITS: FUNDAMENTALS

---

Teach:

- What is a combinational circuit?
- Input-output relationship
- No memory property
- Propagation delay
- How to design a combinational circuit from a specification

General design procedure:

1. Identify inputs/outputs
2. Assign variables
3. Create truth table
4. Obtain Boolean expressions
5. Minimize
6. Implement circuit

---

## UNIT 11 — ADDERS AND SUBTRACTORS

---

Teach completely:

- Half adder
- Full adder
- Half subtractor
- Full subtractor

For each:

- block idea
- truth table
- equations
- circuit realization
- relationship between circuits

Then cover:

- n-bit parallel adder
- Ripple Carry Adder
- Carry propagation
- Propagation delay
- Worst-case delay
- Carry look-ahead concepts if needed for GATE
- Carry generate
- Carry propagate
- Faster addition concepts
- Adder-subtractor circuit

Include numerical timing questions.

---

## UNIT 12 — MULTIPLEXERS

---

Teach:

- Multiplexer concept
- 2:1 MUX
- 4:1 MUX
- 8:1 MUX
- 16:1 MUX
- Select lines
- Relationship:

For  $2^n$  inputs:

number of select lines =  $n$

Teach:

- MUX truth tables



- MUX Boolean equation
- Implementing larger MUX using smaller MUXes
- Implementing Boolean functions using MUX
- Choosing variables as select lines
- Using remaining variables at data inputs
- Minimum MUX questions
- MUX tree
- Propagation-delay questions

Give many GATE-style function-realization questions.

---

## UNIT 13 — DEMULTIPLEXERS, DECODERS AND ENCODERS

---

Teach:

### Demultiplexer

---

- Concept
- Select lines
- Implementations

### Decoder

---

- $n$ -to- $2^n$  decoder
- Enable input
- Decoder expansion
- Boolean-function realization using decoder
- Decoder as minterm generator

### Encoder

---

- Encoder concept
- Standard encoder
- Priority encoder
- Handling multiple active inputs
- Valid bit

Explain differences clearly:

MUX vs DEMUX vs Decoder vs Encoder.

---

## UNIT 14 — COMPARATOR AND OTHER COMBINATIONAL CIRCUITS

---

Teach:

- Equality comparator
- Magnitude comparator
- 1-bit comparator
- Multi-bit comparison

- Cascading comparators

Also cover any other combinational-circuit concept repeatedly relevant to GATE.

---

## UNIT 15 — SEQUENTIAL CIRCUITS FUNDAMENTALS

---

Teach carefully:

- Combinational vs sequential circuits
- Memory
- State
- Feedback
- Clock
- Clock signal
- Positive edge
- Negative edge
- Level triggering
- Edge triggering
- Synchronous systems
- Asynchronous systems

Timing concepts:

- Propagation delay
- Setup time
- Hold time
- Clock-to-Q delay
- Clock skew if relevant
- Maximum operating frequency concepts

Use timing diagrams wherever needed.

---

## UNIT 16 — LATCHES AND FLIP-FLOPS

---

Teach every important device.

### SR latch

---

- NOR implementation
- NAND implementation
- Truth table
- forbidden/invalid states

### SR flip-flop

---

### D latch / D flip-flop

---

### JK flip-flop

---

## T flip-flop

---

For each teach:

- operation
- truth table
- characteristic table
- characteristic equation
- excitation table
- timing behavior
- applications

Create ONE master comparison table.

---

## UNIT 17 — FLIP-FLOP CONVERSIONS

---

Teach how to convert:

- SR !' JK
- SR !' D
- SR !' T
- JK !' SR
- JK !' D
- JK !' T
- D !' SR
- D !' JK
- D !' T
- T !' SR
- T !' JK
- T !' D

Teach the standard conversion procedure using:

- desired flip-flop behavior
- excitation table
- K-map
- input equations

Focus more on patterns commonly asked in GATE.

---

## UNIT 18 — RACE-AROUND AND MASTER-SLAVE

---

Teach:

- Race-around condition
- Why it occurs
- JK flip-flop behavior
- Clock pulse width
- Methods to eliminate race-around

- Edge-triggered solution
- Master-slave JK flip-flop

Explain intuitively and mathematically where useful.

## UNIT 19 — REGISTERS AND SHIFT REGISTERS

Teach:

- Register
- Parallel register
- Shift register

Types:

- SISO
- SIPO
- PISO
- PIPO

Then:

- Left shift
- Right shift
- Bidirectional shift register
- Universal shift register
- Serial transfer
- Parallel transfer
- Register timing questions

Explain what happens after each clock pulse.

## UNIT 20 — COUNTERS

Teach very thoroughly.

Cover:

- Counter concept
- Counter modulus
- MOD-N
- Number of required flip-flops

Formula:

MATH

$$n = \lceil \log_2 N \rceil$$

Teach:

- Asynchronous counter
- Ripple counter
- Synchronous counter

- Up counter
- Down counter
- Up/down counter
- Binary counter
- Decade counter
- MOD-N counter
- Truncated counter
- Counter state sequence
- Counter design
- Maximum count
- Frequency division
- Propagation-delay analysis
- Maximum clock-frequency questions

Then:

- Ring counter
- Johnson/twisted-ring counter

Include formulas for the number of states.

## UNIT 21 — SEQUENTIAL CIRCUIT ANALYSIS

Teach the complete method for analyzing a given sequential circuit.

Cover:

- Present state
- Next state
- Inputs
- Outputs
- State variables
- Flip-flop input equations
- Characteristic equations
- Next-state equations
- State table
- State diagram

Given a circuit, teach me how to derive its behavior systematically.

## UNIT 22 — SEQUENTIAL CIRCUIT DESIGN

Teach:

- Converting specification to state diagram
- State table
- State assignment
- Flip-flop selection
- Excitation requirements

- K-map minimization
- Logic implementation

Do complete examples.

---

## UNIT 23 — FINITE STATEMACHINES

---

Teach:

- FSM concept
- Moore machine
- Mealy machine
- Moore vs Mealy
- Output dependence
- State diagrams
- State tables
- Conversion if relevant
- State reduction/minimization concepts if relevant

Focus especially on GATE circuit questions.

---

## UNIT 24 — SEQUENCE DETECTORS

---

Teach:

- Sequence detector concept
- Overlapping detector
- Non-overlapping detector
- Moore implementation
- Mealy implementation
- State construction
- Handling partial matches
- Designing arbitrary bit-sequence detectors

Solve examples such as detecting:

101 1011 1101

and then harder GATE-style cases.

---

## UNIT 25 — FIXED-POINT REPRESENTATION

---

Teach:

- Fixed-point concept
- Binary point
- Integer bits
- Fraction bits
- Signed fixed-point numbers

- Unsigned fixed-point numbers
- Range
- Resolution
- Precision
- Quantization
- Representable values
- Overflow
- Underflow where applicable

Teach how changing the number of integer/fraction bits affects:

- range
- precision

Provide formulas and numerical questions.

---

## UNIT 26 — FLOATING-POINT FUNDAMENTALS

---

Before IEEE 754, explain:

- Scientific notation
- Binary scientific notation
- Normalization
- Sign
- Significand/mantissa
- Exponent
- Bias
- Precision
- Range

Show examples such as:

MATH

10110.101<sub>2</sub>

becoming normalized binary scientific notation.

---

## UNIT 27 — IEEE 754 FLOATING POINT

---

Teach completely.

### IEEE 754 Single Precision

---

- 32 bits
- 1 sign bit
- 8 exponent bits
- 23 fraction bits
- bias = 127

## IEEE 754 Double Precision

---

- 64 bits
- 1 sign bit
- 11 exponent bits
- 52 fraction bits
- bias = 1023

Teach:

- biased exponent
- hidden/implicit leading 1
- normalized numbers
- subnormal/denormalized numbers
- +0
- -0
- "+"
- "-"
- NaN
- exponent patterns
- significand calculation
- encoding decimal !' IEEE
- decoding IEEE !' value
- smallest/largest numbers where GATE relevant
- precision
- rounding basics
- floating-point errors
- comparison with fixed point

Solve complete bit-pattern numericals.

---

## PYQ REQUIREMENT

---

This is extremely important.

For EVERY major chapter:

1. Identify the kinds of questions GATE has historically asked.
2. Mention the relevant GATE year when confidently known.
3. Never invent a PYQ or falsely label a question as an actual GATE PYQ.
4. If exact wording cannot be provided or verified, give a faithful paraphrase and label it clearly.
5. Separate:

### ACTUAL/PARAPHRASED PYQ

from

### PYQ-STYLE PRACTICE QUESTION

6. Explain why each option is correct/incorrect for MCQs.
7. Show full working for numerical-answer questions.



8. Explain the shortest GATE method after showing the proper method.
9. Identify the concept tested.
10. Explain the trap being tested.

Cover old and recent useful GATE CS Digital Logic PYQ patterns.

---

## QUESTION TYPES

---

Train me for all GATE formats:

### MCQ

One correct option.

### MSQ

One or more correct options.

### NAT

Numerical answer.

For each chapter provide all applicable types.

---

## FOUR LEVELS OF PRACTICE

---

Use four levels.

### L0 — Foundation

Very simple questions that verify basic understanding.

### L1 — Standard

Direct application of one concept.

### L2 — GATE Level

Multi-step or deceptive exam-level questions.

### L3 — Advanced/Trap

Questions combining concepts or containing typical GATE traps.

Do not overwhelm me with hundreds of repetitive questions.

Choose enough problems to cover all important patterns.

---

## FORMAT FOR TEACHING EACH CONCEPT

---

Whenever possible use:

### 1. Intuition

---

Explain what the idea actually means.

## 2. Formal Definition

---

Give the precise exam definition.

## 3. Diagram / Truth Table / Equation

---

Whichever applies.

## 4. Step-by-Step Example

---

Start simple.

## 5. GATE-Level Example

---

Increase difficulty.

## 6. Shortcut

---

Show the fastest safe exam technique.

## 7. Common Trap

---

Explain how students lose marks.

## 8. PYQ Connection

---

Explain how GATE asks it.

## 9. Practice

---

Give selected questions.

## 10. Micro Revision

---

Give a 30-second revision summary.

---

## NUMERICAL SOLVING RULE

---

For every important numerical question teach TWO methods whenever useful:

### METHOD A — NORMAL METHOD

Detailed reasoning for learning.

### METHOD B — GATE METHOD

Fastest safe way to solve during the exam.

Clearly identify where shortcuts are safe and where they can cause mistakes.

---

## FORMULA SHEET

---

Maintain a continuously growing Digital Logic formula/shortcut sheet.

It should eventually contain things such as:

- unsigned ranges
- signed ranges
- complement relationships
- minterm/maxterm rules
- MUX relations
- decoder relations
- counter formulas
- propagation delay formulas
- fixed-point formulas
- IEEE 754 formulas
- all other required formulas

At the end provide one consolidated formula sheet.

---

## TRAP BOOK

---

Maintain a separate **GATE Digital Logic Trap Book**.

Examples:

- Carry confused with overflow
- Sign extension performed incorrectly
- Wrong minterm ordering
- Wrong K-map adjacency
- Diagonal K-map cells treated as adjacent
- Creating non-power-of-two K-map groups
- Forgetting wrap-around groups
- Using don't-cares unnecessarily
- Confusing prime implicant and essential prime implicant
- Wrong MUX select-line count
- Decoder input/output reversal
- Latch vs flip-flop confusion
- Characteristic table vs excitation table confusion
- Race-around misunderstanding
- Ripple vs synchronous counter confusion
- Moore vs Mealy output confusion
- IEEE exponent stored value confused with actual exponent
- Hidden bit incorrectly used for subnormal numbers

Add traps as we progress.

---

## MEMORY TRICKS

---

Give short memory tricks wherever helpful.

Examples should be genuinely useful, not childish or confusing.

Create tricks for:

- Boolean identities
  - XOR/XNOR
  - minterms/maxterms
  - K-map grouping
  - flip-flop tables
  - counter rules
  - IEEE 754
  - signed-number ranges
- 

## NOTES-MAKING MODE

---

Write explanations so that I can directly make notes.

Use:

- clear headings
- short paragraphs
- equations
- tables
- truth tables
- bullet points
- ASCII diagrams when useful
- compact summaries

Do not produce enormous walls of prose.

However, **concise does NOT mean incomplete**.

---

## IMPORTANT CIRCUIT-DIAGRAM RULE

---

Whenever a circuit is important:

1. Explain its purpose.
2. Show Boolean equations.
3. Show truth table.
4. Show a simple text/ASCII representation if possible.
5. Explain signal flow.
6. Explain how GATE may modify or combine the circuit.

Do not assume I can understand a circuit from its name alone.

---

## TIMING-DIAGRAM RULE

---

For questions involving:

- flip-flops
- latches
- clocks
- counters
- registers
- sequential circuits

teach me how to manually trace the waveform.

Always mark:

- clock edge
- input value
- present Q
- next Q

where relevant.

---

## REVISION SYSTEM

---

At the end of each major unit provide:

- 1. One-page revision notes**
- 2. Formula/identity list**
- 3. Common traps**
- 4. 5–10 important questions**
- 5. PYQ patterns**
- 6. What I absolutely must remember**

At the end of the entire subject provide:

- complete Digital Logic revision sheet
  - formula sheet
  - Boolean identity sheet
  - K-map cheat sheet
  - flip-flop master table
  - combinational circuit master table
  - counter master sheet
  - IEEE 754 cheat sheet
  - top GATE traps
  - high-frequency PYQ concepts
  - final mixed mock test
-

## HIGH-PRIORITY LABELS

---

Label topics:

### ~~0=1~~ MUST MASTER

Very frequently useful/high-value GATE concepts.

### ~~0=2~~ IMPORTANT

Important but relatively smaller.

### ~~0=3~~ SUPPORT

Needed mainly as foundation.

Use this to help me allocate my limited time.

---

## COMPLETION TRACKER

---

Maintain a Digital Logic syllabus tracker.

After every chapter show only a compact progress line such as:

**Completed:** Boolean Algebra **Current:** K-Maps **Remaining:** Tabular Method !' Combinational !' Sequential !'  
Number Arithmetic !' Fixed/Floating Point

Never claim the subject is completed until every syllabus item and prerequisite listed in this prompt has actually been covered.

---

## "NEXT" PROTOCOL

---

Do NOT dump the entire Digital Logic course in one response.

Teach it chapter by chapter in the correct prerequisite order.

At the end of each response tell me exactly what comes next.

When I say:

### NEXT

immediately continue from the exact point where we stopped.

Do not restart.

Do not ask me what chapter I want next.

Do not unnecessarily repeat previous explanations.

Maintain continuity until the entire Digital Logic course is complete.

---

## BEFORE DECLARING DIGITAL LOGIC COMPLETE

---

Perform a final coverage audit against:

**Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).**

Check every item and prerequisite.

---

Create a table:

| Topic | Covered? | Revision Done? | PYQs Done? |
|-------|----------|----------------|------------|
|-------|----------|----------------|------------|

If anything is missing, teach it BEFORE declaring the subject finished.

## FINAL EXAM PREPARATION

After completing the syllabus, provide:

### Phase 1 — Chapter Revision

Fast revision of every chapter.

### Phase 2 — PYQ Revision

Organize PYQs by concept rather than randomly.

### Phase 3 — Mixed Tests

Give mixed Digital Logic tests.

### Phase 4 — Error Analysis

Classify my mistakes into:

- Concept gap
- Formula gap
- Calculation error
- Misreading
- Trap
- Time-management problem

### Phase 5 — Final Cheat Sheet

Give a last-day Digital Logic revision sheet.

## TEACHING STYLE

Use **simple English**.

Assume I am capable but learning some concepts for the first time.

Prefer:

**intuition !' definition !' equation/table !' example !' shortcut !' GATE question !' trap !' revision**

rather than textbook-style theory.

Whenever mathematics looks complicated, explain what it means before manipulating equations.

For every truth table, explain WHY the output takes that value rather than asking me to memorize blindly.

# MOST IMPORTANT INSTRUCTION

My problem is lack of time, not lack of willingness.

Therefore give me the **complete information needed for GATE**, while aggressively removing:

- unnecessary historical detail
- irrelevant electronics theory
- non-GATE hardware depth
- repetitive questions
- filler

But never remove a concept merely because it is difficult.

The goal is:

**ZERO IMPORTANT GATE DIGITAL LOGIC GAPS + MAXIMUM TIME EFFICIENCY.**

# START NOW

- First:
1. Show me the final **Digital Logic Master Roadmap** in correct learning order.
  2. Mark ~~Ø=NOT~~ MUST MASTER / ~~Ø=NOT~~ IMPORTANT / ~~Ø=NOT~~ SUPPORT.
  3. Estimate roughly what percentage of the total subject each major unit represents.
  4. Then immediately begin **Chapter 1 from the absolute basics**.
  5. Do not wait for another confirmation.

ASSISTANT

# GATE DIGITAL LOGIC — MASTER COURSE

We'll follow this exact course from now on. I'll keep continuity when you say **NEXT**, and I will not declare Digital Logic complete until we perform the final coverage audit.

# 1. Master Roadmap

The percentages below are **study-weight estimates**, not official GATE marks. They tell you approximately how much effort each block deserves.

| Order | Unit                                          | Priority                     | Approx. Study Weight |
|-------|-----------------------------------------------|------------------------------|----------------------|
| 1     | Number System Fundamentals                    | <del>Ø=NOT</del> SUPPORT     | 5%                   |
| 2     | Signed Number Representation                  | <del>Ø=NOT</del> MUST MASTER | 5%                   |
| 3     | Binary Arithmetic                             | <del>Ø=NOT</del> MUST MASTER | 5%                   |
| 4     | Boolean Algebra Fundamentals                  | <del>Ø=NOT</del> MUST MASTER | 7%                   |
| 5     | Boolean Functions, SOP/POS, Minterms/Maxterms | <del>Ø=NOT</del> MUST MASTER | 6%                   |



| Order | Unit                                      | Priority         | Approx. Study Weight |
|-------|-------------------------------------------|------------------|----------------------|
| 6     | NAND/NOR & Gate Realization               | Ø=ßà IMPORTANT   | 4%                   |
| 7     | Algebraic Minimization                    | Ø=Ý4 MUST MASTER | 4%                   |
| 8     | Karnaugh Maps                             | Ø=Ý4 MUST MASTER | 5%                   |
| 9     | Tabular / Quine–McCluskey Method          | Ø=ßà IMPORTANT   | 4%                   |
| 10    | Combinational Circuit Fundamentals        | Ø=ßà IMPORTANT   | 2%                   |
| 11    | Adders/Subtractors & Carry                | Ø=Ý4 MUST MASTER | 6%                   |
| 12    | Multiplexers                              | Ø=Ý4 MUST MASTER | 7%                   |
| 13    | Decoder/Encoder/DEMUX                     | Ø=Ý4 MUST MASTER | 5%                   |
| 14    | Comparator & Other Combinational Circuits | Ø=ßà IMPORTANT   | 2%                   |
| 15    | Sequential Circuit Fundamentals & Timing  | Ø=Ý4 MUST MASTER | 5%                   |
| 16    | Latches & Flip-Flops                      | Ø=Ý4 MUST MASTER | 7%                   |
| 17    | Flip-Flop Conversion                      | Ø=ßà IMPORTANT   | 3%                   |
| 18    | Race Around & Master-Slave                | Ø=ßà IMPORTANT   | 2%                   |
| 19    | Registers & Shift Registers               | Ø=ßà IMPORTANT   | 3%                   |
| 20    | Counters                                  | Ø=Ý4 MUST MASTER | 6%                   |
| 21    | Sequential Circuit Analysis               | Ø=Ý4 MUST MASTER | 4%                   |
| 22    | Sequential Circuit Design                 | Ø=Ý4 MUST MASTER | 3%                   |
| 23    | FSM — Moore/Mealy                         | Ø=Ý4 MUST MASTER | 3%                   |
| 24    | Sequence Detectors                        | Ø=ßà IMPORTANT   | 2%                   |
| 25    | Fixed-Point Representation                | Ø=Ý4 MUST MASTER | 3%                   |
| 26    | Floating-Point Fundamentals               | Ø=Ý4 MUST MASTER | 2%                   |
| 27    | IEEE-754 Floating Point                   | Ø=Ý4 MUST MASTER | 5%                   |

### Highest-return areas

If your revision time becomes extremely short, prioritize:

**Boolean Algebra !' SOP/POS !' K-map !' MUX/Decoder !' Adders !' Flip-Flops !' Counters !' Sequential Analysis !' Signed Arithmetic !' IEEE-754**

But for first learning, we go in prerequisite order.

## CHAPTER 1 — NUMBER SYSTEM FUNDAMENTALS Ø=ßà

Digital circuits ultimately work with two logical states:

$\{ 0, 1 \}$

Therefore, before Boolean algebra and circuits, you need to be comfortable with **binary numbers**.

## 1. What is a Number System?

A number system is a way of representing quantities using a set of symbols.

For example, decimal uses:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Binary uses:

$\{0, 1\}$

Hexadecimal uses:

$\{0, 1, \dots, 9, A, B, C, D, E, F\}$

## 2. Radix / Base

The **radix** or **base** tells us how many different digits the system contains.

| System      | Base | Allowed digits |
|-------------|------|----------------|
| Binary      | 2    | 0, 1           |
| Octal       | 8    | 0–7            |
| Decimal     | 10   | 0–9            |
| Hexadecimal | 16   | 0–9, A–F       |

Important:

$\boxed{\text{Every digit must be smaller than the base}}$

Example:

$(1021)_2$

is invalid because binary cannot contain digit 2.

Similarly:

$(897)_8$

is invalid because octal cannot contain 8 or 9.

### Memory trick

#### BOHD

- Binary = 2
- Octal = 8
- Hexadecimal = 16
- Decimal = 10

The numerical order is easier to remember as:

$\{2, 8, 10, 16\}$

---

### 3. Positional Representation

---

Consider:

$$[3257_{10}]$$

This actually means:

$$[3 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 7 \times 10^0]$$

In any base  $(r)$ :

$$[(a_{n-1} \dots a_1 a_0)_r]$$

means

$$[a_n r^n + a_{n-1} r^{n-1} + \dots + a_1 r + a_0]$$

This is the most fundamental conversion formula.

---

### 4. Binary !' Decimal

---

Convert:

$$[(101101)_2]$$

Write powers of 2:

$$[1(2^5) + 0(2^4) + 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0)]$$

$$[= 32 + 0 + 8 + 4 + 0 + 1]$$

$$[\boxed{45}]$$

Therefore:

$$[\boxed{101101_2 = 45_{10}}]$$

---

### GATE Shortcut

---

For a binary number, memorize powers of 2:

$$[1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]$$

Take only the weights corresponding to bits equal to 1.

Example:

$$[110101_2]$$

Weights:

$$[32, 16, 8, 4, 2, 1]$$

Bits:

$$[1, 1, 0, 1, 0, 1]$$

Hence:

$$[32 + 16 + 4 + 1 = 53]$$

So:

$$\boxed{110101_2 = 53_{10}}$$

---

## 5. Octal !' Decimal

Convert:

$$(725)_8$$

$$7(8^2) + 2(8^1) + 5(8^0)$$

$$= 7(64) + 2(8) + 5$$

$$= 448 + 16 + 5$$

$$\boxed{469}$$

---

## 6. Hexadecimal !' Decimal

Hexadecimal requires these values:

$$A = 10$$

$$B = 11$$

$$C = 12$$

$$D = 13$$

$$E = 14$$

$$F = 15$$

Example:

$$(2AF)_{16}$$

$$2(16^2) + 10(16^1) + 15$$

$$= 512 + 160 + 15$$

$$\boxed{687}$$

---

## 7. Decimal !' Binary

Use repeated division by 2.

Convert:

$$45_{10}$$

| Division    | Quotient | Remainder |
|-------------|----------|-----------|
| $45 \div 2$ | 22       | 1         |
| $22 \div 2$ | 11       | 0         |
| $11 \div 2$ | 5        | 1         |

| Division   | Quotient | Remainder |
|------------|----------|-----------|
| $5 \div 2$ | 2        | 1         |
| $2 \div 2$ | 1        | 0         |
| $1 \div 2$ | 0        | 1         |

Now read the remainders:

$\boxed{\text{bottom !' top}}$

giving:

$\boxed{101101_2}$

## Memory Rule

For integer conversion:

**Divide repeatedly by the destination base and read the remainders upward.**

## 8. Decimal !' Octal

Convert:

$83_{10}$

$83 \div 8 = 10, \text{quad } R=3$

$10 \div 8 = 1, \text{quad } R=2$

$1 \div 8 = 0, \text{quad } R=1$

Read bottom to top:

$\boxed{123_8}$

Check:

$1(64) + 2(8) + 3 = 83$

## 9. Decimal !' Hexadecimal

Convert:

$254_{10}$

$254 \div 16 = 15, \text{quad } R=14$

Now:

$14 = E$

and

$15 = F$

Therefore:

$$\boxed{254_{10} = FE_{16}}$$

---

## 10. Binary to Octal

---

Why is conversion easy?

Because:

$$8 = 2^3$$

Therefore:

$$\boxed{1 \text{ octal digit} = 3 \text{ binary bits}}$$

Example:

$$10111011_2$$

Group 3 bits from the right:

$$101 \quad 111 \quad 011$$

Convert each:

$$101 = 5$$

$$111 = 7$$

$$011 = 3$$

Therefore:

$$\boxed{10111011_2 = 573_8}$$

---

## 11. Octal to Binary

---

Reverse the process.

Convert:

$$(572)_8$$

Each octal digit becomes 3 bits:

$$5 = 101$$

$$7 = 111$$

$$2 = 010$$

Therefore:

$$\boxed{572_8 = 101111010_2}$$

---

## 12. Binary to Hexadecimal

---

Because:

$$16 = 2^4$$

therefore:

$\boxed{1 \text{ hexadecimal digit} = 4 \text{ bits}}$

Example:

$101111101001_2$

Group:

$1011 \quad 1110 \quad 1001$

Convert:

$1011 = B$

$1110 = E$

$1001 = 9$

Hence:

$\boxed{BE9_{16}}$

## Must-Know Binary !”Hex Table

| Binary | Hex |
|--------|-----|
| 0000   | 0   |
| 0001   | 1   |
| 0010   | 2   |
| 0011   | 3   |
| 0100   | 4   |
| 0101   | 5   |
| 0110   | 6   |
| 0111   | 7   |
| 1000   | 8   |
| 1001   | 9   |
| 1010   | A   |
| 1011   | B   |
| 1100   | C   |
| 1101   | D   |
| 1110   | E   |
| 1111   | F   |

For GATE, this should eventually become automatic.

---

## 13. Hexadecimal to Binary

Example:

$(3AF)_{16}$

Convert each hex digit into 4 bits:

$3 = 0011$

$A = 1010$

$F = 1111$

Therefore:

$\boxed{001110101111}_2$

Leading zero can be removed when bit width is not fixed:

$\boxed{1110101111}_2$

---

## GATE TRAP

If the question specifies a bit-width, **do not remove leading zeros**.

Example:

"Represent  $(3A)_{16}$  using 8 bits."

Correct:

$\boxed{00111010}$

not simply  $(111010)$ .

---

## 14. Fractional Positional Values

For digits to the right of the binary point:

$2^{-1}, 2^{-2}, 2^{-3}, \dots$

Example:

$101.101_2$

Integer part:

$1(2^2) + 0(2^1) + 1(2^0)$

Fraction:

$1(2^{-1}) + 0(2^{-2}) + 1(2^{-3})$

Hence:

$4 + 1 + \frac{1}{2} + \frac{1}{8}$

$= 5 + 0.5 + 0.125$

$\boxed{5.625}$

---



Therefore:

$$\boxed{101.101_2 = 5.625_{10}}$$

---

## 15. Decimal Fraction to Binary

---

Integer and fractional parts use different techniques.

For fractions:

$\boxed{\text{Repeated multiplication by 2}}$

Convert:

$$0.625_{10}$$

Step 1:

$$0.625 \times 2 = 1.25$$

take integer part:

$$1$$

Step 2:

$$0.25 \times 2 = 0.5$$

integer part:

$$0$$

Step 3:

$$0.5 \times 2 = 1.0$$

integer part:

$$1$$

Read from top to bottom:

$$\boxed{0.101_2}$$

---

## Ultimate Conversion Memory Trick

---

**Integer part**

$\boxed{\text{DIVIDE ! ' read UP}}$

**Fractional part**

$\boxed{\text{MULTIPLY ! ' read DOWN}}$

This distinction is frequently tested indirectly.

---

## 16. Example With Integer + Fraction

---

Convert:

$$10.625_{10}$$

---

First convert integer:

$$\lfloor 10_{10} = 1010_2 \rfloor$$

Fraction:

$$\lfloor 0.625 = 0.101_2 \rfloor$$

Combine:

$$\lfloor \boxed{10.625_{10} = 1010.101_2} \rfloor$$

---

## 17. Fractional Conversion Using Octal/Hex Shortcuts

---

Suppose:

$$\lfloor 110101.101011_2 \rfloor$$

### Binary to Octal

---

Group 3 bits around the point:

$$\lfloor 110 \text{ } 101.101 \text{ } 011 \rfloor$$

So:

$$\lfloor 6 \text{ } 5.53 \rfloor$$

Therefore:

$$\lfloor \boxed{65.53_8} \rfloor$$

---

### Binary to Hex

---

Suppose:

$$\lfloor 101101.101_2 \rfloor$$

Group into 4-bit groups.

Integer side:

$$\lfloor 0010 \text{ } 1101 \rfloor$$

Fraction side needs padding on the **right**:

$$\lfloor 1010 \rfloor$$

Therefore:

$$\lfloor 0010 = 2 \rfloor$$

$$\lfloor 1101 = D \rfloor$$

$$\lfloor 1010 = A \rfloor$$

So:

$$\lfloor \boxed{2D.A_{16}} \rfloor$$

---

## Important Padding Rule

---

For grouping conversions:

### Integer side

Pad zeros on the **left**.

Example:

$\backslash[10110\backslash]$

for hexadecimal:

$\backslash[0001\quad0110\backslash]$

### Fraction side

Pad zeros on the **right**.

Example:

$\backslash[.101\backslash]$

becomes:

$\backslash[.1010\backslash]$

---

## 18. Unknown Radix Problems

---

A classic exam idea.

Suppose:

$\backslash[(121)_{r=(16)_{\{10\}}}\backslash]$

Expand:

$\backslash[1r^2+2r+1=16\backslash]$

$\backslash[r^2+2r-15=0\backslash]$

$\backslash[(r+5)(r-3)=0\backslash]$

Possible roots:

$\backslash[ r=-5,\quad3\backslash]$

A number-system base must be valid and positive.

Also the largest digit appearing is 2, so:

$\backslash[ r>2\backslash]$

Therefore:

$\backslash[\boxed{r=3}\backslash]$

---

## Important Base Rule

---

If the largest digit occurring is  $\backslash(d\backslash)$ , then:

$\backslash[\boxed{\text{Base}>d}\backslash]$

---

Example:

$\lfloor (764)_r \rfloor$

contains digit 7.

Therefore:

$\lfloor \boxed{r \geq 8} \rfloor$

---

## 19. Arithmetic in Other Bases

---

### Binary Addition Rules

---

Memorize:

$\lfloor 0+0=0 \rfloor$

$\lfloor 0+1=1 \rfloor$

$\lfloor 1+0=1 \rfloor$

$\lfloor 1+1=10 \rfloor$

because decimal 2 is binary 10.

And:

$\lfloor 1+1+1=11 \rfloor$

---

### Example

---

$\lfloor 1011 \rfloor$

$\lfloor +0110 \rfloor$

From right:

$\lfloor 1+0=1 \rfloor$

Next:

$\lfloor 1+1=10 \rfloor$

write 0, carry 1.

Continue:

$\lfloor \boxed{10001} \rfloor$

Check decimal:

$\lfloor 11+6=17 \rfloor$

and:

$\lfloor 10001_2=17 \rfloor$

Correct.

---

## 20. n-Bit Unsigned Numbers

---

With  $(n)$  bits, how many different patterns are possible?

Each bit has 2 choices:

$[0 \text{ or } 1]$

Therefore:

$[2^n]$

patterns.

Minimum unsigned value:

$[0 \dots 0 = 0]$

Maximum:

$[11 \dots 1]$

which equals:

$[2^{n-1}]$

Thus:

$[\text{Unsigned n-bit range} = 0 \text{ to } 2^{n-1}]$

---

### Examples

---

#### 4 bits

$[2^4 = 16]$

patterns.

Range:

$[0 \text{ to } 15]$

#### 8 bits

$[2^8 = 256]$

patterns.

Range:

$[0 \text{ to } 255]$

#### 16 bits

$[0 \text{ to } 65535]$

---

### Why $(111 \dots 1 = 2^{n-1})$ ?

---

For  $(n)$  bits:

$[2^{n-1} + 2^{n-2} + \dots + 2^1 + 2^0]$

This is a geometric series:

---

$$\boxed{2^{n-1}}$$

A useful property to remember.

---

## 21. Minimum Bits Required to Represent a Number

---

Suppose an unsigned number must represent values:

$$0 \text{ to } N$$

We need:

$$2^n \geq N+1$$

So:

$$n = \lceil \log_2(N+1) \rceil$$

Example:

Need to represent 0–100.

There are:

$$101$$

possible values.

$$2^6 = 64$$

not enough.

$$2^7 = 128$$

enough.

Therefore:

$$\boxed{7 \text{ bits}}$$

---

## GATE Shortcut

---

Instead of calculating logs, compare nearby powers of 2.

Know:

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^{10} = 1024$$

$$2^{16} = 65536$$

$$2^{20} \approx 10^6$$

$$2^{30} \approx 10^9$$

Very useful throughout CS.

---

## 22. Number of Values vs Maximum Value

---

This is a common mistake.

For  $n$  unsigned bits:

### Number of patterns

$$2^n$$

### Maximum numerical value

$$2^n - 1$$

Example 8 bits:

Patterns:

$$256$$

Maximum value:

$$255$$

Do not confuse them.

---

## 23. Fast Base Conversion Method

---

General conversion:

$$\text{Base A} \rightarrow \text{Decimal} \rightarrow \text{Base B}$$

always works.

But if bases are powers of 2, do not waste time converting through decimal.

Examples:

### Binary $\rightarrow$ Octal

Use groups of:

$$3$$

### Binary $\rightarrow$ Hex

Use groups of:

$$4$$

### Octal $\rightarrow$ Hex

Fast method:

$$\text{Octal} \rightarrow \text{Binary} \rightarrow \text{Hex}$$

---

## Example

---

Convert:

$$(753)_8$$

to hexadecimal.

---

Octal to binary:

$$7 = 111$$

$$5 = 101$$

$$3 = 011$$

Thus:

$$111101011$$

Group into 4 from right:

$$0001 \ 1110 \ 1011$$

Therefore:

$$1 \ E \ B$$

$$\boxed{1EB_{16}}$$

This is much faster than decimal conversion.

---

## 24. Terminating vs Repeating Binary Fractions

Not every decimal fraction has a finite binary representation.

For example:

$$0.5 = \frac{1}{2}$$

terminates:

$$0.1_2$$

$$0.25 = \frac{1}{4}$$

terminates:

$$0.01_2$$

But decimal:

$$0.1_{10}$$

does **not** have a finite binary representation.

It repeats.

This becomes very important later in:

### IEEE-754 floating point

because it explains why computers cannot store many decimal fractions exactly.

---

## 25. GATE-Level Conceptual Example

How many bits are required to represent 1000 different unsigned values?

Need:

$$2^n \geq 1000$$



We know:

$$\sqrt[10]{2^9} = 512$$

$$\sqrt[10]{2^{10}} = 1024$$

Therefore:

$$\boxed{10}$$

### Trap

Question says **1000 different values**, not maximum value 1000.

If values are  $(0)$  through  $(1000)$ , there are:

$$\sqrt[10]{1001}$$

values, but answer is still 10 because:

$$\sqrt[10]{1024} \geq 1001$$

---

## 26. L0–L3 Practice

### L0 — Foundation

Convert:

$$\sqrt[10]{10101_2}$$

to decimal.

$$\sqrt[10]{16+4+1}$$

$$\boxed{21}$$

---

### L1 — Standard

Convert:

$$\sqrt[10]{11101011_2}$$

to hexadecimal.

Group:

$$\sqrt[10]{1110 \text{ quad } 1011}$$

$$\sqrt[10]{E \text{ quad } B}$$

Answer:

$$\boxed{EB_{16}}$$

---

### L1

Convert:

$$\sqrt[10]{0.375_{10}}$$

to binary.

$$\lfloor 0.375 \times 2 = 0.75 \rfloor \rightarrow 0$$

$$\lfloor 0.75 \times 2 = 1.5 \rfloor \rightarrow 1$$

$$\lfloor 0.5 \times 2 = 1.0 \rfloor \rightarrow 1$$

Answer:

$$\boxed{0.011_2}$$

---

## L2 — GATE Level

---

What is the minimum number of bits required to represent integers from 0 through 500?

Number of values:

$$\lfloor 501 \rfloor$$

Check powers:

$$\lfloor 2^8 = 256 \rfloor$$

$$\lfloor 2^9 = 512 \rfloor$$

Therefore:

$$\boxed{9}$$

---

## L2 — Unknown Base

---

If:

$$\lfloor (123)_r = 38_{10} \rfloor$$

find  $\lfloor (r) \rfloor$ .

$$\lfloor r^2 + 2r + 3 = 38 \rfloor$$

$$\lfloor r^2 + 2r - 35 = 0 \rfloor$$

$$\lfloor (r+7)(r-5) = 0 \rfloor$$

Valid:

$$\boxed{r=5}$$

---

## L3 — Trap

---

Which is valid?

A.

$$\lfloor (128)_8 \rfloor$$

B.

$$\lfloor (10102)_2 \rfloor$$

C.

$$\lfloor (19F)_{16} \rfloor$$

D.

---

$$\lfloor (987)_{10} \rfloor$$

Check digits.

A has 8 in base 8 !' invalid.

B has 2 in base 2 !' invalid.

C: 1,9,F are valid hexadecimal digits.

D: digits 9,8,7 are valid decimal digits.

Therefore:

$$\lfloor \boxed{C,D} \rfloor$$

This would be an **MSQ-style** question.

## 27. GATE Method vs Normal Method

Suppose:

$$\lfloor 111111111_2 \rfloor$$

Find decimal value.

### Normal Method

$$\lfloor 2^9 + 2^8 + \dots + 2^0 \rfloor$$

Add everything.

### GATE Method

Ten 1s:

$$\lfloor \boxed{2^{10}-1} \rfloor$$

$$\lfloor =1024-1 \rfloor$$

$$\lfloor \boxed{1023} \rfloor$$

Always recognize:

$$\lfloor \boxed{111\dots1_2 = 2^n - 1} \rfloor$$

## 28. Chapter 1 Trap Book

Add these immediately.

### Trap 1 — Illegal digits

Every digit must satisfy:

$$\lfloor \text{digit} < \text{base} \rfloor$$

### Trap 2 — Binary/Octal grouping confusion

Octal:

$$\lfloor \boxed{3 \text{ bits}} \rfloor$$

Hex:

$\boxed{4 \text{ bits}}$

### Trap 3 — Wrong fraction direction

Integer:

divide !' read upwards

Fraction:

multiply !' read downwards

### Trap 4 — Wrong zero padding

Integer side !' pad left.

Fraction side !' pad right.

### Trap 5 — Patterns vs maximum

$2^n$

patterns.

$2^{n-1}$

maximum unsigned value.

### Trap 6 — F means 15

Not 16.

$F_{16} = 15_{10}$

### Trap 7 — Decimal 0.1

Do not assume every decimal fraction terminates in binary.

---

## 29. Chapter 1 Formula Sheet

---

### Positional representation

$\boxed{N = \sum a_i r^i}$

### n-bit unsigned patterns

$\boxed{2^n}$

### n-bit unsigned range

$\boxed{0 \leq N \leq 2^n - 1}$

### Bits required for 0 through $N$

$\boxed{n = \lceil \log_2(N+1) \rceil}$

### Binary !"Octal

$\boxed{3 \text{ bits/digit}}$

### Binary !"Hexadecimal

$\boxed{4 \text{ bits/digit}}$

---

## 30. 30-Second Revision

---

Remember only this:

$\boxed{\text{Binary}=2, \text{Octal}=8, \text{Decimal}=10, \text{Hex}=16}$

$\boxed{1 \text{ octal digit} = 3 \text{ bits}}$

$\boxed{1 \text{ hex digit} = 4 \text{ bits}}$

Integer decimal conversion:

$\boxed{\text{divide and read UP}}$

Fraction conversion:

$\boxed{\text{multiply and read DOWN}}$

Unsigned n-bit:

$\boxed{0 \text{ to } 2^n - 1}$

Number of patterns:

$\boxed{2^n}$

---

## What You Absolutely Must Remember

---

1. Positional weights.
  2. Powers of 2.
  3. Binary !" octal grouping = 3.
  4. Binary !" hex grouping = 4.
  5. Fractional conversion.
  6.  $(2^n)$  patterns.
  7.  $(2^n - 1)$  maximum.
  8. Base must exceed every digit.
  9. Leading zeros may matter when bit-width is specified.
  10. Some decimal fractions repeat forever in binary.
- 

## Digital Logic Progress

---

**Completed:** Chapter 1 — Number System Fundamentals ' **Current:** Chapter 2 — Signed Number Representation  
**Remaining:** Signed Representation !' Binary Arithmetic !' Boolean Algebra !' SOP/POS !' Gate Realization !' Algebraic Minimization !' K-map !' Tabular Method !' Combinational Circuits !' Sequential Circuits !' Fixed Point !' Floating Point

### NEXT

#### Chapter 2 — Signed Number Representation

We will cover:

**Unsigned !' Sign Magnitude !' 1's Complement !' 2's Complement !' all ranges !' positive/negative zero !' negation !' sign extension !' zero extension !' truncation !' GATE traps.**

USER

next

ASSISTANT

## CHAPTER 2 — SIGNED NUMBER REPRESENTATION MUST MASTER

This chapter is extremely important because it connects directly to:

- binary arithmetic
- overflow
- ALU behavior
- fixed-point representation
- computer arithmetic
- many GATE numerical questions

The core problem is simple:

**Binary naturally represents positive values. How do we represent negative numbers?**

We need a convention.

There are four important representations:

1. Unsigned
2. Sign Magnitude
3. 1's Complement
4. 2's Complement

### 1. Unsigned Representation

If all  $(n)$  bits represent magnitude, the number is **unsigned**.

For 4 bits:

$[0000 = 0]$

$[0001 = 1]$

$[\dots]$

$[1111 = 15]$

General range:

$[0 \text{ to } 2^n - 1]$

For 8 bits:

$[0 \text{ to } 255]$

Unsigned representation cannot directly represent negative numbers.

## 2. Why Signed Representations Are Needed

---

Suppose we have:

$\lceil 0101 \rceil$

We can interpret this as:

$\lceil +5 \rceil$

But how should we store:

$\lceil -5? \rceil$

We could reserve one bit to indicate the sign.

That leads to **sign magnitude**.

---

## 3. Sign Magnitude Representation

---

In sign magnitude:

- MSB = sign bit
- remaining bits = magnitude

Convention:

$\lceil 0 \rightarrow + \rceil$

$\lceil 1 \rightarrow - \rceil$

For a 4-bit number:

$\lceil SMMM \rceil$

where:

- $\lceil S \rceil$  = sign
  - $\lceil MMM \rceil$  = magnitude
- 

### Example: +5

---

Magnitude of 5:

$\lceil 101 \rceil$

Sign positive:

$\lceil 0 \rceil$

So:

$\lceil \boxed{0101} \rceil$

---

### Example: -5

---

Magnitude:

$\lceil 101 \rceil$

Negative sign:

---

$\lfloor 1 \rfloor$

So:

$\lfloor \boxed{1101} \rfloor$

---

## 4. Sign Magnitude Range

---

For  $\lfloor n \rfloor$  bits:

1 bit is sign.

Therefore magnitude uses:

$\lfloor n-1 \rfloor$

bits.

Largest magnitude:

$\lfloor 2^{n-1}-1 \rfloor$

So range:

$\lfloor \boxed{-(2^{n-1}-1) \text{ to } +(2^{n-1}-1)} \rfloor$

Example for 4 bits:

$\lfloor -(2^3-1) \text{ to } +(2^3-1) \rfloor$

$\lfloor \boxed{-7 \text{ to } +7} \rfloor$

---

## 5. Two Zeros in Sign Magnitude

---

Positive zero:

$\lfloor 0000 \rfloor$

Negative zero:

$\lfloor 1000 \rfloor$

Both represent numerical zero.

Therefore one bit pattern is wasted.

That is one major disadvantage.

---

## 6. Sign Magnitude Example

---

Interpret:

$\lfloor 11010 \rfloor$

as a 5-bit sign-magnitude number.

Sign bit:

$\lfloor 1 \rfloor$

Therefore negative.

---



Magnitude:

$\lceil 1010_2 = 10 \rceil$

Hence:

$\lceil \boxed{-10} \rceil$

---

## 7. Limitations of Sign Magnitude

---

Main disadvantages:

1. Two representations of zero.
2. Arithmetic hardware becomes inconvenient.
3. Addition/subtraction need special sign handling.

This is why computers usually prefer **2's complement**.

---

## 8. 1's Complement Representation

---

To represent a negative number in 1's complement:

**Write the positive binary representation and invert every bit.**

That means:

$\lceil 0 \rightarrow 1 \rceil$

---

### Example: -5 in 4 bits

---

First:

$\lceil +5 = 0101 \rceil$

Invert every bit:

$\lceil 0101 \rightarrow 1010 \rceil$

Therefore:

$\lceil \boxed{-5 = 1010} \rceil$

in 1's complement.

---

## Memory Trick

---

**1's complement:**

$\lceil \boxed{\text{Flip every bit}} \rceil$

---

## 9. Positive Numbers in 1's Complement

---

Positive numbers are represented normally.

---

Example:

$\lceil +6=0110 \rceil$

Negative 6:

$\lceil 0110 \rightarrow 1001 \rceil$

Therefore:

$\lceil \boxed{-6=1001} \rceil$

---

## 10. Range of 1's Complement

---

Like sign magnitude, it has two zeros.

Therefore:

$\lceil \boxed{-(2^{n-1}-1) \text{ to } +(2^{n-1}-1)} \rceil$

For 4 bits:

$\lceil \boxed{-7 \text{ to } +7} \rceil$

---

## 11. Two Zeros in 1's Complement

---

Positive zero:

$\lceil 0000 \rceil$

Invert:

$\lceil 1111 \rceil$

which is negative zero.

So:

$\lceil \boxed{0000=+0} \rceil$

$\lceil \boxed{1111=-0} \rceil$

Again, two zero representations.

---

## 12. How to Decode a 1's Complement Number

---

Suppose:

$\lceil 10110 \rceil$

is a 5-bit 1's-complement number.

MSB = 1.

So it is negative.

Invert bits:

$\lceil 10110 \rightarrow 01001 \rceil$

$\lceil 01001=9 \rceil$

---

Therefore:

$\boxed{-9}$

---

## 13. 2's Complement Representation

---

This is the most important signed representation.

Almost all GATE questions involving signed arithmetic eventually depend on it.

To obtain the 2's complement of a binary number:

1. Flip every bit.
2. Add 1.

That is:

$\boxed{\text{2's complement} = \text{1's complement} + 1}$

---

### Example: -5 in 4 bits

---

Positive 5:

$0101$

Invert:

$1010$

Add 1:

$1010 + 1 = 1011$

Therefore:

$\boxed{-5 = 1011}$

---

## Memory Trick

---

**1's complement:**

Flip

**2's complement:**

Flip + 1

---

## 14. Another Example

---

Represent:

$-13$

using 8-bit 2's complement.

First:

$13 = 00001101$

---

Invert:

$\text{\texttt{11110010}}$

Add 1:

$\text{\texttt{11110011}}$

Hence:

$\boxed{-13=11110011}$

---

## 15. Range of 2's Complement

---

For  $(n)$  bits:

$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$

This is a must-memorize formula.

---

### Example: 4 bits

---

$-2^3 \text{ to } 2^3-1$

$\boxed{-8 \text{ to } +7}$

---

### Example: 8 bits

---

$-2^7 \text{ to } 2^7-1$

$\boxed{-128 \text{ to } +127}$

---

## 16. Why 2's Complement Has One Extra Negative Number

---

4-bit patterns:

$\text{\texttt{0000}} \text{ to } \text{\texttt{1111}}$

Positive values:

$\text{\texttt{0000}}=0$

to:

$\text{\texttt{0111}}=+7$

Negative values:

$\text{\texttt{1000}}=-8$

to:

$\text{\texttt{1111}}=-1$

So range:

$\boxed{-8 \text{ to } +7}$

There is only one zero:

---

$[0000]$

Therefore the bit pattern that was previously wasted as negative zero can now represent:

$[-8]$

That is why 2's complement has one extra negative value.

---

## 17. Special Value: $(1000\dots0)$

---

In an  $(n)$ -bit 2's-complement system:

$[1000\dots0]$

represents:

$[-2^{n-1}]$

Example:

8 bits:

$[10000000]$

represents:

$[-128]$

This number has no positive counterpart representable in the same bit width.

Because  $(+128)$  requires more than 8-bit signed 2's complement.

---

## 18. Decode a 2's Complement Number

---

Suppose:

$[11101011]$

is an 8-bit 2's-complement number.

MSB = 1.

So negative.

Take 2's complement to find its magnitude.

Invert:

$[00010100]$

Add 1:

$[00010101]$

$[=21]$

Therefore:

$[-21]$

---

## GATE Shortcut for Decoding 2's Complement

---

There is a faster method.

For:

$\lfloor 11101011 \rfloor$

Treat the MSB as having weight:

$\lfloor -2^7 \rfloor$

and the rest normally.

So:

$\lfloor -128+64+32+8+2+1 \rfloor$

$\lfloor -128+107 \rfloor$

$\lfloor \boxed{-21} \rfloor$

This is often faster in the exam.

General formula:

For bits:

$\lfloor b_{n-1}b_{n-2}\dots b_0 \rfloor$

value is:

$\lfloor \boxed{-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i} \rfloor$

## 19. Master Comparison Table

| Representation | n-bit Range                                                      | Zero Representations | How to form -x  |
|----------------|------------------------------------------------------------------|----------------------|-----------------|
| Unsigned       | $\lfloor 0 \rfloor$ to $\lfloor 2^{n-1} \rfloor$                 | 1                    | Not applicable  |
| Sign Magnitude | $\lfloor -(2^{n-1}-1) \rfloor$ to $\lfloor +(2^{n-1}-1) \rfloor$ | 2                    | Change sign bit |
| 1's Complement | $\lfloor -(2^{n-1}-1) \rfloor$ to $\lfloor +(2^{n-1}-1) \rfloor$ | 2                    | Flip all bits   |
| 2's Complement | $\lfloor -2^{n-1} \rfloor$ to $\lfloor 2^{n-1}-1 \rfloor$        | 1                    | Flip + 1        |

This table is very important.

## 20. Same Bit Pattern, Different Meaning

Consider:

$\lfloor 1110 \rfloor$

### Unsigned

$\lfloor 14 \rfloor$

### Sign Magnitude

Sign = 1.

Magnitude:

$\lceil 110 = 6 \rceil$

Therefore:

$\lceil -6 \rceil$

### 1's Complement

Invert:

$\lceil 1110 \rightarrow 0001 \rceil$

Therefore:

$\lceil -1 \rceil$

### 2's Complement

Invert:

$\lceil 0001 \rceil$

Add 1:

$\lceil 0010 \rceil$

Therefore:

$\lceil -2 \rceil$

So the same bits can represent very different numerical values.

This is a common GATE idea.

---

## 21. Important Insight

---

A bit pattern by itself has no inherent numerical meaning.

Example:

$\lceil 11111111 \rceil$

could mean:

Unsigned:

$\lceil 255 \rceil$

1's complement:

$\lceil -0 \rceil$

2's complement:

$\lceil -1 \rceil$

Therefore:

$\lceil \boxed{\text{Representation convention matters}} \rceil$

---

## 22. Quick 2's Complement Trick

---

Instead of "invert + 1", another method is:

Starting from the right:

1. Copy bits until the first 1, including that 1.
2. Complement everything to its left.

Example:

$\backslash$  00101100  $\backslash$

Find 2's complement.

From right, copy:

$\backslash$  100  $\backslash$

Then flip everything left of it:

$\backslash$  00101  $\rightarrow$  11010  $\backslash$

Result:

$\backslash$   $\boxed{11010100}$   $\backslash$

Check using normal method:

$\backslash$  00101100  $\rightarrow$  11010011 + 1 = 11010100  $\backslash$

Correct.

---

## 23. Negation in 2's Complement

A beautiful property:

**Take the 2's complement of a number to get its negative.**

Example:

$\backslash$  00000101 = +5  $\backslash$

2's complement:

$\backslash$  11111011 = -5  $\backslash$

Take 2's complement again:

$\backslash$  00000101 = +5  $\backslash$

So:

$\backslash$   $\boxed{-(-x)=x}$   $\backslash$

---

## 24. Special Exception: Most Negative Number

Consider 8-bit:

$\backslash$  10000000 = -128  $\backslash$

Take 2's complement:

Invert:

$\backslash$  01111111  $\backslash$



Add 1:

$\lfloor 10000000 \rfloor$

You get the same bit pattern.

Why?

Because:

$\lfloor +128 \rfloor$

cannot be represented in signed 8-bit 2's complement.

This is an important overflow edge case.

---

## 25. Sign Extension

Suppose we have a signed 4-bit 2's-complement number and want to convert it into 8 bits.

We must preserve its value.

For signed 2's complement:

$\lfloor \boxed{\text{Repeat the sign bit on the left}} \rfloor$

This is called **sign extension**.

---

### Positive Example

4-bit:

$\lfloor 0101=+5 \rfloor$

Extend to 8 bits:

$\lfloor \boxed{00000101} \rfloor$

---

### Negative Example

4-bit:

$\lfloor 1011=-5 \rfloor$

Extend to 8 bits by copying sign bit 1:

$\lfloor \boxed{11111011} \rfloor$

Still:

$\lfloor -5 \rfloor$

---

### Why Does Sign Extension Work?

Original:

$\lfloor 1011 \rfloor$

Using signed weights:

---

$$\lfloor -8+2+1=-5 \rfloor$$

Extended:

$$\lfloor 11111011 \rfloor$$

$$\lfloor -128+64+32+16+8+2+1 \rfloor$$

$$\lfloor =-5 \rfloor$$

So value is unchanged.

---

## 26. GATE Trap — Zero Extension vs Sign Extension

---

Suppose:

$$\lfloor 1011 \rfloor$$

is 4-bit signed 2's complement.

It means:

$$\lfloor -5 \rfloor$$

If you zero extend:

$$\lfloor 00001011 \rfloor$$

this becomes:

$$\lfloor +11 \rfloor$$

Wrong.

For signed 2's complement:

$$\lfloor \boxed{\text{Use sign extension}} \rfloor$$

---

## 27. Zero Extension

---

For **unsigned numbers**, extend by adding zeros to the left.

Example:

$$\lfloor 1011_2=11 \rfloor$$

Extend from 4 to 8 bits:

$$\lfloor 00001011 \rfloor$$

Still:

$$\lfloor 11 \rfloor$$

Therefore:

$$\lfloor \boxed{\text{Unsigned !' zero extension}} \rfloor$$

---

## 28. Sign Extension in 1's Complement

---

The same general rule applies:

---

repeat the sign bit.

Example:

4-bit 1's complement:

$\lfloor 1010 \rceil = -5$

Extend:

$\lfloor 11111010 \rceil$

This remains  $\lfloor -5 \rceil$ .

---

## 29. Sign Extension in Sign Magnitude

---

Be careful.

Sign magnitude representation has:

- one sign bit
- magnitude bits

To increase width, keep the sign bit and extend the magnitude with zeros.

Example:

4-bit:

$\lfloor 1101 \rceil$

means:

$\lfloor -5 \rceil$

Extend to 8-bit sign magnitude:

$\lfloor \boxed{10000101} \rceil$

Not:

$\lfloor 11111101 \rceil$

This is another reason why blindly copying MSB is associated primarily with complement representations.

---

## 30. Truncation

---

Truncation means reducing the number of bits.

Example:

8-bit unsigned:

$\lfloor 00001101 \rceil$

Reduce to 4 bits:

$\lfloor 1101 \rceil$

Still decimal:

$\lfloor 13 \rceil$

No problem.

---

But truncation can change the value if important high bits are removed.

---

## Example

---

8-bit unsigned:

\[ 00101101 \]

Decimal:

\[ 45 \]

Keep only 4 bits:

\[ 1101 \]

Decimal:

\[ 13 \]

Value changed.

This is truncation overflow/data loss.

---

## 31. Safe Truncation for Positive Signed Numbers

---

Suppose:

\[ 00000101 \]

8-bit 2's complement.

Truncate to 4 bits:

\[ 0101 \]

Still:

\[ +5 \]

Safe because discarded bits were redundant sign bits.

---

## 32. Safe Truncation for Negative Signed Numbers

---

Example:

\[ 11111011=-5 \]

Reduce to 4 bits:

\[ 1011 \]

Still:

\[ -5 \]

Again, safe.

Rule:

---

**Truncation is safe only if the remaining bits still represent the same value.**

A practical way:

After truncation, sign-extend it back.

If you recover the original pattern, the truncation was safe.

## Example

Original:

$\lfloor 11111011 \rfloor$

Truncate:

$\lfloor 1011 \rfloor$

Sign extend:

$\lfloor 11111011 \rfloor$

Matches original.

Therefore safe.

## 33. Minimum Bits Required for Signed 2's Complement

For  $\lfloor n \rfloor$  bits:

$\lfloor -2^{n-1} \leq x \leq 2^{n-1}-1 \rfloor$

Example: minimum bits to represent:

$\lfloor +100 \rfloor$

Need:

$\lfloor 2^{n-1}-1 \geq 100 \rfloor$

Try  $\lfloor n=7 \rfloor$ :

$\lfloor 2^6-1=63 \rfloor$

not enough.

Try  $\lfloor n=8 \rfloor$ :

$\lfloor 2^7-1=127 \rfloor$

enough.

Therefore:

$\lfloor \boxed{8 \text{ bits}} \rfloor$

## Example: Represent -100

Need:

$$\lceil -2^{n-1} \rceil \leq -100$$

Thus:

$$\lceil 2^{n-1} \rceil \geq 100$$

Again:

$$\lceil n=8 \rceil$$

So:

$$\boxed{8 \text{ bits}}$$

## 34. Interesting Asymmetry

What is the minimum number of signed 2's-complement bits required for:

$$\lceil -128 \rceil$$

8 bits:

$$\lceil -128 \text{ to } 127 \rceil$$

So:

$$\boxed{8}$$

What about:

$$\lceil +128 \rceil$$

8-bit maximum:

$$\lceil 127 \rceil$$

So need 9 bits.

$$\boxed{9}$$

This is a classic consequence of asymmetric range.

## 35. Sign Bit Is Not Always Simply "-"

This is an important conceptual point.

In sign magnitude:

MSB directly means sign.

In 2's complement:

MSB is related to sign, but the whole bit pattern contributes to the numerical value.

Example:

$$\lceil 1011 \rceil$$

You should not say:

"sign bit 1, magnitude 011 = 3, so -3."

That would be sign-magnitude interpretation.

For 2's complement:

$\lceil 1011 = -5 \rceil$

So never mix conventions.

---

## 36. GATE-Level Pattern: Bit Pattern Interpretation

---

Suppose an 8-bit register contains:

$\lceil 11110110 \rceil$

What is its decimal value if interpreted as:

**Unsigned?**

$\lceil 128+64+32+16+4+2=246 \rceil$

$\lceil \boxed{246} \rceil$

**2's complement?**

Invert:

$\lceil 00001001 \rceil$

Add 1:

$\lceil 00001010 = 10 \rceil$

Therefore:

$\lceil \boxed{-10} \rceil$

Same storage, different interpretation.

---

## 37. Range Comparison Example

---

For 6 bits:

**Unsigned**

$\lceil 0 \text{ to } 2^6-1 \rceil$

$\lceil \boxed{0 \text{ to } 63} \rceil$

**Sign Magnitude**

$\lceil -(2^5-1) \text{ to } +(2^5-1) \rceil$

$\lceil \boxed{-31 \text{ to } 31} \rceil$

**1's Complement**

$\lceil \boxed{-31 \text{ to } 31} \rceil$

**2's Complement**

$\lceil -2^5 \text{ to } 2^5-1 \rceil$

$\lceil \boxed{-32 \text{ to } 31} \rceil$

---

## 38. Quick Recognition Rules

---

For an  $(n)$ -bit 2's-complement number:

If MSB = 0:

$\boxed{\text{non-negative}}$

Read normally.

If MSB = 1:

$\boxed{\text{negative}}$

Use:

- invert + 1, or
  - negative-MSB-weight shortcut.
- 

## 39. Important 2's Complement Identities

---

For  $(n)$ -bit arithmetic, if unsigned bit pattern value is  $(U)$ :

If MSB = 1, signed value can be obtained by:

$\boxed{U - 2^n}$

This is a fantastic GATE shortcut.

Example:

$\boxed{11110110_2}$

Unsigned value:

$\boxed{246}$

Since MSB = 1 and width = 8:

$\boxed{246 - 256 = -10}$

Therefore:

$\boxed{-10}$

Often faster than complementing bits.

---

## 40. Why the $(U - 2^n)$ Shortcut Works

---

An  $n$ -bit 2's-complement negative pattern corresponds to a large unsigned value.

For example:

$\boxed{11111111}$

unsigned:

$\boxed{255}$

signed:

$\boxed{255 - 256 = -1}$

Similarly:



$\lfloor 10000000 \rfloor$

unsigned:

$\lfloor 128 \rfloor$

signed:

$\lfloor 128-256=-128 \rfloor$

Hence:

$\lfloor \boxed{\text{Signed value}=U-2^n} \rfloor$

for MSB 1.

---

## 41. L0 Practice

Represent  $\lfloor +9 \rfloor$  in 8-bit sign magnitude.

Magnitude:

$\lfloor 0001001 \rfloor$

Add sign 0:

$\lfloor \boxed{00001001} \rfloor$

---

## 42. L0 Practice

Represent  $\lfloor -9 \rfloor$  in 8-bit 1's complement.

$\lfloor +9=00001001 \rfloor$

Flip:

$\lfloor \boxed{11110110} \rfloor$

---

## 43. L0 Practice

Represent  $\lfloor -9 \rfloor$  in 8-bit 2's complement.

$\lfloor 00001001 \rfloor$

Flip:

$\lfloor 11110110 \rfloor$

Add 1:

$\lfloor \boxed{11110111} \rfloor$

---

## 44. L1 Practice

Interpret:

$\lfloor 11010110 \rfloor$

as 8-bit 2's complement.

---

Method A:

Invert:

$\lceil 00101001 \rceil$

Add 1:

$\lceil 00101010 \rceil$

$\lceil =42 \rceil$

Therefore:

$\lceil \boxed{-42} \rceil$

---

## GATE Method

---

Unsigned value:

$\lceil 214 \rceil$

Then:

$\lceil 214-256 \rceil$

$\lceil \boxed{-42} \rceil$

---

## 45. L1 Practice

---

What is the range of a 12-bit 2's-complement number?

$\lceil -2^{11} \text{ to } 2^{11}-1 \rceil$

$\lceil \boxed{-2048 \text{ to } 2047} \rceil$

---

## 46. L2 — GATE-Style Question

---

An 8-bit pattern is:

$\lceil 11111100 \rceil$

Its value under different interpretations is?

### Unsigned

$\lceil 252 \rceil$

### Sign magnitude

Sign = 1.

Magnitude:

$\lceil 1111100_2=124 \rceil$

So:

$\lceil -124 \rceil$

---

### 1's complement

Invert:

$\lfloor 00000011 \rfloor$

So:

$\lfloor -3 \rfloor$

### 2's complement

Unsigned shortcut:

$\lfloor 252-256=-4 \rfloor$

So:

$\lfloor \boxed{-4} \rfloor$

---

## 47. L2 — Bits Required

Minimum number of 2's-complement bits needed to represent:

$\lfloor -1000 \rfloor$

Need:

$\lfloor 2^{n-1} \geq 1000 \rfloor$

We know:

$\lfloor 2^9=512 \rfloor$

$\lfloor 2^{10}=1024 \rfloor$

Therefore:

$\lfloor n-1=10 \rfloor$

$\lfloor \boxed{n=11} \rfloor$

---

## 48. L3 — Trap Question

Which values can be represented in 8-bit signed 2's complement?

A.  $\lfloor (-128) \rfloor$

B.  $\lfloor (+127) \rfloor$

C.  $\lfloor (+128) \rfloor$

D.  $\lfloor (-129) \rfloor$

Range:

$\lfloor -128 \text{ to } 127 \rfloor$

Therefore:

$\lfloor \boxed{A,B} \rfloor$

---

## 49. L3 — Sign Extension Trap

---

4-bit 2's-complement number:

$\boxed{1010}$

What is its 8-bit equivalent?

Because MSB is 1, sign extend:

$\boxed{11111010}$

What value?

4-bit:

$\boxed{1010}$

Unsigned = 10.

Signed:

$\boxed{10-16=-6}$

8-bit:

$\boxed{250-256=-6}$

Correct.

---

## 50. GATE PYQ Patterns

---

Historically, GATE Digital Logic/COA-style questions frequently test these ideas indirectly:

- converting between signed representations
- range of  $(n)$ -bit signed numbers
- interpreting a fixed bit pattern
- overflow after arithmetic
- sign extension
- 2's-complement subtraction
- identifying whether a result fits in a given word size

The highest-value preparation is not memorizing isolated conversions; it is being able to identify the **representation and bit width first**.

We will solve arithmetic-heavy PYQ patterns in the next chapter because that is where representation and overflow combine.

---

## 51. GATE Trap Book — Chapter 2

---

Add these.

### Trap 1 — MSB = sign does not mean remaining bits are magnitude

That interpretation works only for sign magnitude.

For 2's complement:

$\boxed{1011 \neq -3}$

---

It is:

$\boxed{-5}$

---

### Trap 2 — Forgetting bit width

2's complement depends on the number of bits.

Example:

$\boxed{1111}$

4-bit:

$\boxed{-1}$

but if interpreted as unsigned:

$\boxed{15}$

---

### Trap 3 — Wrong range

2's complement:

$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$

not symmetric.

---

### Trap 4 — Two zeros

Sign magnitude and 1's complement have:

$\boxed{+0, -0}$

2's complement has only one zero.

---

### Trap 5 — Zero extending a negative 2's-complement number

Wrong.

Use:

$\boxed{\text{sign extension}}$

---

### Trap 6 — Most negative number

$\boxed{1000\dots0}$

represents:

$\boxed{-2^{n-1}}$

---

### Trap 7 — Taking magnitude incorrectly

For negative 2's complement:

$\boxed{\text{invert} + 1}$

not merely invert.

---

## 52. Chapter 2 Formula Sheet

---

### Unsigned

---

$$\boxed{0 \leq N \leq 2^n - 1}$$

### Sign Magnitude

---

$$\boxed{-(2^{n-1}-1) \leq N \leq 2^{n-1}-1}$$

Two zeros.

### 1's Complement

---

$$\boxed{-(2^{n-1}-1) \leq N \leq 2^{n-1}-1}$$

Two zeros.

### 2's Complement

---

$$\boxed{-2^n \leq N \leq 2^n - 1}$$

One zero.

### Form negative 1's complement

---

$$\boxed{\text{flip}}$$

### Form negative 2's complement

---

$$\boxed{\text{flip} + 1}$$

### Negative 2's-complement shortcut

---

If unsigned interpretation is  $U$ :

$$\boxed{\text{signed} = U - 2^n}$$

when MSB = 1.

## 53. Master Range Memory Trick

---

For  $(n)$  bits:

### Unsigned

$$\boxed{0 \rightarrow 2^n - 1}$$

### Sign magnitude / 1's complement

"Equal positive and negative range":

$$\boxed{\pm(2^{n-1}-1)}$$

### 2's complement

"One extra negative":

$$\boxed{-2^{n-1} \rightarrow 2^{n-1}-1}$$


---

## 54. 30-Second Revision

---

Remember:

### Sign magnitude

$$\boxed{\text{sign + magnitude}}$$

### 1's complement

$$\boxed{\text{flip bits}}$$

### 2's complement

$$\boxed{\text{flip bits} + 1}$$

8-bit 2's-complement range:

$$\boxed{-128 \text{ to } 127}$$

For negative-width extension:

$$\boxed{\text{repeat the sign bit}}$$

And for quick decoding:

$$\boxed{U-2^n}$$

when MSB = 1.

---

## What You Absolutely Must Remember

---

1. The representation must be known before interpreting bits.
2. 2's complement is the most important representation for GATE.
3.  $n$ -bit 2's complement range:  

$$\boxed{-2^{n-1} \text{ to } 2^{n-1}-1}$$
4. Sign magnitude and 1's complement have two zeros.
5. 2's complement has one zero.
6. Negative 2's complement = invert + 1.
7. Sign extension repeats the sign bit.
8. Unsigned widening uses zero extension.
9.  $(100\dots0)$  is the most negative value.
10.  $(111\dots1)$  in 2's complement is always:

$$\boxed{-1}$$


---

## Digital Logic Progress

---

**Completed:** Chapter 1 Number Systems ' ' Chapter 2 Signed Number Representation ' **Current:** Chapter 3 — Binary Arithmetic **Remaining:** Binary Arithmetic ' Boolean Algebra ' Boolean Functions ' Gate Realization ' Algebraic Minimization ' K-map ' Tabular Method ' Combinational Circuits ' Sequential Circuits ' Fixed Point ' Floating Point

NEXT

## Chapter 3 — Binary Arithmetic Ø=Ý4

Next we cover:

**Binary addition !' subtraction !' borrow !' complement subtraction !' signed arithmetic !' carry vs overflow !' overflow detection !' 2's-complement arithmetic !' shifts !' multiplication !' division basics !' GATE numerical traps.**

USER

Next

ASSISTANT

## CHAPTER 3 — BINARY ARITHMETIC Ø=Ý4 MUST MASTER

This chapter connects directly to:

- 2's complement
- overflow
- adders/subtractors
- ALU
- fixed-point arithmetic
- computer arithmetic questions in GATE

The most important exam distinction is:

$\boxed{\text{Carry-out and signed overflow are NOT the same thing}}$

### 1. Binary Addition Basics

Binary addition follows these rules:

| A | B | Sum | Carry |
|---|---|-----|-------|
| 0 | 0 | 0   | 0     |
| 0 | 1 | 1   | 0     |
| 1 | 0 | 1   | 0     |
| 1 | 1 | 0   | 1     |

So:

$1+1=10_2$

because:

$1+1=2_{10}$

and:



$$\lfloor 2_{10}=10_2 \rfloor$$

---

## 2. Addition With Carry-In

---

If three 1s are added:

$$\lfloor 1+1+1=11_2 \rfloor$$

Therefore:

- Sum bit = 1
- Carry = 1

This is exactly the idea behind a **full adder**, which we study later.

---

## 3. Binary Addition Example

---

Add:

$$\lfloor 1011_2+0110_2 \rfloor$$

Rightmost column:

$$\lfloor 1+0=1 \rfloor$$

Next:

$$\lfloor 1+1=10 \rfloor$$

Write 0 and carry 1.

Next:

$$\lfloor 0+1+1=10 \rfloor$$

Write 0 and carry 1.

Next:

$$\lfloor 1+0+1=10 \rfloor$$

Final result:

$$\lfloor \boxed{10001_2} \rfloor$$

Check:

$$\lfloor 1011_2=11 \rfloor$$

$$\lfloor 0110_2=6 \rfloor$$

$$\lfloor 11+6=17 \rfloor$$

$$\lfloor 17=10001_2 \rfloor$$

Correct.

---

## 4. n-Bit Addition

---

Suppose two 4-bit numbers are added.

The result may require:

---

$5 \text{ bits}$

Example:

$1111 + 0001 = 10000$

However, if the hardware register stores only 4 bits, the leftmost carry may be discarded.

Stored result:

$0000$

Carry-out:

$1$

This distinction becomes important for unsigned and signed arithmetic.

---

## 5. Unsigned Addition Overflow

For unsigned  $n$ -bit numbers:

$0 \leq N \leq 2^n - 1$

If the true result exceeds:

$2^n - 1$

unsigned overflow occurs.

For unsigned addition:

$\boxed{\text{Carry out of MSB usually indicates overflow}}$

Example with 4 bits:

$1111 + 0001$

$15 + 1 = 16$

But 4-bit unsigned maximum:

$15$

Therefore overflow.

---

## 6. Binary Subtraction Basics

Rules:

$0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

Problem case:

$0 - 1$

We need to borrow from the next higher bit.

In binary:

$10_2 - 1_2 = 1_2$

because  $(10_{10} = 2_{10})$ .

---

## 7. Binary Subtraction Example

---

Subtract:

$$10110_2 - 00101_2$$

Decimal check:

$$22 - 5 = 17$$

Therefore answer should be:

$$10001_2$$

Direct subtraction gives:

$$\boxed{10001_2}$$

## 8. Why Computers Prefer Complement Subtraction

---

Direct borrowing is inconvenient in digital hardware.

Instead, computers transform subtraction into addition:

$$A - B = A + (-B)$$

For 2's complement:

$$\boxed{A - B = A + \text{2's complement of } B}$$

This means the same adder hardware can perform both addition and subtraction.

Very important later in ALU design.

---

## 9. Subtraction Using 2's Complement

---

Compute:

$$7 - 3$$

using 4 bits.

Represent:

$$7 = 0111$$

$$3 = 0011$$

2's complement of  $(3)$ :

Invert:

$$1100$$

Add 1:

$$1101$$

Now:

---

$$\text{\texttt{0111+1101}}$$

$$\text{\texttt{=1\,0100}}$$

For 4-bit arithmetic, discard carry-out:

$$\text{\texttt{\boxed{0100}}}$$

which is:

$$\text{\texttt{\boxed{4}}}$$

---

## 10. Why Can We Discard Carry?

In  $(n)$ -bit 2's-complement arithmetic, calculations effectively occur modulo:

$$2^n$$

For 4 bits:

$$2^4=16$$

So:

$$7+(-3)$$

is implemented using:

$$7+13=20$$

Modulo 16:

$$20-16=4$$

Hence the carry is discarded.

---

## 11. Negative Result Example

Compute:

$$3-7$$

using 4-bit 2's complement.

$$3=0011$$

$$7=0111$$

2's complement of 7:

$$0111 \rightarrow 1000 \rightarrow 1001$$

Add:

$$0011+1001=1100$$

MSB is 1, so negative.

Find magnitude:

$$1100 \rightarrow 0011 \rightarrow 0100$$

Therefore:

$$\text{\texttt{\boxed{-4}}}$$

---

## 12. Addition in 2's Complement

For signed 2's-complement numbers:

**Add the bit patterns exactly like ordinary binary numbers.**

You do not separately manage signs.

This is one of the biggest advantages of 2's complement.

---

## 13. Example: Positive + Negative

Compute:

$$\lfloor 5+(-3) \rfloor$$

4-bit representation:

$$\lfloor 5=0101 \rfloor$$

$$\lfloor -3=1101 \rfloor$$

Add:

$$\lfloor 0101+1101 \rfloor$$

$$\lfloor =1\,0010 \rfloor$$

Discard carry:

$$\lfloor 0010 \rfloor$$

Therefore:

$$\lfloor \boxed{2} \rfloor$$

---

## 14. Example: Negative + Negative

Compute:

$$\lfloor -3+(-2) \rfloor$$

4 bits:

$$\lfloor -3=1101 \rfloor$$

$$\lfloor -2=1110 \rfloor$$

Add:

$$\lfloor 1101+1110 \rfloor$$

$$\lfloor =1\,1011 \rfloor$$

Discard carry:

$$\lfloor 1011 \rfloor$$

Interpret 1011:

$\lfloor 16? \rfloor$

No—4-bit signed.

Unsigned value is 11.

Signed value:

$\lfloor 11-16=-5 \rfloor$

Therefore:

$\lfloor \boxed{-5} \rfloor$

---

## 15. What Is Overflow?

Overflow means:

**The mathematically correct result cannot be represented using the available number of bits.**

Suppose we use 4-bit signed 2's complement.

Range:

$\lfloor -8\text{ to }+7 \rfloor$

Now compute:

$\lfloor 5+4=9 \rfloor$

But:

$\lfloor 9>7 \rfloor$

So the true result cannot fit.

Therefore:

$\lfloor \boxed{\text{overflow}} \rfloor$

---

## 16. Signed Overflow Rule

In 2's-complement addition, overflow occurs only when:

### Case 1

Two positive numbers produce a negative result.

or

### Case 2

Two negative numbers produce a positive result.

Therefore:

$\lfloor \boxed{\text{Same-sign inputs} + \text{opposite-sign result} \rightarrow \text{overflow}} \rfloor$

---

## 17. Example: Positive Overflow

4-bit:

$$\lfloor 5=0101 \rfloor$$

$$\lfloor 4=0100 \rfloor$$

Add:

$$\lfloor 0101+0100=1001 \rfloor$$

Bit pattern 1001 is negative in 4-bit 2's complement.

But:

$$\lfloor 5+4=9 \rfloor$$

which cannot fit in range:

$$\lfloor -8\text{ to }7 \rfloor$$

Therefore:

$$\lfloor \boxed{\text{overflow}} \rfloor$$

---

## 18. Example: Negative Overflow

---

Compute:

$$\lfloor -5+(-4) \rfloor$$

4 bits:

$$\lfloor -5=1011 \rfloor$$

$$\lfloor -4=1100 \rfloor$$

Add:

$$\lfloor 1011+1100=1\,0111 \rfloor$$

Stored result:

$$\lfloor 0111=+7 \rfloor$$

But true result:

$$\lfloor -9 \rfloor$$

which is outside range.

Therefore:

$$\lfloor \boxed{\text{overflow}} \rfloor$$

---

## 19. Opposite Signs Cannot Overflow in Addition

---

Example:

$$\lfloor +6+(-3) \rfloor$$

True answer lies between the operands:

$$\lfloor 3 \rfloor$$

If one number is positive and one negative, the magnitude tends to reduce rather than increase beyond the

representable extremes.

Therefore:

$\boxed{\text{Adding operands of opposite signs cannot cause signed overflow}}$

---

## 20. Carry vs Overflow Ø=Ý4

---

This is one of the most important Digital Logic traps.

Consider:

$1111+0001=1\,0000$

If interpreted as unsigned:

$15+1=16$

4-bit unsigned range:

$0 \text{ to } 15$

So:

$\boxed{\text{unsigned overflow}}$

If interpreted as signed 2's complement:

$1111=-1$

$0001=+1$

Then:

$-1+1=0$

stored result:

$0000$

Correct.

So:

$\boxed{\text{carry occurred, but signed overflow did NOT}}$

---

## 21. Example: Overflow Without Final Carry

---

4-bit signed:

$0111+0001$

$7+1=8$

Add:

$0111+0001=1000$

No carry out.

But:

$1000=-8$

True result should be  $(+8)$ , which is unrepresentable.

---



Therefore:

$\boxed{\text{signed overflow occurs even without carry-out}}$

Hence:

$\boxed{\text{Carry-out} \neq \text{signed overflow}}$

---

## 22. Hardware Overflow Detection

---

Let:

- $C_{n-1}$  = carry into the sign bit
- $C_n$  = carry out of the sign bit

For 2's-complement addition:

$\boxed{V = C_{n-1} \oplus C_n}$

where  $V$  is overflow flag.

If these carries differ:

$\boxed{\text{overflow}}$

---

## 23. Sign-Based Overflow Formula

---

Let:

- $A_s$  = sign bit of A
- $B_s$  = sign bit of B
- $S_s$  = sign bit of result

Overflow for addition:

$\boxed{V = \overline{A_s \oplus B_s} \cdot (A_s \oplus S_s)}$

Meaning:

- operands have same sign
- result has different sign

You usually do not need to memorize the full Boolean formula if you remember the sign rule.

---

## 24. Signed Subtraction Overflow

---

For:

$A - B$

overflow is possible when A and B have **different signs**, and the result has a sign different from A.

Shortcut:

$\boxed{\text{Different-sign operands in subtraction} + \text{wrong result sign} \rightarrow \text{overflow}}$

---

## 25. Example: Subtraction Overflow

---

4-bit signed:

$$\lfloor 7 - (-2) \rfloor$$

True result:

$$\lfloor 9 \rfloor$$

But maximum is:

$$\lfloor 7 \rfloor$$

Represent:

$$\lfloor 7 = 0111 \rfloor$$

$$\lfloor -2 = 1110 \rfloor$$

Subtracting  $\lfloor -2 \rfloor$  is adding  $\lfloor +2 \rfloor$ :

$$\lfloor 0111 + 0010 = 1001 \rfloor$$

1001 is negative.

So overflow occurred.

---

## 26. Another Subtraction Overflow

---

$$\lfloor -8 - 1 = -9 \rfloor$$

4-bit range:

$$\lfloor -8 \text{ to } 7 \rfloor$$

Cannot represent  $\lfloor -9 \rfloor$ .

So overflow.

---

## 27. 1's-Complement Arithmetic

---

Less important than 2's complement, but still useful.

For 1's-complement addition:

1. Add normally.
2. If carry-out occurs, add it back to LSB.

This is called:

$$\lfloor \boxed{\text{end-around carry}} \rfloor$$

---

## 28. Example: 1's-Complement Addition

---

Using 4-bit 1's complement:

$$\lfloor +5 = 0101 \rfloor$$

$$\lfloor -3: 0011 \rightarrow 1100 \rfloor$$

Add:

$$\lfloor 0101 + 1100 = 1\,0001 \rfloor$$

---

There is an end carry 1.

Add it back:

$$\boxed{0001+1=0010}$$

Therefore:

$$\boxed{2}$$

---

## 29. End-Around Carry Memory Trick

---

### 1's complement

$$\boxed{\text{Add carry back}}$$

### 2's complement

$$\boxed{\text{Discard final carry}}$$

This is a high-value distinction.

---

## 30. Sign-Magnitude Arithmetic

---

In sign magnitude, addition is more complicated.

If signs are same:

- add magnitudes
- retain sign

If signs differ:

- subtract smaller magnitude from larger magnitude
- keep sign of larger magnitude

This extra logic is one reason 2's complement is preferred.

For GATE, understand the principle; do not over-invest in manual sign-magnitude arithmetic unless a question explicitly asks.

---

## 31. Modular Interpretation of n-Bit Arithmetic

---

For unsigned  $n$ -bit arithmetic:

$$\boxed{\text{result stored modulo } 2^n}$$

Example:

8-bit:

$$\boxed{255+3=258}$$

Stored:

$$\boxed{258 \bmod 256 = 2}$$

So bit pattern becomes:

$$\boxed{00000010}$$

with a carry-out.

This modular view makes wraparound behavior easy to understand.

---

## 32. Binary Multiplication

---

Binary multiplication is simpler than decimal because each multiplier digit is only 0 or 1.

Rules:

$$[ 0 \times 0 = 0 ]$$

$$[ 0 \times 1 = 0 ]$$

$$[ 1 \times 0 = 0 ]$$

$$[ 1 \times 1 = 1 ]$$

---

## 33. Multiplication Example

---

Multiply:

$$[ 101_2 \times 11_2 ]$$

$$[ 101 \times 1 = 101 ]$$

Second multiplier bit is also 1, shifted left:

$$[ 1010 ]$$

Add:

$$[ 0101 ]$$

$$[ 1010 ]$$

$$[ =1111 ]$$

Therefore:

$$[ \boxed{101_2 \times 11_2 = 1111_2} ]$$

Check:

$$[ 5 \times 3 = 15 ]$$

Correct.

---

## 34. Width of Product

---

An  $(n)$ -bit unsigned number multiplied by an  $(m)$ -bit unsigned number can require up to:

$$[ \boxed{n+m \text{ bits}} ]$$

Example:

4-bit  $\times$  4-bit can need:

$$[ \boxed{8 \text{ bits}} ]$$

Why?

---

Maximum:

$$\lfloor (2^4-1)^2=225 \rfloor$$

which requires 8 bits.

---

## 35. Left Shift

---

Logical left shift:

$$\lfloor 00101101 \rfloor$$

shift left once:

$$\lfloor 01011010 \rfloor$$

A zero enters at the right.

For an unsigned value, if no significant bit is lost:

$$\lfloor \boxed{\text{left shift by } k} \rightarrow \times 2^k \rfloor$$

Example:

$$\lfloor 00101_2=5 \rfloor$$

Shift left once:

$$\lfloor 01010_2=10 \rfloor$$

So:

$$\lfloor 5 \times 2=10 \rfloor$$

---

## 36. Right Shift

---

Logical right shift:

$$\lfloor 10110000 \rfloor$$

becomes:

$$\lfloor 01011000 \rfloor$$

Zero enters at the left.

For unsigned integers:

$$\lfloor \boxed{\text{right shift by } k} \rightarrow \left\lfloor \frac{x}{2^k} \right\rfloor \rfloor$$

---

## 37. Logical Shift vs Arithmetic Shift $\emptyset = \text{Y}4$

---

### Logical shift

---

Vacated positions are filled with:

$$\lfloor 0 \rfloor$$

Used mainly for:

$$\lfloor \boxed{\text{unsigned data}} \rfloor$$

---

---

## Arithmetic right shift

---

For signed 2's-complement numbers:

**replicate the sign bit.**

Example:

$\lfloor 10110000 \rfloor$

Arithmetic right shift:

$\lfloor \boxed{11011000} \rfloor$

The sign remains negative.

---

## 38. Why Arithmetic Right Shift Copies Sign Bit

---

Suppose:

$\lfloor 11111000 \rfloor$

8-bit 2's complement:

Unsigned = 248.

Signed:

$\lfloor 248-256=-8 \rfloor$

Arithmetic right shift:

$\lfloor 11111100 \rfloor$

This represents:

$\lfloor -4 \rfloor$

So:

$\lfloor \boxed{-8/2=-4} \rfloor$

If we had inserted 0 instead:

$\lfloor 01111100 \rfloor$

that would be positive 124, completely wrong.

---

## 39. Arithmetic Left Shift

---

Arithmetic left shift generally uses the same bit movement as logical left shift:

- shift left
- insert 0 at LSB

Potential issue:

$\lfloor \boxed{\text{overflow}} \rfloor$

because the sign may change.

---

Example:

4-bit signed:

$[0101=5]$

left shift:

$[1010]$

This pattern is:

$[-6]$

The intended mathematical result  $(10)$  is outside 4-bit signed range.

So overflow.

---

## 40. Left Shift Overflow Intuition

---

For signed 2's complement:

If significant high-order bits are lost or the sign changes incorrectly, multiplication by 2 cannot be represented.

Therefore never blindly say:

$[x \ll 1 = 2x]$

without considering bit width and overflow.

---

## 41. Right Shift of Negative Odd Numbers

---

This is a subtle point.

Example:

4-bit:

$[1101=-3]$

Arithmetic right shift:

$[1110=-2]$

Mathematically:

$[-3/2=-1.5]$

Integer interpretation gives:

$[-2]$

So arithmetic right shift of negative numbers generally corresponds to rounding toward negative infinity in ordinary two's-complement shift behavior, not necessarily truncation toward zero.

This can matter in low-level questions.

---

## 42. Multiplication Using Shifts

---

Example:

$[13 \times 8]$

---

Since:

$$8 = 2^3$$

shift 13 left by 3 positions:

$$\boxed{13 \times 8 = 104}$$

In binary:

$$1101 \ll 3 = 1101000$$

---

## 43. Multiplication by Non-Power-of-Two

---

Example:

$$x \times 10$$

Since:

$$10 = 8 + 2$$

$$x \times 10 = (x \ll 3) + (x \ll 1)$$

This is useful conceptually in digital arithmetic.

---

## 44. Binary Division Basics

---

Binary long division is analogous to decimal division.

Example:

$$1100_2 \div 10_2$$

Since:

$$12 \div 2 = 6$$

result:

$$\boxed{110_2}$$

For powers of 2, division is much easier using right shifts.

---

## 45. Division by Powers of Two

---

Unsigned:

$$x / 2^k$$

can be performed using:

$$\boxed{\text{logical right shift by } k}$$

For signed 2's-complement values:

$$\boxed{\text{arithmetic right shift}}$$

is usually used.

---



## 46. Fixed Width Example

---

4-bit unsigned:

$$\lfloor 1110 = 14 \rfloor$$

Right shift once:

$$\lfloor 0111 = 7 \rfloor$$

Correct:

$$\lfloor 14/2 = 7 \rfloor$$

---

## 47. Negation Edge Case Again

---

For  $\lfloor n \rfloor$ -bit 2's complement:

$$\lfloor -2^{n-1} \rfloor$$

cannot be negated within the same width.

Example:

8-bit:

$$\lfloor 10000000 = -128 \rfloor$$

2's complement remains:

$$\lfloor 10000000 \rfloor$$

This indicates overflow if you mathematically attempt:

$$\lfloor -(-128) = +128 \rfloor$$

because +128 is not representable.

---

## 48. GATE Method — Detect Overflow Without Full Conversion

---

Suppose:

$$\lfloor 01101 + 01011 \rfloor$$

5-bit signed.

Both operands:

MSB = 0 !' positive.

Result:

$$\lfloor 11000 \rfloor$$

MSB = 1 !' negative.

Immediately conclude:

$$\lfloor \boxed{\text{overflow}} \rfloor$$

You do not need decimal conversion.

---

## 49. GATE Method — Opposite Signs

---

Suppose:

$$\lfloor 10110 + 01101 \rfloor$$

One negative, one positive.

For signed addition:

$$\lfloor \boxed{\text{overflow impossible}} \rfloor$$

So you can immediately rule out overflow.

---

## 50. Example — Carry but No Overflow

---

4-bit:

$$\lfloor 1100 + 1110 \rfloor$$

Interpret:

$$\lfloor 1100 = -4 \rfloor$$

$$\lfloor 1110 = -2 \rfloor$$

True result:

$$\lfloor -6 \rfloor$$

Add:

$$\lfloor 1100 + 1110 = 1\,1010 \rfloor$$

Carry exists.

Stored:

$$\lfloor 1010 = -6 \rfloor$$

Correct.

Therefore:

$$\lfloor \boxed{\text{carry, but no overflow}} \rfloor$$

---

## 51. Example — Carry and Overflow

---

Can both happen?

Yes.

4-bit signed:

$$\lfloor 1001 + 1010 \rfloor$$

Interpret:

$$\lfloor 1001 = -7 \rfloor$$

$$\lfloor 1010 = -6 \rfloor$$

True result:

$\lfloor -13 \rfloor$

Not representable.

Binary addition:

$\lfloor 1001 + 1010 = 1\,0011 \rfloor$

Carry-out = 1.

Stored result 0011 = +3.

So:

$\lfloor \boxed{\text{carry AND signed overflow}} \rfloor$

Again, they are independent concepts.

---

## 52. Example — No Carry and No Overflow

$\lfloor 0011 + 0010 = 0101 \rfloor$

$\lfloor 3 + 2 = 5 \rfloor$

No carry, no overflow.

---

## 53. Four Possible Cases

Therefore:

| Carry | Signed Overflow | Possible? |
|-------|-----------------|-----------|
| 0     | 0               | Yes       |
| 0     | 1               | Yes       |
| 1     | 0               | Yes       |
| 1     | 1               | Yes       |

So never infer signed overflow solely from carry-out.

---

## 54. Carry Meaning for Unsigned Numbers

For unsigned addition:

$\lfloor \boxed{\text{carry-out = overflow indicator}} \rfloor$

For signed 2's-complement addition:

$\lfloor \boxed{\text{use sign rule or carry-XOR rule}} \rfloor$

This is perhaps the single most important arithmetic distinction in this chapter.

---

## 55. GATE-Style L0 Question

Add:

$$\backslash[1011\_2+0101\_2\backslash]$$

$$\backslash[1011+0101=10000\backslash]$$

Answer:

$$\backslash\boxed{10000\_2}\backslash]$$

---

## 56. L1 — 2's Complement Subtraction

---

Compute 8-bit:

$$\backslash[25-9\backslash]$$

$$\backslash[25=00011001\backslash]$$

$$\backslash[9=00001001\backslash]$$

2's complement of 9:

$$\backslash[11110111\backslash]$$

Add:

$$\backslash[00011001+11110111=1\backslash,00010000\backslash]$$

Discard carry:

$$\backslash[00010000\backslash]$$

$$\backslash\boxed{16}\backslash]$$

---

## 57. L1 — Negative Result

---

Compute:

$$\backslash[9-25\backslash]$$

8-bit.

$$\backslash[9=00001001\backslash]$$

2's complement of 25:

$$\backslash[25=00011001\backslash]$$

Invert:

$$\backslash[11100110\backslash]$$

Add 1:

$$\backslash[11100111\backslash]$$

Now:

$$\backslash[00001001+11100111=11110000\backslash]$$

Interpret:

Unsigned = 240.

Signed:

$$\lfloor 240-256=-16 \rfloor$$

Therefore:

$$\lfloor \boxed{-16} \rfloor$$

---

## 58. L2 — Overflow

---

4-bit 2's complement:

$$\lfloor 0110+0011 \rfloor$$

Values:

$$\lfloor 6+3=9 \rfloor$$

Range:

$$\lfloor -8\text{ to }7 \rfloor$$

Binary result:

$$\lfloor 1001 \rfloor$$

Positive + positive gave negative.

Therefore:

$$\lfloor \boxed{\text{overflow}} \rfloor$$

---

## 59. L2 — No Overflow Despite Carry

---

$$\lfloor 1110+0011 \rfloor$$

Interpret:

$$\lfloor -2+3=1 \rfloor$$

Binary:

$$\lfloor 1110+0011=1\,0001 \rfloor$$

Stored:

$$\lfloor 0001=1 \rfloor$$

Carry exists, but answer is valid.

Therefore:

$$\lfloor \boxed{\text{No signed overflow}} \rfloor$$

---

## 60. L2 — Arithmetic Shift

---

An 8-bit signed 2's-complement register contains:

$$\lfloor 11110100 \rfloor$$

First find value:

Unsigned:

$\lfloor 244 \rfloor$

Signed:

$\lfloor 244-256=-12 \rfloor$

Arithmetic right shift once:

$\lfloor 11111010 \rfloor$

Unsigned:

$\lfloor 250 \rfloor$

Signed:

$\lfloor 250-256=-6 \rfloor$

Therefore:

$\lfloor \boxed{-6} \rfloor$

---

## 61. L3 — Trap: Left Shift

---

4-bit 2's-complement number:

$\lfloor 0100 \rfloor$

represents:

$\lfloor 4 \rfloor$

Shift left once:

$\lfloor 1000 \rfloor$

Bit pattern now represents:

$\lfloor -8 \rfloor$

But intended multiplication:

$\lfloor 4 \times 2=8 \rfloor$

which cannot fit.

Therefore:

$\lfloor \boxed{\text{overflow}} \rfloor$

---

## 62. L3 — Mixed Interpretation

---

4-bit bit pattern:

$\lfloor 1110 \rfloor$

Add:

$\lfloor 0011 \rfloor$

Binary result:

$\lfloor 1\,0001 \rfloor$

---

## Unsigned interpretation

$\lfloor 14+3=17 \rfloor$

4-bit cannot represent 17.

Unsigned overflow.

## Signed interpretation

$\lfloor -2+3=1 \rfloor$

No signed overflow.

Same addition, different overflow interpretation.

---

## 63. GATE PYQ Patterns for This Chapter

---

The recurring concepts you should expect are:

- represent numbers in 2's complement
- add/subtract fixed-width signed numbers
- detect signed overflow
- distinguish carry from overflow
- interpret arithmetic-right-shift behavior
- determine number of bits needed
- calculate result after fixed-width wraparound
- use complement addition rather than direct subtraction

When we later cover adders and ALU, these ideas reappear at the circuit level.

---

## 64. GATE TRAP BOOK — Chapter 3

---

### Trap 1 — Carry = overflow

Wrong for signed arithmetic.

Remember:

$\lfloor \boxed{\text{carry}} \neq \text{signed overflow} \rfloor$

---

### Trap 2 — Ignoring bit width

$\lfloor 0111+0001 \rfloor$

might be 8 in unrestricted arithmetic, but in 4-bit signed arithmetic it overflows.

---

### Trap 3 — Keeping carry in 2's-complement subtraction

For fixed-width 2's-complement arithmetic:

$\lfloor \boxed{\text{discard final carry}} \rfloor$

---

### Trap 4 — Forgetting end-around carry

---

In 1's-complement arithmetic:

$\boxed{\text{carry-out is added back}}$

---

### Trap 5 — Logical right shift on signed negative number

For signed arithmetic use:

$\boxed{\text{arithmetic right shift}}$

to preserve sign.

---

### Trap 6 — Assuming left shift always multiplies correctly

It does only when overflow does not occur.

---

### Trap 7 — Opposite-sign addition overflow

Impossible in normal fixed-width 2's-complement addition.

---

### Trap 8 — Same-sign subtraction rule

Subtraction overflow uses a different sign pattern from addition.

---

## 65. Formula Sheet — Chapter 3

---

### 2's-complement subtraction

$A - B = A + \text{2's complement}(B)$

---

### Unsigned addition overflow

$\boxed{\text{carry-out}=1}$

---

### Signed addition overflow

$\boxed{\text{same-sign operands and opposite-sign result}}$

---

### Hardware overflow

$V = C_{\text{into MSB}} \oplus C_{\text{out of MSB}}$

---

### 1's-complement arithmetic

$\boxed{\text{end-around carry}}$

---

### n-bit arithmetic

$\boxed{\text{mod } 2^n}$

---

### Left shift

---



$$\lfloor \boxed{x \ll k \approx x 2^k} \rfloor$$

if no overflow.

## Unsigned right shift

$$\lfloor \boxed{x \gg k = \left\lfloor x / 2^k \right\rfloor} \rfloor$$

## Product width

$$\lfloor \boxed{n \text{-bit} \times m \text{-bit} \leq n+m \text{ bits}} \rfloor$$

## 66. Carry vs Overflow Master Table

| Situation                      | Carry Used?           | Overflow Test                      |
|--------------------------------|-----------------------|------------------------------------|
| Unsigned addition              | Yes                   | Carry-out                          |
| Signed 2's-complement addition | Not sufficient        | Same-sign != different-sign result |
| 2's-complement subtraction     | Final carry discarded | Check signed range/sign rule       |
| 1's-complement addition        | End carry added back  | Representation-specific            |
| Logical shift                  | Bits may be lost      | Usually not called signed overflow |
| Arithmetic left shift          | Bits may be lost      | Can overflow signed range          |

## 67. 30-Second Revision

Remember these six facts:

1.

$$\lfloor \boxed{A-B=A+(-B)} \rfloor$$

2.

2's complement:

$$\lfloor \boxed{\text{flip} + 1} \rfloor$$

3.

In 2's-complement arithmetic:

$$\lfloor \boxed{\text{discard final carry}} \rfloor$$

4.

Signed overflow:

$$\lfloor \boxed{\text{same sign in != different sign out}} \rfloor$$

for addition.

5.

Unsigned overflow:

$\boxed{\text{carry out}}$

6.

Signed negative right shift:

$\boxed{\text{copy sign bit}}$

---

## What You Absolutely Must Remember

1. Addition rules.
2. Subtraction via 2's complement.
3. Bit width matters.
4. 2's-complement arithmetic is modulo  $(2^n)$ .
5. Carry-out and signed overflow are different.
6. Positive + positive != negative means overflow.
7. Negative + negative != positive means overflow.
8. Opposite-sign addition cannot overflow.
9. 1's complement uses end-around carry.
10. Arithmetic right shift preserves the sign.
11. Left shift can cause overflow.
12.  $(n)$ -bit  $\times$   $(m)$ -bit product may need  $(n+m)$  bits.

---

## Digital Logic Progress

**Completed:** Chapter 1 — Number Systems ' Chapter 2 — Signed Number Representation ' Chapter 3 — Binary Arithmetic '

**Current:** Chapter 4 — Boolean Algebra Fundamentals

**Remaining:** Boolean Algebra !' Boolean Functions/SOP/POS !' Universal Gates !' Algebraic Minimization !' K-map !' Tabular Method !' Combinational Circuits !' Sequential Circuits !' Fixed Point !' Floating Point

**NEXT**

---

## Chapter 4 — Boolean Algebra Fundamentals Ø=Ý4

Next we cover:

**Boolean variables !' AND/OR/NOT !' NAND/NOR !' XOR/XNOR !' complete truth tables !' all Boolean laws !' De Morgan !' absorption !' consensus theorem !' principle of duality !' fast GATE simplification tricks.**

USER

next

## CHAPTER 4 — BOOLEAN ALGEBRA FUNDAMENTALS 0=MUST MASTER

Boolean Algebra is the mathematical language of digital logic.

It is used to:

- describe logic gates
- simplify circuits
- build combinational circuits
- design sequential logic
- minimize hardware
- solve K-map and tabular-method questions

The core idea is very simple:

$\boxed{\text{Boolean variables can take only } 0 \text{ or } 1}$

### 1. Boolean Variable

A Boolean variable has only two possible values:

$\boxed{0, 1}$

Interpretations may be:

- FALSE / TRUE
- LOW / HIGH
- OFF / ON
- 0 V-ish / logic HIGH conceptually
- No / Yes

For GATE Digital Logic, think simply:

$\boxed{0=\text{False}, \quad 1=\text{True}}$

### 2. Basic Boolean Operations

Three fundamental operations:

1. AND
2. OR
3. NOT

From these we build other gates.

### 3. AND Operation

Boolean AND is represented as:

$A \cdot B$

or simply:

$\lceil AB \rceil$

Output is 1 **only when both inputs are 1.**

Truth table:

| A | B | $\lceil AB \rceil$ |
|---|---|--------------------|
| 0 | 0 | 0                  |
| 0 | 1 | 0                  |
| 1 | 0 | 0                  |
| 1 | 1 | 1                  |

Memory:

**AND = all required conditions must be true**

Example:

"Door opens if card valid AND PIN correct."

Both conditions must be true.

## 4. OR Operation

Represented as:

$\lceil A+B \rceil$

Output is 1 if **at least one input is 1.**

| A | B | $\lceil A+B \rceil$ |
|---|---|---------------------|
| 0 | 0 | 0                   |
| 0 | 1 | 1                   |
| 1 | 0 | 1                   |
| 1 | 1 | 1                   |

Memory:

**OR = one or more is enough**

## 5. NOT Operation

NOT complements a Boolean value.

Represented as:

$\lceil \bar{A} \rceil$

or:

$$\neg(A')$$

Truth table:

| A | $\neg(A')$ |
|---|------------|
| 0 | 1          |
| 1 | 0          |

So:

$$\neg(0'=1)$$

$$\neg(1'=0)$$

## 6. Buffer

A buffer does not change the logical value.

$$\neg(Y=A)$$

Truth table:

| A | Y |
|---|---|
| 0 | 0 |
| 1 | 1 |

It is not mathematically exciting, but in actual circuits buffers are used for drive strength, isolation, etc.

For GATE CS, only the logical behavior usually matters.

## 7. NAND Gate

NAND means:

$$\neg(\text{NOT AND})$$

Expression:

$$\neg(Y=(AB)')$$

Truth table:

| A | B | AB | $\neg((AB)')$ |
|---|---|----|---------------|
| 0 | 0 | 0  | 1             |
| 0 | 1 | 0  | 1             |
| 1 | 0 | 0  | 1             |
| 1 | 1 | 1  | 0             |

So NAND is 0 only when both inputs are 1.

## 8. NOR Gate

NOR means:

$\text{NOT OR}$

Expression:

$Y = (A + B)'$

Truth table:

| A | B | $(A + B)$ | $((A + B)')$ |
|---|---|-----------|--------------|
| 0 | 0 | 0         | 1            |
| 0 | 1 | 1         | 0            |
| 1 | 0 | 1         | 0            |
| 1 | 1 | 1         | 0            |

So NOR is 1 only when both inputs are 0.

## 9. XOR Gate $\oplus$

XOR = Exclusive OR.

Expression:

$A \oplus B$

Output is 1 when the inputs are **different**.

Truth table:

| A | B | $(A \oplus B)$ |
|---|---|----------------|
| 0 | 0 | 0              |
| 0 | 1 | 1              |
| 1 | 0 | 1              |
| 1 | 1 | 0              |

Therefore:

$\boxed{A \oplus B = A'B + AB'}$

Memory:

**XOR = different  $\neq$  1**

## 10. XNOR Gate

XNOR is complement of XOR.

$$\lceil A \odot B \rceil$$

or:

$$\lceil (A \oplus B)' \rceil$$

Output is 1 when inputs are the **same**.

| A | B | XNOR |
|---|---|------|
| 0 | 0 | 1    |
| 0 | 1 | 0    |
| 1 | 0 | 0    |
| 1 | 1 | 1    |

Expression:

$$\lceil \boxed{AB + A'B'} \rceil$$

Memory:

**XNOR = same !' 1**

## 11. XOR and XNOR Shortcut

Very useful:

$$\lceil \boxed{A \oplus 0 = A} \rceil$$

$$\lceil \boxed{A \oplus 1 = A'} \rceil$$

$$\lceil \boxed{A \oplus A = 0} \rceil$$

$$\lceil \boxed{A \oplus A' = 1} \rceil$$

For XNOR:

$$\lceil \boxed{A \odot 0 = A'} \rceil$$

$$\lceil \boxed{A \odot 1 = A} \rceil$$

$$\lceil \boxed{A \odot A = 1} \rceil$$

$$\lceil \boxed{A \odot A' = 0} \rceil$$

## 12. XOR as Odd-Parity Function

For multiple inputs:

$$\lceil A \oplus B \oplus C \rceil$$

equals 1 when the number of input 1s is **odd**.

Examples:

$\lceil 0,0,1 \rightarrow 1 \rceil$

one 1s !' odd.

$\lceil 1,1,1 \rightarrow 1 \rceil$

three 1s !' odd.

$\lceil 1,1,0 \rightarrow 0 \rceil$

two 1s !' even.

This becomes useful in parity circuits.

---

### 13. Boolean Algebra Is Not Ordinary Algebra

---

This is critical.

In Boolean algebra:

$\lceil 1+1=1 \rceil$

because  $\lceil (+) \rceil$  means OR, not arithmetic addition.

Also:

$\lceil A+A=A \rceil$

not  $\lceil (2A) \rceil$ .

And:

$\lceil A \cdot A=A \rceil$

You must separate Boolean operations from normal arithmetic.

---

### 14. Identity Laws

---

$\lceil \boxed{A+0=A} \rceil$

$\lceil \boxed{A \cdot 1=A} \rceil$

Why?

OR with 0 changes nothing.

AND with 1 changes nothing.

Memory:

**OR identity = 0    AND identity = 1**

---

### 15. Null / Domination Laws

---

$\lceil \boxed{A+1=1} \rceil$

$\lceil \boxed{A \cdot 0=0} \rceil$

---



Why?

OR with 1 is always 1.

AND with 0 is always 0.

---

## 16. Idempotent Laws

---

$$\boxed{A+A=A}$$

$$\boxed{AA=A}$$

Repeating a Boolean variable does nothing.

---

## 17. Complement Laws

---

$$\boxed{A+A'=1}$$

$$\boxed{AA'=0}$$

A variable and its complement cover all possibilities.

---

## 18. Involution Law

---

Double complement restores the original:

$$\boxed{(A')'=A}$$

---

## 19. Commutative Laws

---

Order does not matter:

$$\boxed{A+B=B+A}$$

$$\boxed{AB=BA}$$

---

## 20. Associative Laws

---

Grouping does not matter:

$$\boxed{A+(B+C)=(A+B)+C}$$

$$\boxed{A(BC)=(AB)C}$$

So:

$$\boxed{A+B+C}$$

and:

$$\boxed{ABC}$$

can usually be written without brackets.

---

## 21. Distributive Laws

---

First one looks familiar:

$$\boxed{A(B+C)=AB+AC}$$

But Boolean algebra also has the dual:

$$\boxed{A+BC=(A+B)(A+C)}$$

This second identity is extremely useful in GATE simplification.

---

## 22. Proving the Second Distributive Law

---

Start with:

$$(A+B)(A+C)$$

Expand:

$$AA+AC+AB+BC$$

Since:

$$AA=A$$

we get:

$$A+AC+AB+BC$$

Using absorption:

$$A+AC=A$$

and:

$$A+AB=A$$

Hence:

$$A+BC$$

Therefore:

$$\boxed{A+BC=(A+B)(A+C)}$$

---

## 23. Absorption Laws $A+AB=A$

---

Very important:

$$\boxed{A+AB=A}$$

and:

$$\boxed{A(A+B)=A}$$

Why?

For:

$$A+AB$$

factor:

$$\boxed{A(1+B)}$$

Since:

$$\boxed{1+B=1}$$

then:

$$\boxed{A \cdot 1 = A}$$

## 24. Absorption Variant

Another extremely useful identity:

$$\boxed{A + A'B = A + B}$$

Proof:

$$\boxed{A + A'B}$$

use:

$$\boxed{X + YZ = (X + Y)(X + Z)}$$

So:

$$\boxed{(A + A')(A + B)}$$

$$\boxed{= 1(A + B)}$$

$$\boxed{= A + B}$$

Similarly:

$$\boxed{A(A' + B) = AB}$$

## 25. De Morgan's Laws ~~0=1~~ MUST MASTER

These are among the most important laws in Digital Logic.

First:

$$\boxed{(AB)' = A' + B'}$$

Meaning:

NOT of AND becomes OR of complements.

Second:

$$\boxed{(A + B)' = A'B'}$$

Meaning:

NOT of OR becomes AND of complements.

Memory trick:

**Break the bar, flip the operator.**

AND !" OR.

## 26. De Morgan Example

---

Simplify:

$$\lceil (A+BC)' \rceil$$

Apply De Morgan:

$$\lceil A'(BC)' \rceil$$

Then:

$$\lceil (BC)'=B'+C' \rceil$$

Therefore:

$$\lceil \boxed{A'(B'+C')} \rceil$$

---

## 27. Larger De Morgan Example

---

$$\lceil (ABC)' \rceil$$

becomes:

$$\lceil \boxed{A'+B'+C'} \rceil$$

Similarly:

$$\lceil (A+B+C)' \rceil$$

becomes:

$$\lceil \boxed{A'B'C'} \rceil$$

---

## 28. Common De Morgan Trap

---

Wrong:

$$\lceil (A+B)'=A'+B' \rceil$$

Correct:

$$\lceil \boxed{(A+B)'=A'B'} \rceil$$

The operator must change.

Likewise:

Wrong:

$$\lceil (AB)'=A'B' \rceil$$

Correct:

$$\lceil \boxed{(AB)'=A'+B'} \rceil$$

---

## 29. Principle of Duality

---

A Boolean identity remains valid if we simultaneously interchange:

$$\lceil +\leftrightarrow\cdot \rceil$$

---

and:

$$\lceil 0 \rightarrow 1 \rceil$$

Variables and complements remain unchanged.

Example:

Original:

$$\lceil A + 0 = A \rceil$$

Dual:

$$\lceil A \cdot 1 = A \rceil$$

Both are true.

---

### 30. Another Duality Example

---

Original:

$$\lceil A + 1 = 1 \rceil$$

Dual:

$$\lceil A \cdot 0 = 0 \rceil$$

This helps you remember many laws in pairs instead of memorizing separately.

---

### 31. Consensus Theorem $XY + X'Z + YZ = XY + X'Z$

---

Important identity:

$$\boxed{AB + A'C + BC = AB + A'C}$$

The term:

$$\lceil BC \rceil$$

is called the **consensus term** and is redundant.

General form:

$$\boxed{XY + X'Z + YZ = XY + X'Z}$$

---

### 32. Why Consensus Term Is Redundant

---

Consider:

$$\lceil AB + A'C + BC \rceil$$

When  $(BC=1)$ :

$$\lceil B=1, C=1 \rceil$$

Now A is either:

**If  $(A=1)$**

then:

---

$$\lceil AB=1 \rceil$$

**If  $\lceil A=0 \rceil$**

then:

$$\lceil A'C=1 \rceil$$

So whenever  $\lceil BC \rceil$  is 1, one of the other two terms is already 1.

Thus  $\lceil BC \rceil$  adds nothing.

Hence:

$$\lceil \boxed{AB+A'C+BC=AB+A'C} \rceil$$

### 33. Dual Consensus Theorem

The dual is:

$$\lceil \boxed{(A+B)(A'+C)(B+C)=(A+B)(A'+C)} \rceil$$

Less commonly used manually, but you should recognize the pattern.

### 34. Useful Theorem: $\lceil (A+A'B) \rceil$

Already seen:

$$\lceil \boxed{A+A'B=A+B} \rceil$$

Likewise:

$$\lceil \boxed{A(A'+B)=AB} \rceil$$

These are high-speed GATE simplification tools.

### 35. Useful Theorem: $\lceil (AB+A'B) \rceil$

Factor B:

$$\lceil AB+A'B \rceil$$

$$\lceil =B(A+A') \rceil$$

$$\lceil =B \rceil$$

Therefore:

$$\lceil \boxed{AB+A'B=B} \rceil$$

Similarly:

$$\lceil \boxed{AB+AB'=A} \rceil$$

### 36. Useful Theorem: $\lceil (A+B)(A'+B) \rceil$

Use:

$$\lceil (X+Y)(X'+Y)=Y \rceil$$

Therefore:

$$\boxed{(A+B)(A'+B)=B}$$

Likewise:

$$\boxed{(A+B)(A+B')=A}$$

These patterns save a lot of time.

---

## 37. XOR Algebraic Forms

---

Remember:

$$\boxed{A \oplus B = A'B + AB'}$$

Another form:

$$\boxed{A \oplus B = (A+B)(A'+B')}$$

XNOR:

$$\boxed{AB + A'B'}$$

and:

$$\boxed{(A+B')(A'+B)}$$

---

## 38. XOR Associativity

---

XOR is associative:

$$\boxed{A \oplus (B \oplus C) = (A \oplus B) \oplus C}$$

and commutative:

$$\boxed{A \oplus B = B \oplus A}$$

Very useful in parity calculations.

---

## 39. XOR Cancellation

---

$$\boxed{A \oplus A = 0}$$

Therefore:

$$\boxed{A \oplus A \oplus B}$$

$$\boxed{= 0 \oplus B}$$

$$\boxed{B}$$

Also:

$$\boxed{A \oplus B \oplus A = B}$$

Very useful in XOR-based circuit problems.

---

## 40. Complement of XOR

---

$$\boxed{(A \oplus B)' = AB + A'B'}$$

which is XNOR.

Also:

$$\boxed{A \oplus B' = A \odot B}$$

and:

$$\boxed{A' \oplus B = A \odot B}$$

Complementing one XOR input changes XOR to XNOR.

Complementing both inputs:

$$\boxed{A' \oplus B' = A \oplus B}$$

## 41. Evaluate Boolean Expressions

Example:

$$F = A + B'C$$

Suppose:

$$A=0, B=0, C=1$$

Then:

$$B' = 1$$

$$B'C = 1$$

Thus:

$$F = 0 + 1$$

$$\boxed{F = 1}$$

## 42. Operator Precedence

Unless brackets change it, typically:

1. NOT
2. AND
3. OR

So:

$$A + B'C$$

means:

$$A + (B'C)$$

not:

$$(A + B')C$$

Always use parentheses when ambiguity exists.



### 43. Simplification Example 1

---

Simplify:

$$\lceil A+AB \rceil$$

By absorption:

$$\lceil \boxed{A} \rceil$$

---

### 44. Simplification Example 2

---

Simplify:

$$\lceil A+A'B \rceil$$

Use shortcut:

$$\lceil \boxed{A+B} \rceil$$

---

### 45. Simplification Example 3

---

Simplify:

$$\lceil AB+AB' \rceil$$

Factor A:

$$\lceil A(B+B') \rceil$$

$$\lceil =A(1) \rceil$$

$$\lceil \boxed{A} \rceil$$

---

### 46. Simplification Example 4

---

Simplify:

$$\lceil (A+B)(A+B') \rceil$$

Use pattern:

$$\lceil \boxed{A} \rceil$$

Or expand:

$$\lceil AA+AB'+AB+BB' \rceil$$

$$\lceil =A+A(B'+B)+0 \rceil$$

$$\lceil =A+A \rceil$$

$$\lceil \boxed{A} \rceil$$

---

### 47. Simplification Example 5

---

Simplify:

$$\lceil AB+A'C+BC \rceil$$

---

Consensus theorem:

$$\boxed{AB + A'C}$$

No further work needed.

---

## 48. Simplification Example 6

---

Simplify:

$$(A+B)' + AB'$$

First De Morgan:

$$A'B' + AB'$$

Factor:

$$B'(A' + A)$$

$$= B'$$

Therefore:

$$\boxed{B'}$$

---

## 49. GATE Method — Look for Complements First

---

Whenever you see:

$$A + A'$$

immediately replace with:

$$1$$

Whenever you see:

$$AA'$$

immediately replace with:

$$0$$

Example:

$$ABC + AB'C$$

factor:

$$AC(B + B')$$

$$= AC$$

Fast.

---

## 50. GATE Method — Look for Absorption

---

Whenever you see:

$$X + XY$$

---

replace with:

$\boxed{X}$

Example:

$\boxed{A+ABC}$

$\boxed{\boxed{A}}$

Similarly:

$\boxed{X(X+Y)=X}$

---

## 51. GATE Method — Detect XOR Patterns

---

If you see:

$\boxed{A'B+AB'}$

do not expand further.

Recognize:

$\boxed{A \oplus B}$

If:

$\boxed{AB+A'B'}$

recognize:

$\boxed{A \odot B}$

---

## 52. GATE Method — Use Consensus

---

Expression:

$\boxed{XY+X'Z+YZ}$

Immediately remove:

$\boxed{YZ}$

Result:

$\boxed{XY+X'Z}$

This avoids unnecessary expansion.

---

## 53. GATE-Style L0 Question

---

Simplify:

$\boxed{A+0}$

Answer:

$\boxed{A}$

---

## 54. L0 Question

---

Simplify:

$$\lceil AA' \rceil$$

Answer:

$$\lceil \boxed{0} \rceil$$

---

## 55. L1 Question

---

Simplify:

$$\lceil AB+A'B \rceil$$

Factor:

$$\lceil B(A+A') \rceil$$

$$\lceil =B \rceil$$

Answer:

$$\lceil \boxed{B} \rceil$$

---

## 56. L1 Question

---

Simplify:

$$\lceil A+A'B \rceil$$

Use shortcut:

$$\lceil \boxed{A+B} \rceil$$

---

## 57. L2 — GATE-Level Simplification

---

Simplify:

$$\lceil AB+A'B+AB' \rceil$$

First:

$$\lceil AB+A'B \rceil$$

$$\lceil =B(A+A') \rceil$$

$$\lceil =B \rceil$$

So:

$$\lceil B+AB' \rceil$$

Use:

$$\lceil X+YZ=(X+Y)(X+Z) \rceil$$

or shortcut:

$$\lceil B+AB'=A+B \rceil$$

---

Therefore:

$$\boxed{A+B}$$

---

## 58. L2 — De Morgan

---

Simplify:

$$\boxed{[(A+B')C]'$$

Apply De Morgan:

$$\boxed{(A+B')'+C'}$$

Now:

$$\boxed{(A+B')'=A'B}$$

Therefore:

$$\boxed{A'B+C'}$$

---

## 59. L2 — XOR

---

If:

$$\boxed{F=A\oplus B\oplus C}$$

for:

$$\boxed{A=1, B=0, C=1}$$

Number of 1s = 2, even.

Therefore:

$$\boxed{F=0}$$

GATE shortcut: no need to expand XOR.

---

## 60. L3 — Trap

---

Simplify:

$$\boxed{A+A'BC}$$

Use:

$$\boxed{A+A'X=A+X}$$

where:

$$\boxed{X=BC}$$

Therefore:

$$\boxed{A+BC}$$

Do not incorrectly simplify it to  $(A+B+C)$ .

---

## 61. L3 — Consensus Recognition

Simplify:

$$\lceil PQ+P'R+QR \rceil$$

Consensus theorem:

$$\lceil \boxed{PQ+P'R} \rceil$$

The term  $\lceil QR \rceil$  is redundant.

## 62. Truth-Table Method for Verifying Identities

If unsure whether two expressions are equivalent, make a truth table.

Example:

Verify:

$$\lceil A+A'B=A+B \rceil$$

| A | B | A' | A'B | $\lceil(A+A'B)\rceil$ | $\lceil(A+B)\rceil$ |
|---|---|----|-----|-----------------------|---------------------|
| 0 | 0 | 1  | 0   | 0                     | 0                   |
| 0 | 1 | 1  | 1   | 1                     | 1                   |
| 1 | 0 | 0  | 0   | 1                     | 1                   |
| 1 | 1 | 0  | 0   | 1                     | 1                   |

Columns match.

Therefore equivalent.

This is slower than algebra but very reliable.

## 63. How Many Rows in a Truth Table?

For  $\lceil(n)\rceil$  Boolean variables:

$$\lceil \boxed{2^n} \rceil$$

rows.

Examples:

2 variables:

$$\lceil 2^2=4 \rceil$$

3 variables:

$$\lceil 2^3=8 \rceil$$

4 variables:

$$\lceil 2^4=16 \rceil$$

This will matter heavily in minterms and K-maps.

---

## 64. Boolean Function Equality

Two Boolean expressions are equivalent if they produce the same output for **every possible input combination**.

So:

$$\forall F_1 = F_2$$

means their truth-table output columns are identical.

---

## 65. Complementing an Entire Function

Suppose:

$$\forall F = AB + C$$

Then:

$$\forall F' = (AB + C)'$$

By De Morgan:

$$\forall (AB)'C'$$

$$\forall (A' + B')C'$$

Therefore:

$$\forall \boxed{F' = (A' + B')C'}$$

Be careful to complement the entire expression, not individual terms randomly.

---

## 66. Example With Nested Complements

Simplify:

$$\forall [(A+B)' + C]'$$

Inside:

$$\forall (A+B)' = A'B'$$

So:

$$\forall (A'B' + C)'$$

Apply De Morgan:

$$\forall (A'B')'C'$$

$$\forall (A+B)C'$$

Therefore:

$$\forall \boxed{(A+B)C'}$$

---

## 67. Boolean Difference From Arithmetic Addition

This deserves repetition.

---

In ordinary arithmetic:

$$\backslash [ 1+1=2 \backslash ]$$

In Boolean OR:

$$\backslash [ 1+1=1 \backslash ]$$

In ordinary arithmetic:

$$\backslash [ A+A=2A \backslash ]$$

In Boolean:

$$\backslash [ A+A=A \backslash ]$$

Do not carry normal algebra rules blindly into Boolean logic.

---

## 68. High-Yield Boolean Identity Sheet

---

Memorize these:

$$\backslash [ A+0=A \backslash ]$$

$$\backslash [ A \cdot 1=A \backslash ]$$

$$\backslash [ A+1=1 \backslash ]$$

$$\backslash [ A \cdot 0=0 \backslash ]$$

$$\backslash [ A+A=A \backslash ]$$

$$\backslash [ AA=A \backslash ]$$

$$\backslash [ A+A'=1 \backslash ]$$

$$\backslash [ AA'=0 \backslash ]$$

$$\backslash [ (A')'=A \backslash ]$$

$$\backslash [ A+AB=A \backslash ]$$

$$\backslash [ A(A+B)=A \backslash ]$$

$$\backslash [ A+A'B=A+B \backslash ]$$

$$\backslash [ A(A'+B)=AB \backslash ]$$

$$\backslash [ (AB)'=A'+B' \backslash ]$$

$$\backslash [ (A+B)'=A'B' \backslash ]$$

$$\backslash [ AB+A'C+BC=AB+A'C \backslash ]$$

These are enough to solve a huge fraction of algebraic simplification questions.

---

## 69. Memory Structure

---

Rather than memorize randomly, organize laws into pairs.

### Identity

$$\backslash [ A+0=A \backslash ]$$

$$\backslash [ A1=A \backslash ]$$



### Dominance

$$A+1=1$$

$$A0=0$$

### Complement

$$A+A'=1$$

$$AA'=0$$

### Absorption

$$A+AB=A$$

$$A(A+B)=A$$

### De Morgan

$$(AB)'=A'+B'$$

$$(A+B)'=A'B'$$

This pairing uses duality.

---

## 70. Common GATE Traps

### Trap 1 — Treating Boolean + as arithmetic +

$$1+1=1$$

not 2.

### Trap 2 — Wrong De Morgan

Do not forget to change AND to OR.

### Trap 3 — XOR confusion

XOR:

$$\boxed{\text{different inputs} \Rightarrow 1}$$

XNOR:

$$\boxed{\text{same inputs} \Rightarrow 1}$$

### Trap 4 — Misusing absorption

$$A+AB=A$$

but:

$$A+BC$$

cannot generally become A.

### Trap 5 — Missing consensus term

$$\{ AB+A'C+BC \}$$

contains redundant  $(BC)$ .

---

### Trap 6 — Incorrect distributive law

Boolean algebra has both:

$$\{ A(B+C)=AB+AC \}$$

and:

$$\{ A+BC=(A+B)(A+C) \}$$

---

### Trap 7 — Forgetting precedence

NOT before AND before OR.

---

## 71. GATE PYQ Connection

This chapter's concepts usually appear inside larger GATE problems rather than as isolated "state the law" questions.

Common historical question patterns include:

- simplify a Boolean expression
- identify equivalent logic circuits
- transform NAND/NOR networks
- recognize XOR/XNOR behavior
- compute complement using De Morgan
- determine whether two functions are equivalent
- simplify before K-map or circuit realization
- exploit consensus/absorption to reduce gate count

The next three chapters build directly on this foundation.

---

## 72. Chapter 4 Formula / Identity Sheet

### AND

$$\{ Y=AB \}$$

### OR

$$\{ Y=A+B \}$$

### NOT

$$\{ Y=A' \}$$

### NAND

$$\{ Y=(AB)' \}$$

### NOR

$$\{ Y=(A+B)' \}$$

## XOR

$$\boxed{A \oplus B = A'B + AB'}$$

## XNOR

$$\boxed{AB + A'B'}$$

## De Morgan

$$\boxed{(AB)' = A' + B'}$$

$$\boxed{(A+B)' = A'B'}$$

## Absorption

$$\boxed{A + AB = A}$$

$$\boxed{A(A+B) = A}$$

## Useful reduction

$$\boxed{A + A'B = A + B}$$

## Consensus

$$\boxed{AB + A'C + BC = AB + A'C}$$

---

## 73. 30-Second Revision

---

Remember:

- AND !' both 1
- OR !' at least one 1
- XOR !' different
- XNOR !' same
- NAND = NOT AND
- NOR = NOT OR

Then memorize:

$$\boxed{A+0=A}$$

$$\boxed{A1=A}$$

$$\boxed{A+1=1}$$

$$\boxed{A0=0}$$

$$\boxed{A+A'=1}$$

$$\boxed{AA'=0}$$

$$\boxed{A+AB=A}$$

$$\boxed{A+A'B=A+B}$$

and:

**De Morgan = break bar + swap AND/OR.**

## What You Absolutely Must Remember

1. Boolean variables are only 0 or 1.
2.  $\vee$  (+) means OR, multiplication means AND.
3. XOR = different.
4. XNOR = same.
5. NAND and NOR are complements of AND and OR.
6. De Morgan's laws must become automatic.
7. Absorption saves many steps.
8.  $\vee(A+A'B=A+B)$  is a high-value shortcut.
9. Consensus theorem removes redundant terms.
10. Boolean algebra is not normal arithmetic algebra.
11. Truth tables can always verify equivalence.
12.  $\vee(n)$  variables give  $\vee(2^n)$  rows.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra Fundamentals '

**Current: Chapter 5 — Boolean Functions, SOP/POS, Minterms & Maxterms**

**Remaining:** Boolean Functions !' Universal Gates !' Algebraic Minimization !' K-map !' Tabular Method !' Combinational Circuits !' Sequential Circuits !' Fixed Point !' Floating Point

### NEXT

**Chapter 5 — Boolean Functions and Representation Ø=Ý4**

We will cover:

**literals !' product/sum terms !' minterms !' maxterms !' canonical SOP/POS !'  $\vee(\vee\Sigma m)$  and  $\vee(\vee\Pi M)$  notation !' truth-table conversion !' minterm/maxterm numbering !' complements !' missing terms !' GATE traps.**

USER

next

ASSISTANT

## CHAPTER 5 — BOOLEAN FUNCTIONS, MINTERMS, MAXTERMS, SOP & POS Ø=Ý4 MUST MASTER

This chapter is the bridge between:

- truth tables
- Boolean expressions
- K-maps
- decoders
- multiplexers

- logic minimization

You should become very comfortable moving between:

$\text{Truth Table} \rightarrow \text{Minterms/Maxterms} \rightarrow \text{SOP/POS}$

## 1. Boolean Function

A Boolean function maps Boolean inputs to a Boolean output.

Example:

$F(A,B)=A+B$

Inputs:

$A, B \in \{0,1\}$

Output:

$F \in \{0,1\}$

Truth table:

| A | B | F=A+B |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 1     |
| 1 | 0 | 1     |
| 1 | 1 | 1     |

## 2. Literal

A **literal** is:

- a Boolean variable
- or its complement

Examples:

$A$

$B'$

$C$

Each is one literal.

Expression:

$AB'C$

contains:

$\boxed{3 \text{ literals}}$

### 3. Product Term

---

A product term is an AND of literals.

Examples:

$$\lceil AB \rceil$$

$$\lceil A'BC \rceil$$

$$\lceil ABC' \rceil$$

Because multiplication means AND.

---

### 4. Sum Term

---

A sum term is an OR of literals.

Examples:

$$\lceil A+B \rceil$$

$$\lceil A'+B+C' \rceil$$

$$\lceil A+B'+C \rceil$$

---

### 5. SOP — Sum of Products

---

SOP means:

**OR of product terms.**

Example:

$$\lceil F=AB+A'C+BC \rceil$$

Individual product terms:

$$\lceil AB, \quad A'C, \quad BC \rceil$$

They are ORed together.

Therefore:

$$\lceil \boxed{\text{SOP}} \rceil$$

---

### 6. POS — Product of Sums

---

POS means:

**AND of sum terms.**

Example:

$$\lceil F=(A+B)(A'+C)(B+C') \rceil$$

Each bracket is a sum term.

---

The brackets are multiplied.

Therefore:

$\boxed{\text{POS}}$

---

## 7. Standard vs Canonical Forms

This distinction is very important.

### Standard SOP

An OR of product terms, but every term does **not necessarily contain every variable**.

Example:

$AB + A'C$

This is SOP.

But if variables are  $(A, B, C)$ , then:

- $(AB)$  misses  $(C)$
- $(A'C)$  misses  $(B)$

Therefore it is not canonical.

---

## 8. Canonical SOP

In canonical SOP:

**Every product term contains every variable exactly once, either complemented or uncomplemented.**

Example with variables  $(A, B, C)$ :

$A'B'C + A'BC + ABC'$

Every product term contains:

$A, B, C$

Therefore:

$\boxed{\text{canonical SOP}}$

Canonical product terms are called:

$\boxed{\text{minterms}}$

---

## 9. Canonical POS

In canonical POS:

**Every sum term contains every variable exactly once.**

Example:

$$\lceil (A+B+C)(A+B'+C)(A'+B+C') \rceil$$

Each sum contains:

$$\lceil A, B, C \rceil$$

Therefore:

$$\lceil \boxed{\text{canonical POS}} \rceil$$

Canonical sum terms are called:

$$\lceil \boxed{\text{maxterms}} \rceil$$

## 10. What Is a Minterm?

A **minterm** is a product term containing all variables exactly once.

For two variables  $(A, B)$ :

Possible minterms:

$$\lceil A'B' \rceil$$

$$\lceil A'B \rceil$$

$$\lceil AB' \rceil$$

$$\lceil AB \rceil$$

There are:

$$\lceil \boxed{2^n} \rceil$$

possible minterms for  $(n)$  variables.

## 11. Key Property of a Minterm

A minterm evaluates to:

$$\lceil \boxed{1} \rceil$$

for exactly **one** input combination.

Example:

$$\lceil A'BC \rceil$$

When is this 1?

We need:

$$\lceil A'=1 \Rightarrow A=0 \rceil$$

$$\lceil B=1 \rceil$$

$$\lceil C=1 \rceil$$

So only for:

$$\lceil (A, B, C) = (0, 1, 1) \rceil$$

That is why a minterm identifies exactly one truth-table row where output is 1.



## 12. Minterm Numbering

For variables ordered:

$\{A, B, C\}$

treat the row as a binary number:

$\{ABC\}$

Then convert to decimal.

Example:

$\{A=0, B=1, C=1\}$

Binary:

$\{011_2\}$

Decimal:

$\{3\}$

Therefore:

$\{A'BC = m_3\}$

## 13. Minterm Rule $\Sigma$

For a minterm:

**Input 0 !' complemented variable**

**Input 1 !' uncomplemented variable**

Memory trick:

**MINTERM follows the bits**

0 !' prime 1 !' normal

## 14. 3-Variable Minterm Table

| Decimal | ABC | Minterm      |
|---------|-----|--------------|
| 0       | 000 | $\{A'B'C'\}$ |
| 1       | 001 | $\{A'B'C\}$  |
| 2       | 010 | $\{A'BC'\}$  |
| 3       | 011 | $\{A'BC\}$   |
| 4       | 100 | $\{AB'C'\}$  |
| 5       | 101 | $\{AB'C\}$   |
| 6       | 110 | $\{ABC'\}$   |

| Decimal | ABC | Minterm     |
|---------|-----|-------------|
| 7       | 111 | $\neg(A)BC$ |

You should eventually create these directly without a table.

## 15. Example

Find minterm corresponding to decimal 5 for variables  $\neg(A,B,C)$ .

Convert:

$$\neg[5 = 101_2]$$

Bits:

$$\neg[A = 1, \neg B = 0, C = 1]$$

Minterm rule:

1 !' normal 0 !' complement

Therefore:

$$\neg[\boxed{m_5 = AB'C}]$$

## 16. Sum of Minterms

Suppose a function is 1 for rows:

$$\neg[1, 3, 5, 7]$$

Then:

$$\neg[F = \sum m(1, 3, 5, 7)]$$

This means:

$$\neg[F = m_1 + m_3 + m_5 + m_7]$$

For three variables:

$$\neg[F = A'B'C + A'BC + AB'C + ABC]$$

## 17. Sigma Notation

Canonical SOP is written:

$$\neg[\boxed{F = \sum m(\text{indices where } F=1)}]$$

So if truth table output is 1 at:

$$\neg[0, 2, 6]$$

then:

$$\neg[\boxed{F = \sum m(0, 2, 6)}]$$

## 18. What Is a Maxterm?

---

A maxterm is a sum term containing all variables exactly once.

For two variables:

$$\lceil A+B \rceil$$

$$\lceil A+B' \rceil$$

$$\lceil A'+B \rceil$$

$$\lceil A'+B' \rceil$$

---

## 19. Key Property of a Maxterm

---

A maxterm evaluates to:

$$\lceil \boxed{0} \rceil$$

for exactly one input combination.

Example:

$$\lceil A+B'+C \rceil$$

When is this 0?

Every OR input must be 0.

So:

$$\lceil A=0 \rceil$$

$$\lceil B'=0 \Rightarrow B=1 \rceil$$

$$\lceil C=0 \rceil$$

Thus:

$$\lceil (A,B,C)=(0,1,0) \rceil$$

Binary:

$$\lceil 010_2=2 \rceil$$

Therefore:

$$\lceil A+B'+C=M_2 \rceil$$

---

## 20. Maxterm Rule $\Sigma=4$

---

Maxterm uses the **opposite convention** from minterm.

For a maxterm:

**Input 0 !' uncomplemented variable**

**Input 1 !' complemented variable**

Memory trick:

**MAXTERM is opposite to the bits**

This rule causes many mistakes.

## 21. 3-Variable Maxterm Table

| Decimal | ABC | Maxterm                 |
|---------|-----|-------------------------|
| 0       | 000 | $\overline{(A+B+C)}$    |
| 1       | 001 | $\overline{(A+B+C')}$   |
| 2       | 010 | $\overline{(A+B'+C)}$   |
| 3       | 011 | $\overline{(A+B'+C')}$  |
| 4       | 100 | $\overline{(A'+B+C)}$   |
| 5       | 101 | $\overline{(A'+B+C')}$  |
| 6       | 110 | $\overline{(A'+B'+C)}$  |
| 7       | 111 | $\overline{(A'+B'+C')}$ |

## 22. Example

Find:

$\overline{[M_5]}$

for  $\overline{(A,B,C)}$ .

$\overline{[5=101_2]}$

For maxterm:

1 !' complement 0 !' normal

Therefore:

$\overline{[M_5=A'+B+C']}$

## 23. Product of Maxterms

Suppose  $\overline{(F=0)}$  at rows:

$\overline{[0,2,4,6]}$

Then:

$\overline{[F=\prod M(0,2,4,6)]}$

This is canonical POS.

## 24. Pi Notation

Canonical POS:

$\boxed{F = \Pi M(\text{indices where } F=0)}$

So:

- $\Sigma m$  lists 1s.
- $\Pi M$  lists 0s.

This is one of the most important memory rules.

## 25. Master Memory Trick

$\Sigma m \rightarrow \text{where } F=1$

$\Pi M \rightarrow \text{where } F=0$

Think:

**SOP collects the ON rows.**

**POS collects the OFF rows.**

## 26. Truth Table !' Canonical SOP

Suppose:

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Rows with:

$F=1$

are:

01, 10

Their decimal indices:

1, 2

Therefore:

$\boxed{F = \Sigma m(1,2)}$

Expand:

$$\lceil m_1 = A'B \rceil$$

$$\lceil m_2 = AB' \rceil$$

So:

$$\lceil \boxed{F = A'B + AB'} \rceil$$

This is XOR.

---

## 27. Same Truth Table !' Canonical POS

---

Rows with:

$$\lceil F = 0 \rceil$$

are:

$$\lceil 00, 11 \rceil$$

Indices:

$$\lceil 0, 3 \rceil$$

Therefore:

$$\lceil \boxed{F = \prod M(0, 3)} \rceil$$

Expand:

$$\lceil M_0 = A + B \rceil$$

$$\lceil M_3 = A' + B' \rceil$$

So:

$$\lceil \boxed{F = (A + B)(A' + B')} \rceil$$

This is also XOR.

---

## 28. SOP !"POS Conversion Shortcut

---

Suppose:

$$\lceil F = \sum m(1, 3, 4, 6) \rceil$$

Assume 3 variables.

Possible indices:

$$\lceil 0, 1, 2, 3, 4, 5, 6, 7 \rceil$$

F is 1 at:

$$\lceil 1, 3, 4, 6 \rceil$$

Therefore F is 0 at:

$$\lceil 0, 2, 5, 7 \rceil$$

So:

$$\lceil \boxed{F = \prod M(0, 2, 5, 7)} \rceil$$

No algebra needed.

---

---

## 29. Complement Relationship

---

If:

$$F = \sum m(1, 2, 5)$$

for 3 variables, then all indices are:

$$0 \text{ through } 7$$

So:

$$F' = 1$$

where  $(F=0)$ .

Thus:

$$\boxed{F' = \sum m(0, 3, 4, 6, 7)}$$

And equivalently:

$$\boxed{F' = \prod M(1, 2, 5)}$$

Important relationships:

$$\boxed{F = \sum m(S) \rightarrow F' = \prod M(S)}$$

and also:

$$\boxed{F = \prod M(S) \rightarrow F' = \sum m(S)}$$

with the same index set.

---

## 30. Why Same Indices Appear in Complement

---

Suppose:

$$F = \sum m(1, 3)$$

This means  $F=1$  at rows 1 and 3.

Therefore  $F'=0$  at those same rows.

So:

$$F' = \prod M(1, 3)$$

Hence:

$$\boxed{\sum m(S) \leftrightarrow \prod M(S) \text{ under complement}}$$

---

## 31. Minterm and Maxterm Relationship

---

For the same index:

$$\boxed{m_i = (M_i)'}$$

and:

$$\boxed{M_i = (m_i)'}$$

---

Example:

$$\{ m_2 = A'BC' \}$$

Then:

$$\{ (m_2)' = (A'BC')' \}$$

Using De Morgan:

$$\{ A+B'+C \}$$

which is:

$$\{ M_2 \}$$

Correct.

---

## 32. Converting Noncanonical SOP to Canonical SOP

---

Suppose:

$$\{ F = A + B'C \}$$

Variables:

$$\{ A, B, C \}$$

Term  $\{A\}$  does not contain  $\{B, C\}$ .

We can introduce missing variables using:

$$\{ X + X' = 1 \}$$

So:

$$\{ A = A(B+B')(C+C') \}$$

Expand:

$$\{ A(BC + BC' + B'C + B'C') \}$$

$$\{ = ABC + ABC' + AB'C + AB'C' \}$$

Now:

$$\{ B'C \}$$

misses A.

Use:

$$\{ A + A' = 1 \}$$

$$\{ B'C = B'C(A + A') \}$$

$$\{ = AB'C + A'B'C \}$$

Combine and remove duplicates.

Final canonical SOP:

$$\{ F = A'B'C + AB'C' + AB'C + ABC' + ABC \}$$

Indices:

$$\{ 1, 4, 5, 6, 7 \}$$



Therefore:

$$F = \sum m(1,4,5,6,7)$$

---

### 33. Faster Method for Noncanonical SOP

---

Instead of expanding blindly, identify truth-table rows.

For:

$$F = A + B'C$$

Term  $(A)$  is 1 whenever:

$$A = 1$$

regardless of B,C.

That gives rows:

$$100, 101, 110, 111$$

indices:

$$4, 5, 6, 7$$

Term  $(B'C)$  means:

$$B = 0, C = 1$$

A can be 0 or 1.

Rows:

$$001, 101$$

indices:

$$1, 5$$

Union:

$$1, 4, 5, 6, 7$$

Thus:

$$F = \sum m(1,4,5,6,7)$$

This is usually faster for GATE.

---

### 34. Converting Noncanonical POS to Canonical POS

---

Suppose:

$$F = (A+B)(B'+C)$$

Variables:

$$A, B, C$$

A sum term missing a variable can be expanded using:

$$X = (X+Y)(X+Y')$$

Example:

$$\lceil A+B \rceil$$

missing C:

$$\lceil \boxed{A+B=(A+B+C)(A+B+C')} \rceil$$

Similarly:

$$\lceil B'+C \rceil$$

missing A:

$$\lceil \boxed{B'+C=(A+B'+C)(A'+B'+C)} \rceil$$

Then combine all maxterms.

---

## 35. Important Expansion Identities

---

For SOP:

If product term  $\lceil(X)\rceil$  is missing variable A:

$$\lceil \boxed{X=X(A+A')=XA+XA'} \rceil$$

For POS:

If sum term  $\lceil(X)\rceil$  is missing A:

$$\lceil \boxed{X=(X+A)(X+A')} \rceil$$

These are the standard canonicalization techniques.

---

## 36. Example: Find Minterm Number Directly

---

Term:

$$\lceil AB'C'D \rceil$$

Variable order:

$$\lceil A,B,C,D \rceil$$

Convert complements to bits:

- A !' 1
- B' !' 0
- C' !' 0
- D !' 1

So:

$$\lceil 1001_2=9 \rceil$$

Therefore:

$$\lceil \boxed{AB'C'D=m_9} \rceil$$


---

## 37. Example: Find Maxterm Number Directly

---

Term:

$$\lceil A'+B+C'+D \rceil$$

For maxterms, complemented means bit 1 and uncomplemented means bit 0.

So:

- $A' \neq 1$
- $B \neq 0$
- $C' \neq 1$
- $D \neq 0$

Bits:

$$\lceil 1010_2=10 \rceil$$

Therefore:

$$\lceil \boxed{A'+B+C'+D=M_{\{10\}}} \rceil$$

### 38. GATE Shortcut: Read Minterm Literally

Minterm:

$$\lceil A'BC'D \rceil$$

Write:

$$\lceil 0 \setminus 1 \setminus 0 \setminus 1 \rceil$$

So:

$$\lceil 0101_2=5 \rceil$$

Thus:

$$\lceil m_5 \rceil$$

### 39. GATE Shortcut: Maxterm Reverse Rule

Maxterm:

$$\lceil A+B'+C+D' \rceil$$

Write opposite of literal appearance:

- $A \neq 0$
- $B' \neq 1$
- $C \neq 0$
- $D' \neq 1$

Thus:

$$\lceil 0101_2=5 \rceil$$

So:

$$\lceil \boxed{M_5} \rceil$$

## 40. Why Is the Maxterm Convention Opposite?

---

Take:

$$M_5$$

for  $(A,B,C=101)$ .

We want the sum term to be 0 for exactly:

$$A=1, B=0, C=1$$

For OR to be zero, every literal must be zero.

To make  $A=1$  produce 0, use:

$$A'$$

To make  $B=0$  produce 0, use:

$$B$$

To make  $C=1$  produce 0, use:

$$C'$$

Thus:

$$M_5 = A' + B + C'$$

That is why the convention reverses.

---

## 41. Number of Minterms/Maxterms

---

For  $(n)$  variables:

$$\boxed{2^n \text{ minterms}}$$

and:

$$\boxed{2^n \text{ maxterms}}$$

Examples:

3 variables:

$$8$$

4 variables:

$$16$$

5 variables:

$$32$$

---

## 42. Number of Possible Boolean Functions

---

This is an important theoretical result.

For  $(n)$  variables, there are:

$$2^{2^n}$$

---

input rows.

For each row, function output can independently be:

$\{0, 1\}$

Therefore number of distinct Boolean functions is:

$$2^{2^n}$$

Examples:

1 variable:

$$2^{2^1} = 4$$

2 variables:

$$2^{2^2} = 16$$

3 variables:

$$2^{2^3} = 256$$

This is a good GATE conceptual fact.

---

## 43. Example

How many Boolean functions exist for 4 variables?

$$2^{2^4}$$

$$= 2^{16}$$

$$65536$$

---

## 44. Function Specified by ON-Set

The rows where:

$$F = 1$$

form the:

$$\{\text{ON-set}\}$$

Example:

$$F = \sum m(1, 2, 7)$$

ON-set:

$$\{1, 2, 7\}$$

---

## 45. OFF-Set

Rows where:

$$F = 0$$

form:

$\boxed{\text{OFF-set}}$

For 3-variable:

$F = \sum m(1,2,7)$

all rows:

$0-7$

OFF-set:

$\boxed{\{0,3,4,5,6\}}$

## 46. Don't-Care Set Preview

Later in K-maps, certain input combinations may not matter.

These are represented as:

$d(\text{dots})$

or:

$X$

Example:

$F = \sum m(1,3,7) + d(0,2)$

Means:

- definitely 1 at 1,3,7
- don't care at 0,2
- definitely 0 elsewhere

We will study this thoroughly in K-maps.

## 47. Canonical SOP from Function Output

Suppose:

| ABC | F |
|-----|---|
| 000 | 1 |
| 001 | 0 |
| 010 | 1 |
| 011 | 1 |
| 100 | 0 |
| 101 | 0 |
| 110 | 1 |
| 111 | 0 |

Rows with output 1:

$$\Sigma (0,2,3,6)$$

Thus:

$$F = \Sigma m(0,2,3,6)$$

Expanded:

$$A'B'C' + A'BC' + A'BC + ABC$$

### 48. Canonical POS of Same Function

Rows with output 0:

$$\Sigma (1,4,5,7)$$

Therefore:

$$F = \Pi M(1,4,5,7)$$

Expanded:

$$(A+B+C')(A'+B+C)(A'+B+C')(A'+B'+C')$$

### 49. Find Complement Quickly

Given:

$$F = \Sigma m(0,2,3,6)$$

Then:

$$F' = \Pi M(0,2,3,6)$$

Also, because  $F'=1$  where  $F=0$ :

$$F' = \Sigma m(1,4,5,7)$$

Therefore:

$$F' = \Pi M(0,2,3,6) = \Sigma m(1,4,5,7)$$

### 50. Key Relationship Table

| Representation       | Indices indicate   |
|----------------------|--------------------|
| $\Sigma m$           | Rows where $(F=1)$ |
| $\Pi M$              | Rows where $(F=0)$ |
| $\Sigma m$ of $(F')$ | Rows where $(F=0)$ |
| $\Pi M$ of $(F')$    | Rows where $(F=1)$ |

## 51. Minterm vs Maxterm Master Table

| Feature               | Minterm            | Maxterm            |
|-----------------------|--------------------|--------------------|
| Operation             | AND                | OR                 |
| Canonical form        | SOP                | POS                |
| Becomes special value | 1                  | 0                  |
| Bit 0 maps to         | complemented       | uncomplemented     |
| Bit 1 maps to         | uncomplemented     | complemented       |
| Notation              | $\backslash(m\_i)$ | $\backslash(M\_i)$ |

## 52. L0 Practice

Find  $\backslash(m\_6)$  for  $\backslash(A,B,C)$ .

$\backslash[6=110\_2]$

Thus:

$\backslash[A=1,\ B=1,\ C=0]$

So:

$\backslash[\boxed{m\_6=ABC}]$

## 53. L0 Practice

Find  $\backslash(M\_6)$ .

Bits:

$\backslash[110]$

Maxterm rule:

1 !' complement 1 !' complement 0 !' normal

Therefore:

$\backslash[\boxed{M\_6=A'+B'+C}]$

## 54. L1 Practice

Given:

$\backslash[F=\Sigma m(0,1,5,7)]$

for 3 variables.

Find canonical POS.

Remaining indices:



$$\{2,3,4,6\}$$

Therefore:

$$\boxed{F = \Pi M(2,3,4,6)}$$

## 55. L1 Practice

Given:

$$F = \Pi M(1,2,6)$$

Find  $(F')$  in canonical SOP.

Because:

$$F' = \Sigma m(1,2,6)$$

Answer:

$$\boxed{F' = \Sigma m(1,2,6)}$$

## 56. L2 — GATE-Level Pattern

Given:

$$F(A,B,C) = A + B'C$$

find canonical minterm form.

As derived:

A produces:

$$\{4,5,6,7\}$$

$(B'C)$  produces:

$$\{1,5\}$$

Union:

$$\{1,4,5,6,7\}$$

Therefore:

$$\boxed{F = \Sigma m(1,4,5,6,7)}$$

## 57. L2 — Determine Expression from Index

Which is  $(M_{11})$  for variables  $(A,B,C,D)$ ?

Convert:

$$11 = 1011_2$$

Maxterm uses opposite convention:

1 !' complement 0 !' normal 1 !' complement 1 !' complement

Therefore:

$$\boxed{A+B+C+D}$$

---

## 58. L2 — Function Count

---

How many Boolean functions of 5 variables exist?

$$2^{2^5}$$

$$=2^{32}$$

Answer:

$$\boxed{2^{32}}$$

---

## 59. L3 — Trap Question

---

For variables  $(A,B,C)$ ,

$$A'BC$$

is which minterm?

Bits:

$$011$$

$$=3$$

Therefore:

$$\boxed{m_3}$$

A common mistake is to interpret the prime as binary 1. That is wrong for minterms.

---

## 60. L3 — Maxterm Trap

---

For variables  $(A,B,C)$ :

$$A+B'+C'$$

Which maxterm?

Literal mapping:

- A uncomplemented !' bit 0
- B' !' bit 1
- C' !' bit 1

So:

$$011_2=3$$

Therefore:

$$\boxed{M_3}$$

---

## 61. L3 — Complement Trap

---

Suppose:

$$F = \sum m(1,4,6)$$

Then which is correct?

A.

$$F' = \sum m(1,4,6)$$

B.

$$F' = \prod M(1,4,6)$$

C.

$$F = \prod M(1,4,6)$$

Correct:

$$\boxed{B}$$

Because the same indexed rows become zeros in  $F'$ .

---

## 62. Normal Method vs GATE Method

Suppose:

$$F = A'B + BC$$

and you need canonical minterms.

### Normal Method

Expand missing variables using:

$$C + C' = 1$$

and:

$$A + A' = 1$$

Then simplify duplicates.

### GATE Method

Find satisfying assignments.

$$(A'B)$$

$A=0$ ,  $B=1$ ,  $C$  free:

$$010, 011$$

indices:

$$2, 3$$

$$(BC)$$

$B=1$ ,  $C=1$ ,  $A$  free:

$$011, 111$$

indices:

$\{3, 7\}$

Union:

$\{2, 3, 7\}$

Therefore:

$\boxed{F = \sum m(2, 3, 7)}$

Much faster.

---

## 63. Common GATE Traps

---

### Trap 1 — Mixing minterm and maxterm rules

Minterm:

$\rightarrow \text{complement}$

Maxterm:

$\rightarrow \text{normal}$

---

### Trap 2 — $\sum m$ lists zeros

Wrong.

$\boxed{\sum m \text{ lists 1s}}$

---

### Trap 3 — $\prod M$ lists ones

Wrong.

$\boxed{\prod M \text{ lists 0s}}$

---

### Trap 4 — Ignoring variable order

For:

$F(A, B, C, D)$

index bits correspond exactly to:

$A, B, C, D$

unless another order is explicitly specified.

---

### Trap 5 — Calling any product term a minterm

Example:

$AB$

is not a minterm of a 3-variable function  $(A, B, C)$ , because C is missing.

---

### Trap 6 — Calling any sum term a maxterm

Similarly:

$$\nabla [A+B]$$

is not a maxterm of a 3-variable function.

### Trap 7 — Forgetting duplicate minterms during expansion

Canonical expansion may produce the same minterm from multiple terms.

Count it only once.

## 64. PYQ Connection

Common GATE patterns include:

- convert Boolean expression into  $\sum m$
- convert  $\sum m \rightarrow \prod M$
- identify a given minterm/maxterm number
- derive canonical SOP/POS from a truth table
- find complement  $(F')$
- determine equivalent expressions
- identify ON-set/OFF-set
- use minterm indices as input to K-map problems
- implement functions using decoders or multiplexers

This chapter is foundational for K-map questions.

## 65. Chapter 5 Formula Sheet

For  $n$  variables:

$$\boxed{\# \text{truth-table rows} = 2^n}$$

$$\boxed{\# \text{minterms} = 2^n}$$

$$\boxed{\# \text{maxterms} = 2^n}$$

$$\boxed{\# \text{Boolean functions} = 2^{2^n}}$$

Canonical SOP:

$$\boxed{F = \sum m(\text{rows where } F=1)}$$

Canonical POS:

$$\boxed{F = \prod M(\text{rows where } F=0)}$$

Complement:

$$\boxed{F = \sum m(S) \rightarrow F' = \prod M(S)}$$

$$\boxed{F = \prod M(S) \rightarrow F' = \sum m(S)}$$

Same index:

$$\boxed{m_i = (M_i)'}$$

---

## 66. 30-Second Revision

---

Remember:

### Minterm

- AND term
- contains every variable
- equals 1 for exactly one row
- bit 0 !' complemented
- bit 1 !' normal

### Maxterm

- OR term
- contains every variable
- equals 0 for exactly one row
- bit 0 !' normal
- bit 1 !' complemented

And:

$\boxed{\Sigma m = \text{1 rows}}$

$\boxed{\Pi M = \text{0 rows}}$

---

## What You Absolutely Must Remember

---

1. Literal = variable or complement.
2. SOP = OR of products.
3. POS = AND of sums.
4. Canonical form means every term contains all variables.
5. Minterms correspond to rows where output is 1.
6. Maxterms correspond to rows where output is 0.
7. Minterm bit mapping: 0 !' prime, 1 !' normal.
8. Maxterm mapping is opposite.
9.  $(\Sigma m \rightarrow \Pi M)$  conversion uses missing indices.
10.  $m_i = (M_i)'$ .
11.  $n$  variables give  $2^n$  minterms/maxterms.
12. Number of Boolean functions:

$\boxed{2^{2^n}}$

---

## Digital Logic Progress

---

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' Boolean Functions/SOP/POS '

**Current:** Chapter 6 — Universal Gates & Logic Realization

**Remaining:** Universal Gates !' Algebraic Minimization !' K-map !' Tabular Method !' Combinational Circuits !'

## NEXT

### Chapter 6 — NAND/NOR Universal Gates & Logic Realization Ø=ßà

We will cover:

why NAND/NOR are universal !' NOT/AND/OR using NAND !' NOT/AND/OR using NOR !' NAND-only and NOR-only realizations !' two-level NAND-NAND/NOR-NOR circuits !' De Morgan transformation !' XOR/XNOR realization !' gate-count traps.

USER

next

ASSISTANT

## CHAPTER 6 — UNIVERSAL GATES & LOGIC REALIZATION Ø=ßà

This chapter teaches how to implement Boolean functions using only:

- NAND gates
- NOR gates

This is important because:

$\boxed{\text{NAND and NOR are universal gates}}$

Meaning:

**Any Boolean function can be implemented using only NAND gates, or only NOR gates.**

### 1. Why Are NAND and NOR Called Universal?

A gate is called **universal** if we can construct the three basic operations:

- NOT
- AND
- OR

using only that gate.

Since every Boolean function can be built from AND, OR, and NOT, any gate capable of generating all three is universal.

Therefore:

$\boxed{\text{NAND is universal}}$

and

$\boxed{\text{NOR is universal}}$

## 2. NAND Gate

Recall:

$$Y = (AB)'$$

Truth table:

| A | B | NAND |
|---|---|------|
| 0 | 0 | 1    |
| 0 | 1 | 1    |
| 1 | 0 | 1    |
| 1 | 1 | 0    |

## 3. NOT Using NAND

Connect both NAND inputs together:

$$Y = (AA)'$$

Since:

$$AA = A$$

therefore:

$$\boxed{Y = A'}$$

ASCII idea:

TEXT

```
A % % , % % \
    %   N A N D % %   A '
A % % %4% % /
```

So one NAND gate can act as NOT.

## 4. AND Using NAND

First NAND gives:

$$(AB)'$$

We need:

$$AB$$

So invert it using another NAND configured as NOT.

$$Y = [(AB)'(AB)']'$$

Therefore:

$$\boxed{Y = AB}$$



Requires:

$\boxed{2 \text{ NAND gates}}$

## 5. OR Using NAND

We want:

$A+B$

Using De Morgan:

$A+B=(A'B)'$

So:

1. Generate  $A'$  using NAND.
2. Generate  $B'$  using NAND.
3. NAND them.

Thus:

$\boxed{A+B=(A'B)'} \quad \square$

Requires:

$\boxed{3 \text{ NAND gates}}$

## 6. NAND Summary

| Required function | NAND implementation     |
|-------------------|-------------------------|
| NOT A             | $A \text{ NAND } A$     |
| AND               | NAND then invert        |
| OR                | invert inputs then NAND |

Memory trick:

**NAND naturally loves SOP.**

You will see why shortly.

## 7. NOR Gate

Recall:

$Y=(A+B)'$

Truth table:

| A | B | NOR |
|---|---|-----|
| 0 | 0 | 1   |

| A | B | NOR |
|---|---|-----|
| 0 | 1 | 0   |
| 1 | 0 | 0   |
| 1 | 1 | 0   |

## 8. NOT Using NOR

Connect both inputs together:

$$Y = (A + A)'$$

Since:

$$A + A = A$$

therefore:

$$\boxed{Y = A'}$$

One NOR gate acts as NOT.

## 9. OR Using NOR

First NOR gives:

$$(A + B)'$$

Invert the result using another NOR.

Therefore:

$$\boxed{Y = A + B}$$

Requires:

$$\boxed{2 \text{ NOR gates}}$$

## 10. AND Using NOR

Using De Morgan:

$$AB = (A' + B')'$$

So:

1. Generate  $(A')$  with NOR.
2. Generate  $(B')$  with NOR.
3. NOR them.

Thus:

$$\boxed{AB = (A' + B')}$$

Requires:

$$\boxed{3 \text{ NOR gates}}$$

## 11. NOR Summary

| Required function | NOR implementation             |
|-------------------|--------------------------------|
| NOT A             | $\overline{A}$ (text{ NOR } A) |
| OR                | NOR then invert                |
| AND               | invert inputs then NOR         |

Memory trick:

**NOR naturally loves POS.**

## 12. NAND–NAND Realization Ø=Ý4

Suppose Boolean function is in SOP form:

$$F = AB + CD$$

We can rewrite using double complement:

$$F = \overline{\overline{AB + CD}}$$

Apply De Morgan inside:

$$F = \overline{(AB)'(CD)'}$$

This directly maps to NAND gates.

First level:

$$X = (AB)'$$

$$Y = (CD)'$$

Second level:

$$F = (XY)'$$

Therefore:

$$\boxed{\text{SOP} \rightarrow \text{NAND-NAND realization}}$$

## 13. Example: NAND–NAND

Implement:

$$F = A'B + BC$$

using NAND only.

First produce product complements:

$$X = (A'B)'$$

$$Y = (BC)'$$

Then:

$$F = (XY)'$$

But we still require  $(A')$ .

Generate:

$$A' = (AA)'$$

So total implementation includes the inverter NAND plus two first-level NANDs and one final NAND.

---

## 14. Two-Level NAND Concept

---

A pure two-level NAND-NAND network corresponds naturally to an SOP expression.

Example:

$$F = P_1 + P_2 + P_3$$

where each  $(P_i)$  is a product term.

Then:

$$F = (P_1' P_2' P_3)'$$

That is exactly:

- Level 1: NAND each product
  - Level 2: NAND their outputs
- 

## 15. Why SOP !' NAND–NAND?

---

Because De Morgan converts:

$$F = P_1 + P_2 + \dots + P_n$$

into:

$$F = \boxed{(P_1' P_2' \dots P_n)'}'$$

which is a NAND structure.

---

## 16. NOR–NOR Realization Ø=Ý4

---

Suppose:

$$F = (A+B)(C+D)$$

This is POS.

Take double complement:

$$F = [((A+B)(C+D))']'$$

Apply De Morgan:

$$F = [(A+B)' + (C+D)']'$$

First level:

$$X = (A+B)'$$

---

$$\lceil Y=(C+D)' \rceil$$

Second level:

$$\lceil F=(X+Y)' \rceil$$

This is NOR.

Therefore:

$$\lceil \boxed{\text{POS}} \rightarrow \text{NOR-NOR realization} \rceil$$

## 17. Core Mapping Rule

Memorize:

$$\lceil \boxed{\text{SOP}} \rightarrow \text{NAND-NAND} \rceil$$

$$\lceil \boxed{\text{POS}} \rightarrow \text{NOR-NOR} \rceil$$

This is extremely useful in GATE.

## 18. Bubble Pushing Concept

A small inversion bubble on a logic-gate input/output indicates NOT.

Bubble pushing uses De Morgan's law to transform circuits.

Core principle:

**Moving a bubble through a gate changes AND to OR.**

For example:

$$\lceil (AB)'=A'+B' \rceil$$

So:

- NAND can be viewed as OR with bubbled inputs
- NOR can be viewed as AND with bubbled inputs

This is useful for visually simplifying gate networks.

## 19. NAND as OR With Bubbled Inputs

From:

$$\lceil (AB)'=A'+B' \rceil$$

a NAND gate can logically be viewed as:

$$\lceil A'+B' \rceil$$

Meaning:

**NAND = OR gate whose inputs are complemented.**

## 20. NOR as AND With Bubbled Inputs

From:

$$\mathcal{V} \left[ (A+B)' = A'B' \right]$$

Thus NOR can be viewed as:

**AND gate whose inputs are complemented.**

This is often useful when tracing a mixed logic diagram.

## 21. Double Negation Trick

If you want to introduce NAND/NOR structure:

$$\mathcal{V} \left[ F = (F')' \right]$$

This does not change the function.

Example:

$$\mathcal{V} \left[ F = AB + CD \right]$$

Insert double negation:

$$\mathcal{V} \left[ F = [(AB + CD)']' \right]$$

Then apply De Morgan.

This is the standard conversion technique.

## 22. NAND-Only Example

Implement:

$$\mathcal{V} \left[ F = AB + C \right]$$

using NAND.

We want SOP:

$$\mathcal{V} \left[ AB + C \right]$$

The term  $\mathcal{V}(C)$  can be viewed as a one-literal product.

Use:

$$\mathcal{V} \left[ F = [(AB)'C']' \right]$$

Generate:

$$\mathcal{V} \left[ X = (AB)' \right]$$

$$\mathcal{V} \left[ C' = (CC)' \right]$$

Final:

$$\mathcal{V} \left[ F = (XC')' \right]$$

So NAND only.

---

## 23. NOR-Only Example

---

Implement:

$$F = (A+B)(C+D)'$$

using NOR only.

First produce:

$$X = (A+B)'$$

Need  $(D)'$ :

$$D' = (D+D)'$$

Then:

$$Y = (C+D')'$$

Final:

$$F = (X+Y)'$$

This is NOR-only.

---

## 24. Gate Count Matters

---

GATE may ask:

**What is the minimum number of NAND gates required?**

You must count carefully.

Example:

Implement:

$$A'$$

using NAND.

Answer:

$$\boxed{1}$$

Implement:

$$AB$$

using NAND:

$$\boxed{2}$$

Implement:

$$A+B$$

using NAND:

$$\boxed{3}$$

---

## 25. But Multi-Input Gates Matter

If the question allows only **2-input NAND gates**, implementing a 3-input NAND requires extra gates.

If it says simply "NAND gates" and diagram shows 3-input NAND allowed, gate count may differ.

Always check:

$\boxed{\text{gate fan-in constraint}}$

## 26. Fan-In

Fan-in = number of inputs a gate accepts.

Examples:

- 2-input NAND !' fan-in 2
- 3-input NAND !' fan-in 3

This matters in implementation questions.

## 27. Multi-Level Realization

Suppose:

$F = (AB + C)(D + E)$

This is not directly a simple two-level SOP.

You may:

1. algebraically transform it
2. convert to NAND/NOR structure
3. possibly use multiple logic levels

GATE can ask for:

- number of gates
- number of logic levels
- propagation delay

## 28. Logic Level

A logic level is one stage of gates from input to output.

Example:

TEXT

A , B !' N A N D !' N A N D !' O u t p u t

has:

$\boxed{2 \text{ logic levels}}$

Ignoring input inverters depending on the convention/question.

Always read the wording carefully.



---

## 29. Why Fewer Logic Levels Matter

Each gate has propagation delay.

If each level contributes delay  $(t_p)$ :

$$\boxed{\text{approximate total delay} = (\text{number of levels})t_p}$$

This is a simplified model but frequently sufficient in Digital Logic questions.

---

## 30. XOR Using Basic Gates

Recall:

$$A \oplus B = A'B + AB'$$

So XOR can be implemented with:

- two NOTs
- two ANDs
- one OR

Naively:

$$\boxed{5 \text{ gates}}$$

But with NAND gates, we can do better.

---

## 31. XOR Using Four NAND Gates

A standard 2-input NAND realization uses four NANDs.

Let:

$$P = (AB)'$$

Then:

$$Q = (AP)'$$

$$R = (BP)'$$

Finally:

$$F = (QR)'$$

Then:

$$\boxed{F = A \oplus B}$$

Total:

$$\boxed{4 \text{ NAND gates}}$$

This is a useful standard result.

---

## 32. Verify the 4-NAND XOR

Let:

$$[ P = (AB)' ]$$

Then:

$$[ Q = (A(AB)')' ]$$

$$[ R = (B(AB)')' ]$$

Final NAND yields XOR.

You do not need to rederive this every time if the standard circuit is given, but understand its behavior.

---

### 33. XNOR From XOR

---

Since:

$$[ A \odot B = (A \oplus B)' ]$$

we can implement XNOR by:

1. generating XOR
2. inverting it

If only NAND is allowed, use one extra NAND as inverter.

Thus standard construction may use:

$$[ \boxed{5 \text{ NAND gates}} ]$$

with the straightforward method.

---

### 34. NAND-Based NOT Shortcut

---

Whenever you need complement:

$$[ A' = (AA)' ]$$

So if the signal is already available in complemented form from a previous NAND output, **do not create another inverter unnecessarily**.

This is crucial for minimum-gate questions.

---

### 35. Example — Reuse Intermediate Complement

---

Suppose first gate outputs:

$$[ X = (AB)' ]$$

If later you need  $((AB)')$ , use X directly.

Do not invert to  $(AB)$  and then invert again.

GATE gate-count traps often exploit this.

---

### 36. NOR-Based XNOR

---

XNOR is naturally:

$$\lceil AB + A'B' \rceil$$

But NOR realizations can exploit POS form:

$$\lceil A \odot B = (A+B')(A'+B) \rceil$$

This may map nicely to NOR-NOR after required complements are available.

---

## 37. NAND/NOR Duality

---

Notice:

**NAND**

$$\lceil (AB)' \rceil$$

**NOR**

$$\lceil (A+B)' \rceil$$

They are duals under:

$$\lceil \cdot \leftrightsquigarrow + \rceil$$

This reflects Boolean duality.

---

## 38. Convert AND–OR to NAND–NAND

---

Suppose circuit computes:

$$\lceil F = AB + CD + EF \rceil$$

AND gates create:

$$\lceil AB, \ CD, \ EF \rceil$$

OR combines them.

Equivalent NAND-NAND:

$$\lceil \boxed{F = [(AB)'(CD)'(EF)']'} \rceil$$

So:

- replace first-level ANDs with NANDs
  - replace final OR with NAND
  - provided the inversion structure matches
- 

## 39. Convert OR–AND to NOR–NOR

---

Suppose:

$$\lceil F = (A+B)(C+D)(E+F) \rceil$$

Equivalent:

$$\lceil \boxed{F = [(A+B)' + (C+D)' + (E+F)']'} \rceil$$

Thus:

- first-level OR !' NOR
- second-level AND !' NOR

---

## 40. NAND-NAND Example

---

Implement:

$$F = A'BC + AB'C'$$

using NAND-NAND.

First:

$$X = (A'BC)'$$

$$Y = (AB'C')'$$

Final:

$$F = (XY)'$$

Need complements:

$$A', B', C'$$

if not already available.

So if only true inputs A,B,C are supplied, extra NAND inverters are needed.

---

## 41. GATE Trap — Complemented Inputs May Be Given

---

A question may explicitly say:

**Both  $A$  and  $A'$  are available.**

Then you **must not** count an inverter for  $A'$ .

This can change the minimum gate count.

---

## 42. GATE Trap — Constant Inputs

---

Sometimes tying one input to 0 or 1 simplifies a gate.

For NAND:

$$A \text{ NAND } 1 = (A \cdot 1)' = A'$$

So NAND with one input fixed at 1 also gives NOT.

For NOR:

$$A \text{ NOR } 0 = (A + 0)' = A'$$

So NOR with one input fixed at 0 gives NOT.

### 43. NAND With Constants

---

$$\text{NAND } 0 = (A \cdot 0)' = 1$$

$$\text{NAND } 1 = A'$$

Useful for conceptual questions.

---

### 44. NOR With Constants

---

$$\text{NOR } 1 = (A + 1)' = 0$$

$$\text{NOR } 0 = A'$$

---

### 45. NAND Used as Buffer

---

Two NAND inverters in series:

$$A \rightarrow A' \rightarrow A$$

Thus:

$$\boxed{\text{2 NAND gates can create a buffer}}$$

Same for NOR.

---

### 46. NAND-NAND Output Form

---

Suppose:

$$F = (X'Y)'$$

By De Morgan:

$$F = X + Y$$

This is why final NAND effectively performs OR on first-level product terms.

---

### 47. NOR-NOR Output Form

---

Suppose:

$$F = (X' + Y')'$$

Wait carefully: if first level produces complemented sums:

$$X = (A + B)'$$

$$Y = (C + D)'$$

Final NOR:

$$F = (X + Y)'$$

Then:

$$F = ((A + B)' + (C + D)')'$$

By De Morgan:

---

$$F = (A+B)(C+D)$$

Correct.

---

## 48. GATE-Style L0

How many NAND gates are required to implement NOT A?

$$A' = (AA)'$$

Answer:

$$\boxed{1}$$

---

## 49. L0

How many 2-input NAND gates are required for:

$$AB$$

One NAND gives:

$$(AB)'$$

Second NAND inverts:

$$AB$$

Answer:

$$\boxed{2}$$

---

## 50. L1

Using only NAND gates, implement:

$$A+B$$

Use:

$$A+B = (A'B')'$$

Need:

- NAND inverter for A
- NAND inverter for B
- final NAND

Answer:

$$\boxed{3 \text{ NAND gates}}$$

---

## 51. L1

Which universal gate naturally implements SOP in two levels?

Answer:

$$\boxed{\text{NAND}}$$

Specifically:

$$\boxed{\text{NAND-NAND}}$$

---

## 52. L1

---

Which universal gate naturally implements POS?

$$\boxed{\text{NOR-NOR}}$$

---

## 53. L2 — Identify Function

---

Suppose:

$$X = (AB)'$$

$$Y = (CD)'$$

$$F = (XY)'$$

Find F.

Substitute:

$$F = [(AB)'(CD)']'$$

Using De Morgan:

$$\boxed{F = AB + CD}$$

---

## 54. L2 — NOR Network

---

Suppose:

$$X = (A+B)'$$

$$Y = (C+D)'$$

$$F = (X+Y)'$$

Then:

$$F = [(A+B)' + (C+D)']'$$

By De Morgan:

$$\boxed{F = (A+B)(C+D)}$$

---

## 55. L2 — Constants

---

Find:

$$A \text{ NAND } A$$

$$(AA)' = A'$$

Answer:

---

$$\boxed{A'}$$

Find:

$$A \text{ NOR } A$$

$$(A+A)' = A'$$

Again:

$$\boxed{A'}$$

## 56. L3 — Gate Count Trap

Suppose both  $A$  and  $A'$  are available.

Implement:

$$F = A'B + AC$$

using NAND-NAND.

Need:

$$X = (A'B)'$$

$$Y = (AC)'$$

Final:

$$F = (XY)'$$

No inverter needed.

So with multi-input/2-input NAND appropriate:

$$\boxed{3 \text{ NAND gates}}$$

If  $A'$  were unavailable, one more NAND would be needed.

## 57. L3 — Identify Mixed Bubble Logic

Expression:

$$F = [(A'B)']'$$

Inner:

$$(A'B)' = A+B$$

Outer complement:

$$F = (A+B)'$$

Or directly:

$$[(A'B)']' = A'B'$$

Careful! Let's simplify correctly.

Double complement:

$$[(A'B)']' = A'B'$$

So:



$$\boxed{F=A'B}$$

This illustrates why nested bars must be tracked carefully.

---

## 58. NAND/NOR Implementation Strategy

---

Whenever GATE asks for universal-gate realization:

### Step 1

Simplify Boolean function first.

### Step 2

Choose appropriate form:

- NAND !' SOP
- NOR !' POS

### Step 3

Apply double negation.

### Step 4

Use De Morgan.

### Step 5

Count required complements.

### Step 6

Reuse already available inverted signals.

### Step 7

Check gate fan-in.

This is the safest procedure.

---

## 59. Normal Method vs GATE Method

---

Suppose:

$$F=AB+AC$$

### Normal method

---

Direct NAND-NAND:

$$F=[(AB)'(AC)']'$$

3 NAND gates.

### Better algebraic method

---

Factor:

$$F=A(B+C)$$

But NAND-only implementation may or may not require fewer gates depending on available gates/fan-in.

Therefore:

**Boolean simplification does not always automatically mean minimum universal-gate count.**

You must consider the target gate technology.

This is a subtle but important idea.

---

## 60. Gate Count vs Literal Count

Two different optimization goals:

### Boolean minimization

Reduce:

- literals
- product terms

### Gate minimization

Reduce actual gates under constraints.

These are related but not always identical.

GATE may exploit this distinction.

---

## 61. Two-Level vs Multi-Level Networks

Example:

$$F = AB + AC + AD$$

Two-level SOP:

$$F = AB + AC + AD$$

requires three product computations plus OR.

But factoring:

$$F = A(B + C + D)$$

can reduce gates, while increasing/differing structure.

Therefore:

$$\boxed{\text{minimum SOP}} \neq \text{always minimum total gate network}$$

---

## 62. Propagation Delay Example

Suppose each NAND has:

$$t_p = 2 \text{ ns}$$

A signal passes through 3 sequential NAND levels.

Approximate worst-case delay:

$3 \times 6 \text{ ns}$

$6 \text{ ns}$

Later we will do timing more rigorously.

---

## 63. Common GATE PYQ Patterns

---

Expect questions such as:

- prove/identify NAND or NOR universality
  - realize a function using only NAND/NOR
  - identify function produced by NAND/NOR network
  - count gates
  - count logic levels
  - convert AND-OR !" NAND-NAND
  - convert OR-AND !" NOR-NOR
  - identify bubble-equivalent circuit
  - exploit complemented inputs already available
  - analyze propagation delay
  - recognize XOR/XNOR implementation
- 

## 64. GATE TRAP BOOK — Chapter 6

---

### Trap 1 — NAND naturally implements POS

Wrong.

$\boxed{\text{NAND-NAND !' SOP}}$

---

### Trap 2 — NOR naturally implements SOP

Wrong.

$\boxed{\text{NOR-NOR !' POS}}$

---

### Trap 3 — Forgetting inverter gates

If  $\neg A$  is required but unavailable, it costs a gate.

---

### Trap 4 — Counting an inverter when complemented input is already supplied

Do not count it twice.

---

### Trap 5 — Ignoring fan-in

3-input NAND allowed vs only 2-input NAND can change answer.

---

### Trap 6 — Misapplying De Morgan

When complement crosses a gate:

$$\boxed{\text{AND} \rightarrow \text{OR}}$$

---

### Trap 7 — Assuming minimum Boolean expression means minimum gates

Not always.

---

### Trap 8 — Counting levels incorrectly

Parallel gates are on the same level.

Only sequential gate stages increase depth.

---

## 65. Formula / Shortcut Sheet

---

### NAND

$$\boxed{A \uparrow B = (AB)'} \quad \downarrow$$

### NOR

$$\boxed{A \downarrow B = (A+B)'} \quad \downarrow$$

### NAND NOT

$$\boxed{A' = A \text{ NAND } A} \quad \downarrow$$

### NOR NOT

$$\boxed{A' = A \text{ NOR } A} \quad \downarrow$$

### NAND-NAND

$$\boxed{\text{Canonical/simple SOP realization}} \quad \downarrow$$

### NOR-NOR

$$\boxed{\text{Canonical/simple POS realization}} \quad \downarrow$$

### XOR

$$\boxed{A \oplus B = A'B + AB'} \quad \downarrow$$

### XNOR

$$\boxed{A \odot B = AB + A'B'} \quad \downarrow$$

---

## 66. Universal-Gate Memory Trick

---

Remember this sentence:

**NAND likes products first, NOR likes sums first.**

Therefore:

$\boxed{\text{SOP} \rightarrow \text{NAND-NAND}}$

$\boxed{\text{POS} \rightarrow \text{NOR-NOR}}$

---

## 67. 30-Second Revision

You must remember:

1. NAND and NOR are universal.
  2. Tie inputs together to NOT.
  3. NAND-NAND realizes SOP.
  4. NOR-NOR realizes POS.
  5. Use double complement + De Morgan.
  6. Complemented inputs may require extra gates.
  7. Fan-in changes gate count.
  8. Gate minimization and algebraic minimization are not always identical.
  9. XOR can be made with 4 standard 2-input NAND gates.
  10. Bubble movement means AND to OR.
- 

## What You Absolutely Must Remember

$\boxed{\text{NAND is universal}}$

$\boxed{\text{NOR is universal}}$

$\boxed{A \text{ NAND } A = A'}$

$\boxed{A \text{ NOR } A = A'}$

$\boxed{\text{SOP} \rightarrow \text{NAND-NAND}}$

$\boxed{\text{POS} \rightarrow \text{NOR-NOR}}$

And always simplify the Boolean function **before** counting gates.

## Digital Logic Progress

**Completed:** Number Systems ' ' Signed Representation ' ' Binary Arithmetic ' ' Boolean Algebra ' ' SOP/POS ' ' Universal Gates '

**Current:** Chapter 7 — Boolean Minimization Using Algebraic Technique

**Remaining:** Algebraic Minimization ' K-map ' Tabular Method ' Combinational Circuits ' Sequential Circuits ' Fixed Point ' Floating Point

## NEXT

Chapter 7 — Algebraic Minimization ~~0-1~~ **MUST MASTER**

Next we cover:

**systematic Boolean simplification ' factoring ' absorption ' consensus ' redundant terms ' SOP/POS minimization ' literal-cost vs gate-cost ' difficult GATE-style algebraic simplification problems.**

---

USER

next

ASSISTANT

## CHAPTER 7 — BOOLEAN MINIMIZATION USING ALGEBRAIC TECHNIQUE Ø=Ý4 MUST MASTER

The goal of minimization is:

**Represent the same Boolean function using a simpler expression.**

Why simplify?

- fewer literals
- fewer gates
- lower hardware cost
- lower power
- potentially lower propagation delay
- easier circuit implementation

For GATE, you must be able to simplify expressions **quickly without blindly expanding everything**.

### 1. What Does "Minimum" Mean?

Consider:

$$F = AB + AB'$$

Factor:

$$F = A(B + B')$$

Since:

$$B + B' = 1$$

$$F = A$$

So:

$$F = AB + AB'$$

and:

$$F = A$$

represent exactly the same function.

But  $F = A$  is much simpler.

### 2. Simplification vs Changing the Function

You can simplify only using valid Boolean identities.

You must preserve:

$\boxed{\text{same output for every possible input}}$

So if two expressions are equivalent:

$$F_1 = F_2$$

their truth tables must match completely.

---

### 3. Most Important Identities for Algebraic Minimization

---

These should become automatic.

#### Identity

$$A + 0 = A$$

$$A \cdot 1 = A$$

#### Domination

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

#### Idempotent

$$A + A = A$$

$$A A = A$$

#### Complement

$$A + A' = 1$$

$$A A' = 0$$

#### Absorption

$$\boxed{A + AB = A}$$

$$\boxed{A(A + B) = A}$$

#### Useful reduction

$$\boxed{A + A'B = A + B}$$

$$\boxed{A(A' + B) = AB}$$

#### Consensus

$$\boxed{AB + A'C + BC = AB + A'C}$$

#### De Morgan

$$(AB)' = A' + B'$$

$$(A + B)' = A'B'$$

## 4. The Best Simplification Strategy

---

Do **not** immediately expand everything.

Use this order:

### Step 1 — Remove obvious complements

Look for:

$$\lceil X+X'=1 \rceil$$

or:

$$\lceil XX'=0 \rceil$$

### Step 2 — Look for absorption

$$\lceil X+XY=X \rceil$$

### Step 3 — Look for common factors

### Step 4 — Look for patterns

$$\lceil XY+XY'=X \rceil$$

$$\lceil X+X'Y=X+Y \rceil$$

### Step 5 — Look for consensus terms

### Step 6 — Apply De Morgan only where useful

### Step 7 — Expand only if simplification requires it

This is much faster than random algebra.

---

## 5. Basic Factoring

---

Simplify:

$$\lceil AB+AC \rceil$$

Factor  $\lceil(A)\rceil$ :

$$\lceil A(B+C) \rceil$$

So:

$$\lceil \boxed{AB+AC=A(B+C)} \rceil$$

Which form is "better" depends on the target implementation.

For literal count:

Original:

$$\lceil AB+AC \rceil$$

has 4 literal occurrences.

Factored:

$$\lceil A(B+C) \rceil$$

has 3.



---

## 6. Complement-Pair Factoring

---

Simplify:

$$\lceil AB+AB' \rceil$$

Factor A:

$$\lceil A(B+B') \rceil$$

$$\lceil =A(1) \rceil$$

$$\lceil \boxed{A} \rceil$$

Pattern:

$$\lceil \boxed{XY+XY'=X} \rceil$$

---

## 7. Another Complement Pattern

---

$$\lceil AB+A'B \rceil$$

Factor B:

$$\lceil B(A+A') \rceil$$

$$\lceil =B \rceil$$

Therefore:

$$\lceil \boxed{AB+A'B=B} \rceil$$

---

## 8. Absorption

---

Simplify:

$$\lceil A+AB \rceil$$

Factor:

$$\lceil A(1+B) \rceil$$

Since:

$$\lceil 1+B=1 \rceil$$

$$\lceil \boxed{A} \rceil$$

Interpretation:

If A is already true, the extra condition  $\lceil (AB) \rceil$  adds no new case.

---

## 9. Extended Absorption

---

$$\lceil A+ABC \rceil$$

Use:

$$\lceil A+AX=A \rceil$$

---

where:

$$\boxed{X=BC}$$

So:

$$\boxed{A+ABC=A}$$

Likewise:

$$\boxed{A(A+B+C)=A}$$

---

## 10. Very Important Pattern

---

Simplify:

$$\boxed{A+A'B}$$

Use:

$$\boxed{A+A'B=A+B}$$

Proof:

$$\boxed{A+A'B}$$

Use distributive law:

$$\boxed{(A+A')(A+B)}$$

$$\boxed{=1(A+B)}$$

$$\boxed{A+B}$$

---

## 11. Dual Pattern

---

$$\boxed{A(A'+B)}$$

Expand:

$$\boxed{AA'+AB}$$

$$\boxed{=0+AB}$$

$$\boxed{AB}$$

So:

$$\boxed{A(A'+B)=AB}$$

---

## 12. Example

---

Simplify:

$$\boxed{A+A'BC}$$

Treat:

$$\boxed{X=BC}$$

Then:

---

$$\lceil A+A'X=A+X \rceil$$

Hence:

$$\lceil \boxed{A+BC} \rceil$$

Do **not** incorrectly simplify to:

$$\lceil A+B+C \rceil$$

### 13. Product-of-Sums Pattern

Simplify:

$$\lceil (A+B)(A+B') \rceil$$

Use:

$$\lceil (X+Y)(X+Y')=X \rceil$$

Therefore:

$$\lceil \boxed{A} \rceil$$

### 14. Another POS Pattern

$$\lceil (A+B)(A'+B) \rceil$$

Common B:

$$\lceil \boxed{B} \rceil$$

This is the dual of:

$$\lceil AB+A'B=B \rceil$$

### 15. Redundant Term Removal

Consider:

$$\lceil A+AB+ABC \rceil$$

Start:

$$\lceil A+AB=A \rceil$$

Then:

$$\lceil A+ABC=A \rceil$$

Therefore:

$$\lceil \boxed{A} \rceil$$

Once a broader term exists, more restrictive versions are often redundant.

### 16. Literal Containment Intuition

Compare:

$$\{ AB \}$$

and:

$$\{ ABC \}$$

Whenever:

$$\{ ABC=1 \}$$

we automatically have:

$$\{ AB=1 \}$$

Therefore in SOP:

$$\{ \boxed{AB+ABC=AB} \}$$

because  $\{ABC\}$  contributes no new 1s.

This is absorption.

## 17. Consensus Theorem

Recall:

$$\{ \boxed{AB+A'C+BC=AB+A'C} \}$$

The  $\{BC\}$  term is redundant.

General form:

$$\{ \boxed{XY+X'Z+YZ=XY+X'Z} \}$$

The redundant term  $\{YZ\}$  is called the **consensus term**.

## 18. Consensus Example

Simplify:

$$\{ PQ+P'R+QR \}$$

Recognize:

$$\{ X=P, \quad Y=Q, \quad Z=R \}$$

Therefore:

$$\{ \boxed{PQ+P'R} \}$$

No expansion needed.

## 19. Why Consensus Works

Suppose:

$$\{ QR=1 \}$$

Then:

$$\{ Q=1, \quad R=1 \}$$

Now P is either 0 or 1.

If:

$$\boxed{P=1}$$

then:

$$\boxed{PQ=1}$$

If:

$$\boxed{P=0}$$

then:

$$\boxed{P'R=1}$$

So  $\boxed{QR}$  never creates an output-1 case not already covered.

Hence redundant.

---

## 20. Dual Consensus

---

$$\boxed{(A+B)(A'+C)(B+C) = (A+B)(A'+C)}$$

Useful for POS minimization.

---

## 21. Example — Multiple Simplification Steps

---

Simplify:

$$\boxed{F=AB+A'B+ABC}$$

First:

$$\boxed{AB+A'B=B(A+A')}$$

$$\boxed{=B}$$

Then:

$$\boxed{F=B+ABC}$$

Since:

$$\boxed{ABC=B(AC)}$$

use absorption:

$$\boxed{B+B(AC)=B}$$

Therefore:

$$\boxed{\boxed{F=B}}$$

---

## 22. Example — Factor Before Expanding

---

Simplify:

$$\boxed{AB+AC+A'B+A'C}$$

---

Group:

$$\boxed{A(B+C)+A'(B+C)}$$

Factor:

$$(B+C)(A+A')$$

$$=(B+C)(1)$$

Therefore:

$$\boxed{B+C}$$

This is far better than full expansion.

---

## 23. Another Grouping Example

---

$$ABC+ABC'+AB'C+AB'C'$$

Factor A:

$$A[BC+BC'+B'C+B'C']$$

Group:

$$B(C+C')+B'(C+C')$$

$$=B+B'$$

$$=1$$

Therefore:

$$\boxed{A}$$

This corresponds to covering all combinations of B and C while A=1.

---

## 24. Simplify Using De Morgan

---

Simplify:

$$(A+B)' + A'B$$

De Morgan:

$$A'B'+A'B$$

Factor:

$$A'(B'+B)$$

$$=A'$$

Therefore:

$$\boxed{A'}$$

---

## 25. Nested Complement Example

---

Simplify:

---

$$\neg [(A+B')C]'$$

Apply De Morgan:

$$\neg (A+B')'+C'$$

Then:

$$\neg (A+B')'=A'B$$

Therefore:

$$\boxed{A'B+C'}$$

## 26. Another De Morgan Example

Simplify:

$$\neg [(AB)'(A+B)]'$$

Apply De Morgan to whole expression:

$$\neg [(AB)']'+(A+B)'$$

$$\neg =AB+A'B'$$

Therefore:

$$\boxed{AB+A'B'}$$

Recognize:

$$\boxed{\text{XNOR}}$$

## 27. XOR Recognition

If you see:

$$\neg A'B+AB'$$

immediately identify:

$$\boxed{A\oplus B}$$

Likewise:

$$\neg AB+A'B'$$

is:

$$\boxed{A\odot B}$$

Recognizing these can save time.

## 28. Simplifying XOR Expressions

Example:

$$\neg A\oplus A\oplus B$$

Since:

$$\lceil A \oplus A = 0 \rceil$$

$$\lceil 0 \oplus B = B \rceil$$

Therefore:

$$\lceil \boxed{B} \rceil$$


---

## 29. Complement Through XOR

---

Useful identities:

$$\lceil A \oplus 0 = A \rceil$$

$$\lceil A \oplus 1 = A' \rceil$$

$$\lceil A \oplus A = 0 \rceil$$

$$\lceil A \oplus A' = 1 \rceil$$

So:

$$\lceil (A \oplus B) \oplus B = A \rceil$$

because:

$$\lceil B \oplus B = 0 \rceil$$


---

## 30. POS Simplification

---

Example:

$$\lceil (A+B)(A+B+C) \rceil$$

Use absorption:

$$\lceil X(X+Y) = X \rceil$$

where:

$$\lceil X = A+B \rceil$$

So:

$$\lceil \boxed{A+B} \rceil$$


---

## 31. Another POS Example

---

$$\lceil (A+B)(A+B') \rceil$$

As before:

$$\lceil \boxed{A} \rceil$$


---

## 32. POS With Consensus

---

Simplify:

$$\lceil (A+B)(A'+C)(B+C) \rceil$$


---



Using dual consensus:

$$\boxed{(A+B)(A'+C)}$$

---

### 33. Converting SOP to Factored Form

---

Example:

$$AB+AC$$

SOP:

$$AB+AC$$

Factored:

$$A(B+C)$$

Both equivalent.

Which is better?

- For NAND-NAND SOP may be preferable.
- For fewer total gates factored form might be preferable.
- For K-map minimal SOP/POS usually preferred.

So "simplest" depends on the optimization criterion.

---

### 34. What Is Literal Cost?

---

A **literal occurrence** is one appearance of a variable or its complement.

Example:

$$AB'+ACD$$

Literal count:

$$2+3=5$$

So literal cost:

$$\boxed{5}$$

---

### 35. Term Count

---

For:

$$AB'+ACD$$

there are:

$$\boxed{2 \text{ product terms}}$$

Sometimes the optimization target is:

- minimum number of terms
- then minimum literals

This becomes important in K-map minimization.

---

---

## 36. Gate Cost vs Literal Cost

---

Expression:

$$\lceil AB+AC \rceil$$

SOP literal count:

$$\lceil 4 \rceil$$

Factored:

$$\lceil A(B+C) \rceil$$

literal count:

$$\lceil 3 \rceil$$

But gate count depends on:

- gate types allowed
- fan-in
- complemented inputs available
- number of levels

Therefore:

$$\lceil \boxed{\text{minimum literal expression does not always equal minimum hardware}} \rceil$$

---

## 37. Example — Algebraic Reduction

---

Simplify:

$$\lceil F=A'BC+ABC+AB'C \rceil$$

First two terms:

$$\lceil A'BC+ABC \rceil$$

Factor:

$$\lceil BC(A'+A) \rceil$$

$$\lceil =BC \rceil$$

Now:

$$\lceil F=BC+AB'C \rceil$$

Factor C:

$$\lceil C(B+AB') \rceil$$

Use:

$$\lceil B+AB'=A+B \rceil$$

Therefore:

$$\lceil \boxed{F=C(A+B)} \rceil$$

Equivalent SOP form:

$$\boxed{AC+BC}$$

---

### 38. Example — Alternative Method

---

Take:

$$F=AC+BC$$

Factor:

$$\boxed{C(A+B)}$$

So different forms can be equally minimal depending on the required structure.

---

### 39. Difficult Example 1

---

Simplify:

$$F=AB+A'BC+ABC$$

First:

$$AB+ABC=AB$$

by absorption.

So:

$$F=AB+A'BC$$

Factor B:

$$B(A+A'C)$$

Use:

$$A+A'C=A+C$$

Hence:

$$F=B(A+C)$$

Therefore:

$$\boxed{F=AB+BC=B(A+C)}$$

---

### 40. Difficult Example 2

---

Simplify:

$$F=A'B'+A'B+AB$$

First:

$$A'B'+A'B$$

factor:

$$A'(B'+B)=A'$$

Thus:

---

$$\boxed{F=A'+AB}$$

Use:

$$\boxed{A'+AB=A'+B}$$

Therefore:

$$\boxed{A'+B}$$

## 41. Difficult Example 3

Simplify:

$$\boxed{F=(A+B)(A'+B)(A+C)}$$

First:

$$\boxed{(A+B)(A'+B)=B}$$

So:

$$\boxed{F=B(A+C)}$$

Therefore:

$$\boxed{AB+BC}$$

or keep:

$$\boxed{B(A+C)}$$

## 42. Difficult Example 4

Simplify:

$$\boxed{F=AB+A'C+BC+AC}$$

Consensus tells us:

$$\boxed{AB+A'C+BC}$$

allows removal of  $BC$ :

$$\boxed{AB+A'C}$$

Then still  $AC$  remains:

$$\boxed{F=AB+A'C+AC}$$

Can  $AC$  be removed?

No, not generally.

Test:

$$\boxed{A=1, B=0, C=1}$$

Then:

- $AB=0$
- $A'C=0$
- $AC=1$

So  $(AC)$  is necessary.

Thus:

$$\boxed{AB+A'C+AC}$$

This is a useful warning:

**Do not remove terms merely because they "look similar."**

### 43. Difficult Example 5

Simplify:

$$F=A+B'C+BC$$

Combine:

$$B'C+BC=C(B'+B)$$

$$=C$$

Therefore:

$$\boxed{F=A+C}$$

Very fast once you spot the common factor C.

### 44. Difficult Example 6

Simplify:

$$F=A'B'C+A'BC+AB'C+ABC$$

Factor C:

$$C(A'B'+A'B+AB'+AB)$$

Group:

$$A'(B'+B)+A(B'+B)$$

$$=(A'+A)$$

$$=1$$

Therefore:

$$\boxed{F=C}$$

### 45. Difficult Example 7

Simplify:

$$F=(A+B+C)(A+B+C')$$

Use:

$$(X+C)(X+C')=X$$

where:

$$\boxed{X=A+B}$$

Therefore:

$$\boxed{A+B}$$

This pattern is worth memorizing:

$$\boxed{(X+Y)(X+Y')=X}$$

---

## 46. Difficult Example 8

Simplify:

$$F=XY+X'Z+YZ+XYZ'$$

Consensus removes:

$$YZ$$

from:

$$XY+X'Z+YZ$$

So:

$$F=XY+X'Z+XYZ'$$

Now:

$$XY+XYZ'=XY$$

Therefore:

$$\boxed{XY+X'Z}$$

---

## 47. Avoid Over-Expansion

Suppose:

$$F=(A+B)(A+C)(A+D)$$

Do not immediately expand into many terms.

Use:

$$(A+B)(A+C)=A+BC$$

Then:

$$(A+BC)(A+D)$$

Using:

$$(X+Y)(X+Z)=X+YZ$$

with  $(X=A)$ :

$$=A+BCD$$

Therefore:

$$\boxed{A+BCD}$$

Very efficient.

---

## 48. Important Product Identity

---

$$\boxed{(A+B)(A+C)=A+BC}$$

Dual of:

$$\boxed{A+BC=(A+B)(A+C)}$$

This is highly useful.

---

## 49. Example

---

Simplify:

$$(A+B)(A+C)(A+D)$$

First:

$$(A+B)(A+C)=A+BC$$

Then:

$$(A+BC)(A+D)$$

$$=A+BCD$$

So:

$$\boxed{A+BCD}$$

---

## 50. Verify a Simplification When Unsure

---

Suppose you suspect:

$$F=A+B'C$$

simplifies to:

$$A+B+C$$

Do not trust intuition.

Test:

$$A=0, B=1, C=0$$

Original:

$$0+0(0)=0$$

Proposed:

$$0+1+0=1$$

Not equivalent.

Therefore the simplification is wrong.

**GATE emergency method:**

Find a **counterexample input**.

You do not always need the entire truth table.

---

## 51. Equivalence Testing Shortcut

To prove two Boolean expressions are **not equivalent**, one differing input is enough.

To prove they are equivalent, you need:

- valid Boolean algebra
- or all truth-table rows
- or another rigorous method

This can help in MCQs.

---

## 52. Redundant Literal Example

Simplify:

$$\lceil ABC + ABC' \rceil$$

Factor:

$$\lceil AB(C + C') \rceil$$

$$\lceil = AB \rceil$$

The variable C disappeared because output is independent of C when  $\lceil AB = 1 \rceil$ .

This is exactly the idea K-map grouping will formalize next.

---

## 53. Why Variables Disappear During Minimization

Suppose:

$$\lceil ABC + ABC' \rceil$$

The two terms differ only in C.

Since C may be:

$$\lceil 0 \text{ or } 1 \rceil$$

while  $A=B=1$ , C no longer affects the output.

Therefore:

$$\lceil \boxed{AB} \rceil$$

This becomes the heart of K-map grouping.

---

## 54. Adjacency Preview

Terms:

$$\lceil A'BC \rceil$$

---



and:

$$\sum [ABC]$$

differ in only A.

Therefore:

$$\sum [A'BC + ABC]$$

$$\sum [BC(A' + A)]$$

$$\sum [\boxed{BC}]$$

So two minterms differing in one variable combine by eliminating that variable.

This is exactly what a pair in a K-map does.

---

## 55. Combining Four Minterms

---

Consider:

$$\sum [A'B'C + A'BC + AB'C + ABC]$$

All combinations of A and B appear while C stays fixed.

Therefore:

$$\sum [\boxed{C}]$$

A group of 4 K-map cells eliminates 2 variables.

---

## 56. General Pattern Behind K-Maps

---

If:

$$\sum [2^k]$$

adjacent minterms are combined, then:

$$\sum [k]$$

changing variables can disappear.

Example:

- pair of 2 !' eliminate 1 variable
- group of 4 !' eliminate 2 variables
- group of 8 !' eliminate 3 variables

We study this formally next.

---

## 57. L0 Practice

---

Simplify:

$$\sum [A + A'B]$$

Answer:

$$\sum [\boxed{A+B}]$$

---

---

## 58. L1 Practice

---

Simplify:

$$\lceil AB+ABC+AB'C \rceil$$

First:

$$\lceil AB+ABC=AB \rceil$$

Then:

$$\lceil AB+AB'C \rceil$$

Factor A:

$$\lceil A(B+B'C) \rceil$$

Use:

$$\lceil B+B'C=B+C \rceil$$

Therefore:

$$\lceil \boxed{A(B+C)} \rceil$$

or:

$$\lceil \boxed{AB+AC} \rceil$$

---

## 59. L1 Practice

---

Simplify:

$$\lceil (A+B)(A+B')(A+C) \rceil$$

First:

$$\lceil (A+B)(A+B')=A \rceil$$

Then:

$$\lceil A(A+C)=A \rceil$$

Therefore:

$$\lceil \boxed{A} \rceil$$

---

## 60. L2 — GATE-Level

---

Simplify:

$$\lceil AB+A'BC+ABC' \rceil$$

Start:

$$\lceil AB+ABC'=AB \rceil$$

Thus:

$$\lceil AB+A'BC \rceil$$

---

Factor B:

$$\lceil B(A+A'C) \rceil$$

$$\lceil =B(A+C) \rceil$$

Answer:

$$\lceil \boxed{AB+BC} \rceil$$

---

## 61. L2 — Consensus

---

Simplify:

$$\lceil XY+X'Z+YZ+XYZ \rceil$$

Consensus removes:

$$\lceil YZ \rceil$$

Also:

$$\lceil XY+XYZ=XY \rceil$$

Therefore:

$$\lceil \boxed{XY+X'Z} \rceil$$

---

## 62. L2 — POS

---

Simplify:

$$\lceil (A+B)(A'+B)(B+C) \rceil$$

First:

$$\lceil (A+B)(A'+B)=B \rceil$$

Then:

$$\lceil B(B+C)=B \rceil$$

Therefore:

$$\lceil \boxed{B} \rceil$$

---

## 63. L3 — Trap

---

Is:

$$\lceil AB+A'C+AC \rceil$$

equal to:

$$\lceil AB+A'C \rceil$$

No.

Test:

$$\lceil A=1, \ B=0, \ C=1 \rceil$$

---

Original:

$$\lceil 0+0+1=1 \rceil$$

Reduced:

$$\lceil 0+0=0 \rceil$$

So  $\lceil AC \rceil$  is necessary.

---

## 64. L3 — Find Redundant Term

---

Given:

$$\lceil F=AB+A'C+BC \rceil$$

which term is redundant?

Consensus term:

$$\lceil \boxed{BC} \rceil$$

Therefore:

$$\lceil \boxed{F=AB+A'C} \rceil$$

---

## 65. Normal Method vs GATE Method

---

Simplify:

$$\lceil A'BC+ABC+AB'C \rceil$$

### Normal method

Perform systematic factoring.

### GATE method

First two terms differ only in A:

$$\lceil A'BC+ABC=BC \rceil$$

Then:

$$\lceil BC+AB'C \rceil$$

Factor:

$$\lceil C(B+AB') \rceil$$

Use:

$$\lceil B+AB'=A+B \rceil$$

Therefore:

$$\lceil \boxed{C(A+B)} \rceil$$

The key is recognizing terms differing in exactly one literal.

---

## 66. Algebraic Minimization Limit

---

A major weakness of algebraic minimization:

**There is no guaranteed obvious path to the minimum expression.**

Different students may perform different manipulations.

Example:

A function may be simplified correctly but not minimally.

This motivates:

- Karnaugh Map
- Quine–McCluskey/tabular method

These give more systematic minimization.

---

## 67. When Algebraic Method Is Best

Use algebraic simplification when:

- expression is small
- obvious factors exist
- complement pairs appear
- absorption applies
- consensus appears
- GATE options can be reduced quickly

Use K-map when:

- function is given as minterms/maxterms
- 3–4 variables are involved
- minimum SOP/POS is explicitly requested

Use tabular method when:

- many variables/minterms
- systematic minimization is required

---

## 68. Common PYQ Patterns

GATE often embeds algebraic minimization inside questions involving:

- equivalent Boolean expressions
- NAND/NOR realization
- minimum gate count
- circuit output
- MUX realization
- K-map options
- redundant product terms
- XOR/XNOR recognition
- consensus theorem

- simplified function from a given circuit

Actual wording varies, so do not rely on memorized expressions; learn the identities.

---

## 69. GATE Trap Book — Chapter 7

---

### Trap 1 — Expanding too early

Expansion often makes the expression larger and hides patterns.

---

### Trap 2 — Removing a term because it "looks redundant"

Use a valid theorem or verify with an input.

---

### Trap 3 — Misusing:

$$\nabla [A + A'B = A + B]$$

Do not turn:

$$\nabla [A + BC]$$

into  $\nabla (A + B + C)$ .

---

### Trap 4 — Missing absorption

$$\nabla [A + ABC = A]$$


---

### Trap 5 — Missing complement pair

$$\nabla [AB + AB' = A]$$


---

### Trap 6 — Wrong consensus term

For:

$$\nabla [AB + A'C + BC]$$

the removable term is:

$$\nabla [\boxed{BC}]$$


---

### Trap 7 — Confusing minimum Boolean form and minimum gate implementation

They are related, but not identical.

---

### Trap 8 — Forgetting POS dual identities

$$\nabla [(A+B)(A+B') = A]$$

is just as useful as SOP identities.

---

## 70. High-Speed Pattern Sheet

---

Memorize:

$$\boxed{X + XY = X}$$

$$\boxed{X(X + Y) = X}$$

$$\boxed{XY + XY' = X}$$

$$\boxed{XY + X'Y = Y}$$

$$\boxed{X + X'Y = X + Y}$$

$$\boxed{X(X' + Y) = XY}$$

$$\boxed{(X + Y)(X + Y') = X}$$

$$\boxed{XY + X'Z + YZ = XY + X'Z}$$

$$\boxed{(X + Y)(X + Z) = X + YZ}$$

These patterns are extremely valuable.

---

## 71. 30-Second Revision

When simplifying:

1. Look for complements.
2. Factor common literals.
3. Apply absorption.
4. Remove consensus.
5. Recognize XOR/XNOR.
6. Use De Morgan only when useful.
7. Avoid unnecessary expansion.
8. Verify suspicious simplifications with a counterexample.

The most useful identities:

$$X + XY = X$$

$$XY + XY' = X$$

$$X + X'Y = X + Y$$

$$XY + X'Z + YZ = XY + X'Z$$

---

## What You Absolutely Must Remember

- Algebraic minimization preserves the function.
- Factor before expanding.
- Absorption removes more specific terms.
- Terms differing in one complemented variable can merge.
- Consensus can remove redundant terms.
- SOP and POS both have useful dual simplification rules.
- Boolean minimum and hardware minimum are not always the same.
- Algebraic minimization is fast but not always systematic enough to guarantee you have found the global minimum.

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization '

**Current:** Chapter 8 — Karnaugh Map

**Remaining:** K-map !' Tabular Method !' Combinational Circuits !' Sequential Circuits !' Fixed Point !' Floating Point

**NEXT**

## Chapter 8 — KARNAUGH MAP ~~Ø=~~MUST MASTER

Next is one of the highest-value Digital Logic chapters:

**Gray-code order !' 2-variable/3-variable/4-variable maps !' adjacency !' wrap-around !' grouping rules !' SOP/POS !' prime implicants !' essential prime implicants !' don't-cares !' multiple minimum solutions !' GATE traps and full minimization problems.**

USER

next

ASSISTANT

## CHAPTER 8 — KARNAUGH MAP (K-MAP) ~~Ø=~~MUST MASTER

K-map is one of the most important minimization techniques in Digital Logic.

Its purpose is:

$\boxed{\text{Convert a Boolean function into a minimum SOP or POS form}}$

without performing long algebraic manipulation.

For GATE, you should be able to look at a K-map and quickly identify:

- valid groups
- invalid groups
- prime implicants
- essential prime implicants
- minimum SOP
- minimum POS
- effect of don't-cares
- multiple minimum solutions

---

### 1. Why Do We Need K-Maps?

Suppose:

$F = A'BC + A'BC' + ABC + ABC'$



Algebraically:

$$\lceil A'B(C+C') + AB(C+C') \rceil$$

$$\lceil A'B + AB \rceil$$

$$\lceil B(A'+A) \rceil$$

$$\lceil B \rceil$$

Correct.

But for larger functions, algebra can become messy.

K-map gives us a **visual systematic method**.

---

## 2. Core Idea Behind K-Map

---

We already learned:

$$\lceil ABC + ABC' = AB \rceil$$

The two terms differ only in C.

Therefore C disappears.

Similarly:

$$\lceil A'BC + ABC = BC \rceil$$

A disappears.

K-map places such logically adjacent minterms next to each other so we can combine them visually.

---

## 3. What Does "Adjacent" Mean?

---

Two minterms are adjacent if their binary representations differ in exactly:

$$\lceil \boxed{1 \text{ bit}} \rceil$$

Example:

$$\lceil 010 \rceil$$

and:

$$\lceil 011 \rceil$$

differ only in the last bit.

Adjacent.

But:

$$\lceil 010 \rceil$$

and:

$$\lceil 101 \rceil$$

differ in all 3 bits.

Not adjacent.

This is based on **Hamming distance 1**.

---

---

## 4. Why Gray Code Is Used

Normal binary order:

$\{00, 01, 10, 11\}$

has a problem:

$01 \rightarrow 10$

changes 2 bits.

Instead K-map uses Gray-code order:

$\{00, 01, 11, 10\}$

Every neighboring pair differs in exactly one bit.

Check:

$00 \rightarrow 01$

one bit changes.

$01 \rightarrow 11$

one bit.

$11 \rightarrow 10$

one bit.

And importantly:

$10 \rightarrow 00$

also one bit.

This creates wrap-around adjacency.

---

## 5. Number of K-Map Cells

For  $n$  variables:

$2^n$  cells

Examples:

| Variables | Cells |
|-----------|-------|
| 2         | 4     |
| 3         | 8     |
| 4         | 16    |
| 5         | 32    |

Each cell corresponds to one minterm.

---

## 6. 2-Variable K-Map

For variables A and B:

| A\B | 0     | 1     |
|-----|-------|-------|
| 0   | $m_0$ | $m_1$ |
| 1   | $m_2$ | $m_3$ |

Equivalent indexing:

TEXT

```

      B
      0  1
A 0  m0 m1
   1  m2 m3

```

## 7. Example — 2 Variable

$F(A,B) = \sum m(1,3)$

Place 1s at:

$m_1, m_3$

Map:

TEXT

```

      B
      0  1
A=0  0  1
A=1  0  1

```

Group the vertical pair.

Across the two cells:

- A changes 0 → 1
- B stays 1

Therefore A disappears.

Result:

$\boxed{F=B}$

## 8. Fundamental Grouping Rule

Group sizes must be powers of two:

$\boxed{1, 2, 4, 8, 16, \dots}$

Allowed:

- 1
- 2

- 4
- 8

Not allowed:

- 3
- 5
- 6
- 7

Memory:

**K-map groups are powers of 2.**

## 9. Why Powers of Two?

Each grouping eliminates changing variables.

A pair:

$$\boxed{2=2^1}$$

eliminates 1 variable.

A group of 4:

$$\boxed{4=2^2}$$

eliminates 2 variables.

A group of 8:

$$\boxed{8=2^3}$$

eliminates 3 variables.

General rule:

$$\boxed{\text{group of } 2^k \text{ cells eliminates } k \text{ variables}}$$

## 10. 3-Variable K-Map

Use A for rows and BC for columns.

Columns must be in Gray order:

$$\boxed{00, 01, 11, 10}$$

Map:

| A \ BC | 00    | 01    | 11    | 10    |
|--------|-------|-------|-------|-------|
| 0      | $m_0$ | $m_1$ | $m_3$ | $m_2$ |
| 1      | $m_4$ | $m_5$ | $m_7$ | $m_6$ |

Notice:

$$\boxed{0, 1, 3, 2}$$

not:

$\{0,1,2,3\}$

## 11. 3-Variable Example

$F = \sum m(0,1,2,3)$

Top row contains all 1s:

TEXT

|     |    |    |    |    |
|-----|----|----|----|----|
|     | BC |    |    |    |
|     | 00 | 01 | 11 | 10 |
| A=0 | 1  | 1  | 1  | 1  |
| A=1 | 0  | 0  | 0  | 0  |

Group all four top cells.

Across group:

- A = 0 always
- B changes
- C changes

Therefore B and C disappear.

For SOP, A=0 corresponds to:

$A'$

So:

$\boxed{F=A'}$

## 12. SOP K-Map Rule

To obtain minimum SOP:

$\boxed{\text{Group 1s}}$

For each group:

- variable always 1 ! write uncomplemented
- variable always 0 ! write complemented
- variable changes ! eliminate it

Then OR the terms.

## 13. Example

Group covers cells:

$m_4, m_5, m_6, m_7$

These correspond to:

$A=1$

while B,C vary.

Therefore:

$\boxed{A}$

## 14. 4-Variable K-Map

Use AB for rows and CD for columns.

Both in Gray-code order:

$\{00, 01, 11, 10\}$

| AB \ CD | 00       | 01       | 11       | 10       |
|---------|----------|----------|----------|----------|
| 00      | $m_0$    | $m_1$    | $m_3$    | $m_2$    |
| 01      | $m_4$    | $m_5$    | $m_7$    | $m_6$    |
| 11      | $m_{12}$ | $m_{13}$ | $m_{15}$ | $m_{14}$ |
| 10      | $m_8$    | $m_9$    | $m_{11}$ | $m_{10}$ |

This indexing must become familiar.

## 15. GATE Trap — Wrong Order

Do NOT arrange:

$\{00, 01, 10, 11\}$

because middle transition changes two bits.

Correct:

$\boxed{\{00, 01, 11, 10\}}$

This is one of the most common K-map mistakes.

## 16. Adjacency in K-Map

Cells are adjacent:

- horizontally
- vertically
- across left/right edges
- across top/bottom edges

They are NOT adjacent diagonally.

## 17. Wrap-Around Adjacency

This is extremely important.

In a 4-variable map:

- first column and last column are adjacent
- first row and last row are adjacent

Why?

Gray-code edges differ by one bit.

Example columns:

$\begin{bmatrix} 00 \\ 10 \end{bmatrix}$

and:

$\begin{bmatrix} 01 \\ 11 \end{bmatrix}$

differ in one bit.

Therefore they are adjacent.

---

## 18. Corner Cells Are Adjacent

The four corners form a valid group of 4.

Why?

Because:

- left edge wraps to right
- top wraps to bottom

Thus corners behave like the four corners of a surface wrapped like a torus.

You do not need the geometry—just remember:

$\boxed{\text{Four corners can form one group of 4}}$

---

## 19. Diagonal Cells Are NOT Adjacent

Example:

TEXT

```
1 0
0 1
```

The two 1s are diagonal.

They differ in two variables.

So:

$\boxed{\text{cannot group diagonally}}$

---

## 20. Grouping Priority

When minimizing SOP:

1. Cover all 1s.

2. Make groups as large as possible.
3. Use the fewest groups possible.
4. Overlap when beneficial.
5. Every group must contain only:
  - 1s
  - and optional don't-cares
6. A group cannot contain a definite 0.

## 21. Why Largest Groups?

Larger group != fewer literals.

For a 4-variable function:

| Group size | Variables remaining |
|------------|---------------------|
| 1          | 4                   |
| 2          | 3                   |
| 4          | 2                   |
| 8          | 1                   |
| 16         | 0                   |

Therefore larger groups generally produce simpler expressions.

## 22. Singleton

If a 1 cannot be grouped with any adjacent 1, it forms a group of 1.

Example cell:

$\lceil m_5 \rceil$

For variables A,B,C:

$\lceil 5=101 \rceil$

So term:

$\lceil \boxed{AB'C} \rceil$

No variable disappears.

## 23. Pair

Suppose group:

$\lceil m_4, m_5 \rceil$

Binary:

$\lceil 100, \lceil 101 \rceil \rceil$



Only C changes.

Therefore C disappears.

Constant:

$\boxed{A=1, B=0}$

So:

$\boxed{AB}$

---

## 24. Quad

Group:

$m_0, m_1, m_2, m_3$

3-variable K-map top row.

A=0 constant.

B,C vary.

Term:

$\boxed{A}$

---

## 25. Octet

In a 4-variable map, group of 8 eliminates 3 variables.

Only one variable remains.

Example:

all cells where A=1:

$m_8$  through appropriate Gray positions

produce:

$\boxed{A}$

---

## 26. Entire Map

If all  $2^n$  cells are 1:

$\boxed{F=1}$

because function is always true.

If all cells are 0:

$\boxed{F=0}$

---

## 27. Example — Simple 3-Variable K-Map

$F = \sum m(0, 1, 4, 5)$

---

Map:

| A BC | 00 | 01 | 11 | 10 |
|------|----|----|----|----|
| 0    | 1  | 1  | 0  | 0  |
| 1    | 1  | 1  | 0  | 0  |

Group the 4 cells:

$\{0,1,4,5\}$

Across these:

- A changes
- B = 0 constant
- C changes

Therefore:

$\boxed{F=B}$

---

## 28. Example — Wrap-Around Pair

$F = \sum m(0,2)$

3-variable map:

- $m_0$  is column 00
- $m_2$  is column 10

They are on left/right ends of same row.

These columns are adjacent through wrap-around.

Binary:

$\{000, 010\}$

Only B changes.

Constants:

$\{A=0, C=0\}$

Thus:

$\boxed{A'C}$

---

## 29. Example — Four Corners

Suppose 4-variable:

$F = \sum m(0,2,8,10)$

These are the four corners.

Binary:

$\{0000, 0010, 1000, 1010\}$

Across them:

- A changes
- B=0
- C changes
- D=0

So term:

$\boxed{B'D}$

This is a classic GATE trap.

---

## 30. Overlapping Groups Ø=Ý4

---

A cell is allowed to belong to more than one group.

Example:

If overlapping allows two groups of 4 instead of several pairs, overlap is beneficial.

Rule:

$\boxed{\text{K-map groups may overlap}}$

The same 1 may be used multiple times.

---

## 31. Why Overlap Is Allowed

---

The Boolean identity:

$A+A=A$

means repeated coverage does not change the function.

Therefore covering the same minterm twice is harmless.

---

## 32. Example of Overlap

---

Suppose a row pattern is:

TEXT  
1 1 1 0

With appropriate adjacent rows, sometimes cells 1 and 2 can participate in separate larger groups.

Do not try to create disjoint groups simply because each cell "should be used once."

That is false.

---

## 33. Can a Group Contain Extra 1s Already Covered?

---

Yes.

In fact, it often should.

A valid group does not care whether its cells were already used.

Objective:

$\boxed{\text{minimum expression}}$

not "use every 1 exactly once."

## 34. Prime Implicant

An **implicant** is a product term that covers one or more 1-cells without covering a 0.

A **prime implicant (PI)** is:

**An implicant that cannot be combined into a larger valid group.**

In K-map language:

$\boxed{\text{Prime implicant = maximal group}}$

Maximal means you cannot expand it further.

## 35. Example

Suppose four adjacent 1s can form a group of 4.

A pair inside that group is still an implicant.

But it is not prime because it can be expanded to the quad.

So:

- pair = implicant
- quad = prime implicant

## 36. Essential Prime Implicant (EPI) Ø=Ý4

A prime implicant is **essential** if it covers at least one minterm not covered by any other prime implicant.

Memory:

**Essential = owns at least one 1 exclusively.**

Such a minterm is sometimes called a unique minterm for that PI.

## 37. How to Find Essential Prime Implicants

For each 1-cell ask:

**How many prime implicants cover this cell?**

If exactly one PI covers it:

$\boxed{\text{that PI is essential}}$

Essential PIs must be included in every minimum cover.

---

## 38. Non-Essential Prime Implicant

---

A PI may be prime but not essential.

Why?

Every cell it covers may also be covered by other PIs.

Such a PI may or may not be selected depending on which cover yields minimum cost.

---

## 39. Example Concept

---

Suppose three PIs:

$P_1, P_2, P_3$

If a minterm is covered only by  $P_1$ , then:

$P_1$

must be selected.

After selecting  $P_1$ , some minterms may already be covered, and you choose among remaining PIs to cover the rest.

---

## 40. SOP Minimization Procedure

---

For:

$F = \sum m(\dots)$

do:

1. Draw correct K-map.
  2. Put 1s in listed cells.
  3. Put 0s elsewhere.
  4. Identify largest possible groups.
  5. Find all prime implicants.
  6. Identify essential prime implicants.
  7. Select EPIs.
  8. Cover any remaining 1s with minimum additional PIs.
  9. Write product term for each selected group.
  10. OR them.
- 

## 41. Example — SOP

---

Consider:

$F(A,B,C) = \sum m(1,3,5,7)$

---

Map has all cells where  $C=1$ .

That's four cells:

$\{001,011,101,111\}$

Group of 4.

Only C remains 1.

Therefore:

$\boxed{F=C}$

---

## 42. Example — SOP With Two Groups

---

$F(A,B,C)=\Sigma m(0,1,2,3,6,7)$

Possible groups:

### Group 1

$m_0, m_1, m_2, m_3$

Top row:

$A'$

### Group 2

$m_2, m_3, m_6, m_7$

Here:

- $B=1$
- A changes
- C changes

Term:

$B$

Thus:

$\boxed{F=A'+B}$

Note that cells  $(m_2, m_3)$  overlap.

That is allowed and useful.

---

## 43. POS Minimization Ø=Ý4

---

For minimum POS:

$\{\text{Group 0s}\}$

Then each zero-group produces a **sum term**.

Rules reverse relative to SOP.

For a zero-group:

- variable constant 0 !' write uncomplemented

- variable constant 1 !' write complemented
- variable changes !' eliminate

Then AND the sum terms.

## 44. Why POS Rule Is Reversed

A maxterm must become zero for its represented cells.

For OR term to be zero, every literal must be zero.

Therefore:

- if variable is 0, use  $\neg(A)$
- if variable is 1, use  $\neg(A')$

Exactly the maxterm rule.

## 45. POS Example

Suppose a zero-group consists of all cells where:

$$\neg[A=0]$$

Then sum term is:

$$\neg[\boxed{A}]$$

because when  $A=0$ , term A becomes zero.

If zero-group has:

$$\neg[A=1]$$

then sum term is:

$$\neg[\boxed{A'}]$$

## 46. Example — POS

Let:

$$\neg[F(A,B)=\neg\bigwedge M(0,2)]$$

Zeros at:

$$\neg[00, \neg 10]$$

These differ in A;  $B=0$  constant.

Group zeros.

For POS:

$B=0$  !' use B.

Thus:

$$\neg[\boxed{F=B}]$$

Check:

$F=0$  whenever  $B=0$ .

Correct.

## 47. SOP vs POS Group Rules

| Task | Group | Constant 0 | Constant 1 |
|------|-------|------------|------------|
| SOP  | 1s    | complement | normal     |
| POS  | 0s    | normal     | complement |

This table is worth memorizing.

## 48. Don't-Care Conditions

Sometimes some input combinations will never occur or their output doesn't matter.

These are called:

$\boxed{\text{don't-cares}}$

Notation:

$d(\dots)$

or:

$X$

Example:

$F = \sum m(1,3,7) + d(0,2,5)$

## 49. How to Use Don't-Cares

A don't-care can be treated as:

$0 \text{ or } 1$

whichever helps simplify the function.

You are **not required** to use it.

Rule:

**Use don't-care only if it creates a larger or better group.**

## 50. Don't-Care Example

Suppose 1s are:

$m_1, m_3$

and don't-cares:



$\{m_5, m_7\}$

Without don't-cares, group  $\{m_1, m_3\}$  gives a pair.

With  $\{m_5, m_7\}$ , all four can form a quad.

The quad may reduce the term further.

Thus don't-cares can reduce literals.

---

## 51. GATE Trap — Must Use Every Don't-Care

---

Wrong.

A don't-care is optional.

You may:

- include it
- ignore it

based on which gives simpler result.

---

## 52. GATE Trap — Don't-Care Must Preserve Assigned Output

---

No.

The whole point is that output is irrelevant for that input.

So you may assign it optimally during minimization.

---

## 53. Don't-Care in POS

---

When minimizing POS, don't-cares may be treated as 0 if they help create larger zero-groups.

Again, optional.

---

## 54. Largest Group vs Fewest Groups

---

Usually create largest possible groups.

But there can be several possible largest groups.

The true goal is:

$\boxed{\text{minimum-cost cover}}$

Typically:

1. minimize number of terms
2. then minimize literals

Do not choose groups purely greedily if that creates more total terms.

---

## 55. Every 1 Must Be Covered

---

For SOP:

$\boxed{\text{Every required 1 must belong to at least one selected group}}$

No required 1 can be left uncovered.

But zeros cannot be included.

Don't-cares may be included.

---

## 56. Every 0 Must Be Covered in POS

---

Analogously, for POS:

$\boxed{\text{Every required 0 must be covered}}$

Selected groups cannot include required 1s.

---

## 57. Redundant Group

---

Suppose a selected group covers only cells already covered by other selected groups and isn't needed for a simpler expression.

Then it is redundant.

Remove it.

Extra groups mean extra terms/gates.

---

## 58. Prime Implicant vs Essential Prime Implicant

---

Do not confuse:

### Prime implicant

Cannot be enlarged.

### Essential prime implicant

Prime implicant that covers a unique required minterm.

Every EPI is a PI.

But:

$\boxed{\text{Not every PI is essential}}$

---

## 59. Example of Literal Formation — SOP

---

Suppose a 4-cell group in a 4-variable K-map has:

- A = 1 constant
- B changes
- C = 0 constant
- D changes

Then:

---

$$\boxed{AC}$$

B,D disappear.

## 60. Example — POS

Suppose a zero-group has:

- $A=1$
- B changes
- $C=0$
- D changes

POS term:

$$A=1 \Rightarrow A'$$

$$C=0 \Rightarrow C$$

So:

$$\boxed{A'+C}$$

## 61. Quick Formula — Literals per Group

For an  $(n)$ -variable K-map and group size  $(2^k)$ :

$$\boxed{\text{literals remaining} = n - k}$$

Example:

4 variables, group of 8:

$$8 = 2^3$$

Remaining literals:

$$4 - 3 = 1$$

## 62. 5-Variable K-Map ~~Ø=8~~ SUPPORT

A 5-variable function has:

$$32$$

cells.

One common representation is two separate 4-variable K-maps:

- one for  $(A=0)$
- one for  $(A=1)$

Corresponding cells between the two maps are adjacent.

So grouping can occur:

- within each map
- across corresponding locations in both maps

You mainly need the principle; GATE CS more often emphasizes smaller maps and conceptual minimization.

---

## 63. 5-Variable Adjacency

If identical cell positions in the two 4-variable maps differ only in A, they are adjacent.

Thus a group may span both planes and eliminate A.

This generalizes K-map into higher dimensions.

---

## 64. Multiple Minimum Solutions

Some K-maps may have more than one equally minimal cover.

Example:

- same number of product terms
- same number of literals
- different chosen overlapping groups

Then more than one minimum expression is valid.

GATE may ask which expressions are minimal/equivalent.

---

## 65. Essential Prime Implicants and Multiple Solutions

If every 1 is covered by EPIs, minimum solution is largely forced.

If some 1s remain after selecting EPIs, different combinations of non-essential PIs may create multiple minimum covers.

---

## 66. Prime Implicant Chart Preview

When many PIs exist, we can create a table showing which minterms each PI covers.

This becomes central in the next chapter:

## Quine–McCluskey / Tabular Method

For K-map, you often identify the cover visually.

---

## 67. Example — Full SOP Problem

Consider:

$$F(A,B,C,D) = \sum m(0,1,2,3,8,9,10,11)$$

These correspond to rows where:

- $B=0$

Let's verify.

For  $B=0$ , AB can be:

$$\quad \text{or} \quad 10$$

---

and CD can be anything.

That's 8 cells.

Group all 8.

A changes, C,D change.

B remains 0.

For SOP:

$\lceil B=0 \rightarrow B' \rceil$

Therefore:

$\lceil \boxed{F=B'} \rceil$

---

## 68. Example — Another 4-Variable Problem

$\lceil F=\Sigma m(4,5,6,7,12,13,14,15) \rceil$

These are all rows where:

$\lceil B=1 \rceil$

Thus group of 8:

$\lceil \boxed{F=B} \rceil$

---

## 69. Example — Two Quads

$\lceil F=\Sigma m(0,1,2,3,8,9,10,11,12,13,14,15) \rceil$

Could group:

- 8 cells where  $B=0$  for rows corresponding certain region
- another 8 where  $A=1$ , depending coverage

But notice there may be a simpler logical expression based on missing cells.

The only missing minterms are:

$\lceil 4,5,6,7 \rceil$

Those correspond:

$\lceil A=0, B=1 \rceil$

So  $F=0$  only when:

$\lceil A'B=1 \rceil$

Hence:

$\lceil F=(A'B)' \rceil$

$\lceil F=A+B' \rceil$

Thus:

$\lceil \boxed{F=A+B'} \rceil$

K-map would yield two octets corresponding to A and B'.

This is a good example of overlap.

---

## 70. Complement Method

---

Sometimes it is easier to minimize  $\overline{F}$  than  $F$ .

If  $F$  has many 1s but few 0s:

1. Group zeros to obtain  $\overline{F}$  as SOP.
2. Complement using De Morgan to get  $F$ .

Or directly use POS minimization.

This can save time.

---

## 71. When to Choose SOP vs POS

---

If map has fewer/better groups of 1s:

$\boxed{\text{use SOP}}$

If map has fewer/better groups of 0s:

$\boxed{\text{use POS}}$

But follow the form requested by the question.

If it explicitly asks minimum SOP, group 1s.

If minimum POS, group 0s.

---

## 72. Common Wrong Grouping Example

---

Suppose three adjacent 1s:

TEXT

1 1 1

You cannot create a group of 3.

Instead you may need:

- two overlapping pairs

Example:

$\overline{[(1,1)]}$

and:

$\overline{[(1,1)]}$

with the middle cell reused.

This is legal.

---

## 73. Group Shapes

---

Valid groups must be rectangular and power-of-two sized.

Possible shapes:

- 1×1
- 1×2
- 2×1
- 1×4
- 4×1
- 2×2
- 2×4
- 4×2
- 4×4

with wrap-around permitted.

Invalid:

- L-shape
- T-shape
- diagonal
- arbitrary 3-cell cluster

---

## 74. GATE Trap — L-Shaped Group

---

Even if four 1s form an L shape, you cannot call them one group of 4 unless they form a valid rectangular adjacency structure.

You may instead use multiple groups with overlap.

---

## 75. GATE Trap — Adjacent Numerically

---

Minterm numbers that differ by 1 are not always K-map adjacent.

Example:

$\backslash m_3=011 \backslash$

and:

$\backslash m_4=100 \backslash$

numerically consecutive, but differ in 3 bits.

Not adjacent.

Adjacency depends on binary Hamming distance:

$\backslash \boxed{1} \backslash$

not decimal difference.

---

## 76. GATE Trap — Cells Far Apart Visually

---

Two cells on opposite edges may be adjacent due to wrap-around.

So physical distance on paper is not the criterion.

Logical bit difference is.

---

## 77. GATE Trap — Grouping Zeros for SOP

---

For SOP:

$\boxed{\text{group 1s}}$

For POS:

$\boxed{\text{group 0s}}$

Do not mix them.

---

## 78. GATE Trap — Complement Direction in POS

---

For SOP:

constant 0 !' complemented.

For POS:

constant 0 !' uncomplemented.

This reversal is frequently tested.

---

## 79. L0 Practice

---

For 3 variables:

$F = \sum m(0,1)$

Binary:

$000, 001$

A=0, B=0 constant.

C changes.

Therefore:

$\boxed{A'B}$

---

## 80. L1 Practice

---

$F = \sum m(0,1,2,3)$

Top row of 3-variable K-map.

A=0.

Therefore:

$\boxed{A}$

---



## 81. L1 Practice — Wrap-Around

---

$$\mathbb{F} = \sum_{m(0,2)}$$

As earlier:

$$\mathbb{F} = 000, 010$$

$A=0, C=0$ .

B changes.

Therefore:

$$\boxed{A'C'}$$

## 82. L2 — Four Corners

---

$$\mathbb{F}(A,B,C,D) = \sum_{m(0,2,8,10)}$$

Four corners.

Constants:

$$B=0, D=0$$

Therefore:

$$\boxed{B'D'}$$

## 83. L2 — Overlap

---

$$\mathbb{F}(A,B,C) = \sum_{m(0,1,2,3,6,7)}$$

Group:

$$0,1,2,3 \rightarrow A'$$

Group:

$$2,3,6,7 \rightarrow B$$

Thus:

$$\boxed{A'+B}$$

Overlap:

$$m_2, m_3$$

is necessary/beneficial.

## 84. L2 — POS

---

Suppose zeros are:

$$m_0, m_1, m_4, m_5$$

3-variable.

These correspond to:

$B=0$ .

Group zeros.

For POS:

$B=0$  !' write B.

Thus:

$\boxed{F=B}$

---

## 85. L3 — Don't-Care

---

Suppose:

$F(A,B,C)=\Sigma m(1,3)+d(5,7)$

Cells:

$1,3,5,7$

can all be treated as 1.

These are exactly all rows with  $C=1$ .

Thus:

$\boxed{F=C}$

Without don't-cares:

$m_1, m_3$

would produce:

$A'C$

So don't-care dramatically simplifies the function.

---

## 86. L3 — Don't-Care Trap

---

Suppose a don't-care doesn't enlarge any selected group.

Then simply ignore it.

Using it unnecessarily can create extra groups, which is worse.

---

## 87. L3 — Group Size

---

A 4-variable map has a block of six 1s.

Can all six be one group?

$\boxed{\text{No}}$

Because:

$6 \neq 2^k$

Use valid overlapping groups of:

---

- 4
  - 2
  - etc.
- 

## 88. Fast K-Map Strategy for GATE

---

When you see a map:

### Scan in this order

1. Groups of 16
2. Groups of 8
3. Groups of 4
4. Pairs
5. Singletons

At each stage:

- look at wrap-around
- look at corners
- allow overlap
- identify cells having only one possible large group

This often reveals essential PIs immediately.

---

## 89. "Lonely 1" Strategy

---

A 1-cell that has very limited grouping choices often forces an essential prime implicant.

Start with such cells rather than randomly grouping dense regions.

This is a good manual minimization strategy.

---

## 90. Prime Implicant Strategy

---

To find all PIs:

- enumerate all maximal possible power-of-two groups
- discard any group completely contained in a larger valid group

The remaining maximal groups are PIs.

---

## 91. Essential PI Strategy

---

For each minterm:

- mark which PIs cover it
- if a minterm has only one covering PI, that PI is essential

Then choose EPIs first.

---

## 92. K-Map and NAND/NOR

---

Once minimum SOP is obtained:

$$F = P_1 + P_2 + \dots$$

it naturally maps to:

$$\boxed{\text{NAND-NAND}}$$

Once minimum POS is obtained:

$$F = S_1 S_2 \dots$$

it naturally maps to:

$$\boxed{\text{NOR-NOR}}$$

So K-map connects directly to circuit implementation.

---

## 93. K-Map and Hazards Preview

---

Sometimes a logically redundant group may be intentionally added to remove a hazard.

So:

- minimum expression
- hazard-free expression

may not always be identical.

Hazards are more advanced and not central to the given official syllabus, so we will not over-invest unless needed.

---

## 94. PYQ Connection

---

Historically, GATE-style K-map questions commonly test:

- identify minimal expression
- prime implicants
- essential prime implicants
- valid grouping
- use of don't-cares
- number of PIs/EPIs
- minimum SOP/POS
- NAND/NOR implementation after minimization
- recognizing wrap-around adjacency
- choosing among equivalent expressions
- combining K-map reasoning with MUX or decoder realization

I am labeling these as **PYQ patterns**, not claiming any specific question/year here.

---

## 95. K-Map Trap Book

---

Add these permanently:

### Trap 1

Using binary order:

$\backslash [00,01,10,11 \backslash]$

instead of Gray:

$\backslash [ \boxed{00,01,11,10} \backslash]$

### Trap 2

Diagonal grouping.

Invalid.

### Trap 3

Forgetting wrap-around.

### Trap 4

Forgetting four corners are adjacent.

### Trap 5

Creating group of 3 or 6.

Invalid.

### Trap 6

Avoiding overlap.

Overlap is allowed.

### Trap 7

Using every don't-care.

Not required.

### Trap 8

Grouping zeros for SOP.

Wrong.

### Trap 9

Grouping ones for POS.

Wrong.

### Trap 10

Wrong complement rule for POS.

### Trap 11

Calling every implicant prime.

Prime means maximal.

### Trap 12

Calling every PI essential.

Essential means it covers a uniquely covered required cell.

---

## 96. K-Map Master Cheat Sheet

---

### Gray order

$\boxed{00, 01, 11, 10}$

### Valid group sizes

$\boxed{1, 2, 4, 8, 16, \dots}$

### SOP

$\boxed{\text{Group 1s}}$

constant 1 !' normal constant 0 !' complement

### POS

$\boxed{\text{Group 0s}}$

constant 0 !' normal constant 1 !' complement

### Group size

$2^k$

eliminates:

$\boxed{k \text{ variables}}$

### Prime implicant

$\boxed{\text{maximal valid group}}$

### Essential PI

$\boxed{\text{covers at least one uniquely covered required cell}}$

### Don't-care

$\boxed{\text{use only when helpful}}$

---

## 97. 30-Second Revision

---

Remember this sequence:

**Gray order !' largest groups !' wrap-around !' overlap !' cover all required cells !' find EPIs !' minimum cover**

For SOP:

$1 \text{ term} \rightarrow \text{product terms}$

For POS:

$0 \text{ term} \rightarrow \text{sum terms}$

And:

$\boxed{\text{pair removes 1 variable}}$

$\boxed{\text{quad removes 2}}$

$\boxed{\text{octet removes 3}}$

---

## What You Absolutely Must Remember

1. K-map uses Gray code.
2. Adjacent cells differ in exactly one bit.
3. Edges wrap around.
4. Corners can form a group.
5. Diagonal cells are never adjacent.
6. Group sizes are powers of 2.
7. Make groups as large as useful.
8. Overlap is allowed.
9. SOP = group 1s.
10. POS = group 0s.
11. Don't-cares are optional helpers.
12. PI = maximal group.
13. EPI = PI with unique required coverage.
14. Larger group means fewer literals.
15. A group of  $(2^k)$  eliminates  $(k)$  variables.

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' **K-Map** '

**Current:** Chapter 9 — Tabular / Quine–McCluskey Method

**Remaining:** Tabular Method !' Combinational Circuits !' Adders/Subtractors !' MUX !' Decoder/Encoder/DEMUX !' Comparator !' Sequential Fundamentals !' Flip-Flops !' Conversions !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

**NEXT**

---

## Chapter 9 — TABULAR / QUINE–McCLUSKEY METHOD ~~0=NOT~~ IMPORTANT

Next we will do a full mechanical minimization:

**group minterms by number of 1s !' combine adjacent terms !' dash notation !' repeated combining !' identify prime implicants !' construct prime-implicant chart !' essential PIs !' choose minimum cover !' don't-care handling !' complete GATE-style example from start to finish.**

USER

next

## CHAPTER 9 — TABULAR / QUINE–McCLUSKEY METHOD ~~0=NOT~~ IMPORTANT

The **Quine–McCluskey method** is a systematic Boolean minimization technique.

Think of it as:

**K-map converted into a table-based algorithm.**

It is especially useful when the number of variables/minterms becomes too large for easy visual K-map grouping.

The method has two major stages:

1. **Generate all Prime Implicants**
2. **Choose the minimum set of Prime Implicants**

### 1. Why Do We Need the Tabular Method?

K-map works beautifully for:

- 2 variables
- 3 variables
- 4 variables

It can also be extended to 5 variables, but it becomes increasingly inconvenient.

Quine–McCluskey gives a mechanical procedure.

Instead of visually checking adjacency, we compare binary minterms.

Two minterms can combine if they differ in exactly:

$\boxed{1 \text{ bit}}$

Example:

$0101$

and:

$0111$

differ only in the third bit.

So they can combine.

### 2. Core Idea

Recall:

$A'BC'D + A'BCD$

These terms differ only in C.

Factor:

$A'BD(C' + C)$

$= A'BD$



So one variable disappears.

In binary:

$\{ 0101 \}$

and:

$\{ 0111 \}$

become:

$\{ 01-1 \}$

The dash means:

**this bit can be either 0 or 1.**

### 3. Dash Notation

Example:

$\{ 01-1 \}$

For variables:

$\{ A,B,C,D \}$

means:

- $A = 0$
- $B = 1$
- $C$  varies
- $D = 1$

Therefore:

$\{ \boxed{A'BD} \}$

The dashed variable disappears.

### 4. First Step — Write Minterms in Binary

Suppose:

$\{ F(A,B,C,D) = \sum m(0,1,2,5,6,7,8,9,10,14) \}$

Convert each minterm to 4-bit binary.

| Minterm | Binary |
|---------|--------|
| 0       | 0000   |
| 1       | 0001   |
| 2       | 0010   |
| 5       | 0101   |

| Minterm | Binary |
|---------|--------|
| 6       | 0110   |
| 7       | 0111   |
| 8       | 1000   |
| 9       | 1001   |
| 10      | 1010   |
| 14      | 1110   |

## 5. Group by Number of 1s

This is the defining first step of the tabular method.

Count the number of 1s in each binary minterm.

### Group 0 — zero 1s

\[ 0=0000 \]

### Group 1 — one 1

\[ 1=0001 \]

\[ 2=0010 \]

\[ 8=1000 \]

### Group 2 — two 1s

\[ 5=0101 \]

\[ 6=0110 \]

\[ 9=1001 \]

\[ 10=1010 \]

### Group 3 — three 1s

\[ 7=0111 \]

\[ 14=1110 \]

## 6. Why Group by Number of 1s?

If two minterms differ in exactly one bit, their number of 1s differs by exactly 1.

Therefore we only compare:

- Group 0 with Group 1
- Group 1 with Group 2
- Group 2 with Group 3
- etc.

No need to compare every minterm with every other minterm.

This saves work.

---

## 7. First Combining Pass

---

Compare adjacent groups.

Example:

$\Sigma(0,1)$

and:

$\Sigma(0,2)$

differ in last bit.

Combine:

$\Sigma(0,1,2)$

This represents minterms:

$\Sigma(0,1,2)$

Similarly:

$\Sigma(0,1,2)$

and:

$\Sigma(0,1,2)$

combine:

$\Sigma(0,1,2)$

representing:

$\Sigma(0,1,2)$

And:

$\Sigma(0,1,2)$

with:

$\Sigma(0,1,2)$

gives:

$\Sigma(0,1,2)$

representing:

$\Sigma(0,1,2)$

---

## 8. Important Rule

---

Two terms combine only if:

1. they differ in exactly one position
2. all existing dash positions match

Example:

$\bar{A}B$

and:

$\bar{A}C$

can combine because only B changes.

Result:

$\bar{A}$

But:

$\bar{A}B$

and:

$\bar{A}C$

cannot be combined directly because dash positions are different.

---

## 9. Mark Combined Terms

Whenever a term participates in a valid combination, mark it:

$\checkmark$

Any term that **cannot be combined further** eventually becomes a:

$\boxed{\text{Prime Implicant}}$

This is crucial.

---

## 10. Small Complete Example

Let's first solve a cleaner function fully:

$F(A,B,C) = \sum m(0,1,2,3,5,7)$

Binary:

| m | ABC | Number of 1s |
|---|-----|--------------|
| 0 | 000 | 0            |
| 1 | 001 | 1            |
| 2 | 010 | 1            |
| 3 | 011 | 2            |
| 5 | 101 | 2            |
| 7 | 111 | 3            |

Group:

### Group 0

$\{ 000 \}$

### Group 1

$\{ 001, 010 \}$

### Group 2

$\{ 011, 101 \}$

### Group 3

$\{ 111 \}$

---

## 11. First Pass Combinations

---

Compare Group 0 and Group 1.

### 000 with 001

$\{ 00- \}$

covers:

$\{ (0,1) \}$

### 000 with 010

$\{ 0-0 \}$

covers:

$\{ (0,2) \}$

---

Compare Group 1 and Group 2.

### 001 with 011

$\{ 0-1 \}$

covers:

$\{ (1,3) \}$

### 001 with 101

$\{ -01 \}$

covers:

$\{ (1,5) \}$

### 010 with 011

$\{ 01- \}$

covers:

$\{ (2,3) \}$

---

Compare Group 2 and Group 3.

---

### 011 with 111

$\backslash [ -11 ]$

covers:

$\backslash [ (3,7) ]$

### 101 with 111

$\backslash [ 1-1 ]$

covers:

$\backslash [ (5,7) ]$

---

## 12. Second Combining Pass

---

Now compare the combined terms.

We can combine terms if they have:

- same dash positions
- one remaining bit difference

Take:

$\backslash [ 00- ]$

and:

$\backslash [ 01- ]$

They differ only in B.

Combine:

$\backslash [ 0-- ]$

This covers:

$\backslash [ 0,1,2,3 ]$

Actually check coverage:

- 00- gives 0,1
- 01- gives 2,3

Combined:

$\backslash [ 0-- ]$

covers 0,1,2,3.

---

Take:

$\backslash [ 0-1 ]$

and:

$\backslash [ 1-1 ]$

They differ only in A.

Combine:

---

$$\bar{A} \bar{B} C$$

covers:

$$\bar{A} B C, \bar{A} B \bar{C}, \bar{A} \bar{B} C, \bar{A} B C$$

Take:

$$\bar{A} B C$$

and:

$$\bar{A} \bar{B} C$$

They differ only in B.

Combine:

$$\bar{A} C$$

again.

Duplicate combination; keep only once.

### 13. Prime Implicants From Example

After all possible combination passes, largest surviving uncombined terms include:

$$\bar{A} \bar{B} C$$

and:

$$\bar{A} B C$$

Translate:

$$\bar{A}(\bar{B}C + BC)$$

$A=0$ , B and C vary.

Therefore:

$$\bar{A}$$

$$A(\bar{B}C + BC)$$

$C=1$ , A and B vary.

Therefore:

$$C$$

Together:

$$\boxed{F = \bar{A} + C}$$

Check:

For minterms:

$$\bar{A} \bar{B} C, \bar{A} B C, A \bar{B} C, A B C$$

$\bar{A}'=1$ .

For:

$\lfloor 5,7 \rfloor$

$C=1$ .

Correct.

---

## 14. Important Interpretation of Dashes

---

For an  $\lfloor n \rfloor$ -variable function:

- no dash !'  $\lfloor n \rfloor$  literals
- 1 dash !'  $\lfloor n-1 \rfloor$  literals
- 2 dashes !'  $\lfloor n-2 \rfloor$  literals
- etc.

So:

$\lfloor \boxed{\text{more dashes = simpler implicant}} \rfloor$

Exactly like larger K-map groups.

---

## 15. Relationship to K-map Group Size

---

One dash means:

$\lfloor 2 \rfloor$

minterms combined.

Two dashes mean:

$\lfloor 4 \rfloor$

minterms combined.

Three dashes mean:

$\lfloor 8 \rfloor$

minterms combined.

Generally:

$\lfloor \boxed{k \text{ dashes}} \rightarrow 2^k \text{ minterms} \rfloor$

---

## 16. Prime Implicant

---

A term that can no longer be combined is a:

$\lfloor \boxed{\text{Prime Implicant}} \rfloor$

This is equivalent to a largest/maximal K-map group.

Important:

A prime implicant does not necessarily appear only in the last table.

A term may stop combining during an earlier stage and become prime there.

So you must preserve all uncombined terms from every pass.

---



---

## 17. Prime Implicant Chart

Finding all prime implicants is only stage 1.

We still need to choose which ones are necessary to cover all required minterms.

Create a table:

- rows = Prime Implicants
- columns = original minterms

Mark X if PI covers that minterm.

---

## 18. Example Prime Implicant Chart

Suppose prime implicants are:

$\{P_1, P_2, P_3\}$

and minterms:

$\{1, 3, 5, 7\}$

Chart:

| PI        | 1 | 3 | 5 | 7 |
|-----------|---|---|---|---|
| $\{P_1\}$ | X | X |   |   |
| $\{P_2\}$ |   | X | X |   |
| $\{P_3\}$ |   |   | X | X |

If minterm 1 is covered only by  $\{P_1\}$ , then  $\{P_1\}$  is essential.

If minterm 7 is covered only by  $\{P_3\}$ , then  $\{P_3\}$  is essential.

---

## 19. Essential Prime Implicant

A PI that covers a minterm covered by no other PI is:

$\boxed{\text{Essential Prime Implicant}}$

Procedure:

1. Find columns with exactly one X.
  2. Select that PI.
  3. Cross out all minterms it covers.
  4. Repeat if new forced choices appear.
- 

## 20. After Selecting EPIs

If all minterms are covered:

Done.

---

If some remain:

Choose the smallest set of non-essential PIs that covers remaining minterms.

Goal:

$\boxed{\text{minimum cover}}$

Usually prioritize:

1. fewer product terms
2. fewer literals

## 21. Prime Implicant Chart Example

Suppose:

$P_1 \rightarrow \{0,1,2,3\}$

$P_2 \rightarrow \{1,3,5,7\}$

and required minterms are:

$\{0,1,2,3,5,7\}$

Chart:

| PI    | 0 | 1 | 2 | 3 | 5 | 7 |
|-------|---|---|---|---|---|---|
| $P_1$ | X | X | X | X |   |   |
| $P_2$ |   | X |   | X | X | X |

Minterm 0 appears only in  $P_1$ .

So  $P_1$  essential.

Minterm 5 appears only in  $P_2$ .

So  $P_2$  essential.

Thus both selected.

## 22. Don't-Care Handling $\emptyset = 4$

Suppose:

$F = \sum m(1,3,7) + d(5)$

Don't-care 5 may be included during the **combination stage**.

This may create larger prime implicants.

However, don't-care minterms do **not** have to be covered in the final prime-implicant chart.

Important rule:

**Use don't-cares to simplify, but only required 1-minterms must be covered.**

## 23. Example With Don't-Care

---

$$F(A,B,C) = \sum m(1,3,7) + d(5)$$

Binary:

$$1 = 001$$

$$3 = 011$$

$$5 = 101 \text{ (don't-care)}$$

$$7 = 111$$

These four patterns all have:

$$C = 1$$

So they combine into:

$$\overline{A}B$$

Thus:

$$\boxed{F = C}$$

Even though  $m_5$  is a don't-care.

---

## 24. Do Not Put Don't-Cares as Required Columns

---

Final PI chart includes:

$$\sum 1,3,7$$

but not necessarily:

$$+ 5$$

because 5 does not need to be covered.

This is a common mistake.

---

## 25. Complete Mechanical Procedure

---

For:

$$F = \sum m(\dots)$$

### Phase A — Generate Prime Implicants

1. Convert each minterm to binary.
2. Group by number of 1s.
3. Compare adjacent groups.
4. Combine terms differing in one bit.
5. Replace differing bit with dash.
6. Mark combined terms.
7. Repeat on newly combined terms.
8. Terms never combined further are PIs.
9. Remove duplicate PIs.

## Phase B — Find Minimum Cover

10. Create prime implicant chart.
  11. Identify EPIs.
  12. Select all EPIs.
  13. Remove covered minterm columns.
  14. Cover remaining minterms with minimum additional PIs.
  15. Translate selected bit patterns into Boolean expressions.
- 

## 26. Translating Dash Terms

---

Suppose variables:

$\{A, B, C, D\}$

Term:

$\{1-0-\}$

Means:

- $A=1 \Rightarrow A$
- B varies  $\Rightarrow$  omit
- $C=0 \Rightarrow C'$
- D varies  $\Rightarrow$  omit

Therefore:

$\boxed{AC'}$

---

## 27. Another Example

---

$\{-01-\}$

Variables A,B,C,D:

- A varies
- $B=0 \Rightarrow B'$
- $C=1 \Rightarrow C$
- D varies

Therefore:

$\boxed{B'C}$

---

## 28. Combination Rule With Dashes

---

Can:

$\{0-01\}$

combine with:

$\{1-01\}$

Yes.

---

Same dash position.

Only A changes.

Result:

$\boxed{--01}$

But:

$\boxed{0-01}$

and:

$\boxed{00-1}$

cannot be combined because their dash locations differ.

---

## 29. Why Dash Positions Must Match

Each dash already represents a variable that changed in an earlier combination.

If dash positions differ, the two terms usually represent sets that are not one valid  $(2^k)$ -sized hypercube.

So the Quine–McCluskey combination rule requires matching dash structure.

---

## 30. Duplicate Terms

Different combination paths may produce the same result.

Example:

$\boxed{00-1}$

may be obtained from different minterm pairs.

Keep only one copy.

Do not count duplicates as separate prime implicants.

---

## 31. GATE Trap — Compare Nonadjacent One-Count Groups

Terms from Group 0 and Group 2 cannot differ in exactly one bit because their number of 1s differs by 2.

So compare only adjacent groups.

---

## 32. GATE Trap — Decimal Difference of 1

Again:

Minterms differing by decimal 1 do not necessarily combine.

Example:

$\boxed{3=011}$

$\boxed{4=100}$

Differ in 3 bits.

Cannot combine.

Binary difference matters.

---

### 33. GATE Trap — Same Number of 1s

Two minterms with same number of 1s cannot differ in exactly one bit.

Why?

Changing one bit from 0 to 1 changes number of 1s by +1.

Changing 1 to 0 changes by -1.

Therefore valid pairs originate in adjacent one-count groups.

---

### 34. Example — Simple Tabular Reduction

$F(A,B,C,D) = \sum m(0,2,8,10)$

Binary:

$0 = 0000$

$2 = 0010$

$8 = 1000$

$10 = 1010$

First combinations:

$0000 + 0010 \rightarrow 00-0$

$0000 + 1000 \rightarrow -000$

$0010 + 1010 \rightarrow -010$

$1000 + 1010 \rightarrow 10-0$

Second combinations:

$00-0 + 10-0 \rightarrow -0-0$

Likewise:

$-000 + -010 \rightarrow -0-0$

Final:

$-0-0$

For ABCD:

A varies B=0 C varies D=0

Therefore:

$\boxed{B'D'}$

Same result we previously saw as the four-corner K-map group.

---

## 35. K-map vs Tabular Method

| Feature                 | K-map     | Quine–McCluskey         |
|-------------------------|-----------|-------------------------|
| Visual                  | Yes       | No                      |
| Best for few variables  | Excellent | More work               |
| Systematic              | Moderate  | Very systematic         |
| Easy by hand            | 2–4 vars  | Small/moderate problems |
| Scales conceptually     | Poor      | Better                  |
| Produces PIs            | Visual    | Algorithmic             |
| Uses binary comparisons | Implicit  | Explicit                |

## 36. Why Tabular Method Is Important for GATE

Even when GATE does not ask you to complete a huge table, the method helps with:

- counting prime implicants
- identifying combinable terms
- understanding minimization
- recognizing PI charts
- don't-care questions
- objective conceptual questions

## 37. Prime Implicant Chart Terminology

Remember:

### Column

A required minterm.

### Row

A prime implicant.

### $X /$

PI covers that minterm.

### Essential row

Contains the only X in at least one minterm column.

## 38. Dominance in PI Chart Ø=βà

For larger problems, a chart can sometimes be simplified using row/column dominance.

Basic intuition:

## Row dominance

If one PI covers every minterm another PI covers and possibly more, while having equal or lower cost, the weaker row may be removable.

## Column dominance

If covering one column automatically ensures coverage requirements relative to another, some columns can be simplified.

For ordinary GATE hand calculations, EPIs and small minimum-cover reasoning are usually enough, but know that dominance exists.

---

## 39. Petrick's Method ~~Ø=64~~ SUPPORT

If no EPI choice resolves the remaining chart and multiple combinations exist, **Petrick's method** can systematically find minimum covers.

Basic idea:

For each uncovered minterm, write the PIs that cover it as a sum.

Example:

$$\backslash [ m_1 : P_1 + P_2 ]$$

$$\backslash [ m_2 : P_2 + P_3 ]$$

Then multiply:

$$\backslash [ (P_1 + P_2)(P_2 + P_3) ]$$

Use Boolean simplification to find minimum PI combinations.

We don't need excessive depth unless a question specifically requires it.

---

## 40. Tiny Petrick Example

$$\backslash [ (P_1 + P_2)(P_2 + P_3) ]$$

Expand:

$$\backslash [ P_1P_2 + P_1P_3 + P_2P_2 + P_2P_3 ]$$

Absorption:

$$\backslash [ P_2 + P_1P_2 = P_2 ]$$

$$\backslash [ P_2 + P_2P_3 = P_2 ]$$

So:

$$\backslash [ P_2 + P_1P_3 ]$$

This means either:

- choose  $\backslash (P_2)$
- or choose both  $\backslash (P_1, P_3)$

If costs are equal per PI,  $\backslash (P_2)$  is better.



## 41. L0 Practice

---

Can these combine?

$\lceil 0101 \rceil$

and:

$\lceil 0111 \rceil$

Yes.

They differ only in C.

Result:

$\boxed{01-1}$

---

## 42. L0 Practice

---

Can:

$\lceil 0101 \rceil$

and:

$\lceil 1100 \rceil$

combine?

Compare:

$\lceil 0,1,0,1 \rceil$

$\lceil 1,1,0,0 \rceil$

Two bits differ.

Therefore:

$\boxed{\text{No}}$

---

## 43. L1 Practice

---

Translate:

$\lceil 1-0 \rceil$

for  $(A,B,C,D)$ .

Constants:

$A=1 \ D=0$

Therefore:

$\boxed{AD'}$

---

## 44. L1 Practice

---

How many minterms are represented by:

---

$\lceil 0 \text{--} 1 \rceil$

Two dashes.

Therefore:

$\lceil 2^2=4 \rceil$

Answer:

$\lceil \boxed{4} \rceil$

---

## 45. L2 Practice

For:

$\lceil F(A,B,C)=\Sigma m(0,1,2,3) \rceil$

tabular combination eventually gives:

$\lceil 0 \text{--} \rceil$

Translate:

$A=0$ .

Therefore:

$\lceil \boxed{A'} \rceil$

---

## 46. L2 — Prime Implicant Chart

Suppose:

| PI | 1 | 2 | 3 |
|----|---|---|---|
| P1 | X | X |   |
| P2 |   | X | X |

Which are essential?

Column 1 has only P1.

So P1 essential.

Column 3 has only P2.

So P2 essential.

Thus:

$\lceil \boxed{P_1, P_2} \rceil$

---

## 47. L3 — Don't-Care Trap

Given:

$\lceil F=\Sigma m(1,3)+d(0,2) \rceil$

Can don't-cares be used to form a group of four?

Yes.

Cells:

$\backslash \begin{matrix} 0,1,2,3 \end{matrix} \backslash$

can be treated as 1s.

If this yields:

$\backslash \begin{matrix} A' \end{matrix} \backslash$

or corresponding simpler term depending on variable order, use them.

But the don't-cares do not need to be covered independently.

---

## 48. L3 — Dash Position Trap

---

Can:

$\backslash \begin{matrix} 0-10 \end{matrix} \backslash$

and:

$\backslash \begin{matrix} 01-0 \end{matrix} \backslash$

combine?

No.

The dash positions differ.

Even though they may look similar, they do not form a valid next-stage combination under the standard tabular rule.

---

## 49. Fast GATE Strategy

---

When a question gives a list of minterms:

1. Convert to binary.
2. Identify pairs differing by 1 bit.
3. Combine.
4. Look for repeated dash patterns.
5. Continue until no further combination.
6. Translate final maximal patterns.
7. If multiple PIs remain, inspect unique minterm coverage.

For a very small 3- or 4-variable function, K-map is usually faster by hand.

---

## 50. When Should You Use Which Method?

---

### Algebraic technique

Best when expression is already small.

### K-map

Best manual method for 2–4 variables.

### Quine–McCluskey

Best when:

- systematic procedure is required
  - many minterms
  - PI chart is asked
  - visual K-map becomes inconvenient
- 

## 51. GATE PYQ Patterns

---

Typical exam patterns include:

- which minterms can combine
- number of prime implicants
- identify essential prime implicants
- interpret dash notation
- minimize a function from minterms
- use don't-cares correctly
- analyze a prime implicant chart
- choose minimum cover

I am treating these as **PYQ-style patterns**, not attributing specific years without verification.

---

## 52. Trap Book — Chapter 9

---

### Trap 1

Grouping by decimal value.

Wrong.

Group by:

$\lfloor \boxed{\text{number of binary 1s}} \rfloor$

### Trap 2

Comparing every group with every group.

Only adjacent one-count groups.

### Trap 3

Combining terms differing in more than one bit.

Invalid.

### Trap 4

Combining terms with mismatched dash positions.

Usually invalid.

### Trap 5

Forgetting uncombined terms from earlier passes.

They may be prime implicants.

### Trap 6

Keeping duplicate combined terms multiple times.

Remove duplicates.

### Trap 7

Putting don't-care columns in final required coverage.

Only real minterms must be covered.

### Trap 8

Assuming every PI is essential.

False.

### Trap 9

Stopping after finding PIs.

You still need the minimum PI cover.

---

## 53. Formula / Shortcut Sheet

For  $(n)$ -bit terms:

$\boxed{\text{Combine if Hamming distance}=1}$

For  $(k)$  dashes:

$\boxed{\text{covers } 2^k \text{ minterms}}$

Literals remaining:

$\boxed{n-k}$

Prime implicant:

$\boxed{\text{cannot combine further}}$

Essential PI:

$\boxed{\text{uniquely covers at least one required minterm}}$

---

## 54. 30-Second Revision

Remember:

**Binary !' group by number of 1s !' compare adjacent groups !' replace one differing bit by dash !' repeat !' collect uncombined PIs !' PI chart !' essential PIs !' minimum cover.**

Also:

$\boxed{\text{Don't-cares help combination but need not be covered}}$

and:

`\boxed{\text{more dashes = fewer literals}}`

---

## What You Absolutely Must Remember

1. Quine–McCluskey is systematic K-map-like minimization.
2. Convert minterms to fixed-width binary.
3. Group by number of 1s.
4. Compare only adjacent groups.
5. Valid combination differs in exactly one bit.
6. Replace differing bit with "-".
7. Matching dash positions are required for later combining.
8. Uncombined maximal terms are prime implicants.
9. Use a prime implicant chart to find EPIs.
10. Don't-cares help minimization but aren't mandatory final coverage.
11. Final answer still needs minimum PI cover.
12. K-map is usually faster for small manual GATE problems.

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' **Tabular Method** '

**Current:** Chapter 10 — Combinational Circuit Fundamentals

**Remaining:** Combinational Fundamentals !' Adders/Subtractors !' MUX !' DEMUX/Decoder/Encoder !' Comparators !' Sequential Fundamentals !' Flip-Flops !' FF Conversion !' Race Around !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

**NEXT**

---

## Chapter 10 — COMBINATIONAL CIRCUITS

Next we move from Boolean minimization into actual circuit design:

**what combinational circuits are !' no-memory property !' propagation delay !' truth-table-to-circuit design procedure !' specification !' inputs/outputs !' Boolean equations !' minimization !' implementation !' GATE design patterns.**

USER

next

ASSISTANT

---

## CHAPTER 10 — COMBINATIONAL CIRCUIT FUNDAMENTALS 0-31 IMPORTANT

Now we move from pure Boolean algebra into **actual digital circuit design**.

A combinational circuit is one where:

$\boxed{\text{Output depends only on present input values}}$

There is:

- no memory
- no previous state
- no clock required for basic operation

Examples:

- adders
  - subtractors
  - multiplexers
  - decoders
  - encoders
  - comparators
- 

## 1. Combinational vs Sequential Circuit

---

This distinction is fundamental.

### Combinational Circuit

---

Output depends on:

$\boxed{\text{current inputs only}}$

Example:

$Y = A + B$

If A and B change, Y changes accordingly.

No need to know what A or B were before.

---

### Sequential Circuit

---

Output may depend on:

$\boxed{\text{current inputs + previous state}}$

Sequential circuits contain memory.

Examples:

- flip-flops
- counters
- registers
- finite-state machines

We will study them later.

---

## 2. Master Comparison

---

| Feature                | Combinational | Sequential        |
|------------------------|---------------|-------------------|
| Memory                 | No            | Yes               |
| Previous state matters | No            | Yes               |
| Clock usually required | No            | Often yes         |
| Feedback common        | Usually no    | Yes               |
| Example                | Adder, MUX    | Counter, register |

Memory trick:

**Combinational = current combination of inputs**

**Sequential = sequence/history matters**

### 3. General Structure

A combinational circuit can be represented as:

```

TEXT
Inputs
A, B, C, ...
!
Combinational Logic
!
Outputs
Y1, Y2, ...

```

Mathematically:

$Y_1 = f_1(A, B, C, \dots)$

$Y_2 = f_2(A, B, C, \dots)$

Every output is a Boolean function of current inputs.

### 4. Fundamental Design Procedure Ø=Ý4

Whenever GATE asks:

**Design a combinational circuit for a given specification.**

Use this exact procedure.

#### Step 1 — Identify inputs

Determine all independent input variables.

#### Step 2 — Identify outputs

Determine what the circuit must generate.



### Step 3 — Assign symbols

Example:

$\{ A, B, C \}$

for inputs and:

$\{ F \}$

for output.

### Step 4 — Construct truth table

List every input combination.

### Step 5 — Determine outputs

Translate the specification into 0/1 values.

### Step 6 — Write Boolean function

Usually as:

- SOP
- POS
- minterms/maxterms

### Step 7 — Minimize

Using:

- Boolean algebra
- K-map
- tabular method

### Step 8 — Implement

Using:

- AND/OR/NOT
- NAND
- NOR
- MUX
- decoder
- etc.

This is the general combinational design pipeline.

---

## 5. Example — Majority Circuit

---

Design a circuit whose output is 1 when at least two of three inputs are 1.

Inputs:

$\{ A, B, C \}$

Output:

$\{ F \}$

Meaning:

$$\{ F=1 \}$$

when number of 1s among A,B,C is at least 2.

## 6. Truth Table

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Rows with F=1:

$$\{ 3,5,6,7 \}$$

Therefore:

$$\{ F=\Sigma m(3,5,6,7) \}$$

K-map minimization gives:

$$\{ \boxed{F=AB+AC+BC} \}$$

## 7. Why Majority Function Is $(AB+AC+BC)$

If at least two inputs are 1, then at least one pair is simultaneously 1.

So:

$$\{ AB=1 \}$$

or:

$$\{ AC=1 \}$$

or:

$$\{ BC=1 \}$$

Therefore:

$$\{ \boxed{F=AB+AC+BC} \}$$

This is also the carry-output expression of a full adder, which we will see soon.

## 8. Example — Even-Parity Detector

Suppose output should be 1 when two inputs A,B are equal.

Truth table:

| A | B | F |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Expression:

$$F = AB + A'B$$

This is:

$$F = A \text{ XNOR } B$$

Thus equality detection for 1 bit is XNOR.

## 9. Example — Odd-Parity Detector

For two inputs, output 1 when they differ.

Truth table:

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Expression:

$$F = A'B + AB'$$

XOR behaves like a 2-input odd-parity detector.

## 10. Multiple Outputs

A combinational circuit can have multiple outputs.

Example:

A full adder has:

Inputs:

$$A, B, C_{in}$$

Outputs:

$S, C_{out}$

Each output is a separate Boolean function:

$S = f_1(A, B, C_{in})$

$C_{out} = f_2(A, B, C_{in})$

This is completely normal.

---

## 11. Propagation Delay $t_p$

Real gates are not instantaneous.

If an input changes at time:

$t = 0$

the output becomes valid after some delay.

This is called:

$\boxed{\text{Propagation delay}}$

Usually denoted:

$t_p$

---

## 12. Why Propagation Delay Matters

Suppose:

TEXT

A → Gate 1 → Gate 2 → Gate 3 → Y

Each gate delay:

$2 \text{ ns}$

Worst-case delay:

$2 + 2 + 2 = 6 \text{ ns}$

Therefore:

$\boxed{Y \text{ may take } 6 \text{ ns to become valid}}$

---

## 13. Critical Path

The **critical path** is:

**The longest-delay path from any input to any output.**

Circuit speed is limited by this path.

If paths are:

$P_1 = 4 \text{ ns}$

$P_2 = 7 \text{ ns}$

$P_3 = 5 \text{ ns}$

Then critical path:

$\boxed{7 \text{ ns}}$

## 14. Gate Delay vs Logic Depth

If all gates have equal delay  $(t_p)$ , and longest path contains  $(L)$  gates:

$\boxed{T_{pd}} \approx L \cdot t_p$

where:

$L = \text{logic depth}$

This simple model is common in GATE numericals.

## 15. Example

Suppose circuit:

$F = (AB + C)'$

Implementation:

1. AND A,B
2. OR with C
3. NOT

If each gate delay is 1 ns:

From A/B to output:

$3 \text{ gates} = 3 \text{ ns}$

From C:

$2 \text{ gates} = 2 \text{ ns}$

Critical path:

$\boxed{3 \text{ ns}}$

## 16. Parallel Gates Do Not Add Delay Sequentially

Suppose:

TEXT

```
A , B ! ' A N D % % %
C , D ! ' A N D % % %
O R ! ' F
```

Both AND gates operate in parallel.

If each gate delay = 2 ns:

First level:

$2 \text{ ns}$

Second OR:

$2 \text{ ns}$

Total:

$4 \text{ ns}$

Not 6 ns.

---

## 17. Fan-In

Fan-in means:

$\boxed{\text{number of inputs accepted by a gate}}$

Examples:

- 2-input AND !' fan-in 2
- 4-input NAND !' fan-in 4

This can affect:

- gate count
  - delay
  - implementation feasibility
- 

## 18. Fan-Out

Fan-out refers to:

**how many gate inputs one output can drive.**

In theoretical GATE Digital Logic questions, it is less frequently central than fan-in, but the concept is useful.

Large fan-out may increase loading and delay in real circuits.

For exam logic diagrams, usually assume the circuit can drive required loads unless told otherwise.

---

## 19. Cost of a Circuit

"Cost" can mean different things depending on question.

It may refer to:

- number of gates
  - number of gate inputs
  - number of literals
  - number of logic levels
  - propagation delay
-

Always identify exactly what the question means by "minimum."

---

## 20. Example — Gate Count

---

Implement:

$$F = AB + CD$$

using basic gates.

Need:

- two AND gates
- one OR gate

Total:

$$\boxed{3 \text{ gates}}$$

If complemented inputs were required, NOT gates might add cost.

---

## 21. Number of Gate Inputs

---

For:

$$F = AB + CD$$

with two 2-input ANDs and one 2-input OR:

Total gate-input count:

$$2 + 2 + 2 = 6$$

Sometimes cost metrics count gate inputs rather than gates.

---

## 22. SOP Circuit Structure

---

Expression:

$$F = AB + A'C + BCD$$

Typical two-level SOP implementation:

First level:

- AND for  $(AB)$
- AND for  $(A'C)$
- AND for  $(BCD)$

Second level:

- OR them

If  $(A')$  not given, one NOT gate is also required.

---

## 23. POS Circuit Structure

---

Expression:

$$F = (A+B)(A'+C)(B+D)$$

First level:

- OR gates

Second level:

- AND

Again, complemented variables may require inverters.

## 24. Functional Completeness

Any combinational Boolean function can be implemented using:

$$\{\text{AND, OR, NOT}\}$$

or using only:

$$\{\text{NAND}\}$$

or only:

$$\{\text{NOR}\}$$

This is why Boolean minimization is directly linked to combinational circuit design.

## 25. Specification !' Truth Table

This is where many students make mistakes.

Suppose:

**Output F should be 1 if exactly two of A,B,C are 1.**

"Exactly two" is different from "at least two."

Rows:

$$011, 101, 110$$

Therefore:

$$F = \sum m(3,5,6)$$

Do not include:

$$111$$

because that has three 1s, not exactly two.

## 26. Exactly-One Detector

Suppose  $F=1$  only when exactly one of A,B,C is 1.

Rows:

$$001, 010, 100$$

Thus:



$$\boxed{F = \sum m(1,2,4)}$$

Expression:

$$\boxed{A'B'C + A'BC' + AB'C}$$

Usually cannot be reduced much without changing behavior.

---

## 27. At-Least-One Detector

---

Output 1 if at least one input is 1.

For A,B,C:

$$\boxed{F = A + B + C}$$

Only zero when:

$$\boxed{A = B = C = 0}$$


---

## 28. All-Ones Detector

---

Output 1 only if all inputs are 1:

$$\boxed{F = ABC}$$

This is simply AND.

---

## 29. All-Zeros Detector

---

Output 1 only if all inputs are 0:

$$\boxed{F = A'B'C'}$$

Equivalent:

$$\boxed{F = (A + B + C)'} \quad \text{using NOR.}$$


---

## 30. Equality Circuit

---

For two bits A and B:

$$\boxed{A = B \rightarrow A \odot B}$$

For two 2-bit numbers:

$$\boxed{A_1 A_0}$$

and:

$$\boxed{B_1 B_0}$$

Equality requires both corresponding bits equal:

$$\boxed{E = (A_1 \odot B_1)(A_0 \odot B_0)}$$

This is the basic idea of a comparator.

---

---

## 31. Parity Circuit

For  $n$  bits:

$$P = A_1 \oplus A_2 \oplus \dots \oplus A_n$$

Output is 1 for an odd number of 1s.

Thus XOR is naturally used for parity generation/checking.

---

## 32. Example — 3-Bit Odd Parity

$$P = A \oplus B \oplus C$$

Output 1 for:

- exactly one 1
- exactly three 1s

Minterms:

$$m(1, 2, 4, 7)$$

Therefore:

$$P = \sum m(1, 2, 4, 7)$$

---

## 33. Even Parity Function

Even parity indicator is complement of odd parity:

$$P_e = (A \oplus B \oplus C)'$$

or multi-input XNOR conceptually.

---

## 34. Don't-Care Inputs in Circuit Design

Some input combinations may never occur.

Example:

BCD digits use only:

$$0000 \text{ through } 1001$$

Inputs:

$$1010 \text{ through } 1111$$

are invalid BCD digits.

If designing a BCD-related combinational circuit, these unused input combinations may be treated as don't-cares if specification permits.

This can simplify the logic significantly.

---

## 35. Example — BCD Validity Detector Preview

A 4-bit BCD input is valid for decimal:

$[0 \text{ to } 9]$

Binary valid range:

$[0000 \text{ to } 1001]$

Invalid:

$[1010, 1011, 1100, 1101, 1110, 1111]$

A validity circuit could detect this.

We won't overdo BCD circuits unless needed, but understand the principle.

## 36. Static vs Dynamic Behavior

Boolean algebra describes the stable logical function.

Real circuits also have timing.

This means an output can temporarily take an incorrect value while signals propagate.

Such temporary errors are called:

$\boxed{\text{hazards/glitches}}$

This is beyond the core syllabus depth, but know the term.

For most GATE questions in this syllabus, focus on:

- Boolean correctness
- propagation delay
- logic levels

## 37. Combinational Circuit Has No State

Suppose input changes:

$[A, B = (0, 0) \rightarrow (1, 1)]$

The output depends only on new input after delay.

There is no stored memory such as:

**"Previously input was 00."**

This is the key difference from sequential circuits.

## 38. No Clock Requirement

Basic combinational circuits react whenever input changes.

They do not need:

$\boxed{\text{clock edges}}$

Examples:

- MUX
- decoder
- adder

Sequential circuits often update based on clock events.

---

## 39. Cascading Combinational Circuits

---

Large combinational systems are built by connecting smaller ones.

Example:

Multiple full adders can be cascaded into:

$\boxed{\text{n-bit ripple-carry adder}}$

Small MUXes can build larger MUXes.

Small decoders can build larger decoders.

This hierarchical design is extremely important.

---

## 40. Example — Circuit Analysis

---

Suppose circuit equations are:

$$X = A \oplus B$$

$$Y = XC$$

$$F = Y + A'B$$

Then:

$$F = C(A \oplus B) + A'B$$

You may then:

- expand XOR
- simplify
- build truth table
- determine function

This is the reverse of circuit design: **circuit analysis**.

---

## 41. Design vs Analysis

---

### Design

Specification:

$\text{truth table} \rightarrow \text{Boolean function} \rightarrow \text{circuit}$

### Analysis

Circuit:

$\rightarrow \text{Boolean equations} \rightarrow \text{simplify} \rightarrow \text{truth table/function}$

Both are tested.

---

## 42. Circuit Analysis Procedure

---

Given a logic diagram:

1. Label intermediate outputs.
2. Write equation for first-level gates.
3. Continue level by level.
4. Substitute into final output.
5. Simplify.
6. If needed, build truth table.

Do not try to mentally evaluate the whole diagram at once.

---

## 43. Example

---

Suppose:

$$X = (AB)'$$

$$Y = (CD)'$$

$$F = (XY)'$$

Then:

$$F = [(AB)'(CD)']'$$

Using De Morgan:

$$\boxed{F = AB + CD}$$

This is NAND-NAND realization of SOP.

---

## 44. Example — Determine Output

---

Suppose:

$$X = A + B$$

$$Y = X'$$

Then:

$$Y = (A + B)'$$

By De Morgan:

$$\boxed{Y = A'B'}$$

This circuit is NOR.

---

## 45. Propagation Delay With Unequal Gate Delays

---

Suppose:

- AND delay = 2 ns
- OR delay = 3 ns
- NOT delay = 1 ns

Circuit:

$$F = A'B + CD$$

Path from A:

NOT:

$$1$$

AND:

$$2$$

OR:

$$3$$

Total:

$$6 \text{ ns}$$

Path from B:

AND + OR:

$$2 + 3 = 5$$

Path from C/D:

$$5 \text{ ns}$$

Critical path:

$$\boxed{6 \text{ ns}}$$

---

## 46. GATE Method for Delay Problems

---

Do not add delay of every gate in circuit.

Instead:

1. List each input-to-output path.
2. Sum delays along that path.
3. Take maximum.

Thus:

$$\boxed{T_{\text{critical}} = \max(\text{path delays})}$$

---

## 47. L0 Practice

---

Which is combinational?

A. Counter B. Register C. Multiplexer D. Flip-flop

Answer:

$\boxed{C}$

MUX output depends only on current inputs/select lines.

---

## 48. L1 Practice

A combinational circuit has 4 input variables.

How many rows are in its full truth table?

$2^4=16$

$\boxed{16}$

---

## 49. L1 — Exact Two Ones

For A,B,C, output 1 if exactly two inputs are 1.

Rows:

$011=3$

$101=5$

$110=6$

Thus:

$\boxed{F=\Sigma m(3,5,6)}$

---

## 50. L2 — At Least Two Ones

For A,B,C:

$F=1$

if at least two inputs are 1.

Includes:

$3,5,6,7$

Minimized:

$\boxed{F=AB+AC+BC}$

---

## 51. L2 — Delay

Circuit:

TEXT

A , B ! ' A N D % % %  
O R ! ' F

C ! ' N O T ! ' A N D %  
with D

Suppose every gate delay = 2 ns.

Path A/B:

$\lceil \text{AND} \rightarrow \text{OR} = 4 \text{ ns} \rceil$

Path C:

$\lceil \text{NOT} \rightarrow \text{AND} \rightarrow \text{OR} = 6 \text{ ns} \rceil$

Path D:

$\lceil \text{AND} \rightarrow \text{OR} = 4 \text{ ns} \rceil$

Critical:

$\boxed{6 \text{ ns}}$

---

## 52. L3 — Design Trap

Output should be 1 when **at least one but not all** of A,B,C is 1.

Which rows?

All rows except:

- 000
- 111

Therefore:

$\boxed{F = \sum m(1,2,3,4,5,6)}$

Important: "at least one" alone would include 111, but "not all" excludes it.

Read English specifications carefully.

---

## 53. L3 — Circuit Classification Trap

A circuit has feedback but no explicit clock.

Is it necessarily combinational?

No.

Feedback can create memory/state, such as latches.

Therefore:

$\boxed{\text{presence of memory/feedback matters, not merely clock}}$

---

## 54. GATE PYQ Patterns

Typical GATE Digital Logic questions involving combinational circuits include:

- derive truth table from specification



- identify combinational vs sequential logic
- determine output function from gate network
- simplify logic network
- calculate propagation delay
- determine critical path
- identify number of gates/levels
- design using MUX/decoder
- analyze adders/subtractors
- compare hierarchical implementations

The next chapters cover the standard combinational building blocks individually.

---

## 55. Trap Book — Chapter 10

---

### Trap 1

Thinking combinational output depends on previous inputs.

It does not.

### Trap 2

Assuming no clock automatically means combinational.

Asynchronous sequential circuits/latches can exist.

### Trap 3

Adding delays of parallel gates.

Only delays along the same path accumulate.

### Trap 4

Confusing "exactly two" with "at least two."

### Trap 5

Not minimizing before implementation.

May lead to excess gates.

### Trap 6

Ignoring complemented-input inverter delay.

It may be on the critical path.

### Trap 7

Assuming minimum literals always gives minimum delay.

Not necessarily.

### Trap 8

Ignoring specification-invalid inputs that may be don't-cares.

---

## 56. Formula / Concept Sheet

---

Combinational output:

$$\boxed{Y = f(\text{present inputs})}$$

Truth-table rows:

$$\boxed{2^n}$$

Critical-path delay:

$$\boxed{T_{pd} = \max(\text{all input-to-output path delays})}$$

Equal-delay gates:

$$\boxed{T_{pd} = L \cdot t_p}$$

where  $L$  is maximum number of gate levels along a path.

---

## 57. 30-Second Revision

---

A combinational circuit:

- has no memory
- output depends only on current inputs
- usually needs no clock
- is represented by Boolean functions

Design flow:

$$\boxed{\text{Specification} \rightarrow \text{Truth Table} \rightarrow \text{Boolean Function} \rightarrow \text{Minimize} \rightarrow \text{Implement}}$$

Timing:

$$\boxed{\text{longest path} = \text{critical path}}$$

---

## What You Absolutely Must Remember

---

1. Combinational = present inputs only.
  2. Sequential = present input + stored state.
  3. Full truth table has  $2^n$  rows for  $n$  inputs.
  4. Every output has its own Boolean function.
  5. Design begins with a precise English specification.
  6. "At least", "exactly", and "at most" mean different things.
  7. Simplify before implementing.
  8. Propagation delays accumulate only along a path.
  9. Critical path determines worst-case delay.
  10. Adders, MUXes, decoders, encoders and comparators are combinational.
- 

## Digital Logic Progress

---

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !'

**Current: Chapter 11 — Adders & Subtractors**

**Remaining:** Adders/Subtractors !' MUX !' DEMUX/Decoder/Encoder !' Comparator !' Sequential Fundamentals !' Flip-Flops !' FF Conversion !' Race Around !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

**NEXT**

## Chapter 11 — ADDERS & SUBTRACTORS 0=1111 MUST MASTER

Next we cover:

**Half Adder !' Full Adder !' Half Subtractor !' Full Subtractor !' truth tables !' equations !' circuit realization !' full adder using half adders !' ripple-carry adder !' carry propagation !' propagation-delay numericals !' carry generate/propagate !' carry look-ahead !' adder-subtractor circuit !' GATE traps.**

USER

next

ASSISTANT

## CHAPTER 11 — ADDERS & SUBTRACTORS 0=1111 MUST MASTER

Adders are among the most important combinational circuits because arithmetic inside processors is built from them.

We will cover:

- Half Adder
- Full Adder
- Half Subtractor
- Full Subtractor
- Full Adder using Half Adders
- Ripple Carry Adder
- Carry propagation
- Carry Generate / Carry Propagate
- Carry Look-Ahead Adder
- Adder-Subtractor circuit
- propagation-delay numericals
- GATE traps

### 1. Half Adder

A **Half Adder** adds two 1-bit inputs.

Inputs:

$\{ A, B \}$

Outputs:

- Sum  $(S)$
- Carry  $(C)$

## 2. Half Adder Truth Table

| A | B | Sum $(S)$ | Carry $(C)$ |
|---|---|-----------|-------------|
| 0 | 0 | 0         | 0           |
| 0 | 1 | 1         | 0           |
| 1 | 0 | 1         | 0           |
| 1 | 1 | 0         | 1           |

Look at Sum:

$[0,1,1,0]$

This is XOR.

Therefore:

$\boxed{S=A \oplus B}$

Carry becomes 1 only when both inputs are 1:

$\boxed{C=AB}$

## 3. Half Adder Circuit

So a Half Adder needs:

- XOR gate for Sum
- AND gate for Carry

TEXT

```
A  % % %, % % % %   X O R  % % %   S
    %
B  % % %

A  % % %, % % % %   A N D  % % %   C
    %
B  % % %
```

Equations:

$\boxed{S=A \oplus B}$

$\boxed{C=AB}$

## 4. Why Is It Called "Half" Adder?

Because it adds only:

$$[A+B]$$

It has no input for a previous-stage carry.

When adding multi-bit numbers, a bit position may also receive:

$$[C_{in}]$$

Therefore Half Adder is insufficient for general multi-bit addition.

We need a:

$$[\boxed{\text{Full Adder}}]$$

## 5. Full Adder

A Full Adder adds three 1-bit inputs:

$$[A, B, C_{in}]$$

Outputs:

- Sum  $(S)$
- Carry-out  $(C_{out})$

## 6. Full Adder Truth Table

| A | B | $(C_{in})$ | S | $(C_{out})$ |
|---|---|------------|---|-------------|
| 0 | 0 | 0          | 0 | 0           |
| 0 | 0 | 1          | 1 | 0           |
| 0 | 1 | 0          | 1 | 0           |
| 0 | 1 | 1          | 0 | 1           |
| 1 | 0 | 0          | 1 | 0           |
| 1 | 0 | 1          | 0 | 1           |
| 1 | 1 | 0          | 0 | 1           |
| 1 | 1 | 1          | 1 | 1           |

## 7. Full Adder Sum Equation

Observe S.

It is 1 when number of input 1s is odd.

Therefore:

$$[\boxed{S=A \oplus B \oplus C_{in}}]$$

This is a 3-input odd-parity function.

## 8. Full Adder Carry Equation

---

Carry-out is 1 when at least two of the three inputs are 1.

Therefore:

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}}$$

This is the 3-input majority function.

Alternative form:

$$\boxed{C_{out} = AB + C_{in}(A \oplus B)}$$

Both are equivalent.

---

## 9. Why Carry Is Majority Function

---

For carry to occur, total input sum must be at least:

$$2$$

So any two 1s are enough.

That gives:

$$AB$$

or:

$$AC_{in}$$

or:

$$BC_{in}$$

Hence:

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}}$$

---

## 10. Full Adder Using Two Half Adders

---

This is a standard circuit.

### First Half Adder

Adds:

$$A + B$$

Outputs:

$$S_1 = A \oplus B$$

$$C_1 = AB$$

### Second Half Adder

Adds:

$$S_1 + C_{in}$$

Outputs:

$$S = S_1 \oplus C_{in}$$

$$C_2 = S_1 C_{in}$$

Final carry:

$$C_{out} = C_1 + C_2$$

So:

$$\boxed{\text{1 Full Adder} = 2 \text{ Half Adders} + 1 \text{ OR gate}}$$

## 11. Verify Sum

$$S = (A \oplus B) \oplus C_{in}$$

Therefore:

$$\boxed{S = A \oplus B \oplus C_{in}}$$

## 12. Verify Carry

$$C_{out} = AB + C_{in}(A \oplus B)$$

Expand XOR:

$$A \oplus B = A'B + AB'$$

Therefore:

$$C_{out} = AB + A'BC_{in} + AB'C_{in}$$

This is equivalent to majority expression:

$$\boxed{AB + AC_{in} + BC_{in}}$$

## 13. Half Subtractor

A Half Subtractor subtracts one bit B from A:

$$A - B$$

Outputs:

- Difference  $(D)$
- Borrow  $(B_{out})$

## 14. Half Subtractor Truth Table

| A | B | Difference D | Borrow |
|---|---|--------------|--------|
| 0 | 0 | 0            | 0      |
| 0 | 1 | 1            | 1      |
| 1 | 0 | 1            | 0      |

| A | B | Difference D | Borrow |
|---|---|--------------|--------|
| 1 | 1 | 0            | 0      |

Difference pattern:

$[0, 1, 1, 0]$

Therefore:

$\boxed{D = A \oplus B}$

Borrow occurs only when:

$A = 0, B = 1$

Therefore:

$\boxed{B_{out} = A'B}$

## 15. Half Adder vs Half Subtractor

Notice:

Both have:

$\boxed{\text{XOR output for Sum/Difference}}$

But second output differs.

Half Adder carry:

$\boxed{AB}$

Half Subtractor borrow:

$\boxed{A'B}$

Good memory pattern.

## 16. Full Subtractor

A Full Subtractor computes:

$A - B - B_{in}$

Inputs:

- A = minuend bit
- B = subtrahend bit
- $B_{in}$  = borrow from previous lower bit

Outputs:

- Difference D
- Borrow-out  $B_{out}$

## 17. Full Subtractor Truth Table



| A | B | $(B_{in})$ | D | $(B_{out})$ |
|---|---|------------|---|-------------|
| 0 | 0 | 0          | 0 | 0           |
| 0 | 0 | 1          | 1 | 1           |
| 0 | 1 | 0          | 1 | 1           |
| 0 | 1 | 1          | 0 | 1           |
| 1 | 0 | 0          | 1 | 0           |
| 1 | 0 | 1          | 0 | 0           |
| 1 | 1 | 0          | 0 | 0           |
| 1 | 1 | 1          | 1 | 1           |

Difference:

$$\boxed{D = A \oplus B \oplus B_{in}}$$

Exactly like Full Adder sum.

## 18. Full Subtractor Borrow Equation

A standard form is:

$$\boxed{B_{out} = A'B + A'B_{in} + BB_{in}}$$

Another useful form:

$$\boxed{B_{out} = A'B + B_{in}(A \odot B)}$$

where  $(A \odot B)$  is XNOR.

## 19. Full Adder vs Full Subtractor

### Sum/Difference

Both:

$$\boxed{A \oplus B \oplus X_{in}}$$

where  $(X_{in})$  is carry-in or borrow-in.

But carry and borrow logic differ.

## 20. Why Addition Hardware Matters More

Computers commonly do not build a completely separate subtraction circuit.

Instead:

$$\boxed{A - B = A + \text{2's complement}(B)}$$

So subtraction can be performed using the same adder hardware.

This leads to:

$\boxed{\text{Adder-Subtractor Circuit}}$

## 21. n-Bit Parallel Addition

To add two n-bit numbers:

$A_{n-1} \dots A_1 A_0$

and:

$B_{n-1} \dots B_1 B_0$

we connect n Full Adders.

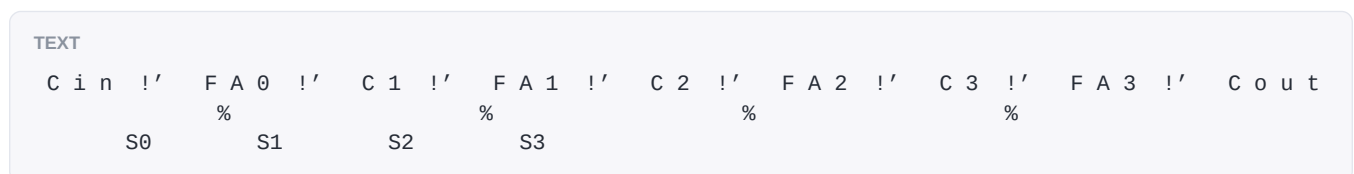
Each bit position produces:

- sum
- carry to next position

## 22. Ripple Carry Adder (RCA)

A Ripple Carry Adder connects full adders serially through the carry chain.

For 4 bits:



Each carry must wait for previous stage.

Therefore carry **ripples** from LSB to MSB.

## 23. Why RCA Is Slow

Suppose the MSB sum depends on carry from all previous stages.

Carry must travel:

$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots$

Thus delay increases approximately linearly with number of bits.

That is the major weakness.

## 24. Ripple Carry Delay

Suppose each full adder carry propagation delay is:

$t_c$

For an n-bit RCA, worst-case carry delay is roughly:

$\boxed{n \cdot t_c}$

depending on exact timing model.

If final sum requires an additional sum-generation delay  $(t_s)$ , then a common model is:

$$T_{\text{MSB sum}} \approx (n-1)t_c + t_s$$

while final carry may be:

$$T_{\text{Cout}} \approx nt_c$$

Always use the exact per-stage delays specified in the question.

---

## 25. Example

4-bit RCA.

Each carry propagation delay:

$$2 \text{ ns}$$

Final carry worst-case:

$$4 \times 2 = 8 \text{ ns}$$

$$\boxed{8 \text{ ns}}$$

---

## 26. More Realistic RCA Timing

Suppose each Full Adder has:

- carry generation from A/B: 2 ns
- carry propagation from  $C_{in}$  to  $C_{out}$ : 1 ns
- sum after carry-in: 2 ns

Then exact longest path may not simply be:

$$n \times 2$$

You must trace the critical carry chain using given delays.

GATE likes this style.

---

## 27. Carry Generate and Carry Propagate $\Phi = \Psi_4$

To speed up addition, define for bit  $i$ :

### Generate

$$\boxed{G_i = A_i B_i}$$

If both A and B are 1, this bit definitely generates a carry.

### Propagate

One common definition:

$$\boxed{P_i = A_i \oplus B_i}$$

If exactly one of them is 1, an incoming carry propagates through.

---

Then:

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

This is fundamental.

---

## 28. Why $(G_i)$ ?

If:

$$A_i = B_i = 1$$

then:

$$1 + 1 = 10$$

A carry occurs regardless of  $(C_i)$ .

Thus:

$$\boxed{G_i = 1}$$

means stage generates carry independently.

---

## 29. Why $(P_i)$ ?

If:

$$A_i \oplus B_i = 1$$

then the local sum of A and B is 1.

If  $(C_i = 1)$ , that extra 1 creates carry.

So incoming carry is passed through.

Hence:

$$\boxed{P_i = 1}$$

means propagate.

---

## 30. Carry Equation

For bit i:

$$C_{i+1} = A_i B_i + C_i (A_i \oplus B_i)$$

Using generate/propagate:

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

---

## 31. Carry Look-Ahead Adder (CLA) Ø=Ý4

Instead of waiting for each carry sequentially, CLA computes carries directly using expanded equations.

For bit 0:

$$C_1 = G_0 + P_0 C_0$$

---

For bit 1:

$$C_2 = G_1 + P_1 C_1$$

Substitute  $C_1$ :

$$C_2 = G_1 + P_1(G_0 + P_0 C_0)$$

$$\boxed{C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0}$$

Now  $C_2$  can be computed without waiting for  $C_1$  physically to ripple through sequentially.

---

## 32. Derive $C_3$

$$C_3 = G_2 + P_2 C_2$$

Substitute:

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

So:

$$\boxed{C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0}$$

## 33. Derive $C_4$

$$\boxed{C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0}$$

This is the 4-bit look-ahead carry formula.

---

## 34. CLA Advantage

RCA:

$$\boxed{\text{carry delay grows roughly linearly with } n}$$

CLA:

$$\boxed{\text{carries are computed more parallelly}}$$

So it is faster.

Tradeoff:

- more gates
  - more wiring
  - larger fan-in
  - higher hardware complexity
- 

## 35. Group Generate and Propagate $G$ and $P$

For a 4-bit block:

Group propagate:

$$\boxed{P_G = P_3 P_2 P_1 P_0}$$

Group generate:

$$\boxed{G_G = G_3 + P_{3G_2} + P_{3P_2G_1} + P_{3P_2P_1G_0}}$$

Then:

$$\boxed{C_4 = G_G + P_{GC_0}}$$

Useful in hierarchical CLA design.

---

## 36. Full Adder Sum Using Propagate

---

Since:

$$P_i = A_i \oplus B_i$$

sum:

$$\boxed{S_i = P_i \oplus C_i}$$

Thus generate/propagate also simplify notation.

---

## 37. Alternative Propagate Definition

---

Some texts define:

$$P_i = A_i + B_i$$

instead of XOR.

Then carry formulas are still valid in an appropriate formulation.

For GATE:

$$\boxed{\text{Always use the definition provided in the question}}$$

This is an important trap.

If no definition is given, XOR-propagate is very common.

---

## 38. Adder-Subtractor Circuit Ø=Ý4

---

Goal:

Perform either:

$$A+B$$

or:

$$A-B$$

with the same adder.

Recall:

$$A-B = A + \overline{B} + 1$$

Use a control input M:

- $(M=0)$  !' addition

- $(M=1)$  !' subtraction
- 

## 39. XOR Trick

---

Feed each B bit through XOR with M:

$$\boxed{B_i = B_i \oplus M}$$

If:

$$M=0$$

then:

$$B_i \oplus 0 = B_i$$

So B unchanged.

If:

$$M=1$$

then:

$$B_i \oplus 1 = B_i'$$

So B is complemented.

Also connect:

$$C_{in} = M$$

---

## 40. Addition Mode

---

If:

$$M=0$$

Then:

$$B_i \oplus 0 = B_i$$

and:

$$C_{in} = 0$$

Thus circuit computes:

$$\boxed{A+B}$$

---

## 41. Subtraction Mode

---

If:

$$M=1$$

Then:

$$B_i \oplus 1 = B_i'$$

and:

---

$$\lceil C_{in}=1 \rceil$$

So adder computes:

$$\lceil A+B'+1 \rceil$$

which is:

$$\lceil \boxed{A-B} \rceil$$

using 2's complement.

---

## 42. Adder-Subtractor Master Equation

$$\lceil \boxed{A+(B\oplus M)+M} \rceil$$

bitwise for B.

When M=0:

$$\lceil A+B \rceil$$

When M=1:

$$\lceil A+B'+1=A-B \rceil$$


---

## 43. Overflow in Adder Circuit

For signed 2's complement, overflow can be detected as:

$$\lceil \boxed{V=C_{n-1}\oplus C_n} \rceil$$

where:

- $\lceil C_{n-1} \rceil$  = carry into MSB
- $\lceil C_n \rceil$  = carry out of MSB

This connects chapter 3 to actual hardware.

---

## 44. Unsigned Carry Flag vs Signed Overflow Flag

In arithmetic hardware, CPU may maintain:

### Carry flag C

Useful for unsigned overflow.

### Overflow flag V

Useful for signed 2's-complement overflow.

They are different.

Again:

$$\lceil \boxed{C \neq V} \rceil$$


---

## 45. Example — 4-Bit Addition

---



Add:

$$\{ 0101+0011 \}$$

$$\{ 5+3=8 \}$$

Binary:

$$\{ 1000 \}$$

As unsigned 4-bit:

8 fits.

As signed 4-bit:

range is:

$$\{ -8 \text{ to } 7 \}$$

So:

$$\{ 1000=-8 \}$$

Hence signed overflow.

Same hardware output, different interpretation.

---

## 46. Carry Chain Example

---

Consider:

$$\{ 0111+0001 \}$$

At LSB:

$$\{ 1+1 \}$$

generates carry.

Carry propagates through higher bits:

$$\{ 1+0+1 \}$$

repeatedly.

Eventually:

$$\{ 1000 \}$$

This is a worst-case type of ripple behavior.

---

## 47. Carry Propagation Worst Case

---

A carry tends to ripple far when intermediate stages satisfy:

$$\{ P_i=1 \}$$

That is:

$$\{ A_i \oplus B_i=1 \}$$

Each stage passes incoming carry to next stage.

---

## 48. Carry Generate Stops Dependence on Earlier Carry

---

If:

$\lceil G_i = 1 \rceil$

then:

$\lceil C_{i+1} = 1 \rceil$

regardless of  $\lceil C_i \rceil$ .

So this stage generates its own carry.

---

## 49. Kill Condition $\lceil C_i = 0 \rceil$

---

Sometimes one also describes:

- Generate
- Propagate
- Kill

If:

$\lceil A_i = B_i = 0 \rceil$

then:

$\lceil G_i = 0, P_i = 0 \rceil$

and:

$\lceil C_{i+1} = 0 \rceil$

regardless of incoming carry.

This stage "kills" the carry.

---

## 50. Half Adder Gate-Level Delay

---

Suppose:

- XOR delay = 3 ns
- AND delay = 2 ns

Then:

Sum valid after:

$\lceil 3 \text{ ns} \rceil$

Carry valid after:

$\lceil 2 \text{ ns} \rceil$

Outputs need not have equal delay.

---

## 51. Full Adder Delay Using Two Half Adders

---

Architecture:

---

1.  $(S_1 = A \oplus B)$
2.  $(C_1 = AB)$
3.  $(S = S_1 \oplus C_{in})$
4.  $(C_2 = S_1 C_{in})$
5.  $(C_{out} = C_1 + C_2)$

Suppose:

- XOR = 2 ns
- AND = 1 ns
- OR = 1 ns

### Sum path from A/B

Two XORs:

$$[2 + 2 = 4 \text{ ns}]$$

### Carry path through XOR !' AND !' OR

$$[2 + 1 + 1 = 4 \text{ ns}]$$

There may be a shorter carry path:

$$[A, B \rightarrow \text{AND} \rightarrow \text{OR}]$$

$$[1 + 1 = 2 \text{ ns}]$$

Worst-case carry:

$$[\boxed{4 \text{ ns}}]$$

## 52. RCA Delay Example

Suppose each full adder has:

- $(C_{in} \rightarrow C_{out})$ : 2 ns
- $(C_{in} \rightarrow S)$ : 3 ns

For 4-bit RCA:

To get MSB sum  $(S_3)$ :

Carry must propagate through first 3 stages:

$$[3 \times 2 = 6]$$

Then final sum:

$$[3]$$

Total:

$$[\boxed{9 \text{ ns}}]$$

Final carry:

$$[4 \times 2 = 8 \text{ ns}]$$

Thus worst output delay:

$$[\boxed{9 \text{ ns}}]$$

This exact approach is very GATE-relevant.

---

### 53. General RCA Delay Model

---

If:

- carry propagation per stage =  $t_c$
- final sum after carry arrives =  $t_s$

Then for  $n$  bits:

$$T_{S_{MSB}} = (n-1)t_c + t_s$$

and:

$$T_{Cout} = nt_c$$

Worst circuit delay:

$$T = \max(T_{S_{MSB}}, T_{Cout})$$

provided the question uses this model.

---

### 54. Why Full Adder Carry May Be Faster Than Sum

---

Carry logic may use:

$$AB + C_{in} \oplus (A \oplus B)$$

while sum requires another XOR.

Therefore carry and sum can have different propagation delays.

Never assume equal delay unless specified.

---

### 55. Binary Incrementer Preview

---

To increment a binary number:

$$A + 1$$

you can use adder logic.

A dedicated incrementer is simpler than a full general adder because second operand is constant.

This may appear in datapath/ALU discussions later.

---

### 56. BCD Adder ~~0=0~~ IMPORTANT

---

BCD represents decimal digits 0–9 using 4 bits.

Examples:

$$5 = 0101$$

$$9 = 1001$$

But:

---

$\{ 1010 \text{ through } 1111 \}$

are invalid BCD digits.

---

## 57. BCD Addition Rule

Add two BCD digits as ordinary binary.

If result is:

- greater than 1001 (decimal 9), or
- produces carry out

then add:

$\{ \boxed{0110} \}$

which is decimal 6.

Why 6?

Because 4-bit binary has 16 states but decimal digit needs only 10.

Correction:

$\{ 16-10=6 \}$

---

## 58. BCD Example

Add decimal:

$\{ 7+5=12 \}$

BCD inputs:

$\{ 0111 \}$

$\{ 0101 \}$

Binary sum:

$\{ 1100 \}$

which is decimal 12 and invalid as one BCD digit.

Add correction:

$\{ 1100+0110=1\,0010 \}$

Interpret as:

- carry 1 = tens digit 1
- lower nibble 0010 = 2

So:

$\{ \boxed{12} \}$

correct BCD result.

---

## 59. BCD Correction Condition

Correction is needed when 4-bit sum is greater than:

$$\lfloor 1001 \rfloor$$

or carry out occurs.

A common logic condition can be derived from output bits, but for GATE the concept is often more important than memorizing a specific detector equation unless explicitly asked.

---

## 60. Half Adder — GATE Quick Facts

---

$$\lfloor \boxed{S=A\oplus B} \rfloor$$

$$\lfloor \boxed{C=AB} \rfloor$$

One XOR + one AND.

---

## 61. Full Adder — GATE Quick Facts

---

$$\lfloor \boxed{S=A\oplus B\oplus C_{in}} \rfloor$$

$$\lfloor \boxed{C_{out}=AB+AC_{in}+BC_{in}} \rfloor$$

or:

$$\lfloor \boxed{C_{out}=AB+C_{in}(A\oplus B)} \rfloor$$

---

## 62. Half Subtractor — GATE Quick Facts

---

$$\lfloor \boxed{D=A\oplus B} \rfloor$$

$$\lfloor \boxed{\text{Borrow}=A'B} \rfloor$$

---

## 63. Full Subtractor — GATE Quick Facts

---

$$\lfloor \boxed{D=A\oplus B\oplus B_{in}} \rfloor$$

$$\lfloor \boxed{B_{out}=A'B+A'B_{in}+BB_{in}} \rfloor$$

---

## 64. L0 Practice

---

For Half Adder:

$$\lfloor A=1, B=1 \rfloor$$

Sum:

$$\lfloor 1\oplus 1=0 \rfloor$$

Carry:

$$\lfloor 1\cdot 1=1 \rfloor$$

Therefore:

$$\lfloor \boxed{S=0, C=1} \rfloor$$

---

---

## 65. L0 Practice

Full Adder:

$$\text{\[ A=1,\ B=0,\ C_{in}=1 \]}$$

Total:

$$\text{\[ 1+0+1=2 \]}$$

Binary:

$$\text{\[ 10 \]}$$

So:

$$\text{\[ \boxed{S=0,\ C_{out}=1} \]}$$

---

## 66. L1 — Equation Recognition

Which is Full Adder carry?

A.

$$\text{\[ A\oplus B\oplus C \]}$$

B.

$$\text{\[ AB+AC+BC \]}$$

C.

$$\text{\[ A+B+C \]}$$

Answer:

$$\text{\[ \boxed{B} \]}$$

---

## 67. L1 — Full Adder Construction

How many Half Adders and OR gates are needed to construct one standard Full Adder?

$$\text{\[ \boxed{2\text{ Half Adders}+1\text{ OR}} \]}$$

---

## 68. L1 — Half Subtractor

For:

$$\text{\[ A=0,\ B=1 \]}$$

Need:

$$\text{\[ 0-1 \]}$$

Borrow 1 from next position.

Difference:

$$\text{\[ 1 \]}$$

Borrow:

$$\lceil 1 \rceil$$

Therefore:

$$\lceil \boxed{D=1, \text{Borrow}=1} \rceil$$

---

## 69. L2 — Carry Propagate

---

Suppose:

$$\lceil A_i=1, \text{B}_i=0 \rceil$$

Then:

$$\lceil P_i=A_i \oplus B_i=1 \rceil$$

$$\lceil G_i=A_i B_i=0 \rceil$$

Thus:

$$\lceil C_{i+1}=C_i \rceil$$

Incoming carry propagates.

---

## 70. L2 — Carry Generate

---

Suppose:

$$\lceil A_i=B_i=1 \rceil$$

Then:

$$\lceil G_i=1 \rceil$$

Thus:

$$\lceil C_{i+1}=1 \rceil$$

regardless of  $\lceil C_i \rceil$ .

---

## 71. L2 — Carry Look-Ahead

---

Given:

$$\lceil G_0=1, P_1=1, G_1=0, C_0=0 \rceil$$

Find  $\lceil C_2 \rceil$ :

$$\lceil C_2=G_1+P_1G_0+P_1P_0C_0 \rceil$$

Since:

$$\lceil G_1=0 \rceil$$

$$\lceil P_1G_0=1 \rceil$$

So:

$$\lceil \boxed{C_2=1} \rceil$$

---



No need for other terms.

---

## 72. L2 — Adder/Subtractor Control

---

If  $M=1$  in:

$$[A + (B \oplus M) + M]$$

then:

$$[B \oplus 1 = B']$$

So:

$$[A + B' + 1]$$

Therefore:

$$[\boxed{A - B}]$$

---

## 73. L3 — RCA Delay

---

8-bit ripple carry adder:

- carry propagation per stage = 1 ns
- final sum after carry = 2 ns

MSB sum delay:

$$[(8-1)(1) + 2]$$

$$[=9 \text{ ns}]$$

Final carry:

$$[8 \text{ ns}]$$

Worst:

$$[\boxed{9 \text{ ns}}]$$

---

## 74. L3 — Signed Overflow

---

4-bit addition:

$$[0110 + 0011]$$

$$[6 + 3 = 9]$$

Binary stored:

$$[1001]$$

Positive + positive != negative result.

Therefore:

$$[\boxed{\text{signed overflow}}]$$

Carry-out is not the deciding rule.

---

---

## 75. L3 — BCD Trap

---

Add:

$\backslash[0101+0011\backslash]$

This is:

$\backslash[5+3=8\backslash]$

Binary result:

$\backslash[1000\backslash]$

Since:

$\backslash[8\leq 9\backslash]$

no correction needed.

Do NOT add 0110 unnecessarily.

---

## 76. GATE PYQ Patterns

---

Common GATE-style patterns include:

- derive/full-adder equations
- identify Full Adder built from Half Adders
- ripple-carry delay
- carry propagation chain
- generate/propagate equations
- compare RCA vs CLA
- signed overflow detection
- use one adder for subtraction
- BCD correction logic
- gate-level timing of adders

These are presented as historical pattern categories, not specific claimed years.

---

## 77. Trap Book — Chapter 11

---

### Trap 1

Half Adder has carry-in.

Wrong.

It has only A,B.

### Trap 2

Full Adder carry is XOR.

Wrong.

Sum is XOR.

### Trap 3

Carry and signed overflow are same.

Wrong.

### Trap 4

RCA delays from all gates simply added.

Trace only the critical path.

### Trap 5

CLA removes all delay.

No. It reduces carry-chain delay; gates still have delay.

### Trap 6

Generate depends on carry-in.

No.

$$G_i = A_i B_i$$

### Trap 7

Ignoring propagate definition supplied in question.

Different conventions exist.

### Trap 8

Subtraction requires separate subtractor hardware.

Not necessarily. 2's-complement adder-subtractor is standard.

### Trap 9

For subtraction mode, complement B but forget +1.

Need both:

$$B \text{ and } C_{in} = 1$$

### Trap 10

Always add 0110 in BCD addition.

Only when result is invalid (>9) or carry occurs.

## 78. Master Comparison Table

| Circuit         | Inputs              | Outputs            | Main equations                        |
|-----------------|---------------------|--------------------|---------------------------------------|
| Half Adder      | A,B                 | S,C                | $S = A \oplus B, C = AB$              |
| Full Adder      | A,B,C <sub>in</sub> | S,C <sub>out</sub> | $S = A \oplus B \oplus C_{in}$        |
| Half Subtractor | A,B                 | D,Borrow           | $D = A \oplus B, \text{Borrow} = A'B$ |
| Full Subtractor | A,B,B <sub>in</sub> | D,B <sub>out</sub> | $D = A \oplus B \oplus B_{in}$        |

| Circuit          | Inputs    | Outputs      | Main equations       |
|------------------|-----------|--------------|----------------------|
| RCA              | n-bit A,B | n-bit S,Cout | cascaded full adders |
| CLA              | n-bit A,B | n-bit S,Cout | look-ahead carry     |
| Adder-Subtractor | A,B,M     | result       | $(A+(B\oplus M)+M)$  |

## 79. Formula Sheet

Half Adder:

$$S = A \oplus B$$

$$C = AB$$

Full Adder:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

Carry:

$$C_{i+1} = G_i + P_i C_i$$

with:

$$G_i = A_i B_i$$

$$P_i = A_i \oplus B_i$$

RCA MSB sum timing model:

$$(n-1)t_c + t_s$$

Adder-subtractor:

$$A + (B \oplus M) + M$$

Signed overflow:

$$V = C_{\text{into MSB}} \oplus C_{\text{out of MSB}}$$

## 80. 30-Second Revision

Remember:

### Half Adder

$$S = \text{XOR}, \quad C = \text{AND}$$

### Full Adder

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = \text{majority}(A, B, C_{in})$$

### Ripple Carry

Simple, but slow because carry moves stage by stage.

## CLA

Use:

$$\lceil G_i, \lceil P_i \rceil$$

to calculate carry faster.

## Add/Subtract

$$\lceil \boxed{M=0} \rightarrow \text{add} \rceil$$

$$\lceil \boxed{M=1} \rightarrow \text{complement } B + \text{add } 1 \rceil$$

## What You Absolutely Must Remember

1. Half Adder equations.
2. Full Adder equations.
3. Full Adder = 2 Half Adders + OR.
4. Half Subtractor equations.
5. Ripple carry delay grows with bit width.
6.  $\lceil G_i = A_i B_i \rceil$ .
7.  $\lceil P_i = A_i \oplus B_i \rceil$  under common convention.
8.  $\lceil C_{i+1} = G_i + P_i C_i \rceil$ .
9. CLA reduces sequential carry propagation.
10. Adder-subtractor uses XOR on B and control as initial carry.
11. Carry flag and signed overflow are different.
12. BCD sums above 9 require +0110 correction.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors '

**Current:** Chapter 12 — Multiplexers

**Remaining:** MUX !' DEMUX/Decoder/Encoder !' Comparator !' Sequential Fundamentals !' Flip-Flops !' FF Conversion !' Race Around !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT

## Chapter 12 — MULTIPLEXERS 0=1 MUST MASTER

Next we cover:

2:1 !' 4:1 !' 8:1 !' 16:1 MUX !' select-line formula !' MUX equation !' implementing Boolean functions using MUX !' choosing select variables !' remaining-variable data inputs !' building larger MUX using smaller MUXes !' minimum-MUX questions !' propagation-delay and GATE traps.

USER

next

ASSISTANT

## CHAPTER 12 — MULTIPLEXERS (MUX) 0=11 MUST MASTER

A **Multiplexer** is one of the highest-yield combinational circuits in GATE.

A MUX selects **one input out of many** and sends that selected input to the output.

Think:

**Many inputs !' one output**

### 1. What Is a Multiplexer?

A multiplexer has:

- data inputs
- select inputs
- one output

If there are:

$2^n$

data inputs, then number of select lines is:

$\boxed{n}$

So:

$\boxed{\text{Select lines} = \log_2(\text{number of inputs})}$

### 2. Examples

| MUX  | Data Inputs | Select Lines | Output |
|------|-------------|--------------|--------|
| 2:1  | 2           | 1            | 1      |
| 4:1  | 4           | 2            | 1      |
| 8:1  | 8           | 3            | 1      |
| 16:1 | 16          | 4            | 1      |
| 32:1 | 32          | 5            | 1      |

Memory trick:

**MUX:  $(2^n)$  inputs need  $(n)$  selectors.**

### 3. 2:1 Multiplexer

Inputs:

$\{I_0, I_1\}$

Select line:

$\{S\}$

Output:

$\{Y\}$

Behavior:

- if  $(S=0)$ , output  $(I_0)$
- if  $(S=1)$ , output  $(I_1)$

Truth table:

| S | Y       |
|---|---------|
| 0 | $(I_0)$ |
| 1 | $(I_1)$ |

### 4. 2:1 MUX Equation

When:

$\{S=0\}$

we want  $(I_0)$ .

Term:

$\{S'I_0\}$

When:

$\{S=1\}$

we want  $(I_1)$ .

Term:

$\{SI_1\}$

Therefore:

$\boxed{Y=S'I_0+SI_1}$

This equation is extremely important.

## 5. Why the Equation Works

If  $(S=0)$ :

$$Y = 1 \cdot I_0 + 0 \cdot I_1$$

$$Y = I_0$$

If  $(S=1)$ :

$$Y = 0 \cdot I_0 + 1 \cdot I_1$$

$$Y = I_1$$

Correct.

## 6. 2:1 MUX Circuit Using Gates

Need:

- NOT gate for  $(S')$
- two AND gates
- one OR gate

Conceptually:

TEXT

```
I 0 % % A N D % % %  
          S ' %  
          O R % % Y  
I 1 % % A N D % % %  
          S
```

Equation:

$$Y = S'I_0 + SI_1$$

## 7. 4:1 Multiplexer

Inputs:

$$I_0, I_1, I_2, I_3$$

Select lines:

$$S_1, S_0$$

Output:

$$Y$$

Truth table:

| $(S_1)$ | $(S_0)$ | Y       |
|---------|---------|---------|
| 0       | 0       | $(I_0)$ |
| 0       | 1       | $(I_1)$ |



| $(S_1)$ | $(S_0)$ | Y       |
|---------|---------|---------|
| 1       | 0       | $(I_2)$ |
| 1       | 1       | $(I_3)$ |

## 8. 4:1 MUX Equation

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

Notice:

The selector terms are exactly minterms of:

$$S_1, S_0$$

## 9. General MUX Equation

For a  $(2^n:1)$  MUX:

$$Y = \sum_{i=0}^{2^n-1} m_i(S) I_i$$

where:

- $m_i(S)$  is the selector minterm
- $I_i$  is corresponding data input

You do not usually need to write the full equation for a large MUX, but understand the structure.

## 10. 8:1 MUX

Inputs:

$$I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7$$

Select lines:

$$S_2, S_1, S_0$$

Because:

$$2^3 = 8$$

Truth relationship:

$$000 \rightarrow I_0$$

$$001 \rightarrow I_1$$

$$010 \rightarrow I_2$$

$$\vdots$$

$$111 \rightarrow I_7$$

## 11. Most Important MUX Formula

If MUX has:

$$\lfloor N \rfloor$$

inputs, where  $(N=2^n)$ ,

then:

$$\boxed{\text{select lines} = \log_2 N}$$

Conversely, if there are  $\lfloor n \rfloor$  select lines:

$$\boxed{\text{data inputs} = 2^n}$$

---

## 12. MUX as a Boolean Function Generator Ø=Ý4

---

A MUX can implement arbitrary Boolean functions.

This is one of the most important GATE applications.

Suppose function:

$$\lfloor F(A,B,C) \rfloor$$

has 3 variables.

One direct way:

Use an:

$$\boxed{8:1 \text{ MUX}}$$

Take:

$$\lfloor A,B,C \rfloor$$

as select lines.

Then put:

$$\lfloor 0 \text{ or } 1 \rfloor$$

on data inputs according to truth table.

---

## 13. Example — Implement Function Using 8:1 MUX

---

Given:

$$\lfloor F(A,B,C) = \Sigma m(1,2,6,7) \rfloor$$

Use:

$$\lfloor A,B,C \rfloor$$

as select lines.

Then data inputs correspond directly to minterm index.

Thus:

$$\lfloor I_0 = 0 \rfloor$$

$$\lfloor I_1 = 1 \rfloor$$

$$\lfloor I_2 = 1 \rfloor$$

$I_3 = 0$

$I_4 = 0$

$I_5 = 0$

$I_6 = 1$

$I_7 = 1$

So the 8:1 MUX implements the function directly.

---

## 14. General Rule

For an  $n$ -variable function:

A:

$\boxed{2^n \text{ MUX}}$

can implement it directly by using all  $n$  variables as select lines and tying data inputs to 0/1.

But GATE often asks something harder:

**Implement an  $n$ -variable function using a smaller MUX.**

Example:

3-variable function using a 4:1 MUX.

That is much more important.

---

## 15. Implementing 3-Variable Function Using 4:1 MUX

A 4:1 MUX has:

2

select lines.

But function has:

3

variables.

So choose any two variables as selectors.

The remaining variable will appear in data inputs as:

0, 1,  $X$ ,  $X'$

This is one of the most important MUX techniques.

---

## 16. Core Strategy

Suppose:

$F(A, B, C)$

Use:

$\{A, B\}$

as select lines.

Then for each selector combination:

$\{AB=00, 01, 10, 11\}$

determine how  $F$  depends on remaining variable  $C$ .

For each case, the data input will become one of:

$\{0, 1, C, \bar{C}\}$

## 17. Example

Implement:

$F(A, B, C) = \sum m(1, 2, 6, 7)$

using 4:1 MUX with  $A, B$  as select lines.

Write truth rows:

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Now group by  $AB$ .

## 18. For $(AB=00)$

Rows:

$C=0 \Rightarrow F=0$

$C=1 \Rightarrow F=1$

This behavior is exactly:

$\{C\}$

Therefore:

$I_0 = C$

---

## 19. For $(AB=01)$

Rows:

$[C=0 \rightarrow F=1]$

$[C=1 \rightarrow F=0]$

This is:

$\boxed{C'}$

Therefore:

$I_1=C'$

---

## 20. For $(AB=10)$

Both outputs:

$0$

Therefore:

$\boxed{I_2=0}$

---

## 21. For $(AB=11)$

Both outputs:

$1$

Therefore:

$\boxed{I_3=1}$

Final implementation:

$\boxed{I_0=C, I_1=C', I_2=0, I_3=1}$

with A,B as select lines.

This is a classic GATE problem.

---

## 22. Fast Pair Pattern

When remaining variable is X, inspect output pair:

| Output for X=0 | Output for X=1 | Data input |
|----------------|----------------|------------|
| 0              | 0              | 0          |
| 1              | 1              | 1          |
| 0              | 1              | X          |
| 1              | 0              | $\neg X$   |

Memorize this table.

It makes smaller-MUX realization very fast.

---

## 23. GATE Memory Trick

---

For the remaining variable X:

$\begin{matrix} 00 \\ \hline \end{matrix} \rightarrow 0$

$\begin{matrix} 11 \\ \hline \end{matrix} \rightarrow 1$

$\begin{matrix} 01 \\ \hline \end{matrix} \rightarrow X$

$\begin{matrix} 10 \\ \hline \end{matrix} \rightarrow X'$

This is extremely useful.

---

## 24. Another Example

---

Implement:

$F(A,B,C) = \sum m(0,2,5,7)$

using 4:1 MUX with A,B selectors.

Write output pairs for C:

**AB=00**

Rows:

$\begin{matrix} m_0=1, \\ m_1=0 \end{matrix}$

Pair:

$\begin{matrix} 10 \end{matrix}$

Thus:

$I_0 = C'$

**AB=01**

Rows:

$\begin{matrix} m_2=1, \\ m_3=0 \end{matrix}$

Again:

$I_1 = C'$

**AB=10**

Rows:

$\begin{matrix} m_4=0, \\ m_5=1 \end{matrix}$

Pair:

$\begin{matrix} 01 \end{matrix}$

Thus:

$$\lceil I_2 = C \rceil$$

**AB=11**

Rows:

$$\lceil m_6 = 0, m_7 = 1 \rceil$$

Thus:

$$\lceil I_3 = C \rceil$$

Therefore:

$$\lceil \boxed{I_0 = I_1 = C', I_2 = I_3 = C} \rceil$$

## 25. Choice of Select Variables Matters

For a function:

$$\lceil F(A, B, C) \rceil$$

you could use:

- A, B as selectors
- A, C
- B, C

All are valid in principle.

But some choices produce simpler data inputs.

Example:

One choice may require:

$$\lceil 0, 1, C, C' \rceil$$

Another may require:

$$\lceil A + B \rceil$$

or more complex expressions if using an even smaller MUX.

So for minimum hardware, selector choice matters.

## 26. Implementing n-Variable Function With $(2^{n-1}:1)$ MUX

General rule:

Use:

$$\lceil n-1 \rceil$$

variables as selectors.

Then each data input is a function of only one remaining variable.

Therefore each input is one of:

$$\lceil \boxed{0, 1, X, X'} \rceil$$

This is extremely convenient.

---

## 27. Implementing n-Variable Function With $(2^{n-2}:1)$ MUX

---

Now only:

$$\lfloor n-2 \rfloor$$

variables are selectors.

Two variables remain.

Therefore each data input may be a Boolean function of two variables.

Possible examples:

$$\lfloor 0, 1, X, X', Y, Y', XY, X+Y, X\oplus Y, \dots \rfloor$$

This becomes more complex.

GATE can ask such questions.

---

## 28. Example — 4-Variable Function With 8:1 MUX

---

For:

$$\lfloor F(A,B,C,D) \rfloor$$

Use:

$$\lfloor A,B,C \rfloor$$

as select lines.

D remains.

Each data input becomes one of:

$$\lfloor 0, 1, D, D' \rfloor$$

This is exactly analogous to 3 variables with a 4:1 MUX.

---

## 29. Direct Shannon Expansion View Ø=βà

---

MUX realization is closely related to Shannon expansion:

$$\lfloor \boxed{F=X'F_{X=0}+XF_{X=1}} \rfloor$$

This is exactly a 2:1 MUX:

$$\lfloor Y=S'I_0+S'I_1 \rfloor$$

Choose:

$$\lfloor S=X \rfloor$$

Then:

$$\lfloor I_0=F_{X=0} \rfloor$$

$$\lfloor I_1=F_{X=1} \rfloor$$

This is the mathematical reason a MUX can implement any Boolean function.

---



---

### 30. Example Using Shannon Expansion

---

Suppose:

$$F(A,B) = A'B + AB'$$

Choose A as select.

When:

$$A = 0$$

$$F = B$$

When:

$$A = 1$$

$$F = B'$$

Thus a 2:1 MUX with:

$$S = A$$

$$I_0 = B$$

$$I_1 = B'$$

implements XOR.

---

### 31. XOR Using 2:1 MUX

---

From above:

$$\boxed{A \oplus B = A'B + AB'}$$

With selector A:

$$I_0 = B$$

$$I_1 = B'$$

So:

$$\boxed{\text{1 MUX + inverter}}$$

if  $B'$  is unavailable.

---

### 32. XNOR Using 2:1 MUX

---

XNOR:

$$AB + A'B'$$

Choose A as select.

If  $A = 0$ :

$$F = B'$$

If  $A = 1$ :

$$F = B$$

Thus:

$$\boxed{I_0 = B', I_1 = B}$$

### 33. Build 4:1 MUX Using 2:1 MUXes $\text{Ø} = 4$

A 4:1 MUX can be built using three 2:1 MUXes.

First level:

- MUX 1 selects between  $(I_0, I_1)$
- MUX 2 selects between  $(I_2, I_3)$

Both controlled by:

$$S_0$$

Second level:

- MUX 3 selects between outputs of first two

controlled by:

$$S_1$$

Total:

$$\boxed{3 \text{ of } 2:1 \text{ MUX}}$$

### 34. General Tree Formula

To build a:

$$2^n:1$$

MUX using 2:1 MUXes:

$$\boxed{2^n - 1}$$

2:1 MUXes are required.

Why?

Example:

8:1:

$$8 - 1 = 7$$

16:1:

$$16 - 1 = 15$$

This is a full binary selection tree.

### 35. Number of Levels

To build:

$$2^n:1$$

MUX using 2:1 MUXes:

$\boxed{n \text{ levels}}$

Because each level resolves one select bit.

Examples:

4:1:

$2 \text{ levels}$

8:1:

$3 \text{ levels}$

16:1:

$4 \text{ levels}$

---

## 36. Delay of MUX Tree

Suppose each 2:1 MUX has delay:

$t$

A  $(2^n:1)$  tree has  $n$  levels.

Therefore worst-case data path delay:

$\boxed{nt}$

Example:

8:1 using 2:1 MUXes:

3 levels.

If each delay = 2 ns:

$\boxed{6 \text{ ns}}$

---

## 37. Build 8:1 Using 4:1 MUXes

One common hierarchy:

- two 4:1 MUXes select among groups of four
- one 2:1 MUX chooses between their outputs

Thus:

$\boxed{2 \times 4:1 + 1 \times 2:1}$

If only 4:1 MUXes are allowed, one 4:1 can sometimes be configured as 2:1 by fixing or duplicating inputs/selects.

But exact answer depends on constraints.

---

## 38. General Larger-MUX Construction

When building a larger MUX using smaller MUXes:

1. partition data inputs into groups

2. use lower select bits inside each group
3. use higher select bits to choose the group

Think:

**low-order select bits choose within group    high-order select bits choose among groups**

## 39. Enable Input

Some MUX ICs have an enable input.

If disabled, output may:

- become 0
- become 1
- high impedance

depending on device.

In theoretical GATE questions, behavior will usually be specified.

Do not assume enable polarity.

## 40. Active-Low Enable

If enable is written:

$\overline{E}$

or has a bubble, it may be active-low.

Meaning:

$E=0$

enables the device.

This is a common hardware-symbol trap.

## 41. MUX as Data Selector

The name "multiplexer" literally means:

**select one of many sources for one destination.**

Applications include:

- choosing ALU operands
- selecting register outputs
- communication routing
- memory buses
- datapath control

For GATE, focus on logic behavior.

---

## 42. MUX vs Decoder

---

Do not confuse:

### MUX

$\boxed{\text{many data inputs !' one output}}$

### Decoder

$\boxed{\text{n-bit input !' one of } 2^n \text{ outputs}}$

MUX selects data.

Decoder activates an output line.

---

## 43. MUX vs DEMUX

---

### MUX

Many inputs !' one output.

### DEMUX

One input !' many outputs.

They are conceptually opposite routing directions.

---

## 44. Minimum MUX Count Questions Ø=Ý4

---

Suppose asked:

**Number of 2:1 MUXes required for a 16:1 MUX?**

Use:

$\lfloor N-1 \rfloor$

where  $(N=16)$ .

Thus:

$\boxed{15}$

---

## 45. Why $\lfloor(N-1)\rfloor$ ?

---

Each 2:1 MUX reduces number of candidate signals by 1.

Starting with N signals, to reduce to one:

$\lfloor N-1 \rfloor$

selection nodes are required.

Also matches a binary tree with N leaves:

---

$\boxed{N-1 \text{ internal nodes}}$

---

## 46. Number of Select Lines for Non-Power-of-Two Inputs

---

Suppose a hypothetical 6:1 MUX.

Minimum bits to select one of six inputs:

$$\lceil \log_2 6 \rceil = 3$$

So:

$$\boxed{3 \text{ select lines}}$$

Some selector combinations remain unused.

However standard MUX sizes are often powers of 2.

---

## 47. Number of Select Combinations

---

With  $n$  select lines:

$$\boxed{2^n}$$

possible selector codes.

So 3 select lines can address:

$$8$$

possible choices.

---

## 48. MUX Function Analysis

---

Suppose 4:1 MUX:

Selectors:

$$A, B$$

Data inputs:

$$I_0 = 0$$

$$I_1 = C$$

$$I_2 = C'$$

$$I_3 = 1$$

Find  $F$ .

Use selector equation:

$$F = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$$

Substitute:

$$F = 0 + A'BC + AB'C + AB$$

Thus:

---

$$\boxed{F = A'BC + AB'C' + AB}$$

Then simplify if needed.

## 49. GATE Faster Method

Instead of expanding immediately, build the truth behavior:

- $AB=00 \rightarrow 0$
- $AB=01 \rightarrow C$
- $AB=10 \rightarrow C'$
- $AB=11 \rightarrow 1$

Then identify minterms if options are given.

This can be faster.

## 50. Example — Find Minterms From MUX

Given:

$$I_0=0, I_1=C, I_2=C', I_3=1$$

Selectors AB.

**AB=00**

No 1s.

**AB=01**

$C=1$  gives:

$$011_2=3$$

**AB=10**

$C'=1$  means  $C=0$ :

$$100_2=4$$

**AB=11**

Both C values:

$$110_2=6$$

$$111_2=7$$

Thus:

$$\boxed{F = \sum m(3, 4, 6, 7)}$$

This is often faster than algebra.

## 51. MUX Implementation From K-Map

Another useful method:

If using a smaller MUX, select variables correspond to some K-map dimensions.

Each MUX data input corresponds to a slice of the function over remaining variables.

For one remaining variable, inspect pairs:

$$\{00,11,01,10\}$$

as:

$$\{0,1,X,X'\}$$

Same method, just visualized through K-map.

---

## 52. L0 Practice

---

How many select lines does a 32:1 MUX need?

$$\{32=2^5\}$$

Therefore:

$$\boxed{5}$$

---

## 53. L0 Practice

---

How many data inputs can 6 select lines choose?

$$\{2^6=64\}$$

Therefore:

$$\boxed{64}$$

---

## 54. L1 Practice

---

2:1 MUX has:

$$\{I_0=A, \quad I_1=B, \quad S=0\}$$

Output:

$$\boxed{A}$$

---

## 55. L1 — 2:1 Equation

---

Given:

$$\{I_0=X, \quad I_1=Y\}$$

Then:

$$\boxed{F=S'X+SY}$$

---

## 56. L1 — Build 8:1

---



How many 2:1 MUXes needed?

$$\lceil \log_2 8 \rceil = 3$$

$$\lceil \log_2 8 \rceil = 3$$

Number of levels:

$$\lceil \log_2 8 \rceil = 3$$

$$\lceil \log_2 8 \rceil = 3$$

---

## 57. L2 — Boolean Function Using 4:1 MUX

---

Given:

$$F(A,B,C) = \sum m(1,3,4,6)$$

Use A,B as selectors.

Rows by AB:

**00**

m0=0, m1=1:

$$01 \rightarrow C$$

Thus:

$$I_0 = C$$

**01**

m2=0, m3=1:

$$I_1 = C$$

**10**

m4=1, m5=0:

$$10 \rightarrow C'$$

$$I_2 = C'$$

**11**

m6=1, m7=0:

$$I_3 = C'$$

Therefore:

$$\boxed{I_0 = C, I_1 = C, I_2 = C', I_3 = C'}$$

---

## 58. L2 — Recognize Simplification

---

Previous implementation actually means:

When A=0, output C.

When A=1, output C'.

So:

$$F = A'C + AC'$$

Hence:

$$\boxed{F = A \oplus C}$$

B does not matter.

Nice GATE observation.

---

## 59. L2 — 4-Variable Using 8:1 MUX

---

Suppose:

$$F(A,B,C,D)$$

Use A,B,C as selectors.

For selector ABC=101:

Relevant minterms are:

$$1010_2=10$$

and:

$$1011_2=11$$

If:

$$F(10)=0$$

and:

$$F(11)=1$$

then data input:

$$\boxed{I_5=D}$$

because  $D = 0$  and  $D = 1$ .

---

## 60. L3 — Pair Pattern Trap

---

For remaining variable X, outputs are:

$$1,0$$

for:

$$X=0,1$$

Then data input is:

$$\boxed{X}$$

Not X.

---

## 61. L3 — Select-Line Trap

---

A 16:1 MUX needs:

$$\lceil \log_2 16 = 4 \rceil$$

select lines.

Do not count 16 select lines.

---

## 62. L3 — MUX Tree Delay

---

64:1 MUX made entirely from 2:1 MUXes.

Levels:

$$\lceil \log_2 64 = 6 \rceil$$

If each MUX delay:

$$1.5 \text{ ns}$$

then data path delay:

$$6 \times 1.5 = 9 \text{ ns}$$

$$\boxed{9 \text{ ns}}$$

---

## 63. L3 — Number of 2:1 MUXes

---

For 64:1:

$$64 - 1 = 63$$

Therefore:

$$\boxed{63}$$

Notice:

- number of MUXes = 63
- levels = 6

Do not confuse them.

---

## 64. GATE PYQ Patterns

---

Typical MUX questions include:

- select-line count
- number of smaller MUXes needed
- implement Boolean function using MUX
- choose data inputs
- identify function produced by a MUX
- find minimum hardware
- calculate MUX tree delay
- realize XOR/XNOR using MUX
- cascade small MUXes

- exploit constants and complemented variables

These are common historical patterns; not tied here to a specific year unless verified.

---

## 65. Trap Book — Chapter 12

---

### Trap 1

For  $(2^n)$  data inputs, using  $(2^n)$  select lines.

Wrong.

$\boxed{n \text{ select lines}}$

### Trap 2

Confusing selector order.

If select lines are  $(S_1, S_0)$ :

$[0 \rightarrow I_0, 01 \rightarrow I_1, 10 \rightarrow I_2, 11 \rightarrow I_3]$

unless question states otherwise.

### Trap 3

Wrong remaining-variable pair mapping.

$[0 \rightarrow 0]$

$[1 \rightarrow 1]$

$[01 \rightarrow X]$

$[10 \rightarrow X']$

### Trap 4

Ignoring which variables were chosen as selectors.

Changing selector variables changes data-input expressions.

### Trap 5

Assuming minimum MUX count equals number of levels.

Not the same.

### Trap 6

For 2:1 MUX tree:

$[N-1]$

MUXes, but:

$\lceil \log_2 N \rceil$

levels.

### Trap 7

Forgetting enable polarity.

Check bubble/notation.

## Trap 8

Expanding algebra when minterm mapping would be faster.

Use selector blocks.

---

## 66. Formula Sheet

---

For a  $(2^n:1)$  MUX:

$\boxed{\text{Select lines}=n}$

$\boxed{\text{Inputs}=2^n}$

2:1 MUX:

$\boxed{Y=S'I_0+SI_1}$

Build N:1 using 2:1 MUXes:

$\boxed{N-1}$

when N is a power of 2.

Tree levels:

$\boxed{\log_2 N}$

MUX tree delay:

$\boxed{T=\log_2 N \cdot t_{\text{MUX}}}$

under equal-delay tree model.

---

## 67. 30-Second Revision

---

Remember:

### MUX

$\boxed{\text{many inputs !' one output}}$

### Select relation

$\boxed{2^n \text{ inputs} \rightarrow n \text{ selects}}$

### 2:1 equation

$\boxed{Y=S'I_0+SI_1}$

### Implement n-variable function

- use  $(2^n:1)$  MUX !' tie data to 0/1
- use  $(2^{n-1}:1)$  MUX !' data inputs become  $(0,1,X,X')$

### Pair rule

$\boxed{00 \rightarrow 0, \quad 11 \rightarrow 1, \quad 01 \rightarrow X, \quad 10 \rightarrow X'}$

### Build larger MUX

$\boxed{N:1 \text{ using } 2:1 \text{ requires } N-1}$

---

---

## What You Absolutely Must Remember

1. MUX selects one of many data inputs.
2.  $(2^n)$  inputs need  $n$  selector bits.
3.  $(Y = S'I_0 + SI_1)$ .
4. A MUX can implement any Boolean function.
5.  $(n)$ -variable function using  $(2^{n-1}:1)$  MUX leaves one variable in data inputs.
6. Memorize the  $(00,11,01,10)$  data-input pair rule.
7. Selector choice can change hardware cost.
8.  $(N:1)$  using  $2:1$  MUXes needs  $(N-1)$  MUXes.
9. Number of tree levels is  $(\log_2 N)$ .
10. Do not confuse MUX with decoder or DEMUX.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors ' !' **MUX** '

**Current: Chapter 13 — DEMUX, Decoder & Encoder**

**Remaining:** DEMUX/Decoder/Encoder !' Comparator !' Sequential Fundamentals !' Latches/Flip-Flops !' FF Conversion !' Race Around !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT

## Chapter 13 — DEMULTIPLEXER, DECODER & ENCODER Ø=Ý4

Next:

**1:2 / 1:4 DEMUX !' decoder as minterm generator !'  $n$ -to- $(2^n)$  decoder !' enable input !' decoder expansion !' Boolean function using decoder !' encoder !' priority encoder !' valid bit !' MUX vs DEMUX vs decoder vs encoder !' GATE implementation and gate-count traps.**

USER

next

ASSISTANT

## CHAPTER 13 — DEMULTIPLEXER, DECODER & ENCODER Ø=Ý4 MUST MASTER

This chapter contains three circuits that students often confuse:

- DEMUX
- Decoder
- Encoder

The easiest way to separate them is:

$\boxed{\text{MUX: many data inputs } \rightarrow \text{ one output}}$

$\boxed{\text{DEMUX: one data input } \rightarrow \text{ one of many outputs}}$

$\boxed{\text{Decoder: binary code } \rightarrow \text{ one active output}}$

$\boxed{\text{Encoder: one active input } \rightarrow \text{ binary code}}$

---

## PART A — DEMULTIPLEXER

---

### 1. What Is a DEMUX?

---

A **Demultiplexer** routes one input to one of several output lines.

Think:

**One input !' many possible destinations**

Only one selected output normally receives the input.

---

### 2. 1:2 DEMUX

---

Inputs:

- Data input  $(D)$
- Select line  $(S)$

Outputs:

$Y_0, Y_1$

Behavior:

If:

$S=0$

then D goes to:

$Y_0$

If:

$S=1$

then D goes to:

$Y_1$

---

### 3. 1:2 DEMUX Equations

---

$\boxed{Y_0 = DS}$

$\boxed{Y_1 = \overline{D}S}$

Truth behavior:

| S | $Y_0$ | $Y_1$ |
|---|-------|-------|
| 0 | D     | 0     |
| 1 | 0     | D     |

## 4. 1:4 DEMUX

One data input:

$D$

Two selectors:

$S_1, S_0$

Four outputs:

$Y_0, Y_1, Y_2, Y_3$

Because:

$2^2=4$

## 5. 1:4 DEMUX Equations

$Y_0 = D S_1' S_0'$

$Y_1 = D S_1' S_0$

$Y_2 = D S_1 S_0'$

$Y_3 = D S_1 S_0$

Thus selector code chooses the output.

## 6. General DEMUX Rule

With  $n$  select lines:

$2^n$  outputs

So:

| Select Lines | Outputs |
|--------------|---------|
| 1            | 2       |
| 2            | 4       |
| 3            | 8       |
| 4            | 16      |



## 7. DEMUX vs MUX

### MUX

$2^n \text{ data inputs} \rightarrow 1 \text{ output}$

### DEMUX

$1 \text{ data input} \rightarrow 2^n \text{ outputs}$

Memory:

**MUX = gather    DEMUX = distribute**

## PART B — DECODER

### 8. What Is a Decoder?

A decoder converts an  $n$ -bit binary input into one of:

$2^n \text{ output lines}$

Normally, exactly one output is active for each input combination.

Example:

2 input bits:

$A, B$

produce:

4

outputs.

Thus:

$2 \text{-to-} 4 \text{ decoder}$

### 9. 2-to-4 Decoder

Inputs:

$A, B$

Outputs:

$Y_0, Y_1, Y_2, Y_3$

Truth table:

| A | B | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
|---|---|-------|-------|-------|-------|
| 0 | 0 | 1     | 0     | 0     | 0     |
| 0 | 1 | 0     | 1     | 0     | 0     |

| A | B | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
|---|---|-------|-------|-------|-------|
| 1 | 0 | 0     | 0     | 1     | 0     |
| 1 | 1 | 0     | 0     | 0     | 1     |

## 10. Decoder Output Equations

$$Y_0 = A'B$$

$$Y_1 = AB$$

$$Y_2 = A'B$$

$$Y_3 = AB$$

Notice something important:

These are exactly:

$$m_0, m_1, m_2, m_3$$

Therefore:

**A decoder is a minterm generator.**

## 11. Decoder as Minterm Generator $2^n = 4$

For an  $n$ -to- $2^n$  decoder:

each output corresponds to exactly one minterm.

Example:

3-to-8 decoder:

$$Y_0 = m_0$$

$$Y_1 = m_1$$

$$\vdots$$

$$Y_7 = m_7$$

This makes decoders extremely useful for implementing Boolean functions.

## 12. Implement Boolean Function Using Decoder

Suppose:

$$F(A,B,C) = \sum m(1,3,6,7)$$

A 3-to-8 decoder gives outputs:

$$Y_0, \dots, Y_7$$

where:

$$Y_i = m_i$$

Therefore:

$$\boxed{F=Y_1+Y_3+Y_6+Y_7}$$

So connect those decoder outputs to an OR gate.

---

## 13. General Decoder Function Realization

---

For:

$$F=\Sigma m(i_1,i_2,\dots)$$

use:

1. an  $n$ -to- $2^n$  decoder
2. OR together outputs corresponding to those minterms

Therefore:

$$\boxed{\text{Decoder + OR can implement canonical SOP}}$$

---

## 14. Example

---

Implement:

$$F(A,B,C)=\Sigma m(0,2,5)$$

using 3-to-8 decoder.

Take:

$$Y_0, Y_2, Y_5$$

Then:

$$\boxed{F=Y_0+Y_2+Y_5}$$

Very straightforward.

---

## 15. Decoder Enable Input

---

Many decoders contain an **enable** input E.

If decoder is enabled:

it behaves normally.

If disabled:

outputs may all become inactive.

For active-high enable:

$$E=1$$

means enabled.

For active-low enable:

$$\overline{E}=0$$

means enabled.

---

Always inspect notation.

---

## 16. Decoder With Enable

---

For a 2-to-4 decoder with active-high enable E:

$$Y_0 = EA'B'$$

$$Y_1 = EA'B$$

$$Y_2 = EAB'$$

$$Y_3 = EAB$$

When:

$$E = 0$$

all outputs become 0.

---

## 17. Active-Low Decoder Outputs

---

Some decoder outputs may be active-low.

Then selected output becomes:

$$\overline{0}$$

instead of 1.

This is often indicated by:

- bubble on output
- bar notation

Example:

$$\overline{Y_0}$$

Do not assume active-high behavior if bubbles are shown.

---

## 18. n-to- $(2^n)$ Decoder Formula

---

$$\boxed{\text{n inputs} \rightarrow 2^n \text{ outputs}}$$

Examples:

| Decoder | Outputs |
|---------|---------|
| 2-to-4  | 4       |
| 3-to-8  | 8       |
| 4-to-16 | 16      |
| 5-to-32 | 32      |

---

## 19. How Many Decoder Inputs Are Needed?

If number of outputs:

$$N = 2^n$$

then:

$$n = \log_2 N$$

Example:

32 outputs:

$$n = 5$$

So:

$$5 \text{ 2-to-32}$$

## 20. Decoder Expansion Ø=4

GATE often asks:

**How many smaller decoders are needed to construct a larger decoder?**

Example:

Build a 3-to-8 decoder using 2-to-4 decoders.

## 21. 3-to-8 From 2-to-4 Decoders

A 3-to-8 decoder has inputs:

$$A, B, C$$

Use one bit, say A, to enable one of two 2-to-4 decoder blocks.

Then B, C are inputs to both smaller decoders.

If:

$$A = 0$$

first decoder active !' outputs 0–3.

If:

$$A = 1$$

second decoder active !' outputs 4–7.

So conceptually need:

$$2 \times \text{2-to-4 decoders}$$

if appropriate enable controls exist and complements are available/generated.

## 22. General Expansion Idea

To build a larger decoder:

- lower-order input bits feed smaller decoders
- higher-order bits control enable lines

Same hierarchy idea as MUX design.

---

## 23. Build 4-to-16 Using 2-to-4 Decoders

---

A common structure:

- one 2-to-4 decoder handles upper 2 bits
- its 4 outputs enable four lower 2-to-4 decoders
- lower 2 bits feed those 4 decoders

Thus:

$\boxed{5 \text{ decoders}}$

assuming every decoder has enable.

One first-stage + four second-stage.

---

## 24. Decoder Tree Formula Concept

---

For hierarchical decoders, exact count depends on:

- decoder sizes
- whether enable exists
- whether active-high/low
- whether extra inverters are required

So don't memorize one universal formula.

Use hierarchy.

---

## 25. Decoder vs DEMUX

---

These circuits look similar.

### Decoder

Has:

$n \text{ input bits}$

and activates one of:

$2^n$

outputs.

### DEMUX

Has:

- one data input D
  - n select bits
-

and routes D to one of:

$2^n$

outputs.

---

## 26. Decoder as DEMUX

A decoder with enable input can behave like a DEMUX.

Use enable E as data input.

Selector bits become decoder input bits.

Then selected output is:

E

while others remain inactive.

So:

$\boxed{\text{Decoder + enable can act as DEMUX}}$

This is a common conceptual question.

---

## PART C — ENCODER

### 27. What Is an Encoder?

An encoder performs roughly the reverse operation of a decoder.

It accepts one active input out of:

$2^n$

input lines and generates an  $n$ -bit binary code.

So:

$\boxed{2^n \text{ inputs} \rightarrow n \text{ output bits}}$

Example:

4-to-2 encoder.

---

### 28. 4-to-2 Encoder

Inputs:

$D_0, D_1, D_2, D_3$

Outputs:

$Y_1, Y_0$

Assume exactly one input active.

Mapping:

---

$\lceil D_0=1 \rightarrow 00 \rceil$

$\lceil D_1=1 \rightarrow 01 \rceil$

$\lceil D_2=1 \rightarrow 10 \rceil$

$\lceil D_3=1 \rightarrow 11 \rceil$

---

## 29. Encoder Truth Table

| Active Input        | $\lceil Y_1 \rceil$ | $\lceil Y_0 \rceil$ |
|---------------------|---------------------|---------------------|
| $\lceil D_0 \rceil$ | 0                   | 0                   |
| $\lceil D_1 \rceil$ | 0                   | 1                   |
| $\lceil D_2 \rceil$ | 1                   | 0                   |
| $\lceil D_3 \rceil$ | 1                   | 1                   |

---

## 30. Encoder Equations

For 4-to-2:

$\lceil \boxed{Y_1 = D_2 + D_3} \rceil$

$\lceil \boxed{Y_0 = D_1 + D_3} \rceil$

Why?

$Y_1=1$  for input 2 or 3.

$Y_0=1$  for input 1 or 3.

---

## 31. Problem With Ordinary Encoder

What if:

$\lceil D_1=1 \rceil$

and:

$\lceil D_3=1 \rceil$

simultaneously?

Ordinary encoder assumes:

$\lceil \boxed{\text{exactly one active input}} \rceil$

If multiple inputs are active, output may become ambiguous.

This leads to:

$\lceil \boxed{\text{Priority Encoder}} \rceil$



## 32. Priority Encoder Ø=Ý4

A priority encoder assigns priority to inputs.

If several inputs are active, the highest-priority input determines output.

Example priority:

$\{ D_3 > D_2 > D_1 > D_0 \}$

If:

$\{ D_3 = 1 \}$

output is always:

$\{ 11 \}$

regardless of lower inputs.

## 33. 4-to-2 Priority Encoder Behavior

Priority:

$\{ D_3 > D_2 > D_1 > D_0 \}$

| $\{ D_3 \}$ | $\{ D_2 \}$ | $\{ D_1 \}$ | $\{ D_0 \}$ | Output |
|-------------|-------------|-------------|-------------|--------|
| 1           | X           | X           | X           | 11     |
| 0           | 1           | X           | X           | 10     |
| 0           | 0           | 1           | X           | 01     |
| 0           | 0           | 0           | 1           | 00     |

X means don't-care.

## 34. What If All Inputs Are 0?

Problem:

$\{ D_0 = 1 \}$

normally gives output:

$\{ 00 \}$

But if no input is active, binary output may also appear as:

$\{ 00 \}$

How can we distinguish?

Use a:

$\{ \boxed{\text{Valid bit}} \}$

## 35. Valid Bit

Valid bit V indicates whether any encoder input is active.

For 4-input encoder:

$$V = D_0 + D_1 + D_2 + D_3$$

If:

$$V = 0$$

no valid input is active.

If:

$$V = 1$$

output code is meaningful.

## 36. Priority Encoder Example

Priority:

$$D_3 > D_2 > D_1 > D_0$$

Suppose:

$$D_3 = 0, D_2 = 1, D_1 = 1, D_0 = 1$$

Highest active input:

$$D_2$$

So output:

$$10$$

Lower active inputs are ignored.

## 37. Encoder General Rule

For:

$$2^n$$

input lines:

output width is:

$$n$$

Examples:

| Encoder | Output bits |
|---------|-------------|
| 4-to-2  | 2           |
| 8-to-3  | 3           |
| 16-to-4 | 4           |

| Encoder | Output bits |
|---------|-------------|
| 32-to-5 | 5           |

### 38. 8-to-3 Encoder

Inputs:

$[D_0, \dots, D_7]$

Outputs:

$[Y_2, Y_1, Y_0]$

If:

$[D_5=1]$

then output is binary 5:

$\boxed{101}$

If:

$[D_7=1]$

output:

$\boxed{111}$

### 39. Decimal-to-BCD Encoder

A decimal-to-BCD encoder has:

$10$

input lines corresponding to decimal digits:

$0 \text{ through } 9$

and outputs 4-bit BCD representation.

Example:

Input line 7 active:

$\boxed{0111}$

This is not exactly a  $(2^n)$ -input encoder because only 10 inputs are used, but the principle is similar.

### 40. Encoder vs Decoder

#### Decoder

Code !' active line.

Example:

$10 \rightarrow Y_2$

## Encoder

Active line !' code.

Example:

$\lceil D_2 \rightarrow 10 \rceil$

Thus they are conceptual inverses.

---

## 41. One-Hot Representation

Decoder outputs often form a:

$\lceil \boxed{\text{one-hot code}} \rceil$

Meaning exactly one output bit is 1.

Example for input 10 in 2-to-4 decoder:

$\lceil 0010 \rceil$

depending on output ordering.

One-hot coding appears frequently in digital systems.

---

## 42. Decoder Output Count

If decoder has 6 inputs:

$\lceil 2^6 = 64 \rceil$

outputs.

Therefore:

$\lceil \boxed{6 \text{-to-} 64} \rceil$

---

## 43. Encoder Input Count

If encoder produces 5 output bits:

$\lceil 2^5 = 32 \rceil$

input lines for a standard full encoder.

Thus:

$\lceil \boxed{32 \text{-to-} 5} \rceil$

---

## 44. Boolean Function Using Decoder — Multiple Outputs

Suppose:

$\lceil F_1 = \Sigma m(1, 2, 5) \rceil$

and:

$\lceil F_2 = \Sigma m(0, 2, 7) \rceil$

---

Use one 3-to-8 decoder.

Then:

$$F_1 = Y_1 + Y_2 + Y_5$$

$$F_2 = Y_0 + Y_2 + Y_7$$

Thus the same decoder can generate minterms for multiple Boolean functions.

Only separate OR networks are needed.

---

## 45. Function Using Active-Low Decoder

---

Suppose decoder outputs are active-low.

Then selected minterm line becomes 0, not 1.

To implement SOP, you may need NAND logic rather than OR.

Example:

If outputs:

$$\bar{m}_1, \bar{m}_3, \bar{m}_5$$

then:

$$F = m_1 + m_3 + m_5$$

Using De Morgan:

$$F = (\bar{m}_1 \bar{m}_3 \bar{m}_5)'$$

So:

$$F = \boxed{\text{NAND selected active-low outputs}}$$

This is a common GATE circuit trick.

---

## 46. Decoder Using NAND Gates

---

A decoder may use NAND gates instead of AND gates.

Then outputs are active-low.

Example 2-to-4 NAND decoder:

$$Y_0 = (A'B)'$$

etc.

Selected output becomes 0.

Always inspect gate type and bubbles.

---

## 47. DEMUX vs Decoder Algebraically

---

1:4 DEMUX:

$$Y_0 = DS_1'S_0'$$

---

Decoder with enable D:

$$Y_0 = DS_1 S_0$$

Exactly the same equation.

Hence:

$$\boxed{\text{Decoder with enable and DEMUX are closely related}}$$

---

## 48. Decoder Propagation Delay

Suppose a decoder uses:

- inverter delay  $t_i$
- AND delay  $t_a$

Some outputs may require complemented input signals.

For a path requiring inversion:

$$T = t_i + t_a$$

For a path using only true inputs:

$$T = t_a$$

Worst-case delay includes the slowest required input path.

---

## 49. Decoder Tree Delay

Large decoders built hierarchically have multiple levels.

If each decoder level takes:

$$t_d$$

and signal passes through L stages:

$$\boxed{T \approx L t_d}$$

as long as the simplified model matches the question.

---

## 50. GATE-Style Example — Decoder Function

A 3-to-8 decoder has active-high outputs.

Output function:

$$F = Y_1 + Y_4 + Y_6$$

Then:

$$\boxed{F = \sum m(1, 4, 6)}$$

No Boolean expansion needed.

---

## 51. GATE-Style Example — Active-Low

Suppose active-low decoder outputs  $(Y_i)$  satisfy:

selected output = 0.

If a NAND gate combines:

$$[Y_2, Y_5, Y_7]$$

Then output:

$$[F = (Y_2 Y_5 Y_7)']$$

Since each:

$$[Y_i = m_i']$$

$$[F = [m_2' m_5' m_7']]$$

By De Morgan:

$$[\boxed{F = m_2 + m_5 + m_7}]$$

Thus:

$$[\boxed{F = \sum m(2, 5, 7)}]$$

Very common conceptual pattern.

---

## 52. Decoder Expansion Example

How many 2-to-4 decoders with enable are needed to construct a 4-to-16 decoder?

Use:

- 1 first-level 2-to-4 decoder
- 4 second-level 2-to-4 decoders

Total:

$$[\boxed{5}]$$

---

## 53. Why 5?

Upper two bits choose one of four lower decoders.

Lower two bits are decoded within selected block.

Thus:

$$[4 \times 4 = 16]$$

outputs.

---

## 54. L0 Practice — DEMUX

A 1:8 DEMUX needs how many select lines?

$$[8 = 2^3]$$

Therefore:

$$[\boxed{3}]$$

---

## 55. L0 — Decoder

A 4-to-16 decoder has:

$$\boxed{4\text{ inputs}, 16\text{ outputs}}$$

---

## 56. L0 — Encoder

An 8-to-3 encoder has:

$$\boxed{8\text{ input lines}, 3\text{ output bits}}$$

---

## 57. L1 — Decoder Equation

For 2-to-4 decoder:

$$Y_2 = ?$$

Index 2:

$$10$$

Therefore:

$$\boxed{Y_2 = AB}$$

---

## 58. L1 — Implement Function

Using 3-to-8 decoder:

$$F = \Sigma m(2, 4, 7)$$

Then:

$$\boxed{F = Y_2 + Y_4 + Y_7}$$

---

## 59. L1 — Encoder

In an ordinary 8-to-3 encoder, if only input  $(D_6)$  is active:

$$6 = 110_2$$

Therefore:

$$\boxed{110}$$

---

## 60. L2 — Priority Encoder

Priority:

$$D_7 > D_6 > \dots > D_0$$

If:



$\lceil D_7=0, \lceil D_6=1, \lceil D_4=1 \rceil$

highest active:

$\lceil D_6 \rceil$

So output:

$\lceil \boxed{110} \rceil$

---

## 61. L2 — Valid Bit

---

For 8 inputs:

$\lceil \boxed{V=D_0+D_1+\cdots+D_7} \rceil$

If all zero:

$\lceil V=0 \rceil$

Otherwise:

$\lceil V=1 \rceil$

---

## 62. L2 — Decoder as DEMUX

---

A 3-to-8 decoder has an enable E.

Treat:

$\lceil E=D \rceil$

as data input.

The 3 decoder inputs become select lines.

Then it functions as:

$\lceil \boxed{1:8\text{ DEMUX}} \rceil$

under compatible active-high behavior.

---

## 63. L3 — Active-Low Trap

---

A decoder output has a bubble.

Selected output therefore becomes:

$\lceil \boxed{0} \rceil$

not 1.

Do not interpret active-low output as ordinary minterm without complementing.

---

## 64. L3 — Ordinary Encoder Ambiguity

---

For a normal 4-to-2 encoder:

$\lceil D_1=1, \lceil D_2=1 \rceil$

---

simultaneously.

Output is not well-defined under the ordinary encoder assumption.

Solution:

$\boxed{\text{Priority Encoder}}$

## 65. L3 — Encoder Output Width

How many output bits are needed to encode 100 input lines?

Need:

$2^n \geq 100$

$2^6 = 64$

$2^7 = 128$

Therefore:

$\boxed{7 \text{ bits}}$

This is not a standard full  $2^n$ -input encoder, but minimum code width is 7.

## 66. MUX vs DEMUX vs Decoder vs Encoder — Master Table

| Circuit | Main Direction                   | Select/Code Relation      | Typical Purpose |
|---------|----------------------------------|---------------------------|-----------------|
| MUX     | Many $\rightarrow$ 1             | $n$ select $2^n$ inputs   | select data     |
| DEMUX   | 1 $\rightarrow$ Many             | $n$ select $2^n$ outputs  | route data      |
| Decoder | $n$ bits $\rightarrow 2^n$ lines | input code activates line | decode/minterms |
| Encoder | $2^n$ lines $\rightarrow n$ bits | active line becomes code  | encode          |

## 67. Memory Trick

### MUX

Many  $\rightarrow$  one

### DEMUX

One  $\rightarrow$  many

### Decoder

Code  $\rightarrow$  line

### Encoder

Line  $\rightarrow$  code

This is enough to separate all four quickly.

---

## 68. GATE PYQ Patterns

---

Typical questions include:

- number of decoder outputs
  - decoder as minterm generator
  - Boolean function using decoder
  - active-low decoder output interpretation
  - decoder expansion
  - decoder as DEMUX
  - ordinary vs priority encoder
  - valid output bit
  - determine code produced by encoder
  - distinguish MUX/DEMUX/decoder/encoder
  - calculate gate or decoder count
- 

## 69. Trap Book — Chapter 13

---

### Trap 1

Decoder has  $2^n$  inputs.

Wrong.

It has:

$n$  inputs,  $2^n$  outputs

### Trap 2

Encoder has  $n$  input lines.

Standard encoder:

$2^n$  inputs,  $n$  outputs

### Trap 3

Decoder and DEMUX are identical in every way.

No.

Decoder uses coded inputs; DEMUX explicitly routes a data input. But decoder with enable can implement DEMUX behavior.

### Trap 4

Ordinary encoder handles multiple active inputs.

No.

Need priority encoder.

### Trap 5

Ignoring active-low bubbles.

Can completely reverse interpretation.

### Trap 6

Assuming enable is active-high.

Check notation.

### Trap 7

For function using active-high decoder, combining selected outputs with AND.

Usually canonical SOP requires OR.

### Trap 8

Don't confuse  $(Y_i)$  with binary value  $(i)$ .

$(Y_i)$  is a physical output line corresponding to minterm  $(i)$ .

---

## 70. Formula Sheet

DEMUX:

$\boxed{n \text{ select lines} \rightarrow 2^n \text{ outputs}}$

Decoder:

$\boxed{n \text{ inputs} \rightarrow 2^n \text{ outputs}}$

Encoder:

$\boxed{2^n \text{ inputs} \rightarrow n \text{ output bits}}$

Decoder function:

$\boxed{Y_i = m_i}$

for active-high decoder.

Function realization:

$\boxed{F = \sum m(i,j,k) \rightarrow F = Y_i + Y_j + Y_k}$

Priority encoder valid bit:

$\boxed{V = \sum i D_i}$

Boolean OR sum.

---

## 71. 30-Second Revision

Remember:

$\boxed{\text{MUX: many ! ' 1}}$

$\boxed{\text{DEMUX: 1 ! ' many}}$

$\boxed{\text{Decoder: code ! ' line}}$

$\boxed{\text{Encoder: line ! ' code}}$

Decoder:

$\lceil n \rceil \rightarrow 2^n$

Encoder:

$2^n \rightarrow n$

And:

$\boxed{\text{Decoder outputs are minterms}}$

Priority encoder solves multiple-active-input ambiguity.

Valid bit tells whether output code is meaningful.

---

## What You Absolutely Must Remember

1. DEMUX routes one input to one selected output.
2.  $n$  select lines give  $2^n$  DEMUX outputs.
3. Decoder has  $n$  inputs and  $2^n$  outputs.
4. Decoder output  $(Y_i)$  corresponds to minterm  $(m_i)$ .
5. Decoder + OR implements SOP functions.
6. Decoder enable can make it act like DEMUX.
7. Encoder performs roughly inverse coding.
8. Ordinary encoder assumes one active input.
9. Priority encoder handles multiple active inputs.
10. Valid bit distinguishes "no input active" from a legitimate zero code.
11. Active-low decoder outputs require careful complement handling.
12. Hierarchical decoder expansion uses upper bits as enables and lower bits as local decoder inputs.

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors ' !' MUX ' !' **DEMUX/Decoder/Encoder** '

**Current: Chapter 14 — Comparators & Other Combinational Circuits**

**Remaining:** Comparator !' Sequential Fundamentals !' Latches/Flip-Flops !' FF Conversion !' Race Around !' Registers !' Counters !' Sequential Analysis/Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

**NEXT**

---

## Chapter 14 — COMPARATORS& OTHER COMBINATIONAL CIRCUITS Ø=βà

Next:

**1-bit comparator !' equality/greater-than/less-than equations !' multi-bit magnitude comparison !' cascading comparators !' priority of MSB !' parity generator/checker !' code converters where useful !' GATE comparison traps.**

---

USER

next

## CHAPTER 14 — COMPARATORS & OTHER COMBINATIONAL CIRCUITS Ø=βà IMPORTANT

A **comparator** checks the relationship between binary numbers.

For two numbers  $\backslash(A\backslash)$  and  $\backslash(B\backslash)$ , a magnitude comparator usually produces:

$\backslash(A > B \backslash)$

$\backslash(A = B \backslash)$

$\backslash(A < B \backslash)$

This chapter is smaller than MUX/decoders, but the concepts are useful and can appear inside larger GATE circuit questions.

### 1. 1-Bit Comparator

Let:

$\backslash(A, B \in \{0, 1\} \backslash)$

We need three outputs:

- $\backslash(G\backslash)$ :  $A > B$
- $\backslash(E\backslash)$ :  $A = B$
- $\backslash(L\backslash)$ :  $A < B$

### 2. Truth Table

| A | B | $\backslash(A > B\backslash)$ | $\backslash(A = B\backslash)$ | $\backslash(A < B\backslash)$ |
|---|---|-------------------------------|-------------------------------|-------------------------------|
| 0 | 0 | 0                             | 1                             | 0                             |
| 0 | 1 | 0                             | 0                             | 1                             |
| 1 | 0 | 1                             | 0                             | 0                             |
| 1 | 1 | 0                             | 1                             | 0                             |

### 3. Equality Output

A equals B when:

- both 0
- both 1

Therefore:

$\backslash(E = A'B' + AB \backslash)$

This is XNOR.

So:

$$\boxed{A=B=A \odot B}$$

or:

$$\boxed{(A \oplus B)}$$

---

## 4. Greater-Than Output

---

$A > B$  only for:

$$\boxed{A=1, B=0}$$

Therefore:

$$\boxed{G=AB'}$$

---

## 5. Less-Than Output

---

$A < B$  only for:

$$\boxed{A=0, B=1}$$

Therefore:

$$\boxed{L=A'B}$$

---

## 6. 1-Bit Comparator Master Equations

---

$$\boxed{A > B = AB'}$$

$$\boxed{A = B = AB + A'B' = A \odot B}$$

$$\boxed{A < B = A'B}$$

These are easy but worth remembering.

---

## 7. Important Property

---

Exactly one of these must be true:

$$\boxed{A > B, \quad A = B, \quad A < B}$$

Therefore:

$$\boxed{G + E + L = 1}$$

for valid binary inputs.

And they are mutually exclusive:

$$\boxed{GE = GL = EL = 0}$$

---

## 8. Multi-Bit Comparator

---

Suppose:

$A = A_1A_0$

$B = B_1B_0$

The most important rule is:

**Compare the most significant differing bit.**

For unsigned binary numbers, the first differing bit from the left decides the result.

## 9. Example

Compare:

$A = 10_2$

$B = 01_2$

MSBs:

$A_1 = 1, B_1 = 0$

Immediately:

$\boxed{A > B}$

The LSBs do not matter.

## 10. Another Example

Compare:

$A = 1011$

$B = 1001$

From left:

First bit:

$1 = 1$

Second:

$0 = 0$

Third:

$1 > 0$

Therefore:

$\boxed{A > B}$

No need to inspect the last bit.

## 11. 2-Bit Equality



Two 2-bit numbers are equal only if both corresponding bits are equal.

Thus:

$$\boxed{A=B}$$

when:

$$(A_1 \odot B_1)=1$$

and:

$$(A_0 \odot B_0)=1$$

Therefore:

$$\boxed{E=(A_1 \odot B_1)(A_0 \odot B_0)}$$

---

## 12. General n-Bit Equality

For:

$$A_{n-1} \dots A_0$$

and:

$$B_{n-1} \dots B_0$$

Equality:

$$\boxed{E=\prod_{i=0}^{n-1}(A_i \odot B_i)}$$

In words:

**XNOR every corresponding bit, then AND all results.**

---

## 13. 2-Bit Greater-Than Equation

$A > B$  if:

### Case 1

MSB decides:

$$A_1=1, B_1=0$$

Term:

$$A_1 B_1'$$

### Case 2

MSBs equal and LSB decides:

$$(A_1 \odot B_1) A_0 B_0'$$

Therefore:

$$\boxed{G=A_1 B_1' + (A_1 \odot B_1) A_0 B_0'}$$

## 14. 2-Bit Less-Than

---

Similarly:

$$\boxed{L = A_1'B_1 + (A_1 \odot B_1)A_0'B_0}$$

---

## 15. General Comparator Logic

---

For n-bit comparison:

Start from MSB.

At bit  $i$ :

A decides it is greater if:

$$A_i = 1, B_i = 0$$

and all higher-order bits are equal.

Conceptually:

$$A > B$$

if:

$$A_{n-1} > B_{n-1}$$

or:

$$A_{n-1} = B_{n-1}$$

and next bit decides, etc.

---

## 16. Lexicographic View

---

Unsigned binary comparison works like comparing words lexicographically:

**First position where they differ decides the order.**

Example:

$$101101$$

vs

$$101011$$

Common prefix:

$$101$$

Next bits:

$$1 > 0$$

Therefore first number larger.

---

## 17. Why MSB Has Highest Priority

---

Because positional weight:

$$[ 2^{n-1} ]$$

is larger than the sum of all lower bit weights:

$$[ 2^{n-2} + \dots + 2^0 ]$$

Indeed:

$$[ 2^{n-2} + \dots + 1 = 2^{n-1} - 1 ]$$

So one higher-order 1 outweighs every lower-order bit combined.

---

## 18. Cascading Comparators

---

Large comparators can be constructed from smaller comparator blocks.

A comparator may accept cascade inputs such as:

- $(I_{A>B})$
- $(I_{A=B})$
- $(I_{A<B})$

which represent comparison results from less-significant blocks.

Higher-order block has priority.

---

## 19. Cascading Principle

---

Suppose comparing two 8-bit numbers using two 4-bit comparators.

Split:

- upper nibble: bits 7–4
- lower nibble: bits 3–0

First compare upper 4 bits.

If upper parts differ:

$$[ \boxed{\text{upper comparator decides}} ]$$

Only if upper parts are equal do lower bits matter.

---

## 20. Example

---

A:

$$[ 1010,0000 ]$$

B:

$$[ 1001,1111 ]$$

Upper 4 bits:

$$[ 1010 > 1001 ]$$

Therefore:

---

$\boxed{A > B}$

Lower nibbles are irrelevant.

---

## 21. Signed vs Unsigned Comparison

---

Be careful.

Bitwise magnitude comparison described above assumes:

$\boxed{\text{unsigned numbers}}$

For signed 2's-complement numbers, MSB is sign-related.

Example 4-bit:

$1000$

unsigned:

$8$

signed 2's complement:

$-8$

Therefore interpretation matters.

---

## 22. Signed Comparison Shortcut

---

For 2's-complement numbers:

### Different sign bits

- sign 0 !' non-negative
- sign 1 !' negative

So if A sign = 0 and B sign = 1:

$\boxed{A > B}$

If A sign = 1 and B sign = 0:

$\boxed{A < B}$

If signs are same, comparison of remaining structure can proceed appropriately.

---

## 23. Signed Comparison Trap

---

Compare 4-bit patterns:

$A = 1001$

$B = 0111$

Unsigned:

$A = 9, B = 7$

So unsigned:

---

$$\lfloor A > B \rfloor$$

Signed 2's complement:

$$\lfloor A = -7, B = +7 \rfloor$$

So signed:

$$\lfloor \boxed{A < B} \rfloor$$

Same bits, different interpretation.

---

## 24. Comparator Implementation Using Subtraction

---

A processor can compare A and B by computing:

$$\lfloor A - B \rfloor$$

Then inspect:

- zero
- sign
- carry/borrow
- overflow

depending on signed/unsigned comparison.

This avoids requiring a completely separate comparator in some architectures.

For Digital Logic, understand the idea; detailed CPU flag semantics belong more to COA.

---

## 25. Equality via XOR

---

If two corresponding bits differ:

$$\lfloor A_i \oplus B_i = 1 \rfloor$$

So numbers are equal if all XOR results are 0.

Thus:

$$\lfloor A = B \rfloor$$

can also be written:

$$\lfloor \boxed{[(A_{n-1} \oplus B_{n-1}) + \dots + (A_0 \oplus B_0)]'} \rfloor$$

Equivalent to XNOR-AND structure.

---

## 26. Example — 4-Bit Equality Detector

---

Inputs:

$$\lfloor A_3 A_2 A_1 A_0 \rfloor$$

and:

$$\lfloor B_3 B_2 B_1 B_0 \rfloor$$

Compute:

---

$$\forall X_i = A_i \oplus B_i$$

If any  $\forall(X_i = 1)$ , numbers differ.

So:

$$\boxed{E = (X_3 + X_2 + X_1 + X_0) \% 2}$$

## PART B — PARITY GENERATOR/CHECKER

### 27. What Is Parity?

Parity adds one extra bit to help detect certain transmission/storage errors.

There are two common conventions:

- even parity
- odd parity

### 28. Even Parity

The total number of 1s, including the parity bit, should be even.

Example data:

$$\forall 1011$$

Number of 1s:

$$\forall 3$$

odd.

To make total even, parity bit must be:

$$\forall 1$$

giving 4 ones.

### 29. Odd Parity

The total number of 1s including parity bit should be odd.

For:

$$\forall 1011$$

already 3 ones.

Therefore odd-parity bit:

$$\forall 0$$

### 30. XOR and Parity

XOR outputs 1 when number of input 1s is odd.

Therefore for data bits:

$$\{ D_0, D_1, \dots, D_{n-1} \}$$

XOR:

$$D_0 \oplus D_1 \oplus \dots$$

is 1 when input has odd count of 1s.

---

## 31. Even-Parity Bit

---

To make total number of 1s even:

$$\boxed{P_e = D_0 \oplus D_1 \oplus \dots \oplus D_{n-1}}$$

Why?

If data has odd number of 1s, XOR=1, so parity bit adds one more '1' even.

If data has even number, XOR=0.

---

## 32. Odd-Parity Bit

---

Odd parity bit is complement:

$$\boxed{P_o = (D_0 \oplus D_1 \oplus \dots \oplus D_{n-1})'}$$

## 33. Example

---

Data:

$$\{ 1101 \}$$

Number of 1s = 3.

XOR:

$$1 \oplus 1 \oplus 0 \oplus 1 = 1$$

Even parity bit:

$$\boxed{1}$$

Odd parity bit:

$$\boxed{0}$$

---

## 34. Parity Checker

---

A parity checker XORs all received data bits plus parity bit.

For even parity:

If transmission is consistent:

$$\boxed{\text{overall XOR} = 0}$$

If overall XOR = 1:

---

an odd number of bit errors has occurred.

---

## 35. What Parity Can Detect

---

Single-bit error changes parity.

Therefore:

$\boxed{\text{single-bit errors are detected}}$

More generally:

$\boxed{\text{any odd number of bit flips is detected}}$

---

## 36. What Parity Cannot Detect

---

If two bits flip, parity changes twice and returns to original parity.

Thus ordinary parity cannot detect all even-numbered errors.

Example:

$1010 \rightarrow 0110$

two bits changed.

Parity may remain same.

Therefore:

$\boxed{\text{simple parity cannot detect every even-number error}}$

---

## 37. Parity Does Not Correct Errors

---

Simple parity tells:

**Something may be wrong.**

It generally does not tell:

**Which bit is wrong.**

So:

$\boxed{\text{detection, not correction}}$

Error-correcting codes such as Hamming code go beyond this basic combinational topic.

---

## PART C — CODE CONVERTERS

---

### 38. Code Converter

---

A code converter is a combinational circuit that converts one binary coding scheme into another.

---



Examples:

- Binary !' Gray
- Gray !' Binary
- BCD !' Excess-3

These are standard combinational design exercises.

---

## 39. Binary to Gray Conversion Ø=Ý4

---

For binary:

$$\backslash [ B_{\{n-1\}} B_{\{n-2\}} \ldots B_0 \backslash ]$$

Gray bits:

$$\backslash [ G_{\{n-1\}} = B_{\{n-1\}} \backslash ]$$

and:

$$\backslash [ \boxed{G_i = B_{\{i+1\}} \oplus B_i} \backslash ]$$

for lower positions.

---

## 40. 4-Bit Binary !' Gray

---

Binary:

$$\backslash [ B_3 B_2 B_1 B_0 \backslash ]$$

Gray:

$$\backslash [ \boxed{G_3 = B_3} \backslash ]$$

$$\backslash [ \boxed{G_2 = B_3 \oplus B_2} \backslash ]$$

$$\backslash [ \boxed{G_1 = B_2 \oplus B_1} \backslash ]$$

$$\backslash [ \boxed{G_0 = B_1 \oplus B_0} \backslash ]$$

---

## 41. Example

---

Convert binary:

$$\backslash [ 1011 \backslash ]$$

to Gray.

First bit:

$$\backslash [ G_3 = 1 \backslash ]$$

Then adjacent XORs:

$$\backslash [ 1 \oplus 0 = 1 \backslash ]$$

$$\backslash [ 0 \oplus 1 = 1 \backslash ]$$

$$\backslash [ 1 \oplus 1 = 0 \backslash ]$$

Thus:

---

$$\boxed{1110}$$


---

## 42. Binary-to-Gray Shortcut

---

Keep MSB unchanged.

Then XOR every pair of adjacent binary bits.

Memory:

**Keep first, XOR neighbors.**

---

## 43. Gray to Binary Conversion Ø=Ý4

---

For Gray:

$$G_{n-1} \dots G_0$$

Binary MSB:

$$B_{n-1} = G_{n-1}$$

Then:

$$\boxed{B_i = B_{i+1} \oplus G_i}$$

So binary bits are reconstructed cumulatively.

---

## 44. Example — Gray to Binary

---

Gray:

$$1110$$

First:

$$B_3 = 1$$

Then:

$$B_2 = B_3 \oplus G_2$$

$$= 1 \oplus 1 = 0$$

Next:

$$B_1 = 0 \oplus 1 = 1$$

Next:

$$B_0 = 1 \oplus 0 = 1$$

Thus:

$$\boxed{1011}$$

Correct.

---

## 45. Gray-Code Property

Adjacent Gray-code values differ in exactly one bit.

This is why Gray code was used earlier in:

$\boxed{\text{K-map ordering}}$

So this chapter reconnects to K-map fundamentals.

## 46. BCD to Excess-3

Excess-3 code is obtained by adding:

$\boxed{3}$

to each decimal digit represented in BCD.

Example decimal 5:

BCD:

$\boxed{0101}$

Add 3:

$\boxed{0101+0011=1000}$

Therefore Excess-3 of 5:

$\boxed{1000}$

## 47. Excess-3 Examples

| Decimal | BCD  | Excess-3 |
|---------|------|----------|
| 0       | 0000 | 0011     |
| 1       | 0001 | 0100     |
| 2       | 0010 | 0101     |
| 3       | 0011 | 0110     |
| 4       | 0100 | 0111     |
| 5       | 0101 | 1000     |
| 6       | 0110 | 1001     |
| 7       | 0111 | 1010     |
| 8       | 1000 | 1011     |
| 9       | 1001 | 1100     |

Invalid BCD inputs 1010–1111 may be used as don't-cares in converter design.

## 48. Code Converter Design Procedure

---

Exactly like any combinational design:

1. List input code words.
  2. List corresponding output code words.
  3. Make one truth table.
  4. Derive one Boolean function per output bit.
  5. Use K-map.
  6. Implement minimized equations.
- 

## PART D — GATE PRACTICE

---

### 49. L0 — Comparator

---

For:

$\{ A=1, B=0 \}$

Outputs:

$\{ A>B=1 \}$

$\{ A=B=0 \}$

$\{ A<B=0 \}$

---

### 50. L0 — Equality

---

For one-bit comparator:

$\{ A=B \}$

is implemented using:

$\{ \boxed{\text{XNOR}} \}$

---

### 51. L1 — 2-Bit Equality

---

$\{ A=10 \}$

$\{ B=10 \}$

Corresponding XNORs:

both 1.

AND:

$\{ 1 \}$

Therefore:

$\{ \boxed{A=B} \}$

---

## 52. L1 — Magnitude

---

Compare:

$\lfloor 10101 \rfloor$

and:

$\lfloor 10011 \rfloor$

First differing position from left:

third bit:

$\lfloor 1 > 0 \rfloor$

Therefore:

$\lfloor \boxed{10101 > 10011} \rfloor$

---

## 53. L2 — Equation

---

For 2-bit unsigned numbers, which expression contributes to  $(A > B)$  when MSBs are equal?

$\lfloor \boxed{(A_1 \odot B_1)A_0B_0'} \rfloor$

Because lower bit matters only if higher bits are equal.

---

## 54. L2 — Signed Trap

---

4-bit patterns:

$\lfloor A = 1110 \rfloor$

$\lfloor B = 0010 \rfloor$

Unsigned:

$\lfloor 14 > 2 \rfloor$

Signed 2's complement:

$\lfloor -2 < 2 \rfloor$

Therefore answer depends on interpretation.

---

## 55. L1 — Even Parity

---

Data:

$\lfloor 1010 \rfloor$

Number of 1s:

$\lfloor 2 \rfloor$

already even.

Therefore even-parity bit:

$\lfloor \boxed{0} \rfloor$

---

---

## 56. L2 — Error Detection

---

Can one parity bit detect three flipped bits?

Three is odd.

Parity changes.

Therefore:

$\boxed{\text{Yes}}$

Can it always detect two flipped bits?

$\boxed{\text{No}}$

---

## 57. L1 — Binary to Gray

---

Binary:

$\boxed{1101}$

First bit:

$\boxed{1}$

Then XOR neighbors:

$\boxed{1 \oplus 1 = 0}$

$\boxed{1 \oplus 0 = 1}$

$\boxed{0 \oplus 1 = 1}$

Gray:

$\boxed{1011}$

---

## 58. L2 — Gray to Binary

---

Gray:

$\boxed{1011}$

Binary:

First:

$\boxed{1}$

Next:

$\boxed{1 \oplus 0 = 1}$

Next:

$\boxed{1 \oplus 1 = 0}$

Next:

$\boxed{0 \oplus 1 = 1}$

---

Therefore:

$$\boxed{1101}$$

---

## 59. L3 — Comparator Critical Idea

Two unsigned 16-bit numbers differ only at bit 12 and lower bits.

At bit 12:

$$A_{12}=1, B_{12}=0$$

All bits above 12 equal.

Then immediately:

$$\boxed{A > B}$$

Lower 12 bits cannot reverse the result.

---

## 60. L3 — Equality Gate Count Idea

To compare two 8-bit numbers for equality using 2-input XNORs and an 8-input AND:

Need:

$$8 \text{ XNOR gates}$$

plus:

$$1 \text{ AND gate}$$

assuming 8-input AND allowed.

If only 2-input ANDs allowed, additional gate tree is required.

Again fan-in constraints matter.

---

## 61. GATE PYQ Patterns

Common question patterns include:

- derive 1-bit comparator equations
  - multi-bit equality
  - identify first significant differing bit
  - cascade comparator blocks
  - signed vs unsigned comparison
  - parity generation/checking
  - determine which errors parity detects
  - binary to decimal conversion
  - code-converter truth tables
  - don't-cares for invalid BCD inputs
-

## 62. Trap Book — Chapter 14

---

### Trap 1

Comparing binary strings from LSB first.

Wrong.

For unsigned magnitude comparison, start at:

$\boxed{\text{MSB}}$

### Trap 2

Using unsigned comparison for signed 2's-complement values.

Interpretation matters.

### Trap 3

Equality using XOR directly.

XOR gives difference.

Equality uses:

$\boxed{\text{XNOR}}$

### Trap 4

Assuming lower bits can override a higher differing bit.

They cannot for unsigned comparison.

### Trap 5

Even-parity bit is always complement of XOR.

Wrong.

Even-parity generator bit is XOR of data bits.

### Trap 6

Parity detects every multi-bit error.

False.

It detects odd-numbered bit errors, not all even-numbered errors.

### Trap 7

Binary! ~~And yG r a y!~~ Binary identical procedure.

No.

Binary! ~~Uses~~ adjacent binary XOR.

Gray! Binary accumulative.

### Trap 8

Invalid BCD inputs must be assigned fixed outputs in a converter.

Often they may be treated as don't-cares if specification allows.



## 63. Formula Sheet

### 1-Bit Comparator

$$\boxed{A > B = AB'}$$

$$\boxed{A = B = AB + A'B'}$$

$$\boxed{A < B = A'B}$$

### n-Bit Equality

$$\boxed{E = \prod_i (A_i \odot B_i)}$$

### Even-Parity Bit

$$\boxed{P_e = D_0 \oplus D_1 \oplus \dots \oplus D_{n-1}}$$

### Odd-Parity Bit

$$\boxed{P_o = P_e'}$$

### Binary to Gray

$$\boxed{G_{n-1} = B_{n-1}}$$

$$\boxed{G_i = B_{i+1} \oplus B_i}$$

### Gray to Binary

$$\boxed{B_{n-1} = G_{n-1}}$$

$$\boxed{B_i = G_{i+1} \oplus G_i}$$

## 64. 30-Second Revision

Comparator:

**First differing MSB decides.**

Equality:

$$\boxed{\text{XNOR corresponding bits, then AND}}$$

Parity:

$$\boxed{\text{XOR detects odd number of 1s}}$$

Binary to Gray:

**keep MSB, XOR neighbors.**

Gray to Binary:

keep MSB, cumulative XOR.

## What You Absolutely Must Remember

- 1-bit comparator equations.
- MSB has comparison priority.
- Multi-bit equality = XNOR each pair + AND.
- Signed and unsigned comparison can give opposite answers.
- Even parity bit is XOR of the data bits.
- Simple parity detects any odd number of bit errors.
- Simple parity does not correct errors.
- Binary! Keep first bit, XOR adjacent bits.
- Gray! Binary! Cumulative XOR.
- Code converters are ordinary combinational circuits designed using truth table + K-map + logic.

## Digital Logic Progress

**Completed:** Number Systems + Signed Representation + Binary Arithmetic + Boolean Algebra + SOP/POS + Universal Gates + Algebraic Minimization + K-map + Tabular Method + Combinational Fundamentals + Adders/Subtractors + MUX + Decoder/Encoder/DEMUX + **Comparators & Supporting Combinational Circuits**

**Current:** Chapter 15 — Sequential Circuit Fundamentals

**Remaining:** Sequential Fundamentals + Latches/Flip-Flops + FF Conversion + Race Around/Master-Slave + Registers + Counters + Sequential Analysis + Sequential Design + FSM + Sequence Detectors + Fixed Point + Floating Point + IEEE-754

**NEXT**

## Chapter 15 — SEQUENTIAL CIRCUIT FUNDAMENTALS 0=MUST MASTER

Next we begin the other major half of Digital Logic:

memory + state + feedback + combinational vs sequential + synchronous/asynchronous + clock waveform + positive/negative edge + level triggering + propagation delay + setup time + hold time + clock-to-Q delay + clock skew + timing constraints + maximum clock-frequency numericals.

USER

next

ASSISTANT

## CHAPTER 15 — SEQUENTIAL CIRCUIT FUNDAMENTALS 0=MUST MASTER

Now we enter the second major half of Digital Logic:

## $\boxed{\text{Sequential Circuits}}$

This chapter is crucial because everything ahead depends on it:

- latches
- flip-flops
- registers
- counters
- finite-state machines
- sequence detectors
- timing numericals

The defining idea is:

**A sequential circuit has memory.**

## 1. Combinational vs Sequential Logic

### Combinational

Output depends only on present inputs:

$$\boxed{Y=f(X)}$$

Example:

- adder
- MUX
- decoder

### Sequential

Output can depend on:

- present input
- previous stored state

Mathematically:

$$\boxed{Y=f(X,Q)}$$

and next state:

$$\boxed{Q^+=g(X,Q)}$$

where:

- $Q$  = present state
- $Q^+$  = next state

## 2. Meaning of State

**State** is the information remembered by the circuit.

Example:

Suppose a circuit must output 1 after it has previously seen the sequence:

$\{101\}$

It must remember some information about earlier inputs.

That remembered information is its state.

### 3. Memory Element

Sequential circuits contain storage devices such as:

- latches
- flip-flops
- registers

These store binary state.

One flip-flop generally stores:

$\{1\text{ bit}\}$

So  $\{n\}$  flip-flops can store:

$\{n\text{ bits}\}$

and potentially represent:

$\{2^n\text{ states}\}$

### 4. General Sequential Circuit Structure

TEXT

```

% % % % % % % % % % % % % % % % % % %
I n p u t s % % % % % % C o m b i n a t i o n a l L o g i c % % % % % % O u t p u t s
% % % % % % % % % % % % % % % % % % %
% N e x t S t a t e
%
% % % % % % % % % % % % % % % % % % %
% S t a t e M e m o r y %
% % % % % % % % % % % % % % % % % % %
% P r e s e n t S t a t e
% % % % % % % % % % % % % % % % % % % b a c k t o l o g i c
```

This feedback of stored state is what makes sequential behavior possible.

### 5. Present State and Next State

Suppose current flip-flop output is:

$Q(t)$

After the appropriate triggering event, it becomes:

$Q(t+1)$

We often write:

$$\boxed{Q^{+}=Q(t+1)}$$

Present state:

$$Q$$

Next state:

$$Q^{+}$$

This notation becomes extremely important in flip-flop equations.

---

## 6. Feedback

Sequential circuits typically use feedback.

That means some output/state information is fed back into the circuit.

In combinational logic, uncontrolled feedback is generally avoided.

In sequential logic, controlled feedback is fundamental.

---

## 7. Synchronous vs Asynchronous Sequential Circuits

Two broad categories:

### Synchronous

State changes are coordinated by a clock.

$$\boxed{\text{State normally changes at defined clock events}}$$

Examples:

- synchronous counters
  - registers
  - most processor state elements
- 

### Asynchronous

State can change directly when inputs change.

No global clock is necessarily used.

These circuits are harder to design reliably because timing becomes more complicated.

For GATE Digital Logic, synchronous circuits are much more important.

---

## 8. Clock Signal

A clock is a periodic binary waveform.

Conceptually:

---

TEXT

1 % % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % %  
0 % % % % % % % % % % % % % % % % % % % % %  
! ' ! '  
edges

It repeatedly alternates between:

$\leftarrow$

## 9. Clock Period

The time for one complete clock cycle is:

$T_{\text{clk}}$

If frequency is:

$f$

then:

$f = \frac{1}{T_{\text{clk}}}$

and:

$T_{\text{clk}} = \frac{1}{f}$

## 10. Example

Clock frequency:

100 MHz

Then:

$T = \frac{1}{100 \times 10^6}$

$= 10^{-8}$  s

10 ns

Useful shortcut:

$1 \text{ GHz} \rightarrow 1 \text{ ns}$

So:

- 500 MHz !' 2 ns
- 250 MHz !' 4 ns
- 2 GHz !' 0.5 ns

## 11. Rising Edge

Transition:

$\rightarrow$

is called:

- rising edge
- positive edge
- positive-going edge

Symbol often shown:

$\uparrow$

---

## 12. Falling Edge

---

Transition:

$\downarrow$

is called:

- falling edge
- negative edge
- negative-going edge

Symbol:

$\downarrow$

---

## 13. Edge-Triggered Device

---

An edge-triggered flip-flop responds at a clock edge.

### Positive-edge triggered

Updates around:

$\boxed{0 \rightarrow 1}$

### Negative-edge triggered

Updates around:

$\boxed{1 \rightarrow 0}$

Between active edges, state remains stored.

---

## 14. Level-Sensitive Device

---

A latch is generally **level-sensitive**.

Example:

An active-high latch may be transparent while:

$\text{CLK}=1$

During this interval, input changes can affect output.

When clock becomes inactive, the value is held.

---

## 15. Latch vs Flip-Flop — Preview

---

A simplified GATE distinction:

### Latch

$\boxed{\text{Level-sensitive}}$

### Flip-flop

$\boxed{\text{Edge-triggered}}$

This distinction is very important.

We will study latches and flip-flops properly next chapter.

---

## 16. Transparent Latch

---

If a latch is enabled:

input can "flow through" to output.

Hence the term:

$\boxed{\text{transparent}}$

Example:

For an active-high D latch:

When:

$E=1$

approximately:

$Q=D$

When:

$E=0$

Q retains previous value.

---

## 17. Why Clocking Is Useful

---

Without coordinated clocking, many signals may change at different times because of propagation delays.

Clocking creates defined moments when state is sampled.

This makes complex digital systems easier to reason about.

---

## 18. Propagation Delay

---

Flip-flop output does not change instantaneously after the clock edge.

The delay from active clock edge to valid Q output is:

$\boxed{t_{CQ}}$

also called:

---



$t_{clk \rightarrow Q}$

---

## 19. Clock-to-Q Delay

---

Example:

Positive clock edge occurs at:

$10 \text{ ns}$

and:

$t_{CQ} = 2 \text{ ns}$

Then new Q may become valid around:

$12 \text{ ns}$

---

## 20. Setup Time $\varnothing = 4$

---

Input data must be stable for some minimum time **before** the active clock edge.

This is called:

$t_{\text{setup}}$

Example:

Active edge at:

$10 \text{ ns}$

setup time:

$2 \text{ ns}$

Then data should already be stable from approximately:

$8 \text{ ns}$

until the edge.

---

## 21. Hold Time $\varnothing = 4$

---

Input data must remain stable for some minimum time **after** the active clock edge.

This is:

$t_{\text{hold}}$

If active edge at 10 ns and:

$t_{\text{hold}} = 1 \text{ ns}$

data should remain stable until at least:

$11 \text{ ns}$

---

## 22. Setup + Hold Window

---

Around the clock edge:

TEXT

```
      stable      stable
% % % % % % % % % % % % % | % % % % % % % % % % % % %
      CLK edge
    <setup>  |  <hold>
```

Input should not change inside this critical sampling window.

Memory trick:

**Setup = before clock   Hold = after clock**

## 23. Why Setup/Hold Matter

If input changes too close to the clock edge, the flip-flop may not reliably decide whether it sampled 0 or 1.

This can cause:

$\boxed{\text{metastability}}$

## 24. Metastability

A flip-flop normally resolves to:

$0 \text{ or } 1$

But if timing requirements are violated, it may temporarily enter an unstable/intermediate condition before eventually resolving.

Important:

**Metastability cannot be eliminated completely in asynchronous sampling, but its probability can be reduced.**

For GATE, mostly know the concept and why setup/hold violations are dangerous.

## 25. Register-to-Register Path $\Delta t$

This timing model is extremely important.

Imagine:

TEXT

```
FF1 % % °   C o m b i n a t i o n a l   L o g i c   % % °   FF2
```

At one clock edge:

FF1 launches data.

Data travels through:

1. FF1 clock-to-Q delay

## 2. combinational logic delay

Then FF2 must receive it early enough to satisfy setup time before next clock edge.

---

## 26. Setup Constraint

For correct operation:

$$T_{\text{clk}} \geq t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}$$

This is the basic no-skew setup equation.

Where:

- $t_{\text{CQ}}^{\text{max}}$  = maximum clock-to-Q delay
- $t_{\text{comb}}^{\text{max}}$  = maximum combinational path delay
- $t_{\text{setup}}$  = setup time of destination flip-flop

---

## 27. Maximum Clock Frequency

Since:

$$T_{\text{clk}}^{\text{min}} = t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}$$

then:

$$f_{\text{max}} = \frac{1}{t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}}$$

This is extremely high-yield for GATE numericals.

---

## 28. Example — Maximum Frequency

Given:

$$t_{\text{CQ}} = 2 \text{ ns}$$

$$t_{\text{comb}} = 5 \text{ ns}$$

$$t_{\text{setup}} = 1 \text{ ns}$$

Minimum period:

$$T_{\text{clk}} = 2 + 5 + 1$$

$$= 8 \text{ ns}$$

Thus:

$$f_{\text{max}} = \frac{1}{8 \text{ ns}}$$

$$= 125 \text{ MHz}$$

Therefore:

$$\boxed{125 \text{ MHz}}$$

---

## 29. GATE Frequency Shortcut

If period is in ns:

$$\boxed{f(\text{MHz}) = \frac{1000}{T(\text{ns})}}$$

Example:

$$T = 8 \text{ ns}$$

$$f = \frac{1000}{8} = 125 \text{ MHz}$$

Fast.

---

## 30. Hold-Time Constraint

Hold checks whether the **new data arrives too early** after the same clock edge.

Basic no-skew constraint:

$$\boxed{t_{CQ}^{\min} + t_{\text{comb}}^{\min} \geq t_{\text{hold}}}$$

Important differences:

Setup uses:

$$\boxed{\text{maximum delays}}$$

Hold uses:

$$\boxed{\text{minimum delays}}$$

---

## 31. Why Minimum Delay for Hold?

Hold asks:

**Could new data reach the destination too soon and disturb the value that was just sampled?**

So we analyze the fastest possible path.

Hence:

$$t_{CQ}^{\min}$$

and:

$$t_{\text{comb}}^{\min}$$

---

## 32. Example — Hold Check

Suppose:

$$t_{CQ}^{\min} = 0.5 \text{ ns}$$

$$t_{\text{comb}}^{\min} = 0.8 \text{ ns}$$

$$t_{\text{hold}} = 1 \text{ ns}$$

Then:

$$0.5 + 0.8 = 1.3 \text{ ns}$$

Since:

$t_{CQ} \geq 1$

hold requirement is satisfied.

### 33. Hold Violation Example

Suppose:

$t_{CQ}^{\min} = 0.2 \text{ ns}$

$t_{\text{comb}}^{\min} = 0.3 \text{ ns}$

$t_{\text{hold}} = 1 \text{ ns}$

Then:

$0.5 < 1$

So:

$\boxed{\text{hold violation}}$

### 34. Fixing Hold Violation

One standard method:

**Add delay to the short/fast data path.**

This is different from setup violations.

To fix setup, we often need:

- faster logic
- longer clock period

To fix hold, increasing the clock period generally does **not** solve the same-edge hold violation.

This is a classic GATE trap.

### 35. Setup vs Hold — Master Comparison

| Feature                 | Setup                      | Hold                |
|-------------------------|----------------------------|---------------------|
| Stability required      | Before edge                | After edge          |
| Delay considered        | Maximum                    | Minimum             |
| Main issue              | Data too late              | Data too early      |
| Helped by slower clock? | Yes                        | Usually no          |
| Common fix              | longer period/faster logic | add data-path delay |

## 36. Critical Path

---

For synchronous timing:

The register-to-register path with largest:

$$t_{CQ} + t_{comb} + t_{setup}$$

determines minimum clock period.

So the longest combinational path is often called the:

$$\boxed{\text{critical path}}$$

---

## 37. Example With Multiple Paths

---

Path 1:

$$t_{CQ}=1, \quad t_{comb}=5, \quad t_{setup}=1$$

Total:

$$7 \text{ ns}$$

Path 2:

$$1+8+1=10 \text{ ns}$$

Path 3:

$$1+4+1=6 \text{ ns}$$

Minimum allowed clock period:

$$\boxed{10 \text{ ns}}$$

Maximum frequency:

$$\boxed{100 \text{ MHz}}$$

Path 2 is critical.

---

## 38. Clock Skew ~~0=0~~ IMPORTANT

---

Ideally, the clock reaches all flip-flops simultaneously.

In reality, clock may arrive at different times.

Difference in arrival times is:

$$\boxed{\text{clock skew}}$$

Suppose clock reaches FF1 at:

$$t=0$$

and FF2 at:

$$t=0.5 \text{ ns}$$

Then there is:

$$0.5 \text{ ns}$$

---

relative skew.

---

### 39. Positive Skew Convention

---

For a path from launch FF to capture FF, define:

$$\boxed{t_{\text{skew}} = t_{\text{capture}} - t_{\text{launch}}}$$

So positive skew means destination/capture clock arrives later.

---

### 40. Setup Constraint With Skew

---

With the above convention:

$$\boxed{T_{\text{clk}} \geq t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}} - t_{\text{skew}}}$$

Positive skew helps setup because the destination samples later.

Negative skew hurts setup.

---

### 41. Hold Constraint With Skew

---

Using the same convention:

$$\boxed{t_{\text{CQ}}^{\text{min}} + t_{\text{comb}}^{\text{min}} \geq t_{\text{hold}} + t_{\text{skew}}}$$

So positive skew hurts hold because destination clock arrives later, increasing the interval during which old data must remain valid relative to launch.

This setup-vs-hold tradeoff is important.

---

### 42. Skew Memory Trick

---

Under:

$$t_{\text{skew}} = t_{\text{capture}} - t_{\text{launch}}$$

Positive skew:

- **helps setup**
- **hurts hold**

Negative skew:

- **hurts setup**
  - **helps hold**
- 

### 43. Example With Positive Skew

---

Given:

$$t_{\text{CQ}} = 1 \text{ ns}$$

$$t_{\text{comb}} = 7 \text{ ns}$$

---

$$t_{\text{setup}} = 1 \text{ ns}$$

$$t_{\text{skew}} = 2 \text{ ns}$$

Minimum clock:

$$T \geq 1 + 7 + 1 - 2$$

$$\boxed{7 \text{ ns}}$$

Without skew it would be 9 ns.

So positive skew helped setup.

## 44. Hold With Same Positive Skew

Suppose:

$$t_{\text{CQ}}^{\text{min}} = 1 \text{ ns}$$

$$t_{\text{comb}}^{\text{min}} = 2 \text{ ns}$$

$$t_{\text{hold}} = 1 \text{ ns}$$

$$t_{\text{skew}} = 2 \text{ ns}$$

Need:

$$1 + 2 \geq 1 + 2$$

$$3 \geq 3$$

Just satisfied.

## 45. Clock Jitter ~~0=64~~ SUPPORT

Clock edges may not occur at perfectly exact periodic times.

Small timing variations are called:

$$\boxed{\text{clock jitter}}$$

In detailed timing analysis, jitter reduces available timing margin.

For core GATE logic, skew is more commonly emphasized.

## 46. Setup Slack

A useful concept:

$$\boxed{\text{Setup Slack} = T_{\text{available}} - T_{\text{required}}}$$

Without skew, one common form:

$$\boxed{\text{Slack} = T_{\text{clk}} - (t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}})}$$

Positive slack !' timing satisfied.

Negative slack !' violation.



## 47. Example

---

Clock period:

$$\lfloor 10 \text{ ns} \rfloor$$

Required path:

$$\lfloor 2+5+1=8 \text{ ns} \rfloor$$

Slack:

$$\lfloor 10-8=2 \text{ ns} \rfloor$$

$$\lfloor \boxed{+2 \text{ ns}} \rfloor$$

Timing is safe by 2 ns.

---

## 48. Setup Violation

---

Suppose:

$$\lfloor T_{\text{clk}}=6 \text{ ns} \rfloor$$

but path requires:

$$\lfloor 8 \text{ ns} \rfloor$$

Then data arrives too late.

Violation:

$$\lfloor \boxed{2 \text{ ns}} \rfloor$$

You may need:

- slower clock
  - faster combinational logic
  - pipelining
  - faster cells
- 

## 49. Pipelining Preview

---

Suppose combinational delay is:

$$\lfloor 20 \text{ ns} \rfloor$$

If we insert an intermediate register and split into approximately:

$$\lfloor 10+10 \rfloor$$

ns stages, maximum clock frequency can increase.

Tradeoff:

- higher throughput
- more registers
- increased latency in cycles

This concept will matter more in COA/pipelining, but it follows directly from sequential timing.

---



sample D.

Then after:

$t_{CQ}$

Q becomes that sampled value.

Changes in D between active edges do not immediately change Q.

---

## 54. Example

---

Positive-edge D FF.

Suppose D values at consecutive rising edges are:

$1, 0, 1, 1$

Then Q after each edge becomes:

$\boxed{1, 0, 1, 1}$

ignoring initial state and adding appropriate  $t_{CQ}$ .

---

## 55. GATE Trap — Sampling Just Before Edge

---

If D changes exactly around active edge, setup/hold information determines whether sampling is valid.

Do not casually assume the new or old value if timing requirement is violated.

Question may explicitly state setup/hold or ideal device.

---

## 56. Clock Duty Cycle

---

Duty cycle is fraction of period clock stays high.

$\boxed{\text{Duty Cycle} = \frac{T_{\text{high}}}{T_{\text{period}}} \times 100\%}$

For a symmetric square wave:

$\boxed{50\%}$

This matters more for level-sensitive latches and some timing circuits.

---

## 57. Example

---

Clock:

$T = 10 \text{ ns}$

High for:

$4 \text{ ns}$

Duty cycle:

$\frac{4}{10} \times 100$

$\boxed{40\%}$

---

---

## 58. Frequency and Period Conversions

---

Useful:

$$1 \text{ MHz} = 10^6 \text{ Hz}$$

$$1 \text{ GHz} = 10^9 \text{ Hz}$$

$$1 \text{ ns} = 10^{-9} \text{ s}$$

Fast relationship:

$$T(\text{ns}) = \frac{1000}{f(\text{MHz})}$$

---

## 59. Example

---

Frequency:

$$250 \text{ MHz}$$

Period:

$$\frac{1000}{250}$$

$$4 \text{ ns}$$

---

## 60. Number of States from Flip-Flops $\emptyset = 4$

---

If circuit has:

$$n$$

binary state bits / flip-flops, maximum possible state combinations:

$$2^n$$

Example:

3 flip-flops:

$$2^3 = 8$$

possible binary states.

Important:

A particular sequential circuit may intentionally use fewer than all possible states.

---

## 61. Minimum Flip-Flops for N States

---

To encode N states:

$$n = \lceil \log_2 N \rceil$$

Example:

10 states:

$$\log_2 10 \approx 3.32$$

Need:

$\boxed{4 \text{ flip-flops}}$

This will matter heavily in counters/FSMs.

---

## 62. Unused States

---

If 3 flip-flops are used for a 6-state machine:

Possible states:

$\boxed{8}$

Used:

$\boxed{6}$

Unused:

$\boxed{2}$

Questions may ask what happens if machine enters unused state.

That relates to self-starting/self-correcting design later.

---

## 63. Synchronous State Transition

---

In a synchronous circuit:

Between active clock edges:

$Q = \text{stable state}$

At active edge:

next state is captured.

Then after  $t_{CQ}$ :

$Q \rightarrow Q^+$

So conceptually:

$\boxed{\text{Compute next state during cycle, capture at next edge}}$

---

## 64. State Equation

---

A sequential system can be represented as:

$\boxed{Q^+ = f(Q, X)}$

and output as:

$\boxed{Y = g(Q, X)}$

or sometimes output depends only on state:

$\boxed{Y = g(Q)}$

These lead to Mealy and Moore machines later.

---

---

## 65. Memoryless vs Stateful Example

Suppose output:

$$Y = A \oplus B$$

No memory.

Combinational.

Suppose output toggles every time input pulse occurs.

To know what output should become, circuit must know its current value.

So state is required.

Sequential.

---

## 66. L0 Practice

Which quantity tells how long data must be stable before clock edge?

A. Hold time B. Setup time C. Clock-to-Q D. Skew

Answer:

$$\boxed{B}$$

---

## 67. L0 Practice

Which is after the edge?

$$\boxed{t_{\text{hold}}}$$

---

## 68. L1 — Frequency

Clock period:

$$5 \text{ ns}$$

Frequency:

$$\frac{1000}{5} = 200 \text{ MHz}$$

$$\boxed{200 \text{ MHz}}$$

---

## 69. L1 — Number of States

5 flip-flops can represent:

$$2^5 = 32$$

states.

$$\boxed{32}$$

---

---

## 70. L1 — Flip-Flops Needed

A machine needs 13 states.

$$\lceil 2^3 = 8 < 13 \rceil$$

$$\lceil 2^4 = 16 \geq 13 \rceil$$

Therefore:

$$\boxed{4}$$

flip-flops.

---

## 71. L2 — Maximum Frequency

Given:

$$t_{CQ}^{\max} = 1.5 \text{ ns}$$

$$t_{\text{comb}}^{\max} = 6 \text{ ns}$$

$$t_{\text{setup}} = 0.5 \text{ ns}$$

Minimum period:

$$1.5 + 6 + 0.5 = 8 \text{ ns}$$

Therefore:

$$\boxed{f_{\max} = 125 \text{ MHz}}$$

---

## 72. L2 — Hold Check

Given:

$$t_{CQ}^{\min} = 0.4 \text{ ns}$$

$$t_{\text{comb}}^{\min} = 0.3 \text{ ns}$$

$$t_{\text{hold}} = 1 \text{ ns}$$

Available:

$$0.7 \text{ ns}$$

Required:

$$1 \text{ ns}$$

Thus:

$$\boxed{\text{Hold violation of } 0.3 \text{ ns}}$$

---

## 73. L2 — Fix Which Violation?

Suppose setup fails.

Possible fix:

---

$\boxed{\text{increase clock period}}$

Suppose hold fails.

Simply increasing clock period:

$\boxed{\text{does not normally fix same-cycle hold violation}}$

Instead add delay to the too-fast path or alter clock/data timing.

---

## 74. L3 — Setup With Skew

---

Given:

$t_{CQ}=1\text{ ns}$

$t_{comb}=8\text{ ns}$

$t_{setup}=1\text{ ns}$

positive skew:

$2\text{ ns}$

Then:

$T_{clk} \geq 1+8+1-2$

$\boxed{8\text{ ns}}$

Frequency:

$\boxed{125\text{ MHz}}$

---

## 75. L3 — Hold With Positive Skew

---

Given:

$t_{CQ}^{\min}=0.5$

$t_{comb}^{\min}=1$

$t_{hold}=0.5$

$t_{skew}=1.2$

Need:

$0.5+1 \geq 0.5+1.2$

$1.5 \geq 1.7$

False.

Hold violation:

$\boxed{0.2\text{ ns}}$

---

## 76. L3 — Critical Path

---

Three combinational paths have maximum delays:

---



$\lceil 4, \lceil 7, \lceil 5 \text{ ns} \rceil \rceil$

Common:

$\lceil t_{\text{CQ}}=1 \text{ ns} \rceil$

$\lceil t_{\text{setup}}=1 \text{ ns} \rceil$

Critical clock period:

$\lceil 1+7+1=9 \text{ ns} \rceil$

Thus:

$\lceil f_{\text{max}}=\frac{1000}{9} \text{ MHz} \rceil$

$\lceil \boxed{\approx 111.1 \text{ MHz}} \rceil$

---

## 77. GATE PYQ Patterns

Sequential fundamentals commonly appear as:

- combinational vs sequential classification
- number of states from flip-flops
- minimum flip-flops for states
- setup/hold interpretation
- register-to-register timing
- maximum clock frequency
- critical-path analysis
- clock skew effects
- timing diagrams
- latch vs flip-flop distinction
- propagation-delay numericals

These are PYQ-style patterns; no specific year is claimed here.

---

## 78. Trap Book — Chapter 15

### Trap 1

Sequential circuit output depends only on current input.

Wrong.

It can depend on stored state.

### Trap 2

Every sequential circuit needs a clock.

Wrong.

Asynchronous sequential circuits exist.

### Trap 3

Latch = edge-triggered.

Usually wrong.

Latch is level-sensitive.

#### **Trap 4**

Flip-flop = level-sensitive.

Standard edge-triggered flip-flops respond to edges.

#### **Trap 5**

Setup means after edge.

Wrong.

$\boxed{\text{Setup = before}}$

#### **Trap 6**

Hold means before edge.

Wrong.

$\boxed{\text{Hold = after}}$

#### **Trap 7**

Use maximum delay for hold.

Wrong.

Hold uses minimum path delay.

#### **Trap 8**

Use minimum delay for setup.

Wrong.

Setup uses maximum path delay.

#### **Trap 9**

Slower clock always fixes hold.

Wrong.

#### **Trap 10**

Ignore clock-to-Q delay in frequency calculation.

Wrong.

#### **Trap 11**

Number of states from n FFs is n.

Wrong.

$\boxed{2^n}$

#### **Trap 12**

Positive skew always good.

No.

With our convention:

- helps setup
- hurts hold

---

## 79. Sequential Timing Formula Sheet

---

Clock:

$$\boxed{f = \frac{1}{T}}$$

State capacity:

$$\boxed{n \text{ FFs} \rightarrow 2^n \text{ possible states}}$$

Minimum FFs for N states:

$$\boxed{\lceil \log_2 N \rceil}$$

Setup constraint without skew:

$$\boxed{T_{\text{clk}} \geq t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}}$$

Maximum frequency:

$$\boxed{f_{\text{max}} = \frac{1}{t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}}}$$

Hold:

$$\boxed{t_{\text{CQ}}^{\text{min}} + t_{\text{comb}}^{\text{min}} \geq t_{\text{hold}}}$$

With skew  $(t_{\text{skew}} = t_{\text{capture}} - t_{\text{launch}})$ :

$$\boxed{T_{\text{clk}} \geq t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}} - t_{\text{skew}}}$$

$$\boxed{t_{\text{CQ}}^{\text{min}} + t_{\text{comb}}^{\text{min}} \geq t_{\text{hold}} + t_{\text{skew}}}$$

---

## 80. 30-Second Revision

---

Sequential logic remembers.

$$\boxed{Q^{+} = f(Q, X)}$$

Remember:

- latch !' level
- flip-flop !' edge
- setup !' before
- hold !' after
- clock-to-Q !' edge to output
- setup uses maximum delay
- hold uses minimum delay
- positive skew helps setup but hurts hold
- $n$  FFs !'  $2^n$  possible states

And the biggest timing formula:

$$\boxed{T_{\text{clk}}^{\text{min}} = t_{\text{CQ}}^{\text{max}} + t_{\text{comb}}^{\text{max}} + t_{\text{setup}}}$$

---

## What You Absolutely Must Remember

1. Sequential circuits contain memory/state.
2. Present state and next state are different concepts.
3. Synchronous circuits coordinate state transitions with a clock.
4. Latches are level-sensitive; flip-flops are normally edge-triggered.
5. Setup time is before the sampling edge.
6. Hold time is after the sampling edge.
7. Clock-to-Q is delay from clock edge to valid output.
8. Setup timing is a longest-path problem.
9. Hold timing is a shortest-path problem.
10. Slowing clock can fix setup, not normally hold.
11. Critical path determines maximum clock frequency.
12.  $n$  flip-flops can represent at most  $2^n$  states.
13. Minimum FFs for  $N$  states:

$\lceil \log_2 N \rceil$

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' **Sequential Fundamentals** '

**Current: Chapter 16 — Latches & Flip-Flops**

**Remaining:** Latches/Flip-Flops !' FF Conversion !' Race Around/Master-Slave !' Registers !' Counters !' Sequential Analysis !' Sequential Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT — Chapter 16: Latches & Flip-Flops Ø=Ý4

We will cover:

SR latch !' gated SR latch !' D latch !' SR/JK/D/T flip-flops !' truth tables !' characteristic tables !' characteristic equations !' excitation tables !' timing diagrams !' asynchronous preset/clear !' toggle behavior !' all high-frequency GATE traps.

---

USER

next

ASSISTANT

## CHAPTER 16 — LATCHES & FLIP-FLOPS Ø=Ý4 MUST MASTER

This is one of the most important chapters in Sequential Logic.

You must become comfortable with:

- SR latch
- gated SR latch
- D latch
- SR flip-flop
- JK flip-flop
- D flip-flop
- T flip-flop
- characteristic tables
- characteristic equations
- excitation tables
- timing diagrams
- asynchronous preset/clear

The single most important idea is:

$\boxed{\text{A flip-flop stores 1 bit}}$

Its current output is:

$Q$

and its next output is:

$Q^+$

## 1. Latch vs Flip-Flop

### Latch

Generally:

$\boxed{\text{level-sensitive}}$

It responds during an active enable level.

### Flip-Flop

Generally:

$\boxed{\text{edge-triggered}}$

It updates at an active clock edge.

Memory:

**Latch = Level**

**Flip-Flop = Edge**

## 2. SR Latch — Basic Memory Element

SR stands for:

- S = Set
- R = Reset

The latch stores one bit.

Two outputs are usually shown:

$Q, \overline{Q}$

Ideally:

$\overline{Q} = \overline{Q}$

### 3. NOR-Based SR Latch

A standard active-high SR latch can be built using two cross-coupled NOR gates.

Inputs:

$S, R$

Behavior:

**S = 1**

Set:

$Q = 1$

**R = 1**

Reset:

$Q = 0$

**S=R=0**

Hold previous state.

**S=R=1**

Forbidden / invalid condition.

### 4. NOR SR Latch Truth Table

| S | R | $Q^{n+1}$ | Meaning   |
|---|---|-----------|-----------|
| 0 | 0 | Q         | Hold      |
| 0 | 1 | 0         | Reset     |
| 1 | 0 | 1         | Set       |
| 1 | 1 | Invalid   | Forbidden |

Memorize:

$\boxed{00 = \text{Hold}}$

$\boxed{01 = \text{Reset}}$

$\boxed{10 = \text{Set}}$

$\boxed{11 = \text{Invalid}}$

for an **active-high NOR SR latch**.

## 5. Why Is $(S=R=1)$ Invalid in NOR Latch?

A NOR output is 0 if any input is 1.

If both external inputs are 1, both latch outputs are forced toward:

$Q=0, \quad Q'=0$

But normally:

$Q'=Q'$

must be complement of Q.

Having:

$Q=Q'=0$

breaks the complementary-output condition.

More importantly, when S and R return simultaneously to 0, final state may depend on gate delays.

Therefore:

$\boxed{S=R=1 \text{ is forbidden for NOR SR latch}}$

## 6. NAND-Based SR Latch $\overline{S}, \overline{R}$

A NAND SR latch normally uses **active-low inputs**:

$\overline{S}, \quad \overline{R}$

Sometimes written:

$S', R'$

This reverses the input interpretation.

## 7. NAND SR Latch Table

| $\overline{S}$ | $\overline{R}$ | $Q^{n+1}$ | Meaning   |
|----------------|----------------|-----------|-----------|
| 1              | 1              | Q         | Hold      |
| 0              | 1              | 1         | Set       |
| 1              | 0              | 0         | Reset     |
| 0              | 0              | Invalid   | Forbidden |

So:

$\boxed{11 = \text{Hold}}$

$\boxed{01 = \text{Set}}$

$\boxed{10 = \text{Reset}}$

$\boxed{00 = \text{Invalid}}$

for active-low NAND latch inputs.

## 8. Critical GATE Trap

Do not memorize only:

**"11 is invalid for SR."**

That is true for an active-high NOR SR latch.

For an active-low NAND SR latch:

$\boxed{00 \text{ is invalid}}$

Always inspect:

- gate type
- bubbles
- active-high vs active-low notation

## 9. Gated SR Latch

A basic SR latch responds whenever S/R change.

A **gated SR latch** introduces enable input E.

When:

$E=0$

the latch holds.

When:

$E=1$

S and R control the state.

For an active-high gated SR latch:

| E | S | R | Action  |
|---|---|---|---------|
| 0 | X | X | Hold    |
| 1 | 0 | 0 | Hold    |
| 1 | 0 | 1 | Reset   |
| 1 | 1 | 0 | Set     |
| 1 | 1 | 1 | Invalid |



---

## 10. D Latch Ø=Ý4

The SR latch has an unwanted invalid input combination.

D latch removes that problem.

D = Data.

Use:

$$\boxed{S=D}$$

$$\boxed{R=D'}$$

Therefore S and R can never be 1 simultaneously.

---

## 11. D Latch Behavior

For active-high enable E:

**E=1**

Latch is transparent:

$$\boxed{Q^{+}=D}$$

**E=0**

Latch holds:

$$\boxed{Q^{+}=Q}$$

Table:

| E | D | $Q^{+}$ |
|---|---|---------|
| 0 | X | Q       |
| 1 | 0 | 0       |
| 1 | 1 | 1       |

---

## 12. D Latch Characteristic Equation

$$\boxed{Q^{+}=ED+E'Q}$$

Why?

If:

$$\boxed{E=1}$$

$$\boxed{Q^{+}=D}$$

If:

$$\boxed{E=0}$$

$$\boxed{Q^{+}=Q}$$

This is exactly a 2:1 MUX equation.

Interesting connection:

A D latch next-state equation behaves like a MUX selecting between old Q and new D.

## 13. Transparent Behavior

Suppose E remains 1 for a long interval.

If D changes:

$[0 \text{ to } 1 \text{ to } 0]$

Q can follow those changes after propagation delay.

This is why a latch is called **transparent** while enabled.

A flip-flop behaves differently: it samples at an edge.

## 14. Edge-Triggered Flip-Flops

A flip-flop updates only at its designated clock edge.

Common types:

- SR
- JK
- D
- T

We need four things for each:

1. characteristic table
2. characteristic equation
3. excitation table
4. behavior shortcut

## 15. Characteristic Table

A **characteristic table** answers:

Given current state and current input, what is the next state?

So:

$\boxed{(Q, \text{ inputs}) \rightarrow Q^{+}}$

## 16. Excitation Table

An **excitation table** answers the reverse question:

Given present state  $Q$  and desired next state  $Q^{+}$ , what flip-flop inputs are required?

So:

$\boxed{Q, Q^{+} \rightarrow \text{required inputs}}$

This becomes essential in sequential circuit design and FF conversion.

## 17. SR Flip-Flop

Inputs:

$S, R$

Characteristic behavior:

| S | R | $Q^{+}$ |
|---|---|---------|
| 0 | 0 | Q       |
| 0 | 1 | 0       |
| 1 | 0 | 1       |
| 1 | 1 | Invalid |

Same logical behavior as active-high SR latch, but sampled at clock edge.

## 18. SR Characteristic Equation

For valid inputs:

$\boxed{Q^{+} = S + R'Q}$

subject to:

$SR \neq 1$

Check:

**S=1, R=0**

$Q^{+} = 1 + 1Q = 1$

Set.

**S=0, R=1**

$Q^{+} = 0 + 0Q = 0$

Reset.

**S=0, R=0**

$Q^{+} = 0 + 1Q = Q$

Hold.

## 19. SR Excitation Table $Q \rightarrow Q^+$

We ask: what inputs are needed for each transition?

| Q | $Q^+$ | S | R |
|---|-------|---|---|
| 0 | 0     | 0 | X |
| 0 | 1     | 1 | 0 |
| 1 | 0     | 0 | 1 |
| 1 | 1     | X | 0 |

Why X?

Example:

For:

$Q \rightarrow 0$

Either:

$S=0, R=0$

holds 0, or:

$S=0, R=1$

resets to 0.

Thus R doesn't matter:

$R = X$

## 20. SR Memory Trick

Transitions:

$0 \rightarrow \boxed{\text{Set}}$

$1 \rightarrow \boxed{\text{Reset}}$

So excitation is intuitive.

## 21. JK Flip-Flop $Q \rightarrow Q^+$ MUST MASTER

JK flip-flop improves SR by removing the invalid condition.

Inputs:

$J, K$

Characteristic table:

| J | K | $Q^+$ | Action |
|---|---|-------|--------|
| 0 | 0 | Q     | Hold   |

| J | K | $Q^{n+1}$ | Action |
|---|---|-----------|--------|
| 0 | 1 | 0         | Reset  |
| 1 | 0 | 1         | Set    |
| 1 | 1 | $Q^n$     | Toggle |

This is fundamental.

## 22. JK Memory Trick

$\boxed{00 = \text{Hold}}$

$\boxed{01 = \text{Reset}}$

$\boxed{10 = \text{Set}}$

$\boxed{11 = \text{Toggle}}$

Compare SR:

SR has 11 invalid.

JK turns that case into toggle.

## 23. JK Characteristic Equation

$Q^{n+1} = JQ^n + K'Q^n$

This is extremely important.

Check:

**J=K=0**

$Q^{n+1} = 0 + 1Q^n = Q^n$

Hold.

**J=1,K=0**

$Q^{n+1} = Q^n + Q^n = 1$

Set.

**J=0,K=1**

$Q^{n+1} = 0 + 0 = 0$

Reset.

**J=K=1**

$Q^{n+1} = Q^n$

Toggle.

## 24. JK Excitation Table Ø=Ý4

| Q | $\mathcal{Q}^{+}$ | J | K |
|---|-------------------|---|---|
| 0 | 0                 | 0 | X |
| 0 | 1                 | 1 | X |
| 1 | 0                 | X | 1 |
| 1 | 1                 | X | 0 |

Notice compared with SR, more don't-cares appear because JK has no invalid input combination.

## 25. Fast JK Excitation Memory

When current:

$\mathcal{Q}=0$

K doesn't matter much because reset of 0 still gives 0.

When current:

$\mathcal{Q}=1$

J often becomes don't-care.

But safest is to memorize the table.

## 26. D Flip-Flop Ø=Ý4

D flip-flop is the simplest.

Input:

$\mathcal{D}$

At active edge:

$\boxed{\mathcal{Q}^{+}=\mathcal{D}}$

Characteristic table:

| D | $\mathcal{Q}^{+}$ |
|---|-------------------|
| 0 | 0                 |
| 1 | 1                 |

Independent of current  $\mathcal{Q}$ .

## 27. D Characteristic Equation

$\boxed{\mathcal{Q}^{+}=\mathcal{D}}$

Nothing more.

This makes D flip-flops very convenient for:

- registers
- state machines
- pipelines

## 28. D Excitation Table

To achieve a desired next state:

$$D = Q^{+}$$

Table:

| Q | $Q^{+}$ | D |
|---|---------|---|
| 0 | 0       | 0 |
| 0 | 1       | 1 |
| 1 | 0       | 0 |
| 1 | 1       | 1 |

Thus:

$$\boxed{D = Q^{+}}$$

## 29. T Flip-Flop

T stands for Toggle.

Input:

$$T$$

Behavior:

**T=0**

Hold.

**T=1**

Toggle.

Characteristic table:

| T | $Q^{+}$ |
|---|---------|
| 0 | Q       |
| 1 | Q'      |

### 30. T Characteristic Equation

$$Q^{+} = T \oplus Q$$

Equivalent SOP:

$$Q^{+} = T'Q + TQ'$$

Check:

T=0:

$$Q^{+} = Q$$

T=1:

$$Q^{+} = Q'$$

### 31. T Excitation Table

| Q | $Q^{+}$ | T |
|---|---------|---|
| 0 | 0       | 0 |
| 0 | 1       | 1 |
| 1 | 0       | 1 |
| 1 | 1       | 0 |

Thus:

$$T = Q \oplus Q^{+}$$

Very useful.

Memory:

**T=1 when state must change.**

**T=0 when state must stay.**

### 32. Master Characteristic Table Ø=Ý4

| FF | Input(s) | Key behavior                          |
|----|----------|---------------------------------------|
| SR | S,R      | 00 Hold, 01 Reset, 10 Set, 11 Invalid |
| JK | J,K      | 00 Hold, 01 Reset, 10 Set, 11 Toggle  |
| D  | D        | $Q^{+} = D$                           |
| T  | T        | 0 Hold, 1 Toggle                      |

This table must become automatic.



### 33. Master Characteristic Equations

#### SR

$$Q^{+} = S + R'Q$$

valid for:

$$SR \neq 1$$

#### JK

$$Q^{+} = JQ' + K'Q$$

#### D

$$Q^{+} = D$$

#### T

$$Q^{+} = T \oplus Q$$

These four equations are high-value formulas.

### 34. Master Excitation Table $\emptyset = \text{Y}4$

| $Q$ | $Q^{+}$ | SR         | JK         | D | T |
|-----|---------|------------|------------|---|---|
| 0   | 0       | $S=0, R=X$ | $J=0, K=X$ | 0 | 0 |
| 0   | 1       | $S=1, R=0$ | $J=1, K=X$ | 1 | 1 |
| 1   | 0       | $S=0, R=1$ | $J=X, K=1$ | 0 | 1 |
| 1   | 1       | $S=X, R=0$ | $J=X, K=0$ | 1 | 0 |

This is one of the most useful tables in the entire sequential-logic syllabus.

### 35. How to Remember Excitation Table

For D:

$$D = Q^{+}$$

For T:

$$T = Q \oplus Q^{+}$$

For SR:

- need 0! 1' Set
- need 1! 0' Reset

For JK:

- same broad idea
- but unused input often becomes don't-care.

---

## 36. Characteristic vs Excitation — Do Not Confuse

---

### Characteristic table

Given FF inputs, find next state.

Example:

$\{ J, K, Q \rightarrow Q^+ \}$

### Excitation table

Given desired transition, find FF inputs.

Example:

$\{ Q, Q^+ \rightarrow J, K \}$

Memory:

**Characteristic = predict**

**Excitation = design**

---

## 37. Timing Diagram — D Flip-Flop Ø=4

---

For positive-edge-triggered D FF:

At every rising edge:

1. inspect D
2. sample D
3. after clock-to-Q delay, Q becomes sampled D

Example:

If D at successive rising edges is:

$\{ 0, 1, 1, 0 \}$

then Q becomes:

$\boxed{0, 1, 1, 0}$

after each active edge.

Changes in D between edges do not directly affect Q.

---

## 38. D FF Timing Example

---

Suppose initial:

$\{ Q=0 \}$

At rising edges:

| Edge | D sampled | New Q |
|------|-----------|-------|
| 1    | 1         | 1     |
| 2    | 0         | 0     |
| 3    | 0         | 0     |
| 4    | 1         | 1     |

Always:

$$Q^{n+1} = D$$

### 39. T FF Timing Diagram

Suppose:

$$T = 1$$

at every active clock edge.

Then Q toggles:

$$Q \rightarrow \bar{Q}$$

Therefore T FF can divide clock frequency by 2.

If input clock frequency:

$$f$$

then Q toggling waveform frequency:

$$f/2$$

This becomes central in counters.

### 40. JK FF With $(J=K=1)$

Exactly same toggle behavior:

$$Q^{n+1} = Q'$$

Therefore:

$$J=K=1$$

makes JK behave like a T FF with:

$$T = 1$$

### 41. JK FF With $(J=K=T)$

If:

$$J=K=T$$

then:

- $T=0 \text{ ! } J=K=0 \text{ ! }$  hold
- $T=1 \text{ ! } J=K=1 \text{ ! }$  toggle

Thus:

$\boxed{\text{JK can implement T FF by setting } J=K=T}$

This previews flip-flop conversion.

---

## 42. D FF From JK Preview

We want:

$Q^{n+1} = D$

One standard way:

$\boxed{J=D, K=D'}$

Then:

- $D=0 \text{ ! } J=0, K=1 \text{ ! }$  reset
- $D=1 \text{ ! } J=1, K=0 \text{ ! }$  set

Hence Q becomes D.

We will systematize conversions next chapter.

---

## 43. T FF From D Preview

Need:

$Q^{n+1} = T \oplus Q$

Since D FF has:

$Q^{n+1} = D$

choose:

$\boxed{D = T \oplus Q}$

Again full conversion methodology comes next.

---

## 44. Asynchronous Preset and Clear Ø=Ý4

Flip-flops may have special inputs:

- PRESET / SET
- CLEAR / RESET

These can force output independently of normal clock.

### Preset

forces:

$Q=1$

## Clear

forces:

$\overline{Q} = 0$

---

## 45. Why Are They Called Asynchronous?

Because they do not wait for the active clock edge.

If asynchronous clear becomes active:

$Q \rightarrow 0$

regardless of:

- D/J/K/T
- clock

subject to device behavior.

---

## 46. Active-Low Preset/Clear

Common notation:

$\overline{\text{PRE}}, \quad \overline{\text{CLR}}$

or bubbles on pins.

Then:

$0 = \text{active}$

Example:

$\overline{\text{CLR}} = 0$

forces:

$\overline{Q} = 0$

---

## 47. Preset/Clear Priority

When an asynchronous control is active, it generally overrides synchronous data inputs.

Conceptually:

$\boxed{\text{Async control} > \text{normal clocked operation}}$

Exact device truth table must be checked if both preset and clear are active simultaneously.

That combination is often invalid/forbidden.

---

## 48. Example

Positive-edge D FF with active-low clear.

Suppose:

---

$\overline{\text{CLR}}=0$

and:

$D=1$

No matter what clock does:

$\boxed{Q=0}$

until clear is released, subject to device specifications.

---

## 49. Synchronous Reset vs Asynchronous Reset

### Synchronous reset

Only takes effect at active clock edge.

### Asynchronous reset

Can reset immediately without clock edge.

This distinction is important.

---

## 50. GATE Timing Trap — Async Reset

If asynchronous clear changes between clock edges:

Q may still change.

Therefore:

**"Flip-flop output changes only at clock edges" is not universally true when asynchronous control inputs exist.**

---

## 51. Edge Symbols

A clock input with triangle typically indicates edge-triggering.

Conceptually:

- triangle only !' positive edge
- triangle + bubble !' negative edge

So:

$\boxed{\text{bubble on clock often means falling-edge triggered}}$

---

## 52. Positive vs Negative Edge

Suppose D changes as follows:

- before rising edge: 1
- before falling edge: 0

Positive-edge D FF samples 1.

Negative-edge D FF samples 0.

Same D waveform, different Q.

Always identify the active edge.

---

## 53. Characteristic Table Example — JK

---

Suppose:

$$\lceil Q=0, \quad J=1, \quad K=1 \rceil$$

JK toggles.

So:

$$\lceil Q^+=1 \rceil$$

Suppose current:

$$\lceil Q=1 \rceil$$

same inputs:

$$\lceil J=K=1 \rceil$$

then:

$$\lceil Q^+=0 \rceil$$

---

## 54. Characteristic Equation Example — JK

---

Given:

$$\lceil J=X \rceil$$

$$\lceil K=1 \rceil$$

Then:

$$\lceil Q^+=JQ'+K'Q \rceil$$

Since:

$$\lceil K'=0 \rceil$$

$$\lceil Q^+=XQ' \rceil$$

So next state depends on X and present state.

This style appears in sequential circuit analysis.

---

## 55. Characteristic Equation Example — T

---

Suppose:

$$\lceil T=XQ \rceil$$

Then:

$$\lceil Q^+=T \oplus Q \rceil$$

---

$$\bar{Q} = (XQ) \oplus Q$$

You can then simplify or evaluate by cases.

Sequential analysis often starts this way.

---

## 56. Input Conditions for Toggle

---

### SR

No clean safe toggle input independent of Q, because:

- to make 0! need S=1,R=0
- to make 1! need S=0,R=1

Input must depend on Q.

### JK

$$\boxed{J=K=1}$$

toggles.

### T

$$\boxed{T=1}$$

toggles.

### D

Need:

$$\boxed{D=Q}$$

to toggle.

This comparison is useful.

---

## 57. Input Conditions for Hold

---

### SR

$$\boxed{S=R=0}$$

### JK

$$\boxed{J=K=0}$$

### T

$$\boxed{T=0}$$

### D

Need:

$$\boxed{D=Q}$$

So D must be fed from current state if you explicitly want hold.

---



## 58. Input Conditions for Set to 1

---

### SR

$$\lceil S=1, R=0 \rceil$$

### JK

$$\lceil J=1, K=0 \rceil$$

or sometimes  $J=1, K=X$  depending current state requirement.

### D

$$\lceil D=1 \rceil$$

### T

Depends on current Q:

- if  $Q=0$  !'  $T=1$
- if  $Q=1$  !'  $T=0$

This is why excitation tables matter.

---

## 59. Flip-Flop Frequency Division

---

A toggle flip-flop changes state at every active edge.

One complete Q cycle requires two clock edges.

Therefore:

$$\lceil f_Q = \frac{f_{clk}}{2} \rceil$$

Cascade n toggle stages:

$$\lceil f_{out} = \frac{f_{clk}}{2^n} \rceil$$

This becomes core counter knowledge.

---

## 60. Example

---

Clock:

$$\lceil 16 \text{ MHz} \rceil$$

A T FF with  $T=1$ :

$$\lceil f_Q = 8 \text{ MHz} \rceil$$

Two cascaded toggle stages:

$$\lceil 4 \text{ MHz} \rceil$$

Three:

$$\lceil 2 \text{ MHz} \rceil$$

---

## 61. L0 Practice — SR

---

NOR-based SR latch:

$$\lceil S=0, R=1 \rceil$$

Result:

$$\lceil \boxed{Q=0} \rceil$$

Reset.

---

## 62. L0 — JK

---

$$\lceil J=K=1 \rceil$$

Action:

$$\lceil \boxed{\text{Toggle}} \rceil$$

---

## 63. L0 — D

---

For D FF:

$$\lceil D=1 \rceil$$

at active edge.

Next:

$$\lceil \boxed{Q^{+}=1} \rceil$$

---

## 64. L0 — T

---

Current:

$$\lceil Q=1 \rceil$$

and:

$$\lceil T=1 \rceil$$

Then:

$$\lceil \boxed{Q^{+}=0} \rceil$$

---

## 65. L1 — Characteristic Equation

---

Which is JK characteristic equation?

$$\lceil \boxed{Q^{+}=JQ'+K'Q} \rceil$$

---

## 66. L1 — Excitation

---

For JK FF, required transition:

---

$\lceil 0 \rightarrow 1 \rceil$

From excitation table:

$\lceil \boxed{J=1, K=X} \rceil$

---

## 67. L1 — T Excitation

---

Transition:

$\lceil 1 \rightarrow 1 \rceil$

No change.

Therefore:

$\lceil \boxed{T=0} \rceil$

---

## 68. L1 — D Excitation

---

Desired:

$\lceil Q^+=0 \rceil$

Regardless of present state:

$\lceil \boxed{D=0} \rceil$

---

## 69. L2 — JK Next State

---

Given:

$\lceil J=0, \quad K=1, \quad Q=1 \rceil$

JK reset:

$\lceil \boxed{Q^+=0} \rceil$

Equation check:

$\lceil Q^+=JQ'+K'Q \rceil$

$\lceil =0+0(1)=0 \rceil$

---

## 70. L2 — T Equation

---

Given:

$\lceil Q=0, \quad T=1 \rceil$

$\lceil Q^+=T \oplus Q \rceil$

$\lceil =1 \oplus 0=1 \rceil$

$\lceil \boxed{1} \rceil$

---

## 71. L2 — Frequency Division

---

A T FF with  $T=1$  receives:

$$\lceil 40 \text{ MHz} \rceil$$

Output frequency:

$$\lceil \frac{40}{2} \rceil$$

$$\lceil \boxed{20 \text{ MHz}} \rceil$$

Four cascaded toggle FFs:

$$\lceil \frac{40}{16} \rceil$$

$$\lceil \boxed{2.5 \text{ MHz}} \rceil$$

---

## 72. L2 — Async Clear

---

D FF:

$$\lceil D=1 \rceil$$

No active clock edge occurs.

But active-low asynchronous clear changes:

$$\lceil \bar{\text{CLR}}:1 \rightarrow 0 \rceil$$

Then:

$$\lceil \boxed{Q \rightarrow 0} \rceil$$

because clear does not require clock.

---

## 73. L3 — Excitation Trap

---

For SR FF transition:

$$\lceil Q=1 \rightarrow Q^+=1 \rceil$$

Which inputs are valid?

Excitation table:

$$\lceil S=X, R=0 \rceil$$

Thus both:

$$\lceil S=0, R=0 \rceil$$

and:

$$\lceil S=1, R=0 \rceil$$

produce  $Q^+=1$ .

Therefore S is don't-care.

---

## 74. L3 — NAND SR Trap

---

For active-low NAND latch:

$$\overline{S} = \overline{R} = 0$$

This is:

$$\boxed{\text{invalid}}$$

Not set+reset simultaneously in a meaningful stable way.

---

## 75. L3 — D vs Latch Timing

---

Suppose D changes multiple times while enable is high.

For D latch:

Q may follow.

For edge-triggered D FF:

Q only samples D at active edge.

Therefore they can produce very different timing waveforms.

---

## 76. L3 — Determine FF Type

---

Suppose characteristic behavior is:

$$Q^{+} = X \oplus Q$$

This exactly matches:

$$\boxed{\text{T flip-flop with } T=X}$$

---

## 77. Race-Around Preview

---

In a level-sensitive JK implementation, if:

$$J=K=1$$

and clock pulse remains active long enough, Q may repeatedly toggle within the same clock pulse.

This is called:

$$\boxed{\text{race-around condition}}$$

We will study it properly in Chapter 18 with master-slave JK.

Do not confuse this with ordinary edge-triggered JK behavior.

---

## 78. GATE PYQ Patterns

---

Common high-frequency patterns include:

- SR/JK/D/T truth tables
  - characteristic equations
  - excitation tables
-

- identify FF from equation
- next-state calculation
- convert one FF to another
- timing diagrams
- active-high/active-low set/reset
- frequency division
- latch vs FF
- asynchronous clear/preset
- race-around condition

These are PYQ-style categories, not specific year claims.

---

## 79. Trap Book — Chapter 16

---

### Trap 1

Latch and flip-flop are identical.

No.

Latch !' level-sensitive.

FF !' edge-triggered.

### Trap 2

SR input 11 always invalid.

Only for the common active-high NOR version.

NAND active-low invalid is 00.

### Trap 3

JK 11 is invalid.

Wrong.

$\boxed{11 = \text{toggle}}$

### Trap 4

D FF holds when  $D=0$ .

Wrong.

It loads 0.

$Q^{n+1} = D$

### Trap 5

$T=1$  means set.

Wrong.

$\boxed{T=1 = \text{toggles}}$

### Trap 6

Characteristic table and excitation table are the same.

No.

One predicts next state; one finds required input.

### Trap 7

Async clear waits for clock.

No.

### Trap 8

Bubble means active-high.

Usually bubble indicates active-low.

### Trap 9

Positive-edge and negative-edge timing can be analyzed identically without checking waveform.

No—sampling moments differ.

### Trap 10

Toggle FF output has same frequency as clock.

No.

$$f_Q = f_{clk}/2$$

when toggling every edge.

---

## 80. Flip-Flop Master Sheet Ø=Ý4

---

### SR

---

Characteristic:

$$Q^{+} = S + R'Q$$

Excitation:

| $Q \mid Q^{+}$ | S | R |
|----------------|---|---|
| 0! 0           | 0 | X |
| 0! 1           | 1 | 0 |
| 1! 0           | 0 | 1 |
| 1! 1           | X | 0 |

---

### JK

---

Characteristic:

$$Q^{+} = JQ' + K'Q$$

Excitation:

| $Q!'$ $Q$ $+$ | $J$ | $K$ |
|---------------|-----|-----|
| $0!'$ $0$     | $0$ | $X$ |
| $0!'$ $1$     | $1$ | $X$ |
| $1!'$ $0$     | $X$ | $1$ |
| $1!'$ $1$     | $X$ | $0$ |

## D

Characteristic:

$$\boxed{Q^{+}=D}$$

Excitation:

$$\boxed{D=Q^{+}}$$

## T

Characteristic:

$$\boxed{Q^{+}=T\oplus Q}$$

Excitation:

$$\boxed{T=Q\oplus Q^{+}}$$

## 81. 30-Second Revision

Remember this master pattern:

### SR

$$\boxed{00\text{ Hold},\ 01\text{ Reset},\ 10\text{ Set},\ 11\text{ Invalid}}$$

### JK

$$\boxed{00\text{ Hold},\ 01\text{ Reset},\ 10\text{ Set},\ 11\text{ Toggle}}$$

### D

$$\boxed{Q^{+}=D}$$

### T

$$\boxed{T=0\text{ Hold},\ T=1\text{ Toggle}}$$

Characteristic equations:

$$\boxed{SR: Q^{+}=S+R'Q}$$

$$\boxed{JK: Q^{+}=JQ'+K'Q}$$

$$\boxed{D: Q^{+}=D}$$

$$\boxed{T: Q^{+}=T\oplus Q}$$



## What You Absolutely Must Remember

1. Latch = level-sensitive.
2. Flip-flop = edge-triggered.
3. SR has a forbidden input combination.
4. JK fixes SR's invalid 11 case by toggling.
5. D stores the sampled input directly.
6. T toggles when T=1.
7. Characteristic table: input/state !' next state.
8. Excitation table: present/desired next state !' required inputs.
9. Learn all four characteristic equations.
10. Learn the excitation table completely.
11. Async preset/clear override normal clocked behavior.
12. Active-low pins are usually indicated by bubbles/bars.
13. Toggle operation divides frequency by 2.
14. Timing diagrams must be read at the correct active clock edge.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' **Latches & Flip-Flops** '

**Current: Chapter 17 — Flip-Flop Conversion**

**Remaining:** FF Conversion !' Race Around/Master-Slave !' Registers !' Counters !' Sequential Analysis !' Sequential Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

### NEXT — Chapter 17: FLIP-FLOP CONVERSION Ø=ßà

Next we will systematically learn:

how to realize SR using JK/D/T !' JK using SR/D/T !' D using SR/JK/T !' T using SR/JK/D !' conversion tables !' excitation-method derivation !' K-map minimization !' direct shortcut formulas !' GATE conversion traps.

USER

next

ASSISTANT

## CHAPTER 17 — FLIP-FLOP CONVERSION Ø=ßà IMPORTANT

Flip-flop conversion means:

**Using one type of flip-flop to behave exactly like another type.**

Example:

- use JK FF hardware to behave like D FF
- use D FF to behave like T FF
- use T FF to behave like JK FF

This is a very common GATE topic because it tests whether you truly understand:

- characteristic tables
- excitation tables
- present state
- next state
- Boolean minimization

---

## 1. The Core Idea

Suppose we are given:

**Implement a D flip-flop using a JK flip-flop.**

The desired device is D.

So desired next state is:

$$Q^{+} = D$$

But actual hardware is JK.

Therefore we must determine what values of J and K will force the JK FF to produce the same  $Q^{+}$ .

That is the entire conversion problem.

---

## 2. Golden Conversion Method

Always follow:

### Step 1

Write the **characteristic behavior of the desired flip-flop**.

### Step 2

For every combination of:

- desired input(s)
- present state Q

find desired:

$$Q^{+}$$

### Step 3

Use the **excitation table of the available flip-flop**.

Find the required available-FF inputs.

#### Step 4

Build conversion table.

#### Step 5

Minimize each required input using K-map/algebra.

#### Step 6

Connect resulting logic to the available FF.

Memory:

**Desired FF tells you what next state you want.**

**Available FF excitation tells you how to produce it.**

### 3. Available vs Desired — Do Not Reverse Them

Suppose:

**D using JK.**

#### Desired

D flip-flop.

So calculate:

$$Q^{+} = D$$

#### Available

JK flip-flop.

So use the JK excitation table to determine J,K.

A common mistake is reversing these roles.

### 4. Excitation Tables We Need

#### SR

| Q | $Q^{+}$ | S | R |
|---|---------|---|---|
| 0 | 0       | 0 | X |
| 0 | 1       | 1 | 0 |
| 1 | 0       | 0 | 1 |
| 1 | 1       | X | 0 |

## JK

| Q | $\neg(Q^{+})$ | J | K |
|---|---------------|---|---|
| 0 | 0             | 0 | X |
| 0 | 1             | 1 | X |
| 1 | 0             | X | 1 |
| 1 | 1             | X | 0 |

## D

$$\boxed{D=Q^{+}}$$

## T

$$\boxed{T=Q \oplus Q^{+}}$$

Keep these available while solving conversions.

## PART A — D FLIP-FLOP USING JK

### 5. Desired Behavior

For D:

$$\boxed{Q^{+}=D}$$

Construct table:

| D | Q | $\neg(Q^{+})$ |
|---|---|---------------|
| 0 | 0 | 0             |
| 0 | 1 | 0             |
| 1 | 0 | 1             |
| 1 | 1 | 1             |

Now use JK excitation.

### 6. Find Required J,K

| D | Q | $\neg(Q^{+})$ | J | K |
|---|---|---------------|---|---|
| 0 | 0 | 0             | 0 | X |
| 0 | 1 | 0             | X | 1 |
| 1 | 0 | 1             | 1 | X |

| D | Q | $Q^{+}$ | J | K |
|---|---|---------|---|---|
| 1 | 1 | 1       | X | 0 |

Minimize:

$$\boxed{J=D}$$

$$\boxed{K=D'}$$

Therefore:

## D using JK:

$$\boxed{J=D, \quad K=D'}$$

## 7. Verify

If  $D=0$ :

$$J=0, \quad K=1$$

JK resets:

$$Q^{+}=0$$

If  $D=1$ :

$$J=1, \quad K=0$$

JK sets:

$$Q^{+}=1$$

Therefore:

$$Q^{+}=D$$

Correct.

## PART B — T FLIP-FLOP USING JK

### 8. Desired T Behavior

T FF:

$$T=0 \rightarrow Q^{+}=Q$$

$$T=1 \rightarrow Q^{+}=Q'$$

JK already has:

$$J=K=0 \rightarrow \text{Hold}$$

$$J=K=1 \rightarrow \text{Toggle}$$

Therefore directly:

$$\boxed{J=T}$$

$$\boxed{K=T}$$

So:

## T using JK:

$$\boxed{J=K=T}$$

This is one of the easiest conversions.

## 9. Verification

T=0:

$$\boxed{J=K=0}$$

Hold.

T=1:

$$\boxed{J=K=1}$$

Toggle.

Exactly matches T FF.

## PART C — SR FLIP-FLOP USING JK

### 10. Desired SR Behavior

Valid SR behavior:

| S | R | Action  |
|---|---|---------|
| 0 | 0 | Hold    |
| 0 | 1 | Reset   |
| 1 | 0 | Set     |
| 1 | 1 | Invalid |

JK can reproduce the first three.

A simple mapping is:

$$\boxed{J=S}$$

$$\boxed{K=R}$$

For valid SR inputs:

- 00 !' JK 00 !' hold
- 01 !' JK 01 !' reset
- 10 !' JK 10 !' set

The SR invalid input 11 is simply excluded.

Therefore:

$$\boxed{J=S, \text{quad } K=R}$$

implements valid SR behavior using JK.

---

## 11. Important Trap

If  $S=R=1$ :

desired SR behavior is invalid.

But JK with  $J=K=1$  toggles.

This does not matter if:

$$\boxed{SR=11 \text{ is treated as an invalid/don't-care input}}$$

The conversion only needs to match valid desired states.

---

## PART D — JK FLIP-FLOP USING D

---

### 12. Key Shortcut

D FF has:

$$\boxed{Q^{+}=D}$$

Therefore whenever you want to implement any desired FF using D:

**Simply set D equal to the desired characteristic equation.**

For JK:

$$\boxed{Q^{+}=JQ'+K'Q}$$

Therefore:

$$\boxed{D=JQ'+K'Q}$$

So:

---

### JK using D:

$$\boxed{D=JQ'+K'Q}$$

Very important shortcut.

---

### 13. Why This Works

D FF always loads its D input.

If:

$$\boxed{D=JQ'+K'Q}$$

then:

$$Q^{+}=D$$

becomes:

$$Q^{+}=JQ'+K'Q$$

which is exactly JK behavior.

## PART E — T FLIP-FLOP USING D

### 14. Desired T Equation

T FF:

$$Q^{+}=T\oplus Q$$

Since D FF gives:

$$Q^{+}=D$$

choose:

$$\boxed{D=T\oplus Q}$$

Therefore:

### T using D:

$$\boxed{D=T\oplus Q}$$

### 15. Verification

T=0:

$$D=0\oplus Q=Q$$

D FF reloads Q !' hold.

T=1:

$$D=1\oplus Q=Q'$$

D FF loads complement !' toggle.

Correct.

## PART F — SR FLIP-FLOP USING D

### 16. Desired SR Equation

SR characteristic equation:

$$Q^{+}=S+R'Q$$

valid for:

$$SR\neq 1$$



Using D:

$$\boxed{D=Q^+}$$

Therefore:

$$\boxed{D=S+R'Q}$$

So:

## SR using D:

$$\boxed{D=S+R'Q}$$

with SR=11 excluded.

## PART G — D FLIP-FLOP USING T

### 17. Use T Excitation

For a T FF:

$$\boxed{T=Q \oplus Q^+}$$

Desired D FF:

$$\boxed{Q^+=D}$$

Therefore:

$$\boxed{T=Q \oplus D}$$

Hence:

## D using T:

$$\boxed{T=D \oplus Q}$$

### 18. Intuition

If current Q already equals D:

$$\boxed{Q=D}$$

then:

$$\boxed{T=0}$$

Hold.

If Q differs from D:

$$\boxed{Q \neq D}$$

then:

$$\boxed{T=1}$$

Toggle.

Thus Q becomes D.

## PART H — JK FLIP-FLOP USING T

### 19. Desired JK Next-State Equation

JK:

$$Q^{+} = JQ' + K'Q$$

Available T FF needs:

$$T = Q \oplus Q^{+}$$

Substitute desired next state.

We can derive by cases.

### 20. Conversion Table

| J | K | Q | Desired $Q^{+}$ | Required T |
|---|---|---|-----------------|------------|
| 0 | 0 | 0 | 0               | 0          |
| 0 | 0 | 1 | 1               | 0          |
| 0 | 1 | 0 | 0               | 0          |
| 0 | 1 | 1 | 0               | 1          |
| 1 | 0 | 0 | 1               | 1          |
| 1 | 0 | 1 | 1               | 0          |
| 1 | 1 | 0 | 1               | 1          |
| 1 | 1 | 1 | 0               | 1          |

Look for pattern.

When:

$$Q = 0$$

T must equal J.

When:

$$Q = 1$$

T must equal K.

Therefore:

$$\boxed{T = JQ' + KQ}$$

So:

## JK using T:

$$\boxed{T = JQ' + KQ}$$

## 21. Verify

If  $Q=0$ :

$$T = J$$

- $J=0$  !' hold at 0
- $J=1$  !' toggle to 1

Correct.

If  $Q=1$ :

$$T = K$$

- $K=0$  !' hold at 1
- $K=1$  !' toggle to 0

Correct.

## PART I — SR FLIP-FLOP USING T

## 22. Use T Excitation Again

Desired SR:

$$Q^+ = S + R'Q$$

Available T:

$$T = Q \oplus Q^+$$

Reason by state.

If  $Q=0$ :

- SR needs set only when  $S=1$
- so  $T=S$

If  $Q=1$ :

- SR needs reset only when  $R=1$
- so  $T=R$

Therefore:

$$\boxed{T = SQ' + RQ}$$

for valid SR inputs.

Thus:

## SR using T:

$$\boxed{T = SQ' + RQ}$$

---

## 23. Verify

Current  $Q=0$ :

$\forall T=S$

If  $S=0$  !' stay 0.

If  $S=1$  !' toggle to 1.

Current  $Q=1$ :

$\forall T=R$

If  $R=0$  !' stay 1.

If  $R=1$  !' toggle to 0.

Correct.

---

## PART J — D FLIP-FLOP USING SR

---

### 24. Desired D Behavior

Need:

$\forall Q^{+}=D$

SR can:

- set when  $D=1$
- reset when  $D=0$

So the obvious mapping is:

$\forall \boxed{S=D}$

$\forall \boxed{R=D'}$

Therefore:

### D using SR:

$\forall \boxed{S=D, \quad R=D'}$

This also guarantees:

$\forall S, R$

are never simultaneously 1.

---

### 25. Verification

$D=0$ :

$\forall S=0, R=1$

reset !'  $Q^{+}=0$ .

---

D=1:

$\backslash [ S=1, R=0 ]$

set !' Q+=1.

Correct.

---

## PART K — T FLIP-FLOP USING SR

---

### 26. Desired T Behavior

---

T=0 !' hold.

T=1 !' toggle.

For SR:

To toggle:

- Q=0 !' set !' S=1,R=0
- Q=1 !' reset !' S=0,R=1

Therefore:

$\backslash \boxed{S=TQ} \backslash$

$\backslash \boxed{R=TQ} \backslash$

Thus:

### T using SR:

---

$\backslash \boxed{S=TQ', \quad R=TQ} \backslash$

---

### 27. Verification

---

T=0:

$\backslash [ S=R=0 ]$

Hold.

T=1:

If Q=0:

$\backslash [ S=1, R=0 ]$

set to 1.

If Q=1:

$\backslash [ S=0, R=1 ]$

reset to 0.

Thus toggle.

---

## PART L — JK FLIP-FLOP USING SR

---

### 28. This One Requires Care

---

Desired JK:

- 00 hold
- 01 reset
- 10 set
- 11 toggle

Available SR cannot accept:

$\nmid S=R=1 \nmid$

So inputs must depend on current Q to avoid invalid condition.

A standard mapping:

$\nmid \boxed{S=JQ} \nmid$

$\nmid \boxed{R=KQ} \nmid$

---

### 29. Verify JK Using SR

---

**J=K=0**

$\nmid S=0, R=0 \nmid$

hold.

**J=1, K=0**

If Q=0:

$\nmid S=1, R=0 \nmid$

set.

If Q=1:

$\nmid S=0, R=0 \nmid$

already 1, hold.

Thus final Q=1.

**J=0, K=1**

If Q=1:

$\nmid R=1 \nmid$

reset.

If Q=0:

hold at 0.

Thus final Q=0.

**J=K=1**

If  $Q=0$ :

$\boxed{S=1, R=0}$

becomes 1.

If  $Q=1$ :

$\boxed{S=0, R=1}$

becomes 0.

Therefore toggle.

Correct.

And SR never receives 11.

---

### 30. JK Using SR Final

---

$\boxed{S=JQ'}$

$\boxed{R=KQ}$

This is a high-value conversion.

---

### 31. Master Conversion Table $\emptyset = Y_4$

---

Here are the main conversions you should remember after understanding the method.

#### Desired D

---

Using JK:

$\boxed{J=D, \quad K=D'}$

Using SR:

$\boxed{S=D, \quad R=D'}$

Using T:

$\boxed{T=D \oplus Q}$

---

#### Desired T

---

Using JK:

$\boxed{J=K=T}$

Using D:

$\boxed{D=T \oplus Q}$

Using SR:

$\boxed{S=TQ', \quad R=TQ}$

---

## Desired JK

---

Using D:

$$\boxed{D = JQ' + K'Q}$$

Using T:

$$\boxed{T = JQ' + KQ}$$

Using SR:

$$\boxed{S = JQ', \quad R = KQ}$$

---

## Desired SR

---

Using JK:

$$\boxed{J = S, \quad K = R}$$

for valid SR combinations.

Using D:

$$\boxed{D = S + R'Q}$$

Using T:

$$\boxed{T = SQ' + RQ}$$

for valid SR combinations.

---

## 32. Do You Need to Memorize All 12?

---

Not necessarily.

For GATE, it is safer to memorize the **conversion method** and a few high-value shortcuts.

Especially remember:

$$\boxed{\text{Desired using D} \rightarrow D = Q^+_{\text{desired}}}$$

and:

$$\boxed{\text{Desired using T} \rightarrow T = Q \oplus Q^+_{\text{desired}}}$$

These two formulas can derive many conversions instantly.

---

## 33. Universal Shortcut — Available D FF

---

If hardware is D:

$$\boxed{Q^+ = D}$$

Therefore:

$$\boxed{D = \text{desired next-state equation}}$$

Examples:

Desired JK:

---



$$\lceil D = JQ' + K'Q \rceil$$

Desired T:

$$\lceil D = T \oplus Q \rceil$$

Desired SR:

$$\lceil D = S + R'Q \rceil$$

This is the easiest available flip-flop.

---

## 34. Universal Shortcut — Available T FF

---

For T:

$$\lceil T = Q \oplus Q^+ \rceil$$

So:

$$\lceil \boxed{T = Q \oplus (\text{desired next state})} \rceil$$

Example desired D:

$$\lceil Q^+ = D \rceil$$

Thus:

$$\lceil T = Q \oplus D \rceil$$


---

## 35. Available JK Strategy

---

When hardware is JK:

Use excitation table.

For:

$$\lceil Q = 0 \rceil$$

J controls whether next state stays 0 or goes 1.

For:

$$\lceil Q = 1 \rceil$$

K controls whether next state stays 1 or goes 0.

This often yields very simple formulas.

---

## 36. Available SR Strategy

---

When hardware is SR:

Be extremely careful to avoid:

$$\lceil S = R = 1 \rceil$$

Use Q feedback when necessary.

For JK using SR:

---

$$S = JQ'$$

$$R = KQ$$

guarantees they cannot be simultaneously 1 because:

$$QQ' = 0$$

## 37. Why Feedback Q Appears

Some conversions need the current state to decide whether to set/reset/toggle.

Example:

T using SR.

When T=1:

- if Q=0 !' set
- if Q=1 !' reset

Therefore inputs must know Q.

Hence:

$$S = TQ'$$

$$R = TQ$$

This is why current state often appears in conversion equations.

## 38. Conversion Through Characteristic Equations

Sometimes you can avoid tables.

Example:

T using D.

Desired:

$$Q^+ = T \oplus Q$$

Available D:

$$Q^+ = D$$

Set equal:

$$D = T \oplus Q$$

Done.

This is the fastest method when equations are simple.

## 39. Equation Method Example — JK Using D

Desired:

$$Q^+ = JQ' + K'Q$$

Available:

$$\lceil Q^+=D \rceil$$

Thus:

$$\lceil \boxed{D=JQ'+K'Q} \rceil$$

Again immediate.

---

## 40. Equation Method Example — D Using T

---

Desired:

$$\lceil Q^+=D \rceil$$

Available T:

$$\lceil Q^+=T\oplus Q \rceil$$

Therefore:

$$\lceil T\oplus Q=D \rceil$$

XOR Q both sides:

$$\lceil T=D\oplus Q \rceil$$

because:

$$\lceil Q\oplus Q=0 \rceil$$

Thus:

$$\lceil \boxed{T=D\oplus Q} \rceil$$


---

## 41. Conversion Cost

---

A conversion may require:

- AND gates
- OR gates
- XOR gates
- NOT gates
- feedback from Q or Q'

Example:

D using JK:

$$\lceil J=D, \quad K=D' \rceil$$

If D' is not already available:

one inverter is needed.

If complementary outputs are available from an upstream source, maybe not.

GATE may ask gate count.

---

## 42. Q and Q' Outputs

---

Most flip-flop symbols may provide both:

$$\boxed{Q, \text{quad } Q'}$$

If  $Q'$  is directly available, do not count a separate NOT gate for  $Q'$ .

Always inspect the circuit/question.

---

### 43. Example — Gate Count

---

Implement T using SR:

$$\boxed{S=TQ'}$$

$$\boxed{R=TQ}$$

If both  $Q$  and  $Q'$  are available:

Need:

$$\boxed{2 \text{ AND gates}}$$

No inverter.

If  $Q'$  is unavailable:

need one extra NOT.

---

### 44. L0 Practice

---

Implement T using JK.

Answer:

$$\boxed{J=K=T}$$

---

### 45. L0 Practice

---

Implement D using SR.

Answer:

$$\boxed{S=D, \text{quad } R=D'}$$

---

### 46. L1 Practice

---

Implement T using D.

Desired:

$$\boxed{Q^+ = T \oplus Q}$$

D FF:

$$\boxed{D=Q^+}$$

Therefore:

$$\boxed{D=T \oplus Q}$$

---

---

## 47. L1 Practice

Implement D using T.

T excitation:

$$T = Q \oplus Q'$$

Desired:

$$Q' = D$$

Therefore:

$$\boxed{T = Q \oplus D}$$

Same XOR expression.

---

## 48. L1 Practice

Implement JK using D.

$$Q'_{JK} = JQ' + K'Q$$

Thus:

$$\boxed{D = JQ' + K'Q}$$

---

## 49. L2 — JK Using SR

Required equations:

$$\boxed{S = JQ'}$$

$$\boxed{R = KQ}$$

Why Q feedback?

To implement toggle safely without ever producing forbidden:

$$\boxed{S = R = 1}$$

---

## 50. L2 — T Using SR

$$\boxed{S = TQ'}$$

$$\boxed{R = TQ}$$

At  $T=0$  !' both 0 !' hold.

At  $T=1$  !' set/reset depending on current Q.

---

## 51. L2 — JK Using T

$$\boxed{T = JQ' + KQ}$$

---

Test:

If  $Q=0$ :

$\lceil T=J \rceil$

If  $Q=1$ :

$\lceil T=K \rceil$

Correct.

---

## 52. L2 — SR Using T

$\lceil \boxed{T=SQ'+RQ} \rceil$

For  $Q=0$ , S decides whether to toggle up.

For  $Q=1$ , R decides whether to toggle down.

---

## 53. L3 — Invalid-State Trap

Suppose converting SR to D.

The desired SR input:

$\lceil S=R=1 \rceil$

is undefined.

When constructing the conversion table, treat this input combination as:

$\lceil \boxed{\text{don't-care / invalid}} \rceil$

Do not force an arbitrary valid next state unless the question specifies one.

---

## 54. L3 — Available vs Desired Trap

Question:

**Implement JK FF using D FF.**

Wrong approach:

Use D excitation table and derive JK input.

Correct:

1. Desired JK determines  $\lceil Q^{+} \rceil$ .

2. Available D requires:

$\lceil D=Q^{+} \rceil$

Therefore:

$\lceil D=JQ'+K'Q \rceil$

## 55. L3 — Q Feedback Trap

Suppose asked:

**Can a T FF be implemented from an SR FF without using Q/Q' feedback?**

In the straightforward realization, no fixed mapping of T alone can distinguish:

- toggle 0! 1
- toggle 1! 0

because SR needs set in one case and reset in the other.

Therefore current Q is needed:

$$\boxed{S=TQ'}$$

$$\boxed{R=TQ}$$

## 56. Which Flip-Flop Is Easiest for State-Machine Design?

D FF is usually simplest because:

$$\boxed{D=Q^+}$$

Once next-state equation is known, connect it directly to D.

With JK/SR/T, you often need excitation equations.

This is one reason D FF is widely used in registers and synchronous state machines.

## 57. Conversion and Excitation Are Directly Connected

Sequential circuit design procedure often looks like:

$$\text{State table} \rightarrow Q^+ \rightarrow \text{FF excitation} \rightarrow \text{input equations}$$

Flip-flop conversion is simply a smaller version of the same idea.

So mastering this chapter prepares you for:

- counter design
- FSM design
- sequence detector design

## 58. GATE-Fast Decision Tree

If asked to convert:

**Available D?**

Write:

$$\boxed{D=Q^+_{{\text{desired}}}}$$

**Available T?**

Write:

$\boxed{T = Q \oplus Q^+_{\text{desired}}}$

### Available JK or SR?

Use excitation table.

This saves time.

---

## 59. Characteristic Equation Sheet

Desired SR:

$$Q^+ = S + R'Q$$

Desired JK:

$$Q^+ = JQ' + K'Q$$

Desired D:

$$Q^+ = D$$

Desired T:

$$Q^+ = T \oplus Q$$

These equations are your starting point.

---

## 60. GATE PYQ Patterns

Common conversion questions include:

- derive D from JK
- implement T using JK
- implement JK using D
- identify required feedback logic
- complete conversion table
- simplify conversion equations using K-map
- determine gate count
- identify equivalent converted circuit
- use excitation table in sequential design

These are PYQ-style patterns, without claiming particular years.

---

## 61. Trap Book — Chapter 17

### Trap 1

Using characteristic table of available FF to determine desired next state.

Wrong.

Desired FF decides next state.

### Trap 2



Using excitation table of desired FF.

Wrong.

Use excitation table of the **available hardware**.

### Trap 3

Ignoring present Q.

Many conversions require Q feedback.

### Trap 4

Forgetting invalid SR combination.

Treat it as invalid/don't-care unless specified.

### Trap 5

Counting NOT gate for Q' when Q' output is already available.

Check diagram.

### Trap 6

Memorizing formulas without knowing direction.

"JK using D" is not the same as "D using JK."

### Trap 7

Thinking T using JK requires complex logic.

Actually:

$$\boxed{J=K=T}$$

### Trap 8

Thinking D using T is  $\boxed{T=D}$ .

Wrong.

$$\boxed{T=D \oplus Q}$$

### Trap 9

Thinking JK using SR can use  $S=J, R=K$ .

That fails at  $J=K=1$  because SR input 11 is invalid.

Correct:

$$\boxed{S=JQ', R=KQ}$$

## 62. Conversion Master Sheet

| Desired FF | Available FF | Required input(s)   |
|------------|--------------|---------------------|
| D          | JK           | $\boxed{J=D, K=D'}$ |
| D          | SR           | $\boxed{S=D, R=D'}$ |

| Desired FF | Available FF | Required input(s)               |
|------------|--------------|---------------------------------|
| D          | T            | $(T=D \oplus Q)$                |
| T          | JK           | $(J=K=T)$                       |
| T          | D            | $(D=T \oplus Q)$                |
| T          | SR           | $(S=TQ', R=TQ)$                 |
| JK         | D            | $(D=JQ' + K'Q)$                 |
| JK         | T            | $(T=JQ' + KQ)$                  |
| JK         | SR           | $(S=JQ', R=KQ)$                 |
| SR         | JK           | $(J=S, K=R)$ on valid SR inputs |
| SR         | D            | $(D=S + R'Q)$                   |
| SR         | T            | $(T=SQ' + RQ)$                  |

## 63. 30-Second Revision

The method:

$\boxed{\text{Desired inputs} + Q} \rightarrow Q^+ \rightarrow \text{available FF excitation} \rightarrow \text{required logic}$

Best shortcuts:

### Available D

$\boxed{D=Q^+_{\text{desired}}}$

### Available T

$\boxed{T=Q \oplus Q^+_{\text{desired}}}$

### T using JK

$\boxed{J=K=T}$

### D using JK

$\boxed{J=D, K=D'}$

### JK using SR

$\boxed{S=JQ', R=KQ}$

## What You Absolutely Must Remember

1. Conversion means making available hardware reproduce desired next-state behavior.
2. Desired FF determines  $(Q^+)$ .
3. Available FF's excitation table determines required inputs.
4. D FF conversion is easiest because  $(D=Q^+)$ .

5. T excitation is  $(T = Q \oplus Q')$ .
6. T using JK:  $(J = K = T)$ .
7. D using JK:  $(J = D, K = D')$ .
8. D using T:  $(T = D \oplus Q)$ .
9. JK using D:  $(D = JQ' + K'Q)$ .
10. JK using SR:  $(S = JQ', R = KQ)$ .
11. Invalid SR inputs may be treated as don't-cares during conversion.
12.  $Q/Q'$  feedback is often essential.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Fundamentals ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' **FF Conversion** '

**Current: Chapter 18 — Race-Around Condition & Master-Slave Flip-Flop**

**Remaining:** Race Around/Master-Slave ' !' Registers ' !' Counters ' !' Sequential Analysis ' !' Sequential Design ' !' FSM ' !' Sequence Detectors ' !' Fixed Point ' !' Floating Point ' !' IEEE-754

## NEXT — Chapter 18

We will cover:

race-around in JK FF ' !' why  $(J = K = 1)$  causes repeated toggling in level-triggered implementation ' !' clock pulse width condition ' !' master-slave JK operation ' !' edge-triggering as solution ' !' timing behavior ' !' GATE traps.

USER

next

ASSISTANT

## CHAPTER 18 — RACE-AROUND CONDITION & MASTER-SLAVE JK FLIP-FLOP

### Ø=0=NOT IMPORTANT

This chapter mainly answers one question:

**Why can a level-triggered JK flip-flop behave incorrectly when  $(J = K = 1)$ , and how do we prevent it?**

The key term is:

$\boxed{\text{Race-Around Condition}}$

## 1. Recall JK Flip-Flop

JK characteristic table:

| J | K | $\neg(Q^{+})$ | Action |
|---|---|---------------|--------|
| 0 | 0 | Q             | Hold   |
| 0 | 1 | 0             | Reset  |
| 1 | 0 | 1             | Set    |
| 1 | 1 | $\neg(Q)$     | Toggle |

So when:

$$\boxed{J=K=1}$$

the desired behavior is:

$$\boxed{\text{one toggle per clock event}}$$

## 2. Where Does Race-Around Occur?

Race-around is associated with a **level-triggered JK implementation** when:

$$J=K=1$$

and the active clock pulse remains present long enough for the output to feed back and toggle repeatedly.

This is not the normal behavior of an ideal edge-triggered JK flip-flop.

## 3. Basic Idea

Suppose initially:

$$Q=0$$

and:

$$J=K=1$$

During the active clock level:

$$Q:0 \rightarrow 1$$

But the JK circuit uses feedback from Q.

After propagation delay, the new  $Q=1$  feeds back.

Since clock is **still active** and  $J=K=1$ :

$$Q:1 \rightarrow 0$$

Again feedback changes.

If enough clock-pulse time remains:

$$Q:0 \rightarrow 1$$

again.

So:

$$0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow \cdots$$

within the **same clock pulse**.

That is race-around.

---

## 4. Why Is It a Problem?

At the end of the clock pulse, the final Q depends on:

- how many toggles occurred
- propagation delay
- clock pulse width

If toggles occur an unpredictable number of times, the final state can be uncertain.

Therefore:

$\boxed{\text{final output becomes timing-dependent}}$

---

## 5. Race-Around Definition

**Race-around condition is repeated toggling of the output of a level-triggered JK flip-flop during one active clock pulse when  $(J=K=1)$ , because feedback has time to affect the circuit repeatedly.**

---

## 6. Essential Conditions

Race-around generally requires:

1. JK configured with:

$\boxed{J=K=1}$

2. Level-sensitive/level-triggered implementation.
3. Active clock pulse sufficiently wide.

The clock must remain active long enough for feedback changes to cause another toggle.

---

## 7. Propagation Delay Connection

Let flip-flop propagation delay be approximately:

$t_p$

After one output change, feedback requires time to affect the next state.

If active clock duration is very short, output may toggle only once.

If clock remains active much longer than the internal propagation delay:

$\boxed{\text{multiple toggles may occur}}$

---

## 8. Simplified Condition

A common conceptual statement is:

$$\boxed{T_{\text{pulse}} > t_p}$$

can permit race-around.

More precisely, the exact condition depends on the internal circuit and total feedback-loop delay.

For GATE conceptual questions, remember:

**Clock pulse width must be small enough that feedback cannot trigger repeated state changes within the same active level.**

## 9. Important Caution

Do not blindly use:

$$T_{\text{pulse}} < t_p$$

as a universal design formula unless the question defines  $t_p$  for the relevant feedback path.

Real circuits have multiple internal delays.

The important exam concept is:

$$\boxed{\text{wide active pulse} + \text{feedback} + J=K=1 \rightarrow \text{race-around risk}}$$

## 10. Timing Example

Suppose:

$$t_p = 10 \text{ ns}$$

and clock remains HIGH for:

$$50 \text{ ns}$$

If the circuit can toggle approximately every 10 ns:

$$0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow 1$$

may occur before clock goes LOW.

Final Q depends on how many toggles fit inside the pulse.

Bad.

## 11. Difference From Normal Toggle Operation

Correct JK behavior for:

$$J=K=1$$

should be:

At each active clock event:

$$Q^+ = Q'$$

Exactly one toggle.

Race-around instead produces:

$Q \rightarrow Q \rightarrow Q \rightarrow \dots$

multiple times during one pulse.

---

## 12. Race-Around vs Propagation Delay

Propagation delay itself is normal.

Every digital gate has delay.

Race-around happens because:

$\boxed{\text{the delayed output is fed back while the device remains enabled}}$

So feedback plus level-sensitive clocking causes the problem.

---

## 13. How Do We Remove Race-Around?

Three major approaches:

1. Use a sufficiently narrow clock pulse.
2. Use a **master-slave JK flip-flop**.
3. Use an **edge-triggered flip-flop**.

For GATE, master-slave and edge-triggering are the important conceptual solutions.

---

## PART A — MASTER-SLAVE JK FLIP-FLOP

---

### 14. What Is Master-Slave?

A master-slave flip-flop consists of two latch stages:

1. Master
2. Slave

connected in cascade.

Conceptually:

```
TEXT
J, K
! "
MASTER
! "
SLAVE
! "
Q
```

The two stages are enabled on opposite clock levels.

---

### 15. Basic Operating Principle

One common implementation:

## Clock HIGH

- Master enabled
- Slave disabled

## Clock LOW

- Master disabled
- Slave enabled

Therefore input information is captured by master during one phase and transferred to slave during the opposite phase.

---

## 16. Why Does This Help?

While master is accepting input:

$\boxed{\text{slave output } Q \text{ is not changing in response to master's internal changes}}$

Then master closes.

Only afterward does slave copy master's stored result.

Thus the external output does not feed back repeatedly through both stages while both are simultaneously transparent.

This breaks the repeated-toggle path.

---

## 17. Two-Phase Intuition

Think:

**Master listens first.**

**Slave updates later.**

They do not both freely follow the input at the same time.

This prevents the level-triggered feedback race.

---

## 18. Example Operation

Assume:

$J=K=1, \quad Q=0$

### During master-active phase

Master determines that next state should toggle to 1.

Slave is closed, so external  $Q$  remains:

$0$

### Clock changes phase

Master closes.

Slave opens and accepts master's stored state:

---



$Q \rightarrow 1$

Only one externally visible toggle occurs.

---

## 19. Next Clock Cycle

On the next complete clock cycle:

Current:

$Q = 1$

Again:

$J = K = 1$

Master computes complement.

Slave later transfers:

$Q \rightarrow 0$

Thus output sequence becomes:

$0, 1, 0, 1, \dots$

one toggle per clock cycle.

---

## 20. Master-Slave Timing Interpretation

Depending on implementation, a master-slave FF may effectively behave like:

- falling-edge triggered
- or rising-edge triggered

Example:

If master is active HIGH and slave active LOW, external Q commonly changes around:

$\boxed{\text{falling edge}}$

because that is when control transfers from master to slave.

---

## 21. Do Not Memorize Wrong Edge Universally

The effective edge depends on:

- latch polarities
- clock inversion
- implementation

So never say:

**"Master-slave is always falling-edge triggered."**

Instead:

$\boxed{\text{Check which phase enables master and slave}}$

---

## 22. Master-Slave Structure

Conceptually:

TEXT

CLK

```
%  
% % % % % % % % %° M A S T E R  
%  
% % N O T % % % % ° S L A V E
```

Thus one stage receives CLK and the other receives its complement.

Only one should be transparent at a time.

---

## 23. Master-Slave vs Edge-Triggered FF

They solve similar problems but are not exactly the same internal structure.

### Master-Slave

Built using two level-sensitive stages.

### Edge-Triggered FF

Designed so input is sampled only around a clock transition.

From a high-level timing perspective, both can ensure approximately one state update per active clock event.

---

## PART B — EDGE TRIGGERING

---

## 24. Why Edge Triggering Solves Race-Around

An edge-triggered JK FF responds only during a very short transition:

$\lceil 0 \rightarrow 1 \rceil$

or:

$\lceil 1 \rightarrow 0 \rceil$

After the edge, the device is no longer transparent to further feedback changes during the rest of the clock level.

Thus:

$\boxed{J=K=1} \rightarrow \text{one toggle per active edge}$

---

## 25. Positive-Edge JK

At every:

$\uparrow \text{CLK}$

evaluate J,K.

Q holds.

causes one toggle per falling edge.

and initial  $Q=0$ :

toggles at active edges

Q does **not** keep toggling while clock remains high.

$\boxed{\text{Race-around is a particular timing/feedback problem}}$

www.gpttopdf.in

## 29. Master-Slave JK Behavior Table

Externally, it still obeys normal JK behavior:

| J | K | $Q^{n+1}$   |
|---|---|-------------|
| 0 | 0 | Q           |
| 0 | 1 | 0           |
| 1 | 0 | 1           |
| 1 | 1 | $\neg(Q^n)$ |

The internal master-slave mechanism changes **timing behavior**, not the intended logical truth table.

## 30. Important Insight

Race-around is not because JK truth table is wrong.

The truth table:

$\neg[1] \rightarrow \text{Toggle}$

is correct.

The problem is **implementation timing** in a level-sensitive circuit.

This distinction matters.

## 31. Race-Around and T Flip-Flop

A T flip-flop with:

$T=1$

also logically toggles.

But a proper edge-triggered T FF toggles once per edge.

Race-around is specifically discussed most often with traditional level-triggered JK implementations because of their feedback mechanism.

## 32. Why SR Doesn't Have This Same Toggle Problem

SR does not have a valid toggle input combination equivalent to JK's:

$J=K=1$

Its corresponding:

$S=R=1$

is invalid in the active-high NOR version.

So the classic repeated-toggle race-around condition is associated with JK.

### 33. Frequency Division Connection

---

If an edge-triggered JK FF has:

$$J=K=1$$

then it toggles once per clock edge.

One full Q cycle requires two clock periods.

Therefore:

$$f_Q = \frac{f_{CLK}}{2}$$

Same as T FF with  $T=1$ .

---

### 34. Example

---

Clock:

$$20 \text{ MHz}$$

JK:

$$J=K=1$$

Output:

$$10 \text{ MHz}$$

assuming one active edge per clock cycle.

---

### 35. Clock Pulse Width

---

Clock period and pulse width are different.

Suppose period:

$$T=10 \text{ ns}$$

with 50% duty cycle.

Then high pulse width:

$$5 \text{ ns}$$

Race-around concerns the duration of the **active level**, not merely total period.

---

### 36. Duty Cycle Connection

---

For active-high level-triggered JK:

If duty cycle increases, HIGH interval increases.

A longer active interval gives more opportunity for repeated toggling.

Again this is implementation-dependent, but conceptually:

$$\text{active pulse width matters}$$

---

## 37. Example

---

Clock frequency:

$50\text{ MHz}$

Period:

$20\text{ ns}$

Duty cycle:

$60\%$

High time:

$0.6 \times 20$

$12\text{ ns}$

If active-high JK feedback can re-toggle within 12 ns, race-around may occur.

---

## 38. L0 Practice

---

Race-around occurs classically when:

A.  $J=0, K=0$  B.  $J=0, K=1$  C.  $J=1, K=0$  D.  $J=K=1$

Answer:

$D$

---

## 39. L0 Practice

---

In normal JK behavior:

$J=K=1$

means:

Toggle

not invalid.

---

## 40. L1 Practice

---

Which implementation is most directly susceptible to classic race-around?

A. Edge-triggered D FF B. Level-triggered JK FF with  $J=K=1$  C. Decoder D. MUX

Answer:

$B$

---

## 41. L1 — Solution

---

Which can prevent race-around?

- Master-slave configuration
-

- Edge-triggering
- suitably limiting active clock pulse width

Therefore all are valid conceptual solutions.

---

## 42. L1 — Master-Slave

---

What are the two storage stages called?

$\boxed{\text{Master and Slave}}$

They operate on opposite clock phases.

---

## 43. L2 — Toggle Sequence

---

Initial:

$\boxed{Q=0}$

Edge-triggered JK:

$\boxed{J=K=1}$

After successive active edges:

$\boxed{1, 0, 1, 0, 1, \dots}$

---

## 44. L2 — Frequency

---

Input clock:

$\boxed{64 \text{ MHz}}$

JK with  $J=K=1$ :

$\boxed{f_Q = \frac{64}{2}}$

$\boxed{32 \text{ MHz}}$

Three cascaded toggle stages:

$\boxed{\frac{64}{8} = 8 \text{ MHz}}$

---

## 45. L2 — Master-Slave Phase

---

Suppose master is transparent while  $CLK=1$  and slave while  $CLK=0$ .

When does external Q typically update?

Around transition:

$\boxed{1 \rightarrow 0}$

i.e. falling edge.

But remember: this depends on implementation.

---

## 46. L3 — Conceptual Trap

Statement:

**"JK flip-flop always suffers from race-around whenever  $J=K=1$ ."**

False.

A properly edge-triggered or master-slave JK FF avoids the classic race-around problem.

Correct statement:

$\boxed{\text{Level-sensitive JK + sufficiently wide active pulse + } J=K=1}$

creates the risk.

## 47. L3 — Another Trap

Statement:

**"Race-around means the output toggles once."**

False.

One toggle is the intended JK behavior.

Race-around means:

$\boxed{\text{multiple toggles within the same active clock pulse}}$

## 48. L3 — Timing Calculation

Suppose:

$f=100 \text{ MHz}$

Clock period:

$\frac{1000}{100}=10 \text{ ns}$

Duty cycle:

40%

Active HIGH pulse:

$0.4(10)=4 \text{ ns}$

If a question states feedback requires at least 6 ns before another toggle can occur, then 4 ns active pulse is too short for a second toggle.

Thus race-around would be avoided under that simplified timing model.

## 49. Master-Slave Advantages

- avoids classic race-around
- isolates input sampling from output update



- permits controlled single state update

But disadvantages include:

- more hardware
- more internal delay
- more complexity than simpler edge-triggered structures

## 50. Master-Slave Propagation

Because state must pass through:

$\text{Master} \rightarrow \text{Slave}$

master-slave configurations can have larger delay than a single latch.

That is one tradeoff for safer timing.

## 51. GATE PYQ Patterns

Typical questions include:

- condition for race-around
- why  $J=K=1$  causes repeated toggling
- clock pulse width and propagation delay
- master-slave operation
- effective triggering edge
- method to eliminate race-around
- output-frequency division for JK toggle
- distinguish edge-triggered and level-triggered behavior

These are PYQ-style patterns, not specific year claims.

## 52. Trap Book — Chapter 18

### Trap 1

JK 11 is invalid.

Wrong.

$\boxed{11 = \text{Toggle}}$

### Trap 2

Every JK FF suffers race-around.

Wrong.

Classic race-around is tied to level-sensitive implementations.

### Trap 3

Race-around = one toggle per clock.

Wrong.

It means multiple toggles in one active pulse.

#### Trap 4

Master and slave are enabled simultaneously.

They are intended to operate on opposite clock phases.

#### Trap 5

Master-slave is always rising-edge triggered.

Wrong.

Effective edge depends on implementation.

#### Trap 6

Clock period and active pulse width are same.

Not necessarily.

Duty cycle matters.

#### Trap 7

Propagation delay is itself a fault.

No.

It is normal. Race-around occurs because feedback has time to act repeatedly while enabled.

#### Trap 8

Edge-triggered JK with  $J=K=1$  keeps toggling throughout clock HIGH.

Wrong.

It toggles once at the active edge.

---

## 53. Formula / Concept Sheet

---

JK characteristic:

$$\boxed{Q^{n+1} = JQ^n + K'Q^n}$$

For:

$$\boxed{J=K=1}$$

$$\boxed{Q^{n+1} = Q^n}$$

Frequency division:

$$\boxed{f_Q = f_{clk}/2}$$

Clock high time:

$$\boxed{T_{H} = D \cdot T}$$

where D is duty-cycle fraction.

Clock period:

$$\boxed{T = \frac{1}{f}}$$

Race-around risk:

$\boxed{J=K=1 + \text{level-sensitive JK} + \text{sufficient active pulse width}}$

## 54. 30-Second Revision

Remember:

**JK 11 means toggle.**

But in a level-sensitive implementation, if the clock stays active too long:

$Q \rightarrow Q \rightarrow Q \rightarrow \dots$

within the same pulse.

That is:

$\boxed{\text{race-around}}$

Solutions:

$\boxed{\text{master-slave}}$

$\boxed{\text{edge-triggered FF}}$

or suitably restrict pulse width.

Master-slave:

**master samples in one phase, slave updates in the opposite phase.**

## What You Absolutely Must Remember

1. Race-around is mainly associated with level-triggered JK when  $(J=K=1)$ .
2. It means repeated toggling within one active clock pulse.
3. Feedback and propagation delay cause the repeated toggles.
4. A sufficiently long active pulse enables the problem.
5. Master-slave configuration prevents the classic feedback race.
6. Master and slave work on opposite clock phases.
7. Edge-triggering also prevents repeated within-pulse toggling.
8. Master-slave effective edge depends on implementation.
9. JK with  $(J=K=1)$  is a divide-by-2 toggle element when operated correctly.
10. Do not confuse intended toggle behavior with race-around.

## Digital Logic Progress

**Completed:** Number Systems ' ' Signed Representation ' ' Binary Arithmetic ' ' Boolean Algebra ' ' SOP/POS ' ' Universal Gates ' ' Algebraic Minimization ' ' K-map ' ' Tabular Method ' ' Combinational Circuits ' ' Adders/Subtractors ' ' MUX ' ' Decoder/Encoder/DEMUX ' ' Comparator ' ' Sequential Fundamentals ' ' Latches/Flip-Flops ' ' FF Conversion ' ' **Race Around & Master-Slave** '

**Current: Chapter 19 — Registers & Shift Registers**

**Remaining:** Registers/Shift Registers !' Counters !' Sequential Analysis !' Sequential Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT — Chapter 19

We will cover:

register basics !' SISO, SIPO, PISO, PIPO !' left/right shift !' universal shift register !' serial/parallel loading !' shift timing diagrams !' frequency/bit movement !' ring counter and Johnson-counter preview !' GATE traps.

USER

next

ASSISTANT

## CHAPTER 19 — REGISTERS & SHIFT REGISTERS 0=14 IMPORTANT

A **register** is a group of flip-flops used to store multiple bits.

If one flip-flop stores:

$1 \text{ bit}$

then  $n$  flip-flops can store:

$n \text{ bits}$

Example:

4 flip-flops can store a 4-bit word such as:

$1011$

Registers are widely used in:

- CPUs
- temporary storage
- data movement
- serial/parallel conversion
- shifting
- counters
- arithmetic operations

### 1. Basic Register

Suppose we have four D flip-flops:

$Q_3, Q_2, Q_1, Q_0$

Together they form a:

$4 \text{-bit register}$

All flip-flops often share the same clock.

At the active edge, all four bits can be loaded simultaneously.

---

## 2. Parallel Register

Suppose inputs are:

$\{D_3, D_2, D_1, D_0\}$

At active clock edge:

$Q_3^+ = D_3$

$Q_2^+ = D_2$

$Q_1^+ = D_1$

$Q_0^+ = D_0$

Thus entire word is loaded at once.

This is:

$\boxed{\text{parallel loading}}$

---

## 3. Register vs Memory

A register is a small high-speed storage structure built from flip-flops.

For GATE Digital Logic, think:

$\boxed{\text{Register = group of flip-flops}}$

Later in COA, registers form part of processor datapaths.

---

## 4. What Is a Shift Register?

A shift register is a register that can move stored bits left or right, usually one position per clock.

Example:

Initial:

$Q_3Q_2Q_1Q_0 = 1011$

After right shift:

$?101$

where the new leftmost bit depends on the serial input.

---

## 5. Four Main Shift-Register Types

You must know:

1. SISO
  2. SIPO
  3. PISO
  4. PIPO
-

The letters mean:

- S = Serial
- P = Parallel
- I = Input
- O = Output

---

## 6. SISO — Serial In Serial Out

---

Data enters one bit at a time.

Data leaves one bit at a time.

$\boxed{\text{SISO}}$

Concept:

TEXT

S e r i a l   I n   ! '   F F 0   ! '   F F 1   ! '   F F 2   ! '   F F 3   ! '   S e r i a l   O u t

Each clock shifts one bit to the next stage.

---

## 7. SISO Example

---

Suppose a 4-bit right-shift register initially contains:

$\boxed{0000}$

Serial input sequence:

$\boxed{1,0,1,1}$

Assume new bit enters left side.

After first clock:

$\boxed{1000}$

After second:

$\boxed{0100}$

After third:

$\boxed{1010}$

After fourth:

$\boxed{1101}$

Exact bit order depends on the shift convention, so always inspect the diagram.

---

## 8. Shift Direction Matters

---

If bits move toward lower-index positions:

$\boxed{Q_3 \text{ to } Q_2 \text{ to } Q_1 \text{ to } Q_0}$

this is commonly called:

$\boxed{\text{right shift}}$

If bits move toward higher-index positions:

$Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$

this is commonly:

$\boxed{\text{left shift}}$

But labels vary with drawings.

Always trace connections instead of trusting visual intuition.

---

## 9. Right-Shift Equations

For a 4-bit D-FF register, suppose serial input is  $(SI)$  entering  $(Q_3)$ .

Then:

$Q_3^{+} = SI$

$Q_2^{+} = Q_3$

$Q_1^{+} = Q_2$

$Q_0^{+} = Q_1$

After one clock, every stored bit moves right by one position.

---

## 10. Left-Shift Equations

Suppose serial input enters  $(Q_0)$ .

Then:

$Q_0^{+} = SI$

$Q_1^{+} = Q_0$

$Q_2^{+} = Q_1$

$Q_3^{+} = Q_2$

---

## 11. SIPO — Serial In Parallel Out

Bits enter one at a time, but after enough clocks, all stored bits are available simultaneously.

$\boxed{\text{SIPO}}$

Application:

**Convert serial data into parallel form.**

Example:

Serial bits arrive:

$[1, 0, 1, 1]$

After 4 clocks, the 4 flip-flop outputs collectively hold the word.

---

## 12. Why SIPO Is Useful

---

Suppose communication sends one bit at a time through one wire.

Processor wants a full 8-bit byte.

SIPO:

$\boxed{\text{serial stream}} \rightarrow \text{parallel word}$

---

## 13. PISO — Parallel In Serial Out

---

PISO does the reverse.

Load several bits simultaneously:

$P_3 P_2 P_1 P_0$

Then shift them out one bit per clock.

$\boxed{\text{parallel word}} \rightarrow \text{serial stream}$

---

## 14. PISO Operation

---

Usually has a control such as:

- LOAD
- SHIFT

### Load mode

Parallel bits are stored simultaneously.

### Shift mode

Bits move one position each clock, producing serial output.

This requires multiplexing at flip-flop inputs.

---

## 15. PIPO — Parallel In Parallel Out

---

All bits enter simultaneously and are read simultaneously.

$\boxed{\text{PIPO}}$

This is basically a standard parallel register.

No serial shifting is required.

---

## 16. Master Register Type Table

---



| Type | Input    | Output   | Main Use           |
|------|----------|----------|--------------------|
| SISO | Serial   | Serial   | delay/bit shifting |
| SIPO | Serial   | Parallel | serial-to-parallel |
| PISO | Parallel | Serial   | parallel-to-serial |
| PIPO | Parallel | Parallel | regular register   |

Memory trick:

**Read the acronym literally.**

## 17. Number of Clock Pulses

For an  $n$ -bit SISO/SIPO register:

To completely load an  $n$ -bit serial word usually requires:

$\boxed{n \text{ clock pulses}}$

For PISO, after parallel load, shifting out  $n$  bits generally needs:

$\boxed{n \text{ shift events}}$

depending on whether the first output bit is considered already available before the first shift.

Read exact wording carefully.

## 18. Delay Through SISO

A bit inserted into the first stage moves one flip-flop per clock.

For an  $n$ -stage shift register, to travel from first stage through all  $n$  stages may require approximately:

$\boxed{n \text{ clock events}}$

depending on input/output location and counting convention.

This is why shift registers can act as digital delay lines.

## 19. Timing Example

4-bit register:

$Q_3Q_2Q_1Q_0=1010$

Right shift with serial input:

$SI=1$

After one active edge:

$Q_3 \leftarrow Q_2$

$Q_2 \leftarrow Q_1$

$$Q_1 = 0$$

$$Q_0 = 1$$

So:

$$\boxed{1101}$$

---

## 20. Another Right-Shift Example

---

Initial:

$$0110$$

Serial input:

$$0$$

After right shift:

$$\boxed{0011}$$

because:

$$Q_3 = 0$$

$$Q_2 = 0$$

$$Q_1 = 1$$

$$Q_0 = 1$$

---

## 21. Arithmetic vs Logical Shift Connection

---

You already studied shifts in binary arithmetic.

For right shift:

### Logical right shift

insert:

$$0$$

at MSB side.

### Arithmetic right shift for signed 2's complement

insert old sign bit.

A shift register can implement either by choosing appropriate serial input.

---

## 22. Arithmetic Right Shift

---

For signed 4-bit number:

$$1010$$

which represents:

$$-6$$

---

Arithmetic right shift replicates MSB 1:

$\boxed{1101}$

which is:

$-3$

So serial input equals old sign bit.

---

## 23. Logical Right Shift

---

Same bit pattern:

$1010$

Logical right shift:

$\boxed{0101}$

which is unsigned 5.

Different operation.

---

## 24. Left Shift

---

A left shift generally inserts 0 at LSB side.

Example:

$0011$

left shift:

$0110$

which corresponds to:

$3 \times 2 = 6$

if no overflow occurs.

---

## 25. Shift Register as Multiply/Divide Hardware

---

Logical left shift by 1:

$\times 2$

if no overflow.

Logical right shift unsigned by 1:

$\lfloor x/2 \rfloor$

Arithmetic right shift approximates signed division by 2 with 2's-complement rounding behavior.

This is one reason shift hardware appears in ALUs.

---

## 26. Bidirectional Shift Register

---

A bidirectional shift register can shift:

- left
- right

Control input selects direction.

Example:

$\text{[ DIR=0 \rightarrow \text{right} ]}$

$\text{[ DIR=1 \rightarrow \text{left} ]}$

Exact polarity is design-dependent.

---

## 27. How Bidirectional Register Is Built

Each D input can be driven through a MUX.

MUX chooses between:

- neighbor from left
- neighbor from right

depending on direction control.

This connects MUX knowledge to register design.

---

## 28. Universal Shift Register $\emptyset = 4$

A **universal shift register** typically supports:

1. Hold
2. Shift right
3. Shift left
4. Parallel load

That makes it "universal."

---

## 29. Typical 2-Bit Mode Control

Suppose control:

$\text{[ S\_1S\_0 ]}$

Example convention:

| $\text{(S\_1S\_0)}$ | Operation     |
|---------------------|---------------|
| 00                  | Hold          |
| 01                  | Shift right   |
| 10                  | Shift left    |
| 11                  | Parallel load |

This is a common mapping, but **never assume it unless given**.

The actual device may use a different control table.

---

### 30. Why a Universal Register Uses MUXes

Each flip-flop needs to choose among several possible next inputs:

- itself !' hold
- left neighbor
- right neighbor
- parallel input

A 4:1 MUX is therefore a natural choice before each D FF.

Thus:

$$\boxed{\text{Universal shift register} = \text{MUX network} + \text{D FFs}}$$

---

### 31. Hold Operation

For a D FF register:

To hold:

$$D_i = Q_i$$

So MUX selects feedback from the same flip-flop.

---

### 32. Parallel Load

For each bit:

$$D_i = P_i$$

At active edge, all bits load simultaneously.

Example:

Parallel input:

$$1010$$

After one clock:

$$Q = 1010$$

---

### 33. Serial Loading

To load a 4-bit value serially requires multiple clock cycles.

Example:

Starting:

$$0000$$

---

and shifting one bit per clock.

This is slower than parallel load but uses fewer external data lines.

Tradeoff:

$\boxed{\text{Serial} = \text{fewer wires, more cycles}}$

$\boxed{\text{Parallel} = \text{more wires, fewer cycles}}$

---

## 34. Shift Register Using JK/T FFs

---

Registers are most naturally built using D flip-flops because:

$D = Q^+$

But other FF types can implement them after deriving appropriate excitation logic.

For GATE, D-FF implementation is the cleanest and most common.

---

## 35. Register With Load Enable

---

Suppose register should:

- hold if  $L=0$
- load D if  $L=1$

Then each flip-flop next state:

$Q_i^+ = L'D_i$

Careful.

If  $L=0$  !' hold Q.

If  $L=1$  !' load  $(D_i)$ .

Thus:

$Q_i^+ = L'Q_i + LD_i$

This is exactly a 2:1 MUX equation.

---

## 36. Register Load Enable Example

---

Current:

$Q = 1010$

Input:

$D = 0111$

If:

$L = 0$

next:

$\boxed{1010}$

If:

---

$L=1$

next:

$\boxed{0111}$

---

## 37. Register Clear

Registers may include clear/reset.

If synchronous clear:

state becomes 0 at active clock edge.

If asynchronous clear:

state can become 0 immediately when clear activates.

Same concept as single flip-flops.

---

## 38. Ring Counter Preview

A ring counter is basically a shift register where the output of the last flip-flop is fed back to the first.

Example 4-bit one-hot initial state:

$1000$

Shift:

$0100$

then:

$0010$

then:

$0001$

then:

$1000$

Thus it cycles through:

$\boxed{4 \text{ states}}$

for 4 flip-flops when initialized with one 1.

We will study counters in detail next.

---

## 39. Ring Counter Key Result

With  $n$  flip-flops:

A standard one-hot ring counter has:

$\boxed{n \text{ useful states}}$

not:

---

$$\lfloor 2^n \rfloor$$

This is a common GATE trap.

## 40. Johnson Counter Preview Ø=Ý4

A Johnson counter is also called:

$$\boxed{\text{twisted ring counter}}$$

Instead of feeding last Q directly back, feed its complement.

For n flip-flops, it normally cycles through:

$$\boxed{2n \text{ states}}$$

This will be covered properly in counters.

## 41. Ring vs Johnson Preview

| Counter | Feedback             | States with n FFs |
|---------|----------------------|-------------------|
| Ring    | $(Q_{\text{last}})$  | $(n!)$            |
| Johnson | $(Q_{\text{last}}')$ | $(2n!)$           |

Memorize this now; we will derive it next chapter.

## 42. Shift Register Frequency

The clock controls movement rate.

If clock:

$$f = 10 \text{ MHz}$$

one bit shifts every:

$$T = \frac{1}{f}$$

$$= 0.1 \mu s$$

$$= 100 \text{ ns}$$

So after 4 clock events:

$$400 \text{ ns}$$

of shift time, ignoring setup and propagation timing details.

## 43. Throughput

A serial register shifting one bit per clock at 20 MHz can move:

$$20 \text{ million bits/s}$$

in the ideal simple interpretation.



Thus:

$$\boxed{\text{serial bit rate}} \approx f_{\text{clk}}$$

if one bit is transferred per cycle.

---

## 44. Serial vs Parallel Transfer Time

---

Suppose 8-bit word.

### Parallel transfer

Can transfer all 8 bits in one clock event.

### Serial transfer

Needs about 8 bit-times.

So parallel is faster but needs 8 data lines instead of 1.

---

## 45. Timing Diagram Strategy

---

When solving a shift-register timing problem:

1. Write initial register contents.
2. Mark active clock edges.
3. At each edge, update all FFs **simultaneously** from old values.
4. Insert serial input.
5. Do not update stage-by-stage using newly changed values in the same edge.

This last point is extremely important.

---

## 46. Simultaneous Update Trap Ø=Ý4

---

Suppose:

$$Q_3Q_2Q_1Q_0=1010$$

right shift with  $(SI=0)$ .

Correct:

$$Q_3^+=0$$

$$Q_2^+=\text{old } Q_3=1$$

$$Q_1^+=\text{old } Q_2=0$$

$$Q_0^+=\text{old } Q_1=1$$

Therefore:

$$\boxed{0101}$$

Do NOT update Q3 first and then use new Q3 when computing Q2.

All flip-flops sample at the same edge.

---

## 47. Why Simultaneous Update Matters

---

Synchronous register behavior uses:

$$Q_i^{n+1} = f(\text{old state})$$

All next-state equations are evaluated from the present state.

Then all Q outputs update after the clock edge.

This same principle applies to synchronous counters and FSMs.

---

## 48. Example — Two Shifts

---

Initial:

$$1010$$

Right shift, SI sequence:

$$0, 1$$

After first edge:

$$0101$$

After second edge, using SI=1:

$$1010$$

So after two shifts:

$$\boxed{1010}$$

in this particular example.

---

## 49. Example — SIPO Loading

---

Initial:

$$0000$$

Assume new serial bit enters left.

Input sequence:

$$1, 1, 0, 1$$

After clock 1:

$$1000$$

after 2:

$$1100$$

after 3:

$$0110$$

after 4:

$$1011$$

---

Final parallel output:

$\boxed{1011}$

---

## 50. Order Trap in Serial Input

---

If input stream is written:

$\boxed{1011}$

you must know whether:

- MSB enters first
- LSB enters first

Final stored word can differ.

Never assume.

Question must specify serial order or diagram should reveal it.

---

## 51. PISO Output Order

---

Similarly, if a parallel word:

$\boxed{1011}$

is loaded, serial output may be:

- 1,1,0,1
- or 1,0,1,1

depending on:

- shift direction
- output end

Trace the circuit.

---

## 52. Register Capacity

---

An n-bit register stores:

$\boxed{n \text{ bits}}$

Number of possible stored words:

$\boxed{2^n}$

Example:

8-bit register:

$\boxed{256}$

possible bit patterns.

Do not confuse:

- storage capacity in bits = 8 bits
-

- number of patterns = 256
- 

## 53. Register State

---

A register's contents form its state.

For 4-bit register:

$\{Q_3Q_2Q_1Q_0\}$

can be any of:

$\{0000\}$

through:

$\{1111\}$

unless the circuit constrains the reachable states.

---

## 54. Shift Register as Sequence Generator

---

With feedback, a shift register can generate repeating sequences.

Examples:

- ring counter
- Johnson counter
- linear-feedback shift registers (LFSRs)

LFSRs are useful but go beyond core GATE Digital Logic requirements unless specifically asked.

---

## 55. L0 Practice

---

How many flip-flops are needed for a 12-bit register?

$\boxed{12}$

---

## 56. L0 — Type

---

Which register converts serial input to parallel output?

$\boxed{\text{SIPO}}$

---

## 57. L0 — Type

---

Which converts parallel to serial?

$\boxed{\text{PISO}}$

---

## 58. L1 — Right Shift

---

Initial:

---

$$\lceil Q_3Q_2Q_1Q_0=1101 \rceil$$

Right shift with:

$$\lceil SI=0 \rceil$$

Next:

$$\lceil \boxed{0110} \rceil$$

because:

$$\lceil Q_3^+=0 \rceil$$

$$\lceil Q_2^+=1 \rceil$$

$$\lceil Q_1^+=1 \rceil$$

$$\lceil Q_0^+=0 \rceil$$

## 59. L1 — Arithmetic Shift

4-bit signed:

$$\lceil 1100 \rceil$$

Arithmetic right shift.

Sign bit:

$$\lceil 1 \rceil$$

Replicate:

$$\lceil \boxed{1110} \rceil$$

## 60. L1 — Logical Shift

Same:

$$\lceil 1100 \rceil$$

logical right shift:

$$\lceil \boxed{0110} \rceil$$

## 61. L2 — Load Enable

Given:

$$\lceil Q^+=L'Q+LD \rceil$$

If:

$$\lceil L=0 \rceil$$

then:

$$\lceil Q^+=Q \rceil$$

If:

$$\lceil L=1 \rceil$$

then:

$$\lceil Q^+=D \rceil$$

Therefore L is:

$$\lceil \boxed{\text{load enable}} \rceil$$

---

## 62. L2 — Universal Shift Register

---

Which set of operations best describes a universal shift register?

A. only hold B. left shift only C. left/right shift, parallel load, hold D. serial output only

Answer:

$$\lceil \boxed{C} \rceil$$

---

## 63. L2 — Clock Count

---

An empty 8-bit SIPO register receives one bit per clock.

How many clocks are needed to load all 8 serial bits?

$$\lceil \boxed{8} \rceil$$

---

## 64. L2 — Ring Counter Preview

---

How many standard one-hot states can a 6-FF ring counter generate?

$$\lceil \boxed{6} \rceil$$

---

## 65. L2 — Johnson Preview

---

How many standard states for a 6-FF Johnson counter?

$$\lceil 2n=12 \rceil$$

$$\lceil \boxed{12} \rceil$$

---

## 66. L3 — Simultaneous Update

---

Initial:

$$\lceil Q_3Q_2Q_1Q_0=1001 \rceil$$

Right shift with SI=1.

Correct equations:

$$\lceil Q_3^+=1 \rceil$$

$$\lceil Q_2^+=1 \rceil$$

---

$\lceil Q_1^{+}=0 \rceil$

$\lceil Q_0^{+}=0 \rceil$

So:

$\lceil \boxed{1100} \rceil$

Do not get 1111 by cascading newly updated values during the same clock.

---

## 67. L3 — Shift Register Timing

---

4-stage SISO, one shift per rising edge.

A bit enters stage 0 on edge 1.

If serial output is taken from stage 3, it reaches that stage after successive transfers.

Be very careful whether the question asks:

- after how many edges is it stored in final stage?
- when is it observable at output?
- is initial output counted at time zero?

Draw a state table rather than guessing.

---

## 68. GATE PYQ Patterns

---

Common patterns include:

- identify SISO/SIPO/PISO/PIPO
- calculate next register contents
- serial/parallel conversion
- shift direction
- logical vs arithmetic shift
- register load enable equations
- number of clock pulses
- universal shift register control
- ring/Johnson basics
- sequence generated by feedback register
- simultaneous state update

These are PYQ-style categories, not specific year claims.

---

## 69. Trap Book — Chapter 19

---

### Trap 1

Using new Q values during same synchronous clock edge.

Wrong.

Use old values for all next-state calculations.

---

### Trap 2

Assuming shift direction from drawing orientation.

Trace actual connections.

### Trap 3

Confusing SIPO and PISO.

SIPO: serial in, parallel out.

PISO: parallel in, serial out.

### Trap 4

Thinking n-bit register has  $(2^n)$  flip-flops.

Wrong.

It has:

$\boxed{n \text{ FFs}}$

### Trap 5

Thinking ring counter has  $(2^n)$  useful states.

Standard one-hot ring:

$\boxed{n}$

### Trap 6

Johnson counter states = n.

Wrong.

$\boxed{2n}$

### Trap 7

Logical and arithmetic right shift are same for negative numbers.

Wrong.

Arithmetic replicates sign bit.

### Trap 8

Assuming serial bit order.

Check whether MSB or LSB enters first.

### Trap 9

Universal register always has the same control coding.

No. Operation set is standard; exact code mapping is device-dependent.

---

## 70. Formula / Concept Sheet

---

n-bit register:

$\boxed{n \text{ flip-flops}}$



possible patterns:

$\boxed{2^n}$

Right shift example:

$\boxed{Q_3^+ = S, \text{ ; } Q_2^+ = Q_3, \text{ ; } Q_1^+ = Q_2, \text{ ; } Q_0^+ = Q_1}$

Load-enable register:

$\boxed{Q_i^+ = L'Q_i + LD_i}$

Ring counter:

$\boxed{n \text{ FFs} \rightarrow n \text{ standard states}}$

Johnson counter:

$\boxed{n \text{ FFs} \rightarrow 2n \text{ standard states}}$

---

## 71. 30-Second Revision

Remember:

### Register

Group of flip-flops storing a multi-bit word.

### Shift register

Moves bits one position per clock.

### Types

$\boxed{\text{SISO, SIPO, PISO, PIPO}}$

### Universal shift register

Usually supports:

- hold
- shift left
- shift right
- parallel load

### Critical timing rule

$\boxed{\text{All FFs update simultaneously from old state}}$

And previews:

$\boxed{\text{Ring: } n \text{ states}}$

$\boxed{\text{Johnson: } 2n \text{ states}}$

---

## What You Absolutely Must Remember

1. One flip-flop stores one bit.
2. n-bit register needs n flip-flops.
3. SISO = serial in serial out.

4. SIPO = serial in parallel out.
5. PISO = parallel in serial out.
6. PIPO = parallel in parallel out.
7. Shift direction must be determined from actual signal connections.
8. Synchronous stages update simultaneously.
9. Logical right shift inserts 0.
10. Arithmetic right shift repeats the sign bit.
11. Universal shift register supports hold, left, right and parallel load.
12. n-FF ring counter has n standard states.
13. n-FF Johnson counter has 2n standard states.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' **Registers & Shift Registers ' !'**

**Current: Chapter 20 — Counters**

**Remaining:** Counters !' Sequential Analysis !' Sequential Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT — Chapter 20: COUNTERS 0=MUST MASTER

Next we cover:

asynchronous/ripple counter !' synchronous counter !' up/down counter !' MOD-N counters !' minimum flip-flops !' frequency division !' propagation-delay numericals !' ring counter !' Johnson counter !' truncated counters !' self-starting concepts !' counter state sequences !' GATE traps.

USER

next

ASSISTANT

## CHAPTER 20 — COUNTERS 0=MUST MASTER

A **counter** is a sequential circuit that moves through a prescribed sequence of states in response to clock pulses.

Typical binary up-counter sequence:

\[  
000\rightarrow001\rightarrow010\rightarrow011\rightarrow100\rightarrow101\rightarrow110\rightarrow111\rightarrow000  
\]

Counters are extremely important for GATE because they combine:

- flip-flops
- state transitions
- propagation delay

- frequency division
- MOD-N concepts
- synchronous vs asynchronous design

## 1. What Is a Counter?

A counter counts clock events.

For example, a 3-bit binary counter can represent:

$0$  to  $7$

because:

$2^3=8$

states are available.

Each clock changes the stored state according to the counter design.

## 2. Main Types of Counters

Two major categories:

### 1. Asynchronous counter

Also called:

$\boxed{\text{Ripple Counter}}$

### 2. Synchronous counter

All flip-flops receive the same clock.

We must understand the difference very clearly.

## 3. Asynchronous / Ripple Counter

In an asynchronous counter:

- external clock drives only the first flip-flop
- output of one flip-flop clocks the next
- state changes ripple from one stage to another

Conceptually:

|      |   |   |   |    |    |    |   |    |    |    |    |
|------|---|---|---|----|----|----|---|----|----|----|----|
| TEXT |   |   |   |    |    |    |   |    |    |    |    |
| C    | L | K | ! | F  | F  | 0  | ! | F  | F  | 1  | !  |
|      |   |   |   | Q0 | Q1 | Q2 |   | Q2 | Q1 | Q0 | Q3 |

The flip-flops do **not** all receive the external clock simultaneously.

## 4. Why Is It Called Ripple?

Suppose Q0 toggles first.

After its propagation delay:

$$\lceil Q_1 \rceil$$

may toggle.

Then after another delay:

$$\lceil Q_2 \rceil$$

may toggle.

So changes move stage by stage like a ripple.

---

## 5. T Flip-Flops in Counters

---

A T flip-flop with:

$$\lceil T=1 \rceil$$

toggles every active clock edge.

Thus it naturally performs binary counting.

Similarly a JK flip-flop with:

$$\lceil J=K=1 \rceil$$

behaves as a toggle FF.

Therefore ripple counters are often drawn using:

$$\lceil \boxed{T=1} \rceil$$

or:

$$\lceil \boxed{J=K=1} \rceil$$

---

## 6. Frequency Division

---

Each toggling flip-flop divides its input frequency by 2.

If:

$$\lceil f_{CLK}=f \rceil$$

then first stage:

$$\lceil f_{Q_0}=\frac{f}{2} \rceil$$

second:

$$\lceil f_{Q_1}=\frac{f}{4} \rceil$$

third:

$$\lceil f_{Q_2}=\frac{f}{8} \rceil$$

and generally:

$$\lceil \boxed{f_{Q_k}=\frac{f}{2^{k+1}}} \rceil$$

if  $Q_0$  is the first stage.

---

## 7. Example

---

Clock:

$$f_{\text{Clock}} = 16 \text{ MHz}$$

3-stage binary ripple counter:

$$f_{Q_0} = 8 \text{ MHz}$$

$$f_{Q_1} = 4 \text{ MHz}$$

$$f_{Q_2} = 2 \text{ MHz}$$

Thus counters are also frequency dividers.

---

## 8. 3-Bit Binary Up Counter

---

State sequence:

$$000$$

$$001$$

$$010$$

$$011$$

$$100$$

$$101$$

$$110$$

$$111$$

then back:

$$000$$

Therefore:

$$\boxed{\text{MOD-8 counter}}$$

because it has 8 distinct states.

---

## 9. MOD of a Counter

---

The **modulus** or MOD number is the number of distinct states before the sequence repeats.

So:

### 3-bit binary counter

$$\text{MOD} = 2^3 = 8$$

### 4-bit binary counter

$$\text{MOD} = 16$$

In general:

$$\boxed{\text{Full binary n-bit counter is MOD-}2^n}$$

---

## 10. Minimum Flip-Flops for MOD-N Counter Ø=Ý4

---

Need minimum  $\lceil n \rceil$  such that:

$$2^n \geq N$$

Therefore:

$$n = \lceil \log_2 N \rceil$$

Example:

MOD-10 counter:

$$2^3 = 8 < 10$$

$$2^4 = 16 \geq 10$$

So need:

$$4 \text{ flip-flops}$$

---

## 11. Unused States

---

A MOD-10 counter using 4 FFs has:

$$16$$

possible binary states.

But only:

$$10$$

are used.

Unused states:

$$16 - 10 = 6$$

Thus:

$$6 \text{ unused states}$$

---

## 12. Decade Counter

---

A MOD-10 counter is also called a:

$$\text{decade counter}$$

Typical count:

$$0000 \rightarrow 0001 \rightarrow \cdots \rightarrow 1001$$

then:

$$0000$$

instead of continuing to 1010.

---

## 13. Truncated Counter

---

A counter using fewer than all  $(2^n)$  possible states is often called a:

$\boxed{\text{truncated counter}}$

Example:

4 FFs normally support MOD-16.

If reset after state 9:

$\boxed{\text{MOD-10 truncated counter}}$

---

## 14. Asynchronous MOD-10 Counter

---

One simple implementation:

Use a 4-bit ripple counter.

Detect state:

$1010_2 = 10$

and asynchronously clear the counter to:

$0000$

Thus stable count sequence becomes approximately:

$0, 1, \dots, 9, 0$

The state 10 triggers reset.

---

## 15. Important Reset-State Idea

---

For a MOD-N ripple counter built from a binary counter, we often detect the **first unwanted state**:

$N$

Then reset to 0.

Example MOD-6:

desired states:

$000, 001, 010, 011, 100, 101$

First unwanted state:

$110_2 = 6$

Detect:

$110$

and clear.

---

## 16. MOD-6 Example

---

Minimum FFs:

$$\lceil 2^2=4<6 \rceil$$

$$\lceil 2^3=8\geq 6 \rceil$$

Need:

$$\lceil \boxed{3} \rceil$$

flip-flops.

Unused states:

$$\lceil 8-6=2 \rceil$$

Typical sequence:

$$\lceil 000\rightarrow 001\rightarrow 010\rightarrow 011\rightarrow 100\rightarrow 101\rightarrow 000 \rceil$$

## 17. Ripple Counter Propagation Delay $\varnothing = 4$

Ripple counters are simple but slow.

If each flip-flop propagation delay is:

$$\lceil t_p \rceil$$

then worst-case state settling time for n stages can be approximately:

$$\lceil \boxed{nt_p} \rceil$$

because changes ripple through all stages.

## 18. Maximum Clock Frequency for Ripple Counter

To let the counter settle before next input clock:

$$\lceil T_{clk} \geq nt_p \rceil$$

Therefore approximately:

$$\lceil \boxed{f_{max} \approx \frac{1}{nt_p}} \rceil$$

under the simplified equal-delay model.

## 19. Example

4-bit ripple counter.

Each FF:

$$\lceil t_p = 5 \text{ ns} \rceil$$

Worst-case ripple delay:

$$\lceil 4 \times 5 = 20 \text{ ns} \rceil$$

Thus:

$$\lceil f_{max} \approx \frac{1}{20 \text{ ns}} \rceil$$

$$\lceil = 50 \text{ MHz} \rceil$$



So:

$\boxed{50 \text{ MHz}}$

approximately.

---

## 20. Temporary Intermediate States Ø=Ý4

---

Because ripple counter FFs do not update simultaneously, outputs may briefly pass through incorrect intermediate states.

Example transition:

$0111 \rightarrow 1000$

Ideally all four relevant bits change.

But ripple sequence may transiently look like:

$0111 \rightarrow 0110 \rightarrow 0100 \rightarrow 0000 \rightarrow 1000$

depending on edge conventions and propagation.

These temporary states can cause glitches in decoded outputs.

---

## 21. Why Ripple Counter Is Asynchronous

---

Only first FF receives external clock.

Other FFs are clocked by outputs of previous FFs.

Therefore transitions are not simultaneous.

Hence:

$\boxed{\text{asynchronous}}$

---

## PART B — SYNCHRONOUS COUNTERS

---

### 22. Synchronous Counter Ø=Ý4

---

In a synchronous counter:

$\boxed{\text{all flip-flops receive the same clock}}$

At the active edge, all FFs evaluate their inputs simultaneously.

Combinational logic determines which FFs should toggle.

---

### 23. Main Advantage

---

Because all FFs share the same clock:

- no ripple through clock chain
  - faster
  - outputs change much closer together
-

Therefore synchronous counters scale better to high clock speeds.

---

## 24. 3-Bit Synchronous Up Counter Using T FFs

---

Let outputs:

$\{Q_2, Q_1, Q_0\}$

For an up counter:

### Q0

toggles every clock:

$\boxed{T_0=1}$

### Q1

toggles when  $Q_0=1$ :

$\boxed{T_1=Q_0}$

### Q2

toggles when both lower bits are 1:

$\boxed{T_2=Q_1Q_0}$

This is extremely important.

---

## 25. General n-Bit Synchronous Up Counter

---

For T FFs:

$\boxed{T_0=1}$

$\boxed{T_1=Q_0}$

$\boxed{T_2=Q_1Q_0}$

$\boxed{T_3=Q_2Q_1Q_0}$

In general:

$\boxed{T_i = \prod_{k=0}^{i-1} Q_k}$

for  $(i>0)$ .

Meaning:

**A higher bit toggles when all lower-order bits are 1.**

---

## 26. Why?

---

Binary increment:

$\{0111+1=1000\}$

A bit toggles when a carry reaches it.

Carry reaches bit  $i$  only when every lower bit is 1.

Thus synchronous counter toggle equations mirror binary carry logic.

---

## 27. JK Implementation

---

Since JK toggles when:

$$J=K=1$$

set:

$$J_i=K_i=T_i$$

Thus:

$$J_0=K_0=1$$

$$J_1=K_1=Q_0$$

$$J_2=K_2=Q_1Q_0$$

etc.

---

## 28. D Flip-Flop Implementation

---

For D FF:

$$D_i=Q_i'$$

So derive next-state expressions.

For LSB:

$$Q_0' = Q_0$$

Thus:

$$\boxed{D_0=Q_0'}$$

For Q1:

$$Q_1' = Q_1 \oplus Q_0$$

So:

$$\boxed{D_1=Q_1 \oplus Q_0}$$

For Q2:

$$Q_2' = Q_2 \oplus (Q_1Q_0)$$

Thus:

$$\boxed{D_2=Q_2 \oplus Q_1Q_0}$$

---

## 29. Synchronous Counter Delay

---

All FFs get clock together, but combinational toggle logic still has delay.

Clock-period constraint resembles ordinary synchronous timing:

---

$$T_{\text{clk}} \geq t_{\text{CQ}} + t_{\text{logic}} + t_{\text{setup}}$$

Unlike ripple counters, delay does not grow simply as  $(n \cdot t_p)$  through flip-flop clock stages.

## 30. Ripple vs Synchronous Master Table

| Feature        | Ripple            | Synchronous              |
|----------------|-------------------|--------------------------|
| Clocking       | first FF only     | all FFs                  |
| State change   | sequential/ripple | near-simultaneous        |
| Speed          | slower            | faster                   |
| Hardware logic | simpler           | more combinational logic |
| Glitch risk    | larger            | lower                    |
| Large counters | poorer            | better                   |

## PART C — DOWN COUNTERS

### 31. Binary Down Counter

A down counter decrements:

$$111 \rightarrow 110 \rightarrow 101 \rightarrow 100 \rightarrow 011 \rightarrow 010 \rightarrow 001 \rightarrow 000 \rightarrow 111$$

For 3 bits, still:

$$\text{MOD-8}$$

but direction is downward.

### 32. Synchronous Down Counter Using T FFs

Q0 still toggles every clock:

$$T_0 = 1$$

Q1 toggles when lower bit is 0:

$$T_1 = Q_0'$$

Q2 toggles when all lower bits are 0:

$$T_2 = Q_1' Q_0'$$

General:

$$T_i = \prod_{k=0}^{i-1} Q_k'$$

for down counting.

---

## 33. Memory Trick

---

### Up counter

higher bit toggles when all lower bits are:

$$\boxed{1}$$

### Down counter

higher bit toggles when all lower bits are:

$$\boxed{0}$$

---

## PART D — UP/DOWN COUNTER

---

## 34. Up/Down Counter

---

A control input U may select direction.

Example:

$$U=1 \rightarrow \text{UP}$$

$$U=0 \rightarrow \text{DOWN}$$

Exact polarity is design-dependent.

---

## 35. Toggle Condition

---

For bit 1:

Need toggle if:

- UP and  $Q_0=1$
- DOWN and  $Q_0=0$

So:

$$T_1 = UQ_0 + U'Q_0'$$

Recognize:

$$\boxed{T_1 = U \odot Q_0}$$

XNOR, for this chosen control convention.

---

## 36. General Up/Down Logic

---

For higher bit  $i$ :

Toggle when:

- UP: all lower bits =1
  - DOWN: all lower bits =0
-

Thus one form:

$$T_i = U(Q_{i-1} \cdot Q_0) + U'(Q_{i-1} \cdot Q_0)$$

---

## PART E — RING COUNTER

---

### 37. Ring Counter Ø=4

---

A ring counter is a shift register with last output fed directly back to first input.

Example 4 FFs, initialized:

1000

Then:

0100

0010

0001

1000

repeat.

---

### 38. Ring Counter Number of States

---

For n FFs with a one-hot pattern:

$\boxed{n \text{ states}}$

So 8 FF ring counter:

$\boxed{\text{MOD-8}}$

even though 8 FFs could theoretically represent:

$2^8=256$

binary patterns.

---

### 39. Ring Counter Initialization

---

A standard one-hot ring counter needs a valid initial pattern such as:

1000

If initialized to:

0000

direct feedback keeps it:

0000

forever.

So initialization matters.

---

---

## 40. Ring Counter Efficiency

---

To obtain MOD-N ring counter:

$\boxed{N \text{ flip-flops}}$

This is hardware-inefficient compared with binary counter, which needs:

$\lceil \log_2 N \rceil$

FFs.

But decoding ring states is very simple.

---

## PART F — JOHNSON COUNTER

---

### 41. Johnson Counter $\text{Ø} = 4$

---

Also called:

$\boxed{\text{twisted ring counter}}$

Instead of feeding last output back directly, feed its complement.

If last output:

$Q_{n-1}$

feedback is:

$\boxed{Q_{n-1}'}$

---

### 42. 4-Bit Johnson Example

---

Suppose initial:

$0000$

Assume complemented last bit enters left.

Sequence:

$0000$

$1000$

$1100$

$1110$

$1111$

$0111$

$0011$

$0001$

$0000$

---

That is:

$$\boxed{2^n \text{ states}}$$

## 43. Johnson Number of States

For n flip-flops:

$$\boxed{2n \text{ states}}$$

Therefore:

- 3 FF !' 6 states
- 4 FF !' 8 states
- 5 FF !' 10 states

## 44. Ring vs Johnson

| Property          | Ring                 | Johnson                      |
|-------------------|----------------------|------------------------------|
| Feedback          | Q                    | Q'                           |
| States with n FFs | n                    | 2n                           |
| Initialization    | important            | important                    |
| Structure         | shift-register based | twisted shift-register based |

## 45. Minimum FF Comparison

Want MOD-8.

### Binary counter

Need:

$$\lceil \log_2 8 \rceil = 3$$

FFs.

### Ring counter

Need:

$$8$$

FFs.

### Johnson counter

Need:

$$2n=8$$

so:

$$n=4$$



FFs.

Therefore:

$\boxed{\text{Binary uses fewest FFs}}$

---

## PART G — STATE SEQUENCE QUESTIONS

---

### 46. Counter State Table

---

Suppose a 3-bit counter sequence is:

$000 \rightarrow 001 \rightarrow 011 \rightarrow 111 \rightarrow 000$

Then MOD is:

$\boxed{4}$

because only four distinct states appear before repeating.

MOD is not determined merely by number of FFs.

---

### 47. MOD Based on Actual Reachable States

---

A circuit with 4 FFs can theoretically represent 16 states.

But if its sequence uses only:

$0000, 0001, 0010, 0011, 0000$

then:

$\boxed{\text{MOD}=4}$

---

### 48. Self-Starting Counter

---

A counter is called **self-starting** if, from unused/invalid states, it eventually enters the intended counting sequence automatically.

Example:

A MOD-6 design using 3 FFs has unused states:

$110, 111$

If entering either state eventually returns to:

$000 \text{ to } 101$

without external reset, it is self-starting.

---

### 49. Lockout

---

If an invalid state leads into a loop of invalid states and never enters valid counting sequence, the counter may be:

$\boxed{\text{locked out}}$

---

This is undesirable.

---

## 50. Counter Design Idea

---

For a custom synchronous MOD-N counter:

1. Write desired state sequence.
2. Create present-state / next-state table.
3. Choose FF type.
4. Use excitation table.
5. Derive FF input equations with K-map.
6. Implement.

This will be taught in the sequential design chapter.

---

## PART H — FREQUENCY DIVISION

---

### 51. Divide-by- $(2^n)$

---

An n-stage binary counter naturally divides frequency by:

$$2^n$$

Final stage frequency:

$$f_{\text{out}} = \frac{f_{\text{clk}}}{2^n}$$

---

### 52. Example

---

Input clock:

$$1 \text{ MHz}$$

10-stage binary counter:

$$f_{\text{out}} = \frac{1 \text{ MHz}}{1024}$$

$$\approx 976.56 \text{ Hz}$$

---

### 53. MOD-N Frequency Division

---

If an entire MOD-N counter sequence repeats once every N input clocks:

$$f_{\text{sequence}} = \frac{f_{\text{clk}}}{N}$$

This refers to one full count cycle.

Individual output waveforms may have different frequencies/duty cycles.

---

### 54. Example — MOD-10

---

Clock:

---

$1 \text{ kHz}$

Complete decade count repeats every 10 pulses.

Sequence repetition rate:

$100 \text{ Hz}$

---

## 55. Duty Cycle

---

Binary counter bits normally have approximately 50% duty cycle.

But outputs from arbitrary MOD-N decoded states may not.

For example, a pulse that becomes 1 for one count out of 10 has:

$10\%$

duty cycle.

---

## PART I — RIPPLE DELAY NUMERICALS

---

### 56. Example

---

5-stage asynchronous counter.

Each FF propagation delay:

$4 \text{ ns}$

Worst-case settling:

$5 \times 4 = 20 \text{ ns}$

Thus approximate maximum frequency:

$\frac{1}{20 \text{ ns}} = 50 \text{ MHz}$

$50 \text{ MHz}$

---

### 57. Important Caveat

---

Real counter timing may include:

- different rise/fall delays
- asynchronous reset delay
- decoder delay
- setup requirements

So use the exact timing model given.

Do not blindly use  $(n \times t_p)$  if additional delays are specified.

---

### 58. MOD Counter With Reset Delay

---

Suppose asynchronous MOD-10 counter:

---

- ripple settling to 1010 takes 20 ns
- reset logic adds 5 ns

Then worst-cycle timing may involve:

$$\boxed{20+5=25\text{ ns}}$$

depending on architecture.

Thus approximate maximum input frequency:

$$\boxed{40\text{ MHz}}$$

if that is the required full recovery time.

## PART J — GATE PRACTICE

### 59. L0 — MOD

A 4-bit full binary counter is:

$$\boxed{\text{MOD-16}}$$

### 60. L0 — Flip-Flops

Minimum FFs for MOD-12?

$$\boxed{2^3=8<12}$$

$$\boxed{2^4=16\geq 12}$$

Answer:

$$\boxed{4}$$

### 61. L0 — Frequency

Clock:

$$\boxed{8\text{ MHz}}$$

First toggle FF output:

$$\boxed{4\text{ MHz}}$$

Second:

$$\boxed{2\text{ MHz}}$$

### 62. L1 — Unused States

MOD-10 with 4 FFs:

$$\boxed{16-10=6}$$

$$\boxed{6}$$

unused states.

---

## 63. L1 — Ring Counter

---

7-FF standard one-hot ring counter:

$\boxed{7 \text{ states}}$

---

## 64. L1 — Johnson

---

7-FF Johnson counter:

$2 \times 7 = 14$

$\boxed{14 \text{ states}}$

---

## 65. L1 — Up Counter T Inputs

---

3-bit synchronous up counter:

$T_0 = 1$

$T_1 = Q_0$

$T_2 = Q_1 Q_0$

---

## 66. L1 — Down Counter T Inputs

---

3-bit synchronous down:

$T_0 = 1$

$T_1 = Q_0'$

$T_2 = Q_1' Q_0'$

---

## 67. L2 — Sequence MOD

---

Counter:

$000 \rightarrow 011 \rightarrow 101 \rightarrow 110 \rightarrow 000$

Distinct states:

$4$

Thus:

$\boxed{\text{MOD-4}}$

even though 3 FFs are present.

---

## 68. L2 — Ripple Delay

---

8-bit ripple counter.

Each FF:

$2\text{ ns}$

Approximate worst delay:

$8 \times 2 = 16\text{ ns}$

Maximum frequency:

$\frac{1}{16\text{ ns}} = 62.5\text{ MHz}$

$62.5\text{ MHz}$

---

## 69. L2 — Output Frequency

---

Input:

$64\text{ MHz}$

6th counter stage if first stage is divide-by-2:

$\frac{64}{2^6}$

$= 1\text{ MHz}$

$1\text{ MHz}$

---

## 70. L2 — MOD-6 Reset

---

3-bit ripple counter should count 0–5.

First invalid count:

$6 = 110$

So detect:

$110$

and reset to 000.

Exact reset gate polarity depends on active-high/active-low clear.

---

## 71. L3 — Counter Type

---

Which is faster for large bit widths?

Generally:

$\text{synchronous counter}$

because carry/toggle decision is generated through combinational logic while all FFs share the same clock.

---

## 72. L3 — Ring Initialization Trap

---

4-FF ring counter starts:

---

$\lfloor 0000 \rfloor$

with direct Q feedback.

What happens?

It remains:

$\lfloor \boxed{0000} \rfloor$

indefinitely.

It does not magically generate one-hot sequence.

---

### 73. L3 — Johnson Sequence

---

4-FF Johnson counter has how many states?

$\lfloor 2n=8 \rfloor$

$\lfloor \boxed{8} \rfloor$

Not:

$\lfloor 16 \rfloor$

---

### 74. L3 — Minimum FFs for Johnson MOD-10

---

Johnson with n FFs has:

$\lfloor 2n \rfloor$

states.

Need:

$\lfloor 2n=10 \rfloor$

So:

$\lfloor \boxed{n=5} \rfloor$

flip-flops.

---

### 75. L3 — Ring MOD-10

---

Standard one-hot ring counter requires:

$\lfloor \boxed{10 \text{ flip-flops}} \rfloor$

for MOD-10.

Binary counter needs only:

$\lfloor 4 \rfloor$

---

### 76. GATE PYQ Patterns

---

Common counter questions test:

---

- synchronous vs ripple behavior
- minimum FF count
- MOD number
- frequency division
- ripple propagation delay
- maximum clock frequency
- counter state sequence
- up/down toggle equations
- truncated counters
- ring counter states
- Johnson counter states
- unused states
- self-starting/lockout behavior
- asynchronous reset decoding

These are PYQ-style patterns, not specific year claims.

---

## 77. Trap Book — Chapter 20

---

### Trap 1

n FFs always means  $\text{MOD} = 2^n$ .

Wrong.

Only if all  $2^n$  states are used.

### Trap 2

Ripple counter FFs all receive external clock.

Wrong.

Only first does.

### Trap 3

Synchronous counter state bits update one after another.

Wrong.

They update together from the old state.

### Trap 4

Up-counter higher bit toggles when lower bits are 0.

Wrong.

Up !' lower bits all 1.

### Trap 5

Down-counter higher bit toggles when lower bits are 1.

Wrong.

Down !' lower bits all 0.



### Trap 6

Ring n FFs gives  $2^n$  useful states.

Wrong.

$$\boxed{n}$$

### Trap 7

Johnson n FFs gives n states.

Wrong.

$$\boxed{2n}$$

### Trap 8

MOD-N always needs N flip-flops.

Only ring counter does in standard one-hot form.

Binary MOD-N needs:

$$\lceil \log_2 N \rceil$$

### Trap 9

Ignoring ripple transient states.

They matter in decoder/glitch questions.

### Trap 10

Assuming unused states automatically return to valid sequence.

Not always.

Check self-starting behavior.

---

## 78. Counter Master Formula Sheet

Binary n-bit counter:

$$\boxed{\text{MOD}=2^n}$$

Minimum FFs for MOD-N:

$$\boxed{n=\lceil \log_2 N \rceil}$$

Unused states:

$$\boxed{2^n - N}$$

Ripple delay:

$$\boxed{T_{\text{settle}} \approx n t_p}$$

Approximate ripple maximum frequency:

$$\boxed{f_{\text{max}} \approx \frac{1}{n t_p}}$$

Synchronous up:

$$\boxed{T_0=1}$$

$$\boxed{T_i = Q_{i-1} Q_{i-2} \cdots Q_0}$$

Synchronous down:

$$\boxed{T_i = Q_{i-1}' Q_{i-2} \cdots Q_0'}$$

Ring:

$$\boxed{n \text{ FFs} \rightarrow n \text{ states}}$$

Johnson:

$$\boxed{n \text{ FFs} \rightarrow 2n \text{ states}}$$

Frequency division:

$$\boxed{f_{Q_k} = \frac{f_{clk}}{2^{k+1}}}$$

---

## 79. 30-Second Revision

Remember:

### Ripple counter

clock change ripples through FFs !' simple but slow.

### Synchronous counter

all FFs share clock !' faster.

### MOD

$$\boxed{\text{number of states before repetition}}$$

### Flip-flops needed

$$\boxed{\lceil \log_2 N \rceil}$$

### Up counter

higher bit toggles when all lower bits are:

$$\boxed{1}$$

### Down counter

higher bit toggles when all lower bits are:

$$\boxed{0}$$

### Ring

$$\boxed{n}$$

states.

### Johnson

$$\boxed{2n}$$

states.

## What You Absolutely Must Remember

1. Counter = sequential state machine driven by clock pulses.
2. Ripple counter is asynchronous.
3. Synchronous counter clocks all FFs together.
4. Full n-bit binary counter has  $(2^n)$  states.
5. Minimum FFs for MOD-N =  $\lceil \log_2 N \rceil$ .
6. Ripple counter has cumulative propagation delay.
7. Each toggle stage divides frequency by 2.
8. Synchronous up counter toggles a higher bit when all lower bits are 1.
9. Synchronous down counter toggles a higher bit when all lower bits are 0.
10. MOD number is based on actual repeating state count.
11. Ring counter: n FFs !' n states.
12. Johnson counter: n FFs !' 2n states.
13. Truncated counters intentionally use fewer than  $(2^n)$  states.
14. Unused states can cause lockout unless the design is self-starting.
15. Ripple counters can briefly pass through intermediate states due to propagation delay.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' Registers ' !' **Counters** '

**Current: Chapter 21 — Sequential Circuit Analysis**

**Remaining:** Sequential Analysis !' Sequential Design !' FSM !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

### NEXT — Chapter 21

We will learn the full reverse-engineering process:

**given a flip-flop circuit !' write FF input equations !' use characteristic equations !' obtain next-state equations !' construct state table !' draw state diagram !' determine output sequence !' identify counter/FSM behavior !' solve GATE sequential-analysis problems.**

USER

next

ASSISTANT

## CHAPTER 21 — SEQUENTIAL CIRCUIT ANALYSIS 0=1111 MUST MASTER

Now we learn the reverse process:

**Given a sequential circuit, determine exactly what it does.**

Typical GATE questions give:

- flip-flops
- gates
- feedback connections
- maybe an external input

and ask you to find:

- next-state equations
- state table
- state diagram
- output sequence
- MOD value
- whether the circuit is a counter/FSM
- reachable/unreachable states

---

## 1. What Does “Analyze a Sequential Circuit” Mean?

For a combinational circuit:

$\text{Circuit} \rightarrow \text{Boolean function}$

For a sequential circuit:

$\boxed{\text{Circuit}} \rightarrow \text{FF input equations} \rightarrow Q^+ \rightarrow \text{State table} \rightarrow \text{State diagram/behavior}$

This is the master workflow.

---

## 2. Present State vs Next State

Suppose two flip-flops store:

$Q_1, Q_0$

Present state:

$\boxed{Q_1 Q_0}$

After the next active clock edge:

$\boxed{Q_1^+ Q_0^+}$

Never mix them.

All next-state calculations use the **old/present Q values**.

---

## 3. The 6-Step Analysis Method Ø=Ý4

Whenever GATE gives a synchronous sequential circuit:

### Step 1

Identify flip-flop type.

## Step 2

Write each flip-flop input equation.

Example:

$$D_1 = Q_1 \oplus Q_0$$

$$D_0 = Q_0'$$

## Step 3

Use characteristic equation of the FF.

## Step 4

Find each next-state equation.

## Step 5

Enumerate all present states and compute next states.

## Step 6

Draw state diagram / identify sequence.

---

## 4. Characteristic Equations Recap

---

You need these instantly.

### D FF

---

$$Q^{+} = D$$

### T FF

---

$$Q^{+} = T \oplus Q$$

### JK FF

---

$$Q^{+} = JQ' + K'Q$$

### SR FF

---

$$Q^{+} = S + R'Q$$

with invalid:

$$S = R = 1$$

for standard active-high SR.

---

## 5. Easiest Case — D Flip-Flop Analysis

---

Suppose circuit has two D FFs:

$$D_1 = Q_0$$

$$D_0 = Q_1'$$

Since D FF has:

$$Q_1^{n+1} = D$$

immediately:

$$Q_1^{n+1} = Q_0^n$$

$$Q_0^{n+1} = Q_1^n$$

Now build state table.

---

## 6. State Table Example

Evaluate all 4 states.

### Present 00

$$Q_1^{n+1} = Q_0^n = 0$$

$$Q_0^{n+1} = Q_1^n = 1$$

Next:

$$01$$

### Present 01

$$Q_1^{n+1} = 1$$

$$Q_0^{n+1} = 1$$

Next:

$$11$$

### Present 11

$$Q_1^{n+1} = 1$$

$$Q_0^{n+1} = 0$$

Next:

$$10$$

### Present 10

$$Q_1^{n+1} = 0$$

$$Q_0^{n+1} = 0$$

Next:

$$00$$

So:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

This circuit is MOD-4.

---

## 7. State Diagram

Represent each state as a node:

TEXT

```
0 0  !'    0 1  !'    1 1  !'    1 0
!'                                     !"
% % % % % % % % % % % % % % % %
```

Every active clock moves to the next state.

---

## 8. Important Observation

The sequence:

$\{00, 01, 11, 10\}$

is Gray-code-like because adjacent states differ by one bit.

A state sequence does not have to be ordinary binary counting.

---

## 9. Analysis Using T Flip-Flops

Suppose:

$T_0 = 1$

$T_1 = Q_0$

Characteristic:

$Q^+ = T \oplus Q$

Therefore:

$Q_0^+ = 1 \oplus Q_0 = Q_0'$

and:

$Q_1^+ = Q_0 \oplus Q_1$

This is exactly a 2-bit synchronous up counter.

---

## 10. Verify 2-Bit Counter

Start:

$\{00\}$

**00**

$Q_0^+ = 1$

$Q_1^+ = 0$

**! 01**

---

## 01

$$\lceil Q_0^+=0 \rceil$$

$$\lceil Q_1^+=1 \rceil$$

!' 10

## 10

$$\lceil Q_0^+=1 \rceil$$

$$\lceil Q_1^+=1 \rceil$$

!' 11

## 11

$$\lceil Q_0^+=0 \rceil$$

$$\lceil Q_1^+=0 \rceil$$

!' 00

Thus:

$$\lceil \boxed{00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00} \rceil$$

---

## 11. T-FF Analysis Shortcut

For T FF:

### T=0

$$\lceil Q^+=Q \rceil$$

Hold.

### T=1

$$\lceil Q^+=Q' \rceil$$

Toggle.

So often you can analyze without writing XOR algebra every time.

---

## 12. JK Flip-Flop Analysis

Suppose one JK FF has:

$$\lceil J=X \rceil$$

$$\lceil K=1 \rceil$$

Characteristic equation:

$$\lceil Q^+=JQ'+K'Q \rceil$$

Since:

$$\lceil K'=0 \rceil$$

$$\lceil \boxed{Q^+=XQ'} \rceil$$



Now analyze by Q.

If  $Q=0$ :

$$Q^+ = X$$

If  $Q=1$ :

$$Q^+ = 0$$

This kind of simplification is common.

## 13. JK Behavioral Shortcut

Instead of equation, use:

| J | K | Action |
|---|---|--------|
| 0 | 0 | Hold   |
| 0 | 1 | Reset  |
| 1 | 0 | Set    |
| 1 | 1 | Toggle |

For many GATE state-table questions, this is faster.

## 14. Example — 2 JK FFs

Suppose:

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0$$

Then:

FF0 toggles every edge.

FF1:

- if  $(Q_0=0)$ :  $(J_1=K_1=0)$  !' hold
- if  $(Q_0=1)$ :  $(J_1=K_1=1)$  !' toggle

This again creates:

$$\boxed{\text{2-bit synchronous up counter}}$$

## 15. Analysis Using SR Flip-Flops

Suppose:

$$S = Q'$$

$$R = Q$$

If  $Q=0$ :

$\lceil S=1, R=0 \rceil$

set !' 1.

If  $Q=1$ :

$\lceil S=0, R=1 \rceil$

reset !' 0.

Thus:

$\lceil \boxed{Q^{+}=Q'} \rceil$

The SR FF behaves as a toggle here.

## 16. Check Invalid SR Inputs

Whenever analyzing SR:

You must verify whether:

$\lceil S=R=1 \rceil$

can ever occur.

If yes:

- state may be invalid/undefined
- question may expect you to identify invalid condition

Do not blindly compute characteristic equation there.

## 17. State Table Format

A standard state table looks like:

| Present State | Input | Next State | Output |
|---------------|-------|------------|--------|
|---------------|-------|------------|--------|

For autonomous counters with no external input:

| Present State | Next State |
|---------------|------------|
|---------------|------------|

Example:

| PS | NS |
|----|----|
| 00 | 01 |
| 01 | 10 |
| 10 | 11 |
| 11 | 00 |

## 18. Circuits With External Input

Suppose state is:

$$[Q]$$

and input:

$$[X]$$

with:

$$[D = X \oplus Q]$$

Then:

$$[Q^{+} = X \oplus Q]$$

Now state transition depends on X.

## 19. State Table With Input

| Q | X | $Q^{+}$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 0       |

Thus:

- $X=0$  !' hold
- $X=1$  !' toggle

So this D-FF circuit behaves like a T FF with:

$$[T = X]$$

## 20. State Diagram With Inputs

For the previous circuit:

From state 0:

- $X=0$  !' 0
- $X=1$  !' 1

From state 1:

- $X=0$  !' 1
- $X=1$  !' 0

Conceptually:

TEXT

```
X=1
0  % % % % % % ! ' 1
! '          ! "
% % % % % % % % %
X=1
```

X=0 gives self-loop at each state.

## 21. Output Equations

A sequential circuit may also have combinational outputs.

Example:

$\forall [Y = Q_1X + Q_0']$

After determining current state/input, evaluate Y.

Important:

Output may depend on:

- state only
- state + input

This leads into Moore and Mealy FSMs later.

## 22. Moore-Like Output

If:

$\forall [Y = f(Q)]$

then output depends only on present state.

Example:

$\forall [Y = Q_1Q_0]$

Only state 11 gives output 1.

## 23. Mealy-Like Output

If:

$\forall [Y = f(Q, X)]$

then output depends on present state and input.

Example:

$\forall [Y = XQ]$

Even within same state, output may change when X changes.

We cover Moore vs Mealy formally in Chapter 23.

## 24. Reachable State

---

A state is **reachable** if the machine can enter it from the chosen initial state through some valid input sequence.

Example:

State diagram:

$\{ 00 \rightarrow 01 \rightarrow 10 \rightarrow 00 \}$

and state 11 goes only to itself.

If initial state is 00:

$\{ 11 \}$

is unreachable.

---

## 25. Unreachable States Matter

---

If a circuit has  $n$  FFs:

$\{ 2^n \}$

bit patterns exist.

But not all must be reachable.

Example:

3 FFs !' 8 patterns.

Counter may use only 6.

Analysis should identify:

- valid cycle
  - unused states
  - whether unused states enter valid cycle
- 

## 26. Self-Starting Analysis

---

Suppose intended sequence:

$\{ 000 \rightarrow 001 \rightarrow 010 \rightarrow 000 \}$

Unused states eventually transition to 000.

Then circuit is self-starting.

If unused states form their own loop:

$\{ 110 \leftrightarrow 111 \}$

then lockout is possible.

---

## 27. Example — Find Reachable Cycle

---

Suppose 2-bit next state:

---

$$Q_1^{n+1} = Q_0^n$$

$$Q_0^{n+1} = Q_1^n$$

Evaluate:

**00**

$$Q_1^{n+1} = 0, Q_0^{n+1} = 0$$

! 00

**01**

$$Q_1^{n+1} = 1, Q_0^{n+1} = 0$$

! 10

**10**

$$Q_1^{n+1} = 0, Q_0^{n+1} = 0$$

! 00

**11**

$$Q_1^{n+1} = 1, Q_0^{n+1} = 0$$

! 10

So 00 is absorbing.

If initialized 01:

$$01 \rightarrow 10 \rightarrow 00 \rightarrow 00$$

## 28. Absorbing State

A state that transitions to itself is a self-loop.

If once entered it can never leave under relevant inputs, it may be an:

$$\boxed{\text{absorbing state}}$$

Example:

$$00 \rightarrow 00$$

## 29. State Sequence From Initial State

GATE may ask:

**Given initial state 01, what are the next four states?**

Do not list every possible state.

Start at given initial state and repeatedly apply next-state equations.

### 30. Example

---

Given:

$$Q_1^{+} = Q_0$$

$$Q_0^{+} = Q_1'$$

Initial:

$$00$$

We previously found:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

Next four states after initial are:

$$\boxed{01, 11, 10, 00}$$

---

### 31. Output Sequence

---

Suppose output:

$$Y = Q_1$$

For state cycle:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

Y values:

- 00 !' 0
- 01 !' 0
- 11 !' 1
- 10 !' 1

So repeating output:

$$\boxed{0011, 0011 \dots}$$

---

### 32. Determine MOD From State Graph

---

If cycle is:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

number of distinct states:

$$4$$

So:

$$\boxed{\text{MOD}-4}$$

Do not determine MOD just from FF count.

---

### 33. Autonomous Sequential Circuit

---

A sequential circuit with no external input:

---

$$\forall Q^{+}=f(Q) \quad \forall$$

is autonomous.

Counters are typical examples.

The next state is fully determined by present state.

---

### 34. Input-Driven Sequential Circuit

---

If:

$$\forall Q^{+}=f(Q,X) \quad \forall$$

then next state depends on external input.

This is typical FSM behavior.

---

### 35. Analysis of a Register-Like Circuit

---

Suppose two D FFs:

$$\forall D_1=X \quad \forall$$

$$\forall D_0=Q_1 \quad \forall$$

Then:

$$\forall Q_1^{+}=X \quad \forall$$

$$\forall Q_0^{+}=Q_1 \quad \forall$$

This is a 2-stage shift register.

At every edge:

- new X enters Q1
- old Q1 shifts into Q0

Recognizing structure can save time.

---

### 36. Analysis of a Toggle Chain

---

Suppose:

$$\forall T_0=1 \quad \forall$$

$$\forall T_1=Q_0 \quad \forall$$

$$\forall T_2=Q_1Q_0 \quad \forall$$

Immediately recognize:

$$\forall \boxed{\text{3-bit synchronous up counter}} \quad \forall$$

You can still derive state table if needed.

---

### 37. Analysis of Down Counter

---



If:

$$\lceil T_0=1 \rceil$$

$$\lceil T_1=Q_0' \rceil$$

$$\lceil T_2=Q_1'Q_0' \rceil$$

Recognize:

$$\lceil \boxed{\text{3-bit synchronous down counter}} \rceil$$

---

## 38. Gate Network !' FF Input Equation

Suppose D input comes from:

TEXT

Q 1 % % N O T % %  
O R % % D 0  
X % % % % % % % %

Then:

$$\lceil \boxed{D_0=Q_1'+X} \rceil$$

Always label intermediate signals if network is complex.

---

## 39. Complex Circuit Strategy

For each FF:

1. trace backward from input pin
2. write Boolean equation
3. simplify if useful
4. only then apply FF characteristic equation

Do not mentally simulate gate network and FF behavior simultaneously.

---

## 40. Example — JK Circuit

Suppose:

$$\lceil J=Q' \rceil$$

$$\lceil K=X \rceil$$

Then:

$$\lceil Q^+=JQ'+K'Q \rceil$$

Substitute:

$$\lceil Q^+=Q'Q'+X'Q \rceil$$

$$\lceil =Q'+X'Q \rceil$$

Use identity:

$$\forall [A + A'B = A + B]$$

with  $\forall (A = Q')$ :

$$\forall [Q' + QX' = Q' + X']$$

Thus:

$$\forall [\boxed{Q^{++} = Q' + X'}]$$

You can then build state table.

## 41. Alternative Behavioral Analysis

For same JK:

$$\forall [J = Q', \text{quad } K = X]$$

Case  $Q=0$ :

$$\forall [J = 1]$$

- $X=0$  !' JK 10 !' set !' 1
- $X=1$  !' JK 11 !' toggle 0! 1

So  $Q^{++}=1$  always.

Case  $Q=1$ :

$$\forall [J = 0]$$

- $X=0$  !' JK 00 !' hold 1
- $X=1$  !' JK 01 !' reset 0

Thus:

$$\forall [Q^{++} = X']$$

when  $Q=1$ .

Same result overall.

Sometimes behavior table is faster than algebra.

## 42. State Assignment

If a state machine is described symbolically with states:

$$\forall [A, B, C, D]$$

actual flip-flops store binary codes such as:

$$\forall [A=00, \text{ } B=01, \text{ } C=10, \text{ } D=11]$$

State assignment matters during circuit implementation.

For analysis of an already-built circuit, binary  $Q$  values are usually the states directly.

## 43. Don't Confuse State Name and Output

A state like:

$\lceil 10 \rceil$

means:

$\lceil Q_1=1, Q_0=0 \rceil$

It does not necessarily mean numeric count 2 unless the machine is intended as a counter.

State bits are just encoded memory.

---

## 44. Timing Diagram Analysis

---

Given a synchronous sequential circuit:

At each active clock edge:

1. evaluate FF inputs from **old state**
2. determine next state
3. after  $\lceil t_{CQ} \rceil$ , Q outputs update

Do not propagate next-state changes through other FF input logic within the same clock event.

---

## 45. Example — Simultaneous Update

---

Suppose:

$\lceil D_1=Q_0 \rceil$

$\lceil D_0=Q_1 \rceil$

Initial:

$\lceil Q_1Q_0=10 \rceil$

At active edge:

$\lceil Q_1^+=\text{old } Q_0=0 \rceil$

$\lceil Q_0^+=\text{old } Q_1=1 \rceil$

So:

$\lceil \boxed{01} \rceil$

Not 00.

---

## 46. Swapping Register

---

The previous circuit:

$\lceil D_1=Q_0, \quad D_0=Q_1 \rceil$

swaps the two bits every clock.

Thus:

$\lceil 10 \rightarrow 01 \rceil$

while:

$\lceil 00 \rightarrow 00 \rceil$

---

$\lceil 11 \rightarrow 11 \rceil$

Recognizing this reduces calculation.

---

## 47. State Transition Graph

For the swapping register:

TEXT

0 0 !°

1 0 !" 0 1

1 1 !°

So there are multiple separate cycles.

Initial state determines which cycle is followed.

---

## 48. Multiple Cycles

A sequential circuit may not have one single state cycle.

Example:

- 00 fixed
- 11 fixed
- 01 !" 10

Therefore asking “what sequence does circuit generate?” is incomplete unless initial state is given.

This is a common trap.

---

## 49. Reset Resolves Initial State

Many practical circuits include reset to force known initial state.

Example:

$\lceil \text{RESET} \rightarrow Q=00 \rceil$

Then only state sequence reachable from 00 matters after reset.

---

## 50. State Transition Matrix Idea Ø=ßá

For small deterministic FSMs, state table fully defines mapping:

$\lceil PS \rightarrow NS \rceil$

You usually do not need matrix methods for GATE Digital Logic.

State table/diagram is enough.

## 51. L0 Practice

---

Given D FF:

$$D = Q'$$

Then:

$$Q^{n+1} = Q'$$

Therefore the FF:

$$\boxed{\text{toggles every clock}}$$

## 52. L0 Practice

---

Given T FF:

$$T = 0$$

Then:

$$\boxed{Q^{n+1} = Q}$$

Hold.

## 53. L1 — Identify Counter

---

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_1 Q_0$$

Answer:

$$\boxed{\text{3-bit synchronous up counter}}$$

## 54. L1 — D Analysis

---

Given:

$$D_1 = Q_1'$$

$$D_0 = Q_0'$$

Every clock complements both bits.

Thus:

$$00 \rightarrow 11$$

and:

$$01 \rightarrow 10$$

## 55. L2 — State Sequence

---

Given:

$$Q_1^{+} = Q_0$$

$$Q_0^{+} = Q_1'$$

Initial state 01.

We know cycle:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

Starting at 01:

$$\boxed{01 \rightarrow 11 \rightarrow 10 \rightarrow 00 \rightarrow 01}$$

## 56. L2 — Output Sequence

For above circuit:

$$Y = Q_1 \oplus Q_0$$

State cycle:

$$00, 01, 11, 10$$

Outputs:

$$0, 1, 0, 1$$

Therefore:

$$\boxed{0101\dots}$$

## 57. L2 — Reachability

Suppose:

$$00 \rightarrow 01$$

$$01 \rightarrow 00$$

$$10 \rightarrow 11$$

$$11 \rightarrow 10$$

If reset initializes 00, which states are reachable?

Only:

$$\boxed{00, 01}$$

States 10, 11 form a disconnected cycle.

## 58. L3 — SR Invalid State

Suppose:

$$S = Q$$

$$R = Q$$

If  $Q=1$ :

$\lceil S=R=1 \rceil$

standard SR FF receives invalid input.

Therefore circuit behavior is not properly defined for  $Q=1$ .

This may be the point of the question.

---

## 59. L3 — Multiple Cycles

Suppose circuit has state transitions:

$\lceil 000 \rightarrow 001 \rightarrow 000 \rceil$

and:

$\lceil 010 \rightarrow 011 \rightarrow 010 \rceil$

and all others eventually enter one of these cycles.

What is “the MOD”?

There is no single unique MOD without specifying which cycle/initial condition is relevant.

Do not blindly say MOD-2 because one loop has 2 states unless the intended operating cycle is identified.

---

## 60. L3 — Sequential Logic Trap

Given:

$\lceil D_1=Q_0 \rceil$

$\lceil D_0=Q_1 \rceil$

Initial 10.

A student updates Q1 first:

$\lceil Q_1=0 \rceil$

then uses this new Q1 to compute  $Q_0=0$ , obtaining 00.

Wrong.

Correct simultaneous update:

$\lceil \boxed{10} \rightarrow 01 \rceil$

Use old state throughout the edge.

---

## 61. Analysis With Input — Example

Suppose two D FFs:

$\lceil Q_1^+=Q_0 \rceil$

$\lceil Q_0^+=X \rceil$

This shifts external input X into a 2-bit register.

For state  $\lceil (Q_1Q_0=10) \rceil$ :

---

**X=0**

$\lceil Q_1^+=0, \lceil Q_0^+=0 \rceil$

next 00.

**X=1**

$\lceil Q_1^+=0, \lceil Q_0^+=1 \rceil$

next 01.

State transition depends on X.

---

## 62. State Table

| PS | X=0 | X=1 |
|----|-----|-----|
| 00 | 00  | 01  |
| 01 | 10  | 11  |
| 10 | 00  | 01  |
| 11 | 10  | 11  |

This is exactly what you should construct for input-driven machines.

---

## 63. Number of State Table Rows

For:

- n FFs !'  $\lceil 2^n \rceil$  states
- m independent binary inputs !'  $\lceil 2^m \rceil$  input combinations

Full state-input table contains:

$\lceil \boxed{2^{n+m}} \rceil$

rows if listing every state-input combination separately.

---

## 64. Example

3 flip-flops + 2 binary external inputs:

$\lceil 2^{3+2}=32 \rceil$

state/input combinations.

---

## 65. GATE-Fast Analysis Strategy

When circuit is small:

**D FF**

Immediately substitute:



$\lceil Q^+=D \rceil$

### **T FF**

Use hold/toggle.

### **JK FF**

Use 00/01/10/11 behavior.

### **SR FF**

Use hold/reset/set and check invalid.

Do not overuse algebra if behavioral analysis is faster.

---

## **66. When Algebra Is Better**

---

Use characteristic equations when:

- FF inputs are Boolean expressions
- several variables are involved
- you need minimized next-state equation
- answer options are Boolean expressions

Example:

$\lceil J=XQ' \rceil$

$\lceil K=X'+Q \rceil$

Algebra may be faster than repeatedly evaluating cases.

---

## **67. When State Table Is Better**

---

Use a state table when:

- only 2–4 FFs
- question asks sequence
- question asks MOD
- question asks reachable states
- you suspect multiple cycles

Enumerating states prevents mistakes.

---

## **68. Analysis Master Checklist**

---

For any given circuit:

1. What FF type?
  2. What are state variables?
  3. What are external inputs?
  4. What are FF input equations?
  5. What are next-state equations?
-

6. Any invalid FF input combinations?
  7. Build complete state table.
  8. Identify reachable states from initial/reset state.
  9. Draw state diagram if needed.
  10. Determine output equation/sequence.
  11. Identify counter/FSM/register behavior.
  12. Check for unused states, lockout, multiple cycles.
- 

## 69. Common GATE Question Types

---

Typical sequential-analysis questions ask you to:

- determine next state
  - derive state table
  - derive state diagram
  - identify MOD
  - determine output sequence
  - identify counter direction
  - determine reachable states
  - detect lockout
  - analyze D/JK/T FF networks
  - identify equivalent machine
  - detect invalid SR behavior
- 

## 70. Trap Book — Chapter 21

---

### Trap 1

Using new Q values immediately while calculating another FF in the same clock edge.

Wrong.

Use old Q values for all next-state calculations.

### Trap 2

Assuming every n-FF circuit is MOD- $(2^n)$ .

Wrong.

Inspect actual state cycle.

### Trap 3

Ignoring initial state.

Different initial states may enter different cycles.

### Trap 4

Ignoring unreachable states.

All binary codes are possible patterns, but not necessarily reachable.

### Trap 5

Using FF excitation table during analysis.

Usually analysis uses **characteristic behavior/equation**.

Excitation is mainly for design.

### Trap 6

For D FF, making analysis complicated.

Just use:

$$\boxed{Q^{+}=D}$$

### Trap 7

For T FF, forgetting T=1 means toggle.

### Trap 8

For JK 11, treating it as invalid.

It toggles.

### Trap 9

For SR, forgetting invalid combination.

### Trap 10

Confusing state value with numeric count.

A state code is just memory unless counter interpretation is intended.

---

## 71. Formula Sheet

---

### D

$$\boxed{Q^{+}=D}$$

### T

$$\boxed{Q^{+}=T\oplus Q}$$

### JK

$$\boxed{Q^{+}=JQ'+K'Q}$$

### SR

$$\boxed{Q^{+}=S+R'Q}$$

State combinations with n FFs:

$$\boxed{2^n}$$

Full state/input combinations with n FFs and m binary inputs:

$$\boxed{2^{n+m}}$$

MOD:

$\boxed{\text{number of states in repeating operational cycle}}$

---

## 72. 30-Second Revision

Sequential circuit analysis:

$\boxed{\text{Circuit} \rightarrow \text{FF input equations} \rightarrow Q^{+} \rightarrow \text{state table} \rightarrow \text{state diagram/sequence}}$

Remember:

- D !' next state equals D
- T !' hold/toggle
- JK !' hold/reset/set/toggle
- SR !' hold/reset/set/invalid
- all FFs in synchronous circuit update from **old state simultaneously**
- initial state determines reachable sequence
- MOD comes from actual repeating states

---

## What You Absolutely Must Remember

1. Analysis means derive behavior from an existing circuit.
2. Write FF input equations first.
3. Use characteristic equations, not excitation tables, for analysis.
4. Present state and next state must remain separate.
5. D FF analysis is direct:  $(Q^{+}=D)$ .
6. T FF analysis is hold/toggle.
7. JK can be solved by either characteristic equation or behavior table.
8. SR invalid inputs must be checked.
9. Build state table systematically.
10. Initial/reset state determines reachable behavior.
11. A circuit can have multiple disjoint cycles.
12. MOD is based on the operational repeating state cycle.
13. Synchronous FFs update simultaneously using old Q values.
14. Output may depend on state only or state + input.

---

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' Registers ' !' Counters ' !' **Sequential Circuit Analysis** '

**Current: Chapter 22 — Sequential Circuit Design**

**Remaining:** Sequential Design !' FSM Moore/Mealy !' Sequence Detectors !' Fixed Point !' Floating Point !' IEEE-754

## NEXT — Chapter 22: SEQUENTIAL CIRCUIT DESIGN Ø=Ý4 MUST MASTER

Next we reverse the process:

**specification !' state sequence/table !' choose FF type !' excitation table !' derive FF input functions !' K-map minimization !' implement synchronous counters/custom state machines !' unused-state handling !' self-starting design !' GATE design problems.**

USER

next

ASSISTANT

## CHAPTER 22 — SEQUENTIAL CIRCUIT DESIGN Ø=Ý4 MUST MASTER

Now we reverse Chapter 21.

Previously:

$\boxed{\text{Circuit}} \rightarrow \text{Behavior}$

Now:

$\boxed{\text{Required Behavior}} \rightarrow \text{Circuit}$

This is one of the most important GATE skills because it combines:

- state tables
- excitation tables
- K-maps
- flip-flops
- counters
- unused states
- next-state design

### 1. What Is Sequential Circuit Design?

Suppose we want a machine with sequence:

$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$

We must find:

**What logic should be connected to the flip-flops so this exact sequence occurs?**

That is sequential circuit design.

### 2. Master Design Procedure Ø=Ý4

Memorize this pipeline:

$\boxed{\text{Specification}} \rightarrow \text{State Diagram} \rightarrow \text{State Table} \rightarrow \text{State Assignment} \rightarrow \text{Choose FF} \rightarrow \text{Excitation Table} \rightarrow \text{K-map} \rightarrow \text{FF Input Equations} \rightarrow \text{Circuit}$

For counters whose binary states are already given, state assignment may already be done.

### 3. Analysis vs Design

#### Analysis

Given FF inputs.

Use characteristic equation:

$$\boxed{\text{Inputs}} \rightarrow Q^{+}$$

#### Design

Given desired:

$$Q \rightarrow Q^{+}$$

Use excitation table:

$$\boxed{Q, Q^{+}} \rightarrow \text{required FF inputs}$$

Memory:

**Analysis !' characteristic**

**Design !' excitation**

Very important.

### 4. Excitation Tables Recap Ø=Ý4

#### D Flip-Flop

$$\boxed{D=Q^{+}}$$

| Q | $Q^{+}$ | D |
|---|---------|---|
| 0 | 0       | 0 |
| 0 | 1       | 1 |
| 1 | 0       | 0 |
| 1 | 1       | 1 |

## T Flip-Flop

$$T = Q \oplus Q^+$$

| Q | $Q^+$ | T |
|---|-------|---|
| 0 | 0     | 0 |
| 0 | 1     | 1 |
| 1 | 0     | 1 |
| 1 | 1     | 0 |

Memory:

state changes !' T=1

state stays !' T=0

## JK Flip-Flop

| Q | $Q^+$ | J | K |
|---|-------|---|---|
| 0 | 0     | 0 | X |
| 0 | 1     | 1 | X |
| 1 | 0     | X | 1 |
| 1 | 1     | X | 0 |

## SR Flip-Flop

| Q | $Q^+$ | S | R |
|---|-------|---|---|
| 0 | 0     | 0 | X |
| 0 | 1     | 1 | 0 |
| 1 | 0     | 0 | 1 |
| 1 | 1     | X | 0 |

## 5. Why D FF Is Easiest

For D:

$$Q^+ = D$$

So once desired next-state function is known:

$$\boxed{D=Q^+}$$

No further excitation conversion needed.

That is why D flip-flops are extremely convenient for synchronous design.

## PART A — DESIGN A 2-BIT SYNCHRONOUS UP COUNTER USING T FFs

### 6. Desired Sequence

We want:

$$00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$$

Let:

$$Q_1 Q_0$$

be the present state.

### 7. State Table

| Present $(Q_1 Q_0)$ | Next $(Q_1^+ Q_0^+)$ |
|---------------------|----------------------|
| 00                  | 01                   |
| 01                  | 10                   |
| 10                  | 11                   |
| 11                  | 00                   |

Now derive T inputs separately.

### 8. Find $(T_0)$

Look at  $Q_0$  transitions:

| PS | $Q_0$ | $(Q_0^+)$ | $(T_0)$ |
|----|-------|-----------|---------|
| 00 | 0     | 1         | 1       |
| 01 | 1     | 0         | 1       |
| 10 | 0     | 1         | 1       |
| 11 | 1     | 0         | 1       |

So:

$$\boxed{T_0=1}$$

LSB toggles every clock.



---

## 9. Find $T_1$

| PS | Q1 | $Q_1^{+1}$ | $T_1$ |
|----|----|------------|-------|
| 00 | 0  | 0          | 0     |
| 01 | 0  | 1          | 1     |
| 10 | 1  | 1          | 0     |
| 11 | 1  | 0          | 1     |

Pattern:

$$T_1 = 1$$

when:

$$Q_0 = 1$$

Therefore:

$$T_1 = Q_0$$

Final design:

$$T_0 = 1, T_1 = Q_0$$

Exactly the rule we learned for synchronous up counters.

---

## 10. Same Counter Using JK FFs

Because a JK FF toggles when:

$$J = K = 1$$

we can use the T conditions:

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = Q_0$$

This is a fast shortcut.

---

## 11. Same Counter Using D FFs

State table:

$$00 \rightarrow 01$$

$$01 \rightarrow 10$$

$$10 \rightarrow 11$$

$$11 \rightarrow 00$$

Since:

$$D_i = Q_i^{+1}$$

derive directly.

For Q0:

$$Q_0 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 0 \rightarrow \dots$$

so:

$$D_0 = Q_0$$

For Q1:

Q1 toggles whenever Q0=1.

Thus:

$$D_1 = Q_1 \oplus Q_0$$

Final:

$$D_0 = Q_0$$

$$D_1 = Q_1 \oplus Q_0$$

## PART B — CUSTOM COUNTER DESIGN

### 12. Example Sequence

Design a synchronous counter:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00 \rightarrow \dots$$

using D FFs.

This is a 4-state custom sequence.

### 13. State Table

| $Q_1 Q_0$ | $Q_1^+ Q_0^+$ |
|-----------|---------------|
| 00        | 01            |
| 01        | 11            |
| 11        | 10            |
| 10        | 00            |

For D FF:

$$D_1 = Q_1^+$$

$$D_0 = Q_0^+$$

### 14. Find $D_1$

From table:

| Q1 | Q0 | $\text{D}_1$ |
|----|----|--------------|
| 0  | 0  | 0            |
| 0  | 1  | 1            |
| 1  | 0  | 0            |
| 1  | 1  | 1            |

Clearly:

$$\boxed{\text{D}_1 = \text{Q}_0}$$

## 15. Find $\text{D}_0$

| Q1 | Q0 | $\text{D}_0$ |
|----|----|--------------|
| 0  | 0  | 1            |
| 0  | 1  | 1            |
| 1  | 0  | 0            |
| 1  | 1  | 0            |

Thus:

$$\boxed{\text{D}_0 = \text{Q}_1'}$$

So the complete circuit:

$$\boxed{\text{D}_1 = \text{Q}_0}$$

$$\boxed{\text{D}_0 = \text{Q}_1'}$$

produces:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

## 16. Verify the Design

Always verify.

For:

$$\text{Q}_1\text{Q}_0 = 10$$

$$\text{D}_1 = \text{Q}_0 = 0$$

$$\text{D}_0 = \text{Q}_1' = 0$$

So next:

$$00$$

Correct.

Verification is especially useful after K-map simplification.

---

## PART C — DESIGN USING JK FFs

---

### 17. Example

Design:

$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$

using JK flip-flops.

Need:

- $J_1, K_1$
- $J_0, K_0$

Use excitation table for each state transition.

---

### 18. Build Excitation Table

| PS | NS | $J_1$ | $K_1$ | $J_0$ | $K_0$ |
|----|----|-------|-------|-------|-------|
| 00 | 01 | 0     | X     | 1     | X     |
| 01 | 11 | 1     | X     | X     | 0     |
| 10 | 00 | X     | 1     | 0     | X     |
| 11 | 10 | X     | 0     | X     | 1     |

Now minimize each column.

---

### 19. Derive $J_1$

Required values:

- state 00 !' 0
- state 01 !' 1
- $Q_1=1$  rows !' X

Using don't-cares, simplest:

$J_1 = Q_0$

---

### 20. Derive $K_1$

Requirements:

- 10 !' 1
- 11 !' 0
- $Q_1=0$  rows are X

Thus:

$$\boxed{K_1=Q_0}$$


---

## 21. Derive $(J_0)$

---

Requirements:

- $00 \rightarrow 1$
- $10 \rightarrow 0$
- $Q_0=1$  rows X

Simplest:

$$\boxed{J_0=Q_1}$$


---

## 22. Derive $(K_0)$

---

Requirements:

- $01 \rightarrow 0$
- $11 \rightarrow 1$
- $Q_0=0$  rows X

Therefore:

$$\boxed{K_0=Q_1}$$

Final:

$$\boxed{J_1=Q_0, \text{quad } K_1=Q_0}$$

$$\boxed{J_0=Q_1, \text{quad } K_0=Q_1}$$


---

## 23. Why Don't-Cares Help

---

JK excitation table contains many X entries.

These Xs can be used in K-map minimization.

That often produces much simpler input logic than treating every X as 0.

Important:

$$\boxed{\text{Excitation-table X is a genuine don't-care}}$$

because either value will produce the desired transition.

---

## PART D — DESIGN USING T FFs

---

### 24. Same Custom Sequence

---

Sequence:

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$$

Use:

---

$$T = Q \oplus Q^+$$

## 25. T Excitation Table

| PS | NS | $T_1$ | $T_0$ |
|----|----|-------|-------|
| 00 | 01 | 0     | 1     |
| 01 | 11 | 1     | 0     |
| 10 | 00 | 1     | 0     |
| 11 | 10 | 0     | 1     |

Observe:

$$T_1 = Q_1 \oplus Q_1^+$$

From table:

$$T_1 = 1$$

for states:

$$\{01, 10\}$$

This is:

$$T_1 = Q_1 \oplus Q_0$$

For  $T_0$ :

1 for 00 and 11.

That's XNOR:

$$T_0 = Q_1 \odot Q_0$$

## 26. Why T FF Can Be Convenient

If desired design naturally describes which bits toggle, T FF often produces simple equations.

Counters are a great example.

D is simplest for arbitrary next-state equations.

T/JK can be simpler for toggle-heavy counters.

## PART E — MOD-N COUNTER DESIGN Ø=4

### 27. Example — MOD-6 Counter

Desired sequence:

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 000$$

Need:

$\lceil 3 \text{ FFs} \rceil$

because:

$\lceil 2^2 < 6 \leq 2^3 \rceil$

Unused states:

$\lceil 110, 111 \rceil$

---

## 28. Two Choices for Unused States

---

We can:

### Option A

Treat them as don't-cares during minimization.

This may simplify logic but can cause unpredictable lockout behavior.

### Option B

Explicitly assign them a next state, often:

$\lceil 000 \rceil$

This can make the counter self-starting.

Both can be valid depending on question.

---

## 29. Self-Starting MOD-6 Design

---

For robust behavior, define:

$\lceil 110 \rightarrow 000 \rceil$

$\lceil 111 \rightarrow 000 \rceil$

Then every possible state eventually enters valid sequence.

This is:

$\lceil \boxed{\text{self-starting}} \rceil$

---

## 30. Why Unused States Matter

---

Suppose you simply mark:

$\lceil 110, 111 = X \rceil$

during K-map minimization.

The final circuit may accidentally produce:

$\lceil 110 \rightarrow 111 \rceil$

$\lceil 111 \rightarrow 110 \rceil$

Then if noise/reset problem puts circuit there, it stays trapped.

This is:

---

$\boxed{\text{lockout}}$

---

## 31. GATE Rule

---

If a problem asks only for minimal logic and says unused states are don't-care:

use them as X.

If it asks for a self-starting counter:

you must analyze and control unused-state transitions.

---

## PART F — DESIGNING A SYNCHRONOUS DOWN COUNTER

---

### 32. 3-Bit Down Sequence

---

$111 \rightarrow 110 \rightarrow 101 \rightarrow 100 \rightarrow 011 \rightarrow 010 \rightarrow 001 \rightarrow 000 \rightarrow 111$

Using T FFs:

$T_0 = 1$

$T_1 = Q_0'$

$T_2 = Q_1'Q_0'$

This can be obtained from excitation table, but knowing pattern saves time.

---

### 33. Why Lower Zeros Matter

---

During decrement:

Example:

$1000 - 1 = 0111$

Borrow passes through lower zeros.

Thus a higher bit toggles when all lower bits are 0.

Compare:

#### Up

carry passes through lower 1s.

#### Down

borrow passes through lower 0s.

---

## PART G — UP/DOWN COUNTER DESIGN

---



## 34. 2-Bit Synchronous Up/Down Counter

---

Let control U:

$$U=1 \rightarrow \text{UP}$$

$$U=0 \rightarrow \text{DOWN}$$

LSB always toggles:

$$T_0 = 1$$

For  $(Q_1)$ :

Up ! toggle when  $(Q_0=1)$

Down ! toggle when  $(Q_0=0)$

Therefore:

$$T_1 = Q_0 \oplus U'Q_0'$$

So:

$$T_1 = U \odot Q_0$$

---

## 35. General Up/Down Counter

---

For higher bit  $(i)$ :

$$T_i = U(Q_{i-1} \odot Q_0) + U'(Q_{i-1} \odot Q_0')$$

for the convention:

$$U=1 \rightarrow \text{UP}$$

---

## PART H — STATE ASSIGNMENT

---

### 36. Symbolic States

---

Suppose specification uses:

$$A, B, C$$

These are symbolic states.

Need enough FFs:

$$\lceil \log_2 3 \rceil = 2$$

Possible assignment:

$$A=00$$

$$B=01$$

$$C=10$$

State 11 unused.

---

## 37. State Assignment Can Affect Logic

---

Different binary assignments can produce different Boolean equations.

For example:

Assignment 1:

$\{ A=00, B=01, C=10 \}$

may produce more complex gates than:

$\{ A=00, B=11, C=10 \}$

There is no universally best simple assignment.

In GATE questions, assignment is often provided or small enough to evaluate directly.

---

## 38. Number of FFs for M States

---

Binary encoding:

$\lceil n = \lceil \log_2 M \rceil \rceil$

Example:

7 states:

$\lceil 3 \text{ FFs} \rceil$

13 states:

$\lceil 4 \text{ FFs} \rceil$

---

## 39. One-Hot Encoding ~~Ø=64~~ SUPPORT

---

Another encoding uses one flip-flop per state.

For 4 states:

$\{ 0001, 0010, 0100, 1000 \}$

This uses:

$4 \text{ FFs}$

instead of 2.

Not usually minimal in FF count, but logic can be simpler.

For core GATE CS, binary state encoding is usually the more important default.

---

## PART I — OUTPUT DESIGN

---

## 40. Sequential Machines Can Have Outputs Too

---

A design may specify both:

- next state

- output

Output equation is derived separately.

Suppose output  $Y=1$  in states:

$$\{01, 11\}$$

Then:

$$Y = Q_0$$

If output is based only on state, this is Moore-type behavior.

If output also depends on  $X$ , it's Mealy-type.

Formal treatment comes next chapter.

## 41. Example With Input

Suppose 1-bit state  $Q$  and input  $X$ .

Requirement:

- if  $X=0$  !' hold
- if  $X=1$  !' toggle

Using D FF.

State equation:

$$Q^+ = Q \oplus X$$

So:

$$\boxed{D = Q \oplus X}$$

That implements the machine directly.

## 42. Same Design Using T FF

Requirement is literally:

- $X=0$  !' hold
- $X=1$  !' toggle

Therefore:

$$\boxed{T = X}$$

Much simpler.

This illustrates why FF choice can affect hardware complexity.

## PART J — DESIGN FROM A STATE DIAGRAM

### 43. Given Diagram

Suppose:

```text id="i0eq58" 00 !' 01 !' 10 !' !" % % % % % % % % % % %

Sequence:

```
\[
00\rightarrow01\rightarrow10\rightarrow00
\]
```

State 11 unused.

This is a MOD-3 counter.

---

# 44. Using D FF

Table:

|                      |
|----------------------|
| PS   NS              |
| ---  ---             |
| 00   01              |
| 01   10              |
| 10   00              |
| 11   X/X or assigned |

If unused 11 is don't-care:

```
\[
D_1=Q_1^+
\]
```

```
\[
D_0=Q_0^+
\]
```

Then K-map with 11 as X.

---

# 45. Derive MOD-3 D Inputs

For valid states:

|                        |
|------------------------|
| Q1   Q0   D1   D0      |
| ---:  ---:  ---:  ---: |
| 0   0   0   1          |
| 0   1   1   0          |
| 1   0   0   0          |
| 1   1   X   X          |

For  $\backslash(D_1\backslash)$ :

Only 01 requires 1.

Using 11 as X:

can group 01 with 11:

```
\[
\boxed{D_1=Q_0}
\]
```

For  $\backslash(D_0\backslash)$ :

Only 00 requires 1.

11 is X but not adjacent enough to simplify to one literal in this 2-var map.

So:

```
\[
\boxed{D_0=Q_1'Q_0'}
\]
```

One minimal implementation.

---

# 46. Check Unused State 11

With:

```
\[
D_1=Q_0
\]
```

```
\[
D_0=Q_1'Q_0'
\]
```

At:

```
\[
Q_1Q_0=11
\]
```

```
\[
D_1=1
\]
```

```
\[
D_0=0
\]
```

So:

```
\[
11\rightarrow 10
\]
```

Then:

```
\[
10\rightarrow 00
\]
```

Thus the counter is actually self-correcting from 11.

Nice result.

---

# 47. Important Insight

Even if unused states were treated as don't-care in K-map, after minimization you should sometimes check where they actually go.

This can reveal whether the minimized circuit is:

- self-starting
- locked out

---

## # PART K – DESIGN USING JK EXCITATION

### # 48. Generic Procedure

Suppose one state bit:

\[  
Q<sub>i</sub>  
\]

has desired transitions.

For every row:

- 0! ' 0 !' \ ( J = 0 , K = X \ )  
- 0! ' 1 !' \ ( J = 1 , K = X \ )  
- 1! ' 0 !' \ ( J = X , K = 1 \ )  
- 1! ' 1 !' \ ( J = X , K = 0 \ )

Do this independently for every state bit.

Then minimize:

\[  
J<sub>i</sub>  
\]

and:

\[  
K<sub>i</sub>  
\]

separately.

---

### # 49. Generic T Procedure

For each bit:

\[  
 $\boxed{T_i = Q_i \oplus Q_i^{+}}$   
\]

So instead of excitation lookup, simply check whether that bit changes.

If yes:

\[  
T<sub>i</sub>=1  
\]

If no:

\[  
T<sub>i</sub>=0  
\]

---

### # 50. Generic D Procedure

For every state bit:

```
\[
\boxed{D_i=Q_i^{+}}
\]
```

Just copy the next-state column into D input's truth table.

This is the fastest design procedure.

---

# PART L – DESIGN COST

# 51. What Does Minimum Design Mean?

Could mean:

- minimum literals
- minimum gates
- minimum FF input logic
- minimum gate levels
- minimum flip-flops

These are not necessarily the same.

Always read question carefully.

---

# 52. Choosing FF Type

There is no universal “best FF.”

But typical intuition:

### D FF

Best for arbitrary next-state functions.

### T FF

Excellent for counters/toggle behavior.

### JK FF

Flexible; many don't-cares in excitation table.

### SR FF

Can be efficient but forbidden state must be avoided.

---

# PART M – SYNCHRONOUS VS ASYNCHRONOUS DESIGN

# 53. Synchronous State Design

All FFs:

```
\[
\boxed{\text{share same clock}}
\]
```

Combinational logic generates next-state inputs.

This chapter mainly deals with synchronous design.

---

#### # 54. Asynchronous Reset Is Different

You may design state logic synchronously but include an asynchronous reset.

Example:

```
\[
RESET\rightarrow Q=000
\]
```

This does not make the entire counter an asynchronous ripple counter.

Be careful with terminology.

---

#### # 55. "Synchronous Counter" Meaning

Means state FFs receive the same clock.

It may still have:

- combinational logic
- asynchronous reset pin

The presence of async reset does not automatically make counting itself asynchronous.

---

#### # PART N – GATE PRACTICE

#### # 56. L0

Which table is primarily used in sequential circuit design?

- A. Characteristic table only
- B. Excitation table
- C. Decoder table
- D. Encoder table

Answer:

```
\[
\boxed{B}
\]
```

---

#### # 57. L0 – D Design

Desired:

```
\[
Q^{+}=XQ'+X'Q
\]
```

Using D FF:

```
\[
\boxed{D=XQ'+X'Q}
\]
```



Recognize:

$$D = X \oplus Q$$

---

# 58. L0 – T Design

Desired:

$$Q^{+} = Q'$$

every clock.

Need:

$$\boxed{T=1}$$

---

# 59. L1 – JK Excitation

Transition:

$$\emptyset \rightarrow 1$$

requires:

$$\boxed{J=1, \quad K=X}$$

---

# 60. L1 – Counter FF Count

Design MOD-13 synchronous counter.

Need:

$$2^3 < 13 \leq 2^4$$

So:

$$\boxed{4 \text{ FFs}}$$

---

# 61. L1 – Unused States

MOD-13 with 4 FFs:

$$16 - 13 = 3$$

$$\boxed{3 \text{ unused states}}$$

---

#### # 62. L2 – T Counter Design

3-bit synchronous up counter:

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_1 Q_0$$

---

#### # 63. L2 – Down Counter

3-bit synchronous down:

$$T_0 = 1$$

$$T_1 = Q_0'$$

$$T_2 = Q_1' Q_0'$$

---

#### # 64. L2 – Self-Starting

A MOD-5 counter uses 3 FFs.

Unused states:

$$101, 110, 111$$

If all eventually transition into valid cycle:

$$000 \rightarrow 100$$

then:

$$\text{self-starting}$$

---

## # 65. L2 – State Design With D

Desired:

\[  
00\rightarrow10  
\]

\[  
10\rightarrow11  
\]

\[  
11\rightarrow01  
\]

\[  
01\rightarrow00  
\]

Find  $\{(D_1, D_0)\}$ .

Table:

| PS | NS | D1 | D0 |
|----|----|----|----|
| 00 | 10 | 1  | 0  |
| 01 | 00 | 0  | 0  |
| 10 | 11 | 1  | 1  |
| 11 | 01 | 0  | 1  |

Observe:

\[  
 $D_1 = Q_0$   
\]

\[  
 $D_0 = Q_1$   
\]

Thus:

\[  
 $\boxed{D_1 = Q_0, \quad D_0 = Q_1}$   
\]

---

## # 66. L3 – Unused State Trap

You design MOD-6 using 3 FFs and treat 110,111 as X.

Can you automatically claim it is self-starting?

\[  
 $\boxed{\text{No}}$   
\]

You must evaluate the final logic at those unused states.

---

## # 67. L3 – Excitation Don't-Care

JK transition:

$$\begin{aligned} & \backslash[ \\ & Q=0 \rightarrow Q^+=0 \\ & \backslash] \end{aligned}$$

Excitation:

$$\begin{aligned} & \backslash[ \\ & J=0, \backslash K=X \\ & \backslash] \end{aligned}$$

The X can be chosen 0 or 1 to simplify K-map.

Do not force it to 0.

---

# 68. L3 – Wrong Table Trap

Suppose designing with T FFs.

Student uses:

$$\begin{aligned} & \backslash[ \\ & Q^+=T \oplus Q \\ & \backslash] \end{aligned}$$

to fill desired next state from T.

But T is what we're trying to find.

Better design relation:

$$\begin{aligned} & \backslash[ \\ & \boxed{T=Q \oplus Q^+} \\ & \backslash] \end{aligned}$$

Use present and desired next states.

---

# 69. L3 – State-Assignment Trap

Machine has 5 symbolic states.

Minimum binary FFs:

$$\begin{aligned} & \backslash[ \\ & \lceil \log_2 5 \rceil = 3 \\ & \backslash] \end{aligned}$$

Not 5, unless one-hot encoding is explicitly used.

---

# 70. GATE-Fast Design Strategy

If you can choose FF type and problem is simple:

### Counter

Try T/JK first.

### Arbitrary FSM

D is usually easiest.

### Asked specifically for JK/SR/T

Use excitation table and exploit don't-cares.

---

# 71. GATE PYQ Patterns

Typical design questions include:

- derive FF input equations from state table
- choose excitation values
- design synchronous up/down counters
- MOD-N counter design
- identify unused states
- determine self-starting behavior
- derive D/JK/T input functions
- minimize FF input logic using K-map
- complete partially specified next-state table
- compare implementations using different FFs

These are PYQ-style patterns, not specific-year claims.

---

# 72. Trap Book – Chapter 22

### Trap 1

Using characteristic table instead of excitation table for design.

Wrong direction.

### Trap 2

For D FF, deriving extra excitation logic unnecessarily.

Just:

$\boxed{D=Q^+}$

### Trap 3

For T FF, forgetting:

$\boxed{T=Q \oplus Q^+}$

### Trap 4

Replacing excitation-table X with 0 immediately.

You lose simplification opportunities.

### Trap 5

Ignoring invalid SR combination.

Never allow  $S=R=1$  in standard SR design.

### Trap 6

Assuming all  $2^n$  states are used.

MOD-N may use only N states.

```

### Trap 7
Ignoring unused states.

They matter for self-starting/lockout.

### Trap 8
Assuming unused state = don't-care automatically.

Only when problem permits.

### Trap 9
Updating states sequentially within one synchronous edge.

All state FFs use old state simultaneously.

### Trap 10
Confusing symbolic state count with flip-flop count.

Use:

\[
\lceil \log_2 M \rceil
\]

for binary encoding.

---

# 73. Sequential Design Formula Sheet

Minimum FFs:

\[
\boxed{n = \lceil \log_2 M \rceil}
\]

D excitation:

\[
\boxed{D = Q^+}
\]

T excitation:

\[
\boxed{T = Q \oplus Q^+}
\]

JK:

\[
\begin{aligned}
0 \rightarrow 0: & \quad J=0, K=X \\
0 \rightarrow 1: & \quad J=1, K=X \\
1 \rightarrow 0: & \quad J=X, K=1 \\
1 \rightarrow 1: & \quad J=X, K=0
\end{aligned}
\]

```

SR:

\[  
0\to0:\ S=0,R=X  
\]

\[  
0\to1:\ S=1,R=0  
\]

\[  
1\to0:\ S=0,R=1  
\]

\[  
1\to1:\ S=X,R=0  
\]

---

# 74. Design Master Workflow

Memorize:

\[  
\boxed{  
\text{Desired behavior}  
\rightarrow  
PS/NS  
\rightarrow  
\text{Excitation}  
\rightarrow  
\text{K-map}  
\rightarrow  
\text{Input equations}  
\rightarrow  
\text{Circuit}  
}  
\]

And after designing:

\[  
\boxed{\text{VERIFY THE STATE SEQUENCE}}  
\]

---

# 75. 30-Second Revision

Sequential design is the reverse of analysis.

Use:

- specification
- state table
- desired next state
- excitation table
- K-map
- FF input equations

Fastest formulas:

\[  
\boxed{D=Q^+}  
\]

```
\[
\boxed{T=Q\oplus Q^+}
\]
```

For counters:

Up:

```
\[
\boxed{T_i=\text{AND of all lower } Q \text{ bits}}
\]
```

Down:

```
\[
\boxed{T_i=\text{AND of all lower } Q' \text{ bits}}
\]
```

Unused states:

> either exploit as don't-cares or deliberately redirect them for self-starting behavior.

---

# What You Absolutely Must Remember

1. Design goes from required behavior to hardware.
2. Analysis uses characteristic behavior; design uses excitation behavior.
3. D FF is easiest because  $(D=Q^+)$ .
4. T FF uses  $(T=Q\oplus Q^+)$ .
5. JK excitation table has useful don't-cares.
6. State table must list present and next states correctly.
7. Minimize every FF input separately.
8. Up-counter higher bit toggles when all lower bits are 1.
9. Down-counter higher bit toggles when all lower bits are 0.
10. MOD-N needs  $\lceil \log_2 N \rceil$  FFs for binary encoding.
11. Unused states may cause lockout.
12. Self-starting means unused states eventually enter valid cycle.
13. Never claim self-starting without checking final equations.
14. State assignment can affect hardware complexity.
15. Always verify the designed sequence after simplification.

## Digital Logic Progress

```
* * Completed : * *   Number Systems ' !' Signed Representation
' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization
Combinational Circuits ' !' Adders/Subtractors ' !' MUX '
Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF C
Registers ' !' Counters ' !' Sequential Analysis ' !' * * Se
```

\*\*Current:\*\* \*\*Chapter 23 – Finite State Machines: Moore & Mealy\*\*

```
* * Remaining : * *   FSM Moore/Mealy !' Sequence Detectors !' F
Fundamentals !' IEEE - 754
```

### NEXT – Chapter 23: FINITE STATE MACHINES Ø=Ý4 MUST MA

Next we cover:

```
* * what an FSM is !' state diagrams/tables !' Moore machin
Moore vs Mealy differences !' state reduction/equivalenc
conversion between Moore and Mealy !' GATE traps. * *
```



next

## CHAPTER 23 — FINITE STATEMACHINES: MOORE & MEALY 0=14 MUST MASTER

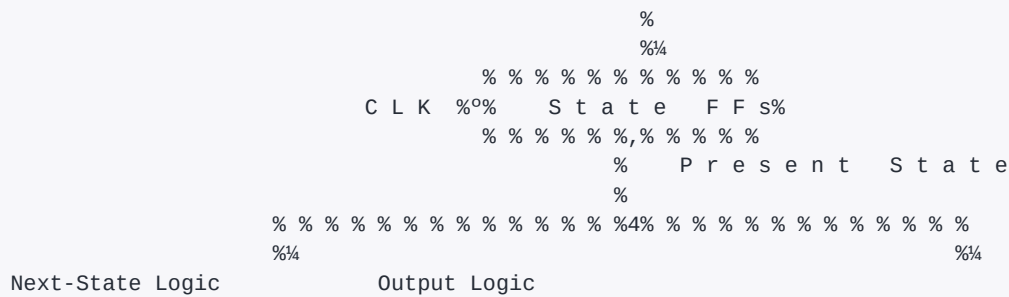
- a finite number of states
- input-dependent state transitions
- output rules

- sequence detectors
- controllers
- communication protocols
- traffic-light logic
- CPU control units
- digital controllers

$$\boxed{\text{Mealy Machine}}$$

That difference separates Moore and Mealy machines.

[illegible]



State FFs store the machine's memory.

### 3. State

A state summarizes relevant information about past inputs.

Example:

For detecting sequence:

[ 101 ]

the machine may need to remember:

- nothing useful seen yet
- saw 1
- saw 10

These become states.

FSMs do **not** normally store the entire input history.

They store only the information necessary to determine future behavior.

### 4. State Diagram

A state diagram uses circles for states and arrows for transitions.

Example:

```

TEXT
1
% % % % % % % % S 1
% % % % % % % % S 1
% 0 % 0
% % % % % % S 2
S 0 % % % % % % S 2

```

Transition arrows show how input changes the state.

The exact labels depend on Moore vs Mealy.

### 5. State Table

An FSM can also be represented using a table.

Example:

| Present State | X=0 Next | X=1 Next |
|---------------|----------|----------|
| S0            | S0       | S1       |
| S1            | S2       | S1       |
| S2            | S0       | S1       |

Output columns may also be included.

## PART A — MOORE MACHINE

### 6. Moore Machine $Q=Y4$

In a Moore machine:

$\boxed{Y=g(Q)}$

Output depends **only on present state**.

The input affects output indirectly by changing the state.

### 7. Moore Output Rule

Suppose:

$S_0 \rightarrow Y=0$

$S_1 \rightarrow Y=0$

$S_2 \rightarrow Y=1$

Whenever machine is in  $(S_2)$ :

$Y=1$

regardless of current input.

### 8. Moore State Diagram Labels

A Moore state is often written:

$\boxed{\text{State/Output}}$

Example:

TEXT

S0/0

S1/0

S2/1

Output is written **inside/on the state**, not on the transition.

## 9. Moore Example

Suppose:

- S0 output = 0
- S1 output = 1

Transitions:

- S0, X=0 !' S0
- S0, X=1 !' S1
- S1, X=0 !' S0
- S1, X=1 !' S1

Diagram conceptually:

TEXT

```
X=1
S 0 / 0 % % % % %° S 1 / 1
%² %
% X = 0 % X = 1
% % % % % % % % % % %
```

Output depends only on whether current state is S0 or S1.

## 10. Moore Output Timing

Since output depends on state FF outputs, output normally changes after a state change.

Thus output usually changes after:

$\boxed{\text{active clock edge} + \text{clock-to-Q delay}}$

ignoring output logic delay.

This makes Moore outputs relatively stable between clock edges.

## PART B — MEALY MACHINE

### 11. Mealy Machine $\emptyset = \mathbb{Y}_4$

In a Mealy machine:

$\boxed{Y = g(Q, X)}$

Output depends on:

- present state
- current input

Therefore output can potentially change when input changes, even without a state transition.

### 12. Mealy Diagram Labels

Mealy transition arrows are usually labeled:

$$\boxed{\text{Input/Output}}$$

Example:

$$\boxed{1/0}$$

means:

- input = 1
- output = 0

## 13. Mealy Example

State S0:

- X=0 !' remain S0, output 0
- X=1 !' go S1, output 1

State S1:

- X=0 !' S0, output 0
- X=1 !' S1, output 1

Transition labels:

TEXT

S0 --1/1--> S1  
S0 --0/0--> S0

Output is associated with transitions.

## 14. Moore vs Mealy — Core Difference

### Moore

$$\boxed{Y=f(\text{state})}$$

### Mealy

$$\boxed{Y=f(\text{state}, \text{input})}$$

Memory:

**Moore = output on state**

**Mealy = output on transition**

## 15. Master Comparison Table Ø=Ý4

| Feature           | Moore | Mealy         |
|-------------------|-------|---------------|
| Output depends on | State | State + input |

| Feature                                | Moore              | Mealy                 |
|----------------------------------------|--------------------|-----------------------|
| Output shown on                        | State              | Transition            |
| Typical label                          | S/Y                | X/Y                   |
| Output can react immediately to input? | Usually no         | Yes                   |
| Number of states                       | Often more         | Often fewer           |
| Output stability                       | Generally better   | More input-sensitive  |
| Output delay                           | after state change | can change with input |

## 16. Why Mealy Often Needs Fewer States

Suppose output should become 1 immediately when a certain input completes a sequence.

A Mealy machine can put output 1 directly on the transition that completes the sequence.

Moore may need an additional state whose output is 1.

Therefore:

$\boxed{\text{Mealy usually uses equal or fewer states than equivalent Moore}}$

Not a strict law for every representation, but a very useful general rule.

## 17. Output Response Difference

Suppose present state remains fixed and X changes:

$0 \rightarrow 1$

### Moore

Output remains the same because state didn't change.

### Mealy

Output may change immediately because:

$Y = f(Q, X)$

This is important in timing questions.

## 18. Glitch Sensitivity

Because Mealy output depends directly on input:

input glitches may propagate to output.

Moore output is based on registered state, so is generally more stable.

Thus:

$\boxed{\text{Mealy can be more glitch-sensitive}}$

---

## 19. Moore State Table Format

Example:

| Present State | X=0 Next | X=1 Next | Output |
|---------------|----------|----------|--------|
| S0            | S0       | S1       | 0      |
| S1            | S2       | S1       | 0      |
| S2            | S0       | S1       | 1      |

Notice:

one output per state.

---

## 20. Mealy State Table Format

| Present State | X=0  | X=1  |
|---------------|------|------|
| S0            | S0/0 | S1/0 |
| S1            | S2/0 | S1/0 |
| S2            | S0/0 | S1/1 |

Each transition carries:

$\boxed{\text{Next State / Output}}$

---

## PART C — STATE ENCODING

---

### 21. Binary Encoding of States

Suppose machine has 4 states:

$\{S_0, S_1, S_2, S_3\}$

Minimum state bits:

$\lceil \log_2 4 \rceil = 2$

Possible assignment:

$S_0 = 00$

$S_1 = 01$

$S_2 = 10$

$S_3 = 11$

---

## 22. Number of FFs for M States

---

$$\lceil n = \lceil \log_2 M \rceil \rceil$$

Example:

5-state FSM:

$$2^2 = 4 < 5$$

$$2^3 = 8$$

Need:

$$3 \text{ FFs}$$

---

## 23. State Assignment Affects Logic

---

Different binary codes may produce different next-state equations.

Example:

Assignment A:

$$S_0=00, S_1=01, S_2=10$$

may produce one gate structure.

Another assignment:

$$S_0=00, S_1=11, S_2=10$$

may simplify certain transitions.

In small GATE problems, assignment is often specified.

---

## 24. One-Hot Encoding

---

For M states, one-hot encoding uses:

$$M \text{ FFs}$$

Exactly one state bit is 1.

Example 3 states:

$$S_0=001$$

$$S_1=010$$

$$S_2=100$$

It uses more FFs but can simplify logic.

For GATE CS, binary encoding is generally more important unless one-hot is explicitly mentioned.

---

## PART D — FSM DESIGN PROCEDURE

---

### 25. Master FSM Design Flow

---



Given an English specification:

### Step 1

Understand what history must be remembered.

### Step 2

Define symbolic states.

### Step 3

Draw state diagram.

### Step 4

Create state table.

### Step 5

Reduce equivalent states if possible.

### Step 6

Assign binary codes.

### Step 7

Choose flip-flop type.

### Step 8

Derive next-state equations.

### Step 9

Derive output equation.

### Step 10

Implement.

---

## 26. State Meaning Is More Important Than State Name

Instead of thinking:

**State A, State B, State C**

think:

- S0 = no useful pattern seen
- S1 = last useful suffix is 1
- S2 = last useful suffix is 10

This makes sequence-detector FSMs much easier.

---

## PART E — STATEEQUIVALENCE AND REDUCTION

## 27. Why Reduce States?

Fewer states may mean:

- fewer flip-flops
- simpler logic
- smaller design

If two states behave identically for all future inputs, they can sometimes be merged.

## 28. Equivalent States — Intuition

Two states are equivalent if:

**For every possible future input sequence, they produce the same output behavior and transition into equivalent states.**

This is a recursive idea.

## 29. Moore State Equivalence

For Moore machines, two states can be equivalent only if:

1. they have the same output
2. for every input, they transition to the same/equivalent states

If outputs differ:

$\boxed{\text{they are definitely not equivalent}}$

## 30. Mealy State Equivalence

For Mealy machines, two states can be equivalent only if for every input:

1. their outputs are the same
2. their next states are the same/equivalent

## 31. Simple Reduction Example

Suppose Moore machine:

| State | Output | X=0 | X=1 |
|-------|--------|-----|-----|
| A     | 0      | A   | B   |
| B     | 1      | C   | B   |
| C     | 1      | C   | B   |

Compare B and C.

Outputs:

$\forall 1=1 \setminus$

For  $X=0$ :

- $B \neq C$
- $C \neq C$

Both lead into same/equivalent class.

For  $X=1$ :

both  $\neq B$ .

Thus B and C may be equivalent and merged.

---

## 32. First Partition Method $\emptyset = \beta_a$

---

For Moore state reduction:

Start by grouping states with same outputs.

Example outputs:

- $A, C, E \neq 0$
- $B, D \neq 1$

Initial partition:

$\setminus \{A, C, E\}, \{B, D\} \setminus$

Then split groups if states transition to different equivalence classes for some input.

Repeat until no more splitting.

---

## 33. Mealy Initial Partition

---

For Mealy:

Group states that produce the same output pattern for all input values.

Example with binary input:

State A outputs:

$\setminus 0 \text{ for } X=0, \quad 1 \text{ for } X=1 \setminus$

State B also:

$\setminus 0, 1 \setminus$

They may stay together initially.

If output pattern differs, split immediately.

---

## 34. Reachable-State Reduction

---

Before deeper equivalence analysis, remove unreachable states if initial/reset state is specified.

An unreachable state can never occur in normal operation.

So it does not need to be retained in a minimal machine.

---

---

## 35. Reachability Example

Initial:

$\{ S_0 \}$

Transitions only reach:

$\{ S_0, S_1, S_2 \}$

State  $(S_3)$  has no incoming reachable path.

Then:

$\boxed{S_3 \text{ is unreachable}}$

and may be removed from the reachable FSM representation.

---

## PART F — MOORE !”MEALY CONVERSION

---

### 36. Mealy to Moore

The output in Mealy is attached to transitions.

In Moore, output must be attached to states.

Therefore if the same Mealy destination state is entered with different output values, it may need to be split into multiple Moore states.

This is why 'Mealy' Moore can increase state count.

---

### 37. Example Principle

Suppose Mealy transitions enter state B with:

- output 0 from one path
- output 1 from another path

A Moore state can only have one fixed output.

Therefore B may need to become:

$\{ B_0 \}$

and:

$\{ B_1 \}$

with outputs 0 and 1 respectively.

---

### 38. Mealy !' Moore General Effect

$\boxed{\text{state count may increase}}$

Also output timing may effectively shift by a state transition/clock depending on interpretation.

You must be careful comparing output sequences.

---

---

## 39. Moore to Mealy

A Moore state's output can usually be moved onto transitions entering or leaving appropriate states.

Often this allows merging states.

Therefore:

$\boxed{\text{Moore} \rightarrow \text{Mealy may reduce state count}}$

---

## 40. Important Conversion Trap

A naive "Moore to Mealy" conversion may appear to shift output timing by one clock.

Why?

Moore output is associated with **being in a state**.

Mealy output is associated with **taking a transition**.

Always compare the intended input/output timing carefully.

---

## PART G — MOORE/MEALY OUTPUT EQUATIONS

### 41. Moore Output Equation

Suppose binary states:

$S_0=00, S_1=01, S_2=10, S_3=11$

and output 1 in S2 and S3.

Then:

$Y=Q_1$

because both states have MSB=1.

---

### 42. Mealy Output Equation

Suppose output 1 when:

$Q=1$

and:

$X=1$

Then:

$\boxed{Y=QX}$

Output can change when X changes while Q stays fixed.

---

## PART H — EXAMPLE FSM ANALYSIS

---

### 43. Moore Example

---

States:

- A/0
- B/1

Transitions:

A:

- $X=0 \rightarrow A$
- $X=1 \rightarrow B$

B:

- $X=0 \rightarrow A$
- $X=1 \rightarrow B$

Initial A.

Input sequence:

\[ 1,1,0,1 \]

Let's trace.

Initial:

\[ A, Y=0 \]

Input 1:

A' B

new Moore output:

\[ 1 \]

Next 1:

B' B

output:

\[ 1 \]

Next 0:

B' A

output:

\[ 0 \]

Next 1:

A' B

output:

\[ 1 \]

State/output tracing is essential.

---

## 44. Mealy Example

Suppose:

A:

- 0/0 !' A
- 1/1 !' B

B:

- 0/0 !' A
- 1/1 !' B

Input:

$\backslash [ 1,1,0,1 \backslash ]$

Outputs occur on transitions:

$\backslash [ 1,1,0,1 \backslash ]$

No separate output state is needed.

---

## 45. Moore Initial Output

Moore machine has an output even before the first new input symbol because output belongs to the initial state.

So for input sequence length  $\backslash (n) \backslash$ , sometimes a trace includes:

$\backslash [ n+1 \backslash ]$

state/output observations including the initial state.

Be careful with what the question asks.

---

## 46. Mealy Output Count

For  $\backslash (n) \backslash$  input symbols, a Mealy machine typically produces:

$\backslash [ n \backslash ]$

transition outputs.

This is a useful difference in sequence tracing.

---

## PART I — FSM IMPLEMENTATION

## 47. State Register

An FSM with  $\backslash (k) \backslash$  state bits uses a register of  $k$  flip-flops.

If D FFs are used:

$\backslash [ D_i = Q_i^{+} \backslash ]$

Next-state logic computes these D inputs.

---

---

## 48. Output Logic

---

### Moore

$$\boxed{Y=g(Q)}$$

### Mealy

$$\boxed{Y=g(Q,X)}$$

Output logic is separate from next-state logic, though logic may be shared in optimized hardware.

---

## 49. FSM Timing

---

At each active clock edge:

1. present state is sampled into next state
2. FF outputs update after  $t_{CQ}$
3. combinational next-state/output logic settles
4. next edge captures next state

This follows the synchronous timing model from Chapter 15.

---

## PART J — COMMON DESIGN EXAMPLE

---

### 50. “Remember Previous Bit”

---

Suppose output should be 1 whenever current input differs from previous input.

We must remember previous input.

So states:

- S0 = previous input was 0
- S1 = previous input was 1

This is an example of turning memory requirement into states.

---

### 51. Mealy Design

---

From S0:

- $X=0 \rightarrow S0, Y=0$
- $X=1 \rightarrow S1, Y=1$

From S1:

- $X=0 \rightarrow S0, Y=1$
- $X=1 \rightarrow S1, Y=0$

Notice:

$$Y=X \oplus Q$$



if Q encodes previous bit.

This is a 2-state Mealy machine.

---

## 52. State Meaning Trick

When designing an FSM, always ask:

**What is the minimum piece of past information I need to remember?**

That answer usually defines the states naturally.

---

## PART K — L0 PRACTICE

---

### 53. Which Model?

Output depends only on current state.

Answer:

`\boxed{\text{Moore}}`

---

### 54. Which Model?

Output depends on state and input.

Answer:

`\boxed{\text{Mealy}}`

---

### 55. Diagram Label

Which machine usually labels transitions as:

`\input/output`

Answer:

`\boxed{\text{Mealy}}`

---

### 56. State Label

Which machine often uses:

`\State/Output`

Answer:

`\boxed{\text{Moore}}`

## PART L — L1 PRACTICE

---

### 57. Number of FFs

---

FSM has 10 states.

Minimum binary state bits:

$$\lceil \log_2 10 \rceil = 4$$

$$\boxed{4}$$

---

### 58. Output Stability

---

Which is usually less directly sensitive to asynchronous input glitches?

$$\boxed{\text{Moore}}$$

because output depends only on stored state.

---

### 59. State Count

---

Which equivalent implementation often uses fewer states?

$$\boxed{\text{Mealy}}$$

---

### 60. Moore Output

---

If states:

$$S_0=00, S_1=01, S_2=10, S_3=11$$

and output 1 only in S3:

$$\boxed{Y = Q_1 Q_0}$$

---

## PART M — L2 PRACTICE

---

### 61. State Equivalence

---

Moore states A and B have different outputs.

Can they be equivalent?

$$\boxed{\text{No}}$$

Different current output immediately distinguishes them.

---

### 62. Mealy Equivalence

---

Two Mealy states produce different outputs for X=1.

Can they be equivalent?

---

$\boxed{\text{No}}$

---

## 63. Reachability

---

Initial state A.

Reachable:

$A \rightarrow B \rightarrow C$

No transition from A/B/C reaches D.

Then:

$\boxed{D \text{ is unreachable}}$

---

## 64. Mealy Timing

---

Current state remains unchanged but input X changes.

Can output change?

$\boxed{\text{Yes}}$

because:

$Y = f(Q, X)$

---

## 65. Moore Timing

---

Current state remains unchanged but input changes.

Can Moore output change purely because of that input change?

$\boxed{\text{No}}$

under the standard Moore definition.

---

## PART N — L3 GATE TRAPS

---

### 66. Trap — “Mealy output changes only on clock edge”

---

False.

Since output depends directly on input:

$Y = g(Q, X)$

it may change between clock edges when X changes.

---

### 67. Trap — “Moore always has exactly one more state than Mealy”

---

False.

Moore **often** uses more states, but the difference is not universally exactly one.

---

---

## 68. Trap — State Names

States:

$\{A, B, C, D\}$

do not automatically require 4 flip-flops.

Binary encoding needs:

$\lceil \log_2 4 \rceil = 2$

---

## 69. Trap — Initial Moore Output

If tracing a Moore FSM, do not forget output associated with the initial state if the question includes output before first transition.

---

## 70. Trap — Equivalent Outputs Only

Two Moore states having same output does **not** automatically mean equivalent.

Their future transitions must also go to equivalent states.

---

## 71. Trap — Same Next States Only

Two Mealy states may transition to same next states but produce different outputs.

Then they are not equivalent.

---

## 72. Trap — Unreachable State

A state present in the table may be irrelevant from the reset state.

Always check reachability before claiming it is part of the operational cycle.

---

## 73. Trap — Output Placement

Moore:

$\boxed{\text{output on state}}$

Mealy:

$\boxed{\text{output on edge}}$

Switching these is one of the most common exam mistakes.

---

## PART O — GATE PYQ PATTERNS

Typical GATE FSM questions include:

---

- identify Moore vs Mealy
- trace output for an input sequence
- minimum state count
- state reduction/equivalence
- unreachable-state elimination
- number of flip-flops
- convert state diagram to table
- derive output equation
- compare Mealy/Moore state counts
- understand output timing
- design sequence detectors

These are PYQ-style categories; no specific year is claimed.

---

## 74. FSM Master Formula Sheet

---

Next state:

$$Q^{+} = f(Q, X)$$

Moore output:

$$Y = g(Q)$$

Mealy output:

$$Y = g(Q, X)$$

Minimum binary FFs for M states:

$$\lceil \log_2 M \rceil$$

D-FF implementation:

$$D_i = Q_i^{+}$$

---

## 75. Moore vs Mealy Master Memory Trick

---

### Moore

**MOORE = output belongs to the MOde/state**

Think:

$$\text{State} \rightarrow \text{Output}$$

### Mealy

Think:

$$\text{State} + \text{Input} \rightarrow \text{Output}$$

or simply:

## 76. 30-Second Revision

FSM:

$\boxed{\text{finite states} + \text{transitions} + \text{outputs}}$

Moore:

$\boxed{Y=f(Q)}$

- output on states
- usually more stable
- may need more states

Mealy:

$\boxed{Y=f(Q,X)}$

- output on transitions
- faster response to input
- often fewer states
- more glitch-sensitive

Equivalent states must produce indistinguishable future behavior.

Remove unreachable states.

Minimum state bits:

$\boxed{\lceil \log_2 M \rceil}$

## What You Absolutely Must Remember

1. An FSM stores only the past information needed for future decisions.
2. Next state depends on present state and input.
3. Moore output depends only on state.
4. Mealy output depends on state and input.
5. Moore outputs are written on states.
6. Mealy outputs are written on transitions.
7. Mealy output can change between clock edges if input changes.
8. Moore usually gives more stable outputs.
9. Mealy often requires fewer states.
10. Same output alone does not prove state equivalence.
11. Equivalent states must have equivalent future behavior for every input.
12. Remove unreachable states when minimizing.
13. Binary encoding of  $M$  states needs  $\lceil \log_2 M \rceil$  FFs.
14. Moore! Moore! reduce states.
15. Mealy! Mealy! increase states.

16. State meaning should represent the minimum useful memory of the past.

17. Sequence detectors are a direct FSM application.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' Registers ' !' Counters ' !' Sequential Analysis ' !' Sequential Design ' !' **FSM: Moore & Mealy** '

**Current: Chapter 24 — Sequence Detectors**

**Remaining:** Sequence Detectors !' Fixed-Point Representation !' Floating-Point Fundamentals !' IEEE-754

## NEXT — Chapter 24: SEQUENCE DETECTORS 0=1 MUST MASTER

Next we will cover:

overlapping vs non-overlapping detectors !' Moore detector design !' Mealy detector design !' state meaning from prefixes/suffixes !' detecting sequences like 101 and 1101 !' output timing !' minimum states !' state tables/diagrams !' repeated/overlapping pattern logic !' GATE traps.

USER

next

ASSISTANT

## CHAPTER 24 — SEQUENCE DETECTORS 0=1 MUST MASTER

A **sequence detector** is an FSM that watches a serial input stream and produces an output when a particular bit pattern appears.

Example target:

$\boxed{101}$

Input stream:

$1101011$

The detector should recognize every occurrence of:

$101$

Sequence detectors are classic GATE problems because they test:

- state meaning
- Moore vs Mealy
- overlap
- state diagrams
- suffix/prefix reasoning
- output timing

## 1. Core Idea

A detector does not need to remember the whole input stream.

It only remembers:

**How much of the target pattern has been matched so far.**

For target:

$\backslash [ 101 \backslash ]$

we may need to remember:

- nothing matched
- matched 1
- matched 10

That is enough.

## 2. Overlapping vs Non-Overlapping Ø=Ý4

This distinction is extremely important.

Suppose target:

$\backslash [ 101 \backslash ]$

and input:

$\backslash [ 10101 \backslash ]$

Occurrences are:

- bits 1–3 !' 101
- bits 3–5 !' 101

They share the middle/ending 1.

That is overlap.

## 3. Overlapping Detector

An overlapping detector allows part of one detected sequence to become the beginning of the next.

For:

$\backslash [ 10101 \backslash ]$

an overlapping 101 detector finds:

$\backslash [ \boxed{2 \text{ occurrences}} \backslash ]$

## 4. Non-Overlapping Detector

A non-overlapping detector does not reuse bits from the just-detected occurrence.

For:



$\backslash [ 10101 \backslash ]$

after detecting the first 101, it starts fresh.

So typically only:

$\backslash [ \boxed{1} \backslash \text{text{ occurrence}} \backslash ]$

is detected.

## 5. Memory Trick

**Overlap = reuse useful suffix**

**Non-overlap = reset/start fresh after detection**

This is the single best way to remember the difference.

## PART A — MEALY DETECTOR FOR 101

### 6. Why Start With Mealy?

Mealy detectors are often simpler because they can produce output on the transition that completes the pattern.

Target:

$\backslash [ 101 \backslash ]$

Define states:

**S0**

Nothing useful matched.

**S1**

Last useful suffix is:

$\backslash [ 1 \backslash ]$

**S2**

Last useful suffix is:

$\backslash [ 10 \backslash ]$

No extra “detected” state is necessary in Mealy.

## 7. State Meaning Table

| State | Meaning                  |
|-------|--------------------------|
| S0    | no useful prefix matched |

| State | Meaning    |
|-------|------------|
| S1    | matched 1  |
| S2    | matched 10 |

Target is:

$\backslash [ 101 \backslash ]$

When we are in S2 and input 1 arrives, sequence is complete.

So Mealy output can become 1 directly on that transition.

## 8. From S0

Target begins with 1.

### Input 0

Still nothing useful:

$\backslash [ S0 \xrightarrow{0/0} S0 \backslash ]$

### Input 1

Matched first bit:

$\backslash [ S0 \xrightarrow{1/0} S1 \backslash ]$

## 9. From S1

State means we've matched:

$\backslash [ 1 \backslash ]$

### Input 0

Now matched:

$\backslash [ 10 \backslash ]$

So:

$\backslash [ S1 \xrightarrow{0/0} S2 \backslash ]$

### Input 1

We have seen:

$\backslash [ 11 \backslash ]$

The latest 1 can itself be the first bit of a new 101.

So remain:

$\backslash [ S1 \backslash ]$

Thus:

$\backslash [ S1 \xrightarrow{1/0} S1 \backslash ]$

This is overlap/suffix reasoning.

## 10. From S2

State means:

$\backslash [ 10 \backslash ]$

### Input 1

We get:

$\backslash [ 101 \backslash ]$

Pattern detected:

$\backslash [ Y=1 \backslash ]$

For overlapping detector, the last input 1 can also start the next occurrence.

So:

$\backslash [ \boxed{S2 \rightarrow \{1/1\} S1} \backslash ]$

### Input 0

We get:

$\backslash [ 100 \backslash ]$

No useful suffix matching beginning of 101.

So:

$\backslash [ \boxed{S2 \rightarrow \{0/0\} S0} \backslash ]$

## 11. Mealy 101 Overlap Detector

State table:

| Present | X=0  | X=1  |
|---------|------|------|
| S0      | S0/0 | S1/0 |
| S1      | S2/0 | S1/0 |
| S2      | S0/0 | S1/1 |

This is a complete overlapping Mealy detector for 101.

## 12. Trace Example

Input:

$\backslash [ 10101 \backslash ]$

Initial:

$\backslash [ S0 \backslash ]$

**Bit 1**

$$S_0 \xrightarrow{1/0} S_1$$

Output 0.

**Bit 0**

$$S_1 \xrightarrow{0/0} S_2$$

Output 0.

**Bit 1**

$$S_2 \xrightarrow{1/1} S_1$$

Detected first 101.

Output 1.

**Bit 0**

$$S_1 \xrightarrow{0/0} S_2$$
**Bit 1**

$$S_2 \xrightarrow{1/1} S_1$$

Second detection.

Output sequence:

$$\boxed{00101}$$

Two detections.

## PART B — NON-OVERLAPPING 101 MEALY DETECTOR

### 13. What Changes?

Only the transition after successful detection changes.

For overlapping:

$$S_2 \xrightarrow{1/1} S_1$$

because the final 1 is reused.

For non-overlapping:

$$\boxed{S_2 \xrightarrow{1/1} S_0}$$

because after detection we restart completely.

That single transition changes overlap behavior.

### 14. Compare

**Overlap**

$$S_2 \xrightarrow{1/1} S_1$$

## Non-overlap

$\lceil S2 \rightarrow \{1/1\} S0 \rceil$

This is a very common GATE trap.

## PART C — MOORE DETECTOR FOR 101

### 15. Moore Needs an Output State

Moore output belongs to a state.

So we need an additional state representing:

**complete 101 has been detected.**

States:

- S0 = nothing useful
- S1 = matched 1
- S2 = matched 10
- S3 = matched 101, output 1

Thus Moore typically needs:

$\lceil \boxed{4 \text{ states}} \rceil$

while Mealy needed:

$\lceil \boxed{3} \rceil$

### 16. Moore State Outputs

| State | Meaning      | Y |
|-------|--------------|---|
| S0    | nothing      | 0 |
| S1    | matched 1    | 0 |
| S2    | matched 10   | 0 |
| S3    | detected 101 | 1 |

### 17. Transitions to S3

From S2:

If input 1:

$\lceil 10+1=101 \rceil$

So:

$\lceil \boxed{S2 \rightarrow \{1\} S3} \rceil$

Now machine is in output-1 state.

---

## 18. Moore Overlap From S3

---

State S3 means the latest bits ended in:

$\backslash 101 \backslash$

For overlapping detection, useful suffixes matter.

The suffix:

$\backslash 1 \backslash$

is also a prefix of target 101.

So after detection, we should conceptually preserve that useful overlap information.

Now inspect next input.

### From S3, input 0

Current useful suffix starts with final 1 from previous sequence.

Append 0:

$\backslash 10 \backslash$

So go:

$\backslash \boxed{S3 \rightarrow \{0\} S2} \backslash$

### From S3, input 1

The latest useful suffix is:

$\backslash 1 \backslash$

So:

$\backslash \boxed{S3 \rightarrow \{1\} S1} \backslash$

---

## 19. Moore 101 Overlap State Table

---

| Present | X=0 | X=1 | Y |
|---------|-----|-----|---|
| S0      | S0  | S1  | 0 |
| S1      | S2  | S1  | 0 |
| S2      | S0  | S3  | 0 |
| S3      | S2  | S1  | 1 |

This is a clean overlapping Moore detector.

---

## 20. Output Timing Difference

---

Mealy detects on the transition completing 101.

Moore enters output state after that transition.

So Moore output appears associated with reaching S3.

Conceptually:

$\boxed{\text{Mealy can assert output one transition earlier}}$

or more precisely:

**Mealy asserts output while processing the completing input; Moore asserts due to the resulting state.**

This is why sequence traces can appear offset by one state observation.

---

## PART D — THE MOST IMPORTANT DESIGN RULE: PREFIX/SUFFIX

---

### 21. How to Choose Next State

---

Whenever a mismatch occurs, do NOT always go to S0.

Instead ask:

**What suffix of what I've seen is also a prefix of the target?**

This is the heart of overlapping sequence detector design.

---

### 22. Example

---

Target:

$\boxed{101}$

Suppose current matched prefix:

$\boxed{1}$

and new input is:

$\boxed{1}$

Now seen:

$\boxed{11}$

Although 11 is not a prefix of 101, its suffix:

$\boxed{1}$

is.

So stay in state representing:

$\boxed{1}$

rather than resetting.

---

## 23. Another Example

Target:

$\backslash[1101\backslash]$

Suppose matched:

$\backslash[110\backslash]$

and next input is 1.

Then:

$\backslash[1101\backslash]$

detected.

For overlap, ask:

**What suffix of 1101 is also a prefix of 1101?**

Suffixes include:

- 1
- 01
- 101

Prefixes include:

- 1
- 11
- 110

Common non-empty longest one:

$\backslash\boxed{1}\backslash$

So after detection, preserve state representing matched prefix 1.

This prefix/suffix method is general.

## PART E — DESIGN 1101 MEALY DETECTOR

### 24. Define States

Target:

$\backslash[1101\backslash]$

Mealy states correspond to proper prefixes:

- S0 = nothing
- S1 = matched 1
- S2 = matched 11
- S3 = matched 110

Completion occurs on a transition from S3 with input 1.



## 25. From S0

---

Input 0:

$\backslash [ S0 \backslash ]$

Input 1:

$\backslash [ S1 \backslash ]$

Thus:

$\backslash [ S0: 0/0 \backslash \text{to } S0, \backslash \text{quad } 1/0 \backslash \text{to } S1 \backslash ]$

---

## 26. From S1 — matched 1

---

Input 1:

Now:

$\backslash [ 11 \backslash ]$

So:

$\backslash [ S1 \backslash \xrightarrow{1/0} S2 \backslash ]$

Input 0:

Now:

$\backslash [ 10 \backslash ]$

No non-empty suffix of 10 is a prefix of 1101.

Target starts with 1.

Suffixes:

- 0
- 10

none equals 1 or 11 or 110.

So:

$\backslash [ S1 \backslash \xrightarrow{0/0} S0 \backslash ]$

---

## 27. From S2 — matched 11

---

Input 0:

$\backslash [ 110 \backslash ]$

So:

$\backslash [ S2 \backslash \xrightarrow{0/0} S3 \backslash ]$

Input 1:

Now recent string:

$\backslash [ 111 \backslash ]$

Longest suffix that is a prefix of 1101?

---

Suffix:

$\backslash [ 11 \backslash ]$

is a valid prefix.

Therefore remain:

$\backslash [ \boxed{S2} \backslash ]$

Thus:

$\backslash [ S2 \xrightarrow{1/0} S2 \backslash ]$

---

## 28. From S3 — matched 110

Input 1:

$\backslash [ 1101 \backslash ]$

Detection:

$\backslash [ Y=1 \backslash ]$

After detection, longest useful suffix is 1.

So for overlap:

$\backslash [ \boxed{S3 \xrightarrow{1/1} S1} \backslash ]$

Input 0:

Recent:

$\backslash [ 1100 \backslash ]$

No useful non-empty suffix matching prefix.

So:

$\backslash [ \boxed{S3 \xrightarrow{0/0} S0} \backslash ]$

---

## 29. 1101 Mealy Table

| State | X=0  | X=1  |
|-------|------|------|
| S0    | S0/0 | S1/0 |
| S1    | S0/0 | S2/0 |
| S2    | S3/0 | S2/0 |
| S3    | S0/0 | S1/1 |

This is an overlapping detector.

---

## PART F — NUMBER OF STATES

## 30. General Mealy Rule

---

For a target sequence of length:

$\lfloor m \rfloor$

a straightforward Mealy detector often needs:

$\lfloor \boxed{m} \text{ states} \rfloor$

representing matched-prefix lengths:

$\lfloor 0, 1, \dots, m-1 \rfloor$

because detection happens on a transition.

Example:

101 length 3 !' typically 3 states.

1101 length 4 !' typically 4 states.

---

## 31. General Moore Rule

A straightforward Moore detector often needs:

$\lfloor \boxed{m+1} \text{ states} \rfloor$

because it needs a separate detected/output state.

Example:

101 !' 4 states.

1101 !' 5 states.

This is a common rule of thumb, not an absolute proof of minimality for every special sequence.

---

## 32. Why "Often", Not "Always"

State minimization can sometimes merge equivalent states in special designs.

So don't blindly claim:

$\lfloor m \text{ and } m+1 \rfloor$

are universally minimal.

But for standard binary sequence detectors, this is a very useful starting expectation.

---

## PART G — OVERLAPPING EXAMPLES

---

### 33. Sequence 11

Input:

$\lfloor 111 \rfloor$

How many overlapping occurrences of 11?

Positions:

- bits 1–2
- bits 2–3

So:

$\boxed{2}$

Non-overlap would typically detect only first pair and then restart.

---

### 34. Sequence 101 in 10101

---

Overlap:

$\boxed{2}$

Non-overlap:

$\boxed{1}$

---

### 35. Sequence 111 in 1111

---

Overlapping windows:

- bits 1–3 = 111
- bits 2–4 = 111

Thus:

$\boxed{2}$

---

### 36. Sequence 010 in 01010

---

Occurrences:

- 1–3
- 3–5

Thus overlap gives:

$\boxed{2}$

---

## PART H — MEALY VS MOORE FOR DETECTORS

---

### 37. Mealy Advantages

---

- usually fewer states
  - output asserted immediately on completing transition
  - compact design
- 

### 38. Mealy Disadvantages

---

Output directly depends on input:

$$Y = f(Q, X)$$

So it may be:

- more glitch-sensitive
- more sensitive to asynchronous input changes

---

## 39. Moore Advantages

- output tied to registered state
- generally more stable
- easier output interpretation

---

## 40. Moore Disadvantages

- often one more state
- output may appear one state-transition later in representation

---

# PART I — FROM STATE MACHINE TO HARDWARE

---

## 41. State Assignment

Suppose Mealy 101 detector states:

$$S_0, S_1, S_2$$

Need:

$$\lceil \log_2 3 \rceil = 2$$

flip-flops.

Example:

$$S_0 = 00$$

$$S_1 = 01$$

$$S_2 = 10$$

State 11 unused.

---

## 42. Using D FFs

Once state table is encoded, derive:

$$D_1 = Q_1^+$$

$$D_0 = Q_0^+$$

Then derive output:

$$Y = f(Q_1, Q_0, X)$$

for Mealy.

This is ordinary sequential circuit design from Chapter 22.

---

## 43. Moore Output Equation

If Moore detected state is:

$\lceil S3=11 \rceil$

and only S3 has output 1:

$\lceil \boxed{Y=Q_1Q_0} \rceil$

assuming that state encoding.

---

## 44. Mealy Output Equation Example

Suppose 101 Mealy encoding:

$\lceil S0=00, \quad S1=01, \quad S2=10 \rceil$

Detection occurs when:

- state = S2 = 10
- input X=1

So output:

$\lceil \boxed{Y=Q_1Q_0'X} \rceil$

with this encoding.

---

## PART J — TRACE INPUT SEQUENCES

---

### 45. Best Method

If GATE gives FSM and input string:

1. mark initial state
2. process one input at a time
3. write next state
4. record output at correct place
5. continue

Do not try to mentally jump several bits.

---

### 46. Mealy Trace Table Template

| Step | Present State | Input | Output | Next State |
|------|---------------|-------|--------|------------|
|------|---------------|-------|--------|------------|

This prevents most mistakes.

---

## 47. Moore Trace Table Template

| Step | Present State | State Output | Input | Next State |
|------|---------------|--------------|-------|------------|
|------|---------------|--------------|-------|------------|

Be clear whether output is recorded before or after transition depending on question wording.

---

## PART K — GATE PRACTICE

---

### 48. L0

Target:

$\boxed{101}$

What does state "matched 10" mean?

It means the most recent useful suffix is:

$\boxed{10}$

which is also a prefix of the target.

---

### 49. L0

Which detector generally uses fewer states?

$\boxed{\text{Mealy}}$

---

### 50. L0

In Mealy 101 detector, detection occurs when current matched prefix is:

$\boxed{10}$

and next input is:

$\boxed{1}$

---

### 51. L1 — Overlap

Target:

$\boxed{101}$

Input:

$\boxed{10101}$

Overlapping detections:

$\boxed{2}$

---

## 52. L1 — Non-Overlap

---

Same target and input:

$\lfloor 10101 \rfloor$

Non-overlapping detections:

$\lfloor \boxed{1} \rfloor$

---

## 53. L1 — Mealy State Count

---

Target length:

$\lfloor 4 \rfloor$

Straightforward Mealy design generally starts with:

$\lfloor \boxed{4 \text{ states}} \rfloor$

---

## 54. L1 — Moore State Count

---

Length 4:

$\lfloor \boxed{5 \text{ states}} \rfloor$

in the standard construction.

---

## 55. L2 — Suffix Reasoning

---

Target:

$\lfloor 101 \rfloor$

Current matched:

$\lfloor 1 \rfloor$

Input:

$\lfloor 1 \rfloor$

What state next?

Seen suffix:

$\lfloor 1 \rfloor$

is still a valid first prefix.

Therefore:

$\lfloor \boxed{\text{remain in matched-1 state}} \rfloor$

---

## 56. L2 — Overlap After Detection

---

Target:

$\lfloor 101 \rfloor$

---



After detecting 101, which suffix can start the next occurrence?

$\boxed{1}$

because the last bit 1 equals the first bit of the pattern.

---

## 57. L2 — 1101

After detecting:

$\boxed{1101}$

longest suffix that is also a prefix is:

$\boxed{1}$

---

## 58. L2 — Mealy Output

101 detector in state "10" sees input 1.

Output:

$\boxed{1}$

For overlapping version, next state:

$\boxed{\text{matched 1}}$

---

## 59. L3 — Wrong Reset Trap

Target:

$\boxed{101}$

State matched 1.

New input 1.

A student resets to S0 because 11 is not a target prefix.

Wrong.

The suffix:

$\boxed{1}$

is still a target prefix.

So:

$\boxed{\text{stay in S1}}$

---

## 60. L3 — Moore Timing Trap

A Moore machine usually requires a state with output 1.

Do not place output label 1 directly on an edge and still call it Moore.

---

## 61. L3 — Mealy Timing Trap

---

A Mealy detector's output may assert on the same transition that receives the final pattern bit.

That is a major reason it often uses fewer states.

---

## 62. L3 — State Meaning Trap

---

For target 1101, after seeing 111, do not reset completely.

The suffix:

$\backslash [ 11 \backslash ]$

matches the first two bits of target.

So stay in "matched 11" state.

---

## 63. L3 — Number of Occurrences

---

Target:

$\backslash [ 11 \backslash ]$

Input:

$\backslash [ 11111 \backslash ]$

Overlapping windows:

$\backslash [ (1,2),(2,3),(3,4),(4,5) \backslash ]$

Thus:

$\backslash [ \boxed{4} \backslash ]$

occurrences.

---

## PART L — GATE PYQ PATTERNS

---

Typical sequence-detector questions include:

- design overlap detector
- design non-overlap detector
- distinguish Moore and Mealy state counts
- trace output for an input string
- infer detected pattern from FSM
- identify next state after mismatch
- derive state diagram from target sequence
- derive output equation
- determine number of states/FFs
- handle repeated-prefix patterns such as 101, 111, 1101

These are PYQ-style categories, not specific-year claims.

---

---

## 64. Trap Book — Chapter 24

---

### Trap 1

After mismatch, always return to S0.

Wrong.

Use longest suffix that is also a target prefix.

### Trap 2

Overlap means detecting identical sequences twice at same position.

No.

It means occurrences may share some bits.

### Trap 3

Mealy always needs a separate detected state.

Usually no.

Detection can happen on a transition.

### Trap 4

Moore output belongs on arrow.

Wrong.

Output belongs to state.

### Trap 5

After detection, overlapping machine always resets.

Wrong.

Preserve useful suffix.

### Trap 6

Non-overlap reuses final bits.

No.

It restarts after detection.

### Trap 7

State name should represent total input history.

No.

Represent only useful matched prefix/suffix information.

### Trap 8

For pattern length  $m$ , Moore is always exactly  $m+1$  minimal states.

It's a strong standard construction, not an absolute universal theorem.

## 65. Sequence Detector Master Strategy Ø=Ý4

For target pattern:

$\backslash [ P ]$

Create states based on:

$\backslash [ \boxed{\text{longest prefix of P currently matched as a suffix}} ]$

For every state and next input bit:

1. append the bit
2. find the longest suffix that is also a prefix of P
3. move to that state
4. if complete pattern formed:
  - Mealy !' output on transition
  - Moore !' enter output state

This is the general method.

## 66. 30-Second Revision

Sequence detector = FSM watching serial bits.

Overlap:

$\backslash [ \boxed{\text{reuse useful suffix}} ]$

Non-overlap:

$\backslash [ \boxed{\text{restart after detection}} ]$

Mealy:

- output on transition
- usually  $\backslash (m)$  states for pattern length m

Moore:

- output on state
- usually  $\backslash (m+1)$  states

Best design question:

**What is the longest suffix of what I have seen that is also a prefix of the target?**

That determines the next state.

## What You Absolutely Must Remember

1. Sequence detector states represent matched-prefix progress.
2. Overlap reuses useful ending bits.
3. Non-overlap restarts after detection.
4. Mealy output occurs on the completing transition.
5. Moore uses an output state.

6. Mealy usually uses fewer states.
7. Do not reset after every mismatch.
8. Always use longest suffix = prefix logic.
9. For 101, after seeing 11, remain in state representing 1.
10. For 101 overlap, after detecting 101, preserve final 1.
11. Pattern length  $m$  usually gives  $m$  Mealy states and  $m+1$  Moore states in the standard construction.
12. Mealy output can react with current input.
13. Moore output is tied to state.
14. When tracing, process one bit at a time.
15. State encoding and hardware implementation follow ordinary FSM design.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' Registers ' !' Counters ' !' Sequential Analysis ' !' Sequential Design ' !' FSM ' !' **Sequence Detectors** '

**Current: Chapter 25 — Fixed-Point Representation**

**Remaining:** Fixed-Point Representation !' Floating-Point Fundamentals !' IEEE-754

**NEXT — Chapter 25: FIXED-POINT REPRESENTATION 0=1 MUST MASTER**

Next we move back to number representation:

**binary point position !' unsigned fixed-point !' signed fixed-point !' Q-format !' range and resolution !' quantization !' fixed-point addition/multiplication !' scaling !' overflow !' GATE numericals and traps.**

USER

next

ASSISTANT

## CHAPTER 25 — FIXED-POINT REPRESENTATION 0=1 MUST MASTER

Now we return to **number representation and arithmetic**.

A fixed-point number is a binary number in which the position of the binary point is **fixed in advance**.

This is different from floating point, where the point effectively moves using an exponent.

### 1. Intuition

Suppose we decide that every 8-bit number will be interpreted as:

`\[ xxxx.xxxx ]`

That means:

- 4 bits before binary point
- 4 bits after binary point

Example:

$[0101.1010]$

means:

$[5 + \frac{12}{16} + \frac{18}{16}]$

$[5.625]$

The binary point is not stored physically as a bit.

Its position is understood from the format.

---

## 2. Fixed Point = Integer With a Scale

This is the best way to think about fixed-point numbers.

Suppose raw stored integer is:

$[N]$

and there are  $f$  fractional bits.

Then actual value is:

$\boxed{\text{Value} = \frac{N}{2^f}}$

for unsigned representation.

This one formula simplifies almost everything.

---

## 3. Example

Stored bits:

$[011010]$

Suppose:

$[f=3]$

fractional bits.

Raw unsigned integer:

$[011010]_2 = 26$

Actual value:

$\frac{26}{2^3} = \frac{26}{8} = 3.25$

Therefore:

$\boxed{011.010_2 = 3.25}$

---

## 4. Place Values

Suppose 6-bit format has 3 fractional bits:

---

$$\lfloor b_5b_4b_3.b_2b_1b_0 \rfloor$$

Weights:

$$\lfloor 2^2, 2^1, 2^0, 2^{-1}, 2^{-2}, 2^{-3} \rfloor$$

So:

$$\lfloor 101.101 \rfloor$$

equals:

$$\lfloor 4+1+\frac{12}{16}+\frac{18}{64} \rfloor$$

$$\lfloor =5.625 \rfloor$$

## 5. Resolution Ø=Ý4

If there are  $\lfloor f \rfloor$  fractional bits, the smallest spacing between adjacent representable values is:

$$\lfloor \boxed{\Delta=2^{-f}} \rfloor$$

This is called:

- resolution
- step size
- quantum

Example:

4 fractional bits:

$$\lfloor \Delta=2^{-4}=\frac{1}{16}=0.0625 \rfloor$$

So representable values differ by:

$$\lfloor \boxed{0.0625} \rfloor$$

## 6. More Fractional Bits

Increasing fractional bits gives:

- better precision
- smaller step size

Example:

| Fraction bits | Resolution                 |
|---------------|----------------------------|
| 1             | $\lfloor (0.5) \rfloor$    |
| 2             | $\lfloor (0.25) \rfloor$   |
| 3             | $\lfloor (0.125) \rfloor$  |
| 4             | $\lfloor (0.0625) \rfloor$ |
| 8             | $\lfloor (1/256) \rfloor$  |

Memory:

## 7. Unsigned Fixed-Point Range

Suppose total bits:

$$\lfloor n \rfloor$$

fractional bits:

$$\lfloor f \rfloor$$

Then raw unsigned integer range:

$$\lfloor 0 \leq N \leq 2^n - 1 \rfloor$$

Actual range:

$$\lfloor \boxed{0 \leq x \leq \frac{2^n - 1}{2^f}} \rfloor$$

Alternative form:

$$\lfloor \boxed{0 \leq x \leq 2^{n-f} - 2^{-f}} \rfloor$$

## 8. Example

8 total bits, 3 fractional bits.

Range:

$$\lfloor 0 \rfloor$$

to:

$$\lfloor \frac{255}{8} \rfloor$$

$$\lfloor = 31.875 \rfloor$$

So:

$$\lfloor \boxed{0 \leq x \leq 31.875} \rfloor$$

Resolution:

$$\lfloor \boxed{0.125} \rfloor$$

## 9. Integer Bits vs Fraction Bits

If total bits:

$$\lfloor n \rfloor$$

and fractional bits:

$$\lfloor f \rfloor$$

then number of bits to left of point:

$$\lfloor \boxed{n-f} \rfloor$$



for an unsigned format.

Example:

8 bits,  $(f=3)$ :

$[ \text{xxxxx.xxx} ]$

5 integer-position bits and 3 fractional bits.

---

## 10. Signed Fixed-Point

---

Signed fixed-point commonly uses:

$[ \text{2's complement} ]$

The stored bit pattern is interpreted exactly like a signed 2's-complement integer, then divided by:

$[ 2^f ]$

This is the easiest rule.

---

## 11. Signed Fixed-Point Formula

---

Let signed raw two's-complement integer be:

$[ N_s ]$

Then:

$[ x = \frac{N_s}{2^f} ]$

So all signed-integer knowledge from Chapter 2 still applies.

---

## 12. Signed Range $\emptyset = \mathbb{Y}_4$

---

For n-bit 2's-complement raw integer:

$[ -2^{n-1} \leq N_s \leq 2^{n-1}-1 ]$

Divide by:

$[ 2^f ]$

Therefore:

$[ -2^{n-f-1} \leq x \leq 2^{n-f-1}-2^{-f} ]$

This is an important formula.

---

## 13. Example

---

8-bit signed fixed point with 3 fractional bits.

Raw signed range:

$[ -128 \text{ to } 127 ]$

Divide by 8:

---

$\lfloor -16 \text{ to } 15.875 \rfloor$

Therefore:

$\lfloor \boxed{-16 \leq x \leq 15.875} \rfloor$

Resolution:

$\lfloor \boxed{0.125} \rfloor$

---

## 14. Decode Positive Example

---

8-bit signed fixed-point,  $(f=4)$ :

$\lfloor 00110100 \rfloor$

Raw integer:

$\lfloor 52 \rfloor$

Actual:

$\lfloor \frac{52}{16} = 3.25 \rfloor$

So:

$\lfloor \boxed{0011.0100 = 3.25} \rfloor$

---

## 15. Decode Negative Example

---

8-bit signed fixed-point,  $(f=4)$ :

$\lfloor 11110100 \rfloor$

Interpret as 8-bit two's complement.

Unsigned value:

$\lfloor 244 \rfloor$

Signed raw integer:

$\lfloor 244 - 256 = -12 \rfloor$

Actual:

$\lfloor \frac{-12}{16} = -0.75 \rfloor$

Thus:

$\lfloor \boxed{11110100 = -0.75} \rfloor$

---

## 16. Fast Negative Decode

---

For n-bit signed fixed point:

1. interpret raw bit pattern as signed integer
2. divide by  $(2^f)$

Example:

$\lfloor 10011000 \rfloor$

---

with  $f=3$ .

Unsigned:

$$152$$

Signed:

$$152 - 256 = -104$$

Actual:

$$-104 / 8 = -13$$

$$\boxed{-13}$$

---

## 17. Encoding a Positive Number

---

Suppose:

$$x = 3.625$$

with:

$$f = 3$$

Scale:

$$x \cdot 2^f$$

$$3.625 \times 8 = 29$$

Encode integer 29:

$$29 = 011101_2$$

For 8 bits:

$$00011101$$

Thus stored representation:

$$\boxed{00011101}$$

with implied binary point 3 bits from right.

---

## 18. Encoding Shortcut

---

To encode real number  $x$ :

$$\boxed{\text{raw} = x \times 2^f}$$

Then encode raw as ordinary integer.

For exact representation, raw should be an integer.

---

## 19. Example — Negative Encoding

---

Represent:

$$-2.75$$

---

in 8-bit signed fixed-point with 2 fractional bits.

Scale:

$$\lfloor -2.75 \times 4 = -11 \rfloor$$

Encode -11 as 8-bit 2's complement.

+11:

$$\lfloor 00001011 \rfloor$$

Invert:

$$\lfloor 11110100 \rfloor$$

+1:

$$\lfloor 11110101 \rfloor$$

Therefore:

$$\lfloor \boxed{11110101} \rfloor$$

---

## 20. Verify

---

Raw:

$$\lfloor 11110101 \rfloor$$

is:

$$\lfloor -11 \rfloor$$

Divide by:

$$\lfloor 4 \rfloor$$

$$\lfloor -2.75 \rfloor$$

Correct.

---

## 21. Fractional Representation Limitation

---

Not every decimal fraction has a finite binary representation.

Example:

$$\lfloor 0.1_{10} \rfloor$$

is repeating in binary.

Therefore a fixed number of fractional bits can only approximate it.

This creates:

$$\lfloor \boxed{\text{quantization error}} \rfloor$$

---

## 22. Quantization

---

Suppose resolution is:

---

$\backslash 0.25 \backslash$

Representable values include:

$\backslash 0, \backslash 0.25, \backslash 0.5, \backslash 0.75, \backslash 1, \backslash \text{dots} \backslash$

Value:

$\backslash 0.6 \backslash$

cannot be represented exactly.

It must be approximated by a nearby representable value.

Possible rounding:

$\backslash 0.5 \backslash$

or:

$\backslash 0.75 \backslash$

depending on rounding rule.

---

## 23. Quantization Error

---

If true value is:

$\backslash x \backslash$

and stored approximation:

$\backslash \hat{x} \backslash$

then:

$\backslash \boxed{e = x - \hat{x}} \backslash$

With nearest rounding, absolute quantization error is typically bounded by approximately half the step:

$\backslash \boxed{|e| \leq \frac{\Delta}{2}} \backslash$

assuming no saturation/overflow and normal rounding.

---

## 24. Example

---

Resolution:

$\backslash \Delta = 0.125 \backslash$

Then maximum nearest-rounding quantization error:

$\backslash \frac{0.125}{2} \backslash$

$\backslash \boxed{0.0625} \backslash$

---

## 25. Truncation vs Rounding

---

Suppose exact scaled result lies between two representable values.

### Truncation

Discard lower fractional bits.

## Rounding

Choose nearest representable value.

Rounding generally gives lower average error.

But GATE questions may specify which operation to use.

Never assume.

---

## PART B — Q FORMAT

---

### 26. What Is Q-Format? $Q_m.n$

---

Fixed-point formats are often written using notation like:

$Q_m.n$

Unfortunately, conventions differ between sources about whether the sign bit is included in  $m$ .

So:

$\boxed{\text{Always use the definition supplied by the question}}$

This is a very important caution.

---

### 27. Common Q-Format Interpretation

---

A common convention:

$Q_m.n$

means:

- $m$  integer bits
- $n$  fractional bits

with total sometimes:

$m+n$

or:

$1+m+n$

depending on whether sign bit is counted separately.

Because conventions vary, never memorize range from Q notation without checking definition.

For GATE, explicit total width and fractional bits are safer.

---

### 28. Safe Method for Any Q Format

---

If question gives:

- total width  $W$
- signed/unsigned

- fractional bits  $f$

ignore notation ambiguity and use:

Unsigned:

$$\left[ 0 \text{ to } \frac{2^W - 1}{2^f} \right]$$

Signed 2's complement:

$$\left[ -\frac{2^{W-1}}{2^f} \right]$$

to:

$$\left[ \frac{2^{W-1} - 1}{2^f} \right]$$

That always works.

## PART C — FIXED-POINT ADDITION

### 29. Addition With Same Scaling $f=4$

Suppose two numbers use the same number of fractional bits.

Then their raw integers can simply be added.

Example:

$$[ A = 1.25 ]$$

$$[ B = 2.5 ]$$

with  $f=4$ .

Raw A:

$$[ 1.25 \times 4 = 5 ]$$

Raw B:

$$[ 2.5 \times 4 = 10 ]$$

Raw sum:

$$[ 15 ]$$

Decode:

$$[ 15/4 = 3.75 ]$$

Correct:

$$[ \boxed{1.25 + 2.5 = 3.75} ]$$

### 30. Binary Example

2 fractional bits:

$$[ 01.01 = 1.25 ]$$

$$[ 10.10 = 2.5 ]$$

Add:

$$\lfloor 01.01 + 10.10 = 11.11 \rfloor$$

$$\lfloor \boxed{3.75} \rfloor$$

Binary points align because scales are identical.

---

## 31. Addition Requires Same Scale

---

If two fixed-point numbers have different fractional-bit counts, you cannot directly add raw integers.

Example:

A uses:

$$\lfloor f_A=2 \rfloor$$

B uses:

$$\lfloor f_B=4 \rfloor$$

They represent different units per LSB.

You must first rescale one operand.

---

## 32. Scaling Before Addition

---

Suppose raw A has  $\lfloor f=2 \rfloor$ , and you want to convert to  $\lfloor f=4 \rfloor$ .

Multiply raw value by:

$$\lfloor 2^{4-2}=4 \rfloor$$

which is equivalent to left shifting by 2 bits.

This preserves numerical value while changing scale.

---

## 33. Example

---

A=1.5 in  $\lfloor f=2 \rfloor$ :

Raw:

$$\lfloor 1.5 \times 4 = 6 \rfloor$$

To express with  $\lfloor f=4 \rfloor$ :

$$\lfloor 6 \times 4 = 24 \rfloor$$

Decode:

$$\lfloor 24/16 = 1.5 \rfloor$$

Correct.

---

## 34. Addition Overflow

---

Fixed-point addition can overflow if result exceeds available range.

Example:

---



4-bit signed with  $f=1$ .

Range:

$[-4 \text{ to } 3.5]$

Add:

$3+1=4$

4 is outside range.

Thus:

$\boxed{\text{overflow}}$

Signed overflow rules are the same as ordinary 2's-complement addition because scaling does not change sign behavior.

---

## 35. Key Insight

If two fixed-point numbers have same scale:

$\boxed{\text{fixed-point addition hardware is ordinary integer addition hardware}}$

The binary point is only an interpretation.

This is one reason fixed-point arithmetic is efficient.

---

## PART D — FIXED-POINT MULTIPLICATION

---

### 36. Multiplication Scaling $\emptyset = f_A + f_B$

Suppose:

$A = \frac{a}{2^{f_A}}$

$B = \frac{b}{2^{f_B}}$

Then:

$AB = \frac{ab}{2^{f_A + f_B}}$

Therefore raw product naturally has:

$\boxed{f_A + f_B}$

fractional bits.

This is very important.

---

### 37. Same-Format Example

Both operands have:

$f=3$

Then product initially has:

$\boxed{6 \text{ fractional bits}}$

If you want result back in original  $(f=3)$  format, you must shift right by:

$$\lfloor 3 \rfloor$$

bits, with appropriate truncation/rounding.

---

## 38. Example

---

Represent:

$$\lfloor 1.5 \rfloor$$

with  $(f=2)$ .

Raw:

$$\lfloor 1.5 \times 4 = 6 \rfloor$$

Represent:

$$\lfloor 2.0 \rfloor$$

Raw:

$$\lfloor 2 \times 4 = 8 \rfloor$$

Multiply raw integers:

$$\lfloor 6 \times 8 = 48 \rfloor$$

Raw product has:

$$\lfloor f=4 \rfloor$$

So actual:

$$\lfloor 48/16 = 3 \rfloor$$

Correct.

---

## 39. Returning Product to $f=2$

---

Raw product:

$$\lfloor 48 \rfloor$$

currently scale:

$$\lfloor 2^4 \rfloor$$

To return to  $(f=2)$ :

divide raw by:

$$\lfloor 2^{4-2} = 4 \rfloor$$

$$\lfloor 48/4 = 12 \rfloor$$

Then decode:

$$\lfloor 12/4 = 3 \rfloor$$

Correct.

So:

$\boxed{\text{same-format multiplication often requires right shift by } f}$

after the full raw product.

---

## 40. Product Width

---

Multiplying:

- n-bit number
- m-bit number

may require up to:

$\boxed{n+m \text{ bits}}$

for full product.

If result is forced back into fewer bits, overflow or precision loss may occur.

---

## 41. Fixed-Point Multiplication Trap

---

Do not multiply two raw fixed-point integers and then interpret the product using the **original** fractional-bit count immediately.

The product's scale changes.

Example:

$f=4$  times  $f=4$  gives:

$\boxed{f=8}$

before rescaling.

---

## PART E — DIVISION

---

### 42. Division Scaling

---

Suppose:

$A = \frac{a}{2^{f_A}}$

$B = \frac{b}{2^{f_B}}$

Then:

$\frac{AB}{2^{f_B - f_A}}$

When both operands use same scale:

$f_A = f_B$

their scale factors cancel.

But if you want a fixed-point quotient with  $f_Q$  fractional bits, raw numerator is usually shifted before integer division.

---

## 43. Practical Same-Scale Division

---

If A and B both have  $f$  fractional bits and desired quotient also has  $f$  fractional bits:

$$\boxed{q_{\text{raw}} \approx \frac{a}{2^f b}}$$

This retains fractional precision.

Detailed division implementation is lower priority than addition/multiplication for GATE Digital Logic, but understand the scaling principle.

---

## PART F — RANGE VS PRECISION TRADEOFF

---

### 44. Same Total Bits

---

Suppose total width fixed at:

$$[8]$$

If we increase number of fractional bits:

- precision improves
- representable magnitude decreases

Example unsigned:

$$(f=0)$$

range:

$$[0 \text{ to } 255]$$

resolution:

$$[1]$$

$$(f=4)$$

range:

$$[0 \text{ to } 15.9375]$$

resolution:

$$[0.0625]$$

$$(f=7)$$

range:

$$[0 \text{ to } 1.9921875]$$

resolution:

$$[0.0078125]$$

---

### 45. Core Tradeoff

---

For fixed total width:

$$\boxed{\text{more fraction bits} \rightarrow \text{better precision, smaller range}}$$

---

$\boxed{\text{fewer fraction bits}} \rightarrow \text{larger range, lower precision}$

This is fundamental.

---

## 46. Overflow vs Quantization

These are different.

### Overflow

Value lies outside representable range.

### Quantization

Value lies within range but not exactly on representable grid.

Example:

Format supports 0 to 7.75 in steps of 0.25.

- 8.5 !' overflow
- 3.6 !' quantization required

---

## 47. Saturation vs Wraparound

When overflow occurs, different systems may:

### Wrap

Keep low bits, like modular integer arithmetic.

### Saturate

Clamp to largest/smallest representable value.

Example maximum:

$\boxed{3.75}$

Trying to store 5:

- wrap !' bit-dependent modular result
- saturate !' 3.75

GATE may specify behavior; do not assume saturation.

---

## PART G — EXTENSION AND TRUNCATION

---

### 48. Increasing Width

Signed fixed-point can be widened using sign extension, provided the number of fractional bits remains unchanged.

Example:

4-bit signed raw:

$\boxed{1011}$

with  $f=2$ .

Sign extend to 8 bits:

$\lfloor 11111011 \rfloor$

Numerical fixed-point value remains identical because raw signed integer is preserved.

---

## 49. Increasing Fractional Precision

---

Suppose same numerical value but increase fractional bits by  $k$ .

Raw integer must be shifted left by  $k$ :

$\lfloor \boxed{N_{\text{new}} = N_{\text{old}} 2^k} \rfloor$

Example:

1.5 with  $f=2$ :

raw:

$\lfloor 6 \rfloor$

Convert to  $f=4$ :

$\lfloor 6 \times 4 = 24 \rfloor$

Same value:

$\lfloor 24/16 = 1.5 \rfloor$

---

## 50. Reducing Fractional Bits

---

If reducing fractional bits by  $k$ :

raw value must be shifted right by  $k$ .

But discarded bits cause:

- truncation error
- or rounding, depending on method

So:

$\lfloor \boxed{\text{reducing fractional precision can lose information}} \rfloor$

---

## PART H — GATE PRACTICE

---

### 51. L0 — Resolution

---

8-bit fixed point has 5 fractional bits.

Resolution:

$\lfloor 2^{-5} \rfloor$

$\lfloor \boxed{\frac{1}{32} = 0.03125} \rfloor$

---

## 52. L0 — Unsigned Decode

---

Bits:

$\lfloor 101101 \rfloor$

with  $\lfloor f=2 \rfloor$ .

Raw integer:

$\lfloor 45 \rfloor$

Actual:

$\lfloor 45/4 \rfloor$

$\lfloor \boxed{11.25} \rfloor$

---

## 53. L1 — Signed Decode

---

8-bit signed:

$\lfloor 11100000 \rfloor$

with  $\lfloor f=4 \rfloor$ .

Raw signed integer:

$\lfloor 224-256=-32 \rfloor$

Actual:

$\lfloor -32/16=-2 \rfloor$

$\lfloor \boxed{-2} \rfloor$

---

## 54. L1 — Range

---

6-bit unsigned fixed point,  $\lfloor f=2 \rfloor$ .

Maximum:

$\lfloor \frac{63}{4}=15.75 \rfloor$

Range:

$\lfloor \boxed{0\text{ to }15.75} \rfloor$

Resolution:

$\lfloor \boxed{0.25} \rfloor$

---

## 55. L1 — Signed Range

---

6-bit signed 2's complement,  $\lfloor f=2 \rfloor$ .

Raw range:

$\lfloor -32\text{ to }31 \rfloor$

Actual:

$$\lfloor -8\text{ to }7.75 \rfloor$$

$$\lfloor \boxed{-8 \leq x \leq 7.75} \rfloor$$

---

## 56. L2 — Encoding

---

Encode:

$$\lfloor 2.375 \rfloor$$

with 3 fractional bits.

Scale:

$$\lfloor 2.375 \times 8 = 19 \rfloor$$

19 in binary:

$$\lfloor 10011 \rfloor$$

In 8 bits:

$$\lfloor \boxed{00010011} \rfloor$$

---

## 57. L2 — Negative Encoding

---

Encode:

$$\lfloor -1.25 \rfloor$$

in 8-bit signed fixed point with  $(f=2)$ .

Scale:

$$\lfloor -1.25 \times 4 = -5 \rfloor$$

8-bit 2's complement of -5:

$$\lfloor \boxed{1111011} \rfloor$$

---

## 58. L2 — Addition

---

Two numbers with  $(f=3)$ :

Raw values:

$$\lfloor A=01100_2=12 \rfloor$$

$$\lfloor B=00101_2=5 \rfloor$$

Actual:

$$\lfloor A=12/8=1.5 \rfloor$$

$$\lfloor B=5/8=0.625 \rfloor$$

Raw sum:

$$\lfloor 17 \rfloor$$

Actual:



$$\lfloor 17/8 \rfloor$$

$$\lfloor \boxed{2.125} \rfloor$$

---

## 59. L2 — Multiplication

---

Both operands  $\lfloor f=2 \rfloor$ .

Raw:

$$\lfloor A=6, \text{quad} B=10 \rfloor$$

Actual:

$$\lfloor 1.5, \text{quad} 2.5 \rfloor$$

Raw product:

$$\lfloor 60 \rfloor$$

Product has:

$$\lfloor f=4 \rfloor$$

Actual:

$$\lfloor 60/16 \rfloor$$

$$\lfloor \boxed{3.75} \rfloor$$

---

## 60. L3 — Product Scaling Trap

---

Two  $\lfloor Q \rfloor$ -style numbers each have 5 fractional bits.

Full raw product has:

$$\lfloor \boxed{10 \text{ fractional bits}} \rfloor$$

before conversion back to desired format.

---

## 61. L3 — Range/Precision

---

An 8-bit unsigned fixed-point representation needs resolution:

$$\lfloor 0.125 \rfloor$$

Since:

$$\lfloor 0.125 = 2^{-3} \rfloor$$

need:

$$\lfloor f=3 \rfloor$$

Maximum value:

$$\lfloor \frac{255}{8} \rfloor$$

$$\lfloor \boxed{31.875} \rfloor$$

---

## 62. L3 — Number of Fractional Bits

---

Resolution required:

$$\left[ \frac{1}{64} \right]$$

Need:

$$\left[ 2^{-f} = \frac{1}{64} \right]$$

Therefore:

$$\boxed{f=6}$$

---

## 63. L3 — Can Number Be Exact?

Can:

$$\left[ 0.3 \right]$$

be represented exactly using a finite number of binary fractional bits?

No.

Because 0.3 has a repeating binary expansion.

Therefore:

$$\boxed{\text{only approximation}}$$

---

## 64. L3 — Overflow

Signed 8-bit fixed point with  $(f=4)$ .

Range:

$$\left[ -8 \text{ to } 7.9375 \right]$$

Can 8.0 be represented?

$$\boxed{\text{No}}$$

It overflows even though 8 looks close to the positive limit.

---

## 65. GATE PYQ Patterns

Typical fixed-point questions may ask:

- decode bit pattern
- encode decimal value
- determine range
- determine resolution
- calculate number of fractional bits
- detect overflow
- compare range/precision
- fixed-point addition

- product scaling
- quantization error
- signed 2's-complement fixed-point interpretation

These are PYQ-style categories, not specific-year claims.

---

## 66. Trap Book — Chapter 25

---

### Trap 1

Binary point is stored as an actual bit.

No.

Its position is implicit.

### Trap 2

Interpret signed fixed point separately from 2's complement.

Use signed raw integer then divide by  $(2^f)$ .

### Trap 3

Resolution depends on total bits only.

No.

$\Delta = 2^{-f}$

### Trap 4

More fractional bits increase range.

Opposite, when total width is fixed.

### Trap 5

Add fixed-point values with different scales directly.

Wrong.

Align scales first.

### Trap 6

Multiplying two  $f$ -fractional-bit numbers gives an  $f$ -fractional-bit raw product.

Wrong.

Initially:

$2f$

fractional bits.

### Trap 7

All decimal fractions terminate in binary.

False.

### Trap 8

Overflow and quantization are same.

No.

### Trap 9

Q-format notation is universal.

No. Check convention.

### Trap 10

Truncating fractional bits preserves exact value.

Usually not.

---

## 67. Fixed-Point Formula Sheet Ø=Ý4

If total bits:

$\lfloor n \rfloor$

fraction bits:

$\lfloor f \rfloor$

### Resolution

$\lfloor \boxed{\Delta=2^{-f}} \rfloor$

### Unsigned range

$\lfloor \boxed{0 \leq x \leq 2^{n-f}-2^{-f}} \rfloor$

### Signed two's-complement range

$\lfloor \boxed{-2^{n-f-1} \leq x \leq 2^{n-f-1}-2^{-f}} \rfloor$

### Decode

$\lfloor \boxed{x=\frac{\text{raw integer}}{2^f}} \rfloor$

### Encode

$\lfloor \boxed{\text{raw}=x2^f} \rfloor$

subject to rounding and range.

### Multiply

$\lfloor \boxed{f_{\text{product}}=f_A+f_B} \rfloor$

---

## 68. 30-Second Revision

Fixed point means:

$\boxed{\text{binary point position is fixed}}$

Think:

$\boxed{\text{stored integer} / 2^f}$

Resolution:

$\boxed{2^{-f}}$

Signed:

**decode raw bit pattern as 2's-complement integer, then divide by  $2^f$ .**

Same-scale addition:

**ordinary integer addition.**

Multiplication:

**fractional-bit counts add.**

And:

$\boxed{\text{more fractional bits} = \text{better precision but smaller range}}$

## What You Absolutely Must Remember

1. Fixed point is scaled integer representation.
2. Value = raw integer /  $2^f$ .
3. Encoding multiplies by  $2^f$ .
4. Resolution =  $2^{-f}$ .
5. Unsigned and signed ranges follow directly from integer ranges divided by  $2^f$ .
6. Signed fixed point usually uses 2's complement.
7. Same-scale addition is ordinary binary addition.
8. Different scales must be aligned before addition.
9. Product fractional-bit count is the sum of operand fractional-bit counts.
10. Returning a product to original format requires rescaling.
11. Increasing fractional bits improves precision but reduces range for fixed width.
12. Quantization and overflow are different.
13. Not every decimal fraction is exactly representable in binary.
14. Q-format conventions vary—use the definition given in the problem.
15. Truncation/rounding can introduce quantization error.

## Digital Logic Progress

**Completed:** Number Systems ' ! Signed Representation ' ! Binary Arithmetic ' ! Boolean Algebra ' ! SOP/POS ' ! Universal Gates ' ! Algebraic Minimization ' ! K-map ' ! Tabular Method ' ! Combinational Circuits ' ! Adders/Subtractors ' ! MUX ' ! Decoder/Encoder/DEMUX ' ! Comparator ' ! Sequential Fundamentals ' ! Latches/Flip-Flops ' ! FF Conversion ' ! Race Around/Master-Slave ' ! Registers ' ! Counters ' ! Sequential

Analysis ' ! Sequential Design ' ! FSM ' ! Sequence Detectors ' ! **Fixed-Point Representation** '

**Current: Chapter 26 — Floating-Point Fundamentals**

**Remaining:** Floating-Point Fundamentals ! IEEE-754 ! final Digital Logic master revision/audit.

**NEXT — Chapter 26: FLOATING-POINT FUNDAMENTALS 0=MUST MASTER**

Next we cover:

scientific notation in binary ! sign, significand/mantissa, exponent ! normalization ! normalized vs denormalized concept ! exponent bias ! floating-point range vs precision ! rounding ! floating-point addition/subtraction/multiplication basics ! overflow/underflow ! why 0.1 is not exact ! GATE traps.

USER

next

ASSISTANT

## CHAPTER 26 — FLOATING-POINT FUNDAMENTALS 0=MUST MASTER

Fixed-point keeps the binary point at a fixed location.

Floating-point allows the effective binary point position to move using an exponent.

Think of it as **scientific notation for binary numbers**.

Decimal scientific notation:

$[ 6250 = 6.25 \times 10^3 ]$

Binary floating-point idea:

$[ 101.101_2 = 1.01101_2 \times 2^2 ]$

That basic idea leads directly to IEEE-754 in the next chapter.

### 1. General Floating-Point Form

A floating-point number can be written conceptually as:

$[ \boxed{(-1)^S \times M \times 2^E} ]$

where:

- $(S)$  = sign
- $(M)$  = significand/mantissa
- $(E)$  = exponent

The exact IEEE interpretation comes next chapter.

### 2. Why Floating Point?

Suppose we want to represent both:

$\backslash[ 0.000000001 \backslash]$

and:

$\backslash[ 1000000000 \backslash]$

with limited bits.

Fixed point struggles because the binary point position is fixed.

Floating point changes the scale through the exponent.

So floating point provides:

$\backslash[ \boxed{\text{very large range}} \backslash]$

at the cost of:

- finite precision
  - rounding
  - more complex arithmetic
- 

### 3. Binary Scientific Notation

---

Example:

$\backslash[ 10110.01_2 \backslash]$

Move binary point left until one nonzero digit remains before it:

$\backslash[ 1.011001_2 \times 2^4 \backslash]$

Therefore:

$\backslash[ \boxed{10110.01_2 = 1.011001_2 \times 2^4} \backslash]$

---

### 4. Why Exponent Is 4

---

Original:

$\backslash[ 10110.01 \backslash]$

Normalized:

$\backslash[ 1.011001 \backslash]$

Binary point moved:

$\backslash[ 4 \backslash]$

places left.

Therefore multiply by:

$\backslash[ 2^4 \backslash]$

---

### 5. Negative Exponent Example

---

$\backslash[ 0.001101_2 \backslash]$

Normalize:

---

$$\lfloor 1.101_2 \times 2^{-3} \rfloor$$

because binary point moved 3 positions right.

Thus:

$$\lfloor \boxed{0.001101_2 = 1.101_2 \times 2^{-3}} \rfloor$$

## 6. Normalization Ø=Ý4

A nonzero binary floating-point number is commonly normalized to:

$$\lfloor \boxed{1.f \times 2^E} \rfloor$$

where:

$$\lfloor 1 \leq M < 2 \rfloor$$

in binary.

Example:

$$\lfloor 110.01_2 \rfloor$$

becomes:

$$\lfloor \boxed{1.1001_2 \times 2^2} \rfloor$$

## 7. Why Normalize?

Without normalization, one number can have many forms.

Example:

$$\lfloor 101.1_2 \rfloor$$

could be written:

$$\lfloor 10.11 \times 2^1 \rfloor$$

or:

$$\lfloor 1.011 \times 2^2 \rfloor$$

or:

$$\lfloor 0.1011 \times 2^3 \rfloor$$

Normalization gives a standard form.

This simplifies representation and comparison.

## 8. Sign Bit

Floating point normally stores sign separately.

Conceptually:

$$\lfloor S=0 \rightarrow \text{positive} \rfloor$$

$$\lfloor S=1 \rightarrow \text{negative} \rfloor$$



Then magnitude is represented using significand and exponent.  
This differs from ordinary 2's-complement integer representation.

---

## 9. Significand / Mantissa

Terminology varies.

Strictly, **significand** is the quantity carrying significant digits.

People often say “mantissa.”

For normalized binary:

$1.101101 \times 2^5$

the significand is:

$1.101101_2$

In IEEE normalized numbers, the leading 1 is often implicit.

We will study that next.

---

## 10. Exponent

Exponent determines scale.

Example:

$1.0101 \times 2^7$

means shift binary point 7 positions right.

Example:

$1.0101 \times 2^{-4}$

means shift binary point 4 positions left.

Thus exponent largely controls:

$\boxed{\text{range}}$

---

## 11. Significand Controls Precision

Suppose only a few significand bits are stored.

Then many nearby real numbers cannot be distinguished.

So:

$\boxed{\text{Exponent bits mainly control range}}$

$\boxed{\text{Significand bits mainly control precision}}$

This is one of the most important floating-point concepts.

---

## 12. Range vs Precision

If we allocate more bits to exponent:

- larger/smaller magnitudes can be represented
- range increases

If we allocate more bits to significand:

- more significant binary digits are preserved
- precision improves

Given fixed total width, there is a tradeoff.

---

## 13. Decimal Analogy

---

Suppose decimal floating point stores only 4 significant digits.

Number:

$\backslash[ 123456 \backslash]$

might become:

$\backslash[ 1.235 \times 10^5 \backslash]$

because only four significant digits are retained.

Information after those digits is rounded away.

Binary floating point behaves similarly.

---

## 14. Floating Point Is Approximate

---

Most real numbers cannot be represented exactly with finite bits.

Example:

$\backslash[ 0.1_{10} \backslash]$

has an infinite repeating binary expansion.

So floating point stores a nearby representable value.

This is why calculations such as:

$\backslash[ 0.1 + 0.2 \backslash]$

may not internally equal exact decimal:

$\backslash[ 0.3 \backslash]$

in binary floating-point arithmetic.

---

## 15. Why 0.1 Repeats in Binary

---

Repeated multiplication:

$\backslash[ 0.1 \times 2 = 0.2 \backslash]$

$\backslash[ 0.2 \times 2 = 0.4 \backslash]$

$\backslash[ 0.4 \times 2 = 0.8 \backslash]$

---

$$\lfloor 0.8 \times 2 = 1.6 \rfloor$$

and the process continues cyclically.

Binary expansion:

$$\lfloor 0.0001100110011 \dots_2 \rfloor$$

So finite storage must truncate or round it.

---

## 16. Floating-Point Precision

Precision means roughly:

**How many significant binary digits can be represented reliably?**

It is not the same as range.

A huge number can still have limited precision.

---

## 17. Spacing Between Representable Numbers

Floating-point spacing is **not uniform**.

This is a major difference from fixed point.

With fixed point:

$$\lfloor \Delta = 2^{-f} \rfloor$$

constant everywhere.

With floating point, spacing grows as magnitude grows.

---

## 18. Why Spacing Grows

Suppose significand has limited precision.

Near:

$$\lfloor 1 \rfloor$$

adjacent values may differ by tiny amount.

Near:

$$\lfloor 2^{20} \rfloor$$

the same number of significand bits corresponds to much larger absolute spacing.

So large floating-point values have coarser absolute resolution.

---

## 19. Example With Toy Precision

Suppose normalized significand stores only:

$$\lfloor 1.xxx \rfloor$$

with three fractional bits.

Near exponent 0:

$\{ 1.000, 1.001, 1.010, \dots \}$

spacing:

$2^{-3}=0.125$

But with exponent 4:

$1.000 \times 2^4$

and:

$1.001 \times 2^4$

difference:

$0.001_2 \times 16$

$= \frac{1}{8} \times 16 = 2$

Thus spacing became:

$\boxed{2}$

---

## 20. Relative Precision

Although absolute spacing grows, relative precision stays roughly similar for normalized values.

That is one key strength of floating point:

$\boxed{\text{approximately constant relative precision across a huge range}}$

---

## PART A — EXPONENT BIAS

---

### 21. Why Bias the Exponent?

Exponent can be negative or positive.

Instead of storing exponent in ordinary signed form, IEEE floating point uses a **biased exponent**.

Conceptually:

$\boxed{E_{\text{stored}} = E_{\text{actual}} + \text{Bias}}$

Therefore:

$\boxed{E_{\text{actual}} = E_{\text{stored}} - \text{Bias}}$

---

### 22. Bias Example

Suppose bias:

$7$

Actual exponent:

$$\lfloor -3 \rfloor$$

Stored exponent:

$$\lfloor -3+7=4 \rfloor$$

Actual exponent:

$$\lfloor +5 \rfloor$$

Stored:

$$\lfloor 5+7=12 \rfloor$$

So all stored exponent fields can be unsigned bit patterns.

---

## 23. General Bias

For an exponent field of  $k$  bits, a common bias form is:

$$\boxed{2^{k-1}-1}$$

IEEE uses this.

Examples:

- 8 exponent bits !' bias 127
- 11 exponent bits !' bias 1023

We will use these heavily next chapter.

---

## 24. Why Not Use 2's Complement Exponent?

Biased representation has useful ordering and encoding properties.

It also reserves extreme exponent patterns for special values in IEEE-754.

For GATE, memorize IEEE bias values rather than worrying about hardware motivation.

---

## PART B — NORMALIZED AND SUBNORMAL IDEA

### 25. Normalized Numbers

Typical normalized binary number:

$$\boxed{1.\text{ftimes}2^E}$$

The leading 1 is guaranteed for nonzero normalized binary numbers.

So IEEE need not physically store that leading 1.

This is called the:

$$\boxed{\text{hidden bit / implicit leading 1}}$$

More next chapter.

---

## 26. What Happens Near Zero?

---

If normalized form always requires:

$[1.f]$

there would be a gap between zero and the smallest normalized number.

IEEE introduces:

$[\text{subnormal / denormal numbers}]$

to represent smaller magnitudes gradually.

---

## 27. Subnormal Concept

---

For subnormal numbers, the significand behaves like:

$[0.f]$

rather than:

$[1.f]$

This allows values closer to zero.

The exact IEEE formula comes next chapter.

---

## 28. Gradual Underflow

---

Subnormal numbers let floating-point magnitude decrease gradually toward zero rather than suddenly jumping from smallest normalized value directly to zero.

This behavior is called:

$[\text{gradual underflow}]$

---

## PART C — FLOATING-POINT ADDITION

---

### 29. Addition Is More Complex Than Fixed Point

---

Suppose:

$1.01 \times 2^5$

plus:

$1.11 \times 2^2$

You cannot directly add significands because exponents differ.

First align exponents.

---

### 30. Floating Addition Steps Ø=Ý4

---

General algorithm:

---

1. Compare exponents.
2. Shift significand of smaller exponent right.
3. Make exponents equal.
4. Add/subtract significands.
5. Normalize result.
6. Round to available precision.
7. Check overflow/underflow.

This sequence is extremely important.

---

## 31. Example

---

Add:

$$\boxed{1.010_2 \times 2^3}$$

and:

$$\boxed{1.100_2 \times 2^1}$$

Align second exponent to 3.

Difference:

$$\boxed{3-1=2}$$

Shift second significand right by 2:

$$\boxed{1.100 \times 2^1 = 0.01100 \times 2^3}$$

Now add:

$$\boxed{1.01000 + 0.01100 = 1.10100}$$

Thus:

$$\boxed{1.101_2 \times 2^3}$$

if precision is sufficient.

---

## 32. Why Small Numbers Can Disappear

---

Suppose:

$$\boxed{1.000000 \times 2^{20}}$$

plus a much smaller number:

$$\boxed{1.000000 \times 2^0}$$

To align exponents, smaller significand must shift right by 20 places.

With limited significand bits, it may shift completely out.

Then computed result may equal the large number unchanged.

This is called:

$$\boxed{\text{absorption / loss of significance}}$$


---

### 33. Floating Addition Is Not Exact Algebra

---

Because of rounding:

$$\lfloor (a+b)+c \rfloor$$

may differ from:

$$\lfloor a+(b+c) \rfloor$$

Therefore floating-point addition is generally:

$$\lfloor \boxed{\text{not associative}} \rfloor$$

This is a famous numerical-computing property.

---

### 34. Example Concept

---

Suppose precision is very limited.

Large:

$$\lfloor a=10^8 \rfloor$$

small:

$$\lfloor b=-10^8 \rfloor$$

$$\lfloor c=1 \rfloor$$

Then depending on rounding order, grouping can produce different results.

The exact decimal example depends on precision, but principle is:

$$\lfloor \boxed{\text{rounding after intermediate operations changes result}} \rfloor$$

---

## PART D — FLOATING-POINT SUBTRACTION

---

### 35. Subtraction

---

Same broad process:

1. align exponents
2. subtract significands
3. normalize
4. round

Subtraction can create another issue:

$$\lfloor \boxed{\text{cancellation}} \rfloor$$

---

### 36. Cancellation

---

Suppose two close numbers:

$$\lfloor 1.101001 \times 2^5 \rfloor$$

and:

---



$$\lfloor 1.101000 \times 2^5 \rfloor$$

subtract.

Many leading bits cancel.

Result becomes very small.

This can expose previously insignificant rounding errors.

This is called catastrophic cancellation in numerical computing.

For GATE Digital Logic, understand basic idea, not deep numerical analysis.

---

## PART E — FLOATING-POINT MULTIPLICATION

---

### 37. Multiplication Is Easier Conceptually

---

If:

$$\lfloor A = M_A 2^{E_A} \rfloor$$

and:

$$\lfloor B = M_B 2^{E_B} \rfloor$$

then:

$$\lfloor AB = (M_A M_B) 2^{E_A + E_B} \rfloor$$

So:

$$\lfloor \boxed{\text{multiply significands}} \rfloor$$

and:

$$\lfloor \boxed{\text{add exponents}} \rfloor$$

Then normalize and round.

---

### 38. Multiplication Steps

---

1. Determine sign.
2. Add exponents.
3. Multiply significands.
4. Normalize.
5. Round.
6. Check exponent range.

Sign:

$$\lfloor \boxed{S_{\text{result}} = S_A \oplus S_B} \rfloor$$

just like ordinary sign multiplication.

---

### 39. Example

---

$$\backslash[ 1.10\_2\backslashtimes2^2 \backslash]$$

times:

$$\backslash[ 1.01\_2\backslashtimes2^3 \backslash]$$

Exponents:

$$\backslash[ 2+3=5 \backslash]$$

Significands:

$$\backslash[ 1.10\_2=1.5 \backslash]$$

$$\backslash[ 1.01\_2=1.25 \backslash]$$

Product:

$$\backslash[ 1.875=1.111\_2 \backslash]$$

Thus:

$$\backslash[ \backslashboxed{1.111\_2\backslashtimes2^5} \backslash]$$

---

## 40. If Product Is Not Normalized

---

Example:

$$\backslash[ 1.11\backslashtimes1.10 \backslash]$$

Significand product may become:

$$\backslash[ 10.xxxx \backslash]$$

Then shift point left once:

$$\backslash[ 10.xxxx\backslashtimes2^E = 1.xxxx\backslashtimes2^{\{E+1\}} \backslash]$$

Normalization may therefore change exponent.

---

## PART F — FLOATING-POINT DIVISION

---

### 41. Division

---

If:

$$\backslash[ \backslashfrac{M\_A2^{\{E\_A\}}}{M\_B2^{\{E\_B\}}} \backslash]$$

then:

$$\backslash[ \backslashboxed{\backslashfrac{M\_A}{M\_B}2^{\{E\_A-E\_B\}}} \backslash]$$

So:

- divide significands
- subtract exponents
- XOR signs
- normalize and round

Lower priority than IEEE decoding, but know the concept.

---

---

## PART G — OVERFLOW AND UNDERFLOW

---

### 42. Floating-Point Overflow

If result exponent is too large to represent:

$\boxed{\text{overflow}}$

Example conceptually:

$1.5 \times 2^{100000}$

with a small exponent field.

IEEE commonly produces infinity in many overflow situations depending on rounding mode.

Exact behavior next chapter.

---

### 43. Underflow

If magnitude is too small for the normal exponent range:

$\boxed{\text{underflow}}$

Subnormal representation may still represent it.

If even smaller than minimum subnormal, it may round to zero.

---

### 44. Overflow vs Underflow

#### Overflow

Magnitude too large.

#### Underflow

Magnitude extremely close to zero.

Do not confuse underflow with a negative number.

---

## PART H — ROUNDING

---

### 45. Why Rounding Is Necessary

Suppose significand capacity is 4 fractional bits but exact result is:

$1.101101101\dots$

Cannot store all bits.

Must reduce to something like:

$1.1011$

or:

---

$\lfloor 1.1100 \rfloor$

depending on rounding rule.

---

## 46. Common Rounding Modes Ø=βà

---

IEEE supports multiple modes.

Most important:

$\lfloor \boxed{\text{round to nearest, ties to even}} \rfloor$

Others include:

- toward zero
- toward  $(+\infty)$
- toward  $(-\infty)$

Detailed next chapter.

---

## 47. Guard, Round, Sticky Bits Ø=βà SUPPORT

---

Hardware may keep extra temporary bits while computing:

- Guard bit
- Round bit
- Sticky bit

These help decide how to round accurately.

For core GATE, know their purpose if they appear:

$\lfloor \boxed{\text{extra bits used to determine rounding}} \rfloor$

---

## PART I — SPECIAL FLOATING-POINT VALUES PREVIEW

---

### 48. IEEE-754 Supports Special Values

---

Unlike ordinary fixed point, IEEE floating point explicitly represents:

- $(+0)$
- $(-0)$
- $(+\infty)$
- $(-\infty)$
- NaN
- subnormal numbers

These are encoded using special exponent patterns.

We cover them fully next chapter.

---

### 49. NaN

---

NaN means:

$\boxed{\text{Not a Number}}$

Examples that can produce NaN include operations such as:

$0/0$

or certain undefined floating operations.

Do not treat NaN as ordinary infinity.

## 50. Infinity

Floating-point infinity represents overflow-like or mathematical infinite results in certain cases.

Examples:

$+\infty, -\infty$

IEEE arithmetic has specific rules.

## PART J — FIXED VS FLOATING POINT

### 51. Master Comparison

| Feature              | Fixed Point       | Floating Point      |
|----------------------|-------------------|---------------------|
| Point position       | fixed             | exponent-based      |
| Range                | smaller           | very large          |
| Precision spacing    | uniform           | magnitude-dependent |
| Arithmetic           | simpler           | more complex        |
| Hardware cost        | lower             | higher              |
| Representation error | yes               | yes                 |
| Scaling              | programmer/format | exponent handles it |

### 52. When Fixed Point Is Better

Fixed point can be attractive when:

- known limited range
- predictable precision required
- simple/cheap hardware
- embedded/DSP constraints

Floating point is better when:

- huge dynamic range needed
- numbers vary widely in magnitude

- scientific/engineering computation
- 

## 53. Dynamic Range

---

Floating point is powerful mainly because exponent allows enormous dynamic range.

A relatively small number of exponent bits can represent values differing by huge powers of 2.

---

## PART K — GATE PRACTICE

---

### 54. L0 — Normalize

---

Normalize:

$\lfloor 101.01_2 \rfloor$

Answer:

$\lfloor \boxed{1.0101_2 \times 2^2} \rfloor$

---

### 55. L0 — Normalize Small Fraction

---

$\lfloor 0.000101_2 \rfloor$

Move point 4 positions right:

$\lfloor \boxed{1.01_2 \times 2^{-4}} \rfloor$

---

### 56. L0 — Bias

---

Exponent field has bias 127.

Stored exponent:

$\lfloor 130 \rfloor$

Actual:

$\lfloor 130 - 127 \rfloor$

$\lfloor \boxed{3} \rfloor$

---

### 57. L1 — Stored Exponent

---

Actual:

$\lfloor -5 \rfloor$

Bias:

$\lfloor 127 \rfloor$

Stored:

$\lfloor -5 + 127 \rfloor$

---

$\boxed{122}$

---

## 58. L1 — Addition Order

---

Why must exponents be aligned before addition?

Because significand bits must represent the same positional weights.

You cannot directly add:

$1.01 \times 2^5$

and:

$1.10 \times 2^1$

without rescaling.

---

## 59. L1 — Multiplication

---

Floating-point multiplication generally:

$\boxed{\text{adds exponents}}$

and:

$\boxed{\text{multiplies significands}}$

---

## 60. L1 — Division

---

Floating division:

$\boxed{\text{subtract exponents}}$

and divide significands.

---

## 61. L2 — Normalize Result

---

Suppose multiplication yields:

$10.101 \times 2^4$

Normalize:

$1.0101 \times 2^5$

Thus:

$\boxed{1.0101 \times 2^5}$

---

## 62. L2 — Exponent Alignment

---

Add:

$1.100 \times 2^4$

---

and:

$$\backslash[ 1.000\backslashtimes2^2 \backslash]$$

Convert second:

$$\backslash[ 1.000\backslashtimes2^2 = 0.01000\backslashtimes2^4 \backslash]$$

Then:

$$\backslash[ 1.10000+0.01000=1.11000 \backslash]$$

Result:

$$\backslash[ \backslashboxed{1.110\backslashtimes2^4} \backslash]$$

---

## 63. L2 — Precision vs Range

---

If one bit is moved from significand field to exponent field:

Generally:

- range increases
- precision decreases

$$\backslash[ \backslashboxed{\backslashtext{Range !' ,precision !" } } \backslash]$$

---

## 64. L2 — Spacing

---

Does floating-point spacing remain constant across all magnitudes?

$$\backslash[ \backslashboxed{\backslashtext{No}} \backslash]$$

Absolute spacing increases as exponent increases.

---

## 65. L3 — Addition Absorption

---

Suppose significand precision is limited.

Adding an extremely small number to a huge one may leave result unchanged.

Why?

Small operand is shifted right during exponent alignment until its significant bits are lost.

---

## 66. L3 — Associativity

---

Is floating-point addition associative?

Generally:

$$\backslash[ \backslashboxed{\backslashtext{No}} \backslash]$$

because intermediate rounding matters.

---



## 67. L3 — 0.1 Exact?

---

Can decimal:

$\backslash[ 0.1 \backslash]$

be represented exactly using finite binary floating-point significand bits?

$\backslash[ \boxed{\text{No}} \backslash]$

because its binary representation repeats indefinitely.

---

## 68. L3 — Underflow

---

A result has magnitude smaller than smallest normalized number.

Does it necessarily become zero immediately?

$\backslash[ \boxed{\text{No}} \backslash]$

IEEE subnormal numbers may represent smaller nonzero magnitudes.

---

## 69. GATE PYQ Patterns

---

Floating-point questions commonly test:

- binary normalization
- exponent bias
- significand/exponent roles
- exponent alignment
- floating-point addition steps
- multiplication exponent rules
- precision vs range
- overflow/underflow
- normalized vs subnormal
- representation error
- IEEE field decoding

The last item becomes the major focus of Chapter 27.

---

## 70. Trap Book — Chapter 26

---

### Trap 1

Floating point stores an actual movable point bit.

No.

Exponent determines scaling.

### Trap 2

Exponent controls precision.

---

Mainly range.

Significand mainly controls precision.

### Trap 3

Normalized binary form starts with 0.xxx.

For nonzero normalized binary:

$\boxed{1.xxx}$

### Trap 4

Add significands before aligning exponents.

Wrong.

### Trap 5

Floating multiplication adds significands.

Wrong.

Multiply significands, add exponents.

### Trap 6

Floating-point spacing is uniform.

Wrong.

### Trap 7

Every decimal number has an exact binary float.

False.

### Trap 8

Floating-point addition is always associative.

False.

### Trap 9

Underflow means negative overflow.

No.

It means magnitude too close to zero.

### Trap 10

Subnormal number has the usual implicit leading 1.

Wrong.

Subnormals use a different significand interpretation.

---

## 71. Floating-Point Fundamentals Formula Sheet

---

General conceptual form:

$\boxed{x=(-1)^S M2^E}$

Normalized binary:

$$\boxed{M=1.f}$$

Biased exponent:

$$\boxed{E_{\text{stored}}=E_{\text{actual}}+\text{Bias}}$$

$$\boxed{E_{\text{actual}}=E_{\text{stored}}-\text{Bias}}$$

Typical k-bit exponent bias:

$$\boxed{\text{Bias}=2^{k-1}-1}$$

Multiplication:

$$\boxed{E_R=E_A+E_B}$$

before bias-handling adjustments.

Division:

$$\boxed{E_R=E_A-E_B}$$

---

## 72. Floating Addition Master Algorithm

Memorize:

$$\boxed{\text{Compare exponents} \rightarrow \text{align} \rightarrow \text{add/subtract significands} \rightarrow \text{normalize} \rightarrow \text{round} \rightarrow \text{check range}}$$

---

## 73. 30-Second Revision

Floating point is binary scientific notation:

$$\boxed{(-1)^S \times M \times 2^E}$$

Remember:

- exponent !' range
- significand !' precision
- normalize to  $(1.f \times 2^E)$
- exponent usually stored with bias
- addition requires exponent alignment
- multiplication multiplies significands and adds exponents
- floating spacing is nonuniform
- rounding causes approximation
- 0.1 is not exact in finite binary
- subnormals fill the gap near zero

---

## What You Absolutely Must Remember

1. Floating point is scientific notation in base 2.
2. Main components: sign, significand, exponent.
3. Normalized binary numbers use  $(1.f \times 2^E)$ .

4. Exponent controls scale/range.
5. Significand controls precision.
6. Biased exponent means stored = actual + bias.
7. Addition must align exponents first.
8. After arithmetic, normalize and round.
9. Multiplication adds exponents and multiplies significands.
10. Division subtracts exponents.
11. Absolute spacing grows with magnitude.
12. Floating point gives roughly constant relative precision, not constant absolute precision.
13. Many decimal fractions are not exactly representable.
14. Floating-point addition is generally not associative.
15. Overflow = magnitude too large.
16. Underflow = magnitude extremely small.
17. Subnormals extend representation toward zero.
18. IEEE-754 adds zero, infinity, NaN and special exponent encodings.

## Digital Logic Progress

**Completed:** Number Systems ' !' Signed Representation ' !' Binary Arithmetic ' !' Boolean Algebra ' !' SOP/POS ' !' Universal Gates ' !' Algebraic Minimization ' !' K-map ' !' Tabular Method ' !' Combinational Circuits ' !' Adders/Subtractors ' !' MUX ' !' Decoder/Encoder/DEMUX ' !' Comparator ' !' Sequential Fundamentals ' !' Latches/Flip-Flops ' !' FF Conversion ' !' Race Around/Master-Slave ' !' Registers ' !' Counters ' !' Sequential Analysis ' !' Sequential Design ' !' FSM ' !' Sequence Detectors ' !' Fixed Point ' !' **Floating-Point Fundamentals** '

**Current: Chapter 27 — IEEE-754 Floating-Point Representation**

**Remaining: IEEE-754 only**, followed by the **complete Digital Logic final revision + syllabus coverage audit**.

### NEXT — Chapter 27: IEEE-754 ~~0=1~~ MUST MASTER

Next we finish the actual syllabus with:

**single precision !' double precision !' sign/exponent/fraction fields !' bias 127/1023 !' hidden 1 !' normalized-number decoding !' encoding decimal/binary values !' zero !' subnormal numbers !' infinity !' NaN !' smallest/largest values !' precision !' special exponent patterns !' IEEE arithmetic traps !' GATE numericals.**

USER

next

ASSISTANT

## CHAPTER 27 — IEEE-754 FLOATING-POINT REPRESENTATION ~~0=1~~ MUST MASTER

This is the final core syllabus chapter.

IEEE-754 is the standard representation used for binary floating-point numbers.

For GATE, the highest-value skills are:

- identify sign/exponent/fraction fields
- know single vs double precision
- know bias values
- decode normalized numbers
- encode numbers
- understand zero, subnormal, infinity, NaN
- know hidden leading 1
- know exponent special cases
- solve range/precision numericals

## 1. General IEEE-754 Structure

A floating-point word is divided into:

$\boxed{\text{Sign} \mid \text{Exponent} \mid \text{Fraction}}$

Conceptually:

TEXT

| Sign | Exponent | Fraction / Mantissa field |

For normalized numbers:

$\boxed{x = (-1)^S \times (1.F)_2 \times 2^{E - \text{Bias}}}$

where:

- $(S)$  = sign bit
- $(E)$  = unsigned stored exponent
- $(F)$  = stored fraction field
- leading 1 is implicit

## 2. IEEE-754 Single Precision Ø=Ý4

Single precision uses:

$\boxed{32 \text{ bits}}$

Fields:

| Field    | Bits |
|----------|------|
| Sign     | 1    |
| Exponent | 8    |
| Fraction | 23   |

So:

$\boxed{1 + 8 + 23 = 32}$

Bias:

$\boxed{127}$

### 3. IEEE-754 Double Precision $\emptyset = 1023$

Double precision uses:

$\boxed{64 \text{ bits}}$

Fields:

| Field    | Bits |
|----------|------|
| Sign     | 1    |
| Exponent | 11   |
| Fraction | 52   |

Bias:

$\boxed{1023}$

Thus:

$\boxed{1+11+52=64}$

### 4. Master Table

| Format | Total | Sign | Exponent | Fraction | Bias |
|--------|-------|------|----------|----------|------|
| Single | 32    | 1    | 8        | 23       | 127  |
| Double | 64    | 1    | 11       | 52       | 1023 |

This table must be memorized.

Memory:

**Single = 1-8-23, bias 127**

**Double = 1-11-52, bias 1023**

### 5. Hidden / Implicit Leading 1 $\emptyset = 1$

For normalized binary numbers:

$1.F$

always begins with 1.

So IEEE does not store this leading 1.

Example:

Normalized significand:

$$\boxed{1.10101}$$

Fraction field stores only:

$$\boxed{10101}$$

This gives one extra bit of effective precision.

---

## 6. Effective Precision

---

Single stores 23 fraction bits but normalized significand has:

$$\boxed{24 \text{ significant binary bits}}$$

because:

$$1 + \text{23 stored bits}$$

Double:

$$\boxed{53 \text{ significant binary bits}}$$

because:

$$1 + 52$$

---

## 7. Normalized Number Formula $x = (-1)^S \cdot 2^{E-Bias}$

---

For exponent field:

$$1 \leq E \leq 2^k - 2$$

the number is normalized.

Formula:

$$x = (-1)^S \cdot 2^{E-Bias}$$

This is the main decoding formula.

---

## 8. Sign Bit

---

$$S=0 \rightarrow +$$

$$S=1 \rightarrow -$$

So sign multiplier is:

$$\boxed{(-1)^S}$$

---

## 9. Exponent Decoding

---

Single precision:

$$e = E - 127$$

Double precision:

---

$$\boxed{e=E-1023}$$

where E is interpreted as an unsigned integer.

---

## 10. Example — Simple Normalized Decode

---

Suppose simplified IEEE-like fields:

$$S=0$$

stored exponent:

$$130$$

fraction:

$$010000\dots$$

Single precision bias:

$$127$$

Actual exponent:

$$130-127=3$$

Significand:

$$1.010_2$$

$$=1.25$$

Therefore:

$$1.25 \times 2^3 = 10$$

So value:

$$\boxed{10}$$

---

## 11. Full Example — Encode (5.75)

---

Convert:

$$5.75_{10}$$

to binary.

Integer:

$$5=101_2$$

Fraction:

$$0.75=.11_2$$

Thus:

$$5.75=101.11_2$$

Normalize:

$$1.0111_2 \times 2^2$$



## 12. Sign

---

Positive:

$\boxed{S=0}$

---

## 13. Exponent

---

Actual:

$e=2$

Single precision stored exponent:

$E=2+127=129$

129 in binary:

$\boxed{10000001}$

---

## 14. Fraction

---

Normalized significand:

$1.0111$

Do not store leading 1.

Fraction:

$011100000000000000000000$

Thus single representation:

$\boxed{0 \mid 10000001 \mid 011100000000000000000000}$

---

## 15. Encode Negative Number

---

Represent:

$-5.75$

Everything same except:

$S=1$

So:

$\boxed{1 \mid 10000001 \mid 011100000000000000000000}$

Sign does not require 2's complementing the fraction.

This is a major trap.

---

## 16. IEEE Float Is Not a 2's-Complement Number

---

For a negative floating-point value, you do **not** take 2's complement of the whole 32-bit pattern.

Instead:

---

$\boxed{\text{set sign bit } S=1}$

Exponent and significand encode magnitude.

---

## 17. Example — Decode a Field Pattern

---

Suppose single precision fields:

$S=1$

$E=10000010_2$

$F=010000000000000000000000$

Exponent:

$10000010_2=130$

Actual:

$130-127=3$

Significand:

$1.010_2=1.25$

Value:

$(-1)^1(1.25)2^3$

$=-10$

$\boxed{-10}$

---

## PART A — SPECIAL EXPONENT PATTERNS

---

### 18. Why Exponent 0 and All-1s Are Special

---

Not every exponent bit pattern represents an ordinary normalized exponent.

For IEEE:

#### Exponent all zeros

Used for:

- zero
- subnormal numbers

#### Exponent all ones

Used for:

- infinity
- NaN

This is critical.

---

### 19. Single Precision Exponent Patterns

---

8 exponent bits.

**All zero:**

$\backslash[00000000\backslash]$

**All one:**

$\backslash[11111111\backslash]$

Ordinary normalized exponent fields are:

$\backslash[00000001\backslash]$

through:

$\backslash[11111110\backslash]$

i.e. stored values:

$\backslash[1\text{ to }254\backslash]$

---

## 20. Double Precision

---

11 exponent bits.

Normalized stored exponent values:

$\backslash[1\text{ to }2046\backslash]$

because:

- 0 reserved
  - 2047 reserved
- 

## PART B — ZERO

---

### 21. IEEE Zero

---

Exponent:

$\backslash[E=0\backslash]$

Fraction:

$\backslash[F=0\backslash]$

Then value is zero.

Because sign is separate, IEEE has:

$\backslash[\boxed{+0}\backslash]$

and:

$\backslash[\boxed{-0}\backslash]$

---

### 22. Positive Zero

---

Single:

---

$\boxed{0 \mid 00000000 \mid 000000000000000000000000}$

---

## 23. Negative Zero

---

$\boxed{1 \mid 00000000 \mid 000000000000000000000000}$

Numerically they compare equal in ordinary arithmetic, but their bit patterns differ.

---

## PART C — SUBNORMAL NUMBERS

---

### 24. Subnormal Condition $E=0$

---

Exponent:

$E=0$

Fraction:

$F \neq 0$

Then number is subnormal.

There is no hidden leading 1.

Significand becomes:

$\boxed{0.F}$

---

### 25. Subnormal Formula

---

For subnormal:

$\boxed{x = (-1)^S(0.F)_2 \cdot 2^{1-Bias}}$

Notice exponent is:

$1-Bias$

not:

$0-Bias$

This is important.

---

### 26. Single Precision Subnormal Exponent

---

Bias:

$127$

So subnormal scaling exponent:

$1-127=-126$

Thus:

$\boxed{x = (-1)^S(0.F)_2 \cdot 2^{-126}}$

---

---

## 27. Why Subnormals Exist

They fill the gap between:

$\sqrt{0}$

and smallest normalized positive number.

This gives:

$\sqrt{\boxed{\text{gradual underflow}}}$

---

## 28. Smallest Positive Normalized Single

Smallest normal exponent field:

$\sqrt{00000001}$

Actual exponent:

$\sqrt{1-127=-126}$

Smallest significand:

$\sqrt{1.000\dots}$

Therefore:

$\sqrt{\boxed{2^{-126}}}$

---

## 29. Smallest Positive Subnormal Single

Fraction field minimum nonzero:

$\sqrt{00\dots001}$

Fraction value:

$\sqrt{2^{-23}}$

Subnormal scaling:

$\sqrt{2^{-126}}$

So:

$\sqrt{2^{-23}2^{-126}}$

$\sqrt{\boxed{2^{-149}}}$

Very important result.

---

## 30. Smallest Positive Normal Double

Double bias:

$\sqrt{1023}$

Smallest normal actual exponent:

---

$$\lfloor 1-1023=-1022 \rfloor$$

Thus:

$$\lfloor \boxed{2^{-1022}} \rfloor$$

### 31. Smallest Positive Subnormal Double

Fraction LSB:

$$\lfloor 2^{-52} \rfloor$$

times:

$$\lfloor 2^{-1022} \rfloor$$

gives:

$$\lfloor \boxed{2^{-1074}} \rfloor$$

## PART D — INFINITY

### 32. Infinity Condition

Exponent:

$$\lfloor \boxed{\text{all ones}} \rfloor$$

Fraction:

$$\lfloor \boxed{0} \rfloor$$

Then:

$$\lfloor S=0 \rightarrow +\infty \rfloor$$

$$\lfloor S=1 \rightarrow -\infty \rfloor$$

### 33. Single Infinity

$$\lfloor 0|11111111|000\dots0 \rfloor$$

is:

$$\lfloor +\infty \rfloor$$

$$\lfloor 1|11111111|000\dots0 \rfloor$$

is:

$$\lfloor -\infty \rfloor$$

## PARTE — NaN

### 34. NaN Condition Ø=Ý4

Exponent:

$\boxed{\text{all ones}}$

Fraction:

$\boxed{\neq 0}$

Then:

$\boxed{\text{NaN}}$

Not a Number.

## 35. Examples Producing NaN

Conceptually:

$0/0$

$\infty - \infty$

$\sqrt{-1}$

in ordinary real floating arithmetic may produce NaN.

## 36. NaN vs Infinity

Do not confuse:

### Infinity

$E = \text{all 1s}, F = 0$

### NaN

$E = \text{all 1s}, F \neq 0$

This is a high-frequency trap.

## 37. Quiet and Signaling NaN

IEEE distinguishes:

- quiet NaN
- signaling NaN

For most GATE CS Digital Logic questions, simply knowing NaN encoding category is enough.

## PART F — MASTER SPECIAL-CASE TABLE

| Exponent | Fraction | Meaning   |
|----------|----------|-----------|
| all 0    | all 0    | $(+0/-0)$ |
| all 0    | nonzero  | Subnormal |

| Exponent                | Fraction | Meaning     |
|-------------------------|----------|-------------|
| neither all 0 nor all 1 | any      | Normalized  |
| all 1                   | all 0    | $\pm\infty$ |
| all 1                   | nonzero  | NaN         |

Memorize this table completely.

## 38. GATE-Fast Classification

Look at exponent first.

### Exponent all 0?

Check fraction:

- zero != zero
- nonzero != subnormal

### Exponent all 1?

Check fraction:

- zero != infinity
- nonzero != NaN

### Otherwise

normalized number.

## PART G — EXPONENT RANGE

## 39. Single Precision Normal Exponents

Stored exponent:

$[1 \text{ to } 254]$

Actual:

$[1 - 127 = -126]$

through:

$[254 - 127 = 127]$

So normalized exponent range:

$[-126 \text{ to } +127]$

## 40. Double Precision Normal Exponents

Stored:

$[1 \text{ to } 2046]$



Actual:

$$\lfloor 1-1023=-1022 \rfloor$$

to:

$$\lfloor 2046-1023=1023 \rfloor$$

Thus:

$$\lfloor \boxed{-1022 \text{ to } +1023} \rfloor$$

---

## PART H — LARGEST FINITE VALUE

---

### 41. Largest Single Precision Finite Number

---

Use:

- largest normal exponent before all ones:

$$\lfloor E=254 \rfloor$$

- actual exponent:

$$\lfloor 127 \rfloor$$

- maximum significand:

$$\lfloor 1.111\dots111 \rfloor$$

with 23 fraction 1s.

This significand equals:

$$\lfloor 2-2^{-23} \rfloor$$

Therefore:

$$\lfloor \boxed{(2-2^{-23})2^{127}} \rfloor$$

Equivalent:

$$\lfloor \boxed{2^{128}-2^{104}} \rfloor$$

---

### 42. Largest Double Finite

---

Maximum significand:

$$\lfloor 2-2^{-52} \rfloor$$

Maximum exponent:

$$\lfloor 1023 \rfloor$$

Therefore:

$$\lfloor \boxed{(2-2^{-52})2^{1023}} \rfloor$$

---

### 43. Why Significand Maximum Is $(2-2^{-p})$

---

For single fraction:

$$\lfloor 1.111\ldots111 \rfloor$$

with 23 bits after point.

Fraction part:

$$\lfloor 1-2^{-23} \rfloor$$

including leading 1:

$$\lfloor 2-2^{-23} \rfloor$$

---

## PART I — PRECISION

---

### 44. Single Precision Effective Bits

---

Normalized significand:

$$\lfloor 1+\text{23 fraction bits} \rfloor$$

Thus:

$$\lfloor \boxed{24\text{ binary significant bits}} \rfloor$$

Approximate decimal significant digits:

$$\lfloor \boxed{7\text{ decimal digits}} \rfloor$$

roughly.

---

### 45. Double Precision

---

Effective binary precision:

$$\lfloor \boxed{53\text{ bits}} \rfloor$$

Approximate decimal significant digits:

$$\lfloor \boxed{15\text{--}16} \rfloor$$

roughly.

---

### 46. Why Decimal Digits Are Approximate

---

Because:

$$\lfloor \log_{10}(2) \approx 0.3010 \rfloor$$

24 binary bits:

$$\lfloor 24(0.301) \approx 7.22 \rfloor$$

53 bits:

$$\lfloor 53(0.301) \approx 15.95 \rfloor$$

---

## PART J — SPACING / ULP

---

## 47. Spacing in a Binade

---

For normalized numbers with exponent  $(e)$ , single precision spacing is:

$$\boxed{2^{e-23}}$$

because 23 explicit fraction bits separate adjacent significands.

Double:

$$\boxed{2^{e-52}}$$

---

## 48. Example Near 1.0

---

Single precision near:

$$1 = 1.000 \dots \times 2^0$$

next larger representable number differs by:

$$\boxed{2^{-23}}$$

---

## 49. Machine Epsilon — Common Definition

---

For binary single precision, distance from 1 to next representable larger number:

$$\boxed{2^{-23}}$$

For double:

$$\boxed{2^{-52}}$$

Be careful: some numerical-analysis texts define unit roundoff differently as half of this. Use question's definition.

---

## 50. Spacing Near $(2^k)$

---

Single:

$$\boxed{2^{k-23}}$$

So spacing doubles whenever exponent increases by 1.

---

## PART K — ENCODING ALGORITHM

---

### 51. Decimal/Binary !' IEEE Normalized

---

For a normal nonzero value:

#### Step 1

Determine sign.

#### Step 2

Convert magnitude to binary.

---

### Step 3

Normalize:

$$\lfloor 1.F \times 2^e \rfloor$$

### Step 4

Compute stored exponent:

$$\lfloor E = e + \text{Bias} \rfloor$$

### Step 5

Convert E to exponent field bits.

### Step 6

Drop leading 1 and store fraction bits.

### Step 7

Round if more bits than field allows.

---

## 52. Example — Encode $(13.25)$ Single Precision

---

Convert:

$$\lfloor 13 = 1101 \rfloor$$

$$\lfloor 0.25 = .01 \rfloor$$

So:

$$\lfloor 13.25 = 1101.01_2 \rfloor$$

Normalize:

$$\lfloor 1.10101 \times 2^3 \rfloor$$

Sign:

$$\lfloor S = 0 \rfloor$$

Exponent:

$$\lfloor 3 + 127 = 130 \rfloor$$

130:

$$\lfloor 10000010 \rfloor$$

Fraction:

$$\lfloor 101010000000000000000000 \rfloor$$

So:

$$\lfloor \boxed{0|10000010|101010000000000000000000} \rfloor$$

---

## 53. Encode $(-0.75)$

---

Binary:

---

$\lfloor 0.75 = .11_2 \rfloor$

Normalize:

$\lfloor 1.1 \times 2^{-1} \rfloor$

Sign:

$\lfloor S = 1 \rfloor$

Exponent:

$\lfloor -1 + 127 = 126 \rfloor$

126:

$\lfloor 01111110 \rfloor$

Fraction:

$\lfloor 1000000000000000000000 \rfloor$

Thus:

$\lfloor \boxed{1|01111110|1000000000000000000000} \rfloor$

---

## PART L — DECODING ALGORITHM

---

### 54. IEEE Bits !' Number

---

#### Step 1

Separate S/E/F.

#### Step 2

Classify exponent:

- all 0?
- all 1?
- normal?

#### Step 3

If normal:  $\lfloor e = E - \text{Bias} \rfloor$

#### Step 4

Significand:  $\lfloor 1.F \rfloor$

#### Step 5

Compute:  $\lfloor (-1)^S (1.F) 2^e \rfloor$

---

### 55. Decode Example

---

Single:

$\lfloor 0|10000001|0100000000000000000000 \rfloor$

S:

$\backslash 0 \backslash$

Exponent:

$\backslash 10000001_2=129 \backslash$

Actual:

$\backslash 129-127=2 \backslash$

Significand:

$\backslash 1.01_2=1.25 \backslash$

Value:

$\backslash 1.25 \times 4=5 \backslash$

$\backslash \boxed{5} \backslash$

---

## 56. Decode Another

---

$\backslash 1|01111101|100000000000000000000000 \backslash$

Exponent:

$\backslash 01111101_2=125 \backslash$

Actual:

$\backslash 125-127=-2 \backslash$

Significand:

$\backslash 1.1_2=1.5 \backslash$

Value:

$\backslash -1.5 \times 2^{-2} \backslash$

$\backslash =-1.5/4 \backslash$

$\backslash \boxed{-0.375} \backslash$

---

## PART M — HEX REPRESENTATION

---

### 57. IEEE Patterns Are Often Given in Hex

---

32 bits = 8 hex digits.

Example:

$\backslash 0x3F800000 \backslash$

This is famously single-precision:

$\backslash \boxed{1.0} \backslash$

Let's see why.

---

## 58. $\backslash(0x3F800000\backslash)$

---

Binary:

$\backslash[00111111100000000000000000000000\backslash]$

Split:

Sign:

$\backslash[0\backslash]$

Exponent:

$\backslash[01111111=127\backslash]$

Actual exponent:

$\backslash[0\backslash]$

Fraction:

all 0.

Significand:

$\backslash[1.0\backslash]$

Thus:

$\backslash[\boxed{1.0}\backslash]$

---

## 59. $\backslash(0xBF800000\backslash)$

---

Only sign bit changes.

Therefore:

$\backslash[\boxed{-1.0}\backslash]$

Useful pattern.

---

## 60. $\backslash(0x40000000\backslash)$

---

Fields:

Sign 0.

Exponent:

$\backslash[10000000=128\backslash]$

Actual:

$\backslash[1\backslash]$

Fraction:

0.

So:

$\backslash[1.0\times 2^1\backslash]$

---

---

## PART N — FLOATING-POINT ARITHMETIC TRAPS

---

### 61. Addition

---

Remember:

 $\boxed{\text{align exponents first}}$ 

Then:

- add significands
  - normalize
  - round
- 

### 62. Multiplication

---

For normal conceptual operands:

 $S_R = S_A \oplus S_B$ 

actual exponents:

 $e_R = e_A + e_B$ 

Significands multiply.

Then normalize/round.

Do not simply add **stored biased exponents** without bias correction.

---

### 63. Stored Exponent Multiplication Trap

---

Suppose stored exponents:

 $E_A = e_A + B$  $E_B = e_B + B$ 

Then:

 $E_A + E_B = e_A + e_B + 2B$ 

But desired stored result:

 $E_R = e_A + e_B + B$ 

Therefore:

 $\boxed{E_R = E_A + E_B - B}$ 

before normalization adjustment.

---

### 64. Division Stored Exponent

---

For division:



$$[ e_R = e_A - e_B ]$$

Stored:

$$[ E_R = e_R + B ]$$

Thus:

$$[ \boxed{ E_R = E_A - E_B + B } ]$$

before normalization adjustment.

---

## 65. Rounding May Change Exponent

---

Suppose rounded significand becomes:

$$[ 10.000 \dots ]$$

Then renormalize:

$$[ 1.000 \dots \times 2^{e+1} ]$$

So exponent can increase because of rounding.

---

## PART O — ROUND TO NEAREST, TIES TO EVEN

---

### 66. Basic Idea

---

Choose representable value nearest to exact result.

If exact result is exactly halfway between two representable values, choose the one whose least significant retained bit is:

$$[ \boxed{0} ]$$

i.e. even.

This reduces long-term rounding bias.

---

### 67. Tiny Binary Example

---

Suppose keep 3 fractional bits.

Exact:

$$[ 1.0101000 \dots ]$$

Candidates:

$$[ 1.010 ]$$

and:

$$[ 1.011 ]$$

It's exactly halfway.

Retained LSB:

- 1.010 ends in 0
-

- 1.011 ends in 1

Choose:

$\boxed{1.010}$

ties-to-even.

## PART P — GATE PRACTICE

### 68. L0 — Single Fields

IEEE single precision:

$\boxed{1\text{ sign}, 8\text{ exponent}, 23\text{ fraction}}$

Bias:

$\boxed{127}$

### 69. L0 — Double

$\boxed{1, 11, 52}$

Bias:

$\boxed{1023}$

### 70. L0 — Normal Formula

$\boxed{(-1)^S (1.F) 2^{E-Bias}}$

### 71. L1 — Classify

Exponent all ones, fraction zero:

$\boxed{\infty}$

### 72. L1 — Classify

Exponent all ones, fraction nonzero:

$\boxed{\text{NaN}}$

### 73. L1 — Classify

Exponent all zeros, fraction nonzero:

$\boxed{\text{subnormal}}$

## 74. L1 — Positive Zero

---

Exponent:

$\backslash 0 \backslash$

Fraction:

$\backslash 0 \backslash$

Sign:

$\backslash 0 \backslash$

Answer:

$\backslash \boxed{+0} \backslash$

---

## 75. L1 — Effective Precision

---

Single normalized significant bits:

$\backslash \boxed{24} \backslash$

Double:

$\backslash \boxed{53} \backslash$

---

## 76. L2 — Single Actual Exponent

---

Stored:

$\backslash 200 \backslash$

Actual:

$\backslash 200-127 \backslash$

$\backslash \boxed{73} \backslash$

---

## 77. L2 — Double Actual Exponent

---

Stored:

$\backslash 1000 \backslash$

Actual:

$\backslash 1000-1023 \backslash$

$\backslash \boxed{-23} \backslash$

---

## 78. L2 — Smallest Normal Single

---

$\backslash \boxed{2^{-126}} \backslash$

---

## 79. L2 — Smallest Subnormal Single

---

$\boxed{2^{-149}}$

---

## 80. L2 — Single Maximum Normal Exponent

---

$\boxed{127}$

Not 128.

Because stored 255 is reserved.

---

## 81. L2 — Largest Finite Single

---

$\boxed{(2-2^{-23})2^{127}}$

---

## 82. L2 — Decode

---

Fields:

$S=0$

$E=127$

$F=1000\dots$

Actual exponent:

$0$

Significand:

$1.1_2=1.5$

Thus:

$\boxed{1.5}$

---

## 83. L2 — Decode Negative

---

$S=1, \quad E=128, \quad F=0$

Actual exponent:

$1$

Significand:

$1$

Value:

$\boxed{-2}$

---

## 84. L3 — Hidden Bit Trap

---

For normalized fraction field all zero, significand is not 0.

It is:

$\boxed{1.0}$

because hidden 1 exists.

---

## 85. L3 — Subnormal Hidden Bit

---

For subnormal numbers, hidden bit is:

$\boxed{0}$

not 1.

So significand is:

$0.F$

---

## 86. L3 — Exponent 0 Trap

---

Normal exponent decoding:

$E - \text{Bias}$

cannot be blindly applied to exponent field 0.

Exponent 0 is a special case.

Subnormal effective exponent is:

$\boxed{1 - \text{Bias}}$

---

## 87. L3 — Exponent All Ones Trap

---

Do not interpret:

$255 - 127 = 128$

for single exponent 255 as an ordinary exponent.

255 is reserved for:

- infinity
  - NaN
- 

## 88. L3 — Sign of NaN

---

NaN may have sign bit 0 or 1, but sign is not interpreted as ordinary positive/negative numerical value.

For GATE classification, fraction nonzero + exponent all ones means NaN regardless of sign.

---

## 89. L3 — Negative Float Encoding

---

To encode  $(-x)$ , do not two's complement the positive float pattern.

---

Just alter sign bit while encoding magnitude normally.

---

## 90. L3 — Bias in Multiplication

---

If single stored exponents are:

$$\lfloor E_A = 130 \rfloor$$

$$\lfloor E_B = 129 \rfloor$$

then provisional stored result exponent:

$$\lfloor 130 + 129 - 127 \rfloor$$

$$\lfloor = 132 \rfloor$$

before any normalization shift.

Actual check:

$$\lfloor e_A = 3 \rfloor$$

$$\lfloor e_B = 2 \rfloor$$

sum:

$$\lfloor 5 \rfloor$$

stored:

$$\lfloor 5 + 127 = 132 \rfloor$$

Correct.

---

## PART Q — VERY HIGH-VALUE GATE MASTER TABLE

---

### 91. IEEE Single Precision Cheat Sheet

---

$$\lfloor \boxed{32 = 1 + 8 + 23} \rfloor$$

Bias:

$$\lfloor \boxed{127} \rfloor$$

Normal exponent:

$$\lfloor \boxed{-126 \text{ to } 127} \rfloor$$

Smallest positive normal:

$$\lfloor \boxed{2^{-126}} \rfloor$$

Smallest positive subnormal:

$$\lfloor \boxed{2^{-149}} \rfloor$$

Largest finite:

$$\lfloor \boxed{(2 - 2^{-23}) 2^{127}} \rfloor$$

Precision:

$$\lfloor \boxed{24 \text{ significant binary bits}} \rfloor$$

---

---

## 92. IEEE Double Precision Cheat Sheet

---

$$\boxed{64=1+11+52}$$

Bias:

$$\boxed{1023}$$

Normal exponent:

$$\boxed{-1022\text{ to }1023}$$

Smallest positive normal:

$$\boxed{2^{-1022}}$$

Smallest positive subnormal:

$$\boxed{2^{-1074}}$$

Largest finite:

$$\boxed{(2 \cdot 2^{-52}) 2^{1023}}$$

Precision:

$$\boxed{53\text{ binary significant bits}}$$

---

## 93. Special Values Cheat Sheet

---

$$E=0, F=0$$

$$\boxed{\pm 0}$$

$$E=0, F \neq 0$$

$$\boxed{\text{subnormal}}$$

$$0 < E < \text{all}1$$

$$\boxed{\text{normalized}}$$

$$E=\text{all}1, F=0$$

$$\boxed{\pm \infty}$$

$$E=\text{all}1, F \neq 0$$

$$\boxed{\text{NaN}}$$

---

## 94. GATE-Fast Decoding Shortcut

---

Whenever you see an IEEE bit pattern:

**First:**

Look at exponent.

If special !' classify immediately.

If normal:

1. sign
2. exponent minus bias
3. prepend hidden 1
4. evaluate significand
5. multiply by  $(2^e)$

This is much faster than converting entire 32-bit pattern to decimal first.

---

## 95. GATE-Fast Encoding Shortcut

---

Given decimal number:

1. sign
2. binary conversion
3. normalize
4. exponent + bias
5. drop leading 1
6. pad/round fraction

Do not directly convert decimal integer to 32-bit binary and split.

---

## 96. Trap Book — Chapter 27

---

### Trap 1

Single has 24 stored fraction bits.

Wrong.

23 stored, 24 effective for normals.

### Trap 2

Double has 53 stored fraction bits.

Wrong.

52 stored, 53 effective.

### Trap 3

Exponent 0 means actual exponent  $(-Bias)$ .

Not for subnormals.

Use:

$(1 - Bias)$

### Trap 4

All-ones exponent is largest ordinary exponent.

Wrong.

Reserved for infinity/NaN.



### Trap 5

Fraction all zero means number is zero.

Only if exponent is also zero.

For a normalized number, fraction all zero means significand 1.0.

### Trap 6

NaN = infinity.

No.

Fraction distinguishes them.

### Trap 7

Negative float uses 2's complement.

No.

Separate sign bit.

### Trap 8

Subnormal has implicit leading 1.

No.

Leading bit is 0.

### Trap 9

Single normal actual exponents are  $(-127)$  to  $(128)$ .

Wrong.

$\boxed{-126 \text{ to } 127}$

### Trap 10

Bias for k exponent bits is  $(2^{k-1})$ .

IEEE bias is:

$\boxed{2^{k-1}-1}$

---

## 97. 30-Second IEEE Revision

---

Single:

$\boxed{1/8/23, \text{ bias}=127}$

Double:

$\boxed{1/11/52, \text{ bias}=1023}$

Normal:

$\boxed{(-1)^S (1.F) 2^{E-\text{Bias}}}$

Subnormal:

$\boxed{(-1)^S (0.F) 2^{1-\text{Bias}}}$

Special exponent patterns:

- $0 + F = 0$  !' zero
- $0 + F \neq 0$  subnormal
- all  $1 + F = 0$  !' infinity
- all  $1 + F \neq 0$  NaN

Single:

$\boxed{2^{-126}}$  smallest normal

$\boxed{2^{-149}}$  smallest subnormal

## What You Absolutely Must Remember

1. Single = 32 bits = 1/8/23, bias 127.
2. Double = 64 bits = 1/11/52, bias 1023.
3. Normalized value:

$(-1)^S 2^E (1.F)$

4. Leading 1 is implicit only for normalized values.
5. Exponent all zero is special.
6. Exponent all one is special.
7.  $(E=0, F=0)$  !' signed zero.
8.  $(E=0, F \neq 0)$  !' subnormal.
9.  $(E=all1, F=0)$  !' infinity.
10.  $(E=all1, F \neq 0)$  !' NaN.
11. Subnormal uses  $(0.F)$ , not  $(1.F)$ .
12. Subnormal exponent scaling is  $(1-Bias)$ .
13. Single normal exponent range =  $(-126)$  to  $(127)$ .
14. Double normal exponent range =  $(-1022)$  to  $(1023)$ .
15. Smallest positive single normal =  $(2^{-126})$ .
16. Smallest positive single subnormal =  $(2^{-149})$ .
17. Effective precision: single 24 bits, double 53 bits.
18. Negative floats use sign bit, not 2's complement.
19. Stored exponents require bias correction in arithmetic.
20. Always classify the exponent field before decoding.

## DIGITAL LOGIC CORE SYLLABUS STATUS

You have now completed all 27 planned chapters:

1. Number Systems '
2. Signed Number Representation '
3. Binary Arithmetic '
4. Boolean Algebra '
5. SOP/POS, Minterms/Maxterms '
6. NAND/NOR Universal Gates '
7. Algebraic Minimization '
8. Karnaugh Maps '

9. Quine–McCluskey / Tabular Method '
10. Combinational Circuit Fundamentals '
11. Adders/Subtractors '
12. Multiplexers '
13. DEMUX/Decoder/Encoder '
14. Comparators & Supporting Combinational Circuits '
15. Sequential Fundamentals & Timing '
16. Latches & Flip-Flops '
17. Flip-Flop Conversion '
18. Race-Around & Master-Slave '
19. Registers & Shift Registers '
20. Counters '
21. Sequential Circuit Analysis '
22. Sequential Circuit Design '
23. Moore & Mealy FSMs '
24. Sequence Detectors '
25. Fixed-Point Representation '
26. Floating-Point Fundamentals '
27. IEEE-754 '

The **teaching syllabus** is **now complete**, but your Digital Logic preparation is not finished until we do the final consolidation.

## NEXT — FINAL DIGITAL LOGIC MASTER REVISION + COVERAGE AUDIT

Next I will give you the final exam-oriented package:

**complete syllabus audit !' one-page-style master formula sheet !' Boolean identity sheet !' K-map cheat sheet !' flip-flop master table !' combinational circuit master table !' counter master sheet !' IEEE-754 cheat sheet !' top GATE traps !' high-frequency question patterns !' last-minute revision order !' mixed GATE-style test !' final ZERO-GAP audit.**

USER

next

ASSISTANT

## DIGITAL LOGIC — FINAL MASTER REVISION + ZERO-GAP COVERAGE AUDIT

You have completed the **full Digital Logic teaching sequence**.

Now this chapter is not new theory. It is the **exam consolidation layer**:

$\boxed{\text{Concepts} \rightarrow \text{Formulas} \rightarrow \text{Traps} \rightarrow \text{Speed} \rightarrow \text{Mock}}$

Use this as your last revision source before PYQs/mock tests.

## PART 1 — OFFICIAL GATE SYLLABUS AUDIT

Official syllabus:

**Boolean algebra and minimization – algebraic technique, Karnaugh map, tabular method. Design of combinational and sequential circuits. Number representation and arithmetic (fixed and floating point).**

Coverage:

| Official syllabus area       | Coverage   |
|------------------------------|------------|
| Boolean Algebra              | ' Complete |
| Algebraic Minimization       | ' Complete |
| Karnaugh Maps                | ' Complete |
| Tabular / Quine–McCluskey    | ' Complete |
| Combinational Circuit Design | ' Complete |
| Adders/Subtractors           | ' Complete |
| MUX/DEMUX                    | ' Complete |
| Decoder/Encoder              | ' Complete |
| Comparator/Code Converters   | ' Complete |
| Sequential Fundamentals      | ' Complete |
| Latches/Flip-Flops           | ' Complete |
| FF Conversion                | ' Complete |
| Registers                    | ' Complete |
| Counters                     | ' Complete |
| Sequential Analysis          | ' Complete |
| Sequential Design            | ' Complete |
| Moore/Mealy FSM              | ' Complete |
| Sequence Detectors           | ' Complete |
| Number Systems               | ' Complete |
| Signed Representations       | ' Complete |
| Binary Arithmetic            | ' Complete |
| Fixed Point                  | ' Complete |
| Floating Point               | ' Complete |
| IEEE-754                     | ' Complete |

## Important honesty

---

The **conceptual syllabus is complete**.

But there are still 3 exam-preparation layers after learning:

1. actual GATE PYQs
2. timed mixed practice
3. repeated revision

So:

$\boxed{\text{Syllabus learning = complete}}$

but:

$\boxed{\text{GATE mastery = requires PYQs + revision + mocks}}$

---

## PART 2 — MASTER BOOLEAN IDENTITY SHEET Ø=Ý4

---

These should become automatic.

### Basic

---

$$A+0=A$$

$$A\cdot 1=A$$

$$A+1=1$$

$$A\cdot 0=0$$

$$A+A=A$$

$$AA=A$$

$$A+A'=1$$

$$AA'=0$$

$$(A')'=A$$

---

### Absorption

---

$$\boxed{A+AB=A}$$

$$\boxed{A(A+B)=A}$$

---

### Extended Absorption

---

$$\boxed{A+A'B=A+B}$$

$$\boxed{A(A'+B)=AB}$$

---

### Complement Pair

---

$$\boxed{AB+AB'=A}$$

$$\boxed{AB+A'B=B}$$

## POS Patterns

$$\boxed{(A+B)(A+B')=A}$$

$$\boxed{(A+B)(A'+B)=B}$$

## Distributive

$$A(B+C)=AB+AC$$

Very important dual form:

$$\boxed{A+BC=(A+B)(A+C)}$$

## De Morgan

$$\boxed{(AB)'=A'+B'}$$

$$\boxed{(A+B)'=A'B'}$$

Memory:

**Break the bar and flip the operator.**

## Consensus

$$\boxed{AB+A'C+BC=AB+A'C}$$

Consensus term:

$$BC$$

is redundant.

Dual:

$$(A+B)(A'+C)(B+C)$$

$$\boxed{=(A+B)(A'+C)}$$

## XOR

$$\boxed{A \oplus B = A'B + AB'}$$

$$A \oplus 0 = A$$

$$A \oplus 1 = A'$$

$$A \oplus A = 0$$

$\lceil A \oplus A' = 1 \rceil$

Multi-input XOR:

$\lceil \boxed{1 \text{ for odd number of 1s}} \rceil$

---

## XNOR

$\lceil \boxed{A \odot B = AB + A'B'} \rceil$

1 when inputs are equal.

---

## PART 3 — SOP / POS / MINTERM / MAXTERM CHEAT SHEET

---

### Minterm

---

For input value:

- 1 !' normal variable
- 0 !' complemented variable

Example:

$\lceil A=1, B=0, C=1 \rceil$

minterm:

$\lceil \boxed{AB'C} \rceil$

index:

$\lceil 101_2 = 5 \rceil$

so:

$\lceil m_5 = AB'C \rceil$

---

### Maxterm

---

Rule reverses:

- input 0 !' normal variable
- input 1 !' complemented variable

For 101:

$\lceil \boxed{A'+B+C'} \rceil$

So:

$\lceil M_5 = A'+B+C' \rceil$

---

## Canonical Forms

---

$\lceil \boxed{F = \Sigma m(\text{rows where } F=1)} \rceil$

$\lceil \boxed{F = \Pi M(\text{rows where } F=0)} \rceil$

---

Complement relation:

$$\forall F = \sum m(S)$$

implies:

$$\forall \boxed{F' = \prod M(S)}$$

---

## PART 4 — K-MAP MASTER CHEAT SHEET Ø=Ý4

---

Gray order:

$$\forall \boxed{00, 01, 11, 10}$$

Never:

$$\forall 00, 01, 10, 11$$

for adjacent K-map ordering.

---

## Group Sizes

---

Only:

$$\forall \boxed{1, 2, 4, 8, 16, \dots}$$

No:

- 3
  - 6
  - L-shape
  - diagonal group
- 

## SOP

---

Group:

$$\forall \boxed{1s}$$

For a grouped variable:

- constant 1 !' normal
  - constant 0 !' complemented
  - changes !' eliminate
- 

## POS

---

Group:

$$\forall \boxed{0s}$$

For grouped variable:

- constant 0 !' normal
  - constant 1 !' complemented
-



- changes !' eliminate

---

## Adjacency

Valid:

- horizontal
- vertical
- left-right wrap
- top-bottom wrap
- four corners

Invalid:

- diagonal

---

## Group Size Rule

A group of:

$2^k$

cells removes:

$\lfloor \frac{k}{\text{variables}} \rfloor$

Examples:

pair !' eliminate 1

quad !' eliminate 2

octet !' eliminate 3

---

## Prime Implicant

$\lfloor \text{A maximal valid group} \rfloor$

Essential Prime Implicant:

**Covers at least one required 1 that no other PI covers.**

---

## Don't-Care

For SOP:

use X as 1 only when helpful.

For POS:

use X as 0 only when helpful.

You do **not** have to cover don't-care cells.

## PART 5 — QUINE–McCLUSKEY FAST PROCEDURE

1. Write minterms in fixed-width binary.
2. Group by number of 1s.
3. Compare adjacent groups only.
4. Combine terms differing in exactly one bit.
5. Replace changed bit by  $-$ .
6. Mark combined terms.
7. Repeat.
8. Uncombined terms = prime implicants.
9. Build PI chart.
10. Find essential PIs.
11. Cover remaining minterms minimally.

Example:

$\bar{A}B\bar{C}D$

for variables ABCD means:

$\boxed{AC}$

because B,D disappear.

## PART 6 — UNIVERSAL GATES

### NAND

NOT:

$\boxed{A' = A \text{ NAND } A}$

NAND is naturally suited to:

$\boxed{\text{SOP} \rightarrow \text{NAND-NAND}}$

### NOR

NOT:

$\boxed{A' = A \text{ NOR } A}$

NOR naturally implements:

$\boxed{\text{POS} \rightarrow \text{NOR-NOR}}$

Memory:

**NAND likes products.**

**NOR likes sums.**

## PART 7 — COMBINATIONAL CIRCUIT MASTER TABLE Ø=Ý4

| Circuit             | Important formula                 |
|---------------------|-----------------------------------|
| Half Adder          | $(S=A \oplus B, C=AB)$            |
| Full Adder          | $(S=A \oplus B \oplus C_{in})$    |
| Full Adder Carry    | $(C_{out}=AB+C_{in}(A \oplus B))$ |
| Half Subtractor     | $(D=A \oplus B, Borrow=A'B)$      |
| Full Subtractor     | $(D=A \oplus B \oplus B_{in})$    |
| 2:1 MUX             | $(Y=S'I_0+SI_1)$                  |
| 2! 4 Decoder        | outputs = minterms                |
| Comparator equality | XNOR                              |
| n-bit equality      | AND of XNORs                      |
| Even parity         | XOR behavior                      |

## 8. FULL ADDER MASTER FORMULAS

$$S=A \oplus B \oplus C_{in}$$

$$C_{out}=AB+AC_{in}+BC_{in}$$

Alternative:

$$C_{out}=AB+C_{in}(A \oplus B)$$

Carry generate:

$$G_i=A_iB_i$$

Propagate:

$$P_i=A_i \oplus B_i$$

Then:

$$C_{i+1}=G_i+P_iC_i$$

## 9. ADDER-SUBTRACTOR

Control M:

$$B_i^*=B_i \oplus M$$

$$C_{in}=M$$

M=0:

$$A+B$$

M=1:

$$\lfloor A+B'+1=A-B \rfloor$$

## 10. SIGNED OVERFLOW Ø=Ý4

For 2's-complement addition:

$$\lfloor \boxed{V=C_{\text{in to MSB}} \oplus C_{\text{out of MSB}}} \rfloor$$

Equivalent rule:

**Add two same-sign operands and get opposite-sign result.**

Carry-out is **not** the same as signed overflow.

## PART 8 — MUX MASTER SHEET

A  $(2^n:1)$  MUX needs:

$$\lfloor \boxed{n \text{ select lines}} \rfloor$$

2:1:

$$\lfloor \boxed{Y=S'I_0+SI_1} \rfloor$$

## MUX as Function Generator

For n-variable Boolean function:

Use:

$$\lfloor 2^n:1 \rfloor$$

MUX and connect data inputs to:

$$\lfloor 0/1 \rfloor$$

Or use:

$$\lfloor 2^{n-1}:1 \rfloor$$

and data inputs become:

$$\lfloor 0,1,X,X' \rfloor$$

for remaining variable X.

Mapping:

| $F(X=0), F(X=1)$ | Data |
|------------------|------|
| 0,0              | 0    |
| 1,1              | 1    |
| 0,1              | X    |
| 1,0              | X'   |

---

## Shannon Expansion

$$\boxed{F = X'F_{X=0} + XF_{X=1}}$$

Exactly a 2:1 MUX.

---

## PART 9 — DECODER / ENCODER / DEMUX

---

### Decoder

---

n inputs:

$$\boxed{2^n \text{ outputs}}$$

Outputs represent minterms.

Thus decoder + OR gates can implement SOP functions.

---

### Encoder

---

$$2^n \text{ input lines} \rightarrow \text{output bits}$$

Ordinary encoder assumes one active input.

Priority encoder resolves multiple active inputs.

Valid bit distinguishes:

- no active input
  - input 0 active
- 

### DEMUX

---

$$\boxed{1 \text{ data input} \rightarrow \text{many outputs}}$$

Decoder with enable can often act as DEMUX.

---

## PART 10 — COMPARATOR

---

1-bit:

$$A > B = AB'$$

$$A = B = AB + A'B' = A \odot B$$

$$A < B = A'B$$

For multibit unsigned values:

**Compare the most significant differing bit.**

---

## PART 11 — BINARY/GRAY

Binary !' Gray:

$$\boxed{G_{\text{MSB}} = B_{\text{MSB}}}$$

then:

$$\boxed{G_i = B_{i+1} \oplus B_i}$$

Memory:

**Keep first, XOR neighbors.**

Gray !' Binary:

$$B_{\text{MSB}} = G_{\text{MSB}}$$

$$\boxed{B_i = B_{i+1} \oplus G_i}$$

Cumulative XOR.

## PART 12 — SEQUENTIAL TIMING MASTER SHEET Ø=Ý4

Frequency:

$$\boxed{f = \frac{1}{T}}$$

Useful:

$$\boxed{f(\text{MHz}) = \frac{1000}{T(\text{ns})}}$$

### Setup Constraint

$$\boxed{T_{\text{clk}} \geq t_{\text{CQ(max)}} + t_{\text{comb(max)}} + t_{\text{setup}}}$$

Hence:

$$\boxed{f_{\text{max}} = \frac{1}{t_{\text{CQ(max)}} + t_{\text{comb(max)}} + t_{\text{setup}}}}$$

### Hold Constraint

$$\boxed{t_{\text{CQ(min)}} + t_{\text{comb(min)}} \geq t_{\text{hold}}}$$

Memory:

**Setup !' longest/max path.**

**Hold !' shortest/min path.**

Slowing clock generally does not fix same-edge hold violation.

## PART 13 — FLIP-FLOP MASTER TABLE Ø=Ý4

### Characteristic Behavior

| FF | Inputs | Behavior           |
|----|--------|--------------------|
| SR | 00     | Hold               |
| SR | 01     | Reset              |
| SR | 10     | Set                |
| SR | 11     | Invalid            |
| JK | 00     | Hold               |
| JK | 01     | Reset              |
| JK | 10     | Set                |
| JK | 11     | Toggle             |
| D  | D      | $\mathbb{Q}^{+}=D$ |
| T  | 0      | Hold               |
| T  | 1      | Toggle             |

### Characteristic Equations

SR:

$$\mathbb{Q}^{+}=S+R'\mathbb{Q}$$

JK:

$$\mathbb{Q}^{+}=J\mathbb{Q}'+K'\mathbb{Q}$$

D:

$$\mathbb{Q}^{+}=D$$

T:

$$\mathbb{Q}^{+}=T\oplus \mathbb{Q}$$

### Excitation Master Table

| $\mathbb{Q}'\ \mathbb{Q}^{+}$ | SR  | JK  | D | T |
|-------------------------------|-----|-----|---|---|
| 0! 0                          | 0,X | 0,X | 0 | 0 |
| 0! 1                          | 1,0 | 1,X | 1 | 1 |
| 1! 0                          | 0,1 | X,1 | 0 | 1 |

| $Q!'$ $Q +$ | SR  | JK  | D | T |
|-------------|-----|-----|---|---|
| 1! 1        | X,0 | X,0 | 1 | 0 |

This table is essential for:

- FF conversion
- sequential design
- counter design

## PART 14 — LATCH VS FLIP-FLOP

Latch:

$\boxed{\text{level-sensitive}}$

Flip-flop:

$\boxed{\text{edge-triggered}}$

D latch active:

$Q^{+}=D$

D latch disabled:

$Q^{+}=Q$

### SR Latch Trap

NOR active-high:

$11=\text{invalid}$

NAND active-low:

$00=\text{invalid}$

Never mix these.

## PART 15 — FF CONVERSION MASTER SHORTCUTS

### Available D

$\boxed{D=Q^{+}_{\text{desired}}}$

### Available T

$\boxed{T=Q \oplus Q^{+}_{\text{desired}}}$

Important conversions:

T using JK:

$\boxed{J=K=T}$

D using JK:



$$\boxed{J=D, K=D'}$$

JK using D:

$$\boxed{D=JQ'+K'Q}$$

D using T:

$$\boxed{T=D \oplus Q}$$

JK using SR:

$$\boxed{S=JQ', R=KQ}$$

## PART 16 — RACE-AROUND

Classic race-around:

$$\boxed{J=K=1}$$

in a level-triggered JK implementation with active pulse long enough.

Output may:

$$Q \rightarrow Q \rightarrow Q \rightarrow Q \dots$$

during the same pulse.

Solutions:

- master-slave
- edge-triggering
- sufficiently narrow pulse

## PART 17 — REGISTERS

n-bit register:

$$\boxed{n \text{ FFs}}$$

possible bit patterns:

$$\boxed{2^n}$$

Types:

- SISO
- SIPO
- PISO
- PIPO

Universal shift register usually supports:

- hold
- left shift
- right shift
- parallel load

## Critical synchronous rule

All FFs update using:

$$Q \leftarrow \boxed{\text{old state}}$$

at the same active edge.

Never use newly updated Q from another FF during the same edge.

## PART 18 — COUNTER MASTER SHEET Ø=4

Full n-bit binary counter:

$$\boxed{\text{MOD}=2^n}$$

Minimum FFs for MOD-N:

$$\boxed{\lceil \log_2 N \rceil}$$

Unused states:

$$\boxed{2^n - N}$$

## Synchronous Up Counter Using T FFs

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_1 Q_0$$

$$T_i = \prod_{k=0}^{i-1} Q_k$$

Memory:

**UP !' all lower bits 1.**

## Synchronous Down Counter

$$T_0 = 1$$

$$T_1 = Q_0'$$

$$T_2 = Q_1' Q_0'$$

Memory:

**DOWN !' all lower bits 0.**

## Ripple Counter Delay

Equal FF delay  $(t_p)$ :

$$\boxed{t_{\text{worst}} \approx nt_p}$$

Approximate:

$$\boxed{f_{\max} \approx \frac{1}{n t_p}}$$

when that simplified model is intended.

---

## Frequency Division

First toggle stage:

$$\boxed{f/2}$$

nth stage:

$$\boxed{f/2^n}$$

---

## Ring Counter

n FFs:

$$\boxed{n \text{ standard states}}$$

---

## Johnson Counter

n FFs:

$$\boxed{2n \text{ standard states}}$$

Memory:

**Ring = n.**

**Johnson = 2n.**

---

## PART 19 — SEQUENTIAL ANALYSIS VS DESIGN

---

### Analysis

$$\boxed{\text{Circuit} \rightarrow \text{FF inputs} \rightarrow Q, Q^+ \rightarrow \text{state table}}$$

Use:

$$\boxed{\text{characteristic equation}}$$

---

### Design

$$\boxed{\text{desired states} \rightarrow Q, Q^+ \rightarrow \text{excitation} \rightarrow \text{logic}}$$

Use:

$$\boxed{\text{excitation table}}$$

Memory:

**Analysis = predict.**

**Design = construct.**

## PART 20 — FSM MASTER SHEET Ø=Ý4

Next state:

$\boxed{Q^{+}=f(Q,X)}$

### Moore

$\boxed{Y=f(Q)}$

Output placed:

$\boxed{\text{on state}}$

### Mealy

$\boxed{Y=f(Q,X)}$

Output placed:

$\boxed{\text{on transition}}$

Mealy usually:

- fewer states
- faster input response
- more glitch-sensitive

Moore generally:

- more stable output
- sometimes more states

### States to FFs

M states using binary encoding:

$\boxed{\lceil \log_2 M \rceil}$

## PART 21 — SEQUENCE DETECTOR MASTER TRICK

Target pattern P.

State represents:

$\boxed{\text{longest prefix of P currently matched as suffix}}$

On each new bit:

1. append bit
2. find longest suffix also equal to a prefix
3. transition there

---

## Overlap

$\boxed{\text{reuse useful suffix}}$

Example target 101:

After detecting 101, suffix 1 can start next occurrence.

---

## Non-overlap

After detection:

$\boxed{\text{restart}}$

---

## Standard State Counts

Pattern length m:

Mealy:

$\boxed{\text{often } m}$

Moore:

$\boxed{\text{often } m+1}$

These are standard constructions, not universal minimal-state theorems.

---

## PART 22 — NUMBER REPRESENTATIONMASTER SHEET

---

### Unsigned n-bit

$\boxed{0 \text{ to } 2^n - 1}$

---

### Sign-Magnitude

$\boxed{-(2^{n-1} - 1)}$

to:

$\boxed{+(2^{n-1} - 1)}$

Two zeros.

## 1's Complement

---

Same range:

$$\mathbb{V}[-(2^{n-1}-1) \mathbb{V}]$$

to:

$$\mathbb{V}[(2^{n-1}-1) \mathbb{V}]$$

Two zeros.

Negative:

$$\mathbb{V}[\boxed{\text{invert all bits}} \mathbb{V}]$$

Addition may require end-around carry.

---

## 2's Complement Ø=Ý4

---

$$\mathbb{V}[\boxed{-2^{n-1}} \text{ to } 2^{n-1}-1 \mathbb{V}]$$

One zero.

Negative:

$$\mathbb{V}[\boxed{\text{invert} + 1} \mathbb{V}]$$

Fast decode if MSB=1:

$$\mathbb{V}[\boxed{\text{value}=\text{unsigned}-2^n} \mathbb{V}]$$

---

## Sign Extension

---

For 2's complement:

$$\mathbb{V}[\boxed{\text{repeat sign bit}} \mathbb{V}]$$

Unsigned extension:

$$\mathbb{V}[\boxed{\text{insert zeros}} \mathbb{V}]$$

---

## PART 23 — BINARY ARITHMETIC

---

Subtraction:

$$\mathbb{V}[\boxed{A-B=A+\text{2's complement}(B)} \mathbb{V}]$$

Discard final carry for fixed-width two's-complement result.

---

## n×m Multiplication

---

Full product can need:

$$\mathbb{V}[\boxed{n+m \text{ bits}} \mathbb{V}]$$

---

## Arithmetic Right Shift

For signed 2's complement:

$\boxed{\text{replicate sign bit}}$

Logical right shift:

$\boxed{\text{insert 0}}$

## PART 24 — FIXED-POINT MASTER SHEET Ø=Ý4

Let:

- total bits = n
- fractional bits = f

Value:

$\boxed{x = \frac{\text{raw integer}}{2^f}}$

Resolution:

$\boxed{2^{-f}}$

## Unsigned Range

$\boxed{0 \leq x \leq 2^{n-f} - 2^{-f}}$

## Signed 2's-Complement Range

$\boxed{-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}}$

## Encode

$\boxed{\text{raw} = x 2^f}$

with rounding if necessary.

## Multiply

Fractional bits add:

$\boxed{f_{\text{product}} = f_A + f_B}$

Same-scale addition uses ordinary integer addition.

## Tradeoff

More fractional bits:

$\boxed{\text{precision !' , range !" } \}$

for fixed total width.

---

## PART 25 — IEEE-754 MASTER SHEET Ø=Ý4Ø=Ý4

---

### Single

---

$\boxed{32=1+8+23}$

Bias:

$\boxed{127}$

Effective precision:

$\boxed{24\text{ binary bits}}$

Normal exponent:

$\boxed{-126\text{ to }127}$

Smallest normal:

$\boxed{2^{-126}}$

Smallest subnormal:

$\boxed{2^{-149}}$

Largest finite:

$\boxed{(2 \cdot 2^{-23}) 2^{127}}$

---

### Double

---

$\boxed{64=1+11+52}$

Bias:

$\boxed{1023}$

Effective precision:

$\boxed{53\text{ bits}}$

Normal exponent:

$\boxed{-1022\text{ to }1023}$

Smallest normal:

$\boxed{2^{-1022}}$

Smallest subnormal:

$\boxed{2^{-1074}}$

---

### Normal Number

---

$\boxed{(-1)^S (1.F) 2^{E-\text{Bias}}}$

---



## Subnormal

$\lfloor \boxed{(-1)^S(0.F)2^{1-Bias}} \rfloor$

## Special Cases

| E        | F       | Meaning      |
|----------|---------|--------------|
| all 0    | all 0   | $\pm 0$      |
| all 0    | nonzero | Subnormal    |
| ordinary | any     | Normal       |
| all 1    | all 0   | $\pm \infty$ |
| all 1    | nonzero | NaN          |

This table should be memorized exactly.

## PART 26 — TOP 30 GATE TRAPS Ø=Ý4

1. Carry-out "1" signed overflow.
2. Opposite-sign 2's-complement addition cannot overflow.
3.  $(-2^{n-1})$  is the most-negative 2's-complement value.
4. Its negation cannot be represented in same width.
5. Minterm uses 0! complement.
6. Maxterm uses opposite rule.
7. Gray K-map order is 00,01,11,10.
8. K-map wraps around edges.
9. Diagonal cells are not adjacent.
10. Don't-cares are optional.
11. PI "EPI.
12. NAND-NAND suits SOP.
13. NOR-NOR suits POS.
14. Half adder has no carry-in.
15. Full-adder sum is XOR of three inputs.
16. MUX  $(2^n:1)$  needs n select bits.
17. Decoder outputs correspond to minterms.
18. JK 11 toggles; SR 11 is invalid for active-high SR.
19. NAND SR active-low invalid case is 00.
20. Characteristic and excitation tables are reverse-direction tools.
21. Setup uses maximum delay.
22. Hold uses minimum delay.

23. Slowing clock does not normally fix hold.
  24. Ripple counter "synchronous counter.
  25. Ring n FFs !' n states.
  26. Johnson n FFs !' 2n states.
  27. Mealy output belongs on transitions.
  28. Moore output belongs on states.
  29. IEEE negative numbers do not use 2's complement encoding.
  30. IEEE exponent all ones is special, not an ordinary maximum exponent.
- 

## PART 27 — HIGHEST-RETURN REVISION ORDER

---

If you have only **2 hours**, revise in this order:

### Tier 1 — Must Do

1. Boolean identities
2. SOP/POS
3. K-map
4. MUX
5. Adders
6. Flip-flop characteristic + excitation tables
7. Counters
8. 2's-complement arithmetic
9. IEEE-754

### Tier 2

10. Decoder/Encoder
11. Sequential analysis/design
12. Moore/Mealy
13. Sequence detectors
14. Registers
15. Fixed point

### Tier 3

16. Quine–McCluskey details
  17. code converters
  18. master-slave/race-around
  19. parity/comparator supporting topics
- 

## PART 28 — 15-MINUTE LAST-MINUTE SHEET

---

Memorize:

$$\nabla [A + A'B = A + B \nabla]$$

$$\nabla [A(A' + B) = AB \nabla]$$

$$\nabla [(A + B)(A + B') = A \nabla]$$

$$\lceil AB+A'B=B \rceil$$

$$\lceil (A+B)'=A'B' \rceil$$

$$\lceil (AB)'=A'+B' \rceil$$

K-map:

$$\lceil 00,01,11,10 \rceil$$

Full adder:

$$\lceil S=A\oplus B\oplus C \rceil$$

$$\lceil C=AB+AC+BC \rceil$$

MUX:

$$\lceil Y=S'I_0+SI_1 \rceil$$

FF equations:

$$\lceil SR:Q^+=S+R'Q \rceil$$

$$\lceil JK:Q^+=JQ'+K'Q \rceil$$

$$\lceil D:Q^+=D \rceil$$

$$\lceil T:Q^+=T\oplus Q \rceil$$

Counter:

$$\lceil n=\lceil \log_2 N \rceil \rceil$$

Ring:

$$\lceil n \rceil$$

Johnson:

$$\lceil 2n \rceil$$

2's complement:

$$\lceil [-2^{n-1}, 2^{n-1}-1] \rceil$$

IEEE:

$$\lceil \boxed{\text{single } 1/8/23, 127} \rceil$$

$$\lceil \boxed{\text{double } 1/11/52, 1023} \rceil$$

## PART 29 — MIXED GATE-STYLE MOCK TEST

Do these without immediately checking answers.

### Q1 — MCQ

Simplify:

$$\lceil F=A+A'B \rceil$$

A.  $\lceil A \rceil$  B.  $\lceil (A+B) \rceil$  C.  $\lceil (AB) \rceil$  D.  $\lceil (A'+B) \rceil$

## Q2 — NAT

How many Boolean functions exist for 3 variables?

## Q3 — MCQ

For:

$$F = \sum m(1, 3, 5, 7)$$

with variables ABC, what is F?

A.  $A \cdot B$  B.  $B \cdot C$  C.  $C \cdot D$  D.  $\neg(C)$

## Q4 — MSQ

Which are valid K-map groups?

A. 2 cells B. 3 cells C. 4 cells D. 8 cells

## Q5 — NAT

Minimum number of select lines needed for a 32:1 MUX?

## Q6 — MCQ

A full adder has:

$$A \oplus B = C_{in}$$

What is  $(S, C_{out})$ ?

A. (0,0) B. (1,0) C. (0,1) D. (1,1)

## Q7 — MCQ

For a JK flip-flop:

$$J = K = 1$$

next state is:

A. 0 B. 1 C. Q D.  $\neg(Q)$

## Q8 — NAT

Minimum FFs needed for MOD-13 synchronous counter?

## Q9 — NAT

How many states does a standard 6-FF Johnson counter have?

---

### Q10 — MCQ

Which relationship belongs to a D flip-flop?

A.  $Q^{+}=D$  B.  $Q^{+}=D \oplus Q$  C.  $Q^{+}=D+Q$  D.  $Q^{+}=D'Q$

---

### Q11 — MCQ

For a Moore machine, output depends on:

A. input only B. state only C. state and input D. clock only

---

### Q12 — NAT

A sequence detector for 101 receives:

$\{ 10101 \}$

How many overlapping occurrences are present?

---

### Q13 — NAT

An 8-bit unsigned fixed-point representation has 3 fractional bits.

What is its maximum value?

---

### Q14 — MCQ

IEEE-754 single precision uses:

A.  $1/7/24$  B.  $1/8/23$  C.  $1/11/20$  D.  $1/10/21$

---

### Q15 — MCQ

IEEE exponent = all ones, fraction nonzero represents:

A. zero B. subnormal C. infinity D. NaN

---

## PART 30 — MOCK ANSWERS

---

### Q1

$\boxed{B}$

$A+A'B=A+B$

---

### Q2

---

Number of Boolean functions of n variables:

$$\lceil 2^{2^n} \rceil$$

For n=3:

$$\lceil 2^8 \rceil$$

$$\lceil \boxed{256} \rceil$$

---

### Q3

Minterms:

$$\lceil 1,3,5,7 \rceil$$

all have:

$$\lceil C=1 \rceil$$

Therefore:

$$\lceil \boxed{F=C} \rceil$$

Answer:

$$\lceil \boxed{C} \rceil$$

---

### Q4

Valid sizes are powers of 2.

$$\lceil \boxed{A,C,D} \rceil$$

---

### Q5

$$\lceil 2^n=32 \rceil$$

$$\lceil n=5 \rceil$$

$$\lceil \boxed{5} \rceil$$

---

### Q6

$$\lceil 1+1+1=3=11_2 \rceil$$

So:

$$\lceil S=1, \quad C=1 \rceil$$

$$\lceil \boxed{D} \rceil$$

---

### Q7

JK 11:

$$\lceil \boxed{Q^+=Q'} \rceil$$

---

Answer D.

---

### Q8

---

$$\sqrt[3]{2^3=8}<13$$

$$\sqrt[4]{2^4=16}$$

$$\boxed{4}$$

---

### Q9

---

Johnson:

$$\sqrt{2n}$$

$$\sqrt{2(6)}=12$$

$$\boxed{12}$$

---

### Q10

---

$$\boxed{Q^+=D}$$

Answer A.

---

### Q11

---

Moore:

$$\boxed{Y=f(Q)}$$

Answer B.

---

### Q12

---

10101:

- positions 1–3
- positions 3–5

$$\boxed{2}$$

---

### Q13

---

Unsigned:

$$\frac{2^8-1}{2^3}=\frac{255}{8}$$

$$\boxed{31.875}$$

---

## Q14

$\boxed{\frac{1}{8/23}}$

Answer B.

## Q15

All-ones exponent + nonzero fraction:

$\boxed{\text{NaN}}$

Answer D.

## PART 31 — SCORE INTERPRETATION

For this short mixed test:

### 14–15 correct

Very strong conceptual revision.

### 11–13

Good, but revise mistakes immediately.

### 8–10

Concepts exist, speed/retention needs work.

### Below 8

Revisit red-zone topics before beginning difficult mixed PYQs.

This 15-question set is only a diagnostic, not a substitute for actual GATE PYQs.

## PART 32 — FINAL FORMULA MEMORY TREE

Think of Digital Logic as 7 boxes:

### BOX 1 — Number Representation

$\text{\text{binary, signed, arithmetic, fixed, IEEE}}$

### BOX 2 — Boolean Logic

$\text{\text{identities, SOP/POS, K-map, Q-M}}$

### BOX 3 — Combinational Blocks

$\text{\text{adder, MUX, decoder, encoder, comparator}}$

### BOX 4 — Storage

$\text{\text{latches, SR/JK/D/T}}$



## BOX 5 — Sequential Structures

\[ \text{registers, counters} \]

## BOX 6 — State Machines

\[ \text{analysis, design, Moore, Mealy, detector} \]

## BOX 7 — Timing

\[ \text{setup, hold, delay, frequency} \]

If you can recall these seven boxes, you can reconstruct almost the entire subject.

## PART 33 — FINAL COVERAGE AUDIT

| Topic                      | Covered? | Revision Sheet? | Practice in Course? | Actual GATE PYQs? |
|----------------------------|----------|-----------------|---------------------|-------------------|
| Number Systems             | '        | '               | '                   | #0                |
| Signed Representation      | '        | '               | '                   | #0                |
| Binary Arithmetic          | '        | '               | '                   | #0                |
| Boolean Algebra            | '        | '               | '                   | #0                |
| SOP/POS                    | '        | '               | '                   | #0                |
| Universal Gates            | '        | '               | '                   | #0                |
| Algebraic Minimization     | '        | '               | '                   | #0                |
| K-map                      | '        | '               | '                   | #0                |
| Tabular/Q-M                | '        | '               | '                   | #0                |
| Combinational Fundamentals | '        | '               | '                   | #0                |
| Adders/Subtractors         | '        | '               | '                   | #0                |
| MUX                        | '        | '               | '                   | #0                |
| Decoder/Encoder/DEMUX      | '        | '               | '                   | #0                |
| Comparator/Code Converters | '        | '               | '                   | #0                |
| Sequential Timing          | '        | '               | '                   | #0                |
| Latches/FFs                | '        | '               | '                   | #0                |
| FF Conversion              | '        | '               | '                   | #0                |
| Race Around                | '        | '               | '                   | #0                |
| Registers                  | '        | '               | '                   | #0                |
| Counters                   | '        | '               | '                   | #0                |

| Topic                 | Covered? | Revision Sheet? | Practice in Course? | Actual GATE PYQs? |
|-----------------------|----------|-----------------|---------------------|-------------------|
| Sequential Analysis   | '        | '               | '                   | #0                |
| Sequential Design     | '        | '               | '                   | #0                |
| FSM                   | '        | '               | '                   | #0                |
| Sequence Detector     | '        | '               | '                   | #0                |
| Fixed Point           | '        | '               | '                   | #0                |
| Floating Fundamentals | '        | '               | '                   | #0                |
| IEEE-754              | '        | '               | '                   | #0                |

## FINAL STATUS

### Syllabus learning

$\boxed{\text{100\% COMPLETE}}$

### Important conceptual revision material

$\boxed{\text{COMPLETE}}$

### PYQ-style practice

$\boxed{\text{DONE THROUGHOUT THE COURSE}}$

### Actual year-wise GATE PYQs

$\boxed{\text{NOT YET COMPLETED IN THIS COURSE}}$

That distinction is important. I will not falsely label PYQ-style questions as actual GATE PYQs.

## What You Should Do Now

Your best next sequence is:

$\boxed{\text{Actual Digital Logic PYQs} \rightarrow \text{mistake notebook} \rightarrow \text{second revision} \rightarrow \text{timed mixed test} }$

Do **not** restart Digital Logic theory from zero unless PYQs expose a genuine weak area.

## DIGITAL LOGIC — COURSE STATUS

$\boxed{\text{COMPLETE ' } }$

### NEXT

The highest-value continuation is **Digital Logic GATE PYQ Training — topic-wise, one set at a time**, starting with:

**Boolean Algebra + SOP/POS + K-map !' MUX/Adders !' Flip-Flops/Counters !' Number Representation/IEEE-754 !' mixed timed PYQ revision.**

USER

next

ASSISTANT

We're moving from theory to **actual GATE PYQ training** now. I'll start with **Set 1: Boolean Algebra + SOP/POS + K-map**, using only questions I can verify from real GATE papers and paraphrasing them rather than inventing year labels.

USER

next

ASSISTANT

## DIGITAL LOGIC — ACTUAL GATE PYQ TRAINING SET 1 Ø=Ý4

### Boolean Algebra + SOP/POS + K-Map

From this point onward, I'll distinguish carefully between **verified actual GATE questions** and my own PYQ-style practice. The questions below are verified past GATE CSE questions; I'm condensing/paraphrasing the wording while preserving the mathematics.

## PYQ 1 — BOOLEAN ALGEBRA

### GATE CSE 2024 Set 2

For Boolean variable  $x$ , determine which statements are **false**:

1.  $x \cdot 1 = x$
2.  $x + 1 = x$
3.  $x \cdot x = 0$
4.  $x + \bar{x} = 1$

This was a GATE CSE 2024 Set 2 MSQ.

### Solution

Check one by one.

#### Statement 1

$$x \cdot 1 = x$$

Identity law.

' True.

### Statement 2

$$\forall x+1=x$$

But dominance law says:

$$\forall \boxed{x+1=1}$$

So statement 2 is false.

### Statement 3

$$\forall x \cdot x=0$$

But idempotent law says:

$$\forall \boxed{x \cdot x=x}$$

So statement 3 is false.

### Statement 4

$$\forall x+\bar{x}=1$$

Complement law.

' True.

Therefore false statements:

$$\forall \boxed{2, 3}$$

### GATE shortcut

Immediately remember:

$$\forall X+1=1$$

$$\forall X \cdot 0=0$$

$$\forall X+X=X$$

$$\forall XX=X$$

### Trap

Do not use ordinary algebra.

In ordinary arithmetic  $(x+x=2x)$ , but in Boolean algebra:

$$\forall \boxed{x+x=x}$$

---

## PYQ 2 — MINTERM + MAXTERM + MINIMIZATION

### GATE CSE 2024 Set 1

Given:

$$\forall F(X,Y,Z)=\Sigma m(3,5,6,7)$$

the question asked which statements were correct, including whether:

$$\forall F=\Pi M(0,1,2,4)$$

and whether:

$$\forall F=XY+YZ+XZ$$

### Step 1 — Convert SOP indices to POS indices

---

For 3 variables:

$\{ 0,1,2,3,4,5,6,7 \}$

F=1 for:

$\{ 3,5,6,7 \}$

Therefore F=0 for:

$\{ 0,1,2,4 \}$

Hence:

$\boxed{F=\prod M(0,1,2,4)}$

Correct.

---

### Step 2 — Minimize

---

Minterms:

$\{ 3=011 \}$

$\{ 5=101 \}$

$\{ 6=110 \}$

$\{ 7=111 \}$

These are exactly the input combinations where at least two variables are 1.

Therefore:

$\boxed{F=XY+XZ+YZ}$

---

### Step 3 — Independence Check

---

Does F depend on X?

Yes.

Example:

$\{ Y=0,Z=1 \}$

Then:

$\{ F=X \}$

So changing X changes F.

Does F depend on Y?

Yes.

Example:

---

$$\{ X=0, Z=1 \}$$

Then:

$$\{ F=Y \}$$

Therefore F depends on both.

Correct choices in the original problem were the first two statements.

## GATE memory

$$\sum m(S) \rightarrow \prod M(\text{complement of } S)$$

For n variables, list all:

$$0 \dots 2^n - 1$$

and take the missing indices.

## PYQ 3 — K-MAP WITH DON'T-CARES

### GATE CSE 2014 Set 3

Given:

$$F(P, Q, R, S)$$

with the listed minterm positions

$$\{ 0, 2, 5, 7, 8, 10, 13, 15 \}$$

and with  $\{2, 7, 8, 13\}$  specified as don't-care terms, GATE asked for the minimal SOP.

The correct minimized result is:

$$\boxed{\bar{Q}\bar{S} + QS}$$

## How to See It Fast

The useful K-map grouping produces two large groups.

One corresponds to:

$$\{ Q=0, S=0 \}$$

giving:

$$\bar{Q}\bar{S}$$

The other corresponds to:

$$\{ Q=1, S=1 \}$$

giving:

$$QS$$

Thus:

$$\boxed{F = \bar{Q}\bar{S} + QS}$$

Recognize this?

$$\boxed{F=Q\odot S}$$

XNOR.

The other variables P and R disappear completely.

---

## Why Don't-Cares Are Powerful

---

Don't-care terms may be included in groups if they help create:

- larger groups
- fewer literals
- fewer product terms

But:

$$\boxed{\text{you are not required to cover them}}$$

---

## GATE-fast approach

---

If the final K-map pattern says:

- both bits 00 !' 1
- both bits 11 !' 1

then immediately think:

$$\boxed{\text{XNOR}}$$

$$\boxed{AB+A'B'}$$

---

## PYQ 4 — ESSENTIAL PRIME IMPLICANTS

---

### GATE CSE 2018

GATE gave:

$$F(P,Q,R,S)=\Sigma m(0,2,5,7,9,11)+d(3,8,10,12,14)$$

and asked for the **number of essential prime implicants**.

Answer:

$$\boxed{3}$$

---

## First Recall the Difference

---

### Implicant

---

Any valid group of:

$$\boxed{1,2,4,8,\ldots}$$

1-cells, optionally including don't-cares.

---

## Prime Implicant

---

A group that cannot be enlarged further.

## Essential Prime Implicant

---

A PI covering at least one **required 1** that no other PI covers.

---

## Critical Don't-Care Rule

---

While finding prime implicants:

$\nexists$  can behave like 1

But when deciding whether a PI is **essential**, a don't-care cell itself does not force essentiality.

You must find a real required minterm uniquely covered.

The verified solution identifies:

$\boxed{3 \text{ essential prime implicants}}$

for this function.

---

## Most important trap

---

Suppose a PI uniquely covers only a don't-care cell.

Is it essential?

$\boxed{\text{No}}$

Because nobody requires the don't-care output to be 1.

---

## PYQ 5 — K-MAP + NOR REALIZATION

---

### GATE CSE 2019

GATE asked for the minimum number of **2-input NOR gates** needed for:

$f = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$

assuming both variables and their complements are available.

---

## Step 1 — Minimize the Function

---

K-map simplification gives:

$f = BD + B'D'$

This is:

$\boxed{B \odot D}$

XNOR.

But because the question asks for NOR implementation, convert to POS:

---



$$\lceil BD+B'D' \rceil$$

Using algebra:

$$\lceil \boxed{f=(B'+D)(B+D')} \rceil$$

## Step 2 — NOR-NOR Realization

First gate:

$$\lceil G_1=(B'+D)' \rceil$$

Second:

$$\lceil G_2=(B+D')' \rceil$$

Third NOR:

$$\lceil (G_1+G_2)' \rceil$$

Substitute:

$$\lceil [(B'+D)'+(B+D')]' \rceil$$

By De Morgan:

$$\lceil =(B'+D)(B+D') \rceil$$

which is exactly f.

Therefore:

$$\lceil \boxed{3} \rceil$$

2-input NOR gates are sufficient, and this is the verified GATE answer.

## Why Not More Gates?

Because the problem explicitly says:

**inputs and complements are available.**

So  $\lceil B' \rceil$  and  $\lceil D' \rceil$  do not require extra NOT gates.

This detail saves gates.

## GATE Lesson

When asked:

### NAND gates?

First look for:

$$\lceil \boxed{\text{minimum SOP}} \rceil$$

because:

$$\lceil \text{SOP} \rightarrow \text{NAND-NAND} \rceil$$

NOR gates?

First look for:

$\boxed{\text{minimum POS}}$

because:

$\text{POS} \rightarrow \text{NOR-NOR}$

SET 1 — MASTER LEARNINGS

These five actual PYQs test five different abilities:

| PYQ             | Main skill                          |
|-----------------|-------------------------------------|
| GATE 2024 Set 2 | Boolean identities                  |
| GATE 2024 Set 1 | $\sum m$ , $\prod M$ , minimization |
| GATE 2014 Set 3 | K-map + don't-cares                 |
| GATE 2018       | PI vs EPI                           |
| GATE 2019       | K-map + NOR gate realization        |

1-Minute Revision

Remember:

$X+1=1$

$XX=X$

$X+\overline{X}=1$

$\sum m(S) \rightarrow \prod M(\overline{S})$

Don't-care:

$\boxed{\text{use only if helpful}}$

EPI:

$\boxed{\text{covers a required 1 uniquely}}$

NAND:

$\boxed{\text{SOP}}$

NOR:

$\boxed{\text{POS}}$

XNOR:

$\boxed{AB+A'B'}$

PYQ PROGRESS

## Set 1

Boolean Algebra + SOP/POS + K-map '

**NEXT — ACTUAL GATE PYQ SET 2**

## Adders + MUX + Decoder + Combinational Circuits

---

Next we move through verified GATE questions on:

**MUX function realization !' adder/carry logic !' propagation delay !' decoder realization !' combinational circuit analysis !' gate-count tricks.**